

TCP/IP Illustrated, Volume 1

The Protocols
SECOND EDITION

Kevin R. Fall
W. Richard Stevens



1

Introduction

Effective communication depends on the use of a common language. This is true for humans and other animals as well as for computers. When a set of common behaviors is used with a common language, a *protocol* is being used. The first definition of a protocol, according to the *New Oxford American Dictionary*, is

The official procedure or system of rules governing affairs of state or diplomatic occasions.

We engage in many protocols every day: asking and responding to questions, negotiating business transactions, working collaboratively, and so on. Computers also engage in a variety of protocols. A collection of related protocols is called a *protocol suite*. The design that specifies how various protocols of a protocol suite relate to each other and divide up tasks to be accomplished is called the *architecture* or *reference model* for the protocol suite. TCP/IP is a protocol suite that implements the Internet architecture and draws its origins from the *ARPANET Reference Model* (ARM) [RFC0871]. The ARM was itself influenced by early work on packet switching in the United States by Paul Baran [B64] and Leonard Kleinrock [K64], in the U.K. by Donald Davies [DBSW66], and in France by Louis Pouzin [P73]. Other protocol architectures have been specified over the years (e.g., the ISO protocol architecture [Z80], Xerox's XNS [X85], and IBM's SNA [I96]), but TCP/IP has become the most popular. There are several interesting books that focus on the history of computer communications and the development of the Internet, such as [P07] and [W02].

It is worth mentioning that the TCP/IP architecture evolved from work that addressed a need to provide interconnection of multiple *different* packet-switched computer networks [CK74]. This was accomplished using a set of *gateways* (later called *routers*) that provided a translation function between each otherwise incompatible network. The resulting "concatenated" network or *catenet* (later called *internetwork*) would be much more useful, as many more nodes offering a wide variety of services could communicate. The types of uses that a global network might offer were envisioned years before the protocol architecture was fully developed.

In 1968, for example, J. C. R. Licklider and Bob Taylor foresaw the potential uses for a global interconnected communication network to support “supercommunities” [LT68]:

Today the on-line communities are separated from one another functionally as well as geographically. Each member can look only to the processing, storage and software capability of the facility upon which his community is centered. But now the move is on to interconnect the separate communities and thereby transform them into, let us call it, a supercommunity. The hope is that interconnection will make available to all members of all the communities the programs and data resources of the entire supercommunity . . . The whole will constitute a labile network of networks—ever-changing in both content and configuration.

Thus, it is apparent that the global network concept underpinning the ARPANET and later the Internet was designed to support many of the types of uses we enjoy today. However, getting to this point was neither simple nor obvious. The success resulted from paying careful attention to design and engineering, innovative users and developers, and the availability of sufficient resources to move from concept to prototype and, eventually, to commercial networking products.

This chapter provides an overview of the Internet architecture and TCP/IP protocol suite, to provide some historical context and to establish an adequate background for the remaining chapters. Architectures (both protocol and physical) really amount to a set of design decisions about what features should be supported and where such features should be logically implemented. Designing an architecture is more art than science, yet we shall discuss some characteristics of architectures that have been deemed desirable over time. The subject of network architecture has been undertaken more broadly in the text by Day [D08], one of few such treatments.

1.1 Architectural Principles

The TCP/IP protocol suite allows computers, smartphones, and embedded devices of all sizes, supplied from many different computer vendors and running totally different software, to communicate with each other. By the turn of the twenty-first century it has become a necessity for modern communication, entertainment, and commerce. It is truly an *open system* in that the definition of the protocol suite and many of its implementations are publicly available at little or no charge. It forms the basis for what is called the *global Internet*, or the *Internet*, a wide area network (WAN) of about two billion users that literally spans the globe (as of 2010, about 30% of the world’s population). Although many people consider the Internet and the *World Wide Web* (WWW) to be interchangeable terms, we ordinarily refer to the Internet in terms of its ability to provide basic communication of messages between computers. We refer to WWW as an application that uses the Internet for

communication. It is perhaps the most important Internet application that brought Internet technology to world attention in the early 1990s.

Several goals guided the creation of the Internet architecture. In [C88], Clark recounts that the primary goal was to “develop an effective technique for multiplexed utilization of existing interconnected networks.” The essence of this statement is that the Internet architecture should be able to interconnect multiple distinct networks and that multiple activities should be able to run simultaneously on the resulting interconnected network. Beyond this primary goal, Clark provides a list of the following second-level goals:

- Internet communication must continue despite loss of networks or gateways.
- The Internet must support multiple types of communication services.
- The Internet architecture must accommodate a variety of networks.
- The Internet architecture must permit distributed management of its resources.
- The Internet architecture must be cost-effective.
- The Internet architecture must permit host attachment with a low level of effort.
- The resources used in the Internet architecture must be accountable.

Many of the goals listed could have been supported with somewhat different design decisions from those ultimately selected. However, a few design options were gaining momentum when these architectural principles were being formulated that influenced the designers in the particular choices they made. We will mention some of the more important ones and their consequences.

1.1.1 Packets, Connections, and Datagrams

Up to the 1960s, the concept of a network was based largely on the telephone network. It was developed to connect telephones to each other for the duration of a call. A call was normally implemented by establishing a *connection* from one party to another. Establishing a connection meant that a circuit (initially, a physical electrical circuit) was made between one telephone and another for the duration of a call. When the call was complete, the connection was cleared, allowing the circuit to be used by other users' calls. The call duration and identification of the connection endpoints were used to perform billing of the users. When established, the connection provided each user a certain amount of *bandwidth* or *capacity* to send information (usually voice sounds). The telephone network progressed from its analog roots to digital, which greatly improved its reliability and performance. Data inserted into one end of a circuit follows some preestablished path through the network switches and emerges on the other side in a predictable fashion,

usually with some upper bound on the time (*latency*). This gives predictable service, as long as a circuit is available when a user needs one. Circuits allocate a pathway through the network that is reserved for the duration of a call, even if they are not entirely busy. This is a common experience today with the phone network—as long as a call is taking place, even if we are not saying anything, we are being charged for the time.

One of the important concepts developed in the 1960s (e.g., in [B64]) was the idea of *packet switching*. In packet switching, “chunks” (packets) of digital information comprising some number of bytes are carried through the network somewhat independently. Chunks coming from different sources or senders can be mixed together and pulled apart later, which is called *multiplexing*. The chunks can be moved around from one switch to another on their way to a destination, and the path might be subject to change. This has two potential advantages: the network can be more resilient (the designers were worried about the network being physically attacked), and there can be better utilization of the network links and switches because of *statistical multiplexing*.

When packets are received at a packet switch, they are ordinarily stored in *buffer memory* or *queue* and processed in a *first-come-first-served* (FCFS) fashion. This is the simplest method for *scheduling* the way packets are processed and is also called *first-in-first-out* (FIFO). FIFO buffer management and on-demand scheduling are easily combined to implement statistical multiplexing, which is the primary method used to intermix traffic from different sources on the Internet. In statistical multiplexing, traffic is mixed together based on the arrival statistics or timing pattern of the traffic. Such multiplexing is simple and efficient, because if there is any network capacity to be used and traffic to use it, the network will be busy (high utilization) at every bottleneck or choke point. The downside of this approach is limited predictability—the performance seen by any particular application depends on the statistics of other applications that are sharing the network. Statistical multiplexing is like a highway where the cars can change lanes and ultimately intersperse in such a way that any point of constriction is as busy as it can be.

Alternative techniques, such as *time-division multiplexing* (TDM) and *static multiplexing*, typically reserve a certain amount of time or other resources for data on each connection. Although such techniques can lead to more predictability, a feature useful for supporting constant bit rate telephone calls, they may not fully utilize the network capacity because reserved bandwidth may go unused. Note that while circuits are straightforwardly implemented using TDM techniques, *virtual circuits* (VCs) that exhibit many of the behaviors of circuits but do not depend on physical circuit switches can be implemented atop connection-oriented packets. This is the basis for a protocol known as X.25 that was popular until about the early 1990s when it was largely replaced with Frame Relay and ultimately *digital subscriber line* (DSL) technology and cable modems supporting Internet connectivity (see Chapter 3).

The VC abstraction and connection-oriented packet networks such as X.25 required some information or *state* to be stored in each switch for each connection. The reason is that each packet carries only a small bit of overhead information that provides an index into a state table. For example, in X.25 the 12-bit *logical channel identifier* (LCI) or *logical channel number* (LCN) serves this purpose. At each switch, the LCI or LCN is used in conjunction with the *per-flow state* in each switch to determine the next switch along the path for the packet. The per-flow state is established prior to the exchange of data on a VC using a signaling protocol that supports connection establishment, clearing, and status information. Such networks are consequently called *connection-oriented*.

Connection-oriented networks, whether built on circuits or packets, were the most prevalent form of networking for many years. In the late 1960s, another option was developed known as the *datagram*. Attributed in origin to the CYCLADES [P73] system, a datagram is a special type of packet in which all the identifying information of the source and final destination resides inside the packet itself (instead of in the packet switches). Although this tends to require larger packets, per-connection state at packet switches is no longer required and a *connectionless* network could be built, eliminating the need for a (complicated) signaling protocol. Datagrams were eagerly embraced by the designers of the early Internet, and this decision had profound implications for the rest of the protocol suite.

One other related concept is that of *message boundaries* or *record markers*. As shown in Figure 1-1, when an application sends more than one chunk of information into the network, the fact that more than one chunk was written may or

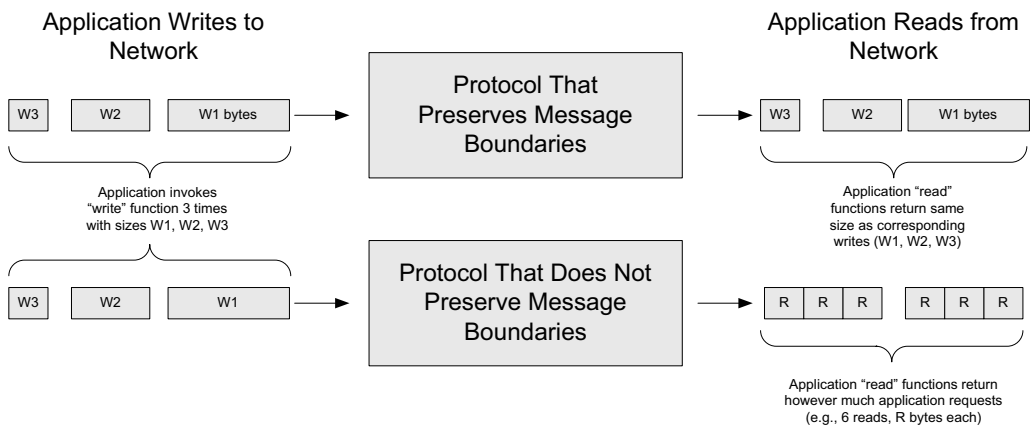


Figure 1-1 Applications write messages that are carried in protocols. A message boundary is the position or byte offset between one write and another. Protocols that preserve message boundaries indicate the position of the sender's message boundaries at the receiver. Protocols that do not preserve message boundaries (e.g., streaming protocols like TCP) ignore this information and do not make it available to a receiver. As a result, applications may need to implement their own methods to indicate a sender's message boundaries if this capability is required.

may not be preserved by the communication protocol. Most datagram protocols preserve message boundaries. This is natural because the datagram itself has a beginning and an end. However, in a circuit or VC network, it is possible that an application may write several chunks of data, all of which are read together as one or more different-size chunks by a receiving application. These types of protocols do not preserve message boundaries. In cases where an underlying protocol fails to preserve message boundaries but they are needed by an application, the application must provide its own.

1.1.2 The End-to-End Argument and Fate Sharing

When large systems such as an operating system or protocol suite are being designed, a question often arises as to where a particular feature or function should be placed. One of the most important principles that influenced the design of the TCP/IP suite is called the *end-to-end argument* [SRC84]:

The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system. Therefore, providing that questioned function as a feature of the communication itself is not possible. (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)

This argument may seem fairly straightforward upon first reading but can have profound implications for communication system design. It argues that correctness and completeness can be achieved *only* by involving the application or ultimate user of the communication system. Efforts to correctly implement what the application is “likely” to need are doomed to incompleteness. In short, this principle argues that important functions (e.g., error control, encryption, delivery acknowledgment) should usually not be implemented at low levels (or layers; see Section 1.2.1) of large systems. However, low levels may provide capabilities that make the job of the endpoints somewhat easier and consequently may improve performance. A nuanced reading reveals that this argument suggests that low-level functions should not aim for perfection because a perfect guess at what the application may require is unlikely to be possible.

The end-to-end argument tends to support a design with a “dumb” network and “smart” systems connected to the network. This is what we see in the TCP/IP design, where many functions (e.g., methods to ensure that data is not lost, controlling the rate at which a sender sends) are implemented in the end hosts where the applications reside. The selection of which functions are implemented together in the same computer or network or software stack is the subject of another related principle known as fate sharing [C88].

Fate sharing suggests placing all the necessary state to maintain an active communication association (e.g., virtual connection) at the same location with

the communicating endpoints. With this reasoning, the only type of failure that destroys communication is one that also destroys one or more of the endpoints, which obviously destroys the overall communication anyhow. Fate sharing is one of the design philosophies that allows virtual connections (e.g., those implemented by TCP) to remain active even if connectivity within the network has failed for a (modest) period of time. Fate sharing also supports a “dumb network with smart end hosts” model, and one of the ongoing tensions in today’s Internet is what functions reside in the network and what functions do not.

1.1.3 Error Control and Flow Control

There are some circumstances where data within a network gets damaged or lost. This can be for a variety of reasons such as hardware problems, radiation that modifies bits while being transmitted, being out of range in a wireless network, and other factors. Dealing with such errors is called *error control*, and it can be implemented in the systems constituting the network infrastructure, or in the systems that attach to the network, or some combination. Naturally, the end-to-end argument and fate sharing would suggest that error control be implemented close to or within applications.

Usually, if a small number of bit errors are of concern, a number of mathematical codes can be used to detect and repair the bit errors when data is received or while it is in transit [LC04]. This task is routinely performed within the network. When more severe damage occurs in a packet network, entire packets are usually resent or *retransmitted*. In circuit-switched or VC-switched networks such as X.25, retransmission tends to be done inside the network. This may work well for applications that require strict in-order, error-free delivery of their data, but some applications do not require this capability and do not wish to pay the costs (such as connection establishment and potential retransmission delays) to have their data reliably delivered. Even a reliable file transfer application does not really care in what order the chunks of file data are delivered, provided it is eventually satisfied that all chunks are delivered without errors and can be reassembled back into the original order.

As an alternative to the overhead of reliable, in-order delivery implemented within the network, a different type of service called *best-effort delivery* was adopted by Frame Relay and the Internet Protocol. With best-effort delivery, the network does not expend much effort to ensure that data is delivered without errors or gaps. Certain types of errors are usually detected using error-detecting codes or *checksums*, such as those that might affect where a datagram is directed, but when such errors are detected, the errant datagram is merely discarded without further action.

If best-effort delivery is successful, a fast sender can produce information at a rate that exceeds the receiver’s ability to consume it. In best-effort IP networks, slowing down a sender is achieved by *flow control* mechanisms that operate outside the network and at higher levels of the communication system. In particular,

TCP handles this type of problem, and we shall discuss it in detail in Chapters 15 and 16. This is consistent with the end-to-end argument: TCP, which resides at the end hosts, handles rate control. It is also consistent with fate sharing: the approach allows some elements of the network infrastructure to fail without necessarily affecting the ability of the devices outside the network to communicate (as long as some communication path continues to operate).

1.2 Design and Implementation

Although a protocol architecture may suggest a certain approach to implementation, it usually does not include a mandate. Consequently, we make a distinction between the protocol architecture and the *implementation architecture*, which defines how the concepts in a protocol architecture may be rendered into existence, usually in the form of software.

Many of the individuals responsible for implementing the protocols for the ARPANET were familiar with the software structuring of operating systems, and an influential paper describing the “THE” multiprogramming system [D68] advocated the use of a hierarchical structure as a way to deal with verification of the logical soundness and correctness of a large software implementation. Ultimately, this contributed to a design philosophy for networking protocols involving multiple *layers* of implementation (and design). This approach is now called *layering* and is the usual approach to implementing protocol suites.

1.2.1 Layering

With layering, each layer is responsible for a different facet of the communications. Layers are beneficial because a layered design allows developers to evolve different portions of the system separately, often by different people with somewhat different areas of expertise. The most frequently mentioned concept of protocol layering is based on a standard called the *Open Systems Interconnection* (OSI) model [Z80] as defined by the International Organization for Standardization (ISO). Figure 1-2 shows the standard OSI layers, including their names, numbers, and a few examples. The Internet’s layering model is somewhat simpler, as we shall see in Section 1.3.

Although the OSI model suggests that seven logical layers may be desirable for modularity of a protocol architecture implementation, the TCP/IP architecture is normally considered to consist of five. There was much debate about the relative benefits and deficiencies of the OSI model, and the ARPANET model that preceded it, during the early 1970s. Although it may be fair to say that TCP/IP ultimately “won,” a number of ideas and even entire protocols from the ISO protocol suite (protocols standardized by ISO that follow the OSI model) have been adopted for use with TCP/IP (e.g., IS-IS [RFC3787]).

	Number	Name	Description/Example
Hosts	7	Application	Specifies methods for accomplishing some user-initiated task. Application-layer protocols tend to be devised and implemented by application developers. Examples include FTP, Skype, etc.
	6	Presentation	Specifies methods for expressing data formats and translation rules for applications. A standard example would be conversion of EBCDIC to ASCII coding for characters (but of little concern today). Encryption is sometimes associated with this layer but can also be found at other layers.
	5	Session	Specifies methods for multiple connections constituting a communication session. These may include closing connections, restarting connections, and checkpointing progress. ISO X.225 is a session-layer protocol.
	4	Transport	Specifies methods for connections or associations between multiple programs running on the same computer system. This layer may also implement reliable delivery if not implemented elsewhere (e.g., Internet TCP, ISO TP4).
All Networked Devices	3	Network or Internetwork	Specifies methods for communicating in a multihop fashion across potentially different types of link networks. For packet networks, describes an abstract packet format and its standard addressing structure (e.g., IP datagram, X.25 PLP, ISO CLNP).
	2	Link	Specifies methods for communication across a single link, including “media access” control protocols when multiple systems share the same media. Error detection is commonly included at this layer, along with link-layer address formats (e.g., Ethernet, Wi-Fi, ISO 13239/HDLC).
	1	Physical	Specifies connectors, data rates, and how bits are encoded on some media. Also describes low-level error detection and correction, plus frequency assignments. We mostly stay clear of this layer in this text. Examples include V.92, Ethernet 1000BASE-T, SONET/SDH.

Figure 1-2 The standard seven-layer OSI model as specified by the ISO. Not all protocols are implemented by every networked device (at least in theory). The OSI terminology and layer numbers are widely used.

As described briefly in Figure 1-2, each layer has a different responsibility. From the bottom up, the *physical* layer defines methods for moving digital information across a communication medium such as a phone line or fiber-optic cable. Portions of the Ethernet and Wireless LAN (Wi-Fi) standards are here, although we do not delve into this layer very much in this text. The *link* or *data-link* layer includes those protocols and methods for establishing connectivity to a neighbor sharing the same medium. Some link-layer networks (e.g., DSL) connect only two neighbors. When more than one neighbor can access the same shared network, the network is said to be a *multi-access* network. Wi-Fi and Ethernet are examples of such multi-access link-layer networks, and specific protocols are used to mediate which stations have access to the shared medium at any given time. We discuss these in Chapter 3.

Moving up the layer stack, the *network* or *internetwork* layer is of great interest to us. For packet networks such as TCP/IP, it provides an interoperable packet format that can use different types of link-layer networks for connectivity. The layer also includes an addressing scheme for hosts and routing algorithms that choose where packets go when sent from one machine to another. Above layer 3 we find protocols that are (at least in theory) implemented only by end hosts, including the *transport* layer. Also of great interest to us, it provides a flow of data between sessions and can be quite complex, depending on the types of services it provides

(e.g., reliable delivery on a packet network that might drop data). *Sessions* represent ongoing interactions between applications (e.g., when “cookies” are used with a Web browser during a Web login session), and session-layer protocols may provide capabilities such as connection initiation and restart, plus *checkpointing* (saving work that has been accomplished so far). Above the session layer we find the *presentation* layer, which is responsible for format conversions and standard encodings for information. As we shall see, the Internet protocols do not include a formal session or presentation protocol layer, so these functions are implemented by applications if needed.

The top layer is the *application* layer. Applications usually implement their own application-layer protocols, and these are the ones most visible to users. There is a wide variety of application-layer protocols, and programmers are constantly inventing new ones. Consequently, the application layer is where there is the greatest amount of innovation and where new capabilities are developed and deployed.

1.2.2 Multiplexing, Demultiplexing, and Encapsulation in Layered Implementations

One of the major benefits of a layered architecture is its natural ability to perform *protocol multiplexing*. This form of multiplexing allows multiple different protocols to coexist on the same infrastructure. It also allows multiple instantiations of the same protocol object (e.g., connections) to be used simultaneously without being confused.

Multiplexing can occur at different layers, and at each layer a different sort of *identifier* is used for determining which protocol or stream of information belongs together. For example, at the link layer, most link technologies (such as Ethernet and Wi-Fi) include a *protocol identifier* field value in each packet to indicate which protocol is being carried in the link-layer frame (IP is one such protocol). When an object (packet, message, etc.), called a *protocol data unit* (PDU), at one layer is carried by a lower layer, it is said to be *encapsulated* (as opaque data) by the next layer down. Thus, multiple objects at layer N can be multiplexed together using encapsulation in layer $N - 1$. Figure 1-3 shows how this works. The identifier at layer $N - 1$ is used to determine the correct receiving protocol or program at layer N during demultiplexing.

In Figure 1-3, each layer has its own concept of a message object (a PDU) corresponding to the particular layer responsible for creating it. For example, if a layer 4 (transport) protocol produces a packet, it would properly be called a layer 4 PDU or *transport PDU* (TPDU). When a layer is provided a PDU from the layer above it, it usually “promises” to not look into the contents of the PDU. This is the essence of encapsulation—each layer treats the data from above as opaque, uninterpretable information. Most commonly a layer prepends the PDU with its own header, although trailers are used by some protocols (not TCP/IP). The header is used for multiplexing data when sending, and for the receiver to perform demultiplexing,

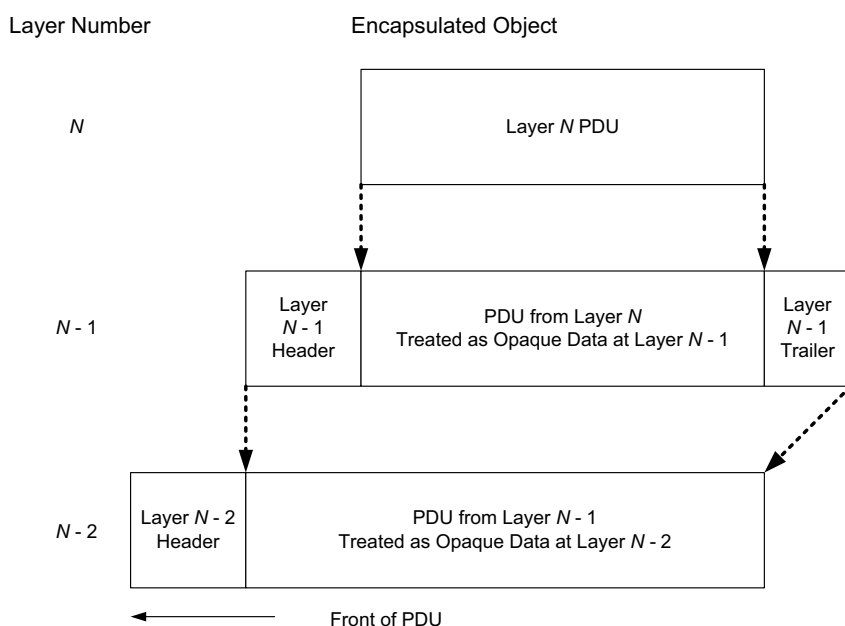


Figure 1-3 Encapsulation is usually used in conjunction with layering. Pure encapsulation involves taking the PDU of one layer and treating it as opaque (uninterpreted) data at the layer below. Encapsulation takes place at each sender, and decapsulation (the reverse operation) takes place at each receiver. Most protocols use headers during encapsulation; a few also use trailers.

based on a demultiplexing (demux) identifier. In TCP/IP networks such identifiers are commonly hardware addresses, IP addresses, and port numbers. The header may also include important state information, such as whether a virtual circuit is being set up or has already completed setup. The resulting object is another PDU.

One other important feature of layering suggested by Figure 1-2 is that in pure layering not all networked devices need to implement all the layers. Figure 1-4 shows that in some cases a device needs to implement only a few layers if it is expected to perform only certain types of processing.

In Figure 1-4, a somewhat idealized small internet includes two end systems, a switch, and a router. In this figure, each number corresponds to a type of protocol at a particular layer. As we can see, each device implements a different subset of the layer stack. The host on the left implements three different link-layer protocols (D, E, and F) with corresponding physical layers and three different transport-layer protocols (A, B, and C) that run on a single type of network-layer protocol. End hosts implement all the layers, switches implement up to layer 2 (this switch implements D and G), and routers implement up to layer 3. Routers are capable of interconnecting different types of link-layer networks and must implement the link-layer protocols for each of the network types they interconnect.

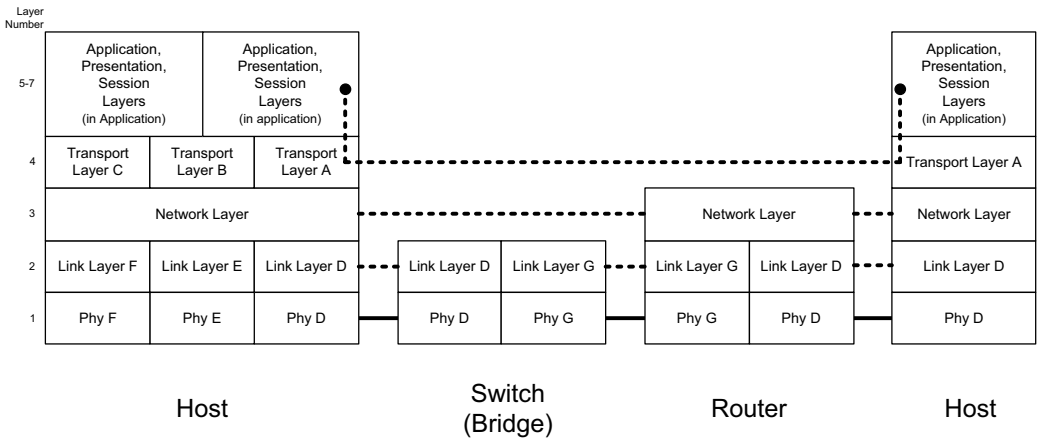


Figure 1-4 Different network devices implement different subsets of the protocol stack. End hosts tend to implement all the layers. Routers implement layers below the transport layer, and switches implement link-layer protocols and below. This idealized structure is often violated because routers and switches usually include the ability to act as a host (e.g., to be managed and set up) and therefore need an implementation of all of the layers even if they are rarely used.

The internet of Figure 1-4 is somewhat idealized because today's switches and routers often implement more than the protocols they are absolutely required to implement for forwarding data. This is for a number of reasons, including management. In such circumstances, devices such as routers and switches must sometimes act as hosts and support services such as remote login. To do this, they usually must implement transport and application protocols.

Although we show only two hosts communicating, the link- and physical-layer networks (labeled as D and G) might have multiple hosts attached. If so, then communication is possible between any pair of systems that implement the appropriate higher-layer protocols. In Figure 1-4 we can differentiate between an *end system* (the two hosts on either side) and an *intermediate system* (the router in the middle) for a particular protocol suite. Layers above the network layer use *end-to-end* protocols. In our picture these layers are needed only on the end systems. The network layer, however, provides a *hop-by-hop* protocol and is used on the two end systems and every intermediate system. The switch or bridge is not ordinarily considered an intermediate system because it is not addressed using the internet-working protocol's addressing format, and it operates in a fashion that is largely transparent to the network-layer protocol. From the point of view of the routers and end systems, the switch or bridge is essentially invisible.

A router, by definition, has two or more network interfaces (because it connects two or more networks). Any system with multiple interfaces is called *multihomed*. A host can also be multihomed, but unless it specifically forwards packets from one interface to another, it is not called a router. Also, routers need not be

special hardware boxes that only move packets around an internet. Most TCP/IP implementations, for example, allow a multihomed host to act as a router also, if properly configured to do so. In this case we can call the system either a host (when an application such as *File Transfer Protocol* (FTP) [RFC0959] or the Web is used) or a router (when it is forwarding packets from one network to another). We will use whichever term makes sense given the context.

One of the goals of an internet is to hide all of the details of the physical layout (the topology) and lower-layer protocol heterogeneity from the applications. Although this is not obvious from our two-network internet in Figure 1-4, the application layers should not care (and do not care) that even though each host is attached to a network using link-layer protocol D (e.g., Ethernet), the hosts are separated by a router and switch that use link-layer G. There could be 20 routers between the hosts, with additional types of physical interconnections, and the applications would run without modification (although the performance might be somewhat different). Abstracting the details in this way is what makes the concept of an internet so powerful and useful.

1.3 The Architecture and Protocols of the TCP/IP Suite

So far we have discussed architecture, protocols, protocol suites, and implementation techniques in the abstract. In this section, we discuss the architecture and particular protocols that constitute the *TCP/IP suite*. Although this has become the established term for the protocols used on the Internet, there are many protocols beyond TCP and IP in the collection or family of protocols used with the Internet. We begin by noting how the ARPANET reference model of layering, which ultimately formed the basis for the Internet's protocol layering, differs somewhat from the OSI layering discussed earlier.

1.3.1 The ARPANET Reference Model

Figure 1-5 depicts the layering inspired by the ARPANET reference model, which was ultimately adopted by the TCP/IP suite. The structure is simpler than the OSI model, but real implementations include a few specialized protocols that do not fit cleanly into the conventional layers.

Starting from the bottom of Figure 1-5 and working our way up the stack, the first layer we see is 2.5, an “unofficial” layer. There are several protocols that operate here, but one of the oldest and most important is called the *Address Resolution Protocol* (ARP). It is a specialized protocol used with IPv4 and only with multi-access link-layer protocols (such as Ethernet and Wi-Fi) to convert between the addresses used by the IP layer and the addresses used by the link layer. We examine this protocol in Chapter 4. In IPv6 the address-mapping function is part of ICMPv6, which we discuss in Chapter 8.

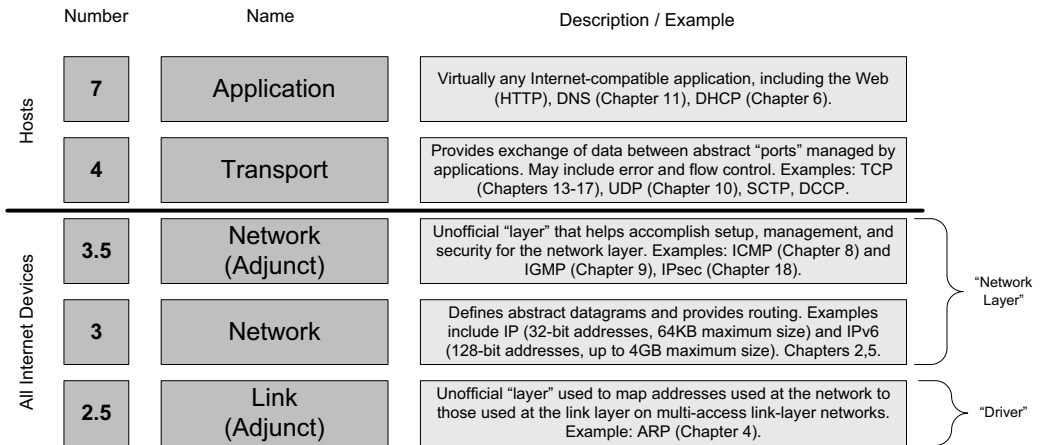


Figure 1-5 Protocol layering based on the ARM or TCP/IP suite used in the Internet. There are no official session or presentation layers. In addition, there are several "adjunct" or helper protocols that do not fit well into the standard layers yet perform critical functions for the operation of the other protocols. Some of these protocols are not used with IPv6 (e.g., IGMP and ARP).

At layer number 3 in Figure 1-5 we find IP, the main network-layer protocol for the TCP/IP suite. We discuss it in detail in Chapter 5. The PDU that IP sends to link-layer protocols is called an *IP datagram* and may be as large as 64KB (and up to 4GB for IPv6). In many cases we shall use the simpler term *packet* to mean an IP datagram when the usage context is clear. Fitting large packets into link-layer PDUs (called *frames*) that may be smaller is handled by a function called *fragmentation* that may be performed by IP hosts and some routers when necessary. In fragmentation, portions of a larger datagram are sent in multiple smaller datagrams called *fragments* and put back together (called *reassembly*) when reaching the destination. We discuss fragmentation in Chapter 10.

Throughout the text we shall use the term IP to refer to both IP versions 4 and 6. We use the term IPv6 to refer to IP version 6, and IPv4 to refer to IP version 4, currently the most popular version. When discussing architecture, the details of IPv4 versus IPv6 matter little. When we delve into the way particular addressing and configuration functions work (Chapter 2 and Chapter 6), for example, these details will become more important.

Because IP packets are datagrams, each one contains the address of the layer 3 sender and recipient. These addresses are called *IP addresses* and are 32 bits long for IPv4 and 128 bits long for IPv6; we discuss them in detail in Chapter 2. This difference in IP address size is the characteristic that most differentiates IPv4 from IPv6. The destination address of each datagram is used to determine where each datagram should be sent, and the process of making this determination and sending the datagram to its next hop is called *forwarding*. Both routers and hosts perform forwarding, although routers tend to do it much more often. There are three

types of IP addresses, and the type affects how forwarding is performed: *unicast* (destined for a single host), *broadcast* (destined for all hosts on a given network), and *multicast* (destined for a set of hosts that belong to a multicast group). Chapter 2 looks at the types of addresses used with IP in more detail.

The *Internet Control Message Protocol* (ICMP) is an adjunct to IP, and we label it as a layer 3.5 protocol. It is used by the IP layer to exchange error messages and other vital information with the IP layer in another host or router. There are two versions of ICMP: ICMPv4, used with IPv4, and ICMPv6, used with IPv6. ICMPv6 is considerably more complex and includes functions such as address autoconfiguration and Neighbor Discovery that are handled by other protocols (e.g., ARP) on IPv4 networks. Although ICMP is used primarily by IP, it is also possible for applications to use it. Indeed, we will see that two popular diagnostic tools, *ping* and *traceroute*, use ICMP. ICMP messages are encapsulated within IP datagrams in the same way transport layer PDUs are.

The *Internet Group Management Protocol* (IGMP) is another protocol adjunct to IPv4. It is used with multicast addressing and delivery to manage which hosts are members of a *multicast group* (a group of receivers interested in receiving traffic for a particular multicast destination address). We describe the general properties of broadcasting and multicasting, along with IGMP and the *Multicast Listener Discovery* protocol (MLD, used with IPv6), in Chapter 9.

At layer 4, the two most common Internet transport protocols are vastly different. The most widely used, the *Transmission Control Protocol* (TCP), deals with problems such as packet loss, duplication, and reordering that are not repaired by the IP layer. It operates in a connection-oriented (VC) fashion and does not preserve message boundaries. Conversely, the *User Datagram Protocol* (UDP) provides little more than the features provided by IP. UDP allows applications to send datagrams that preserve message boundaries but imposes no rate control or error control.

TCP provides a reliable flow of data between two hosts. It is concerned with things such as dividing the data passed to it from the application into appropriately sized chunks for the network layer below, acknowledging received packets, and setting timeouts to make certain the other end acknowledges packets that are sent, and because this reliable flow of data is provided by the transport layer, the application layer can ignore all these details. The PDU that TCP sends to IP is called a *TCP segment*.

UDP, on the other hand, provides a much simpler service to the application layer. It allows datagrams to be sent from one host to another, but there is no guarantee that the datagrams reach the other end. Any desired reliability must be added by the application layer. Indeed, about all that UDP provides is a set of port numbers for multiplexing and demultiplexing data, plus a data integrity checksum. As we can see, UDP and TCP differ radically even though they are at the same layer. There is a use for each type of transport protocol, which we will see when we look at the different applications that use TCP and UDP.

There are two additional transport-layer protocols that are relatively new and available on some systems. As they are not yet very widespread, we do not devote much discussion to them, but they are worth being aware of. The first is the *Datagram Congestion Control Protocol* (DCCP), specified in [RFC4340]. It provides a type of service midway between TCP and UDP: connection-oriented exchange of unreliable datagrams but with congestion control. Congestion control comprises a number of techniques whereby a sender is limited to a sending rate in order to avoid overwhelming the network. We discuss it in detail with respect to TCP in Chapter 16.

The other transport protocol available on some systems is called the *Stream Control Transmission Protocol* (SCTP), specified in [RFC4960]. SCTP provides reliable delivery like TCP but does not require the sequencing of data to be strictly maintained. It also allows for multiple streams to logically be carried on the same connection and provides a message abstraction, which differs from TCP. SCTP was designed for carrying signaling messages on IP networks that resemble those used in the telephone network.

Above the transport layer, the application layer handles the details of the particular application. There are many common applications that almost every implementation of TCP/IP provides. The application layer is concerned with the details of the application and not with the movement of data across the network. The lower three layers are the opposite: they know nothing about the application but handle all the communication details.

1.3.2 Multiplexing, Demultiplexing, and Encapsulation in TCP/IP

We have already discussed the basics of protocol multiplexing, demultiplexing, and encapsulation. At each layer there is an identifier that allows a receiving system to determine which protocol or data stream belongs together. Usually there is also addressing information at each layer. This information is used to ensure that a PDU has been delivered to the right place. Figure 1-6 shows how demultiplexing works in a hypothetical Internet host.

Although it is not really part of the TCP/IP suite, we shall begin bottom-up and mention how demultiplexing from the link layer is performed, using Ethernet as an example. We discuss several link-layer protocols in Chapter 3. An arriving Ethernet frame contains a 48-bit destination address (also called a link-layer or MAC—Media Access Control—address) and a 16-bit field called the *Ethernet type*. A value of 0x0800 (hexadecimal) indicates that the frame contains an IPv4 datagram. Values of 0x0806 and 0x86DD indicate ARP and IPv6, respectively. Assuming that the destination address matches one of the receiving system's addresses, the frame is received and checked for errors, and the *Ethernet Type* field value is used to select which network-layer protocol should process it.

Assuming that the received frame contains an IP datagram, the Ethernet header and trailer information is removed, and the remaining bytes (which constitute the frame's *payload*) are given to IP for processing. IP checks a number of items, including the destination IP address in the datagram. If the destination

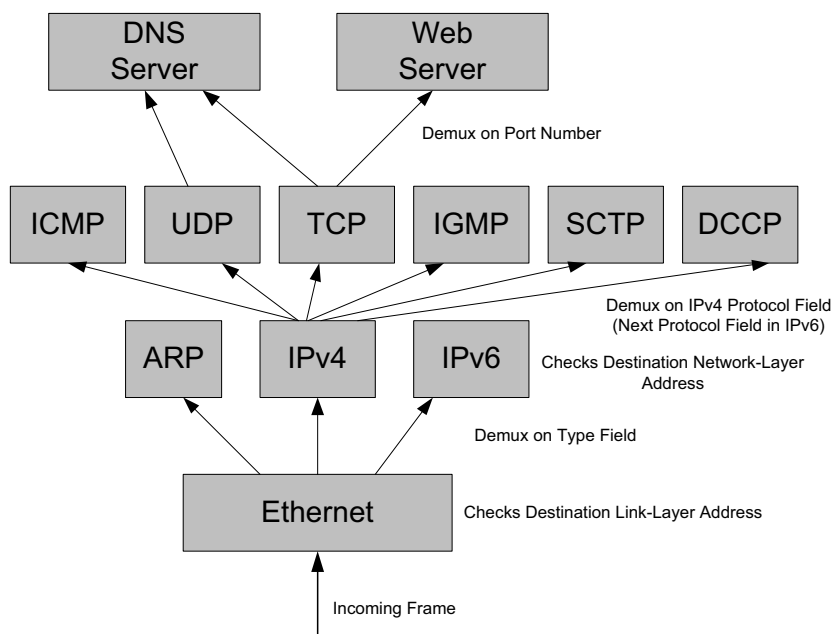


Figure 1-6 The TCP/IP stack uses a combination of addressing information and protocol demultiplexing identifiers to determine if a datagram has been received correctly and, if so, what entity should process it. Several layers also check numeric values (e.g., checksums) to ensure that the contents have not been damaged in transit.

address matches one of its own and the datagram contains no errors in its header (IP does not check its payload), the 8-bit IPv4 *Protocol* field (called *Next Header* in IPv6) is checked to determine which protocol to invoke next. Common values include 1 (ICMP), 2 (IGMP), 4 (IPv4), 6 (TCP), and 17 (UDP). The value of 4 (and 41, which indicates IPv6) is interesting because it indicates the possibility that an IP datagram may appear inside the payload area of an IP datagram. This violates the original concepts of layering and encapsulation but is the basis for a powerful technique known as tunneling, which we discuss more in Chapter 3.

Once the network layer (IPv4 or IPv6) determines that the incoming datagram is valid and the correct transport protocol has been determined, the resulting datagram (reassembled from fragments if necessary) is passed to the transport layer for processing. At the transport layer, most protocols (including TCP and UDP) use port numbers for demultiplexing to the appropriate receiving application.

1.3.3 Port Numbers

Port numbers are 16-bit nonnegative integers (i.e., range 0–65535). These numbers are abstract and do not refer to anything physical. Instead, each IP address has 65,536 associated port numbers for each transport protocol that uses port numbers

(most do), and they are used for determining the correct receiving application. For client/server applications (see Section 1.5.1), a server first “binds” to a port number, and subsequently one or more clients establish connections to the port number using a particular transport protocol on a particular machine. In this sense, port numbers act more like telephone number extensions, except they are usually assigned by standards.

Standard port numbers are assigned by the Internet Assigned Numbers Authority (IANA). The set of numbers is divided into special ranges, including the *well-known* port numbers (0–1023), the *registered* port numbers (1024–49151), and the *dynamic/private* port numbers (49152–65535). Traditionally, servers wishing to bind to (i.e., offer service on) a well-known port require special privileges such as administrator or “root” access.

The range of well-known ports is used for identifying many well-known services such as the *Secure Shell Protocol* (SSH, port 22), FTP (ports 20 and 21), *Telnet* remote terminal protocol (port 23), e-mail/*Simple Mail Transfer Protocol* (SMTP, port 25), *Domain Name System* (DNS, port 53), the *Hypertext Transfer Protocol* or Web (HTTP and HTTPS, ports 80 and 443), *Interactive Mail Access Protocol* (IMAP and IMAPS, ports 143 and 993), *Simple Network Management Protocol* (SNMP, ports 161 and 162), *Lightweight Directory Access Protocol* (LDAP, port 389), and several others. Protocols with multiple ports (e.g., HTTP and HTTPS) often have different port numbers depending on whether *Transport Layer Security* (TLS) is being used with the base application-layer protocol (see Chapter 18).

Note

If we examine the port numbers for these standard services and other standard TCP/IP services (Telnet, FTP, SMTP, etc.), we see that most are odd numbers. This is historical, as these port numbers are derived from the NCP port numbers. (NCP, the Network Control Protocol, preceded TCP as a transport-layer protocol for the ARPANET.) NCP was simplex, not full duplex, so each application required two connections, and an even-odd pair of port numbers was reserved for each application. When TCP and UDP became the standard transport layers, only a single port number was needed per application, yet the odd port numbers from NCP were used.

The registered port numbers are available to clients or servers with special privileges, but IANA keeps a reserved registry for particular uses, so these port numbers should generally be avoided when developing new applications unless an IANA allocation has been procured. The dynamic/private port numbers are essentially unregulated. As we will see, in some circumstances (e.g., on clients) the value of the port number matters little because the port number being used is transient. Such port numbers are also called *ephemeral* port numbers. They are considered to be temporary because a client typically needs one only as long as the user running the client needs service, and the client does not need to be found by

the server in order to establish a connection. Servers, conversely, generally require names and port numbers that do not change often in order to be found by clients.

1.3.4 Names, Addresses, and the DNS

With TCP/IP, each link-layer interface on each computer (including routers) has at least one IP address. IP addresses are enough to identify a host, but they are not very convenient for humans to remember or manipulate (especially the long addresses used with IPv6). In the TCP/IP world, the DNS is a distributed database that provides the mapping between host names and IP addresses (and vice versa). Names are set up in a hierarchy, ending in *domains* such as .com, .org, .gov, .in, .uk, and .edu. Perhaps surprisingly, DNS is an application-layer protocol and thus depends on the other protocols in order to operate. Although most of the TCP/IP suite does not use or care about names, typical users (e.g., those using Web browsers) use names frequently, so if the DNS fails to function properly, normal Internet access is effectively disabled. Chapter 11 looks into the DNS in detail.

Applications that manipulate names can call a standard API function (see Section 1.5.3) to look up the IP address (or addresses) corresponding to a given host's name. Similarly, a function is provided to do the reverse lookup—given an IP address, look up the corresponding host name. Most applications that take a host name as input also take an IP address. Web browsers support this capability. For example, the *Uniform Resource Locators* (URLs) `http://131.243.2.201/index.html` and `http://[2001:400:610:102::c9]/index.html` can be typed into a Web browser and are both effectively equivalent to `http://ee.lbl.gov/index.html` (at the time of writing; the second example requires IPv6 connectivity to be successful).

1.4 Internets, Intranets, and Extranets

As suggested previously, the Internet has developed as the aggregate network resulting from the interconnection of constituent networks over time. The lower-case *internet* means multiple networks connected together, using a common protocol suite. The uppercase *Internet* refers to the collection of hosts around the world that can communicate with each other using TCP/IP. The Internet is an internet, but the reverse is not true.

One of the reasons for the phenomenal growth in networking during the 1980s was the realization that isolated groups of stand-alone computers made little sense. A few stand-alone systems were connected together into a *network*. Although this was a step forward, during the 1990s we realized that separate networks that could not interoperate were not as valuable as a bigger network that could. This notion is the basis for the so-called Metcalfe's Law, which states roughly that the value of a computer network is proportional to the square of the number of connected endpoints (e.g., users or devices). The Internet idea, and its supporting protocols, would make possible the interconnection of different networks. This deceptively simple concept turns out to be remarkably powerful.

2

The Internet Address Architecture

2.1 Introduction

This chapter deals with the structure of network-layer addresses used in the Internet, also known as IP addresses. We discuss how addresses are allocated and assigned to devices on the Internet, the way hierarchy in address assignment aids routing scalability, and the use of special-purpose addresses, including broadcast, multicast, and anycast addresses. We also discuss how the structure and use of IPv4 and IPv6 addresses differ.

Every device connected to the Internet has at least one IP address. Devices used in private networks based on the TCP/IP protocols also require IP addresses. In either case, the forwarding procedures implemented by IP routers (see Chapter 5) use IP addresses to identify where traffic is going. IP addresses also indicate where traffic has come from. IP addresses are similar in some ways to telephone numbers, but whereas telephone numbers are often known and used directly by end users, IP addresses are often shielded from a user's view by the Internet's DNS (see Chapter 11), which allows most users to use names instead of numbers. Users are confronted with manipulating IP addresses when they are required to set up networks themselves or when the DNS has failed for some reason. To understand how the Internet identifies hosts and routers and delivers traffic between them, we must understand the role of IP addresses. We are therefore interested in their administration, structure, and uses.

When devices are attached to the global Internet, they are assigned addresses that must be coordinated so as to not duplicate other addresses in use on the network. For private networks, the IP addresses being used must be coordinated to avoid similar overlaps within the private networks. Groups of IP addresses are *allocated* to users and organizations. The recipients of the allocated addresses then

assign addresses to devices, usually according to some network “numbering plan.” For global Internet addresses, a hierarchical system of administrative entities helps in allocating addresses to users and service providers. Individual users typically receive address allocations from *Internet service providers* (ISPs) that provide both the addresses and the promise of routing traffic in exchange for a fee.

2.2 Expressing IP Addresses

The vast majority of Internet users who are familiar with IP addresses understand the most popular type: IPv4 addresses. Such addresses are often represented in so-called dotted-quad or dotted-decimal notation, for example, 165.195.130.107. The dotted-quad notation consists of four decimal numbers separated by periods. Each such number is a nonnegative integer in the range [0, 255] and represents one-quarter of the entire IP address. The dotted-quad notation is simply a way of writing the whole IPv4 address—a 32-bit nonnegative integer used throughout the Internet system—using convenient decimal numbers. In many circumstances we will be concerned with the binary structure of the address. A number of Internet sites, such as <http://www.subnetmask.info> and <http://www.subnet-calculator.com>, now contain calculators for converting between formats of IP addresses and related information. Table 2-1 gives a few examples of IPv4 addresses and their corresponding binary representations, to get started.

Table 2-1 Example IPv4 addresses written in dotted-quad and binary notation

Dotted-Quad Representation	Binary Representation
0.0.0.0	00000000 00000000 00000000 00000000
1.2.3.4	00000001 00000010 00000011 00000100
10.0.0.255	00001010 00000000 00000000 11111111
165.195.130.107	10100101 11000011 10000010 01101011
255.255.255.255	11111111 11111111 11111111 11111111

In IPv6, addresses are 128 bits in length, four times larger than IPv4 addresses, and generally speaking are less familiar to most users. The conventional notation adopted for IPv6 addresses is a series of four hexadecimal (“hex,” or base-16) numbers called *blocks* or *fields* separated by colons. An example IPv6 address containing eight blocks would be written as 5f05:2000:80ad:5800:0058:0800:2023:1d71. Although not as familiar to users as decimal numbers, hexadecimal numbers make the task of converting to binary somewhat simpler. In addition, a number of agreed-upon simplifications have been standardized for expressing IPv6 addresses [RFC4291]:

1. Leading zeros of a block need not be written. In the preceding example, the address could have been written as 5f05:2000:80ad:5800:58:800:2023:1d71.
2. Blocks of all zeros can be omitted and replaced by the notation ::. For example, the IPv6 address 0:0:0:0:0:0:1 can be written more compactly as ::1. Similarly, the address 2001:0db8:0:0:0:0:0:2 can be written more compactly as 2001:db8::2. To avoid ambiguities, the :: notation may be used only once in an IPv6 address.
3. Embedded IPv4 addresses represented in the IPv6 format can use a form of hybrid notation in which the block immediately preceding the IPv4 portion of the address has the value ffff and the remaining part of the address is formatted using dotted-quad. For example, the IPv6 address ::ffff:10.0.0.1 represents the IPv4 address 10.0.0.1. This is called an *IPv4-mapped IPv6 address*.
4. A conventional notation is adopted in which the low-order 32 bits of the IPv6 address can be written using dotted-quad notation. The IPv6 address ::0102:f001 is therefore equivalent to the address ::1.2.240.1. This is called an *IPv4-compatible IPv6 address*. Note that IPv4-compatible addresses are not the same as IPv4-mapped addresses; they are compatible only in the sense that they can be written down or manipulated by software in a way similar to IPv4 addresses. This type of addressing was originally required for transition plans between IPv4 and IPv6 but is now no longer required [RFC4291].

Table 2-2 presents some examples of IPv6 addresses and their binary representations.

Table 2-2 Examples of IPv6 addresses and their binary representations

Hex Notation	Binary Representation
5f05:2000:80ad:5800:58:800:2023:1d71	0101111100000101 0010000000000000 1000000010101101 0101100000000000 0000000001011000 0000100000000000 0010000000100011 0001110101110001
::1	0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000001
::1.2.240.1 or ::102:f001	0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000100000010 1111000000000001

In some circumstances (e.g., when expressing a URL containing an address) the colon delimiter in an IPv6 address may be confused with another separator such as the colon used between an IP address and a port number. In such circumstances, bracket characters, [and], are used to surround the IPv6 address. For example, the URL

```
http://[2001:0db8:85a3:08d3:1319:8a2e:0370:7344]:443/
```

refers to port number 443 on IPv6 host 2001:0db8:85a3:08d3:1319:8a2e:0370:7344 using the HTTP/TCP/IPv6 protocols.

The flexibility provided by [RFC4291] resulted in unnecessary confusion due to the ability to represent the same IPv6 address in multiple ways. To remedy this situation, [RFC5952] imposes some rules to narrow the range of options while remaining compatible with [RFC4291]. They are as follows:

1. Leading zeros must be suppressed (e.g., 2001:0db8::0022 becomes 2001:db8::22).
2. The :: construct must be used to its maximum possible effect (most zeros suppressed) but not for only 16-bit blocks. If multiple blocks contain equal-length runs of zeros, the first is replaced with ::.
3. The hexadecimal digits a through f should be represented in lowercase.

In most cases, we too will abide by these rules.

2.3 Basic IP Address Structure

IPv4 has 4,294,967,296 possible addresses in its address space, and IPv6 has 340,282,366,920,938,463,463,374,607,431,768,211,456. Because of the large number of addresses (especially for IPv6), it is convenient to divide the address space into chunks. IP addresses are grouped by type and size. Most of the IPv4 address chunks are eventually subdivided down to a single address and used to identify a single network interface of a computer attached to the Internet or to some private intranet. These addresses are called *unicast* addresses. Most of the IPv4 address space is unicast address space. Most of the IPv6 address space is not currently being used. Beyond unicast addresses, other types of addresses include broadcast, multicast, and anycast, which may refer to more than one interface, plus some special-purpose addresses we will discuss later. Before we begin with the details of the current address structure, it is useful to understand the historical evolution of IP addresses.

2.3.1 Classful Addressing

When the Internet's address structure was originally defined, every unicast IP address had a *network* portion, to identify the network on which the interface using

the IP address was to be found, and a *host* portion, used to identify the particular host on the network given in the network portion. Thus, some number of contiguous bits in the address became known as the *net number*, and remaining bits were known as the *host number*. At the time, most hosts had only a single network interface, so the terms *interface address* and *host address* were used somewhat interchangeably.

With the realization that different networks might have different numbers of hosts, and that each host requires a unique IP address, a partitioning was devised wherein different-size allocation units of IP address space could be given out to different sites, based on their current and projected number of hosts. The partitioning of the address space involved five *classes*. Each class represented a different trade-off in the number of bits of a 32-bit IPv4 address devoted to the network number versus the number of bits devoted to the host number. Figure 2-1 shows the basic idea.

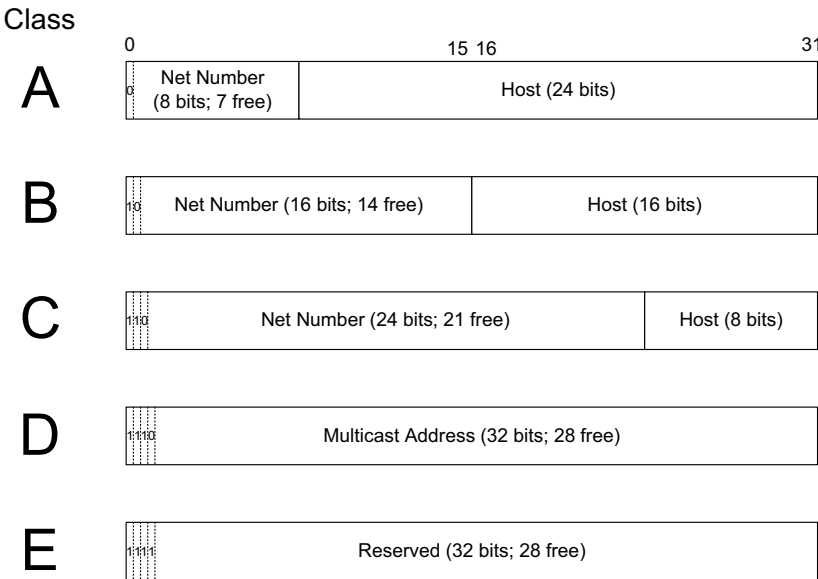


Figure 2-1 The IPv4 address space was originally divided into five classes. Classes A, B, and C were used for assigning addresses to interfaces on the Internet (unicast addresses) and for some other special-case uses. The classes are defined by the first few bits in the address: 0 for class A, 10 for class B, 110 for class C, and so on. Class D addresses are for multicast use (see Chapter 9), and class E addresses remain reserved.

Here we see that the five classes are named A, B, C, D, and E. The A, B, and C class spaces were used for unicast addresses. If we look more carefully at this addressing structure, we can see how the relative sizes of the different classes and their corresponding address ranges really work. Table 2-3 gives this class structure (sometimes called *classful addressing* structure).

Table 2-3 The original (“classful”) IPv4 address space partitioning

Class	Address Range	High-Order Bits	Use	Fraction of Total	Number of Nets	Number of Hosts
A	0.0.0.0–127.255.255.255	0	Unicast/special	1/2	128	16,777,216
B	128.0.0.0–191.255.255.255	10	Unicast/special	1/4	16,384	65,536
C	192.0.0.0–223.255.255.255	110	Unicast/special	1/8	2,097,152	256
D	224.0.0.0–239.255.255.255	1110	Multicast	1/16	N/A	N/A
E	240.0.0.0–255.255.255.255	1111	Reserved	1/16	N/A	N/A

The table indicates how the classful addressing structure was used primarily to have a way of allocating unicast address blocks of different sizes to users. The partitioning into classes induces a trade-off between the number of available network numbers of a given size and the number of hosts that can be assigned to the given network. For example, a site allocated the class A network number 18.0.0.0 (MIT) has 2^{24} possible addresses to assign as host addresses (i.e., using IPv4 addresses in the range 18.0.0.0–18.255.255.255), but there are only 127 class A networks available for the entire Internet. A site allocated a class C network number, say, 192.125.3.0, would be able to assign only 256 hosts (i.e., those in the range 192.125.3.0–192.125.3.255), but there are more than two million class C network numbers available.

Note

These numbers are not exact. Several addresses are not generally available for use as unicast addresses. In particular, the first and last addresses of the range are not generally available. In our example, the site assigned address range 18.0.0.0 would really be able to assign as many as $2^{24} - 2 = 16,777,214$ unicast IP addresses.

The classful approach to Internet addressing lasted mostly intact for the first decade of the Internet’s growth (to about the early 1980s). After that, it began to show its first signs of scaling problems—it was becoming too inconvenient to centrally coordinate the allocation of a new class A, B, or C network number every time a new network segment was added to the Internet. In addition, assigning class A and B network numbers tended to waste too many host numbers, whereas class C network numbers could not provide enough host numbers to many new sites.

2.3.2 Subnet Addressing

One of the earliest difficulties encountered when the Internet began to grow was the inconvenience of having to allocate a new network number for any new network segment that was to be attached to the Internet. This became especially

cumbersome with the development and increasing use of local area networks (LANs) in the early 1980s. To address the problem, it was natural to consider a way that a site attached to the Internet could be allocated a network number centrally that could then be subdivided locally by site administrators. If this could be accomplished without altering the rest of the Internet's core routing infrastructure, so much the better.

Implementing this idea would require the ability to alter the line between the network portion of an IP address and the host portion, but only for local purposes at a site; the rest of the Internet would “see” only the traditional class A, B, and C partitions. The approach adopted to support this capability is called *subnet addressing* [RFC0950]. Using subnet addressing, a site is allocated a class A, B, or C network number, leaving some number of remaining host bits to be further allocated and assigned within a site. The site may further divide the host portion of its base address allocation into a *subnetwork* (subnet) number and a host number. Essentially, subnet addressing adds one additional field to the IP address structure, but without adding any bits to its length. As a result, a site administrator is able to trade off the number of subnetworks versus the number of hosts expected to be on each subnetwork without having to coordinate with other sites.

In exchange for the additional flexibility provided by subnet addressing, a new cost is imposed. Because the definition of the *Subnet* and *Host* fields is now site-specific (not dictated by the class of the network number), all routers and hosts at a site require a new way to determine where the *Subnet* field of the address and the *Host* field of the address are located within the address. Before subnets, this information could be derived directly by knowing whether a network number was from class A, B, or C (as indicated by the first few bits in the address). As an example, using subnet addressing, an IPv4 address might have the form shown in Figure 2-2.

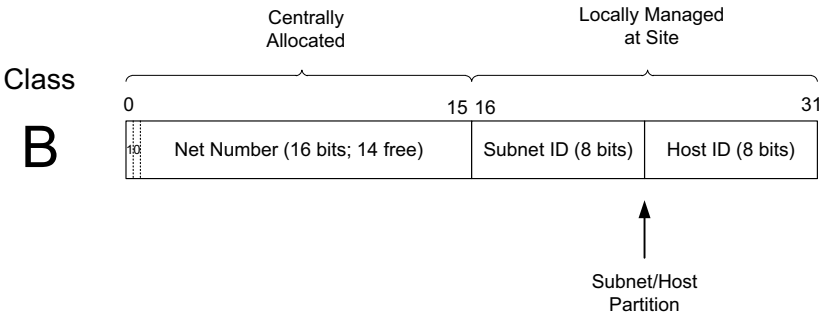


Figure 2-2 An example of a subnetted class B address. Using 8 bits for the subnet ID provides for 256 subnets with 254 hosts on each of the subnets. This partitioning may be altered by the network administrator.

Figure 2-2 is an example of how a class B address might be “subnetted.” Assume that some site in the Internet has been allocated a class B network number. The first 16 bits of every address the site will use are fixed at some particular number because these bits have been allocated by a central authority. The last 16 bits (which would have been used only to create host numbers in the class B network without subnets) can now be divided by the site network administrator as needs may dictate. In this example, 8 bits have been chosen for the subnet number, leaving 8 bits for host numbers. This particular configuration allows the site to support 256 subnetworks, and each subnetwork may contain up to 254 hosts (now the first and last addresses for each subnetwork are not available, as opposed to losing only the first and last addresses of the entire allocated range). Recall that the subnetwork structure is known only by hosts and routers where the subnetting is taking place. The remainder of the Internet still treats any address associated with the site just as it did prior to the advent of subnet addressing. Figure 2-3 shows how this works.

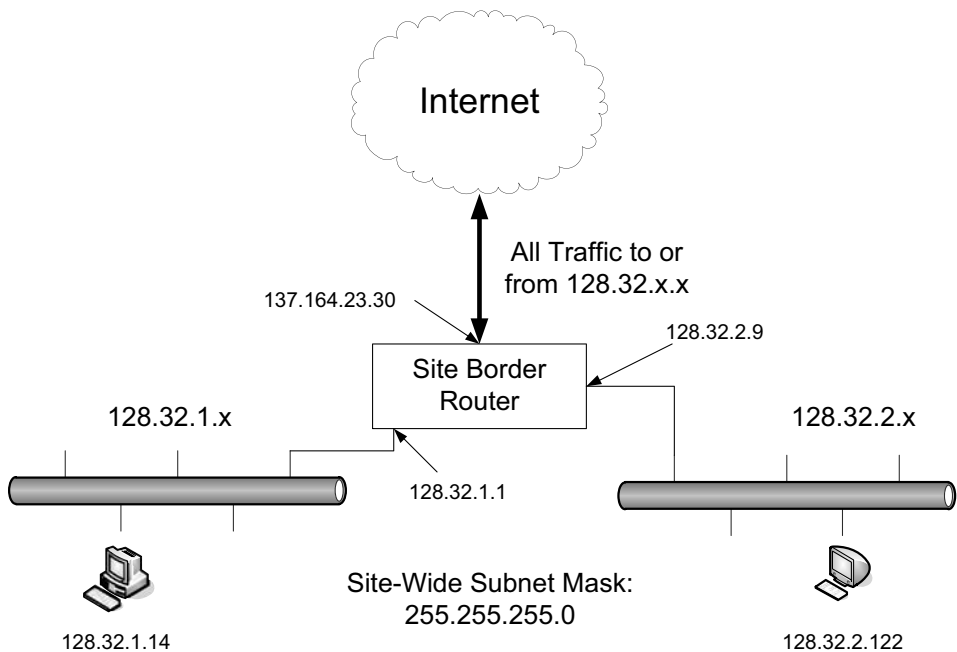


Figure 2-3 A site is allocated the classical class B network number 128.32. The network administrator decides to apply a site-wide subnet mask of 255.255.255.0, giving 256 subnetworks where each subnetwork can hold $256 - 2 = 254$ hosts. The IPv4 address of each host on the same subnet has the subnet number in common. All of the IPv4 addresses of hosts on the left-hand LAN segment start with 128.32.1, and all of those on the right start with 128.32.2.

This figure shows a hypothetical site attached to the Internet with one border router (i.e., one attachment point to the Internet) and two internal local area networks. The value of x could be anything in the range $[0, 255]$. Each of the Ethernet networks is an IPv4 subnetwork of the overall network number 128.32, a class B address allocation. For other sites on the Internet to reach this site, all traffic with destination addresses starting with 128.32 is directed by the Internet routing system to the border router (specifically, its interface with IPv4 address 137.164.23.30). At this point, the border router must distinguish among different subnetworks within the 128.32 network. In particular, it must be able to distinguish and separate traffic destined for addresses of the form 128.32.1. x from those destined for addresses of the form 128.32.2. x . These represent subnetwork numbers 1 and 2, respectively, of the 128.32 class B network number. In order to do this, the router must be aware of where the subnet ID is to be found within the addresses. This is accomplished by a configuration parameter we will discuss next.

2.3.3 Subnet Masks

The *subnet mask* is an assignment of bits used by a host or router to determine how the network and subnetwork information is partitioned from the host information in a corresponding IP address. Subnet masks for IP are the same length as the corresponding IP addresses (32 bits for IPv4 and 128 bits for IPv6). They are typically configured into a host or router in the same way as IP addresses—either statically (typical for routers) or using some dynamic system such as the *Dynamic Host Configuration Protocol* (DHCP; see Chapter 6). For IPv4, subnet masks may be written in the same way an IPv4 address is written (i.e., dotted-decimal). Although not originally required to be arranged in this manner, today subnet masks are structured as some number of 1 bits followed by some number of 0 bits. Because of this arrangement, it is possible to use a shorthand format for expressing masks that simply gives the number of contiguous 1 bits in the mask (starting from the left). This format is now the most common format and is sometimes called the *prefix length*. Table 2-4 presents some examples for IPv4.

Table 2-4 IPv4 subnet mask examples in various formats

Dotted-Decimal Representation	Shorthand (Prefix Length)	Binary Representation
128.0.0.0	/1	10000000 00000000 00000000 00000000
255.0.0.0	/8	11111111 00000000 00000000 00000000
255.192.0.0	/10	11111111 11000000 00000000 00000000
255.255.0.0	/16	11111111 11111111 00000000 00000000
255.255.254.0	/23	11111111 11111111 11111110 00000000
255.255.255.192	/27	11111111 11111111 11111111 11100000
255.255.255.255	/32	11111111 11111111 11111111 11111111

Table 2-5 IPv6 subnet mask examples in various formats

Hex Notation	Shorthand (Prefix Length)	Binary Representation
ffff:ffff:ffff:ffff::	/64	1111111111111111 1111111111111111 1111111111111111 1111111111111111 0000000000000000 0000000000000000 0000000000000000 0000000000000000
ff00::	/8	1111111100000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000

Table 2-5 presents some examples for IPv6.

Masks are used by routers and hosts to determine where the network/sub-network portion of an IP address ends and the host part begins. A bit set to 1 in the subnet mask means the corresponding bit position in an IP address should be considered part of a combined network/subnetwork portion of an address, which is used as the basis for forwarding datagrams (see Chapter 5). Conversely, a bit set to 0 in the subnet mask means the corresponding bit position in an IP address should be considered part of the host portion. For example, in Figure 2-4 we can see how the IPv4 address 128.32.1.14 is treated when a subnet mask of 255.255.255.0 is applied to it.

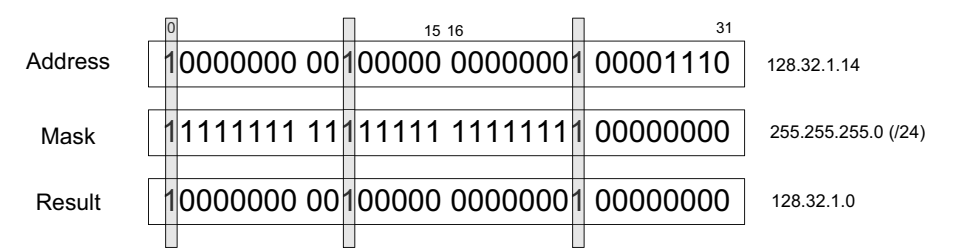


Figure 2-4 An IP address can be combined with a subnet mask using a bitwise AND operation in order to form the network/subnetwork identifier (prefix) of the address used for routing. In this example, applying a mask of length 24 to the IPv4 address 128.32.1.14 gives the prefix 128.32.1.0/24.

Here we see how each bit in the address is ANDed with each corresponding bit in the subnet mask. Recalling the bitwise AND operation, a result bit is only ever a 1 if the corresponding bits in both the mask and the address are 1. In this example, we see that the address 128.32.1.14 belongs to the subnet 128.32.1.0/24. In Figure 2-3, this is precisely the information required by the border router to

determine to which subnetwork a datagram destined for the system with address 128.32.1.14 should be forwarded. Note again that the rest of the Internet routing system does not require knowledge of the subnet mask because routers outside the site make routing decisions based only on the network number portion of an address and not the combined network/subnetwork or host portions. Consequently, subnet masks are purely a local matter at the site.

2.3.4 Variable-Length Subnet Masks (VLSM)

So far we have discussed how a network number allocated to a site can be subdivided into ranges assigned to multiple subnetworks, each of the same size and therefore able to support the same number of hosts, based on the operational expectations of the network administrator. We now observe that it is possible to use a different-length subnet mask applied to the same network number in different portions of the same site. Although doing this complicates address configuration management, it adds flexibility to the subnet structure because different subnetworks may be set up with different numbers of hosts. *Variable-length subnet masks* (VLSM) are now supported by most hosts, routers, and routing protocols. To understand how VLSM works, consider the network topology illustrated in Figure 2-5, which extends Figure 2-3 with two additional subnetworks using VLSM.

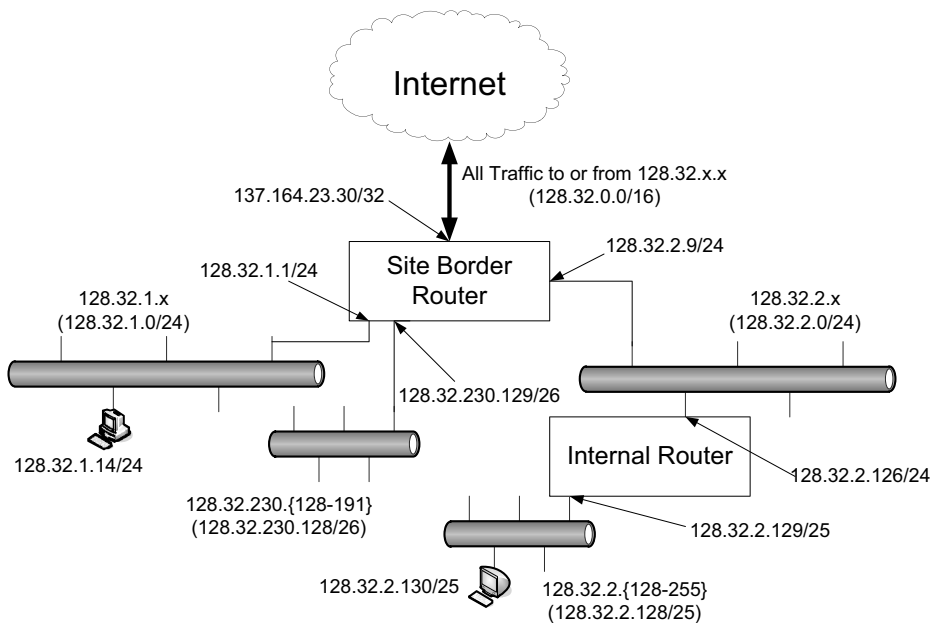


Figure 2-5 VLSM can be used to partition a network number into subnetworks with a differing number of hosts on each subnet. Each router and host is configured with a subnet mask in addition to its IP address. Most software supports VLSM, except for some older routing protocols (e.g., RIP version 1).

In the more complicated and realistic example shown in Figure 2-5, three different subnet masks are used within the site to subnet the 128.32.0.0/16 network: /24, /25, and /26. Doing so provides for a different number of hosts on each subnet. Recall that the number of hosts is constrained by the number of bits remaining in the IP address that are not used by the network/subnet number. For IPv4 and a /24 prefix, this allows for $32 - 24 = 8$ bits (256 hosts); for /25, half as many (128 hosts); and for /26, half further still (64 hosts). Note that each interface on each host and router depicted is now given both an IP address and a subnet mask, but the mask differs across the network topology. With an appropriate dynamic routing protocol running among the routers (e.g., OSPF, IS-IS, RIPv2), traffic is able to flow correctly among hosts at the same site or to/from the outside of the site across the Internet.

Although it may not seem obvious, there is a common case where a subnet contains only two hosts. When routers are connected together by a point-to-point link requiring an IP address to be assigned at each end, it is common practice to use a /31 network prefix with IPv4, and it is now also a recommended practice to use a /127 prefix for IPv6 [RFC6164].

2.3.5 Broadcast Addresses

In each IPv4 subnetwork, a special address is reserved to be the *subnet broadcast address*. The subnet broadcast address is formed by setting the network/subnetwork portion of an IPv4 address to the appropriate value and all the bits in the *Host* field to 1. Consider the left-most subnet from Figure 2-5. Its prefix is 128.32.1.0/24. The subnet broadcast address is constructed by inverting the subnet mask (i.e., changing all the 0 bits to 1 and vice versa) and performing a bitwise OR operation with the address of any of the computers on the subnet (or, equivalently, the network/subnetwork prefix). Recall that the result of a bitwise OR operation is 1 if either input bit is 1. Using the IPv4 address 128.32.1.14, this computation can be written as shown in Figure 2-6.

	0	15	16	31	
Address	10000000	00100000	00000001	00001110	128.32.1.14
Complement of Mask	00000000	00000000	00000000	11111111	0.0.0.255
OR Result	10000000	00100000	00000001	11111111	128.32.1.255

Figure 2-6 The subnet broadcast address is formed by ORing the complement of the subnet mask with the IPv4 address. In this case of a /24 subnet mask, all of the remaining $32 - 24 = 8$ bits are set to 1, giving a decimal value of 255 and the subnet broadcast address of 128.32.1.255.

As shown in the figure, the subnet broadcast address for the subnet 128.32.1.0/24 is 128.32.1.255. Historically, a datagram using this type of address as its destination has also been known as a *directed broadcast*. Such a broadcast can, at least theoretically, be routed through the Internet as a single datagram until reaching the target subnetwork, at which point it becomes a collection of broadcast datagrams that are delivered to all hosts on the subnetwork. Generalizing this idea further, we could form a datagram with the destination IPv4 address 128.32.255.255 and launch it into the Internet attached to the network depicted in Figure 2-3 or Figure 2-5. This would address all hosts at the target site.

Note

Directed broadcasts were found to be such a big problem from a security point of view that they are effectively disabled on the Internet today. [RFC0919] describes the various types of broadcasts for IPv4, and [RFC1812] suggests that support for forwarding directed broadcasts by routers should not only be available but enabled by default. This policy was reversed by [RFC2644] so that by default routers must now disable the forwarding of directed broadcasts and are even free to omit support for the capability altogether.

In addition to the subnet broadcast address, the *special-use* address 255.255.255.255 is reserved as the *local net broadcast* (also called *limited broadcast*), which is never forwarded by routers. (See Section 2.5 for more detail on special-use addresses.) Note that although routers may not forward broadcasts, subnet broadcasts and local net broadcasts destined for the same network to which a computer is attached should be expected to work unless explicitly disabled by end hosts. Such broadcasts do not require action by a router; link-layer broadcast mechanisms, if available, are used for supporting them (see Chapter 3). Broadcast addresses are typically used with protocols such as UDP/IP (Chapter 10) or ICMP (Chapter 8) because these protocols do not involve two-party conversations as in TCP/IP. IPv6 lacks any broadcast addresses; for places where broadcast addresses might be used in IPv4, IPv6 instead uses exclusively multicast addresses (see Chapter 9).

2.3.6 IPv6 Addresses and Interface Identifiers

In addition to being longer than IPv4 addresses by a factor of 4, IPv6 addresses also have some additional structure. Special prefixes used with IPv6 addresses indicate the *scope* of an address. The scope of an IPv6 address refers to the portion of the network where it can be used. Important examples of scopes include *node-local* (the address can be used only for communication on the same computer), *link-local* (used only among nodes on the same network link or IPv6 prefix), or *global* (Internet-wide). In IPv6, most nodes have more than one address in use, often on the same network interface. Although this is supported in IPv4 as well, it

is not nearly as common. The set of addresses required in an IPv6 node, including multicast addresses (see Section 2.5.2), is given in [RFC4291].

Note

Another scope level called *site-local* using prefix fec0::/10 was originally supported by IPv6 but was deprecated for use with unicast addressing by [RFC3879]. The primary problems include how to handle such addresses given that they may be reused in more than one site and a lack of clarity on precisely how to define a “site.”

Link-local IPv6 addresses (and some global IPv6 addresses) use *interface identifiers* (IIDs) as a basis for unicast IPv6 address assignment. IIDs are used as the low-order bits of an IPv6 address in all cases except where the address begins with the binary value 000, and as such they must be unique within the same network prefix. IIDs are ordinarily 64 bits long and are formed either directly from the underlying link-layer MAC address of a network interface using a *modified EUI-64 format* [EUI64], or by another process that randomizes the value in hopes of providing some degree of privacy against address tracking (see Chapter 6).

In IEEE standards, EUI stands for *extended unique identifier*. EUI-64 identifiers start with a 24-bit *Organizationally Unique Identifier* (OUI) followed by a 40-bit *extension identifier* assigned by the organization, which is identified by the first 24 bits. The OUIs are maintained and allocated by the IEEE registration authority [IEEEERA]. EUIs may be “universally administered” or “locally administered.” In the Internet context, such addresses are typically of the universally administered variety.

Many IEEE standards-compliant network interfaces (e.g., Ethernet) have used shorter-format addresses (48-bit EUIs) for years. The only significant difference between the EUI-48 and EUI-64 formats is their length (see Figure 2-7).

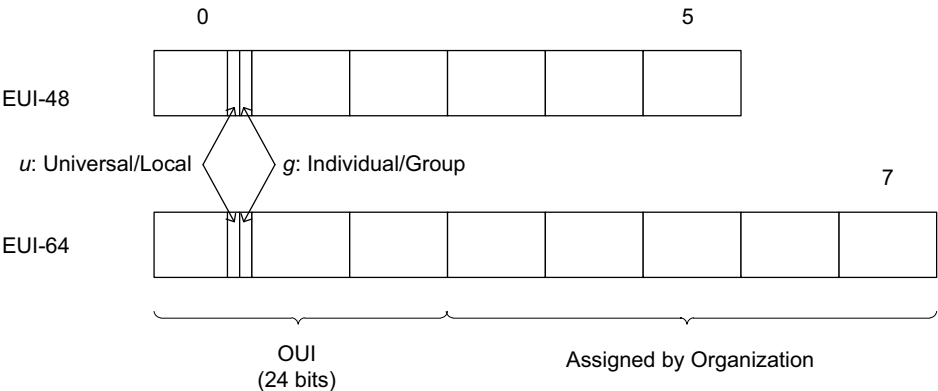


Figure 2-7 The EUI-48 and EUI-64 formats defined by the IEEE. These are used within IPv6 to form interface identifiers by inverting the *u* bit.

The OUI is 24 bits long and occupies the first 3 bytes of both EUI-48 and EUI-64 addresses. The low-order 2 bits of the first bytes of these addresses are designated the *u* and *g* bits, respectively. The *u* bit, when set, indicates that the address is locally administered. The *g* bit, when set, indicates that the address is a group or multicast-type address. For the moment, we are concerned only with cases where the *g* bit is not set.

An EUI-64 can be formed from an EUI-48 by copying the 24-bit OUI value from the EUI-48 address to the EUI-64 address, placing the 16-bit value 1111111111111110 (hex FFFE) in the fourth and fifth bytes of the EUI-64 address, and then copying the remaining organization-assigned bits. For example, the EUI-48 address 00-11-22-33-44-55 would become 00-11-22-FF-FE-33-44-55 in EUI-64. This mapping is the first step used by IPv6 in constructing its interface identifiers when such underlying EUI-48 addresses are available. The modified EUI-64 used to form IIDs for IPv6 addresses simply inverts the *u* bit.

When an IPv6 interface identifier is needed for a type of interface that does not have an EUI-48-bit address provided by its manufacturer, but has some other type of underlying address (e.g., AppleTalk), the underlying address is left-padded with zeros to form the interface identifier. Interface identifiers created for interfaces that lack any form of other identifier (e.g., tunnels, serial links) may be derived from some other interface on the same node (that is not on the same subnet) or from some identifier associated with the node. Lacking any other options, manual assignment is a last resort.

2.3.6.1 Examples

Using the Linux `ifconfig` command, we can investigate the way a link-local IPv6 address is formed:

```
Linux% ifconfig eth1
eth1      Link encap:Ethernet  HWaddr 00:30:48:2A:19:89
          inet addr:12.46.129.28  Bcast:12.46.129.127
          Mask:255.255.255.128
          inet6 addr: fe80::230:48ff:fe2a:1989/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1359970341 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1472870787 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4021555658 (3.7 GiB)  TX bytes:3258456176 (3.0 GiB)
          Base address:0x3040 Memory:f8220000-f8240000
```

Here we can see how the Ethernet's hardware address 00:30:48:2A:19:89 is mapped to an IPv6 address. First, it is converted to EUI-64, forming the address 00:30:48:ff:fe2a:19:89. Next, the *u* bit is inverted, forming the IID value 02:30:48:ff:fe2a:19:89. To complete the link-local IPv6 address, we use the reserved link-local prefix fe80::/10 (see Section 2.5). Together, these form the complete address, fe80::230:48ff:fe2a:1989. The presence of /64 is the

standard length used for identifying the subnetwork/host portion of an IPv6 address derived from an IID as required by [RFC4291].

Another interesting example is from a Windows system with IPv6. In this case, we see a special *tunnel endpoint*, which is used to carry IPv6 traffic through networks that otherwise support only IPv4:

```
c:\> ipconfig /all
...
Tunnel adapter Automatic Tunneling Pseudo-Interface:

    Connection-specific DNS Suffix  . : foo
    Description . . . . . : Automatic Tunneling
                             Pseudo-Interface

    Physical Address. . . . . : 0A-99-8D-87
    Dhcp Enabled. . . . . : No
    IP Address. . . . . : fe80::5efe:10.153.141.135%2
    Default Gateway . . . . . :
    DNS Servers . . . . . : fec0:0:0:ffff::1%2
                           fec0:0:0:ffff::2%2
                           fec0:0:0:ffff::3%2

    NetBIOS over Tcpip. . . . . : Disabled
...
```

In this case, we can see a special tunneling interface called ISATAP [RFC5214]. The so-called physical address is really the hexadecimal encoding of an IPv4 address: 0A-99-8D-87 is the same as 10.153.141.135. Here, the OUI used (00-00-5E) is the one assigned to the IANA [IANA]. It is used in combination with the hex value *fe*, indicating an embedded IPv4 address. This combination is then combined with the standard link-local prefix *fe80::/10* to give the address *fe80::5efe:10.153.141.135*. The *%2* appended to the end of the address is called a *zone ID* in Windows and indicates the interface index number on the computer corresponding to the IPv6 address. IPv6 addresses are often created by a process of automatic configuration, a process we discuss in more detail in Chapter 6.

2.4 CIDR and Aggregation

In the early 1990s, after the adoption of subnet addressing to ease one form of growing pains, the Internet started facing a serious set of scaling problems. Three particular issues were considered so important as to require immediate attention:

1. By 1994, over half of all class B addresses had already been allocated. It was expected that the class B address space would be exhausted by about 1995.
2. The 32-bit IPv4 address was thought to be inadequate to handle the size of the Internet anticipated by the early 2000s.

3. The number of entries in the global routing table (one per network number), about 65,000 in 1995, was growing. As more and more class A, B, and C routing entries appeared, routing performance would suffer.

These three issues were attacked by a group in the IETF called ROAD (for ROuting and ADdressing), starting in 1992. They considered problems 1 and 3 to be of immediate concern, and problem 2 as requiring a long-term solution. The short-term solution they proposed was to effectively remove the class breakdown of IP addresses and also promote the ability to aggregate hierarchically assigned IP addresses. These measures would help problems 1 and 3. IPv6 was envisioned to deal with problem 2.

2.4.1 Prefixes

In order to help relieve the pressure on the availability of IPv4 addresses (especially class B addresses), the classful addressing scheme was generalized using a scheme similar to VLSM, and the Internet routing system was extended to support *Classless Inter-Domain Routing* (CIDR) [RFC4632]. This provided a way to conveniently allocate contiguous address ranges that contained more than 255 hosts but fewer than 65,536. That is, something other than single class B or multiple class C network numbers could be allocated to sites. Using CIDR, any address range is not predefined as being part of a class but instead requires a mask similar to a subnet mask, sometimes called a *CIDR mask*. CIDR masks are not limited to a site but are instead visible to the global routing system. Thus, the core Internet routers must be able to interpret and process masks in addition to network numbers. This combination of numbers, called a network *prefix*, is used for both IPv4 and IPv6 address management.

Eliminating the predefined separation of network and host number within an IP address makes finer-grain allocation of IP address ranges possible. As with classful addressing, dividing the address spaces into chunks is most easily achieved by grouping numerically contiguous addresses for use as a type or for some particular special purpose. Such groupings are now commonly expressed using a prefix of the address space. An n -bit prefix is a predefined value for the first n bits of an address. The value of n (the length of the prefix) is typically expressed as an integer in the range 0–32 for IPv4 and 0–128 for IPv6. It is generally appended to the base IP address following a / character. Table 2-6 gives some examples of prefixes and their corresponding IPv4 or IPv6 address ranges.

In the table, the bits defined and fixed by the prefix are enclosed in a box. The remaining bits may be set to any combination of 0s and 1s, thereby covering the possible address range. Clearly, a smaller prefix length corresponds to a larger number of possible addresses. In addition, the earlier classful addressing approach is easily generalized by this scheme. For example, the class C network number 192.125.3.0 can be written as the prefix 192.125.3.0/24 or 192.125.3/24. Classful A and B network numbers can be expressed using /8 and /16 prefix lengths, respectively.

Table 2-6 Examples of prefixes and their corresponding IPv4 or IPv6 address range

Prefix	Prefix (Binary)	Address Range
0.0.0.0/0	00000000 00000000 00000000 00000000	0.0.0.0–255.255.255.255
128.0.0.0/1	10000000 00000000 00000000 00000000	128.0.0.0–255.255.255.255
128.0.0.0/24	10000000 00000000 00000000 00000000	128.0.0.0–128.0.0.255
198.128.128.192/27	11000110 10000000 10000000 11000000	198.128.128.192–198.128.128.223
165.195.130.107/32	10100101 11000011 10000010 01101011	165.195.130.107
2001:db8::/32	00100000000000001 0000110110111000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000	2001:db8::–2001:db8:ffff:ffff

2.4.2 Aggregation

Removing the classful structure of IP addresses made it possible to allocate IP address blocks in a wider variety of sizes. Doing so, however, did not address the third concern from the list of problems; it did not help to reduce the number of routing table entries. A routing table entry tells a router where to send traffic. Essentially, the router inspects the destination IP address in an arriving datagram, finds a matching routing table entry, and from the entry extracts the “next hop” for the datagram. This is somewhat like driving to a particular address in a car and in every intersection along the way finding a sign indicating what direction to take to get to the next intersection on the way to the destination. If you consider the number of signs that would have to be present at every intersection for every possible destination neighborhood, you get some sense of the problem facing the Internet in the early 1990s.

At the time, few techniques were known to dramatically reduce the number of routing table entries while maintaining shortest-path routes to all destinations in the Internet. The best-known approach was published in a study of *hierarchical routing* [KK77] in the late 1970s by Kleinrock and Kamoun. They observed that if the network topology were arranged as a tree¹ and addresses were assigned in a way that was “sensitive” to this topology, very small routing tables could be used while still maintaining shortest-path routes to all destinations. Consider Figure 2-8.

In this figure, circles represent routers and lines represent network links between them. The left-hand and right-hand sides of the diagram show tree-shaped networks. The difference between them is the way addresses have been assigned to the routers. In the left-hand (a) side, addresses are essentially random—there is no direct relationship between the addresses and the location of

1. In graph theory, a tree is a connected graph with no cycles. For a network of routers and links, this means that there is only one simple (nonduplicative) path between any two routers.

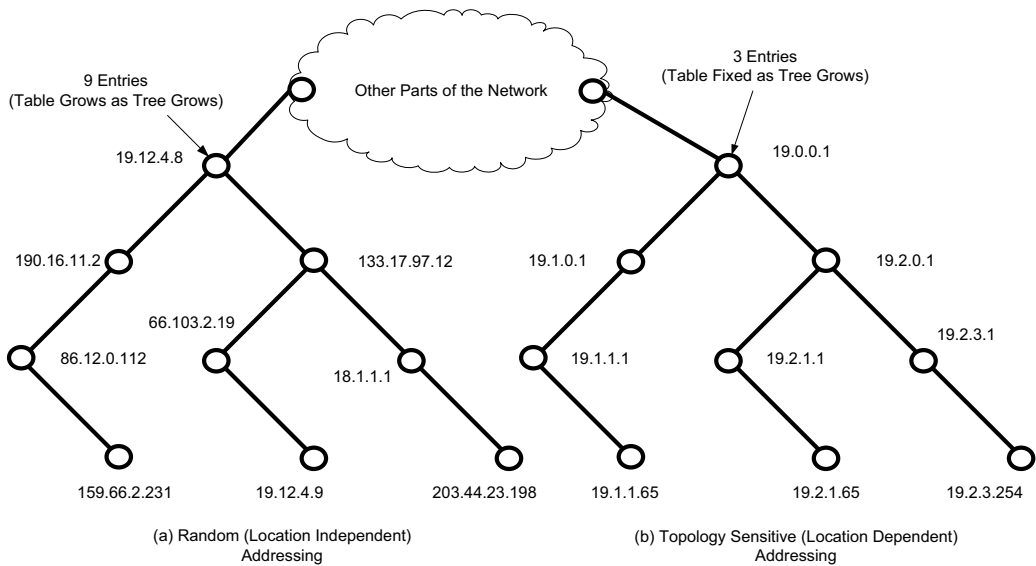


Figure 2-8 In a network with a tree topology, network addresses can be assigned in a special way so as to limit the amount of routing information (“state”) that needs to be stored in a router. If addresses are not assigned in this way (left side), shortest-path routes cannot be guaranteed without storing an amount of state proportional to the number of nodes to be reached. While assigning addresses in a way that is sensitive to the tree topology saves state, if the network topology changes, a reassignment of addresses is generally required.

the routers in the tree. On the right-hand (b) side of the diagram, the addresses are assigned based upon where the router is located in the tree. If we consider the number of entries each top router requires, we see that there is a significant difference.

The root (top) of the tree on the left is the router labeled 19.12.4.8. In order to know a next hop for every possible destination, it needs an entry for all the routers “below” it in the tree: 190.16.11.2, 86.12.0.112, 159.66.2.231, 133.17.97.12, 66.103.2.19, 18.1.1.1, 19.12.4.9, and 203.44.23.198. For any other destination, it simply routes to the cloud labeled “Other Parts of the Network.” This results in a total of nine entries. In contrast, the root of the right-hand tree is labeled 19.0.0.1 and requires only three entries in its routing table. Note that all of the routers on the left side of the right tree begin with the prefix 19.1 and all to the right begin with 19.2. Thus, the table in router 19.0.0.1 need only show 19.1.0.1 as the next hop for any destination starting with 19.1, whereas 19.2.0.1 is the next hop for any destination starting with 19.2. Any other destination goes to the cloud labeled “Other Parts of the Network.” This results in a total of three entries. Note that this behavior is recursive—any router in the (b) side of the tree never requires more entries than the number of links it has. This is a direct result of the special method used to assign the addresses. Even

Table 2-7 IPv4 special-use addresses (defined January 2010)

Prefix	Special Use	Reference
0.0.0.0/8	Hosts on the local network. May be used only as a source IP address.	[RFC1122]
10.0.0.0/8	Address for private networks (intranets). Such addresses never appear on the public Internet.	[RFC1918]
127.0.0.0/8	Internet host loopback addresses (same computer). Typically only 127.0.0.1 is used.	[RFC1122]
169.254.0.0/16	“Link-local” addresses—used only on a single link and generally assigned automatically. See Chapter 6.	[RFC3927]
172.16.0.0/12	Address for private networks (intranets). Such addresses never appear on the public Internet.	[RFC1918]
192.0.0.0/24	IETF protocol assignments (IANA reserved).	[RFC5736]
192.0.2.0/24	TEST-NET-1 addresses approved for use in documentation. Such addresses never appear on the public Internet.	[RFC5737]
192.88.99.0/24	Used for 6to4 relays (anycast addresses).	[RFC3068]
192.168.0.0/16	Address for private networks (intranets). Such addresses never appear on the public Internet.	[RFC1918]
198.18.0.0/15	Used for benchmarks and performance testing.	[RFC2544]
198.51.100.0/24	TEST-NET-2. Approved for use in documentation.	[RFC5737]
203.0.113.0/24	TEST-NET-3. Approved for use in documentation.	[RFC5737]
224.0.0.0/4	IPv4 multicast addresses (formerly class D); used only as destination addresses.	[RFC5771]
240.0.0.0/4	Reserved space (formerly class E), except 255.255.255.255.	[RFC1112]
255.255.255.255/32	Local network (limited) broadcast address.	[RFC0919] [RFC0922]

In IPv6, a number of address ranges and individual addresses are used for specific purposes. They are listed in Table 2-8 [RFC5156].

For both IPv4 and IPv6, address ranges not designated as special, multicast, or reserved are available to be assigned for unicast use. Some unicast address space (prefixes 10/8, 172.16/12, and 192.168/16 for IPv4 and fc00::/7 for IPv6) is reserved for building private networks. Addresses from these ranges can be used by cooperating hosts and routers within a site or organization, but not across the global Internet. Thus, these addresses are sometimes called *nonroutable* addresses. That is, they will not be routed by the public Internet.

The management of private, nonroutable address space is entirely a local decision. The IPv4 private addresses are very common in home networks and for the internal networks of moderately sized and large enterprises. They are frequently used in combination with *network address translation* (NAT), which rewrites IP addresses inside IP datagrams as they enter the Internet. We discuss NAT in detail in Chapter 7.

Table 2-8 IPv6 special-use addresses (defined April 2008)

Prefix	Special Use	Reference
::/0	Default route entry. Not used for addressing.	[RFC5156]
::/128	The unspecified address; may be used as a source IP address.	[RFC4291]
::1/128	The IPv6 host loopback address; not used in datagrams sent outside the local host.	[RFC4291]
::ffff:0:0/96	IPv4-mapped addresses. Such addresses never appear in packet headers. For internal host use only.	[RFC4291]
::[ipv4-address]/96	IPv4-compatible addresses. Deprecated; not to be used.	[RFC4291]
2001::/32	Teredo addresses.	[RFC4380]
2001:10::/28	Overlay Routable Cryptographic Hash Identifiers. Such addresses never appear on the public Internet.	[RFC4843]
2001:db8::/32	Address range used for documentation and for examples. Such addresses never appear on the public Internet.	[RFC3849]
2002::/16	6to4 addresses of 6to4 tunnel relays.	[RFC3056]
3ffe::/16	Used by 6bone experiments. Deprecated; not to be used.	[RFC3701]
5f00::/16	Used by 6bone experiments. Deprecated; not to be used.	[RFC3701]
fc00::/7	Unique, local unicast addresses; not used on the global Internet.	[RFC4193]
fe80::/10	Link-local unicast addresses.	[RFC4291]
ff00::/8	IPv6 multicast addresses; used only as destination addresses.	[RFC4291]

2.5.1 Addressing IPv4/IPv6 Translators

In some networks, it may be attractive to perform translation between IPv4 and IPv6 [RFC6127]. A framework for this has been developed for unicast translations [RFC6144], and one is currently under development for multicast translations [IDv-4v6mc]. One of the basic functions is to provide automated, algorithmic translation of addresses. Using the “well-known” IPv6 prefix 64:ff9b::/96 or another assigned prefix, [RFC6052] specifies how this is accomplished for unicast addresses.

The scheme makes use of a specialized address format called an *IPv4-embedded IPv6 address*. This type of address contains an IPv4 address inside an IPv6 address. It can be encoded using one of six formats, based on the length of the IPv6 prefix, which is required to be one of the following: 32, 40, 48, 56, 64, or 96. The formats available are shown in Figure 2-10.

In the figure, the prefix is either the well-known prefix or a prefix unique to the organization deploying translators. Bits 64–71 must be set to 0 to maintain compatibility with identifiers specified in [RFC4291]. The suffix bits are reserved and should be set to 0. The method to produce an IPv4-embedded IPv6 address is then simple: concatenate the IPv6 prefix with the 32-bit IPv4 address, ensuring that the bits 63–71 are set to 0 (inserting if necessary). Append the suffix as 0 bits until a 128-bit address is produced. IPv4-embedded IPv6 addresses using

Prefix Length	0	31	39	47	63	71	79	87	95	103	127
32	IPv6 Prefix (32 bits)		IPv4 Address (32 bits)		u		Suffix (56 bits)				
40	IPv6 Prefix (40 bits)		IPv4 Address (first 24 bits)		u		IPv4 Address (last 8 bits)	Suffix (48 bits)			
48	IPv6 Prefix (48 bits)			IPv4 Address (first 16 bits)		u		IPv4 Address (last 16 bits)	Suffix (40 bits)		
56	IPv6 Prefix (56 bits)				IPv4 Address (first 8 bits)	u		IPv4 Address (last 24 bits)		Suffix (32 bits)	
64	IPv6 Prefix (64 bits)					u		IPv4 Address (32 bits)		Suffix (24 bits)	
96	IPv6 Prefix (96 bits)									IPv4 Address (32 bits)	

Figure 2-10 IPv4 addresses can be embedded within IPv6 addresses, forming an IPv4-embedded IPv6 address. Six different formats are available, depending on the IPv6 prefix length in use. The well-known prefix 64:ff9b::/96 can be used for automatic translation between IPv4 and IPv6 unicast addresses.

the 96-bit prefix option may be expressed using the convention for IPv6-mapped addresses mentioned previously (Section 2.2(3) of [RFC4291]). For example, embedding the IPv4 address 198.51.100.16 with the well-known prefix produces the address 64:ff9b::198.51.100.16.

2.5.2 Multicast Addresses

Multicast addressing is supported by IPv4 and IPv6. An IP multicast address (also called *group* or *group address*) identifies a group of host interfaces, rather than a single one. Generally speaking, the group could span the entire Internet. The portion of the network that a single group covers is known as the group's *scope* [RFC2365]. Common scopes include *node-local* (same computer), *link-local* (same subnet), *site-local* (applicable to some site), *global* (entire Internet), and *administrative*. Administrative scoped addresses may be used in an area of the network that has been manually configured into routers. A site administrator may configure routers as *admin-scope boundaries*, meaning that multicast traffic of the associated group is not forwarded past the router. Note that the site-local and administrative scopes are available for use only with multicast addressing.

Under software control, the protocol stack in each Internet host is able to join or leave a multicast group. When a host sends something to a group, it creates a datagram using one of its own (unicast) IP addresses as the source address and a multicast IP address as the destination. All hosts in scope that have joined the

group should receive any datagrams sent to the group. The sender is not generally aware of the hosts receiving the datagram unless they explicitly reply. Indeed, the sender does not even know in general how *many* hosts are receiving its datagrams.

The original multicast service model, described so far, has become known as *any-source multicast* (ASM). In this model, any sender may send to any group; a receiver joins the group by specifying only the group address. A newer approach, called *source-specific multicast* (SSM) [RFC3569][RFC4607], uses only a single sender per group (also see the errata to [RFC4607]). In this case, when joining a group, a host specifies the address of a *channel*, which comprises both a group address *and* a source IP address. SSM was developed to avoid some of the complexities in deploying the ASM model. Although neither form of multicast is widely available throughout the Internet, it seems that SSM is now the more likely candidate for adoption.

Understanding and implementing wide area multicasting has been an ongoing effort within the Internet community for more than a decade, and a large number of protocols have been developed to support it. Full details of how global Internet multicasting works are therefore beyond the scope of this text, but the interested reader is directed to [IMR02]. Details of how local IP multicast operates are given in Chapter 9. For now, we shall discuss the format and meaning of IPv4 and IPv6 multicast addresses.

2.5.3 IPv4 Multicast Addresses

For IPv4, the class D space (224.0.0.0–239.255.255.255) has been reserved for supporting multicast. With 28 bits free, this provides for the possibility of $2^{28} = 268,435,456$ host groups (each host group is an IP address). This address space is divided into major sections based on the way they are allocated and handled with respect to routing [IP4MA]. Those major sections are presented in Table 2-9.

The blocks of addresses up to 224.255.255.255 are allocated for the exclusive use of certain application protocols or organizations. These are allocated as the result of action by the IANA or by the IETF. The local network control block is limited to the local network of the sender; datagrams sent to those addresses are never forwarded by multicast routers. The All Hosts group (224.0.0.1) is one group in this block. The internetwork control block is similar to the local network control range but is intended for control traffic that needs to be routed off the local link. An example from this block is the *Network Time Protocol* (NTP) multicast group (224.0.1.1) [RFC5905].

The first ad hoc block was constructed to hold addresses that did not fall into either the local or internetwork control blocks. Most of the allocations in this range are for commercial services, some of which do not (or never will) require global address allocations; they may eventually be returned in favor of GLOP² addressing (see the next paragraphs). The SDP/SAP block contains addresses used by

2. GLOP is not an acronym but instead simply a name for a portion of address space.

Table 2-9 Major sections of IPv4 class D address space used for supporting multicast

Range (Inclusive)	Special Use	Reference
224.0.0.0–224.0.0.255	Local network control; not forwarded	[RFC5771]
224.0.1.0–224.0.1.255	Internetwork control; forwarded normally	[RFC5771]
224.0.2.0–224.0.255.255	Ad hoc block I	[RFC5771]
224.1.0.0–224.1.255.255	Reserved	[RFC5771]
224.2.0.0–224.2.255.255	SDP/SAP	[RFC4566]
224.3.0.0–224.4.255.255	Ad hoc block II	[RFC5771]
224.5.0.0–224.255.255.255	Reserved	[IP4MA]
225.0.0.0–231.255.255.255	Reserved	[IP4MA]
232.0.0.0–232.255.255.255	Source-specific multicast (SSM)	[RFC4607] [RFC4608]
233.0.0.0–233.251.255.255	GLOP	[RFC3180]
233.252.0.0–233.255.255.255	Ad hoc block III (233.252.0.0/24 is reserved for documentation)	[RFC5771]
234.0.0.0–234.255.255.255	Unicast-prefix-based IPv4 multicast addresses	[RFC6034]
235.0.0.0–238.255.255.255	Reserved	[IP4MA]
239.0.0.0–239.255.255.255	Administrative scope	[RFC2365]

applications such as the session directory tool (SDR) [H96] that send multicast session announcements using the *Session Announcement Protocol* (SAP) [RFC2974]. Originally a component of SAP, the newer *Session Description Protocol* (SDP) [RFC4566] is now used not only with IP multicast but also with other mechanisms to describe multimedia sessions.

The other major address blocks were created somewhat later in the evolution of IP multicast. The SSM block is used by applications employing SSM in combination with their own unicast source IP address in forming SSM channels, as described previously. In the GLOP block, multicast addresses are based on the *autonomous system* (AS) number of the host on which the application allocating the address resides. AS numbers are used by Internet-wide routing protocols among ISPs in order to aggregate routes and apply routing policies. Each such AS has a unique AS number. Originally, AS numbers were 16 bits but have now been extended to 32 bits [RFC4893]. GLOP addresses are generated by placing a 16-bit AS number in the second and third bytes of the IPv4 multicast address, leaving room for 1 byte to represent the possible multicast addresses (i.e., up to 256 addresses). Thus, it is possible to map back and forth between a 16-bit AS number and the GLOP multicast address range associated with an AS number. Although this computation is simple, several online calculators have been developed to do it, too.³

3. For example, <http://gigapop.uoregon.edu/glop/>.

The most recent of the IPv4 multicast address allocation mechanisms associates a number of multicast addresses with an IPv4 unicast address prefix. This is called *unicast-prefix-based* multicast addressing (UBM) and is described in [RFC6034]. It is based on a similar structure developed earlier for IPv6 that we discuss in Section 2.5.4. The UBM IPv4 address range is 234.0.0.0 through 234.255.255.255. A unicast address allocation with a /24 or shorter prefix may make use of UBM addresses. Allocations with fewer addresses (i.e., a /25 or longer prefix) must use some other mechanism. UBM addresses are constructed as a concatenation of the 234/8 prefix, the allocated unicast prefix, and the multicast group ID. Figure 2-11 shows the format.



Figure 2-11 The IPv4 UBM address format. For unicast address allocations of /24 or shorter, associated multicast addresses are allocated based on a concatenation of the prefix 234/8, the assigned unicast prefix, and the multicast group ID. Allocations with shorter unicast prefixes therefore contain more unicast and multicast addresses.

To determine the set of UBM addresses associated with a unicast allocation, the allocated prefix is simply prepended with the 234/8 prefix. For example, the unicast IPv4 address prefix 192.0.2.0/24 has a single associated UBM address, 234.192.0.2. It is also possible to determine the owner of a multicast address by simply “left-shifting” the multicast address by 8 bit positions. We know that the multicast address range 234.128.32.0/24 is allocated to UC Berkeley, for example, because the corresponding unicast IPv4 address space 128.32.0.0/16 (the “left-shifted” version of 234.128.32.0) is owned by UC Berkeley (as can be determined using a WHOIS query; see Section 2.6.1.1).

UBM addresses may offer advantages over the other types of multicast address allocations. For example, they do not carry the 16-bit restriction for AS numbers used by GLOP addressing. In addition, they are allocated as a consequence of already-existing unicast address space allocations. Thus, sites wishing to use multicast addresses already know which addresses they can use without further coordination. Finally, UBM addresses are allocated at a finer granularity than GLOP addresses, which correspond to AS number allocations. In today’s Internet, a single AS number may be associated with multiple sites, frustrating the simple mapping between address and owner supported by UBM.

The administratively scoped address block can be used to limit the distribution of multicast traffic to a particular collection of routers and hosts. These are the multicast analogs of private unicast IP addresses. Such addresses should not be used for distributing multicast into the Internet, as most of them are blocked at enterprise boundaries. Large sites sometimes subdivide administratively scoped

multicast addresses to cover specific useful scopes (e.g., work group, division, and geographical area).

2.5.4 IPv6 Multicast Addresses

For IPv6, which is considerably more aggressive in its use of multicast, the prefix `ff00::/8` has been reserved for multicast addresses, and 112 bits are available for holding the group number, providing for the possibility of

$$2^{112} = 5,192,296,858,534,827,628,530,496,329,220,096$$

groups. Its general format is as shown in Figure 2-12.

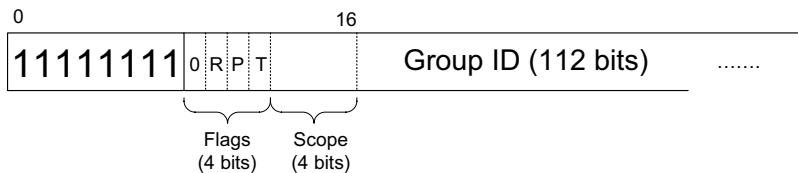


Figure 2-12 The base IPv6 multicast address format includes 4 *flag* bits (*0*, reserved; *R*, contains rendezvous point; *P*, uses unicast prefix; *T*, is transient). The 4-bit *Scope* value indicates the scope of the multicast (global, local, etc.). The *Group ID* is encoded in the low-order 112 bits. If the *P* or *R* bit is set, an alternative format is used.

The second byte of the IPv6 multicast address includes a 4-bit *Flags* field and a 4-bit *Scope ID* field in the second nibble. The *Scope* field is used to indicate a limit on the distribution of datagrams addressed to certain multicast addresses. The hexadecimal values 0, 3, and f are reserved. The hex values 6, 7, and 9 through d are unassigned. The values are given in Table 2-10, which is based on Section 2.7 of [RFC4291].

Table 2-10 Values of the IPv6 *Scope* field

Value	Scope
0	Reserved
1	Interface-/machine-local
2	Link-/subnet-local
3	Reserved
4	Admin
5	Site-local
6–7	Unassigned
8	Organizational-local
9–d	Unassigned
e	Global
f	Reserved

Many IPv6 multicast addresses allocated by the IANA for permanent use intentionally span multiple scopes. Each of these is defined with a certain offset relative to every scope (such addresses are called *scope-relative* or *variable-scope* for this reason). For example, the variable-scope multicast address `ff0x::101` is reserved for NTP servers by [IP6MA]. The `x` indicates variable scope; Table 2-11 shows some of the addresses defined by this reservation.

Table 2-11 Example permanent variable-scope IPv6 multicast address reservations for NTP (101)

Address	Meaning
<code>ff01::101</code>	All NTP servers on the same machine
<code>ff02::101</code>	All NTP servers on the same link/subnet
<code>ff04::101</code>	All NTP servers within some administratively defined scope
<code>ff05::101</code>	All NTP servers at the same site
<code>ff08::101</code>	All NTP servers at the same organization
<code>ff0e::101</code>	All NTP servers in the Internet

In IPv6, the multicast address format given in Figure 2-12 is used when the *P* and *R* bit fields are set to 0. When *P* is set to 1, two alternative methods exist for multicast addresses that do not require global agreement on a per-group basis. These are described in [RFC3306] and [RFC4489]. In the first, called *unicast-prefix-based* IPv6 multicast address assignment, a unicast prefix allocation provided by an ISP or address allocation authority also effectively allocates a collection of multicast addresses, thereby limiting the amount of global coordination required for avoiding duplicates. With the second method, *link-scoped* IPv6 multicast, interface identifiers are used, and multicast addresses are based on a host's IID. To understand how these various formats work, we need to first understand the use of the bit fields in the IPv6 multicast address in more detail. They are defined in Table 2-12.

Table 2-12 IPv6 multicast address flags

Bit Field (Flag)	Meaning	Reference
<i>R</i>	Rendezvous point flag (0, regular; 1, RP address included)	[RFC3956]
<i>P</i>	Prefix flag (0, regular; 1, address based on unicast prefix)	[RFC3306]
<i>T</i>	Transient flag (0, permanently assigned; 1, transient)	[RFC4291]

The *T* bit field, when set, indicates that the included group address is temporary or dynamically allocated; it is not one of the standard addresses defined in [IP6MA]. When the *P* bit field is set to 1, the *T* bit must also be set to 1. When this happens, a special format of IPv6 multicast addresses based on unicast address prefixes is enabled, as shown in Figure 2-13.

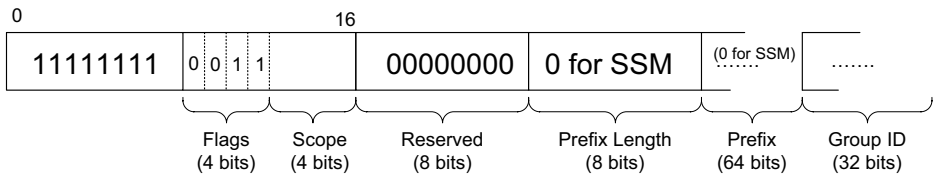


Figure 2-13 IPv6 multicast addresses can be created based upon unicast IPv6 address assignments [RFC3306]. When this is done, the *P* bit field is set to 1, and the unicast prefix is carried in the address, along with a 32-bit group ID. This form of multicast address allocation eases the need for global address allocation agreements.

We can see here how using unicast-prefix-based addressing changes the format of the multicast address to include space for a unicast prefix and its length, plus a smaller (32-bit) group ID. The purpose of this scheme is to provide a way of allocating globally unique IPv6 multicast addresses without requiring a new global mechanism for doing so. Because IPv6 unicast addresses are already allocated globally in units of prefixes (see Section 2.6), it is possible to use bits of this prefix in multicast addresses, thereby leveraging the existing method of unicast address allocation for multicast use. For example, an organization receiving a unicast prefix allocation of `3ffe:ffff:1::/48` would also consequently receive a unicast-based multicast prefix allocation of `ff3x:30:3ffe:ffff:1::/96`, where *x* is any valid scope. SSM is also supported using this format by setting the prefix length and prefix fields to 0, effectively requiring the prefix `ff3x::/32` (where *x* is any valid scope value) for use in all such IPv6 SSM multicast addresses.

To create unique multicast addresses of link-local scope, a method based on IIDs can be used [RFC4489], which is preferred to unicast-prefix-based allocation when only link-local scope is required. In this case, another form of IPv6 multicast address structure is used (see Figure 2-14).

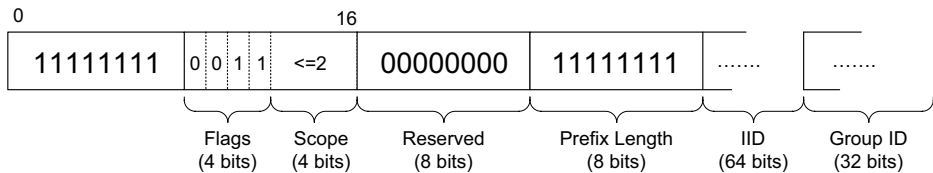


Figure 2-14 The IPv6 link-scoped multicast address format. Applicable only to link- (or smaller) scoped addresses, the multicast address can be formed by combining an IPv6 interface ID and a group ID. The mapping is straightforward, and all such addresses use prefixes of the form `ff3x:0011/32`, where *x* is the scope ID and is less than 3.

The address format shown in Figure 2-14 is very similar to the format in Figure 2-13, except that the *Prefix Length* field is set to 255, and instead of a prefix being carried in the subsequent field, an IPv6 IID is instead. The advantage of

this structure over the previous one is that no prefix need be supplied in forming the multicast address. In ad hoc networks where no routers may be available, an individual machine can form unique multicast addresses based on its own IID without having to engage in a complex agreement protocol. As stated before, this format works only for link- or node-local multicast scoping, however. When larger scopes are required, either unicast-prefix-based addressing or permanent multicast addresses are used. As an example of this format, a host with IID 02-11-22-33-44-55-66-77 would use multicast addresses of the form ff3x:0011:0211:2233:4455:6677:gggg:gggg, where x is a scope value of 2 or less and gggg:gggg is the hexadecimal notation for a 32-bit multicast group ID.

The bit field we have yet to discuss is the *R* bit field. It is used when unicast-prefix-based multicast addressing is used (the *P* bit is set) along with a multicast routing protocol that requires knowledge of a rendezvous point.

Note

A *rendezvous point* (RP) is the IP address of a router set up to handle multicast routing for one or more multicast groups. RPs are used by the PIM-SM protocol [RFC4601] to help senders and receivers participating in the same multicast group to find each other. One of the problems encountered in deploying Internet-wide multicast has been locating rendezvous points. This scheme overloads the IPv6 multicast address to include an RP address. Therefore, it is simple to find an RP from a group address by just selecting the appropriate subset of bits.

When the *P* bit is set, the modified format for a multicast address shown in Figure 2-15 is used.

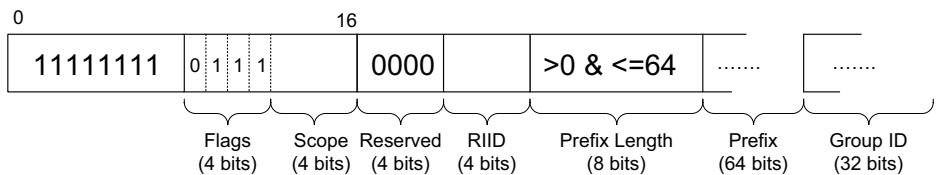


Figure 2-15 The unicast IPv6 address of an RP can be embedded inside an IPv6 multicast address [RFC3956]. Doing so makes it straightforward to find an RP associated with an address for routing purposes. An RP is used by the multicast routing system in order to coordinate multicast senders with receivers when they are not on the same subnetwork.

The format shown in Figure 2-15 is similar to the one shown in Figure 2-13, but SSM is not used (so the prefix length cannot be zero). In addition, a new 4-bit field called the *RIID* is introduced. To form the IPv6 address of an RP based on a multicast address of the form in Figure 2-15, the number of bits indicated in the *Prefix Length* field are extracted from the *Prefix* field and placed as the upper bits in a fresh IPv6 address. Then, the contents of the *RIID* field are used as the

low-order 4 bits of the RP address. The rest is filled with zeros. As an example, consider a multicast address `ff75:940:2001:db8:dead:beef:f00d:face`. In this case, the scope is 5 (site-local), the *RIID* field has the value 9, and the prefix length is $0x40 = 64$ bits. The prefix itself is therefore `2001:db8:dead:beef`, so the RP address is `2001:db8:dead:beef::9`. More examples are given in [RFC3956].

As with IPv4, there are a number of reserved IPv6 multicast addresses. These addresses are grouped by scope, except for the variable-scope addresses mentioned before. Table 2-13 gives a list of the major reservations from the IPv6 multicast space. Consult [IP6MA] for additional information.

Table 2-13 Reserved addresses within the IPv6 multicast address space

Address	Scope	Special Use	Reference
<code>ff01::1</code>	Node	All nodes	[RFC4291]
<code>ff01::2</code>	Node	All routers	[RFC4291]
<code>ff01::fb</code>	Node	mDNSv6	[IDChes]
<code>ff02::1</code>	Link	All nodes	[RFC4291]
<code>ff02::2</code>	Link	All routers	[RFC4291]
<code>ff02::4</code>	Link	DVMRP routers	[RFC1075]
<code>ff02::5</code>	Link	OSPFv2	[RFC2328]
<code>ff02::6</code>	Link	OSPFv2 designated routers	[RFC2328]
<code>ff02::9</code>	Link	RIPv2 routers	[RFC2080]
<code>ff02::a</code>	Link	EIGRP routers	[EIGRP]
<code>ff02::d</code>	Link	PIM routers	[RFC5059]
<code>ff02::16</code>	Link	MLDv2-capable routers	[RFC3810]
<code>ff02::6a</code>	Link	All snoopers	[RFC4286]
<code>ff02::6d</code>	Link	LL-MANET-routers	[RFC5498]
<code>ff02::fb</code>	Link	mDNSv6	[IDChes]
<code>ff02::1:2</code>	Link	All DHCP agents	[RFC3315]
<code>ff02::1:3</code>	Link	LLMNR	[RFC4795]
<code>ff02::1:ffxx:xxxx</code>	Link	Solicited-node address range	[RFC4291]
<code>ff05::2</code>	Site	All routers	[RFC4291]
<code>ff05::fb</code>	Site	mDNSv6	[IDChes]
<code>ff05::1:3</code>	Site	All DHCP servers	[RFC3315]
<code>ff0x::</code>	Variable	Reserved	[RFC4291]
<code>ff0x::fb</code>	Variable	mDNSv6	[IDChes]
<code>ff0x::101</code>	Variable	NTP	[RFC5905]
<code>ff0x::133</code>	Variable	Aggregate Server Access Protocol	[RFC5352]
<code>ff0x::18c</code>	Variable	All ACs address (CAPWAP)	[RFC5415]
<code>ff3x::/32</code>	(Special)	SSM block	[RFC4607]

2.5.5 Anycast Addresses

An *anycast* address is a unicast IPv4 or IPv6 address that identifies a different host depending on where in the network it is used. This is accomplished by configuring Internet routers to advertise the same unicast routes from multiple locations in the Internet. Thus, an anycast address refers not to a single host in the Internet, but to the “most appropriate” or “closest” single host that is responding to the anycast address. Anycast addressing is used most frequently for finding a computer that provides a common service [RFC4786]. For example, a datagram sent to an anycast address could be used to find a DNS server (see Chapter 11), a 6to4 gateway that encapsulates IPv6 traffic in IPv4 tunnels [RFC3068], or RPs for multicast routing [RFC4610].

2.6 Allocation

IP address space is *allocated*, usually in large chunks, by a collection of hierarchically organized *authorities*. The authorities are generally organizations that allocate address space to various owners—usually ISPs or other smaller authorities. Authorities are most often involved in allocating portions of the global unicast address space, but other types of addresses (multicast and special-use) are also sometimes allocated. The authorities can make allocations to users for an undetermined amount of time, or for a limited time (e.g., for running experiments). The top of the hierarchy is the IANA [IANA], which has wide-ranging responsibility for allocating IP addresses and other types of numbers used in the Internet protocols.

2.6.1 Unicast

For unicast IPv4 and IPv6 address space, the IANA delegates much of its allocation authority to a few *regional Internet registries* (RIRs). The RIRs coordinate with each other through an organization formed in 2003 called the Number Resource Organization (NRO) [NRO]. At the time of writing (mid-2011), the set of RIRs includes those shown in Table 2-14, all of which participate in the NRO. Note in addition that, as of early 2011, all the remaining unicast IPv4 address space held by IANA for allocation had been handed over to these RIRs.

These entities typically deal with relatively large address blocks [IP4AS] [IP6AS]. They allocate address space to smaller registries operating in countries (e.g., Australia and Singapore) and to large ISPs. ISPs, in turn, provide address space to their customers and themselves. When users sign up for Internet service, they are ordinarily provided a (typically small) fraction or range of their ISP’s address space in the form of an address prefix. These address ranges are owned and managed by the customer’s ISP and are called *provider-aggregatable* (PA) addresses because they consist of one or more prefixes that can be aggregated with other prefixes the ISP owns. Such addresses are also sometimes called *non-portable* addresses. Switching providers typically requires customers to change the

Table 2-14 Regional Internet registries that participate in the NRO

RIR Name	Area of Responsibility	Reference
AfriNIC—African Network Information Center	Africa	http://www.afrinic.net
APNIC—Asia Pacific Network Information Center	Asia/Pacific Area	http://www.apnic.net
ARIN—American Registry for Internet Numbers	North America	http://www.arin.net
LACNIC—Regional Latin America and Caribbean IP Address Registry	Latin America and some Caribbean islands	http://lacnic.net/en/index.html
RIPE NCC—Réseaux IP Européens	Europe, Middle East, Central Asia	http://www.ripe.net

IP prefixes on all computers and routers they have that are attached to the Internet (an often unpleasant operation called *renumbering*).

An alternative type of address space is called *provider-independent* (PI) address space. Addresses allocated from PI space are allocated to the user directly and may be used with any ISP. However, because such addresses are owned by the customer, they are not numerically adjacent to the ISP's own addresses and are therefore not aggregatable. An ISP being asked to provide routing for a customer's PI addresses may require additional payment for service or simply not agree to support such a configuration. In some sense, an ISP that agrees to provide routing for a customer's PI addresses is taking on an extra cost relative to other customers by having to increase the size of its routing tables. On the other hand, many sites prefer to use PI addresses, and might be willing to pay extra for them, because it helps to avoid the need to renumber when switching ISPs (avoiding what has become known as *provider lock*).

2.6.1.1 Examples

It is possible to use the Internet WHOIS service to determine how address space has been allocated. For example, we can form a query for information about the IPv4 address 72.1.140.203 by accessing the corresponding URL <http://whois.arin.net/rest/ip/72.1.140.203.txt>:

```

NetRange:      72.1.140.192 - 72.1.140.223
CIDR:          72.1.140.192/27
OriginAS:
NetName:       SPEK-SEA5-PART-1
NetHandle:     NET-72-1-140-192-1
Parent:        NET-72-1-128-0-1
NetType:       Reassigned
RegDate:       2005-06-29
Updated:       2005-06-29
Ref:           http://whois.arin.net/rest/net/NET-72-1-140-192-1

```

Here we see that the address 72.1.140.203 is really part of the network called SPEK-SEA5-PART-1, which has been allocated the address range 72.1.140.192/27. Furthermore, we can see that SPEK-SEA5-PART-1's address range is a portion of the PA address space called NET-72-1-128-0-1. We can formulate a query for information about this network by visiting the URL <http://whois.arin.net/rest/net/NET-72-1-128-0-1.txt>:

```
NetRange:      72.1.128.0 - 72.1.191.255
CIDR:          72.1.128.0/18
OriginAS:
NetName:       SPEAKEASY-6
NetHandle:     NET-72-1-128-0-1
Parent:        NET-72-0-0-0-0
NetType:       Direct Allocation
RegDate:       2004-09-09
Updated:       2009-05-19
Ref:           http://whois.arin.net/rest/net/NET-72-1-128-0-1
```

This record indicates that the address range 72.1.128.0/18 (called by the “handle” or name NET-72-1-128-0-1) has been directly allocated out of the address range 72.0.0.0/8 managed by ARIN. More details on data formats and the various methods ARIN supports for WHOIS queries can be found at [WRWS].

We can look at a different type of result using one of the other RIRs. For example, if we search for information regarding the IPv4 address 193.5.93.80 using the Web query interface at <http://www.ripe.net/whois>, we obtain the following result:

```
% This is the RIPE Database query service.
% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See http://www.ripe.net/db/support/db-terms-conditions.pdf
%
% Note: This output has been filtered.
%       To receive output for a database update, use the "-B" flag.
% Information related to '193.5.88.0 - 193.5.95.255'
inetnum:      193.5.88.0 - 193.5.95.255
netname:      WIPONET
descr:        World Intellectual Property Organization
descr:        UN Specialized Agency
descr:        Geneva
country:      CH
admin-c:      AM4504-RIPE
tech-c:       AM4504-RIPE
status:       ASSIGNED PI
mnt-by:       CH-UNISOURCE-MNT
mnt-by:       DE-COLT-MNT
source:       RIPE # Filtered
```

Here, we can see that the address 193.5.93.80 is a portion of the 193.5.88.0/21 block allocated to WIPO. Note that the status of this block is **ASSIGNED PI**, meaning that this particular block of addresses is of the provider-independent variety. The reference to RPSL indicates that the database records are in the *Routing Policy Specification Language* [RFC2622][RFC4012], used by ISPs to express their routing policies. Such information allows network operators to configure routers to help minimize Internet routing instabilities.

2.6.2 Multicast

In IPv4 and IPv6, multicast addresses (i.e., group addresses) can be described based on their scope, the way they are determined (statically, dynamically by agreement, or algorithmically), and whether they are used for ASM or SSM. Guidelines have been constructed for allocation of these groups ([RFC5771] for IPv4; [RFC3307] for IPv6) and the overall architecture is detailed in [RFC6308]. The groups that are not of global scope (e.g., administratively scoped addresses and IPv6 link-scoped multicast addresses) can be reused in various parts of the Internet and are either configured by a network administrator out of an administratively scoped address block or selected automatically by end hosts. Globally scoped addresses that are statically allocated are generally fixed and may be hard-coded into applications. This type of address space is limited, especially in IPv4, so such addresses are really intended for uses applicable to any Internet site. Algorithmically determined globally scoped addresses can be created based on AS numbers, as in GLOP, or an associated unicast prefix allocation. Note that SSM can use globally scoped addresses (i.e., from the SSM block), administratively scoped addresses, or unicast-prefix-based IPv6 addresses where the prefix is effectively zero.

As we can see from the relatively large number of protocols and the complexity of the various multicast address formats, multicast address management is a formidable issue (not to mention global multicast routing [RFC5110]). From a typical user's point of view, multicasting is used rarely and may be of limited concern. From a programmer's point of view, it may be worthwhile to support multicast in application designs, and some insight has been provided into how to do so [RFC3170]. For network administrators faced with implementing multicast, some interaction with the service provider is likely necessary. In addition, some guidelines for multicast address allocation have been developed by vendors [CGEMA].

2.7 Unicast Address Assignment

Once a site has been allocated a range of unicast IP addresses, typically from its ISP, the site or network administrator must determine how to assign addresses in the address range to each network interface and how to set up the subnet structure. If the site has only a single physical network segment (e.g., most private homes), this process is relatively straightforward. For larger enterprises, especially those

3

Link Layer

3.1 Introduction

In Chapter 1, we saw that the purpose of the link layer in the TCP/IP protocol suite is to send and receive IP datagrams for the IP module. It is also used to carry a few other protocols that help support IP, such as ARP (see Chapter 4). TCP/IP supports many different link layers, depending on the type of networking hardware being used: wired LANs such as Ethernet, *metropolitan area networks* (MANs) such as cable TV and DSL connections available through service providers, and wired voice networks such as telephone lines with modems, as well as the more recent wireless networks such as Wi-Fi (wireless LAN) and various wireless data services based on cellular technology such as HSPA, EV-DO, LTE, and WiMAX. In this chapter we shall look at some of the details involved in using the Ethernet and Wi-Fi link layers, how the *Point-to-Point Protocol* (PPP) is used, and how link-layer protocols can be carried inside other (link- or higher-layer) protocols, a technique known as tunneling. Covering the details of every link technology available today would require a separate text, so we instead focus on some of the most commonly used link-layer protocols and how they are used by TCP/IP.

Most link-layer technologies have an associated protocol format that describes how the corresponding PDUs must be constructed in order to be carried by the network hardware. When referring to link-layer PDUs, we usually use the term *frame*, so as to distinguish the PDU format from those at higher layers such as packets or segments, terms used to describe network- and transport-layer PDUs, respectively. Frame formats usually support a variable-length frame size ranging from a few bytes to a few kilobytes. The upper bound of the range is called the *maximum transmission unit* (MTU), a characteristic of the link layer that we shall encounter numerous times in the remaining chapters. Some network technologies, such as modems and serial lines, do not impose their own maximum frame size, so they can be configured by the user.

3.2 Ethernet and the IEEE 802 LAN/MAN Standards

The term *Ethernet* generally refers to a set of standards first published in 1980 and revised in 1982 by Digital Equipment Corp., Intel Corp., and Xerox Corp. The first common form of Ethernet is now sometimes called “10Mb/s Ethernet” or “shared Ethernet,” and it was adopted (with minor changes) by the IEEE as standard number 802.3. Such networks were usually arranged like the network shown in Figure 3-1.

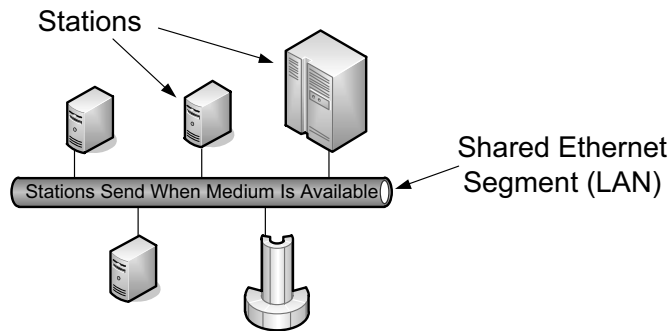


Figure 3-1 A basic shared Ethernet network consists of one or more stations (e.g., workstations, supercomputers) attached to a shared cable segment. Link-layer PDUs (frames) can be sent from one station to one or more others when the medium is determined to be free. If multiple stations send at the same time, possibly because of signal propagation delays, a collision occurs. Collisions can be detected, and they cause sending stations to wait a random amount of time before retrying. This common scheme is called carrier sense, multiple access with collision detection.

Because multiple stations share the same network, this standard includes a distributed algorithm implemented in each Ethernet network interface that controls when a station gets to send data it has. The particular method, known as *carrier sense, multiple access with collision detection* (CSMA/CD), mediates which computers can access the shared medium (cable) without any other special agreement or synchronization. This relative simplicity helped to promote the low cost and resulting popularity of Ethernet technology.

With CSMA/CD, a station (e.g., computer) first looks for a signal currently being sent on the network and sends its own frame when the network is free. This is the “carrier sense” portion of the protocol. If some other station happens to send at the same time, the resulting overlapping electrical signal is detected as a collision. In this case, each station waits a random amount of time before trying again. The amount of time is selected by drawing from a uniform probability distribution that doubles in length each time a subsequent collision is detected.

Eventually, each station gets its chance to send or times out trying after some number of attempts (16 in the case of conventional Ethernet). With CSMA/CD, only one frame is traveling on the network at any given time. Access methods such as CSMA/CD are more formally called *Media Access Control* (MAC) protocols. There are many types of MAC protocols; some are based on having each station try to use the network independently (contention-based protocols like CSMA/CD), and others are based on prearranged coordination (e.g., by allocating time slots for each station to send).

Since the development of 10Mb/s Ethernet, faster computers and infrastructure have driven the need for ever-increasing speeds in LANs. Given the popularity of Ethernet, significant innovation and effort have managed to increase its speed from 10Mb/s to 100Mb/s to 1000Mb/s to 10Gb/s, and now to even more. The 10Gb/s form is becoming popular in larger data centers and large enterprises, and speeds as high as 100Gb/s have been demonstrated. The very first (research) Ethernet ran at 3Mb/s, but the DIX (Digital, Intel, Xerox) standard ran at 10Mb/s over a shared physical cable or set of cable segments interconnected by electrical repeaters. By the early 1990s, the shared cable had largely been replaced by twisted-pair wiring (resembling telephone wires and often called “10BASE-T”). With the development of 100Mb/s (also called “fast Ethernet,” the most popular version of which is known as “100BASE-TX”), contention-based MAC protocols have become less popular. Instead, the wiring between each LAN station is often not shared but instead provides a dedicated electrical path in a star topology. This can be accomplished with Ethernet *switches*, as shown in Figure 3-2.

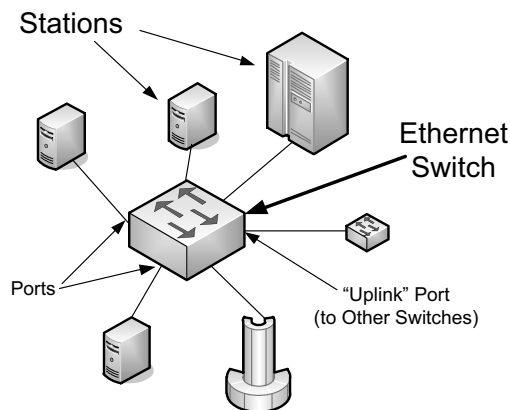


Figure 3-2 A switched Ethernet network consists of one or more stations, each of which is attached to a switch port using a dedicated wiring path. In most cases where switched Ethernet is used, the network operates in a full-duplex fashion and the CSMA/CD algorithm is not required. Switches may be cascaded to form larger Ethernet LANs by interconnecting switch ports, sometimes called “uplink” ports.

At present, switches are commonly used, providing each Ethernet station with the ability to send and receive data simultaneously (called “full-duplex Ethernet”). Although half-duplex (one direction at a time) operation is still supported even by 1000Mb/s Ethernet (1000BASE-T), it is rarely used relative to full-duplex Ethernet. We shall discuss how switches process PDUs in more detail later.

One of the most popular technologies used to access the Internet today is wireless networking, the most common for wireless local area networks (WLANs) being an IEEE standard known as Wireless Fidelity or *Wi-Fi*, and sometimes called “wireless Ethernet” or 802.11. Although this standard is distinct from the 802 wired Ethernet standards, the frame format and general interface are largely borrowed from 802.3, and all are part of the set of IEEE 802 LAN standards. Thus, most of the capabilities used by TCP/IP for Ethernet networks are also used for Wi-Fi networks. We shall explore each of these in more detail. First, however, it is useful to get a bigger picture of all of the IEEE 802 standards that are relevant for setting up home and enterprise networks. We also include references to those IEEE standards governing MAN standards, including IEEE 802.16 (WiMAX) and the standard for media-independent handoffs in cellular networks (IEEE 802.21).

3.2.1 The IEEE 802 LAN/MAN Standards

The original Ethernet frame format and operation were described by industry agreement, mentioned earlier. This format was known as the DIX format or Ethernet II format. This type of Ethernet network, with slight modification, was later standardized by the IEEE as a form of CSMA/CD network, called 802.3. In the world of IEEE standards, standards with the prefix 802 define the operations of LANs and MANs. The most popular 802 standards today include 802.3 (essentially Ethernet) and 802.11 (WLAN/Wi-Fi). These standards have evolved over time and have changed names as freestanding amendments (e.g., 802.11g) are ultimately incorporated in revised standards. Table 3-1 shows a fairly complete list of the IEEE 802 LAN and MAN standards relevant to supporting the TCP/IP protocols, as of mid-2011.

Table 3-1 LAN and MAN IEEE 802 standards relevant to the TCP/IP protocols (2011)

Name	Description	Official Reference
802.1ak	Multiple Registration Protocol (MRP)	[802.1AK-2007]
802.1AE	MAC Security (MACSec)	[802.1AE-2006]
802.1AX	Link Aggregation (formerly 802.3ad)	[802.1AX-2008]
802.1d	MAC Bridges	[802.1D-2004]
802.1p	Traffic classes/priority/QoS	[802.1D-2004]
802.1q	Virtual Bridged LANs/Corrections to MRP	[802.1Q-2005/Cor1-2008]
802.1s	Multiple Spanning Tree Protocol (MSTP)	[802.1Q-2005]

Table 3-1 LAN and MAN IEEE 802 standards relevant to the TCP/IP protocols (2011) (*continued*)

Name	Description	Official Reference
802.1w	Rapid Spanning Tree Protocol (RSTP)	[802.1D-2004]
802.1X	Port-Based Network Access Control (PNAC)	[802.1X-2010]
802.2	Logical Link Control (LLC)	[802.2-1998]
802.3	Baseline Ethernet and 10Mb/s Ethernet	[802.3-2008] (Section One)
802.3u	100Mb/s Ethernet ("Fast Ethernet")	[802.3-2008] (Section Two)
802.3x	Full-duplex operation and flow control	[802.3-2008]
802.3z/802.3ab	1000Mb/s Ethernet ("Gigabit Ethernet")	[802.3-2008] (Section Three)
802.3ae	10Gb/s Ethernet ("Ten-Gigabit Ethernet")	[802.3-2008] (Section Four)
802.3ad	Link Aggregation	[802.1AX-2008]
802.3af	Power over Ethernet (PoE) (to 15.4W)	[802.3-2008] (Section Two)
802.3ah	Access Ethernet ("Ethernet in the First Mile (EFM)")	[802.3-2008] (Section Five)
802.3as	Frame format extensions (to 2000 bytes)	[802.3-2008]
802.3at	Power over Ethernet enhancements ("PoE+", to 30W)	[802.3at-2009]
802.3ba	40/100Gb/s Ethernet	[802.3ba-2010]
802.11a	54Mb/s Wireless LAN at 5GHz	[802.11-2007]
802.11b	11Mb/s Wireless LAN at 2.4GHz	[802.11-2007]
802.11e	QoS enhancement for 802.11	[802.11-2007]
802.11g	54Mb/s Wireless LAN at 2.4GHz	[802.11-2007]
802.11h	Spectrum/power management extensions	[802.11-2007]
802.11i	Security enhancements/replaces WEP	[802.11-2007]
802.11j	4.9–5.0GHz operation in Japan	[802.11-2007]
802.11n	6.5–600Mb/s Wireless LAN at 2.4 and 5GHz using optional MIMO and 40MHz channels	[802.11n-2009]
802.11s (draft)	Mesh networking, congestion control	Under development
802.11y	54Mb/s wireless LAN at 3.7GHz (licensed)	[802.11y-2008]
802.16	Broadband Wireless Access Systems (WiMAX)	[802.16-2009]
802.16d	Fixed Wireless MAN Standard (WiMAX)	[802.16-2009]
802.16e	Fixed/Mobile Wireless MAN Standard (WiMAX)	[802.16-2009]
802.16h	Improved Coexistence Mechanisms	[802.16h-2010]
802.16j	Multihop Relays in 802.16	[802.16j-2009]
802.16k	Bridging of 802.16	[802.16k-2007]
802.21	Media Independent Handovers	[802.21-2008]

Other than the specific types of LAN networks defined by the 802.3, 802.11, and 802.16 standards, there are some related standards that apply across all of the IEEE standard LAN technologies. Common to all three of these is the 802.2 standard that defines the *Logical Link Control* (LLC) frame header common among many of the 802 networks' frame formats. In IEEE terminology, LLC and MAC are "sublayers" of the link layer, where the LLC (mostly frame format) is generally common to each type of network and the MAC layer may be somewhat different. While the original Ethernet made use of CSMA/CD, for example, WLANs often make use of CSMA/CA (CA is "collision avoidance").

Note

Unfortunately the combination of 802.2 and 802.3 defined a different frame format from Ethernet II until 802.3x finally rectified the situation. It has been incorporated into [802.3-2008]. In the TCP/IP world, the encapsulation of IP datagrams is defined in [RFC0894] and [RFC2464] for Ethernet networks, although the older LLC/SNAP encapsulation remains published as [RFC1042]. While this is no longer much of an issue, it was once a source of concern, and similar issues occasionally arise [RFC4840].

The frame format has remained essentially the same until fairly recently. To get an understanding of the details of the format and how it has evolved, we now turn our focus to these details.

3.2.2 The Ethernet Frame Format

All Ethernet (802.3) frames are based on a common format. Since its original specification, the frame format has evolved to support additional functions. Figure 3-3 shows the current layout of an Ethernet frame and how it relates to a relatively new term introduced by IEEE, the *IEEE packet* (a somewhat unfortunate term given its uses in other standards).

The Ethernet frame begins with a *Preamble* area used by the receiving interface's circuitry to determine when a frame is arriving and to determine the amount of time between encoded bits (called *clock recovery*). Because Ethernet is an asynchronous LAN (i.e., precisely synchronized clocks are not maintained in each Ethernet interface card), the space between encoded bits may differ somewhat from one interface card to the next. The preamble is a recognizable pattern (0xAA typically), which the receiver can use to "recover the clock" by the time the *start frame delimiter* (SFD) is found. The SFD has the fixed value 0xAB.

Note

The original Ethernet encoded bits using a *Manchester Phase Encoding* (MPE) with two voltage levels. With MPE, bits are encoded as voltage transitions rather than absolute values. For example, the bit 0 is encoded as a transition from -0.85 to +0.85V, and a 1 bit is encoded as a +0.85 to -0.85V transition (0V indicates

that the shared wire is idle). The 10Mb/s Ethernet specification required network hardware to use an oscillator running at 20MHz, because MPE requires two clock cycles per bit. The bytes 0xAA (10101010 in binary) present in the Ethernet preamble would be a square wave between +0.85 and -0.85V with a frequency of 10MHz. Manchester encoding was replaced with different encodings in other Ethernet standards to improve efficiency.

This basic frame format includes 48-bit (6-byte) *Destination (DST)* and *Source (SRC)* Address fields. These addresses are sometimes known by other names such as “MAC address,” “link-layer address,” “802 address,” “hardware address,” or “physical address.” The destination address in an Ethernet frame is also allowed to address more than one station (called “broadcast” or “multicast”; see Chapter 9). The broadcast capability is used by the ARP protocol (see Chapter 4) and multicast capability is used by the ICMPv6 protocol (see Chapter 8) to convert between network-layer and link-layer addresses.

Following the source address is a *Type* field that doubles as a *Length* field. Ordinarily, it identifies the type of data that follows the header. Popular values used with TCP/IP networks include IPv4 (0x0800), IPv6 (0x86DD), and ARP (0x0806). The value 0x8100 indicates a Q-tagged frame (i.e., one that can carry a “virtual LAN” or VLAN ID according to the 802.1q standard). The size of a basic Ethernet frame is 1518 bytes, but the more recent standard extended this size to 2000 bytes.

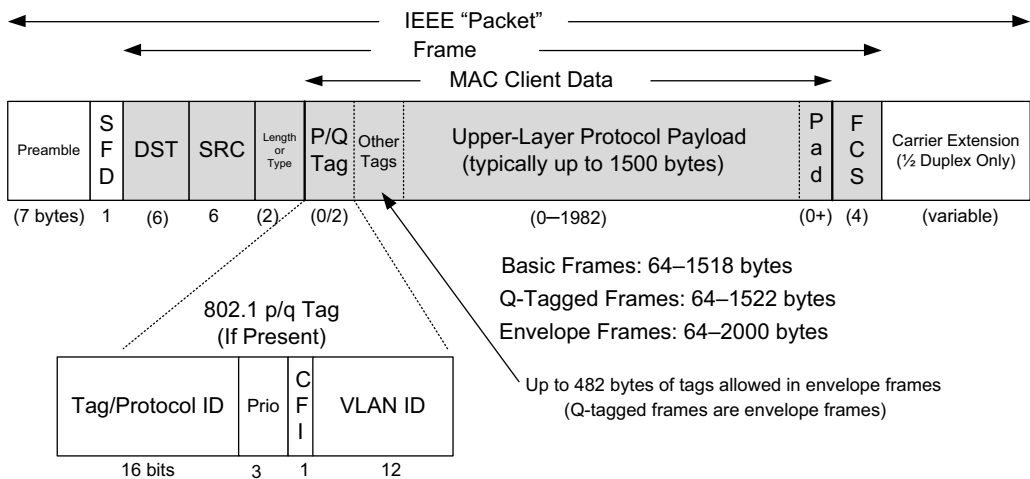


Figure 3-3 The Ethernet (IEEE 802.3) frame format contains source and destination addresses, an overloaded *Length/Type* field, a field for data, and a frame check sequence (a CRC32). Additions to the basic frame format provide for a tag containing a VLAN ID and priority information (802.1p/q) and more recently for an extensible number of tags. The preamble and SFD are used for synchronizing receivers. When half-duplex operation is used with Ethernet running at 100Mb/s or more, additional bits may be appended to short frames as a carrier extension to ensure that the collision detection circuitry operates properly.

Note

The original IEEE (802.3) specification treats the *Length/Type* field as a *Length* field instead of a *Type* field. The field is thereby *overloaded* (used for more than one purpose). The trick is to look at the value of the field. Today, if the value in the field is greater than or equal to 1536, the field must contain a type value, which is assigned by standards to have values exceeding 1536. If the value of the field is 1500 or less, the field indicates the length. The full list of types is given by [ETHERTYPES].

Following the *Destination* and *Source Address* fields, [802.3-2008] provides for a variable number of *tags* that contain various protocol fields defined by other IEEE standards. The most common of these are the tags used by 802.1p and 802.1q, which provide for virtual LANs and some *quality-of-service* (QoS) indicators. These are discussed in Section 3.2.3.

Note

The current [802.3-2008] standard incorporates the frame format modifications of 802.3 as that provides for a maximum of 482 bytes for holding “tags” to be carried with each Ethernet frame. These larger frames, called *envelope frames*, may be up to 2000 bytes in length. Frames containing 802.1p/q tags, called *Q-tagged frames*, are also envelope frames. However, not all envelope frames are necessarily Q-tagged frames.

Following the fields discussed so far is the data area or *payload* portion of the frame. This is the area where higher-layer PDUs such as IP datagrams are placed. Traditionally, the payload area for Ethernet has always been 1500 bytes, representing the MTU for Ethernet. Most systems today use the 1500-byte MTU size for Ethernet, although it is generally possible to configure a smaller value if this is desired. The payload sometimes is *padded* (appended) with 0 bytes to ensure that the overall frame meets the minimum length requirements we discuss in Section 3.2.2.2.

3.2.2.1 Frame Check Sequence/Cyclic Redundancy Check (CRC)

The final field of the Ethernet frame format follows the payload area and provides an integrity check on the frame. The *Cyclic Redundancy Check* (CRC) field at the end includes 32 bits and is sometimes known as the IEEE/ANSI standard CRC32 [802.3-2008]. To use an n -bit CRC for detection of data transmission in error, the message to be checked is first appended with n 0 bits, forming the *augmented message*. Then, the augmented message is divided (using modulo-2 division) by an $(n + 1)$ -bit value called the *generator polynomial*, which acts as the divisor. The value placed in the CRC field of the message is the one’s complement of the remainder of this division (the quotient is discarded). Generator polynomials are standardized

for a number of different values of n . For Ethernet, which uses $n = 32$, the CRC32 generator polynomial is the 33-bit binary number 10000010011000001000111011011011. To get a feeling for how the remainder is computed using long (mod-2) binary division, we can examine a simpler case using CRC4. The ITU has standardized the value 10011 for the CRC4 generator polynomial in a standard called G.704 [G704]. If we wish to send the 16-bit message 1001111000101111, we first begin with the long (mod-2) binary division shown in Figure 3-4.

	1000011000000101	Quotient (Discarded)
10011	10011110001011110000	Message
	10011	
	00001	
	00000	
	00011	
	00000	
	00110	
	00000	
	01100	
	00000	
	11000	
	10011	
	10111	
	10011	
	01000	
	00000	
	10001	
	10011	
	00101	
	00000	
	01011	
	00000	
	10111	
	10011	
	01000	
	00000	
	10000	
	10011	
	01110	
	00000	
	11100	
	10011	
	1111	Remainder

Figure 3-4 Long (mod-2) binary division demonstrating the computation of a CRC4

In this figure, we see that the remainder after division is the 4-bit value 1111. Ordinarily, the one's complement of this value (0000) would be placed in a CRC or *Frame Check Sequence (FCS)* field in the frame. Upon receipt, the receiver performs the same division and checks whether the value in the *FCS* field matches the computed remainder. If the two do not match, the frame was likely damaged in transit and is usually discarded. The CRC family of functions can be used to provide a strong indicator of corrupted messages because any change in the bit pattern is highly likely to cause a change in the remainder term.

3.2.2.2 Frame Sizes

There is both a minimum and a maximum size of Ethernet frames. The minimum is 64 bytes, requiring a minimum data area (payload) length of 48 bytes (no tags). In cases where the payload is smaller, pad bytes (value 0) are appended to the end of the payload portion to ensure that the minimum length is enforced.

Note

The minimum was important for the original 10Mb/s Ethernet using CSMA/CD. In order for a transmitting station to know which frame encountered a collision, a limit of 2500m (five 500m cable segments with four repeaters) was placed upon the length of an Ethernet network. Given that the propagation rate for electrons in copper is about .77c or 231M m/s, and given the transmission time of 64 bytes to be $(64 * 8 / 10,000,000) = 51.2\mu\text{s}$ at 10Mb/s, a minimum-size frame could consume about 11,000m of cable. With a maximum of 2500m of cable, the maximum round-trip distance from one station to another is 5000m. The designers of Ethernet included a factor of 2 overdesign in fixing the minimum frame size, so in all compliant cases (and many noncompliant cases), the last bit of an outgoing frame would still be in the process of being transmitted after the time required for its signal to arrive at a maximally distant receiver and return. If a collision is detected, the transmitting station thus knows with certainty which frame collided—the one it is currently transmitting. In this case, the station sends a *jamming signal* (high voltage) to alert other stations, which then initiate a random binary exponential backoff procedure.

The maximum frame size of conventional Ethernet is 1518 bytes (including the 4-byte CRC and 14-byte header). This value represents a sort of trade-off: if a frame contains an error (detected on receipt by an incorrect CRC), only 1.5KB need to be retransmitted to repair the problem. On the other hand, the size limits the MTU to not more than 1500 bytes. In order to send a larger message, multiple frames are required (e.g., 64KB, a common larger size used with TCP/IP networks, would require at least 44 frames).

The unfortunate consequence of requiring multiple Ethernet frames to hold a larger upper-layer PDU is that each frame contributes a fixed overhead (14 bytes header, 4 bytes CRC). To make matters worse, Ethernet frames cannot be squished together on the network without any space between them, in order to allow the

Ethernet hardware receiver circuits to properly recover data from the network and to provide the opportunity for other stations to interleave their traffic with the existing Ethernet traffic. The Ethernet II specification, in addition to specifying a 7-byte preamble and 1-byte SFD that precedes any Ethernet frame, also specifies an *inter-packet gap* (IPG) of 12 byte times (9.6 μ s at 10Mb/s, 960ns at 100Mb/s, 96ns at 1000Mb/s, and 9.6ns at 10,000Mb/s). Thus, the per-frame efficiency for Ethernet II is at most $1500/(12 + 8 + 14 + 1500 + 4) = 0.975293$, or about 98%. One way to improve efficiency when moving large amounts of data across an Ethernet would be to make the frame size larger. This has been accomplished using Ethernet *jumbo frames* [JF], a nonstandard extension to Ethernet (in 1000Mb/s Ethernet switches primarily) that typically allows the frame size to be as large as 9000 bytes. Some environments make use of so-called *super jumbo frames*, which are usually understood to carry more than 9000 bytes. Care should be taken when using jumbo frames, as these larger frames are not interoperable with the smaller 1518-byte frame size used by most legacy Ethernet equipment.

3.2.3 802.1p/q: Virtual LANs and QoS Tagging

With the growing use of switched Ethernet, it has become possible to interconnect every computer at a site on the same Ethernet LAN. The advantage of doing this is that any host can directly communicate with any other host, using IP and other network-layer protocols, and requiring little or no administrator configuration. In addition, broadcast and multicast traffic (see Chapter 9) is distributed to all hosts that may wish to receive it without having to set up special multicast routing protocols. While these represent some of the advantages of placing many stations on the same Ethernet, having broadcast traffic go to every computer can create an undesirable amount of network traffic when many hosts use broadcast, and there may be some security reasons to disallow complete any-to-any station communication.

To address some of these problems with running large, multiuse switched networks, IEEE extended the 802 LAN standards with a capability called *virtual LANs* (VLANs) in a standard known as 802.1q [802.1Q-2005]. Compliant Ethernet switches isolate traffic among hosts to common VLANs. Note that because of this isolation, two hosts attached to the same switch but operating on different VLANs require a router between them for traffic to flow. Combination switch/router devices have been created to address this need, and ultimately the performance of routers has been improved to match the performance of VLAN switching. Thus, the appeal of VLANs has diminished somewhat, in favor of modern high-performance routers. Nonetheless, they are still used, remain popular in some environments, and are important to understand.

Several methods are used to specify the station-to-VLAN mapping. Assigning VLANs by port is a simple and common method, whereby the switch port to which the station is attached is assigned a particular VLAN, so any station so attached becomes a member of the associated VLAN. Other options include MAC-address-based VLANs that use tables within Ethernet switches to map a station's

MAC address to a corresponding VLAN. This can become difficult to manage if stations change their MAC addresses (which they do sometimes, thanks to the behavior of some users). IP addresses can also be used as a basis for assigning VLANs.

When stations in different VLANs are attached to the same switch, the switch ensures that traffic does not leak from one VLAN to another, irrespective of the types of Ethernet interfaces being used by the stations. When multiple VLANs must span multiple switches (*trunking*), it becomes necessary to label Ethernet frames with the VLAN to which they belong before they are sent to another switch. Support for this capability uses a tag called the *VLAN tag* (or header), which holds 12 bits of *VLAN identifier* (providing for 4096 VLANs, although VLAN 0 and VLAN 4095 are reserved). It also contains 3 bits of priority for supporting QoS, defined in the 802.1p standard, as indicated in Figure 3-3. In many cases, the administrator must configure the ports of the switch to be used to send 802.1p/q frames by enabling trunking on the appropriate ports. To make this job somewhat easier, some switches support a *native VLAN* option on trunked ports, meaning that untagged frames are by default associated with the native VLAN. Trunking ports are used to interconnect VLAN-capable switches, and other ports are typically used to attach stations. Some switches also support proprietary methods for VLAN trunking (e.g., the Cisco *Inter-Switch Link* (ISL) protocol).

802.1p specifies a mechanism to express a QoS identifier on each frame. The 802.1p header includes a 3-bit-wide *Priority* field indicating a QoS level. This standard is an extension of the 802.1q VLAN standard. The two standards work together and share bits in the same header. With the 3 available bits, eight classes of service are defined. Class 0, the lowest priority, is for conventional, best-effort traffic. Class 7 is the highest priority and might be used for critical routing or network management functions. The standards specify how priorities are encoded in packets but leave the policy that governs which packets should receive which class, and the underlying mechanisms implementing prioritized services, to be defined by the implementer. Thus, the way traffic of one priority class is handled relative to another is implementation- or vendor-defined. Note that 802.1p can be used independently of VLANs if the *VLAN ID* field in the 802.1p/q header is set to 0.

The Linux command for manipulating 802.1p/q information is called **vconfig**. It can be used to add and remove virtual interfaces associating VLAN IDs to physical interfaces. It can also be used to set 802.1p priorities, change the way virtual interfaces are identified, and influence the mapping between packets tagged with certain VLAN IDs and how they are prioritized during protocol processing in the operating system. The following commands add a virtual interface to interface **eth1** with VLAN ID 2, remove it, change the way such virtual interfaces are named, and add a new interface:

```
Linux# vconfig add eth1 2
Added VLAN with VID == 2 to IF -:eth1:-
Linux# ifconfig eth1.2
```

```

eth1.2 Link encap:Ethernet HWaddr 00:04:5A:9F:9E:80
        BROADCAST MULTICAST MTU:1500 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:0 (0.0 b) TX bytes:0 (0.0 b)

Linux# vconfig rem eth1.2
Removed VLAN -:eth1.2:-
Linux# vconfig set_name_type VLAN_PLUS_VID
Set name-type for VLAN subsystem. Should be visible in
        /proc/net/vlan/config
Linux# vconfig add eth1 2
Added VLAN with VID == 2 to IF -:eth1:-
Linux# ifconfig vlan0002
vlan0002 Link encap:Ethernet HWaddr 00:04:5A:9F:9E:80
        BROADCAST MULTICAST MTU:1500 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:0
        RX bytes:0 (0.0 b) TX bytes:0 (0.0 b)

```

Here we can see that the default method of naming virtual interfaces in Linux is based on concatenating the associated physical interface with the VLAN ID. For example, VLAN ID 2 associated with the interface `eth1` is called `eth1.2`. This example also shows how an alternative naming method can be used, whereby the VLANs are enumerated by the names `vlan<n>` where `<n>` is the identifier of the VLAN. Once this is set up, frames sent on the VLAN device are tagged with the VLAN ID, as expected. We can see this using Wireshark, as shown in Figure 3-5.

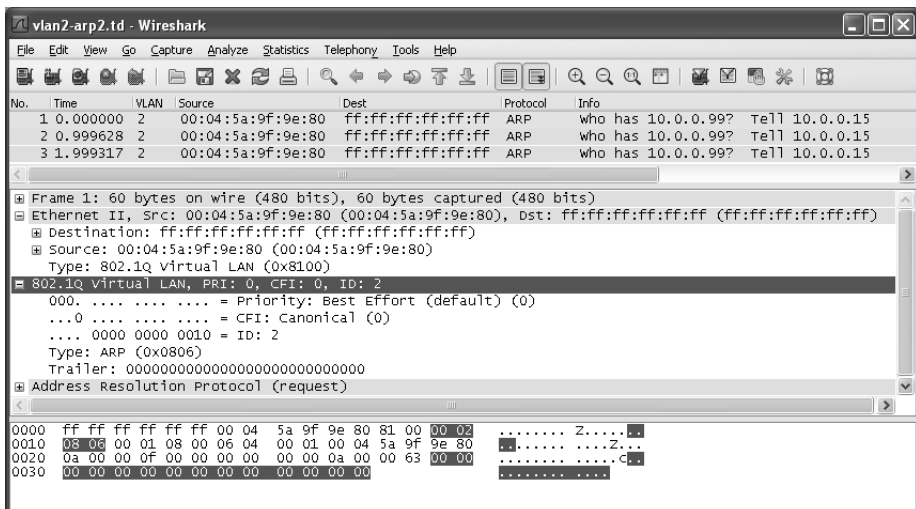


Figure 3-5 Frames tagged with the VLAN ID as shown in Wireshark. The default columns and settings have been changed to display the VLAN ID and raw Ethernet addresses.

This figure shows an ARP packet (see Chapter 4) carried on VLAN 2. We can see that the frame size is 60 bytes (not including CRC). The frame is encapsulated using the Ethernet II encapsulation with type 0x8100, indicating a VLAN. Other than the VLAN header, which indicates that this frame belongs to VLAN 2 and has priority 0, this frame is unremarkable. All the other fields are as we would expect with a regular ARP packet.

3.2.4 802.1AX: Link Aggregation (Formerly 802.3ad)

Some systems equipped with multiple network interfaces are capable of *bonding* or *link aggregation*. With link aggregation, two or more interfaces are treated as one in order to achieve greater reliability through redundancy or greater performance by splitting (striping) data across multiple interfaces. The IEEE Amendment 802.1AX [802.1AX-2008] defines the most common method for performing link aggregation and the *Link Aggregation Control Protocol* (LACP) to manage such links. LACP uses IEEE 802 frames of a particular format (called LACPDUs).

Using link aggregation on Ethernet switches that support it can be a cost-effective alternative to investing in switches with high-speed network ports. If more than one port can be aggregated to provide adequate bandwidth, higher-speed ports may not be required. Link aggregation may be supported not only on network switches but across multiple *network interface cards* (NICs) on a host computer. Often, aggregated ports must be of the same type, operating in the same mode (i.e., half- or full-duplex).

Linux has the capability to implement link aggregation (bonding) across different types of devices using the following commands:

```
Linux# modprobe bonding
Linux# ifconfig bond0 10.0.0.111 netmask 255.255.255.128
Linux# ifenslave bond0 eth0 wlan0
```

This set of commands first loads the bonding driver, which is a special type of device driver supporting link aggregation. The second command creates the `bond0` interface with the IPv4 address information provided. Although providing the IP-related information is not critical for creating an aggregated interface, it is typical. Once the `ifenslave` command executes, the bonding device, `bond0`, is labeled with the MASTER flag, and the `eth0` and `wlan0` devices are labeled with the SLAVE flag:

```
bond0 Link encap:Ethernet HWaddr 00:11:A3:00:2C:2A
       inet addr:10.0.0.111 Bcast:10.0.0.127 Mask:255.255.255.128
       inet6 addr: fe80::211:a3ff:fe00:2c2a/64 Scope:Link
       UP BROADCAST RUNNING MASTER MULTICAST MTU:1500 Metric:1
       RX packets:2146 errors:0 dropped:0 overruns:0 frame:0
       TX packets:985 errors:0 dropped:0 overruns:0 carrier:0
       collisions:18 txqueuelen:0
       RX bytes:281939 (275.3 KiB) TX bytes:141391 (138.0 KiB)
```

```
eth0 Link encap:Ethernet HWaddr 00:11:A3:00:2C:2A
    UP BROADCAST RUNNING SLAVE MULTICAST MTU:1500 Metric:1
    RX packets:1882 errors:0 dropped:0 overruns:0 frame:0
    TX packets:961 errors:0 dropped:0 overruns:0 carrier:0
    collisions:18 txqueuelen:1000
    RX bytes:244231 (238.5 KiB) TX bytes:136561 (133.3 KiB)
    Interrupt:20 Base address:0x6c00
wlan0 Link encap:Ethernet HWaddr 00:11:A3:00:2C:2A
    UP BROADCAST SLAVE MULTICAST MTU:1500 Metric:1
    RX packets:269 errors:0 dropped:0 overruns:0 frame:0
    TX packets:24 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:38579 (37.6 KiB) TX bytes:4830 (4.7 KiB)
```

In this example, we have bonded together a wired Ethernet interface with a Wi-Fi interface. The master device, `bond0`, is assigned the IPv4 address information we would typically assign to either of the individual interfaces, and it receives the first slave's MAC address by default. When IPv4 traffic is sent out of the `bond0` virtual interface, there are a number of possibilities as to which of the slave interfaces will carry it. In Linux, the options are selected using arguments provided when the bonding driver is loaded. For example, a mode option determines whether round-robin delivery is used between the interfaces, one interface acts as a backup to the other, the interface is selected based on performing an XOR of the MAC source and destination addresses, frames are copied to all interfaces, 802.3ad standard link aggregation is performed, or more advance load-balancing options are used. The second mode is used for high-availability systems that can fail over to a redundant network infrastructure if one link has ceased functioning (detectable by MII monitoring; see [BOND] for more details). The third mode is intended to choose the slave interface based on the traffic flow. With enough different destinations, traffic between the two stations is pinned to one interface. This can be useful when trying to minimize reordering while also trying to load-balance traffic across multiple slave interfaces. The fourth mode is for fault tolerance. The fifth mode is for use with 802.3ad-capable switches, to enable dynamic aggregation over homogeneous links.

The LACP protocol is designed to make the job of setting up link aggregation simpler by avoiding manual configuration. Typically the LACP “actor” (client) and “partner” (server) send LACPDUs every second once enabled. LACP automatically determines which member links can be aggregated into a *link aggregation group* (LAG) and aggregates them. This is accomplished by sending a collection of information (MAC address, port priority, port number, and key) across the link. A receiving station can compare the values it sees from other ports and perform the aggregation if they match. Details of LACP are covered in [802.1AX-2008].

3.3 Full Duplex, Power Save, Autonegotiation, and 802.1X Flow Control

When Ethernet was first developed, it operated only in half-duplex mode using a shared cable. That is, data could be sent only one way at one time, so only one station was sending a frame at any given point in time. With the development of switched Ethernet, the network was no longer a single piece of shared wire, but instead many sets of links. As a result, multiple pairs of stations could exchange data simultaneously. In addition, Ethernet was modified to operate in full duplex, effectively disabling the collision detection circuitry. This also allowed the physical length of the Ethernet to be extended, because the timing constraints associated with half-duplex operation and collision detection were removed.

In Linux, the `ethtool` program can be used to query whether full duplex is supported and whether it is being used. This tool can also display and set many other interesting properties of an Ethernet interface:

```
Linux# ethtool eth0
```

```
Settings for eth0:
```

```
Supported ports: [ TP MII ]
Supported link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
Supports auto-negotiation: Yes
Advertised link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
Advertised auto-negotiation: Yes
Speed: 10Mb/s
Duplex: Half
Port: MII
PHYAD: 24
Transceiver: internal
Auto-negotiation: on
Current message level: 0x00000001 (1)
Link detected: yes
```

```
Linux# ethtool eth1
```

```
Settings for eth1:
```

```
Supported ports: [ TP ]
Supported link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Full
Supports auto-negotiation: Yes
Advertised link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Full
Advertised auto-negotiation: Yes
Speed: 100Mb/s
Duplex: Full
Port: Twisted Pair
PHYAD: 0
Transceiver: internal
Auto-negotiation: on
```

```
Supports Wake-on: umbg
Wake-on: g
Current message level: 0x00000007 (7)
Link detected: yes
```

In this example, the first Ethernet interface (`eth0`) is attached to a half-duplex 10Mb/s network. We can see that it is capable of *autonegotiation*, which is a mechanism originating with 802.3u to enable interfaces to exchange information such as speed and capabilities such as half- or full-duplex operation. Autonegotiation information is exchanged at the physical layer using signals sent when data is not being transmitted or received. We can see that the second Ethernet interface (`eth1`) also supports autonegotiation and has set its rate to 100Mb/s and operation mode to full duplex. The other values (Port, PHYAD, Transceiver) identify the physical port type, its address, and whether the physical-layer circuitry is internal or external to the NIC. The current message-level value is used to configure log messages associated with operating modes of the interface; its behavior is specific to the driver being used. We discuss the wake-on values after the following example.

In Windows, details such as these are available by navigating to Control Panel | Network Connections and then right-clicking on the interface of interest, selecting Properties, and then clicking the Configure box and selecting the Advanced tab. This brings up a menu similar to the one shown in Figure 3-6 (this particular example is from an Ethernet interface on a Windows 7 machine).

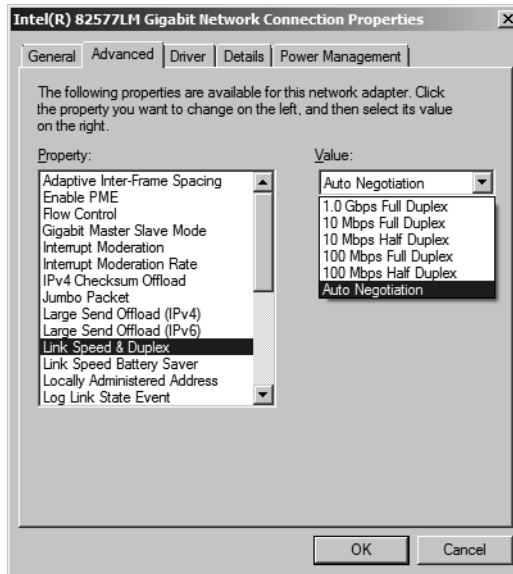


Figure 3-6 Advanced tab of network interface properties in Windows (7). This control allows the user to supply operating parameters to the network device driver.

In Figure 3-6, we can see the special features that can be configured using the adapter's device driver. For this particular adapter and driver, 802.1p/q tags can be enabled or disabled, as can flow control and wake-up capabilities (see Section 3.3.2). The speed and duplex can be set by hand, or to the more typical autonegotiation option.

3.3.1 Duplex Mismatch

Historically, there have been some interoperability problems using autonegotiation, especially when a computer and its associated switch port are configured using different duplex configurations or when autonegotiation is disabled at one end of the link but not the other. In this case, a so-called *duplex mismatch* can occur. Perhaps surprisingly, when this happens the connection does not completely fail but instead may suffer significant performance degradation. When the network has moderate to heavy traffic in both directions (e.g., during a large data transfer), a half-duplex interface can detect incoming traffic as a collision, triggering the exponential backoff function of the CSMA/CD Ethernet MAC. At the same time, the data triggering the collision is lost and may require higher-layer protocols such as TCP to retransmit. Thus, the performance degradation may be noticed only when there is sufficient traffic for the half-duplex interface to be receiving data at the same time it is sending, a situation that does not generally occur under light load. Some researchers have attempted to build analysis tools to detect this unfortunate situation [SC05].

3.3.2 Wake-on LAN (WoL), Power Saving, and Magic Packets

In both the Linux and Windows examples, we saw some indication of power management capabilities. In Windows the *Wake-Up Capabilities* and in Linux the *Wake-On* options are used to bring the network interface and/or host computer out of a lower-power (sleep) state based on the arrival of certain kinds of packets. The kinds of packets used to trigger the change to full-power state can be configured. In Linux, the Wake-On values are zero or more bits indicating whether receiving the following types of frames trigger a wake-up from a low-power state: any physical-layer (PHY) activity (p), unicast frames destined for the station (u), multicast frames (m), broadcast frames (b), ARP frames (a), magic packet frames (g), and magic packet frames including a password. These can be configured using options to `ethtool`. For example, the following command can be used:

```
Linux# ethtool -s eth0 wol umgb
```

This command configures the `eth0` device to signal a wake-up if any of the frames corresponding to the types `u`, `m`, `g`, or `b` is received. Windows provides a similar capability, but the standard user interface allows only magic packet frames and a predefined subset of the `u`, `m`, `b`, and `a` frame types. Magic packets contain

a special repeated pattern of the byte value 0xFF. Often, such frames are sent as a form of UDP packet (see Chapter 10) encapsulated in a broadcast Ethernet frame. Several tools are available to generate them, including wol [WOL]:

```
Linux# wol 00:08:74:93:C8:3C
Waking up 00:08:74:93:C8:3C...
```

The result of this command is to construct a magic packet, which we can view using Wireshark (see Figure 3-7).

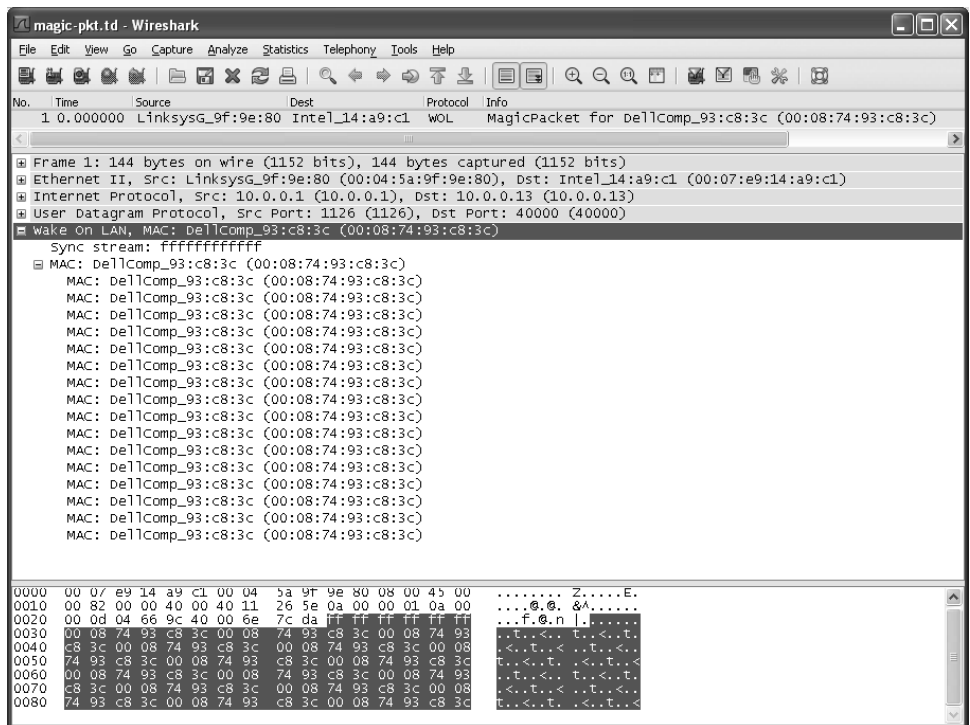


Figure 3-7 A magic packet frame in Wireshark begins with 6 0xFF bytes and then repeats the MAC address 16 times.

The packet shown in Figure 3-7 is mostly a conventional UDP packet, although the port numbers (1126 and 40000) are arbitrary. The most unusual part of the packet is the data area. It contains an initial 6 bytes with the value 0xFF. The rest of the data area includes the destination MAC address 00:08:74:93:C8:3C repeated 16 times. This data payload pattern defines the magic packet.

3.3.3 Link-Layer Flow Control

Operating an extended Ethernet LAN in full-duplex mode and across segments of different speeds may require the switches to buffer (store) frames for some period of time. This happens, for example, when multiple stations send to the same destination (called output port contention). If the aggregate traffic rate headed for a station exceeds the station's link rate, frames start to be stored in the intermediate switches. If this situation persists for a long time, frames may be dropped.

One way to mitigate this situation is to apply *flow control* to senders (i.e., slow them down). Some Ethernet switches (and interfaces) implement flow control by sending special signal frames between switches and NICs. Flow control signals to the sender that it must slow down its transmission rate, although the specification leaves the details of this to the implementation. Ethernet uses an implementation of flow control called *PAUSE messages* (also called *PAUSE frames*), specified by 802.3x [802.3-2008].

PAUSE messages are contained in MAC control frames, identified by the Ethernet *Length/Type* field having the value 0x8808 and using the MAC control opcode of 0x0001. A receiving station seeing this is advised to slow its rate. PAUSE frames are always sent to the MAC address 01:80:C2:00:00:01 and are used only on full-duplex links. They include a *hold-off* time value (specified in *quantas* equal to 512 bit times), indicating how long the sender should pause before continuing to transmit.

The MAC control frame is a frame format using the regular encapsulation from Figure 3-3, but with a 2-byte opcode immediately following the *Length/Type* field. PAUSE frames are essentially the only type of frames that uses MAC control frames. They include a 2-byte quantity encoding the hold-off time. Implementation of the "entire" MAC control layer (basically, just 802.3x flow control) is optional.

Using Ethernet-layer flow control may have a significant negative side effect, and for this reason it is typically not used. When multiple stations are sending through a switch (see the next section) that is becoming overloaded, the switch may naturally send PAUSE frames to all hosts. Unfortunately, the utilization of the switch's memory may not be symmetric with respect to the sending hosts, so some may be penalized (flow-controlled) even though they were not responsible for much of the traffic passing through the switch.

3.4 Bridges and Switches

The IEEE 802.1d standard specifies the operation of bridges, and thus switches, which are essentially high-performance bridges. A bridge or switch is used to join multiple physical link-layer networks (e.g., a pair of physical Ethernet segments) or groups of stations. The most basic setup involves connecting two switches to form an extended LAN, as shown in Figure 3-8.

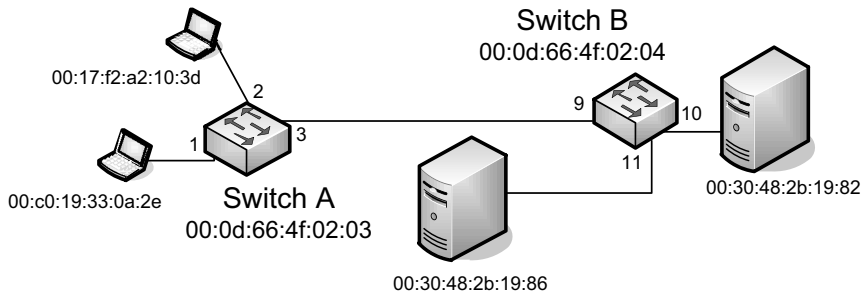


Figure 3-8 A simple extended Ethernet LAN with two switches. Each switch port has a number for reference, and each station (including each switch) has its own MAC address.

Switches A and B in the figure have been interconnected to form an extended LAN. In this particular example, client systems are connected to A and servers to B, and ports are numbered for reference. Note that every network element, including each switch, has its own MAC address. Nonlocal MAC addresses are “learned” by each bridge over time so that eventually every switch knows the port upon which every station can be reached. These lists are stored in tables (called *filtering databases*) within each switch on a per-port (and possibly per-VLAN) basis. As an example, after each switch has learned the location of every station, these databases would contain the information shown in Figure 3-9.

Station	Port
00:17:f2:a2:10:3d	2
00:c0:19:33:0a:2e	1
00:0d:66:4f:02:03	
00:0d:66:4f:02:04	3
00:30:48:2b:19:82	3
00:30:48:2b:19:86	3

Switch A's Database

Station	Port
00:17:f2:a2:10:3d	9
00:c0:19:33:0a:2e	9
00:0d:66:4f:02:03	9
00:0d:66:4f:02:04	
00:30:48:2b:19:82	10
00:30:48:2b:19:86	11

Switch B's Database

Figure 3-9 Filtering databases on switches A and B from Figure 3-8 are created over time (“learned”) by observing the source address on frames seen on switch ports.

When a switch (bridge) is first turned on, its database is empty, so it does not know the location of any stations except itself. Whenever it receives a frame destined for a station other than itself, it makes a copy for each of the ports other than the one on which the frame arrived and sends a copy of the frame out of each

one. If switches (bridges) never learned the location of stations, every frame would be delivered across every network segment, leading to unwanted overhead. The learning capability reduces overhead significantly and is a standard feature of switches and bridges.

Today, most operating systems support the capability to bridge between network interfaces, meaning that a standard computer with multiple interfaces can be used as a bridge. In Windows, for example, interfaces may be bridged together by navigating to the Network Connections menu from the Control Panel, highlighting the interfaces to bridge, right-clicking the mouse, and selecting Bridge Connections. When this is done, a new icon appears that represents the bridging function itself. Most of the normal network properties associated with the interfaces are gone and instead appear on the bridge device (see Figure 3-10).

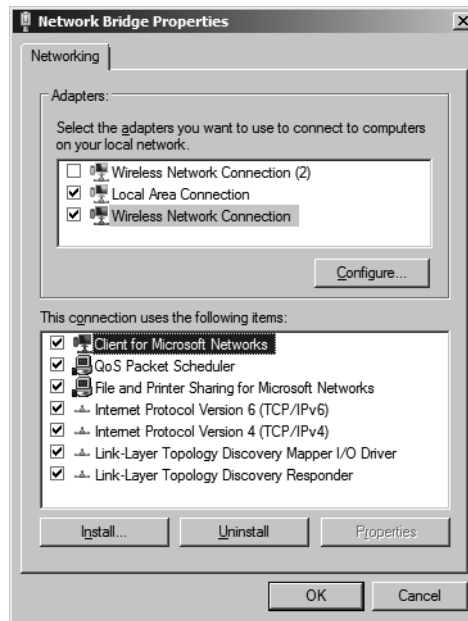


Figure 3-10 In Windows, the bridge device is created by highlighting the network interfaces to be bridged, right-clicking, and selecting the Bridge Network Interfaces function. Once the bridge is established, further modifications are made to the bridge device.

Figure 3-10 shows the Properties panels for the network bridge virtual device on Windows 7. The bridge device's properties include a list of the underlying devices being bridged and the set of services running on the bridge (e.g., the Microsoft Networks client, File and Printer Sharing, etc.). Linux works in a similar way, using command-line arguments. We use the topology shown in Figure 3-11 for this example.

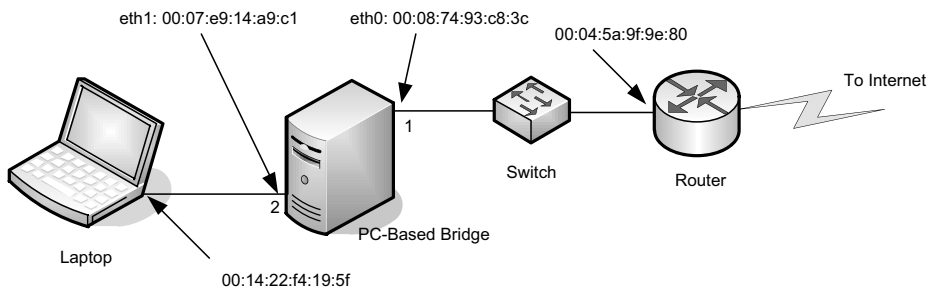


Figure 3-11 In this simple topology, a Linux-based PC is configured to operate as a bridge between the two Ethernet segments it interconnects. As a learning bridge, it accumulates tables of which port should be used to reach the various other systems on the extended LAN.

The simple network in Figure 3-11 uses a Linux-based PC with two Ethernet ports as a bridge. Attached to port 2 is a single station, and the rest of the network is attached to port 1. The following commands enable the bridge:

```
Linux# brctl addbr br0
Linux# brctl addif br0 eth0
Linux# brctl addif br0 eth1
Linux# ifconfig eth0 up
Linux# ifconfig eth1 up
Linux# ifconfig br0 up
```

This series of commands creates a bridge device `br0` and adds the interfaces `eth0` and `eth1` to the bridge. Interfaces can be removed using the `brctl delif` command. Once the interfaces are established, the `brctl showmacs` command can be used to inspect the filter databases (called *forwarding databases* or *fdb*s in Linux terminology):

```
Linux# brctl show
bridge name bridge id          STP enabled interfaces
br0          8000.0007e914a9c1 no          eth0 eth1

Linux# brctl showmacs br0
port no mac addr is local? ageing timer
 1 00:04:5a:9f:9e:80 no 0.79
 2 00:07:e9:14:a9:c1 yes 0.00
 1 00:08:74:93:c8:3c yes 0.00
 2 00:14:22:f4:19:5f no 0.81
 1 00:17:f2:e7:6d:91 no 2.53
 1 00:90:f8:00:90:b7 no 17.13
```

The output of this command reveals one other detail about bridges. Because stations may move around, have their network cards replaced, have their MAC address changed, or other things, once the bridge discovers that a MAC address

is reachable via a certain port, this information cannot be assumed to be correct forever. To deal with this issue, each time an address is learned, a timer is started (commonly defaulted to 5 minutes). In Linux, a fixed amount of time associated with the bridge is applied to each learned entry. If the address in the entry is not seen again within the specified “ageing” time, the entry is removed, as indicated here:

```
Linux# brctl setageing br0 1
Linux# brctl showmacs br0
port no mac addr is local? ageing timer
 1 00:04:5a:9f:9e:80 no 0.76
 2 00:07:e9:14:a9:c1 yes 0.00
 1 00:08:74:93:c8:3c yes 0.00
 2 00:14:22:f4:19:5f no 0.78
 1 00:17:f2:e7:6d:91 no 0.00
```

Here, we have set the `ageing` value unusually low for demonstration purposes. When an entry is removed because of aging, subsequent frames for the removed destination are once again sent out of every port except the receiving one (called *flooding*), and the entry is placed anew into the filtering database. The use of filtering databases and learning is really a performance optimization—if the tables are empty, the network experiences more overhead but still functions. Next we turn our attention to the case where more than two bridges are interconnected with redundant links. In this situation, flooding of frames could lead to a sort of flooding catastrophe with frames looping forever. Obviously, we require a way of dealing with this problem.

3.4.1 Spanning Tree Protocol (STP)

Bridges may operate in isolation, or in combination with other bridges. When more than two bridges are in use (or in general when switch ports are cross-connected), the possibility exists for a cascading, looping set of frames to be formed. Consider the network shown in Figure 3-12.

Assume that the switches in Figure 3-12 have just been turned on and their filtering databases are empty. When station S sends a frame, switch B replicates the frame on ports 7, 8, and 9. So far, the initial frame has been “amplified” three times. These frames are received by switches A, D, and C. Switch A produces copies of the frame on ports 2 and 3. Switches D and C produce more copies on ports 20, 22 and 13, 14, respectively. The amplification factor has grown to 6, with copies of the frames traveling in both directions among switches A, C, and D. Once these frames arrive, the forwarding databases begin to oscillate as the bridge attempts to figure out which port is really the one through which station S should be reached. Obviously, this situation is intolerable. If it were allowed to occur, bridges used in such configurations would be useless. Fortunately, there is a protocol that is used to avoid this situation called the *Spanning Tree Protocol* (STP). We describe STP in

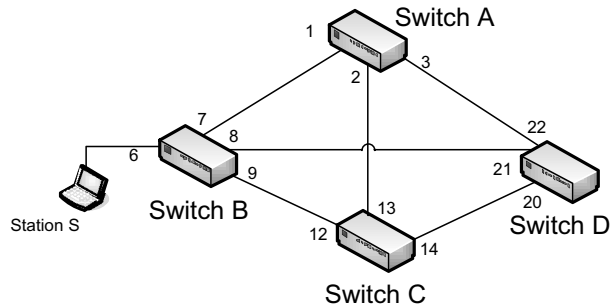


Figure 3-12 An extended Ethernet network with four switches and multiple redundant links. If simple flooding were used in forwarding frames through this network, a catastrophe would occur because of excess multiplying traffic (a so-called broadcast storm). This type of situation requires the use of the STP.

some detail to explain why some approach to duplicate suppression is needed for bridges and switches. In the current standard [802.1D-2004], conventional STP is replaced with the *Rapid Spanning Tree Protocol* (RSTP), which we describe after the conventional STP preliminaries.

STP works by disabling certain ports at each bridge so that topological loops are avoided (i.e., no duplicate paths between bridges are permitted), yet the topology is not partitioned—all stations can be reached. Mathematically, a spanning tree is a collection of all of the nodes and some of the edges of a graph such that there is a path or route from any node to any other node (*spanning* the graph), but there are no loops (the edge set forms a *tree*). There can be many spanning trees on a graph. STP finds one of them for the graph formed by bridges as nodes and links as edges. Figure 3-13 illustrates the idea.

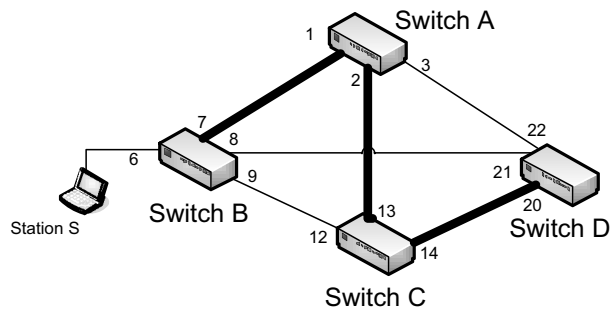


Figure 3-13 Using STP, the B-A, A-C, and C-D links have become active on the spanning tree. Ports 6, 7, 1, 2, 13, 14, and 20 are in the forwarding state; all other ports are blocked (i.e., not forwarding). This keeps frames from looping and avoids broadcast storms. If a configuration change occurs or a switch fails, the blocked ports are changed to the forwarding state and the bridges compute a new spanning tree.

In this figure, the dark lines represent the links in the network selected by STP for forwarding frames. None of the other links are used—ports 8, 9, 12, 21, 22, and 3 are *blocked*. With STP, the various problems raised earlier do not occur, as frames are created only as the result of another frame arriving. There is no amplification. Furthermore, looping is avoided because there is only one path between any two stations. The spanning tree is formed and maintained by bridges using a distributed algorithm running in each bridge.

As with forwarding databases, STP must deal with the situation where bridges are turned off and on, interface cards are replaced, or MAC addresses are changed. Clearly, such changes could affect the operation of the spanning tree, so the STP adapts to these changes. The adaptation is implemented using an exchange of special frames called *Bridge Protocol Data Units* (BPDUs). These frames are used for forming and maintaining the spanning tree. The tree is “grown” from a bridge elected by the others and known as the “root bridge.”

As mentioned previously, there are many possible spanning trees for a given network. Determining which one might be the best to use for forwarding frames depends on a set of *costs* that can be associated with each link and the location of the root bridge. Costs are simply integers that are (recommended to be) inversely proportional to the link speeds. For example, a 10Mb/s link has a recommended cost of 100, and 100Mb/s and 1000Mb/s links have recommended cost values of 19 and 4, respectively. STP operates by computing least-cost paths to the root bridge using these costs. If multiple links must be traversed, the corresponding cost is simply the sum of the link costs.

3.4.1.1 Port States and Roles

To understand the basic operation of STP, we need to understand the operation of the state machine for each port at each bridge, as well as the contents of BPDUs. Each port in each bridge may be in one of five states: blocking, listening, learning, forwarding, and disabled. The relationship among them can be seen in the state transition diagram shown in Figure 3-14.

The normal transitions for ports on the spanning tree are indicated in Figure 3-14 by solid arrows, and the smaller arrows with dashed lines indicate changes due to administrative configuration. After initialization, a port enters the blocking state. In this state, it does not learn addresses, forward frames, or transmit BPDUs, but it does monitor received BPDUs in case it needs to be included in the future on a path to the root bridge, in which case the port transitions to the listening state. In the listening state, the port is now permitted to send as well as receive BPDUs but not learn addresses or forward data. After a typical forwarding delay timeout of 15s, a port enters the learning state. Here it is permitted to do all procedures except forward data. It waits another forwarding delay before entering the forwarding state and commencing to forward frames.

Related to the port state machine, each port is said to have a *role*. This terminology becomes more important with RSTP (see Section 3.4.1.6). A port may have the role of *root port*, *designated port*, *alternate port*, or *backup port*. Root ports are those

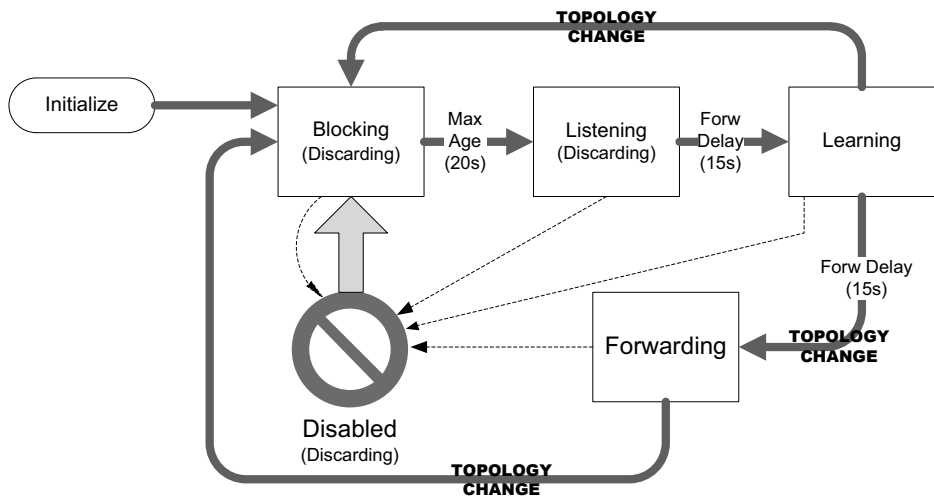


Figure 3-14 Ports transition among four major states in normal STP operation. In the blocking state, frames are not forwarded, but a topology change or timeout may cause a transition to the listening state. The forwarding state is the normal state for active switch ports carrying data traffic. The state names in parentheses indicate the port states according to the RSTP.

ports at the end of an edge on the spanning tree headed toward the root. Designated ports are ports in the forwarding state acting as the port on the least-cost path to the root from the attached segment. Alternate ports are other ports on an attached segment that could also reach the root but at higher cost. They are not in the forwarding state. A backup port is a port connected to the same segment as a designated port *on the same bridge*. Thus, backup ports could easily take over for a failing designated port without disrupting any of the rest of the spanning tree topology but do not offer an alternate path to the root should the entire bridge fail.

3.4.1.2 BPDU Structure

To determine the links in the spanning tree, STP uses BPDUs that adhere to the format shown in Figure 3-15.

The format shown in Figure 3-15 applies to both the original STP as well as the newer RSTP (see Section 3.4.1.6). BPDUs are always sent to the group address 01:80:C2:00:00:00 (see Chapter 9 for details of link-layer group and Internet multicast addressing) and are not forwarded through a bridge without modification. In the figure, the *DST*, *SRC*, and *L/T (Length/Type)* fields are part of the conventional Ethernet (802.3) header of the frame carrying the example BPDU. The 3-byte *LLC/SNAP* header is defined by 802.1 and for BPDUs is set to the constant 0x424203. Not all BPDUs are encapsulated using LLC/SNAP, but this is a common option.

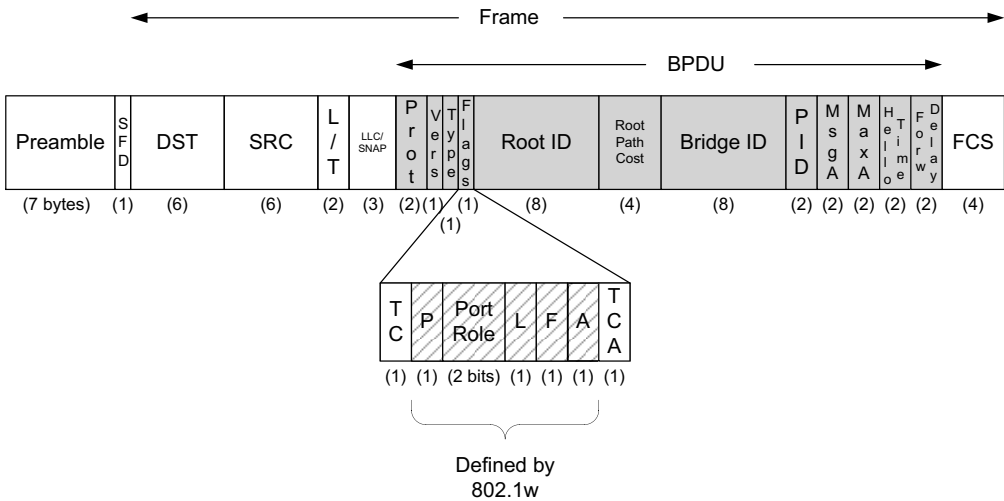


Figure 3-15 BPDUs are carried in the payload area of 802 frames and exchanged between bridges to establish the spanning tree. Important fields include the source, root node, cost to root, and topology change indication. With 802.1w and [802.1D-2004] (including Rapid STP or RSTP), additional fields indicate the state of the ports.

The *Protocol (Prot)* field gives the protocol ID number, set to 0. The *Version (Vers)* field is set to 0 or 2, depending on whether STP or RSTP is in use. The *Type* field is assigned similarly. The *Flags* field contains *Topology Change (TC)* and *Topology Change Acknowledgment (TCA)* bits, defined by the original 802.1d standard. Additional bits are defined for *Proposal (P)*, *Port Role* (00, unknown; 01, alternate; 10, root; 11, designated), *Learning (L)*, *Forwarding (F)*, and *Agreement (A)*. These are discussed in the context of RSTP in Section 3.4.1.6. The *Root ID* field gives the identifier of the root bridge in the eyes of the sender of the frame, whose MAC address is given in the *Bridge ID* field. Both of these ID fields are encoded in a special way that includes a 2-byte *Priority* field immediately preceding the MAC address. The priority values can be manipulated by management software in order to force the spanning tree to be rooted at any particular bridge (Cisco, for example, uses a default value of 0x8000 in its Catalyst switches).

The root path cost is the computed cost to reach the bridge specified in the *Root ID* field. The *PID* field is the port identifier and gives the number of the port from which the frame was sent appended to a 1-byte configurable *Priority* field (default 0x80). The *Message A (MsgA)* field gives the message age (see the next paragraph). The *Maximum Age (MaxA)* field gives the maximum age before timeout (default: 20s). The *Hello Time* field gives the time between periodic transmissions of configuration frames. The *Forward Delay (Forw Delay)* field gives the time spent in the learning and listening states. All of the age and time fields are given in units of 1/256s.

The *Message Age* field is not a fixed value like the other time-related fields. When the root bridge sends a BPDU, it sets this field to 0. Any bridge receiving the frame emits frames on its non-root ports with the *Message Age* field incremented by 1. In essence, the field acts as a hop count, giving the number of bridges by which the BPDU has been processed before being received. When a BPDU is received on a port, the information it contains is kept in memory and participates in the STP algorithm until it is timed out, which happens at time ($MaxA - MsgA$). Should this time pass on a root port without receipt of another BPDU, the root bridge is declared “dead” and the bridge starts the root bridge election process over again.

3.4.1.3 Building the Spanning Tree

The first job of STP is to elect the root bridge. The root bridge is discovered as the bridge in the network (or VLAN) with the smallest identifier (priority combined with MAC address). When a bridge initializes, it assumes itself to be the root bridge and sends configuration BPDUs with the *Root ID* field matching its own bridge ID, but if it detects a bridge with a smaller ID, it ceases sending its own frames and instead adopts the frame it received containing the smaller ID to be the basis for further BPDUs it sends. The port where the BPDU with the smaller root ID was received is then marked as the root port (i.e., the port on the path to the root bridge). The remaining ports are placed in either blocked or forwarding states.

3.4.1.4 Topology Changes

The next important job of STP is to handle topology changes. Although we could conceivably use the basic database aging mechanism described earlier to adapt to changing topologies, this is a poor approach because the aging timers can take a long time (5 minutes) to delete incorrect entries. Instead, STP incorporates a way to detect topology changes and inform the network about them quickly. In STP, a topology change occurs when a port has entered the blocking or forwarding states. When a bridge detects a connectivity change (e.g., a link goes down), the bridge notifies its parent bridges on the tree to the root by sending *topology change notification* (TCN) BPDUs out of its root port. The next bridge on the tree to the root acknowledges the TCN BPDUs to the notifying bridge and also forwards them on toward the root. Once informed of the topology change, the root bridge sets the *TC* bit field in subsequent periodic configuration messages. Such messages are relayed by every bridge in the network and are received by ports in either the blocking or forwarding states. The setting of this bit field allows bridges to reduce their aging time to that of the forward delay timer, on the order of seconds instead of the 5 minutes normally recommended for the aging time. This allows database entries that may now be incorrect to be purged and relearned more quickly, yet it also allows stations that are actively communicating to not have their entries deleted erroneously.

3.4.1.5 Example

In Linux, the bridge function disables STP by default, on the assumption that topologies are relatively simple in most cases where a regular computer is being

used as a bridge. To enable STP on the example bridge we are using so far, we can do the following:

```
Linux# brctl stp br0 on
```

The consequences of executing this command can be inspected as follows:

```
Linux# brctl showstp br0
```

```
br0
```

bridge id	8000.0007e914a9c1		
designated root	8000.0007e914a9c1		
root port	0	path cost	0
max age	19.99	bridge max age	19.99
hello time	1.99	bridge hello time	1.99
forward delay	14.99	bridge forward delay	14.99
ageing time	0.99		
hello timer	1.26	tcn timer	0.00
topology change timer	3.37	gc timer	3.26

```
flags                                TOPOLOGY_CHANGE TOPOLOGY_CHANGE_DETECTED
```

```
eth0 (0)
```

port id	0000	state	forwarding
designated root	8000.0007e914a9c1	path cost	100
designated bridge	8000.0007e914a9c1	message age timer	0.00
designated port	8001	forward delay timer	0.00
designated cost	0	hold timer	0.26

```
flags
```

```
eth1 (0)
```

port id	0000	state	forwarding
designated root	8000.0007e914a9c1	path cost	19
designated bridge	8000.0007e914a9c1	message age timer	0.00
designated port	8002	forward delay timer	0.00
designated cost	0	hold timer	0.26

```
flags
```

Here we can see the STP setup for a simple bridged network. The bridge device, `br0`, holds information for the bridge as a whole. This includes the bridge ID (8000.0007e914a9c1), derived from the smallest MAC address on the PC-based bridge (port 1) of Figure 3-11. The major configuration parameters (e.g., hello time, topology change timer, etc.) are given in seconds. The `flags` values indicate a recent topology change, which is expected given the fact that the network was recently connected. The rest of the output describes per-port information for `eth0`

(bridge port 1) and `eth1` (bridge port 2). Note that the path cost for `eth0` is about ten times greater than the cost of `eth1`. This is consistent with the observation that `eth0` is a 10Mb/s Ethernet network and `eth1` is a full-duplex 100Mb/s network.

We can use Wireshark to look at a BPDU. In Figure 3-16 we see the contents of a 52-byte BPDU. The length of 52 bytes (less than the Ethernet minimum of 64 bytes because the Linux capture facility removed the padding) is derived from the *Length/Type* field of the Ethernet header by adding 14, in this case giving the length of 52. The destination address is the group address, 01:80:C2:00:00:00, as expected. The payload length is 38 bytes, the value contained in the *Length* field. The *SNAP/LLC* field contains the constant 0x424243, and the encapsulated frame is a spanning tree (version 0) frame. The rest of the protocol fields indicate that the station 00:07:e9:14:a9:c1 believes it is the root of the spanning tree, using priority 32768 (a low priority), and the BPDU has been sent from port 2 with priority 0x80. It also indicates a maximum age of 20s, a hello time of 2s, and a forwarding delay of 15s.

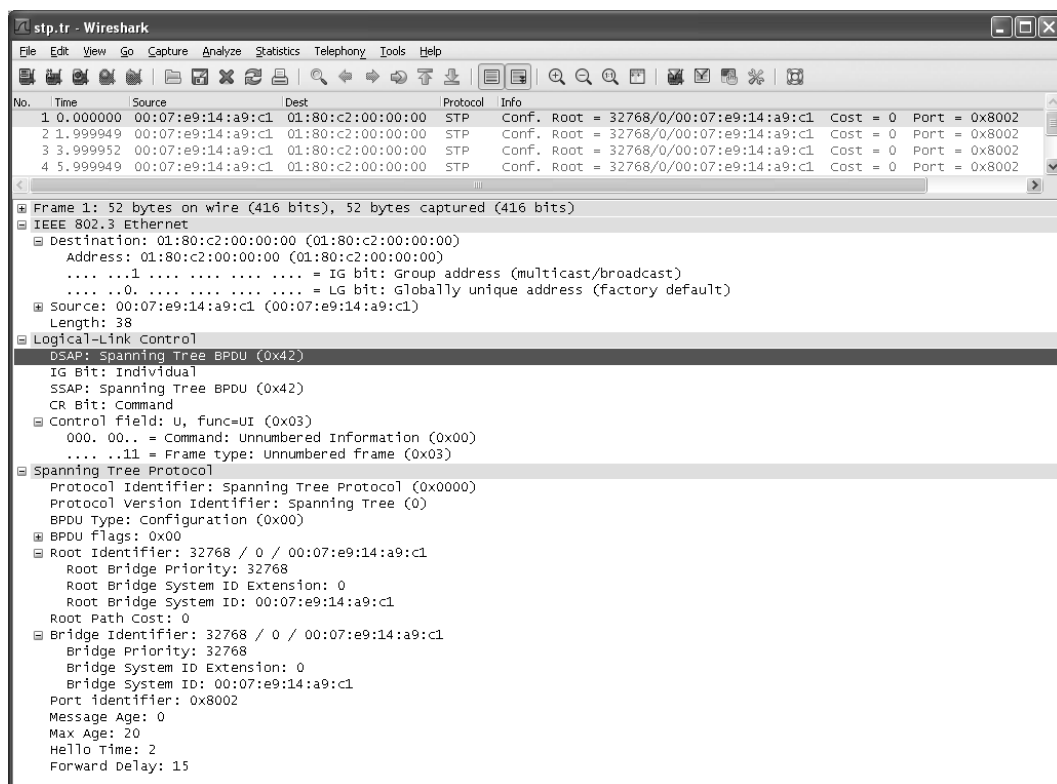


Figure 3-16 Wireshark showing a BPDU. The Ethernet destination is a group address for bridges (01:80:c2:00:00:00).

3.4.1.6 Rapid Spanning Tree Protocol (RSTP) (Formerly 802.1w)

One of the perceived problems with conventional STP is that a change in topology is detected only by the failure to receive a BPDU in a certain amount of time. If the timeout is large, the convergence time (time to reestablish data flow along the spanning tree) could be larger than desired. The IEEE 802.1w standard (now part of [802.1D-2004]) specifies enhancements to the conventional STP and adopts the new name *Rapid Spanning Tree Protocol* (RSTP). The main improvement in RSTP over STP is to monitor the status of each port and upon indication of failure to *immediately* trigger a topology change indication. In addition, RSTP uses all 6 bits in the *Flag* field of the BPDU format to support agreements between bridges that avoid some of the need for timers to initiate protocol operations. It reduces the normal STP five port states to three (discarding, learning, and forwarding, as indicated by the state names in parentheses in Figure 3-14). The discarding state in RSTP absorbs the disabled, blocking, and listening states in conventional STP. RSTP also creates a new port role called an *alternate port*, which acts as an immediate backup should a root port cease to operate.

RSTP uses only one type of BPDU, so there are no special topology change BPDUs, for example. RSTP BPDUs, as they are called, use version and type number 2 instead of 0. In RSTP, any switch detecting a topology change sends BPDUs indicating a topology change, and any switch receiving them clears its filtering databases immediately. This change can significantly affect the protocol's convergence time. Instead of waiting for the topology change to migrate to the root bridge and back followed by the forwarding delay wait time, entries are cleared immediately. Overall, convergence time can be cut from tens of seconds down to a fraction of a second in most cases.

RSTP makes a distinction between *edge ports* (those attached only to end stations) and normal spanning tree ports and also between point-to-point links and shared links. Edge ports and ports on point-to-point links do not ordinarily form loops, so they are permitted to skip the listening and learning states and move directly to the forwarding state. Of course, the assumption of being an edge port could be violated if, for example, two ports were cross-connected, but this is handled by reclassifying ports as spanning tree ports if they ever carry any form of BPDUs (simple end stations do not normally generate BPDUs). Point-to-point links are inferred from the operating mode of the interface; if the interface is running in full-duplex mode, the link is classified as a point-to-point link.

In regular STP, BPDUs are ordinarily relayed from a notifying or root bridge. In RSTP, BPDUs are sent periodically by all bridges as "keepalives" to determine if connections to neighbors are operating properly. This is what most higher-layer routing protocols do also. If a bridge fails to receive an updated BPDU within three times the hello interval, the bridge concludes that it has lost its connection with its neighbor. Note that in RSTP, topology changes are not induced as a result of edge ports being connected or disconnected as they are in regular STP. When a topology change is detected, the notifying bridge sends BPDUs with the *TC* bit

field set, not only to the root but also to all other bridges. Doing so allows the entire network to be notified of the topology change much faster than with conventional STP. When a bridge receives these messages, it flushes all table entries except those associated with edge ports and restarts the learning process.

Many of RSTP's features were developed by Cisco Systems and other companies that had for some time provided proprietary enhancements to regular STP in their products. The IEEE committee incorporated many of these enhancements into the updated 802.1d standard, which covers both types of STP, so extended LANs can run regular STP on some segments and RSTP on others (although the RSTP benefits are lost). RSTP has been extended to include VLANs [802.1Q-2005]—a protocol called the *Multiple Spanning Tree Protocol* (MSTP). This protocol retains the RSTP (and hence STP) BPDU format, so backward compatibility is possible, but it also supports the formation of multiple spanning trees (one for each VLAN).

3.4.2 802.1ak: Multiple Registration Protocol (MRP)

The *Multiple Registration Protocol* (MRP) provides a general method for registering attributes among stations in a bridged LAN environment. [802.1ak-2007] defines two particular “applications” of MRP called MVRP (for registering VLANs) and MMRP (for registering group MAC addresses). MRP replaces the earlier GARP framework; MVRP and MMRP replace the older GVRP and GMRP protocols, respectively. All were originally defined by 802.1q.

With MVRP, once an end station is configured as a member of a VLAN, this information is communicated to its attached switch, which in turn propagates the fact of the station's participation in the VLAN to other switches. This allows switches to augment their filtering tables based on station VLAN IDs and allows changes of VLAN topology without necessarily triggering a recalculation of the existing spanning tree via STP. Avoiding STP recalculation was one of the reasons for migrating from GVRP to MVRP.

MMRP is a method for stations to register their interest in group MAC addresses (multicast addresses). This information may be used by switches to establish the ports through which multicast traffic must be delivered. Without such a facility, switches would have to broadcast all multicast traffic, potentially leading to unwanted overhead. MMRP is a layer 2 protocol with similarities to IGMP and MLD, layer 3 protocols, and the “IGMP/MLD snooping” capability supported in many switches. We discuss IGMP, MLD and snooping in Chapter 9.

3.5 Wireless LANs—IEEE 802.11(Wi-Fi)

One of the most popular technologies being used to access the Internet today is *wireless fidelity* (Wi-Fi), also known by its IEEE standard name 802.11, effectively a wireless version of Ethernet. Wi-Fi has developed to become an inexpensive, highly convenient way to provide connectivity and performance levels acceptable

5

The Internet Protocol (IP)

5.1 Introduction

IP is the workhorse protocol of the TCP/IP protocol suite. All TCP, UDP, ICMP, and IGMP data gets transmitted as IP datagrams. IP provides a best-effort, connectionless datagram delivery service. By “best-effort” we mean there are no guarantees that an IP datagram gets to its destination successfully. Although IP does not simply drop all traffic unnecessarily, it provides no guarantees as to the fate of the packets it attempts to deliver. When something goes wrong, such as a router temporarily running out of buffers, IP has a simple error-handling algorithm: throw away some data (usually the last datagram that arrived). Any required reliability must be provided by the upper layers (e.g., TCP). IPv4 and IPv6 both use this basic best-effort delivery model.

The term *connectionless* means that IP does not maintain any connection state information about related datagrams within the network elements (i.e., within the routers); each datagram is handled independently from all other others. This also means that IP datagrams can be delivered out of order. If a source sends two consecutive datagrams (first A, then B) to the same destination, each is routed independently and can take different paths, and B may arrive before A. Other things can happen to IP datagrams as well: they may be duplicated in transit, and they may have their data altered as the result of errors. Again, some protocol above IP (usually TCP) has to handle all of these potential problems in order to provide an error-free delivery abstraction for applications.

In this chapter we take a look at the fields in the IPv4 (see Figure 5-1) and IPv6 (see Figure 5-2) headers and describe how IP forwarding works. The official specification for IPv4 is given in [RFC0791]. A series of RFCs describe IPv6, starting with [RFC2460].

5.2 IPv4 and IPv6 Headers

Figure 5-1 shows the format of an IPv4 datagram. The normal size of the IPv4 header is 20 bytes, unless *options* are present (which is rare). The IPv6 header is twice as large but never has any options. It may have *extension headers*, which provide similar capabilities, as we shall see later. In our pictures of headers and datagrams, the most significant bit is numbered 0 at the left, and the least significant bit of a 32-bit value is numbered 31 on the right.

The 4 bytes in a 32-bit value are transmitted in the following order: bits 0–7 first, then bits 8–15, then 16–23, and bits 24–31 last. This is called *big endian* byte ordering, which is the byte ordering required for all binary integers in the TCP/IP headers as they traverse a network. It is also called *network byte order*. Computer CPUs that store binary integers in other formats, such as the little endian format used by most PCs, must convert the header values into network byte order for transmission and back again for reception.

5.2.1 IP Header Fields

The first field (only 4 bits or one nibble wide) is the *Version* field. It contains the version number of the IP datagram: 4 for IPv4 and 6 for IPv6. The headers for both IPv4 and IPv6 share the location of the *Version* field but no others. Thus, the two protocols are not directly interoperable—a host or router must handle either IPv4 or IPv6 (or both, called *dual stack*) separately. Although other versions of IP have been proposed and developed, only versions 4 and 6 have any significant amount of use. The IANA keeps an official registry of these version numbers [IV].

The *Internet Header Length (IHL)* field is the number of 32-bit words in the IPv4 header, including any options. Because this is also a 4-bit field, the IPv4 header is limited to a maximum of fifteen 32-bit words or 60 bytes. Later we shall see how this limitation makes some of the options, such as the Record Route option, nearly useless today. The normal value of this field (when no options are present) is 5. There is no such field in IPv6 because the header length is fixed at 40 bytes.

Following the header length, the original specification of IPv4 [RFC0791] specified a *Type of Service (ToS)* byte, and IPv6 [RFC2460] specified the equivalent *Traffic Class* byte. Use of these never became widespread, so eventually this 8-bit field was split into two smaller parts and redefined by a set of RFCs ([RFC3260] [RFC3168] [RFC2474] and others). The first 6 bits are now called the *Differentiated Services Field (DS Field)*, and the last 2 bits are the *Explicit Congestion Notification (ECN)* field or indicator bits. These RFCs now apply to both IPv4 and IPv6. These fields are used for special processing of the datagram when it is forwarded. We discuss them in more detail in Section 5.2.3.

The *Total Length* field is the total length of the IPv4 datagram in bytes. Using this field and the *IHL* field, we know where the data portion of the datagram starts, and its length. Because this is a 16-bit field, the maximum size of an IPv4 datagram (including header) is 65,535 bytes. The *Total Length* field is required in

the header because some lower-layer protocols that carry IPv4 datagrams do not (accurately) convey the size of encapsulated datagrams on their own. Ethernet, for example, pads small frames to be a minimum length (64 bytes). Even though the minimum Ethernet payload size is 46 bytes (see Chapter 3), an IPv4 datagram can be smaller (as few as 20 bytes). If the *Total Length* field were not provided, the IPv4 implementation would not know how much of a 46-byte Ethernet frame was really an IP datagram, as opposed to padding, leading to possible confusion.

Although it is possible to send a 65,535-byte IP datagram, most link layers (such as Ethernet) are not able to carry one this large without fragmenting it (chopping it up) into smaller pieces. Furthermore, a host is not required to be able to receive an IPv4 datagram larger than 576 bytes. (In IPv6 a host must be able to process a datagram at least as large as the MTU of the link to which it is attached, and the minimum link MTU is 1280 bytes.) Many applications that use the UDP protocol (see Chapter 10) for data transport (e.g., DNS, DHCP, etc.) use a limited data size of 512 bytes to avoid the 576-byte IPv4 limit. TCP chooses its own datagram size based on additional information (see Chapter 15).

When an IPv4 datagram is fragmented into multiple smaller fragments, each of which itself is an independent IP datagram, the *Total Length* field reflects the length of the particular fragment. Fragmentation is described in detail along with UDP in Chapter 10. In IPv6, fragmentation is not supported by the header, and the length is instead given by the *Payload Length* field. This field measures the length of the IPv6 datagram *not* including the length of the header; extension headers, however, are included in the *Payload Length* field. As with IPv4, the 16-bit size of the field limits its maximum value to 65,535. With IPv6, however, it is the *payload* length that is limited to 64KB, not the entire datagram. In addition, IPv6 supports a *jumbogram* option (see Section 5.3.1.2) that provides for the possibility, at least theoretically, of single packets with payloads as large as 4GB (4,294,967,295 bytes)!

The *Identification* field helps identify each datagram sent by an IPv4 host. To ensure that the fragments of one datagram are not confused with those of another, the sending host normally increments an internal counter by 1 each time a datagram is sent (from one of its IP addresses) and copies the value of the counter into the IPv4 *Identification* field. This field is most important for implementing fragmentation, so we explore it further in Chapter 10, where we also discuss the *Flags* and *Fragment Offset* fields. In IPv6, this field shows up in the Fragmentation extension header, as we discuss in Section 5.3.3.

The *Time-to-Live* field, or *TTL*, sets an upper limit on the number of routers through which a datagram can pass. It is initialized by the sender to some value (64 is recommended [RFC1122], although 128 or 255 is not uncommon) and decremented by 1 by every router that forwards the datagram. When this field reaches 0, the datagram is thrown away, and the sender is notified with an ICMP message (see Chapter 8). This prevents packets from getting caught in the network forever should an unwanted routing loop occur.

Note

The *TTL* field was originally specified to be the maximum lifetime of an IP datagram in seconds, but routers were also always required to decrement the value by at least 1. Because virtually no routers today hold on to a datagram longer than 1s under normal operation, the earlier rule is now ignored or forgotten, and in IPv6 the field has been renamed to its de facto use: *Hop Limit*.

The *Protocol* field in the IPv4 header contains a number indicating the type of data found in the payload portion of the datagram. The most common values are 17 (for UDP) and 6 (for TCP). This provides a demultiplexing feature so that the IP protocol can be used to carry payloads of more than one protocol type. Although this field originally specified the transport-layer protocol the datagram is encapsulating, it is now understood to identify the encapsulated protocol, which may or not be a transport protocol. For example, other encapsulations are possible, such as IPv4-in-IPv4 (value 4). The official list of the possible values of the *Protocol* field is given in the assigned numbers page [AN]. The *Next Header* field in the IPv6 header generalizes the *Protocol* field from IPv4. It is used to indicate the type of header following the IPv6 header. This field may contain any values defined for the IPv4 *Protocol* field, or any of the values associated with the IPv6 extension headers described in Section 5.3.

The *Header Checksum* field is calculated *over the IPv4 header only*. This is important to understand because it means that the payload of the IPv4 datagram (e.g., TCP or UDP data) is not checked for correctness by the IP protocol. To help ensure that the payload portion of an IP datagram has been correctly delivered, other protocols must cover any important data that follows the header with their own data-integrity-checking mechanisms. We shall see that almost all protocols encapsulated in IP (ICMP, IGMP, UDP, and TCP) have a checksum in their own headers to cover their header and data and also to cover certain parts of the IP header they deem important (a form of “layering violation”). Perhaps surprisingly, the IPv6 header does not have any checksum field.

Note

Omitting the checksum field from the IPv6 header was a somewhat controversial decision. The reasoning behind this action is roughly as follows: Higher-layer protocols requiring correctness in the IP header are required to compute their own checksums over the data they believe to be important. A consequence of errors in the IP header is that the data is delivered to the wrong destination, is indicated to have come from the wrong source, or is otherwise mangled during delivery. Because bit errors are relatively rare (thanks to fiber-optic delivery of Internet traffic) and stronger mechanisms are available to ensure correctness of the other fields (higher-layer checksums or other checks), it was decided to eliminate the field from the IPv6 header.

The algorithm used in computing a checksum is also used by most of the other Internet-related protocols that use checksums and is sometimes known as the *Internet checksum*. Note that when an IPv4 datagram passes through a router, its header checksum must change as a result of decrementing the *TTL* field. We discuss the methods for computing the checksum in more detail in Section 5.2.2.

Every IP datagram contains the *Source IP Address* of the sender of the datagram and the *Destination IP Address* of where the datagram is destined. These are 32-bit values for IPv4 and 128-bit values for IPv6, and they usually identify a single interface on a computer, although multicast and broadcast addresses (see Chapter 2) violate this rule. While a 32-bit address can accommodate a seemingly large number of Internet entities (4.5 billion), there is widespread agreement that this number is inadequate, a primary motivation for moving to IPv6. The 128-bit address of IPv6 can accommodate a huge number of Internet entities. As was restated in [H05], IPv6 has 3.4×10^{38} (340 undecillion) addresses. Quoting from [H05] and others: “The optimistic estimate would allow for 3,911,873,538,269,506,102 addresses per square meter of the surface of the planet Earth.” It certainly seems as if this should last a very, very long time indeed.

5.2.2 The Internet Checksum

The Internet checksum is a 16-bit mathematical sum used to determine, with reasonably high probability, whether a received message or portion of a message matches the one sent. Note that the Internet checksum algorithm is not the same as the common *cyclic redundancy check* (CRC) [PB61], which offers stronger protection.

To compute the IPv4 header checksum for an outgoing datagram, the value of the datagram’s *Checksum* field is first set to 0. Then, the 16-bit one’s complement sum of the header is calculated (the entire header is considered a sequence of 16-bit words). The 16-bit one’s complement of this sum is then stored in the *Checksum* field to make the datagram ready for transmission. One’s complement addition can be implemented by “end-round-carry addition”: when a carry bit is produced using conventional (two’s complement) addition, the carry is added back in as a 1 value. Figure 5-3 presents an example, where the message contents are represented in hexadecimal.

When an IPv4 datagram is received, a checksum is computed across the whole header, including the value of the *Checksum* field itself. Assuming there are no errors, the computed checksum value is always 0 (a one’s complement of the value FFFF). Note that for any nontrivial packet or header, the value of the *Checksum* field in the packet can never be FFFF. If it were, the sum (prior to the final one’s complement operation at the sender) would have to have been 0. No sum can ever be 0 using one’s complement addition unless all the bytes are 0—something that never happens with any legitimate IPv4 header. When the header is found to be bad (the computed checksum is nonzero), the IPv4 implementation discards the received datagram. No error message is generated. It is up to the higher layers to somehow detect the missing datagram and retransmit if necessary.

Sending

Message: E3 4F 23 96 44 27 99 F3 [00 00] ← Checksum Field = 0000
 Two's Complement Sum: 1E4FF
 One's Complement Sum: E4FF+1 = E500
 One's Complement: $\sim(E500) = \sim(1110\ 0101\ 0000\ 0000) = 0001\ 1010\ 1111\ 1111 = 1AFF$ (the checksum)

Receiving

Message + Checksum = E34F + 2396 + 4427 + 99F3 + 1AFF = E500 + 1AFF = FFFF
 $\sim(\text{Message} + \text{Checksum}) = 0000$

Figure 5-3 The Internet checksum is the one's complement of a one's complement 16-bit sum of the data being checksummed (zero padding is used if the number of bytes being summed is odd). If the data being summed includes a *Checksum* field, the field is first set to 0 prior to the checksum operation and then filled in with the computed checksum. To check whether an incoming block of data that contains a *Checksum* field (header, payload, etc.) is valid, the same type of checksum is computed over the whole block (including the *Checksum* field). Because the *Checksum* field is essentially the inverse of the checksum of the rest of the data, computing the checksum on correctly received data should produce a value of 0.

5.2.2.1 Mathematics of the Internet Checksum

For the mathematically inclined, the set of 16-bit hexadecimal values $V = \{0001, \dots, \text{FFFF}\}$ and the one's complement sum operation $+$ together form an Abelian group. For the combination of a set and an operator to be a group, several properties need to be obeyed: closure, associativity, existence of an identity element, and existence of inverses. To be an Abelian (commutative) group, commutativity must also be obeyed. If we look closely, we see that all of these properties are indeed obeyed:

- For any X, Y in V , $(X + Y)$ is in V [closure]
- For any X, Y, Z in V , $X + (Y + Z) = (X + Y) + Z$ [associativity]
- For any X in V , $e + X = X + e = X$ where $e = \text{FFFF}$ [identity]
- For any X in V , there is an X' in V such that $X + X' = e$ [inverse]
- For any X, Y in V , $(X + Y) = (Y + X)$ [commutativity]

What is interesting about the set V and the group $\langle V, + \rangle$ is that we have deleted the number 0000 from consideration. If we put the number 0000 in the set V , then $\langle V, + \rangle$ is not a group any longer. To see this, we first observe that 0000 and FFFF appear to perform the role of zero (additive identity) using the $+$ operation. For example, $\text{AB12} + 0000 = \text{AB12} = \text{AB12} + \text{FFFF}$. However, in a group there can be only one identity element. If we have some element 12AB, and assume the identity

element is 0000, then we need some inverse X' so that $(12AB + X') = 0000$, but we see that no such value of X' exists in V that satisfies the criteria. Therefore, we need to exclude 0000 from consideration as the identity element in $\langle V, + \rangle$ by removing it from the set V to make this structure a true group. For an introduction to abstract algebra, the reader may wish to consult a detailed text on the subject, such as the popular book by Pinter [P90].

5.2.3 *DS Field and ECN (Formerly Called the ToS Byte or IPv6 Traffic Class)*

The third and fourth fields of the IPv4 header (second and third fields of the IPv6 header) are the *Differentiated Services* (called *DS Field*) and *ECN* fields. Differentiated Services (called *DiffServ*) is a framework and set of standards aimed at supporting differentiated classes of service (i.e., beyond just best-effort) on the Internet [RFC2474][RFC2475][RFC3260]. IP datagrams that are marked in certain ways (by having some of these bits set according to predefined patterns) may be forwarded differently (e.g., with higher priority) than other datagrams. Doing so can lead to increased or decreased queuing delay in the network and other special effects (possibly with associated special fees imposed by an ISP). A number is placed in the *DS Field* termed the *Differentiated Services Code Point* (DSCP). A “code point” refers to a particular predefined arrangement of bits with agreed-upon meaning. Typically, datagrams have a DSCP assigned to them when they are given to the network infrastructure that remains unmodified during delivery. However, policies (such as how many high-priority packets are allowed to be sent in a period of time) may cause a DSCP in a datagram to be changed during delivery.

The pair of *ECN* bits in the header is used for marking a datagram with a *congestion indicator* when passing through a router that has a significant amount of internally queued traffic. Both bits are set by persistently congested ECN-aware routers when forwarding packets. The use case envisioned for this function is that when a marked packet is received at the destination, some protocol (such as TCP) will notice that the packet is marked and indicate this fact back to the sender, which would then slow down, thereby easing congestion before a router is forced to drop traffic because of overload. This mechanism is one of several aimed at avoiding or dealing with network congestion, which we explore in more detail in Chapter 16. Although the *DS Field* and *ECN* field are not obviously closely related, the space for them was carved out of the previously defined IPv4 *Type of Service* and IPv6 *Traffic Class* fields. For this reason, they are often discussed together, and the terms “ToS byte” and “Traffic Class byte” are still in widespread use.

Although the original uses for the ToS and Traffic Class bytes are not widely supported, the structure of the *DS Field* has been arranged to provide some backward compatibility with them. To get a clear understanding of how this has been accomplished, we first review the original structure of the *Type of Service* field [RFC0791] as shown in Figure 5-4.

0	2	3	4	5	6	7
Precedence (3 bits)		D	T	R	Reserved (0)	

Figure 5-4 The original IPv4 *Type of Service* and IPv6 *Traffic Class* field structures. The *Precedence* subfield was used to indicate which packets should receive higher priority (larger values mean higher priority). The *D*, *T*, and *R* subfields refer to delay, throughput, and reliability. A value of 1 in these fields corresponds to a desire for low delay, high throughput, and high reliability, respectively.

The *D*, *T*, and *R* subfields are for indicating that the datagram should receive good treatment with respect to delay, throughput, and reliability. A value of 1 indicates better treatment (low delay, high throughput, high reliability, respectively). The precedence values range from 000 (routine) to 111 (network control) with increasing priority (see Table 5-1). They are based on a call preemption scheme called *Multilevel Precedence and Preemption* (MLPP) dating back to the U.S. Department of Defense's AUTOVON telephone system [A92], in which lower-precedence calls could be preempted by higher-precedence calls. These terms are still in use and are being incorporated into VoIP systems.

Table 5-1 The original IPv4 *Type of Service* and IPv6 *Traffic Class* precedence subfield values

Value	Precedence Name
000	Routine
001	Priority
010	Immediate
011	Flash
100	Flash Override
101	Critical
110	Internetwork Control
111	Network Control

In defining the *DS Field*, the precedence values have been taken into account [RFC2474] so as to provide a limited form of backward compatibility. Referring to Figure 5-5, the 6-bit *DS Field* holds the DSCP, providing support for 64 distinct code points. The particular value of the DSCP tells a router the forwarding treatment or special handling the datagram should receive. The various forwarding treatments are expressed as *per-hop behavior* (PHB), so the DSCP value effectively tells a router which PHB to apply to the datagram. The default value for the DSCP is generally 0, which corresponds to routine, best-effort Internet traffic. The 64 possible DSCP values are broadly divided into a set of pools for various uses, as given in [DSCPREG] and shown in Table 5-2.

classes, effectively generalizing the precedence concept. Traffic from one class is forwarded separately from other classes. Within a traffic class, a datagram is assigned a *drop precedence*. Datagrams of higher drop precedence in a class are handled preferentially (i.e., are forwarded with higher priority) over those with lower drop precedence in the same class. Combining the traffic class and drop precedence, the name *AF_{ij}* corresponds to assured forwarding class *i* with drop precedence *j*. For example, a datagram marked with AF32 is in traffic class 3 with drop precedence 2.

Table 5-3 The *DS Field* values are designed to be somewhat compatible with the IP *Precedence* subfield specified for the *Type of Service* and IPv6 *Traffic Class* field. AF and EF provide enhanced services beyond simple best-effort.

Name	Value	Reference	Description
CS0	000000	[RFC2474]	Class selector (best-effort/routine)
CS1	001000	[RFC2474]	Class selector (priority)
CS2	010000	[RFC2474]	Class selector (immediate)
CS3	011000	[RFC2474]	Class selector (flash)
CS4	100000	[RFC2474]	Class selector (flash override)
CS5	101000	[RFC2474]	Class selector (CRITIC/ECP)
CS6	110000	[RFC2474]	Class selector (internetwork control)
CS7	111000	[RFC2474]	Class selector (control)
AF11	001010	[RFC2597]	Assured Forwarding (class 1, dp 1)
AF12	001100	[RFC2597]	Assured Forwarding (1,2)
AF13	001110	[RFC2597]	Assured Forwarding (1,3)
AF21	010010	[RFC2597]	Assured Forwarding (2,1)
AF22	010100	[RFC2597]	Assured Forwarding (2,2)
AF23	010110	[RFC2597]	Assured Forwarding (2,3)
AF31	011010	[RFC2597]	Assured Forwarding (3,1)
AF32	011100	[RFC2597]	Assured Forwarding (3,2)
AF33	011110	[RFC2597]	Assured Forwarding (3,3)
AF41	100010	[RFC2597]	Assured Forwarding (4,1)
AF42	100100	[RFC2597]	Assured Forwarding (4,2)
AF43	100110	[RFC2597]	Assured Forwarding (4,3)
EF PHB	101110	[RFC3246]	Expedited Forwarding
VOICE-ADMIT	101100	[RFC5865]	Capacity-Admitted Traffic

The *Expedited Forwarding* (EF) service provides the appearance of an uncongested network—that is, EF traffic should receive relatively low delay, jitter, and loss. Intuitively, this requires the rate of EF traffic going out of a router to be at least as large as the rate coming in. Consequently, EF traffic will only ever have to wait in a router queue behind other EF traffic.

Delivering differentiated services in the Internet has been an ongoing effort for over a decade. Although much of the standardization effort in terms of mechanisms took place in the late 1990s, only in the twenty-first century are some of its capabilities being realized and implemented. Some guidance on how to configure systems to take advantage of these capabilities is given in [RFC4594]. The complexity of differentiated services is due, in part, to the linkage between differentiated services and the presumed differentiated pricing structure and consequent issues of fairness that would go along with it. Such economic relationships can be complex and are outside the scope of the present discussion. For more information on this and related topics, please see [MB97] and [W03].

5.2.4 IP Options

IP supports a number of options that may be selected on a per-datagram basis. Most of these options were introduced in [RFC0791] at the time IPv4 was being designed, when the Internet was considerably smaller and when threats from malicious users were less of a concern. As a consequence, many of the options are no longer practical or desirable because of the limited size of the IPv4 header or concerns regarding security. With IPv6, most of the options have been removed or altered and are not an integral part of the basic IPv6 header. Instead, they are placed after the IPv6 header in one or more extension headers. An IP router that receives a datagram containing options is usually supposed to perform special processing on the datagram. In some cases IPv6 routers process extension headers, but many headers are designed to be processed only by end hosts. In some routers, datagrams with options or extensions are not forwarded as fast as ordinary datagrams. We briefly discuss the IPv4 options as background and then look at how IPv6 implements extension headers and options. Table 5-4 shows most of the IPv4 options that have been standardized over the years.

Table 5-4 gives the reserved IPv4 options for which descriptive RFCs can be found. The complete list is periodically updated and is available online [IPPA-RAM]. The options area always ends on a 32-bit boundary. Pad bytes with a value of 0 are added if necessary. This ensures that the IPv4 header is always a multiple of 32 bits (as required by the *IHL* field). The “Number” column in Table 5-4 is the number of the option. The “Value” column indicates the number placed inside the option *Type* field to indicate the presence of the option. These values from the two columns are not necessarily the same because the *Type* field has additional structure. In particular, the first (high-order) bit indicates whether the option should be copied into fragments if the associated datagram is fragmented. The next 2 bits indicate the option’s *class*. Currently, all options in Table 5-4 use option class 0 (control) except Timestamp and Traceroute, which are both class 2 (debugging and measurement). Classes 1 and 3 are reserved.

Most of the standardized options are rarely or never used in the Internet today. Options such as Source and Record Route, for example, require IPv4 addresses to be placed inside the IPv4 header. Because there is only limited space in the header

Table 5-4 Options, if present, are carried in IPv4 packets immediately after the basic IPv4 header. Options are identified by an 8-bit option *Type* field. This field is subdivided into three subfields: *Copy* (1 bit), *Class* (2 bits), and *Number* (5 bits). Options 0 and 1 are a single byte long, and most others are variable in length. Variable options consist of 1 byte of type identifier, 1 byte of length, and the option itself.

Name	Number	Value	Length	Description	Reference	Comments
End of List	0	0	1	Indicates no more options.	[RFC0791]	If required
No Op	1	1	1	Indicates no operation to perform (used for padding).	[RFC0791]	If required
Source Routing	3 9	131 137	Variable	Sender lists router “waypoints” for packet to traverse when forwarded. Loose means other routers can be included between waypoints (3,131). Strict means all waypoints have to be traversed exactly in order (9,137).	[RFC0791]	Rare, often filtered
Security and Handling Labels	2 5	130 133	11	Specifies how to include security labels and handling restrictions with IP datagrams in U.S. military environments.	[RFC1108]	Historic
Record Route	7	7	Variable	Records the route taken by a packet in its header.	[RFC0791]	Rare
Timestamp	4	68	Variable	Records the time of day at a packet’s source and destination.	[RFC0791]	Rare
Stream ID	8	136	4	Carries the 16-bit SATNET stream identifier.	[RFC0791]	Historic
EIP	17	145	Variable	Extended Internet Protocol (an experiment in the early 1990s)	[RFC1385]	Historic
Traceroute	18	82	Variable	Adds a route-tracing option and ICMP message (an experiment in the early 1990s).	[RFC1393]	Historic
Router Alert	20	148	4	Indicates that a router needs to interpret the contents of the datagram.	[RFC2113] [RFC5350]	Occasional
Quick-Start	25	25	8	Indicates fast transport protocol start (experimental).	[RFC4782]	Rare

(60 bytes total, of which 20 are devoted to the basic IPv4 header), these options are not very useful in today's IPv4 Internet where the number of router hops in an average Internet path is about 15 [LFS07]. In addition, the options are primarily for diagnostic purposes and make the construction of firewalls more cumbersome and risky. Thus, IPv4 options are typically disallowed or stripped at the perimeter of enterprise networks by firewalls (see Chapter 7).

Within enterprise networks, where the average path length is smaller and protection from malicious users may be less of a concern, options can still be useful. In addition, the Router Alert option represents somewhat of an exception to the problems with the other options for use on the Internet. Because it is designed primarily as a performance optimization and does not change fundamental router behavior, it is permitted more often than the other options. As suggested previously, some router implementations have a highly optimized internal pathway for forwarding IP traffic containing no options. The Router Alert option informs routers that a packet requires processing beyond the conventional forwarding algorithms. The experimental Quick-Start option at the end of the table is applicable to both IPv4 and IPv6, and we describe it in the next section when discussing IPv6 extension headers and options.

5.3 IPv6 Extension Headers

In IPv6, special functions such as those provided by options in IPv4 can be enabled by adding extension headers that follow the IPv6 header. The routing and timestamp functions from IPv4 are supported this way, as well as some other functions such as fragmentation and extra-large packets that were deemed to be rarely used for most IPv6 traffic (but still desired) and thereby did not justify allocating bits in the IPv6 header to support them. With this arrangement, the IPv6 header is fixed at 40 bytes, and extension headers are added only when needed. In choosing the IPv6 header to be of a fixed size, and requiring that extension headers be processed only by end hosts (with one exception), the designers of IPv6 have made the design and construction of high-performance routers easier because the demands on packet processing at routers can be simpler than with IPv4. In practice, packet-processing performance is governed by many factors, including the complexity of the protocol, the capabilities of the hardware and software in the router, and traffic load.

Extension headers, along with headers of higher-layer protocols such as TCP or UDP, are chained together with the IPv6 header to form a cascade of headers (see Figure 5-6). The *Next Header* field in each header indicates the type of the subsequent header, which could be an IPv6 extension header or some other type. The value of 59 indicates the end of the header chain. The possible values for the *Next Header* field are available at [IP6PARAM], and most are provided in Table 5-5.

As we can see from Table 5-5, the IPv6 extension header mechanism distinguishes some functions (e.g., routing and fragmentation) from options. The order

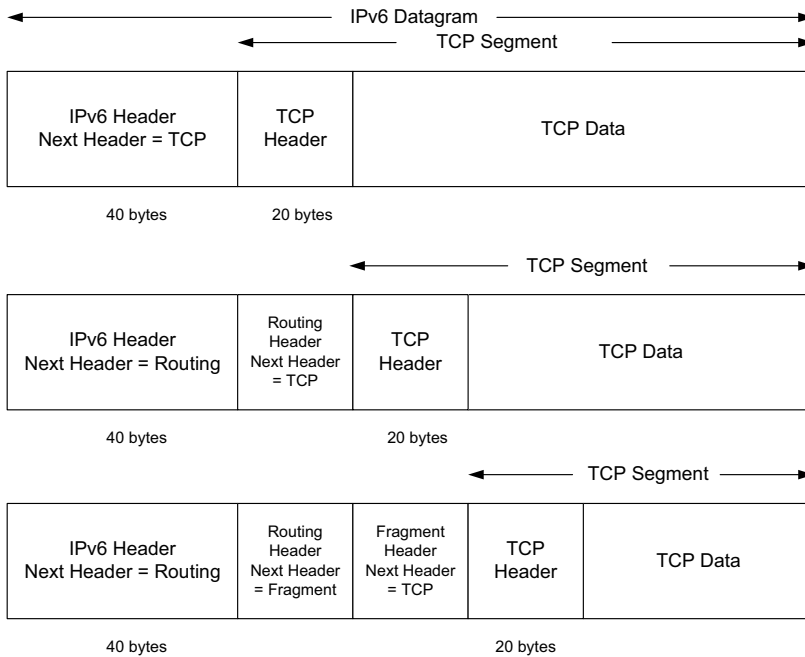


Figure 5-6 IPv6 headers form a chain using the *Next Header* field. Headers in the chain may be IPv6 extension headers or transport headers. The IPv6 header appears at the beginning of the datagram and is always 40 bytes long.

Table 5-5 The values for the IPv6 *Next Header* field may indicate extensions or headers for other protocols. The same values are used with the IPv4 *Protocol* field, where appropriate.

Header Type	Order	Value	References
IPv6 header	1	41	[RFC2460][RFC2473]
Hop-by-Hop Options (HOPOPT)	2	0	[RFC2460]; must immediately follow IPv6 header
Destination Options	3,8	60	[RFC2460]
Routing	4	43	[RFC2460][RFC5095]
Fragment	5	44	[RFC2460]
Encapsulating Security Payload (ESP)	7	50	(See Chapter 18)
Authentication (AH)	6	51	(See Chapter 18)
Mobility (MIPv6)	9	135	[RFC6275]
(None—no next header)	Last	59	[RFC2460]
ICMPv6	Last	58	(See Chapter 8)
UDP	Last	17	(See Chapter 10)
TCP	Last	6	(See Chapters 13–17)
Various other upper-layer protocols	Last	—	See [AN] for complete list

of the extension headers is given as a recommendation, except for the location of the Hop-by-Hop Options, which is mandatory, so an IPv6 implementation must be prepared to process extension headers in the order in which they are received. Only the Destination Options header can be used twice—the first time for options pertaining to the destination IPv6 address contained in the IPv6 header and the second time (position 8) for options pertaining to the final destination of the datagram. In some cases (e.g., when the Routing header is used), the *Destination IP Address* field in the IPv6 header changes as the datagram is forwarded to its ultimate destination.

5.3.1 IPv6 Options

As we have seen, IPv6 provides a more flexible and extensible way of incorporating extensions and options as compared to IPv4. Those options from IPv4 that ceased to be useful because of space limitations in the IPv4 header appear in IPv6 as variable-length extension headers or options encoded in special extension headers that can accommodate today’s much larger Internet. Options, if present, are grouped into either *Hop-by-Hop Options* (those relevant to every router along a datagram’s path) or *Destination Options* (those relevant only to the recipient). Hop-by-Hop Options (called HOPOPTs) are the only ones that need to be processed by every router a packet encounters. The format for encoding options within the Hop-by-Hop and Destination Options extension headers is common.

The Hop-by-Hop and Destination Options headers are capable of holding more than one option. Each of these options is encoded as *type-length-value* (TLV) sets, according to the format shown in Figure 5-7.

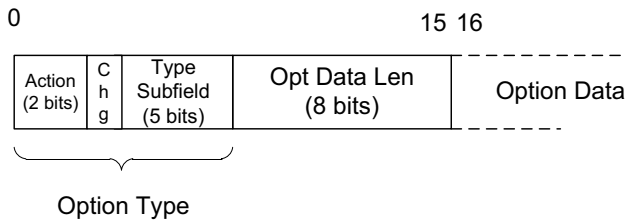


Figure 5-7 Hop-by-hop and Destination Options are encoded as TLV sets. The first byte gives the option type, including subfields indicating how an IPv6 node should behave if the option is not recognized, and whether the option data might change as the datagram is forwarded. The *Opt Data Len* field gives the size of the option data in bytes.

The TLV structure shown in Figure 5-7 includes 2 bytes followed by a variable-length number of data bytes. The first byte indicates the type of the option and includes three subfields. The first subfield gives the action to be taken by an IPv6 node attempting to process the option that does not recognize the 5-bit option *Type* subfield. Its possible values are presented in Table 5-6.

Table 5-6 The 2 high-order bits in an IPv6 TLV option type indicate whether an IPv6 node should forward or drop the datagram if the option is not recognized, and whether a message indicating the datagram's fate should be sent back to the sender.

Value	Action
00	Skip option, continue processing
01	Discard the datagram (silently)
10	Discard the datagram and send an ICMPv6 Parameter Problem message to the source address
11	Same as 10, but send the ICMPv6 message only if the offending packet's destination was not multicast

If an unknown option were included in a datagram destined for a multicast destination, a large number of nodes could conceivably generate traffic back to the source. This can be avoided by use of the 11-bit pattern for the *Action* subfield. The flexibility of the *Action* subfield is useful in the development of new options. A newly specified option can be carried in datagrams and simply ignored by those routers that do not understand it, helping to promote incremental deployment of new options. The *Change* bit field (*Chg* in Figure 5-7) is set to 1 when the option data may be modified as the datagram is forwarded. The options shown in Table 5-7 have been defined for IPv6.

Table 5-7 Options in IPv6 are carried in either Hop-by-Hop (H) or Destination (D) Options extension headers. The option *Type* field contains the value from the "Type" column with the *Action* and *Change* subfields denoted in binary. The "Length" column contains the value of the *Opt Data Len* byte from Figure 5-7. The Pad1 option is the only one lacking this byte.

Option Name	Header	Action	Change	Type	Length	References
Pad1	HD	00	0	0	N/A	[RFC2460]
PadN	HD	00	0	1	var	[RFC2460]
Jumbo Payload	H	11	0	194	4	[RFC2675]
Tunnel Encapsulation Limit	D	00	0	4	4	[RFC2473]
Router Alert	H	00	0	5	4	[RFC2711]
Quick-Start	H	00	1	6	8	[RFC4782]
CALIPSO	H	00	0	7	8+	[RFC5570]
Home Address	D	11	0	201	16	[RFC6275]

5.3.1.1 Pad1 and PadN

IPv6 options are aligned to 8-byte offsets, so options that are naturally smaller are padded with 0 bytes to round out their lengths to the nearest 8 bytes. Two padding options are available to support this, called Pad1 and PadN. The Pad1 option (type 0) is the only option that lacks *Length* and *Value* fields. It is simply 1 byte long and

contains the value 0. The PadN option (type 1) inserts 2 or more bytes of padding into the options area of the header using the format of Figure 5-7. For n bytes of padding, the *Opt Data Len* field contains the value $(n - 2)$.

5.3.1.2 IPv6 Jumbo Payload

In some TCP/IP networks, such as those used to interconnect supercomputers, the normal 64KB limit on the IP datagram size can lead to unwanted overhead when moving large amounts of data. The IPv6 Jumbo Payload option specifies an IPv6 datagram with payload size larger than 65,535 bytes, called a *jumbogram*. This option need not be implemented by nodes attached to links with MTU sizes below 64KB. The Jumbo Payload option provides a 32-bit field for holding the payload size for datagrams with payloads of sizes between 65,535 and 4,294,967,295 bytes.

When a jumbogram is formed for transmission, its normal *Payload Length* field is set to 0. As we shall see later, the TCP protocol makes use of the *Payload Length* field in order to compute its checksum using the Internet checksum algorithm described previously. When the Jumbo Payload option is used, TCP must be careful to use the length value from the option instead of the regular *Length* field in the base header. Although this procedure is not difficult, larger payloads can lead to an increased chance of undetected error [RFC2675].

5.3.1.3 Tunnel Encapsulation Limit

Tunneling refers to the encapsulation of one protocol in another that does not conform to traditional layering (see Chapters 1 and 3). For example, IP datagrams may be encapsulated inside the payload portion of another IP datagram. Tunneling can be used to form virtual overlay networks, in which one network (e.g., the Internet) acts as a well-connected link layer for another layer of IP [TWEF03]. Tunnels can be nested in the sense that datagrams that are in a tunnel may themselves be placed in a tunnel, in a recursive fashion.

When sending an IP datagram, a sender does not ordinarily have much control over how many tunnel levels are ultimately used for encapsulation. Using this option, however, a sender can specify this limit. A router intending to encapsulate an IPv6 datagram into a tunnel first checks for the presence and value of the Tunnel Encapsulation Limit option. If the limit value is 0, the datagram is discarded and an ICMPv6 Parameter Problem message (see Chapter 8) is sent to the source of the datagram (i.e., the previous tunnel entry point). If the limit is nonzero, the tunnel encapsulation is permitted, but the newly formed (encapsulating) IPv6 datagram must include a Tunnel Encapsulation Limit option whose value is 1 less than the option value in the arriving datagram. In effect, the encapsulation limit acts like the IPv4 *TTL* or IPv6 *Hop Limit* field, but for levels of tunnel encapsulation instead of forwarding hops.

5.3.1.4 Router Alert

The Router Alert option indicates that the datagram contains information that needs to be processed by a router. It is used for the same purpose as the IPv4 Router Alert option. [RTAOPTS] gives the current set of values for the option.

5.3.1.5 Quick-Start

The Quick-Start (QS) option is used in conjunction with the experimental Quick-Start procedure for TCP/IP specified in [RFC4782]. It is applicable to both IPv4 and IPv6 but at present is suggested only for private networks and not the global Internet. The option includes a value encoding the sender's desired transmission rate in bits per second, a QS TTL value, and some additional information. Routers along the path may agree that supporting the desired rate is acceptable, in which case they decrement the QS TTL and leave the rate request unchanged when forwarding the containing datagram. When they disagree (i.e., wish to support a lower rate), they can reduce the number to an acceptable rate. Routers that do not recognize the QS option do not decrement the QS TTL. A receiver provides feedback to the sender, including the difference between the received datagram's IPv4 *TTL* or IPv6 *Hop Limit* field and its QS TTL, along with the resulting rate that may have been adjusted by the routers along the forward path. This information is used by the sender to determine its sending rate (which, for example, may exceed the rate TCP it would otherwise use). Comparison of the TTL values is used to ensure that every router along the path participates in the QS negotiation; if any routers are found to be decrementing the IPv4 *TTL* (or IPv6 *Hop Limit*) field and not modifying the QS TTL value, QS is not enabled.

5.3.1.6 CALIPSO

This option is used for supporting the *Common Architecture Label IPv6 Security Option* (CALIPSO) [RFC5570] in certain private networks. It provides a method to label datagrams with a security-level indicator, along with some additional information. In particular, it is intended for use in multilevel secure networking environments (e.g., government, military, and banking) where the security level of all data must be indicated by some form of label.

5.3.1.7 Home Address

This option holds the "home" address of the IPv6 node sending the datagram when IPv6 mobility options are in use. Mobile IP (see Section 5.5) specifies a set of procedures for handling IP nodes that may change their point of network attachment without losing their higher-layer network connections. It has a concept of a node's "home," which is derived from the address prefix of its typical location. When roaming away from home, the node is generally assigned a different IP address. This option allows the node to provide its normal home address in addition to its (presumably temporarily assigned) new address while traveling. The home address can be used by other IPv6 nodes when communicating with the mobile node. If the Home Address option is present, the Destination Options header containing it must appear after a Routing header and before the Fragment, Authentication, and ESP headers (see Chapter 18), if any of them is also present. We discuss this option in more detail in the context of Mobile IP.

5.3.2 Routing Header

The IPv6 Routing header provides a mechanism for the sender of an IPv6 datagram to control, at least in part, the path the datagram takes through the network. At present, two different versions of the routing extension header have been specified, called type 0 (RH0) and type 2 (RH2), respectively. RH0 has been deprecated because of security concerns [RFC5095], and RH2 is defined in conjunction with Mobile IP. To best understand the Routing header, we begin by discussing RH0 and then investigate why it has been deprecated and how it differs from RH2. RH0 specifies one or more IPv6 nodes to be “visited” as the datagram is forwarded. The header is shown in Figure 5-8.

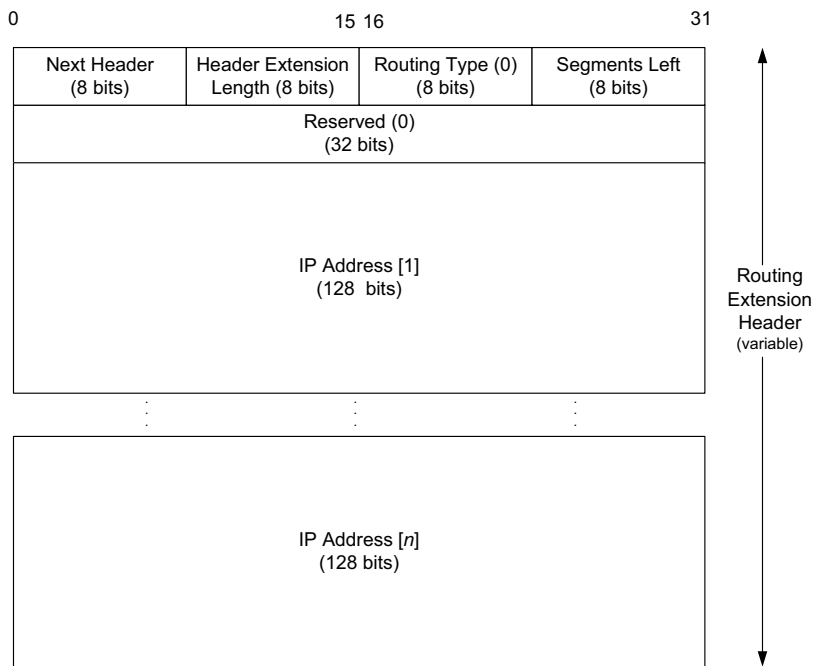


Figure 5-8 The now-deprecated Routing header type 0 (RH0) generalizes the IPv4 loose and strict Source Route and Record Route options. It is constructed by the sender to include IPv6 node addresses that act as waypoints when the datagram is forwarded. Each address can be specified as a loose or strict address. A strict address must be reached by a single IPv6 hop, whereas a loose address may contain one or more other hops in between. The IPv6 *Destination IP Address* field in the base header is modified to contain the next waypoint address as the datagram is forwarded.

The IPv6 Routing header shown in Figure 5-8 generalizes the loose Source and Record Route options from IPv4. It also supports the possibility of routing on identifiers other than IPv6 addresses, although this feature is not standardized

and is not discussed further here. For standardized routing on IPv6 addresses, RH0 allows the sender to specify a vector of IPv6 addresses for nodes to be visited.

The header contains an 8-bit *Routing Type* identifier and an 8-bit *Segments Left* field. The type identifier for IPv6 addresses is 0 for RH0 and 2 for RH2. The *Segments Left* field indicates how many route segments remain to be processed—that is, the number of explicitly listed intermediate nodes still to be visited before reaching the final destination. The block of addresses starts with a 32-bit reserved field set by the sender to 0 and ignored by receivers. The addresses are nonmulticast IPv6 addresses to be visited as the datagram is forwarded.

A Routing header is not processed until it reaches the node whose address is contained in the *Destination IP Address* field of the IPv6 header. At this time, the *Segments Left* field is used to determine the next hop address from the address vector, and this address is swapped with the *Destination IP Address* field in the IPv6 header. Thus, as the datagram is forwarded, the *Segments Left* field grows smaller, and the list of addresses in the header reflects the node addresses that forwarded the datagram. The forwarding procedure is better understood with an example (see Figure 5-9).

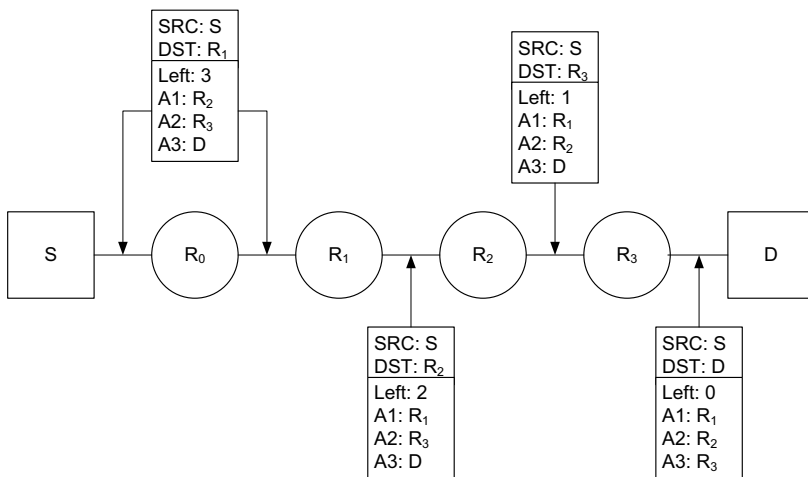


Figure 5-9 Using an IPv6 Routing header (RH0), the sender (S) is able to direct the datagram through the intermediate nodes R₂ and R₃. The other nodes traversed are determined by the normal IPv6 routing. Note that the destination address in the IPv6 header is updated at each hop specified in the Routing header.

In Figure 5-9 we can see how the Routing header is processed by intermediate nodes. The sender (S) constructs the datagram with destination address R₁ and a Routing header (type 0) containing the addresses R₂, R₃, and D. The final destination of the datagram is the last address in the list (D). The *Segments Left* field (labeled “Left” in Figure 5-9) starts at 3. The datagram is forwarded toward R₁ automatically by S and R₀. Because R₀’s address is not present in the datagram,

no modifications of the Routing header or addresses are performed by R_0 . Upon reaching R_r , the destination address from the base header is swapped with the first address listed in the Routing header and the *Segments Left* field is decremented.

As the datagram is forwarded, the process of swapping the destination address with the next address from the address list in the Routing header repeats until the last destination listed in the Routing header is reached.

We can arrange to include a Routing header with a simple command-line option to the `ping6` command in Windows XP (Windows Vista and later include only the `ping` command, which incorporates IPv6 support):

```
C:\> ping6 -r -s 2001:db8::100 2001:db8::1
```

This command arranges to use the source address `2001:db8::100` when sending a ping request to `2001:db8::1`. The `-r` option arranges for a Routing header (RH0) to be included. We can see the outgoing request using Wireshark (see Figure 5-10).

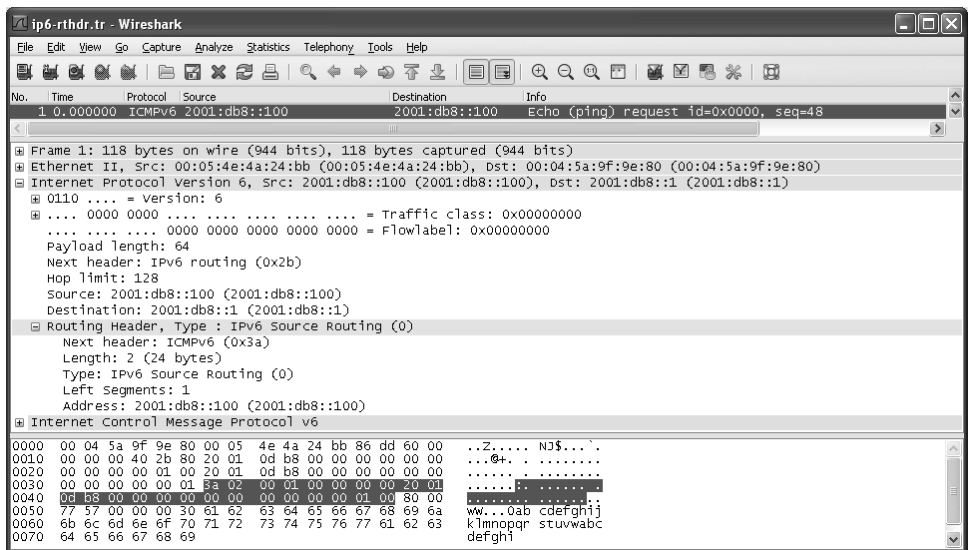


Figure 5-10 The ping request appears as an ICMPv6 Echo Request in Wireshark. The IPv6 header includes a *Next Header* field indicating that the packet contains a type 0 Routing header, followed by an ICMPv6 header. The number of segments in the RH0 left to be processed is one (2001:db8::100).

The ping message appears as an ICMPv6 Echo Request packet (see Chapter 8). By following the *Next Header* field values, we can see that the base header is followed by a Routing header. In the Routing header, we can see that the type is 0 (indicating an RH0), and there is one segment (hop) left to process. The hop is specified by the first slot in the address list (number 0): `2001:db8::100`.

As mentioned previously, RH0 has been deprecated by [RFC5095] because of a security concern that allows RH0 to be used to increase the effectiveness of DoS attacks. The problem is that RH0 allows the same address to be specified in multiple locations within the Routing header. This can lead to traffic being forwarded many times between two or more hosts or routers along a particular path. The potentially high traffic loads that can be created along particular paths in the network can cause disruption to other traffic flows competing for bandwidth across the same path. Consequently, RH0 has been deprecated and only RH2 remains as the sole Routing header supported by IPv6. RH2 is equivalent to RH0 except it has room for only a single address and uses a different value in the *Routing Type* field.

5.3.3 Fragment Header

The Fragment header is used by an IPv6 source when sending a datagram larger than the path MTU of the datagram's intended destination. Path MTU and how it is determined are discussed in more detail in Chapter 13, but 1280 bytes is a network-wide link-layer minimum MTU for IPv6 (see section 5 of [RFC2460]). In IPv4, any host or router can fragment a datagram if it is too large for the MTU on the next hop, and fields within the second 32-bit word of the IPv4 header indicate the fragmentation information. In IPv6, only the sender of the datagram is permitted to perform fragmentation, and in such cases a Fragment header is added.

The Fragment header includes the same information as is found in the IPv4 header, but the *Identification* field is 32 bits instead of the 16 that are used for IPv4. The larger field provides the ability for more fragmented packets to be outstanding in the network simultaneously. The Fragment header uses the format shown in Figure 5-11.

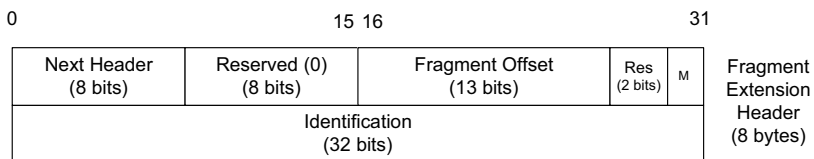


Figure 5-11 The IPv6 Fragment header contains a 32-bit *Identification* field (twice as large as the *Identification* field in IPv4). The *M* bit field indicates whether the fragment is the last of an original datagram. As with IPv4, the *Fragment Offset* field gives the offset of the payload into the original datagram in 8-byte units.

Referring to Figure 5-11, the *Reserved* field and 2-bit *Res* field are both zero and ignored by receivers. The *Fragment Offset* field indicates where the data that follows the Fragment header is located, as a positive offset in 8-byte units, relative to the “fragmentable part” (see the next paragraph) of the original IPv6 datagram. The *M* bit field, if set to 1, indicates that more fragments are contained in the datagram. A value of 0 indicates that the fragment contains the last bytes of the original datagram.

The datagram serving as input to the fragmentation process is called the “original packet” and consists of two parts: the “unfragmentable part” and the “fragmentable part.” The unfragmentable part includes the IPv6 header and any included extension headers required to be processed by intermediate nodes to the destination (i.e., all headers up to and including the Routing header, otherwise the Hop-by-Hop Options extension header if only it is present). The fragmentable part constitutes the remainder of the datagram (i.e., Destination Options header, upper-layer headers, and payload data).

When the original packet is fragmented, multiple fragment packets are produced, each of which contains a copy of the unfragmentable part of the original packet, but for which each IPv6 header has the *Payload Length* field altered to reflect the size of the fragment packet it describes. Following the unfragmentable part, each new fragment packet contains a Fragment header with an appropriately assigned *Fragment Offset* field (e.g., the first fragment contains offset 0) and a copy of the original packet’s *Identification* field. The last fragment has its *M (More Fragments)* bit field set to 0.

The following example illustrates the way an IPv6 source might fragment a datagram. In the example shown in Figure 5-12, a payload of 3960 bytes is fragmented such that no fragment’s total packet size exceeds 1500 bytes (a typical MTU for Ethernet), yet the fragment data sizes still are arranged to be multiples of 8 bytes.

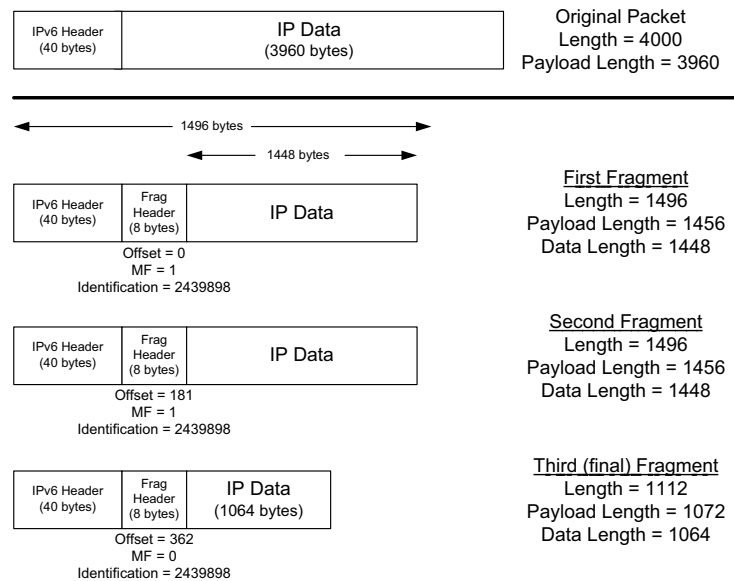


Figure 5-12 An example of IPv6 fragmentation where a 3960-byte payload is split into three fragment packets of size 1448 bytes or less. Each fragment contains a Fragment header with the identical *Identification* field. All but the last fragment have the *More Fragments* field (*M*) set to 1. The offset is given in 8-byte units—the last fragment, for example, contains data beginning at offset $(362 * 8) = 2896$ bytes from the beginning of the original packet’s data. The scheme is similar to fragmentation in IPv4.

In Figure 5-12 we see how the larger original packet has been fragmented into three smaller packets, each containing a Fragment header. The IPv6 header's *Payload Length* field is modified to reflect the size of the data and newly formed Fragment header. The Fragment header in each fragment contains a common *Identification* field, and the sender ensures that no distinct original packets are assigned the same field value within the expected lifetime of a datagram on the network.

The *Offset* field in the Fragment header is given in 8-byte units, so fragmentation is performed at 8-byte boundaries, which is why the first and second fragments contain 1448 data bytes instead of 1452. Thus, all but the last fragment (possibly) is a multiple of 8 bytes. The receiver must ensure that all fragments of an original datagram have been received before performing *reassembly*. The reassembly procedure aggregates the fragments, forming the original datagram. As with fragmentation in IPv4 (see Chapter 10), fragments may arrive out of order at the receiver but are reassembled in order to form a datagram that is given to other protocols for processing.

We can see the construction of an IPv6 fragment using this command on Windows 7:

```
C:\> ping -l 3952 ff01::2
```

Figure 5-13 shows the Wireshark output of the activity on the network as it runs.

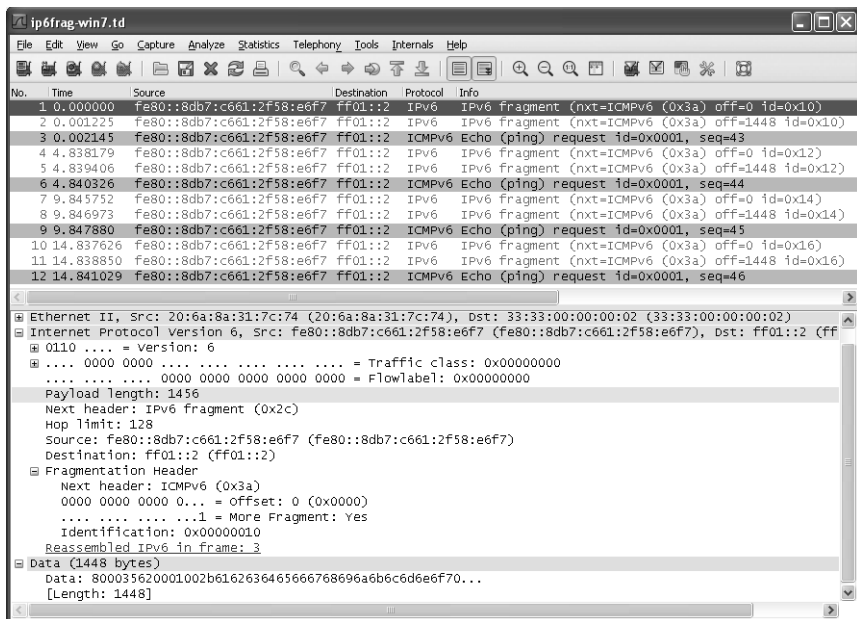


Figure 5-13 The ping program generates ICMPv6 packets (see Chapter 8) containing 3960 IPv6 payload bytes in this example. These packets are fragmented to produce three packet fragments, each of which is small enough to fit in the Ethernet MTU size of 1500 bytes.

In Figure 5-13 we see the fragments constituting four ICMPv6 Echo Request messages sent to the IPv6 multicast address `ff01::2`. Each request requires fragmentation because the `-l 3952` option indicates that 3952 data bytes are to be carried in the data area of each ICMPv6 message (leading to an IPv6 payload length of 3960 bytes due to the 8-byte ICMPv6 header). The IPv6 source address is link-local. To determine the target's link-layer multicast address, a mapping procedure specific to IPv6 is performed, described in Chapter 9. The ICMPv6 Echo Request (generated by the `ping` program) spans several fragments, which Wireshark reassembles to display once it has processed all the constituent fragments. Figure 5-14 shows the second fragment in more detail.

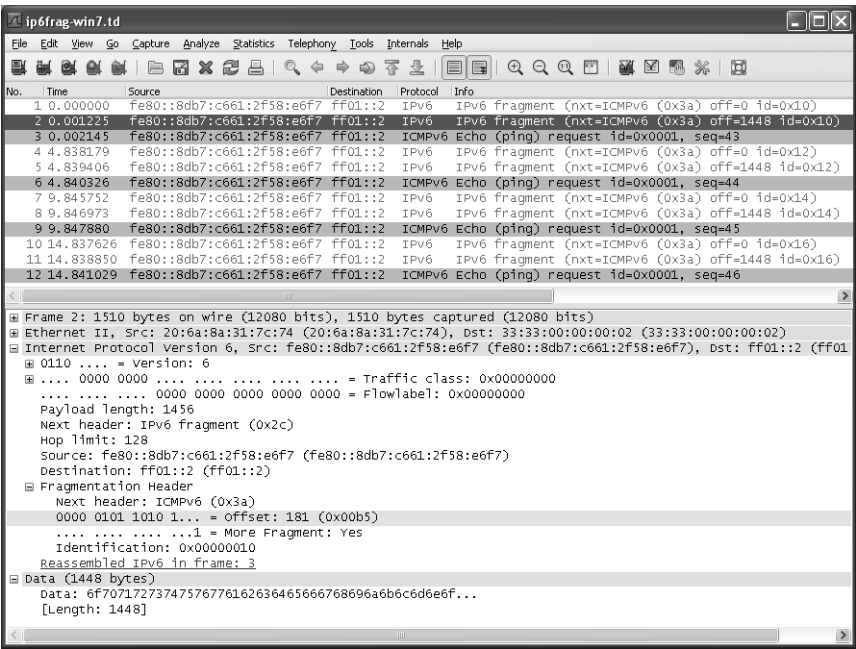


Figure 5-14 The second fragment of an ICMPv6 Echo Request contains 1448 IPv6 payload bytes including the 8-byte Fragment header. The presence of the Fragment header indicates that the overall datagram was fragmented at the source, and the *Offset* field of 181 indicates that this fragment contains data starting at byte offset 1448. The *More Fragments* bit field being set indicates that other fragments are needed to reassemble the datagram. All fragments from the same original datagram contain the same *Identification* field (2 in this case).

In Figure 5-14 we see the IPv6 header, with payload length 1448 bytes, as expected. The *Next Header* field contains the value 44 (0x2c) we saw in Table 5-5, indicating that a Fragment header follows the IPv6 header. The Fragment header indicates that the following header is for ICMPv6, meaning there are no more

extension headers. Also, the *Offset* field is 181, meaning this fragment contains data at byte offset 1448 in the original datagram. We know it is not the last fragment because the *More Fragments* field is set (displayed as Yes by Wireshark). Figure 5-15 shows the final fragment of the initial ICMPv6 Echo Request datagram.

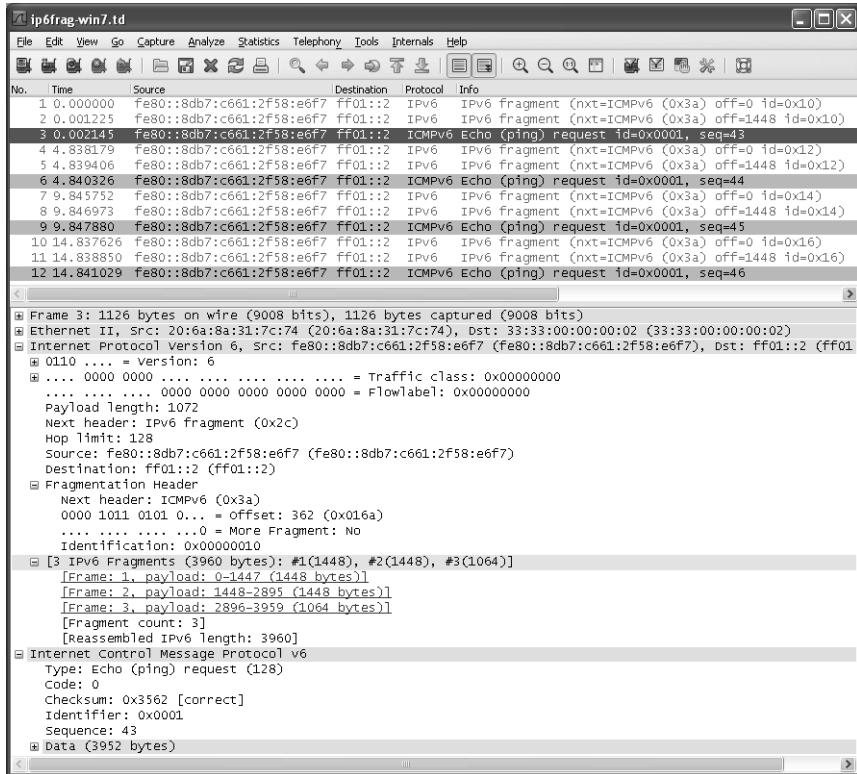


Figure 5-15 The last fragment of the first ICMPv6 Echo Request datagram has an offset of $362 * 8 = 2896$ and payload length of 1072 bytes (1064 bytes of the original datagram's payload plus 8 bytes of Fragment header). The *More Fragments* bit field being set to 0 indicates that this is the last fragment, and the original datagram's total payload length is $2896 + 1064 = 3960$ bytes (3956 bytes of ICMP data plus 8 bytes for the ICMPv6 header; see Chapter 8).

In Figure 5-15 we see that the *Offset* field has the value 362, but this is in 8-byte units, meaning that the byte offset relative to the original datagram is $362 * 8 = 2896$. The *Total Length* field has the value 1072, which includes 8 bytes for the Fragment header. Wireshark computes the fragmentation pattern for us, indicating that the first and second fragments contained the first and second sets of 1448 bytes, and the final fragment contained 1064. All in all, the fragmentation process added $40 * 2 + 8 * 3 = 104$ bytes to be carried by the network layer (two additional

IPv6 headers plus an 8-byte Fragment header for each fragment). If we add link-layer overhead, the total comes to $104 + (2 \times 18) = 140$ bytes. (Each new Ethernet frame includes a 14-byte header and a 4-byte CRC.)

5.4 IP Forwarding

Conceptually, IP forwarding is simple, especially for a host. If the destination is directly connected to the host (e.g., a point-to-point link) or on a shared network (e.g., Ethernet), the IP datagram is sent directly to the destination—a router is not required or used. Otherwise, the host sends the datagram to a single router (called the *default* router) and lets the router deliver the datagram to its destination. This simple scheme handles most host configurations.

In this section we investigate the details of this simple situation and also how IP forwarding works when the situation is not as simple. We begin by noting that most hosts today can be configured to be routers as well as hosts, and many home networks use an Internet-connected PC to act as a router (and also a firewall, as we discuss in Chapter 7). What differentiates a host from a router to IP is how IP datagrams are handled: a host never forwards datagrams it does not originate, whereas routers do.

In our general scheme, the IP protocol can receive a datagram either from another protocol on the same machine (TCP, UDP, etc.) or from a network interface. The IP layer has some information in memory, usually called a *routing table* or *forwarding table*, which it searches each time it receives a datagram to send. When a datagram is received from a network interface, IP first checks if the destination IP address is one of its own IP addresses (i.e., one of the IP addresses associated with one of its network interfaces) or some other address for which it should receive traffic such as an IP broadcast or multicast address. If so, the datagram is delivered to the protocol module specified by the *Protocol* field in the IPv4 header or *Next Header* field in the IPv6 header. If the datagram is not destined for one of the IP addresses being used locally by the IP module, then (1) if the IP layer was configured to act as a router, the datagram is forwarded (that is, handled as an outgoing datagram as described in Section 5.4.2); or (2) the datagram is silently discarded. Under some circumstances (e.g., no route is known in case 1), an ICMP message may be sent back to the source indicating an error condition.

5.4.1 Forwarding Table

The IP protocol standards do not dictate the precise data required to be in a forwarding table, as this choice is left up to the implementer of the IP protocol. Nevertheless, several key pieces of information are generally required to implement the forwarding table for IP, and we shall discuss these now. Each entry in the routing or forwarding table contains the following information fields, at least conceptually:

- **Destination:** This contains a 32-bit field (or 128-bit field for IPv6) used for matching the result of a masking operation (see the next bulleted item). The destination can be as simple as zero, for a “default route” covering all destinations, or as long as the full length of an IP address, in the case of a “host route” that describes only a single destination.
- **Mask:** This contains a 32-bit field (128-bit field for IPv6) applied as a bitwise AND mask to the destination IP address of a datagram being looked up in the forwarding table. The masked result is compared with the set of destinations in the forwarding table entries.
- **Next-hop:** This contains the 32-bit IPv4 address or 128-bit IPv6 address of the next IP entity (router or host) to which the datagram should be sent. The next-hop entity is typically on a network shared with the system performing the forwarding lookup, meaning the two share the same network prefix (see Chapter 2).
- **Interface:** This contains an identifier used by the IP layer to reference the network interface that should be used to send the datagram to its next hop. For example, it could refer to a host’s 802.11 wireless interface, a wired Ethernet interface, or a PPP interface associated with a serial port. If the forwarding system is also the sender of the IP datagram, this field is used in selecting which source IP address to use on the outgoing datagram (see Section 5.6.2.1).

IP forwarding is performed on a *hop-by-hop* basis. As we can see from this forwarding table information, the routers and hosts do not contain the complete forwarding path to any destination (except, of course, those destinations that are directly connected to the host or router). IP forwarding provides the IP address of only the next-hop entity to which the datagram is sent. It is assumed that the next hop is really “closer” to the destination than the forwarding system is, and that the next-hop router is directly connected to (i.e., shares a common network prefix with) the forwarding system. It is also generally assumed that no “loops” are constructed between the next hops so that a datagram does not circulate around the network until its TTL or hop limit expires. The job of ensuring correctness of the routing table is given to one or more routing protocols. Many different *routing protocols* are available to do this job, including RIP, OSPF, BGP, and IS-IS, to name a few (see, for example, [DC05] for more detail on routing protocols).

5.4.2 IP Forwarding Actions

When the IP layer in a host or router needs to send an IP datagram to a next-hop router or host, it first examines the destination IP address (D) in the datagram. Using the value D , the following *longest prefix match* algorithm is executed on the forwarding table:

1. Search the table for all entries for which the following property holds: $(D \wedge m_j) = d_j$, where m_j is the value of the mask field associated with the forwarding entry e_j having index j , and d_j is the value of the destination field associated with e_j . This means that the destination IP address D is bitwise ANDed with the mask in each forwarding table entry (m_j), and the result is compared against the destination in the same forwarding table entry (d_j). If the property holds, the entry (e_j here) is a “match” for the destination IP address. When a match happens, the algorithm notes the entry index (j here) and how many bits in the mask m_j were set to 1. The more bits set to 1, the “better” the match.
2. The best matching entry e_k (i.e., the one with the largest number of 1 bits in its mask m_k) is selected, and its next-hop field n_k is used as the next-hop IP address in forwarding the datagram.

If no matches in the forwarding table are found, the datagram is undeliverable. If the undeliverable datagram was generated locally (on this host), a “host unreachable” error is normally returned to the application that generated the datagram. On a router, an ICMP message is normally sent back to the host that sent the datagram.

In some circumstances, more than one entry may match an equal number of 1 bits. This can happen, for example, when more than one default route is available (e.g., when attached to more than one ISP, called *multihoming*). The end-system behavior in such cases is not set by standards and is instead specific to the operating system’s protocol implementation. A common behavior is for the system to simply choose the first match. More sophisticated systems may attempt to *load-balance* or split traffic across the multiple routes. Studies suggest that multihoming can be beneficial not only for large enterprises, but also for residential users [THL06].

5.4.3 Examples

To get a solid understanding of how IP forwarding works both in the simple local environment (e.g., same LAN) and in the somewhat more complicated multihop (global Internet) environment, we look at two cases. The first case, where all systems are using the same network prefix, is called *direct delivery*, and the other case is called *indirect delivery* (see Figure 5-16).

5.4.3.1 Direct Delivery

First consider a simple example. Our Windows XP host (with IPv4 address S and MAC address $\underline{S}$), which we will just call S , has an IP datagram to send to our Linux host (IPv4 address D , MAC address $\underline{D}$), which we will call D . These systems are interconnected using a switch. Both hosts are on the same Ethernet (see inside front cover). Figure 5-16 (top) shows the delivery of the datagram. When the IP layer in S receives a datagram to send from one of the upper layers such as TCP or UDP, it searches its forwarding table. We would expect the forwarding table on S to contain the information shown in Table 5-8.

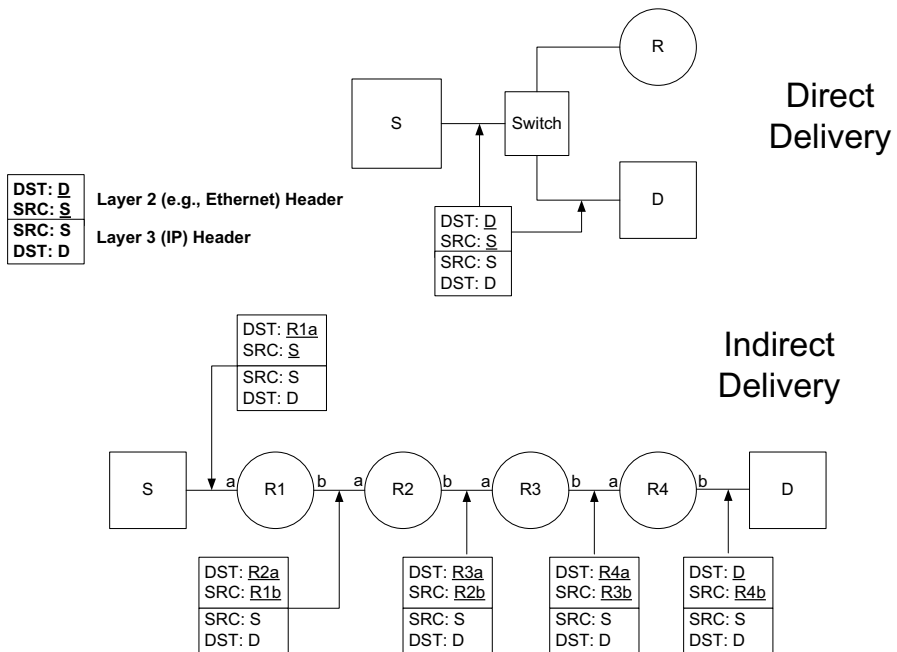


Figure 5-16 Direct delivery does not require the presence of a router—IP datagrams are encapsulated in a link-layer frame that directly identifies the source and destination. Indirect delivery involves a router—data is forwarded to the router using the router’s link-layer address as the destination link-layer address. The router’s IP address does not appear in the IP datagram (unless the router itself is the source or destination, or when source routing is used).

In Table 5-8, the destination IPv4 address D (10.0.0.9) matches both the first and second forwarding table entries. Because it matches the second entry better (25 bits instead of none), the “gateway” or next-hop address is 10.0.0.100, the address S. Thus, the gateway portion of the entry contains the address of the sending host’s own network interface (no router is referenced), indicating that direct delivery is to be used to send the datagram.

Table 5-8 The (unicast) IPv4 forwarding table at host S contains only two entries. Host S is configured with IPv4 address and subnet mask 10.0.0.100/25. Datagrams destined for addresses in the range 10.0.0.1 through 10.0.0.126 use the second forwarding table entry and are sent using direct delivery. All other datagrams use the first entry and are given to router R with IPv4 address 10.0.0.1.

Destination	Mask	Gateway (Next Hop)	Interface
0.0.0.0	0.0.0.0	10.0.0.1	10.0.0.100
10.0.0.0	255.255.255.128	10.0.0.100	10.0.0.100

The datagram is encapsulated in a lower-layer frame destined for the target host D. If the lower-layer address of the target host is unknown, the ARP protocol (for IPv4; see Chapter 4) or Neighbor Solicitation (for IPv6; see Chapter 8) operation may be invoked at this point to determine the correct lower-layer address, D. Once known, the destination address in the datagram is D's IPv4 address (10.0.0.9), and D is placed in the *Destination IP Address* field in the lower-layer header. The switch delivers the frame to D based solely on the link-layer address D; it pays no attention to the IP addresses.

5.4.3.2 Indirect Delivery

Now consider another example. Our Windows host has an IP datagram to send to the host ftp.uu.net, whose IPv4 address is 192.48.96.9. Figure 5-16 (bottom) shows the conceptual path of the datagram through four routers. First, the Windows machine searches its forwarding table but does not find a matching prefix on the local network. It uses its default route entry (which matches every destination, but with no 1 bits at all). The default entry indicates that the appropriate next-hop gateway is 10.0.0.1 (the “a side” of the router R1). This is a typical scenario for a home network.

Recall that in the direct delivery case, the source and destination IP addresses correspond to those associated with the source and destination hosts. The same is true for the lower-layer (e.g., Ethernet) addresses. In indirect delivery, the IP addresses correspond to the source and destination hosts as before, but the lower-layer addresses do not. Instead, the lower-layer addresses determine which machines receive the frame containing the datagram on a per-hop basis. In this example, the lower-layer address needed is the Ethernet address of the next-hop router R1's a-side interface, the lower-layer address corresponding to IPv4 address 10.0.0.1. This is accomplished by ARP (or a Neighbor Solicitation request if this example were using IPv6) on the network interconnecting S and R1. Once R1 responds with its a-side lower-layer address, S sends the datagram to R1. Delivery from S to R1 takes place based on processing only the lower-layer headers (more specifically, the lower-layer destination address). Upon receipt of the datagram, R1 checks its forwarding table. The information in Table 5-9 would be typical.

Table 5-9 The forwarding table at R1 indicates that address translation should be performed for traffic. The router has a private address on one side (10.0.0.1) and a public address on the other (70.231.132.85). Address translation is used to make datagrams originating on the 10.0.0.0/25 network appear to the Internet as though they had been sent from 70.231.132.85.

Destination	Mask	Gateway (Next Hop)	Interface	Note
0.0.0.0	0.0.0.0	70.231.159.254	70.231.132.85	NAT
10.0.0.0	255.255.255.128	10.0.0.100	10.0.0.1	NAT

When R1 receives the datagram, it realizes that the datagram's destination IP address is not one of its own, so it forwards the datagram. Its forwarding table is searched and the default entry is used. The default entry in this case has a next hop within the ISP servicing the network, 70.231.159.254 (this is R2's a-side interface). This address happens to be within SBC's DSL network called by the somewhat cumbersome name `adsl-70-231-159-254.dsl.snfc21.sbcglobal.net`. Because this router is in the global Internet and the Windows machine's source address is the private address 10.0.0.100, R1 performs *Network Address Translation* (NAT) on the datagram to make it routable on the Internet. The NAT operation results in the datagram having the new source address 70.231.132.85, which corresponds to R1's b-side interface. Networks that do not use private addressing (e.g., ISPs and larger enterprises) avoid the last step and the original source address remains unchanged. NAT is described in more detail in Chapter 7.

When router R2 (inside the ISP) receives the datagram, it goes through the same steps that the local router R1 did (except for the NAT operation). If the datagram is not destined for one of its own IP addresses, the datagram is forwarded. In this case, the router usually has not only a default route but several others, depending on its connectivity to the rest of the Internet and its own local policies.

Note that IPv6 forwarding varies only slightly from conventional IPv4 forwarding. Aside from the larger addresses, IPv6 uses a slightly different mechanism (Neighbor Solicitation messages) to ascertain the lower-layer address of its next hop. It is described in more detail in Chapter 8, as it is part of ICMPv6. In addition, IPv6 has both link-local addresses and global addresses (see Chapter 2). While global addresses behave like regular IP addresses, link-local addresses can be used only on the same link. In addition, because all the link-local addresses share the same IPv6 prefix (`fe80::/10`), a multihomed host may require user input to determine which interface to use when sending a datagram destined for a link-local destination.

To illustrate the use of link-local addresses, we start with our Windows XP machine, assuming IPv6 is enabled and operational:

```
C:\> ping6 fe80::204:5aff:fe9f:9e80
```

```
Pinging fe80::204:5aff:fe9f:9e80 with 32 bytes of data:
```

```
No route to destination.
```

```
Specify correct scope-id or use -s to specify source address.
```

```
...
```

```
C:\> ping6 fe80::204:5aff:fe9f:9e80%6
```

```
Pinging fe80::204:5aff:fe9f:9e80%6
```

```
from fe80::205:4eff:fe4a:24bb%6 with 32 bytes of data:
```

```
Reply from fe80::204:5aff:fe9f:9e80%6: bytes=32 time=1ms
```

```
Reply from fe80::204:5aff:fe9f:9e80%6: bytes=32 time=1ms
```

```
Reply from fe80::204:5aff:fe9f:9e80%6: bytes=32 time=1ms
Reply from fe80::204:5aff:fe9f:9e80%6: bytes=32 time=1ms
```

```
Ping statistics for fe80::204:5aff:fe9f:9e80%6:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 1ms, Average = 1ms
```

Here we see that failing to specify which interface to use for outbound link-local traffic results in an error. In Windows XP, we can specify either a scope ID or a source address. In this example we specify the scope ID as an interface number using the %6 extension to the destination address. This informs the system to use interface number 6 as the correct interface when sending the ping traffic.

To see the path taken to an IP destination, we can use the `tracert` program (called `tracert` on Windows, which has a slightly different set of options) with the `-n` option to not convert IP addresses to names:

```
Linux% tracert -n ftp.uu.net
tracert to ftp.uu.net (192.48.96.9), 30 hops max, 38 byte packets
 1  70.231.159.254  9.285 ms  8.404 ms  8.887 ms
 2  206.171.134.131  8.412 ms  8.764 ms  8.661 ms
 3  216.102.176.226  8.502 ms  8.995 ms  8.644 ms
 4  151.164.190.185  8.705 ms  8.673 ms  9.014 ms
 5  151.164.92.181  9.149 ms  9.057 ms  9.537 ms
 6  151.164.240.134  9.680 ms  10.389 ms  11.003 ms
 7  151.164.41.10  11.605 ms  37.699 ms  11.374 ms
 8  12.122.79.97  13.449 ms  12.804 ms  13.126 ms
 9  12.122.85.134  15.114 ms  15.020 ms  13.654 ms
    MPLS Label=32307 CoS=5 TTL=1 S=0
10  12.123.12.18  16.011 ms  13.555 ms  13.167 ms
11  192.205.33.198  15.594 ms  15.497 ms  16.093 ms
12  152.63.57.102  15.103 ms  14.769 ms  15.128 ms
13  152.63.34.133  77.501 ms  77.593 ms  76.974 ms
14  152.63.38.1  77.906 ms  78.101 ms  78.398 ms
15  207.18.173.162  81.146 ms  81.281 ms  80.918 ms
16  198.5.240.36  77.988 ms  78.007 ms  77.947 ms
17  198.5.241.101  81.912 ms  82.231 ms  83.115 ms
```

This program lists each of the IP hops traversed while sending a series of datagrams to the destination `ftp.uu.net` (192.48.96.9). The `tracert` program uses a combination of UDP datagrams (with increasing TTL over time) and ICMP messages (used to detect each hop when the UDP datagrams expire) to accomplish its task. Three UDP packets are sent at each TTL value, providing three round-trip-time measurements to each hop. Traditionally, `tracert` has carried only IP information, but here we also see the following line:

```
MPLS Label=32307 CoS=5 TTL=1 S=0
```

This indicates that *Multiprotocol Label Switching* (MPLS) [RFC3031] is being used on the path, and the label ID is 32307, class of service is 5, TTL is 1, and the message is not the bottom of the MPLS label stack ($S = 0$; see [RFC4950]). MPLS is a form of link-layer network capable of carrying multiple network-layer protocols. Its interaction with ICMP is described in [RFC4950], and its handling of IPv4 packets containing options is described in [RFC6178]. Many network operators use it for traffic engineering purposes (i.e., controlling where network traffic flows through their networks).

5.4.4 Discussion

In the examples we have just seen there are a few key points that should be kept in mind regarding the operation of IP unicast forwarding:

1. Most of the hosts and routers in this example used a default route consisting of a single forwarding table entry of this form: mask 0, destination 0, next hop <some IP address>. Indeed, most hosts and most routers at the edge of the Internet can use a default route for everything other than destinations on local networks because there is only one interface available that provides connectivity to the rest of the Internet.
2. The source and destination IP addresses in the datagram never change once in the regular Internet. This is always the case unless either source routing is used, or when other functions (such as NAT, as in the example) are encountered along the data path. Forwarding decisions at the IP layer are based on the destination address.
3. A different lower-layer header is used on each link that uses addressing, and the lower-layer destination address (if present) always contains the lower-layer address of the next hop. Therefore, lower-layer headers routinely change as the datagram is moved along each hop toward its destination. In our example, both Ethernet LANs encapsulated a link-layer header containing the next hop's Ethernet address, but the DSL link did not. Lower-layer addresses are normally obtained using ARP (see Chapter 4) for IPv4 and ICMPv6 Neighbor Discovery for IPv6 (see Chapter 8).

5.5 Mobile IP

So far we have discussed the conventional ways that IP datagrams are forwarded through the Internet, as well as private networks that use IP. One assumption of the model is that a host's IP address shares a prefix with its nearby hosts and routers. If such a host should move its point of network attachment, yet remain connected to the network at the link layer, all of its upper-layer (e.g., TCP) connections would fail

Firewalls and Network Address Translation (NAT)

7.1 Introduction

During the early years of the Internet and its protocols, most network designers and developers were from universities or other entities engaged in research. These researchers were generally friendly and cooperative, and the Internet system was not especially resilient to attack, but not many people were interested in attacking it, either. By the late 1980s and especially the early to mid-1990s the Internet had gained the interest of the mass population and ultimately people interested in compromising its security. Successful attacks became commonplace, and many problems were caused by bugs or unplanned protocol operations in the software implementations of Internet hosts. Because some sites had a large number of end systems with various versions of operating system software, it became very difficult for system administrators to ensure that all the various bugs in these end systems had been fixed. Furthermore, for obsolete systems, this task was all but impossible. Fixing the problem would have required a way to control the Internet traffic to which the end hosts were exposed. Today, this is provided by a *firewall*—a type of router that restricts the types of traffic it forwards.

As firewalls were being deployed to protect enterprises, another problem was becoming important: the number of available IPv4 addresses was diminishing, with a threat of exhaustion. Something would have to be done with the way addresses were allocated and used. One of the most important mechanisms developed to deal with this, aside from IPv6, is called *Network Address Translation* (NAT). With NAT, Internet addresses need not be globally unique, and as a consequence they can be reused in different parts of the Internet, called *address realms*. Allowing the same addresses to be reused in multiple realms greatly eased the problem of address exhaustion. As we shall see, NAT can also be synergistically combined with firewalls to produce combination devices that have become the

most popular types of routers used to connect end users, including home networks and small enterprises, to the Internet. We shall now explore both firewalls and NATs in further detail.

7.2 Firewalls

Given the enormous management problems associated with trying to keep end system software up-to-date and bug-free, the focus of resisting attacks expanded from securing end systems to restricting the Internet traffic allowed to flow to end systems by filtering out some traffic using firewalls. Today, firewalls are common, and several different types have evolved.

The two major types of firewalls commonly used include *proxy firewalls* and *packet-filtering firewalls*. The main difference between them is the layer in the protocol stack at which they operate, and consequently the way IP addresses and port numbers are used. The packet-filtering firewall is an Internet router that drops datagrams that (fail to) meet specific criteria. The proxy firewall operates as a multihomed server host from the viewpoint of an Internet client. That is, it is the endpoint of TCP and UDP transport associations; it does not typically route IP datagrams at the IP protocol layer.

7.2.1 Packet-Filtering Firewalls

Packet-filtering firewalls act as Internet routers and *filter* (drop) some traffic. They can generally be configured to discard or forward packets whose headers meet (or fail to meet) certain criteria, called *filters*. Simple filters include range comparisons on various parts of the network-layer or transport-layer headers. The most popular filters involve undesired IP addresses or options, types of ICMP messages, and various UDP or TCP services, based on the port numbers contained in each packet. As we shall see, the simplest packet-filtering firewalls are stateless, whereas the more sophisticated ones are stateful. Stateless packet-filtering firewalls treat each datagram individually, whereas stateful firewalls are able associate packets with either previously observed packets or packets that arrive in the future to make inferences about datagrams or streams—either those belonging to a single transport association or those IP fragments that constitute an IP datagram (see Chapter 10). IP fragmentation can significantly complicate a firewall's job, and stateless packet-filtering firewalls are easily confused by fragments.

A typical packet-filtering firewall is shown in Figure 7-1. Here, the firewall is an Internet router with three network interfaces: an “inside,” an “outside,” and a third “DMZ” interface. The DMZ subnet provides access to an extranet or DMZ where servers are deployed for Internet users to access. Network administrators install filters or *access control lists* (ACLs, basically policy lists indicating what types of packets to discard or forward) in the firewall. Typically, these filters conservatively block traffic from the outside that may be harmful and liberally allow traffic to travel from inside to outside.

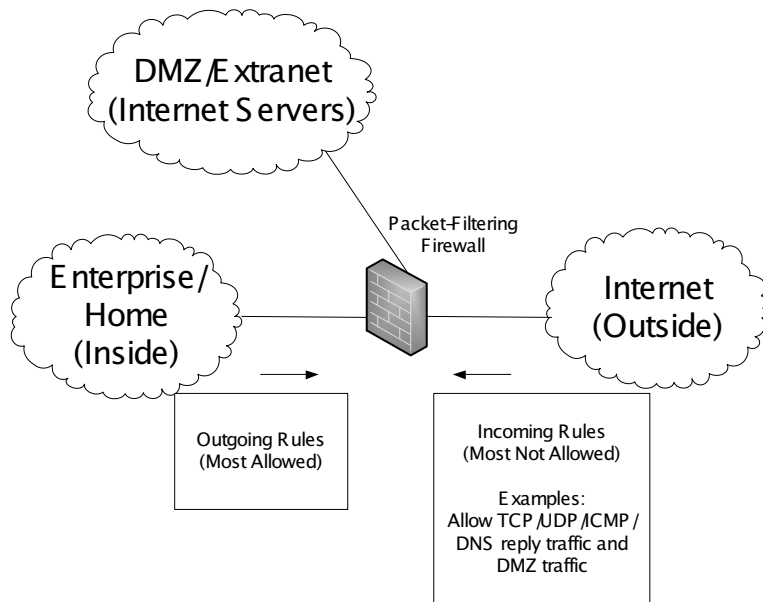


Figure 7-1 A typical packet-filtering firewall configuration. The firewall acts as an IP router between an “inside” and an “outside” network, and sometimes a third “DMZ” or extranet network, allowing only certain traffic to pass through it. A common configuration allows all traffic to pass from inside to outside but only a small subset of traffic to pass in the reverse direction. When a DMZ is used, only certain services are permitted to be accessed from the Internet.

7.2.2 Proxy Firewalls

Packet-filtering firewalls act as routers that selectively drop packets. Other types of firewalls, called *proxy firewalls*, are not really Internet routers in the true sense. Instead, they are essentially hosts running one or more *application-layer gateways* (ALGs)—hosts with more than one network interface that relay traffic of certain types between one connection/association and another at the application layer. They do not typically perform IP forwarding as routers do, although more sophisticated proxy firewalls are now available that combine various functions.

Figure 7-2 illustrates a proxy firewall. For this type of firewall, clients on the inside of the firewall are usually configured in a special way to associate (or connect) with the proxy instead of the actual end host providing the desired service. (Applications capable of operating with proxy firewalls this way include configuration options for it.) These firewalls act as multihomed hosts, and their IP forwarding capability, if present, is typically disabled. As with packet-filtering firewalls, a common configuration is to have an “outside” interface assigned a globally routable IP address and for its “inner” interface to be configured with a private IP address. Thus, proxy firewalls support the use of private address realms.

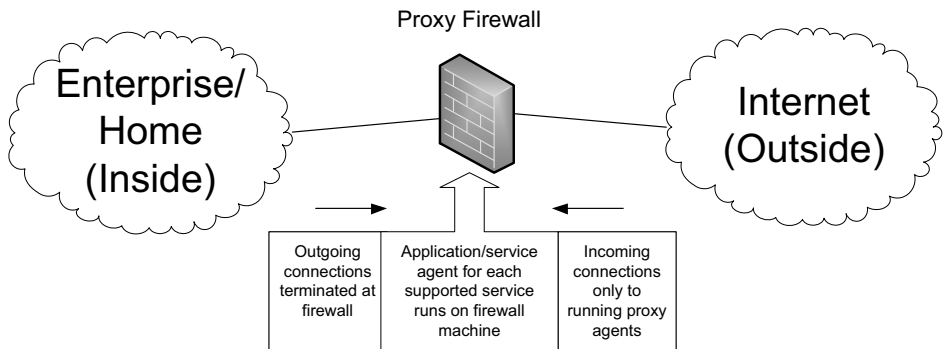


Figure 7-2 The proxy firewall acts as a multihomed Internet host, terminating TCP connections and UDP associations at the application layer. It does not act as a conventional IP router but rather as an ALG. Individual applications or proxies for each service supported must be enabled for communication to take place through the proxy firewall.

While this type of firewall can be quite secure (some people believe this type is fundamentally more secure than packet-filtering firewalls), this security comes at a cost of brittleness and lack of flexibility. In particular, because this style of firewall must contain a proxy for each transport-layer service, any new services to be used must have a corresponding proxy installed and operated for connectivity to take place through the proxy. In addition, each client must typically be configured to find the proxy (e.g., using the Web Proxy Auto-Discovery Protocol, or WPAD [XIDAD], although there are some alternatives—so-called capturing proxies that catch all traffic of a certain type regardless of destination address). With respect to deployment, these firewalls work well in environments where all types of network services being accessed are known with certainty in advance, but they may require significant intervention from network operators to support additional services.

The two most common forms of proxy firewalls are *HTTP proxy firewalls* [RFC2616] and *SOCKS firewalls* [RFC1928]. The first type, also called *Web proxies*, work only for the HTTP and HTTPS (Web) protocols, but because these protocols are so popular, such proxies are commonly used. These proxies act as Web servers for internal clients and as Web clients when accessing external Web sites. Such proxies often also operate as *Web caches*. These caches save copies of Web pages so that subsequent accesses can be served directly from the cache instead of from the originating Internet Web server. Doing so can reduce latency to display Web pages and improve the experience of users accessing the Web. Some Web proxies are also used as *content filters*, which attempt to block access to certain Web sites based on a “blacklist” of prohibited sites. Conversely, a number of so-called *tunneling proxy servers* are available on the Internet. These servers (e.g., psiphon, CGIProxy) essentially perform the opposite function—to allow users to avoid being blocked by content filters.

The SOCKS protocol is more generic than HTTP for proxy access and is applicable to more services than just the Web. Two versions of SOCKS are currently in use: version 4 and version 5. Version 4 provides the basic support for proxy traversal, and version 5 adds strong authentication, UDP traversal, and IPv6 addressing. To use a SOCKS proxy, an application must be written to use SOCKS (it must be “socksified”) and configured to know about the location of the proxy and which version of SOCKS to use. Once this is accomplished, the client uses the SOCKS protocol to request the proxy to perform network connections and, optionally, DNS lookups.

7.3 Network Address Translation (NAT)

NAT is essentially a mechanism for allowing the same sets of IP addresses to be reused in different parts of the Internet. The primary motivation for the creation of NAT was the limited and diminishing availability of IP address space. The most common use case for a NAT is when a site with a single Internet connection is assigned a small range of IP addresses (perhaps only a single address), but there are multiple computers requiring Internet access. When all incoming and outgoing traffic passes through a single NAT device that partitions the inside (private) address realm from the global Internet address realm, all the internal systems can be provided Internet connectivity as clients using locally assigned, private IP addresses. Allowing privately addressed systems to offer services on the Internet, however, is somewhat more complicated. We discuss this case in Section 7.3.4.

NAT was introduced to solve two problems: address depletion and concerns regarding the scalability of routing. At the time of its introduction (early 1990s), NAT was suggested as a stopgap, temporary measure to be used until the deployment of some protocol with a larger number of addresses (ultimately, IPv6) became widespread. Routing scalability was being tackled with the development of Classless Inter-Domain Routing (CIDR; see Chapter 2). NAT is popular because it reduces the need for globally routable Internet addresses but also because it offers some degree of natural firewall capability and requires little configuration. Perhaps ironically, the development and eventual widespread use of NAT has contributed to significantly slow the adoption of IPv6. Among its other benefits, IPv6 was intended to make NAT unnecessary [RFC4864].

Despite its popularity, NAT has several drawbacks. The most obvious is that offering Internet-accessible services from the private side of a NAT requires special configuration because privately addressed systems are not directly reachable from the Internet. In addition, for a NAT to work properly, every packet in both directions of a connection or association must pass through the same NAT. This is because the NAT must actively rewrite the addressing information in each packet in order for communication between a privately addressed system and a conventionally addressed Internet host to work. In many ways, NATs run counter to a fundamental tenet of the Internet protocols: the “smart edge” and “dumb middle.” To do their job, NATs require connection state on a *per-association* (or

per-connection) basis and must operate across multiple protocol layers, unlike conventional routers. Modifying an address at the IP layer also requires modifying checksums at the transport layer (see Chapters 10 and 13 regarding the pseudo-header checksum to see why).

NAT poses problems for some application protocols, especially those that send IP addressing information inside the application-layer payload. The *File Transfer Protocol* (FTP) [RFC0959] and SIP [RFC5411] are among the best-known protocols of this type. They require a special application-layer gateway function that rewrites the application content in order to work unmodified with NAT or other NAT traversal methods that allow the applications to determine how to work with the specific NAT they are using. A more complete list of considerations regarding NAT appears in [RFC3027]. Despite their numerous problems, NATs are very widely used, and most network routers (including essentially all low-end home routers) support it. Today, NATs are so prevalent that application designers are encouraged to make their applications “NAT-friendly” [RFC3235]. It is worth mentioning that despite its shortcomings, NAT supports the basic protocols (e.g., e-mail, Web browsing) that are needed by millions of client systems accessing the Internet every day.

A NAT works by rewriting the identifying information in packets transiting through a router. Most commonly this happens for two directions of a data transfer. In its most basic form, NAT involves rewriting the source IP address of packets as they are forwarded in one direction and the destination IP addresses of packets traveling in the reverse direction (see Figure 7-3). This allows the source IP address in outgoing packets to become one of the NAT router’s Internet-facing interfaces instead of the originating host’s. Thus, to a host on the Internet, packets coming from any of the hosts on the privately addressed side of the NAT appear to be coming from a globally routable IP address of the NAT router.

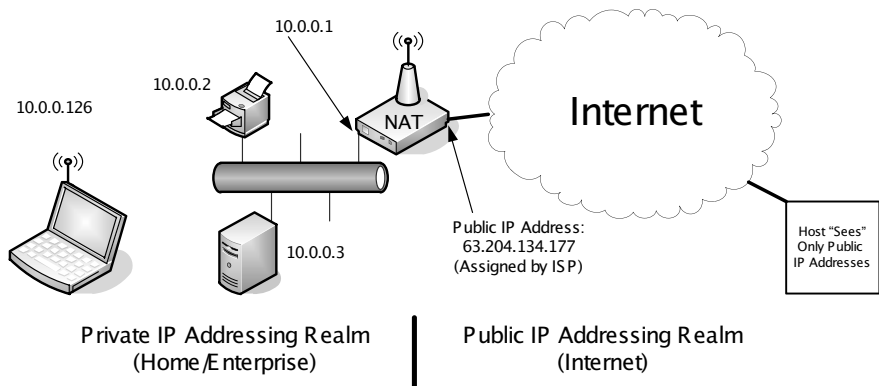
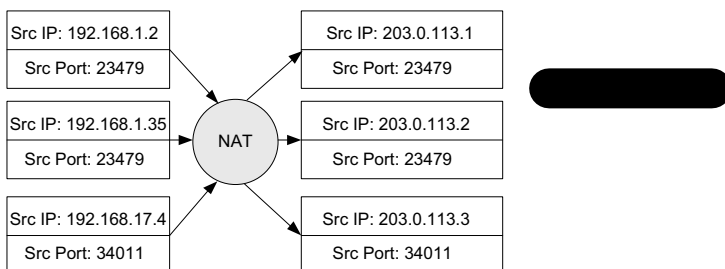


Figure 7-3 A NAT isolates private addresses and the systems using them from the Internet. Packets with private addresses are not routed by the Internet directly but instead must be translated as they enter and leave the private network through the NAT router. Internet hosts see traffic as coming from a public IP address of the NAT.

Most NATs perform both *translation* and *packet filtering*, and the packet-filtering criteria depend on the dynamics of the NAT state. The choice of packet-filtering policy may have a different granularity—for example, the treatment of unsolicited packets (those not associated with packets originating from behind the NAT) received by the NAT may depend on source and destination IP address and/or source and destination port number. The behavior may vary between NATs or in some cases vary over time through the same NAT. This presents challenges for applications that must operate behind a wide variety of NATs.

7.3.1 Traditional NAT: Basic NAT and NAPT

The precise behavior of a NAT remained unspecified for many years. Nonetheless, a taxonomy of NAT types has emerged, based largely on observing how different implementations of the NAT idea behave. The so-called *traditional NAT* includes both *basic NAT* and *Network Address Port Translation* (NAPT) [RFC3022]. Basic NAT performs rewriting of IP addresses only. In essence, a private address is rewritten to be a public address, often from a pool or range of public addresses supplied by an ISP. This type of NAT is not the most popular because it does not help to dramatically reduce the need for IP addresses—the number of globally routable addresses must equal or exceed the number of internal hosts that wish to access the Internet simultaneously. A much more popular approach, NAPT involves using the transport-layer identifiers (i.e., ports for TCP and UDP, query identifiers for ICMP) to differentiate which host on the private side of the NAT is associated with a particular packet (see Figure 7-4). This allows a large number of internal hosts (i.e., multiple thousands) to access the Internet simultaneously using a limited number of public addresses, often only a single one. We shall ordinarily use the term NAT to include both traditional NAT and NAPT unless the distinction is important in a particular context.



The addresses used in a private addressing realm “behind” or “inside” a NAT are not enforced by anyone other than the local network administrator. Thus, it is possible for a private realm to make use of global address space. In principle, this is acceptable. However, when such global addresses are owned and being used by another entity on the Internet, local systems in the private realm would most likely be unable to reach the public systems using the same addresses because the close proximity of the local systems would effectively “mask” the visibility of the farther-away systems using the same addresses. To avoid this undesirable situation, there are three IPv4 address ranges reserved for use with private addressing realms [RFC1918]: 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16. These address ranges are often used as default values for address pools in embedded DHCP servers (see Chapter 6).

As suggested earlier, a NAT provides some degree of security similar to that of a firewall. By default, all systems on the private side of the NAT cannot be reached from the Internet. In most NAT deployments, the internal systems use private addresses. Consequently, communications between hosts in the private addressing realm and those in the public realm can be facilitated only with participation from the NAT, according to its usage policies and behavior. While a large variety of policies may be used in practice, a common policy allows almost all outgoing and returning traffic (associated with outgoing traffic) to pass through the NAT but blocks almost all incoming new connection requests. This behavior inhibits “probing” attacks that attempt to ascertain which IP addresses have active hosts available to exploit. In addition, a NAT (especially a NAPT) “hides” the number and configuration of internal addresses from the outside. Some users feel this topology information is proprietary and should remain confidential. NAT helps by providing so-called topology hiding.

As we shall now explore, NATs are tailored to the protocols and applications that they need to support, so it is difficult to discuss NAT behavior without also mentioning the particular protocol(s) it is being asked to handle. Thus, we now turn to how NAT behaves with each major transport protocol and how it may be used in mixed IPv4/IPv6 environments. Many of the behavioral specifics for NATs have been the subject of the IETF Behavior Engineering for Hindrance Avoidance (BEHAVE) working group. BEHAVE has produced a number of documents, starting in 2007, that clarify consistent behaviors for NATs. These documents are useful for application writers and NAT developers so that a consistent expectation can be established as to how NATs should operate.

7.3.1.1 NAT and TCP

Recall from Chapter 1 that the primary transport-layer protocol for the Internet, TCP, uses an IP address and port number to identify each end of a connection. A connection is identified by the combination of two ends; each unique TCP connection is identified by two IP addresses and two port numbers. When a TCP connection starts, an “active opener” or client usually sends a synchronization (SYN) packet to a “passive opener” or server. The server responds with its own

SYN packet, which also includes an acknowledgment (ACK) of the client's SYN. The client then responds with an ACK to the server. This "three-way handshake" establishes the connection. A similar exchange with finish (FIN) packets is used to gracefully close a connection. The connection can also be forcefully closed right away using a reset (RST) packet. (See Chapter 13 for more detail on TCP connections.) The behavioral requirements for traditional NAT with TCP are defined in [RFC5382] and relate primarily to the TCP three-way handshake.

Referring to the example home network in Figure 7-3, consider a TCP connection initiated by the wireless client at 10.0.0.126 destined for the Web server on the host `www.isoc.org` (IPv4 address 212.110.167.157). Using the following notation to indicate IPv4 addresses and port numbers—(source IP:source port; destination IP:destination port)—the packet initiating the connection on the private segment might be addressed as (10.0.0.126:9200; 212.110.167.157:80). The NAT/firewall device, acting as the default router for the client, receives the first packet. The NAT notices that the incoming packet is a new connection (because the SYN bit in the TCP header is turned on; see Chapter 13). If policy permits (which it typically does because this is an outgoing connection), the source IP address is modified in the packet to reflect the routable IP address of the NAT router's external interface. Thus, when the NAT forwards this packet, the addressing is (63.204.134.177:9200; 212.110.167.157:80). In addition to forwarding the packet, the NAT creates internal state to remember the fact that a new connection is being handled by the NAT (called a *NAT session*). At a minimum, this state includes an entry (called a *NAT mapping*) containing the source port number and IP address of the client. This becomes useful when the Internet server replies. The server replies to the endpoint (63.204.134.177:9200), the external NAT address, using the port number chosen initially by the client. This behavior is called *port preservation*. By matching the destination port number on the received datagram against the appropriate NAT mapping, the NAT is able to ascertain the internal IP address of the client that made the initial request. In our example, this address is 10.0.0.126, so the NAT rewrites the response packet from (212.110.167.157:80; 63.204.134.177:9200) to (212.110.167.157:80; 10.0.0.126:9200) and forwards it. The client then receives a response to its request and for most purposes is now connected to the server.

This example conveys how a basic NAT session is established in the normal case, but not how the session is cleared. Session state is removed if FINs are exchanged, but not all TCP connections are cleared gracefully. Sometimes a computer is simply turned off, which can leave stale NAT mappings in the NAT's memory. Thus, a NAT must also remove mappings thought to have "gone dead" because of a lack of traffic (or if an RST segment indicates some other form of problem).

Most NATs include a simplified version of the TCP connection establishment procedures and can distinguish between connection success and failure. In particular, when an outgoing SYN segment is observed, a *connection timer* is activated, and if no ACK is seen before the timer expires, the session state is cleared. If an ACK does arrive, the timer is canceled and a *session timer* is created, with a

considerably longer timeout (e.g., hours instead of minutes). When this happens, the NAT may send an additional packet to the internal endpoint, just to double-check if the session is indeed dead (called *probing*). If it receives an ACK, the NAT realizes that the connection is still active, resets the session timer, and does not delete the session. If it receives either no response (after a *close timer* has expired) or an RST segment, the connection has gone dead, and the state is cleared.

[RFC5382], a product of the BEHAVE working group, notes that a TCP connection can be configured to send “keepalive” packets (see Chapter 17), and the default rate is one packet every 2 hours, if enabled. Otherwise, a TCP connection can remain established indefinitely. While a connection is being set up or cleared, however, the maximum idle time is 4 minutes. Consequently, [RFC5382] requires (REQ-5) that a NAT wait at least 2 hours and 4 minutes before concluding that an established connection is dead and at least 4 minutes before concluding that a partially opened or closed connection is dead.

One of the tricky problems for a TCP NAT is handling peer-to-peer applications operating on hosts residing on the private sides of multiple NATs [RFC5128]. Some of these applications use a *simultaneous open* whereby each end of the connection acts as a client and sends SYN packets more or less simultaneously. TCP is able to handle this case by responding with SYN + ACK packets that complete the connection faster than with the three-way handshake, but many existing NATs do not handle it properly. [RFC5382] addresses this by requiring (REQ-2) that a NAT handle all valid TCP packet exchanges, and simultaneous opens in particular. Some peer-to-peer applications (e.g., network games) use this behavior. In addition, [RFC5382] specifies that an inbound SYN for a connection about which the NAT knows nothing should be silently discarded. This can occur when a simultaneous open is attempted but the external host’s SYN arrives at the NAT before the internal host’s SYN. Although this may seem unlikely, it can happen as a result of clock skew, for example. If the incoming external SYN is dropped, the internal SYN has time to establish a NAT mapping for the same connection represented by the external SYN. If no internal SYN is forthcoming in 6s, the NAT may signal an error to the external host.

7.3.1.2 NAT and UDP

The NAT behavioral requirements for unicast UDP are defined in [RFC4787]. Most of the same issues arise when performing NAT on a collection of UDP datagrams as arise when performing NAT on TCP. UDP is somewhat different, however, because there are no connection establishment and clearing procedures as there are in TCP. More specifically, there are no indicators such as the SYN, FIN, and RST bits to indicate that a session is being created or destroyed. Furthermore, the participants in an association may not be completely clear. UDP does not use a 4-tuple to identify a connection like TCP; instead, it can rely on only the two endpoint address/port number combinations. To handle these issues, UDP NATs use a *mapping timer* to clear NAT state if a binding has not been used “recently.” There is considerable variation in the values used for this timer to determine what

“recently” means, but [RFC4787] requires the timer to be at least 2 minutes and recommends that it be 5 minutes. A related consideration is when the timer should be considered refreshed. Timers can be refreshed when packets travel from the inside to the outside of the NAT (NAT outbound refresh behavior) or vice versa (NAT inbound refresh behavior). [RFC4787] requires NAT outbound refresh behavior to be true. Inbound behavior may or may not be true.

As we discussed in Chapter 5 (and will see again in Chapter 10), UDP and IP packets can be fragmented. Fragmentation allows for a single IP packet to span multiple chunks (fragments), each of which is treated as an independent datagram. However, because of the layering of UDP above IP, an IP fragment other than the first one does not contain the port number information needed by NAPT to operate properly. This also applies to TCP and ICMP. Thus, in general, fragments cannot be handled properly by simple NATs or NAPT.

7.3.1.3 NAT and Other Transport Protocols (DCCP, SCTP)

Although TCP and UDP are by far the most widely used Internet transport protocols, there are two other protocols for which NAT behaviors have been defined or are being defined. The *Datagram Congestion Control Protocol* (DCCP) [RFC4340] provides a congestion-controlled datagram service. [RFC5597] gives NAT behavioral requirements with respect to DCCP, and [RFC5596] gives a modification to DCCP to support a TCP-like simultaneous open procedure for use with DCCP. The *Stream Control Transmission Protocol* (SCTP) [RFC4960] provides a reliable messaging service that can accommodate hosts with multiple addresses. Considerations for NAT with SCTP are given in [HBA09] and [IDSNAT].

7.3.1.4 NAT and ICMP

ICMP, the Internet Control Message Protocol, is detailed in Chapter 8. It provides status information about IP packets and can also be used for making certain measurements and gathering information about the state of the network. The NAT behavioral requirements for ICMP are defined in [RFC5508]. There are two issues involved when NAT is used for ICMP. ICMP has two categories of messages: informational and error. Error messages generally contain a (partial or full) copy of the IP packet that induced the error condition. They are sent from the point where an error was detected, often in the middle of the network, to the original sender. Ordinarily, this presents no difficulty, but when an ICMP error message passes through a NAT, the IP addresses in the included “offending datagram” need to be rewritten by the NAT in order for them to make sense to the end client (called *ICMP fix-up*). For informational messages, the same issues arise, but in this case most message types are of a query/response or client/server nature and include a *Query ID* field that is handled much like port numbers for TCP or UDP. Thus, a NAT handling these types of messages can recognize outgoing informational requests and set a timer in anticipation of a returning response.

7.3.1.5 NAT and Tunneled Packets

In some cases, tunneled packets (see Chapter 3) are to be sent through a NAT. When this happens, not only must a NAT rewrite the IP header, but it may also have to rewrite the headers or payloads of other packets that are encapsulated in them. One example of this is the Generic Routing Encapsulation (GRE) header used with the Point-to-Point Tunneling Protocol (PPTP; see Chapter 3). When the GRE header is passed through a NAT, its *Call-ID* field could conflict with the NAT's (or with other hosts' tunneled connections). If the NAT fails to handle this mapping appropriately, communication is not possible. As we might imagine, additional levels of encapsulation serve only to complicate a NAT's job further.

7.3.1.6 NAT and Multicast

So far we have discussed only unicast IP traffic with NATs. NATs can be configured to support multicast traffic (see Chapter 9), although this is rare. [RFC5135] gives the requirements for handling multicast traffic through NATs. In effect, to support multicast traffic a NAT is augmented with an IGMP proxy (see [RFC4605] and Chapter 9). In addition, the destination IP addresses and port numbers of packets traveling from a host on the outside to the inside of NAT are *not* modified. For traffic flowing from inside to outside, the source addresses and port numbers may be modified according to the same behaviors as with unicast UDP.

7.3.1.7 NAT and IPv6

Given the tremendous popularity of NAT for IPv4, it is natural to wonder whether NAT will be used with IPv6. At present, this is a contentious issue [RFC5902]. To many protocol designers, NAT arose as a necessary but undesirable “wart” that has added a tremendous amount of complexity to the design of every other protocol. Consequently, there is staunch resistance to supporting the use of NAT with IPv6 based on the idea that saving address space is unnecessary with IPv6 and that other desirable NAT features (e.g., firewall-like functionality, topology hiding, and privacy) can be better achieved using *Local Network Protection* (LNP) [RFC4864]. LNP represents a collection of techniques with IPv6 that match or exceed the properties of NATs.

Aside from its packet-filtering properties, NAT supports the coexistence of multiple address realms and thereby helps to avoid the problem of a site having to change its IP addresses when it switches ISPs. For example, [RFC4193] defines *Unique Local IPv6 Unicast Addresses* (ULAs) that could conceivably be used with an experimental version of IPv6-to-IPv6 prefix translation called *NPTv6* [RFC6296]. It uses an algorithm instead of a table to translate IPv6 addresses to (different) IPv6 addresses (e.g., in different realms) based on their prefix and as a result does not require keeping per-connection state as with conventional NAT. In addition, the algorithm modifies addresses in such a way that the resulting checksum computation for common transport protocols (TCP, UDP) remains the same. This significantly reduces the complexity of NAT because it does not have to modify the

data in a packet beyond the network layer and does not require access to transport layer port numbers in order to operate properly. However, applications that require access to a NAT's external address must still use a NAT traversal method or depend on an ALG. In addition, NPTv6 does not by itself offer the packet-filtering capabilities of a firewall, so additional deployment considerations must be made.

7.3.2 Address and Port Translation Behavior

There has been considerable variation in the way NATs operate. Most of the details relate to the specifics of the address and port mappings. One of the primary goals of the BEHAVE working group in IETF was to clarify the common behaviors and set guidelines as to which are the most appropriate. To better understand the issues involved, we begin with a generic NAT mapping example (see Figure 7-5).

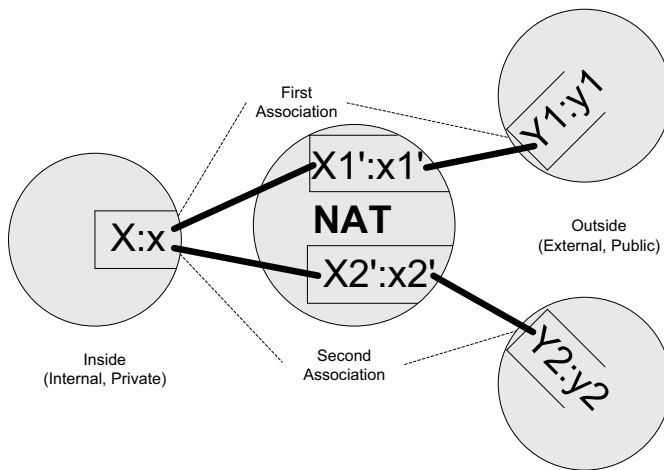


Figure 7-5 A NAT's address and port behavior is characterized by what its mappings depend on. The inside host uses IP address:port $X:x$ to contact $Y1:y1$ and then $Y2:y2$. The address and port used by the NAT for these associations are $X1':x1'$ and $X2':x2'$, respectively. If $X1':x1'$ equals $X2':x2'$ for any $Y1:y1$ or $Y2:y2$, the NAT has endpoint-independent mappings. If $X1':x1'$ equals $X2':x2'$ if and only if $Y1$ equals $Y2$, the NAT has address-dependent mappings. If $X1':x1'$ equals $X2':x2'$ if and only if $Y1:y1$ equals $Y2:y2$, the NAT has address-and-port-dependent mappings. A NAT with multiple external addresses (i.e., where $X1'$ may not equal $X2'$) has an address pooling behavior of arbitrary if the outside address is chosen without regard to inside or outside address. Alternatively, it may have a pooling behavior of paired, in which case the same $X1$ is used for any association with $Y1$.

In Figure 7-5, we use the notation $X:x$ to indicate that a host in the private addressing realm (inside host) uses IP address X with port number x (for ICMP, the query ID is used instead of the port number). The IP address X is ordinarily

chosen from the private IPv4 address space defined in [RFC1918]. To reach the remote address/port combination $Y:y$, the NAT establishes a mapping using an external (usually a public, globally routable) address $X1'$ and port number $x1'$. Assuming that the internal host contacts $Y1:y1$ followed by $Y2:y2$, the NAT establishes mappings $X1':x1'$ and $X2':x2'$, respectively. In most cases, $X1'$ equals $X2'$ because most sites use only a single globally routable IP address. The mapping is said to be *reused* if $x1'$ equals $x2'$. If $x1'$ and $x2'$ equal x , the NAT implements port preservation, mentioned earlier. In some cases, port preservation is not possible, so the NAT must deal with port collisions as suggested by Figure 7-4.

Table 7-1 and Figure 7-5 summarize the various NAT port and address behaviors based on definitions from [RFC4787]. Table 7-1 also gives filtering behaviors that use similar terminology and that we discuss in Section 7.3.3. For all common transports, including TCP and UDP, the required NAT address- and port-handling behavior is endpoint-independent (a similar behavior is recommended for ICMP). The purpose of this requirement is to help applications that attempt to determine the external addresses used for their traffic to work more reliably. We discuss this in more detail in Section 7.4 when we discuss NAT traversal.

Table 7-1 A NAT's overall behavior is defined by both its translation and filtering behaviors. Each of these may be independent of host address, dependent on address, or dependent on both address and port number.

Behavior Name	Translation Behavior	Filtering Behavior
Endpoint-independent	$X1':x1' = X2':x2'$ for all $Y2:y2$ (required)	Allows any packets for $X1:x1$ as long as any $X1':x1'$ exists (recommended for greatest transparency)
Address-dependent	$X1':x1' = X2':x2'$ iff $Y1 = Y2$	Allows packets for $X1:x1$ from $Y1:y1$ as long as $X1$ has previously contacted $Y1$ (recommended for more stringent filtering)
Address- and port-dependent	$X1':x1' = X2':x2'$ iff $Y1:y1 = Y2:y2$	Allows packets for $X1:x1$ from $Y1:y1$ as long as $X1$ has previously contacted $Y1:y1$

As stated previously, a NAT may have several external addresses available to use. The set of addresses is typically called the *NAT pool* or *NAT address pool*. Most moderate to large-scale NATs use address pools. Note that NAT address pools are distinct from the DHCP address pools discussed in Chapter 6, although a single device may need to handle both NAT and DHCP address pools. One question in such environments is that when a single host behind the NAT opens multiple simultaneous connections, is each assigned the same external IP address (called *address pairing*) or not? A NAT's *IP address pooling behavior* is said to be *arbitrary* if there is no restriction on which external address is used for any association. It is said to be *paired* if it implements address pairing. Pairing is the recommended NAT behavior for all transports. If pairing is not used, the communication peer

of an internal host may erroneously conclude that it is communicating with different hosts. For NATs with only a single external address, this is obviously not a problem.

A very brittle type of NAT overloads not only addresses but also ports (called *port overloading*). In this case, the traffic of multiple internal hosts may be rewritten to the same external IP address *and port number*. This is a dangerous prospect because if multiple hosts associate with a service on the same external host, it is no longer possible to determine the appropriate destination for traffic returning from the external host. For TCP, this is a consequence of all four elements of the connection identifier (source and destination address and port numbers) being identical in the external network among the various connections. Such behavior is now disallowed.

Some NATs implement a special feature called *port parity*. Such NATs attempt to preserve the “parity” (evenness or oddness) of port numbers. Thus, if $x1$ is even, $x1'$ is even and vice versa. Although not as strong as port preservation, such behavior is sometimes useful for specific application protocols that use special port numberings (e.g., the *Real-Time Protocol*, abbreviated RTP, has traditionally used multiple ports, but there are proposed methods for avoiding this issue [RFC5761]). Port parity preservation is a recommended NAT feature but not a requirement. It is also expected to become less important over time as more sophisticated NAT traversal methods become widespread.

7.3.3 Filtering Behavior

When a NAT creates a binding for a TCP connection, UDP association, or various forms of ICMP traffic, not only does it establish the address and port mappings, but it must also determine its filtering behavior for the returning traffic if it acts as a firewall, which is the common case. The type of filtering a NAT performs, although logically distinct from its address- and port-handling behavior, is often related. In particular, the same terminology is used: endpoint-independent, address-dependent, and address- and port-dependent.

A NAT's filtering behavior is usually related to whether it has established an address mapping. Clearly, a NAT lacking any form of address mapping is unable to forward any traffic it receives from the outside to the inside because it would not know which internal destination to use. For the most common case of outgoing traffic, when a binding is established, filtering is disabled for relevant return traffic. For NATs with endpoint-independent behavior, as soon as any mapping is established for an internal host, any incoming traffic is permitted, regardless of source. For address-dependent filtering behavior, traffic destined for $X1:x1$ is permitted from $Y1:y1$ only if $Y1$ had been previously contacted by $X1:x1$. For those NATs with address- and port-dependent filtering behavior, traffic destined for $X1:x1$ is permitted from $Y1:y1$ only if $Y1:y1$ had been previously contacted by $X1:x1$. The difference between the last two is that the last form takes the port number $y1$ into account.

7.3.4 Servers behind NATs

One of the most obvious problems with NATs is that a system wishing to provide a service from behind a NAT is not directly reachable from the outside. Consider the example in Figure 7-3 once again. If the host with address 10.0.0.3 is to provide a service to the Internet, it cannot be reached without participation from the NAT, for at least two reasons. First, the NAT is acting as the Internet router, so it must agree to forward the incoming traffic destined for 10.0.0.3. Second, and more important, the IP address 10.0.0.3 is not routable through the Internet and cannot be used to identify the server by hosts in the Internet. Instead, the external address of the NAT must be used to find the server, and the NAT must arrange to properly rewrite and forward the appropriate traffic to the server so that it can operate. This process is most often called *port forwarding* or *port mapping*.

With port forwarding, incoming traffic to a NAT is forwarded to a specific configured destination behind the NAT. By employing NAT with port forwarding, it is possible to allow servers to provide services to the Internet even though they may be assigned private, nonroutable addresses. Port forwarding typically requires static configuration of the NAT with the address of the server and the associated port number whose traffic should be forwarded. The port forwarding directive acts like an always-present static NAT mapping. If the server's IP address is changed, the NAT must be updated with the new addressing information. Port forwarding also has the limitation that it has only one set of port numbers for each of its (IP address, transport protocol) combinations. Thus, if the NAT has only a single external IP address, it can forward only a single port of the same transport protocol to at most one internal machine (e.g., it could not support two independent Web servers on the inside being remotely accessible using TCP port 80 from the outside).

7.3.5 Hairpinning and NAT Loopback

An interesting issue arises when a client wishes to reach a server and both reside on the same, private side of the same NAT. NATs that support this scenario implement so-called *hairpinning* or *NAT loopback*. Referring to Figure 7-6, assume that host *X1* attempts to establish a connection to host *X2*. If *X1* knows the private addressing information, *X2:x2*, there is no problem because the connection can be made directly. However, in some cases *X1* knows only the public address information, *X2':x2'*. In these cases, *X1* attempts to contact *X2* using the NAT with destination *X2':x2'*. The hairpinning process takes place when the NAT notices the existence of the mapping between *X2':x2'* and *X2:x2* and forwards the packet to *X2:x2* residing on the private side of the NAT. At this point a question arises as to which source address is contained in the packet heading to *X2:x2*—*X1:x1* or *X1':x1'*?

If the NAT presents the hairpinned packet to *X2* with source addressing information *X1':x1'*, the NAT is said to have “external source IP address and port” hairpinning behavior. This behavior is required for TCP NAT [RFC5382]. The

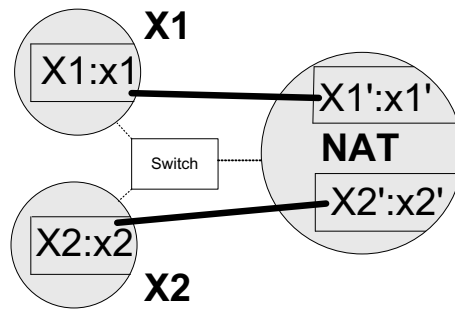


Figure 7-6 A NAT that implements hairpinning or NAT loopback allows a client to reach a server on the same side of the NAT using the server's external IP address and port numbers. That is, $X1$ can reach $X2:x2$ using the addressing information $X2':x2'$.

justification for requiring this behavior is for applications that identify their peers using globally routable addresses. In our example, $X2$ may be expecting an incoming connection from $X1'$ (e.g., because of coordination from a third-party system).

7.3.6 NAT Editors

Together, packets using the UDP and TCP transport protocols account for most of the IP traffic carried on the Internet. These transport protocols, by themselves, can be supported by NAT without additional complexity because their formats are well understood. When application-layer protocols used in conjunction with them carry transport-layer or lower-layer information such as IP addresses, the NAT problem becomes considerably more complicated. The most common example is FTP [RFC0959]. In normal operation, it communicates transport- and network-layer endpoint information (an IP address and port number) so that additional connections can be made when bulk data is to be transferred. This requires a NAT to rewrite not only the IP addresses and port numbers in the IP and TCP portions of a datagram, but also some of the application payload itself. NATs with this capability are sometimes called *NAT editors*. If a NAT changes the size of a packet's application payload, considerable work may be required. For example, TCP numbers every byte in the data transfer using sequence numbers (see Chapter 15), so if the size of a packet is changed, the sequence numbers also require modification. PPTP [RFC2637] also requires a NAT editor for transparent operation (see Chapter 3).

7.3.7 Service Provider NAT (SPNAT) and Service Provider IPv6 Transition

A relatively recent development involves the idea of moving NATs from the customer premises into the ISP. This is sometimes called *service provider NAT* (SPNAT), *carrier-grade NAT* (CGN), or *large-scale NAT* (LSN) and is intended to further mitigate the IPv4 address depletion problem. With SPNAT, it is conceivable

that many ISP customers could share a single global IPv4 address. In effect, this moves the point of aggregation from the edge of the customer to the edge of the ISP. In its basic form, there is no functional difference between conventional NAT and SPNAT; the difference is really in the proposed domain of use. However, moving the NAT function from customer to ISP raises security concerns and brings into question whether individual end users are able to deploy Internet servers and control firewall policy [MBCB08]. A study from 2009 found that a significant number of users accept incoming connections, largely because of peer-to-peer programs [ANM09].

SPNAT can help with the IPv4 address depletion problem, but IPv6 is viewed as the ultimate solution. For a number of reasons already discussed, however, IPv6 deployment has lagged expectations. Originally, a scheme known as dual-stack (see [RFC4213]), whereby each system uses both IPv6 and IPv4 addresses, was intended to support transition to IPv6, but this approach was anticipated to be temporary and rendered unnecessary long before the depletion of IPv4 addresses. An arguably more pragmatic approach is now being undertaken that combines tunneling, address translation, and dual-stack systems in various configurations. We'll discuss some of these in Section 7.6 after exploring the methods that have been developed for dealing with existing NATs.

7.4 NAT Traversal

As an alternative to the complexity of placing ALGs and NAT editors in NAT devices, an application may attempt to perform its own *NAT traversal*. Usually this involves the application trying to ascertain the external IP address and port numbers that will be used when its traffic passes through a NAT and modifying its protocol operations accordingly. If an application is distributed across the network (e.g., has multiple clients and servers, some of which are not behind NATs), the servers can be used to shuttle (copy) data between the clients that connect from behind NATs or enable such clients to discover each other's NAT bindings and possibly facilitate direct communication. Using a server to copy data between clients is usually a last-resort option because of the overheads involved and potential for abuse. Consequently, most approaches attempt to provide for some method that allows direct communication.

Direct methods have been popular for peer-to-peer file sharing, games, and communication applications. However, such techniques are often confined to a particular application, meaning that each new distributed application requiring NAT traversal tends to implement its own method(s). This can lead to redundancy and interoperability problems, ultimately increasing users' frustration and cost. To combat this situation, a standard approach for handling NAT traversal has been established, and it depends on a collection of several distinct, subordinate protocols that we discuss in the following sections. For now, we begin with one of the more robust yet nonstandard approaches used by distributed applications. We then move on to standardized frameworks for NAT traversal.

7.4.1 Pinholes and Hole Punching

As discussed previously, a NAT typically includes both traffic rewriting and filtering capabilities. When a NAT mapping is established, traffic for a particular application is usually permitted to traverse the NAT in both directions. Such mappings are narrow; they usually apply only to a single application for its duration of execution. These types of mappings are called *pinholes*, because they are designed to permit only a certain temporary traffic flow (e.g., a pair of IP address and port number combinations). Pinholes are usually established and removed dynamically as a consequence of communication between programs.

A method that attempts to allow two or more systems, each behind a NAT, to communicate directly using pinholes is called *hole punching*. It is described for UDP in Section 3.3 of [RFC5128] and for TCP in Section 3.4. To punch a hole, a client contacts a known server using an outgoing connection that establishes a mapping in its local NAT. When another client contacts the same server, the server has connections to each of the clients and knows their external addressing information. It then exchanges the client external addressing information between the clients. Once this information is known, a client can attempt a direct connection to the other client(s). The popular Skype peer-to-peer application uses this approach (and some others).

Referring to Figure 7-7, assume client A contacts server S1 followed by client B. S1 will have learned A's and B's external addressing information: IPv4 addresses 192.0.2.201 and 203.0.113.100, respectively. By sending B's information to A and vice versa, A can attempt to contact B directly at its external address (and vice versa). Whether this will work depends on the type of NATs that have been deployed. NAT state for the (A,S1) connection lives in N1 and NAT state for (B,S1) lives in both N2 and N3. If all NATs are endpoint-independent, this is sufficient for direct connections to be possible. Any other type of NAT will not accept traffic from other than S1 and will thus prohibit direct communication. Said another way, this approach fails if both hosts are behind NATs with address-dependent or address- and port-dependent mapping behavior.

7.4.2 UNilateral Self-Address Fixing (UNSAF)

Applications employ a number of methods to determine the addresses their traffic will use when passed through a NAT. This is called *fixing* (learning and maintaining) the addressing information. There are indirect and direct methods for address fixing. The indirect methods involve inferring a NAT's behavior by exchanging traffic through the NAT. The direct methods involve a direct conversation between the application and the NAT itself using one or more special protocols (that are not currently IETF standards). Considerable effort within IETF has gone into development of the indirect methods, and they are widely supported in certain applications, with VoIP applications being the most popular. Some of the direct methods are now supported by some NATs. These methods also provide for

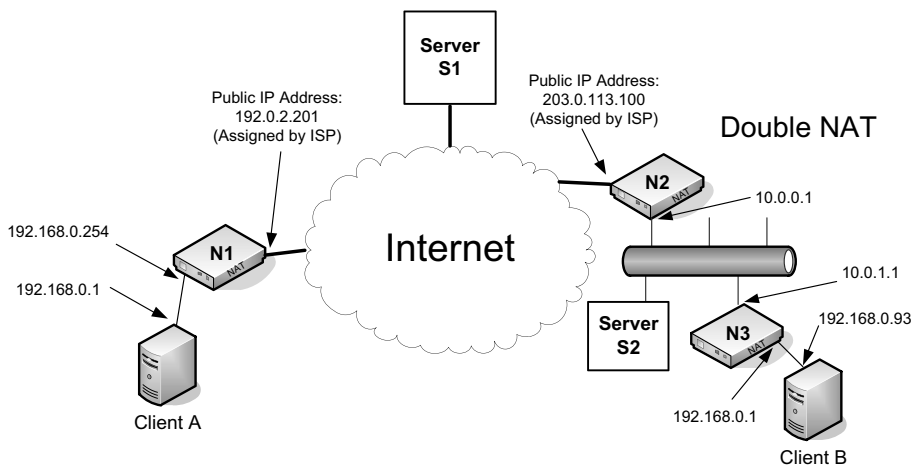


Figure 7-7 Applications running on clients behind a NAT may require help from a server to engage in direct communication. In hole punching, a server, often specialized for a specific application, provides rendezvous information among clients that first establish NAT state and then perform direct communication, if possible. Some applications attempt to “fix” (determine and maintain) the addresses (and port numbers) their traffic will be assigned when passing through a NAT using standard generic protocols. These methods may encounter troubles in certain situations such as environments with multiple levels of NAT. In this example, client A’s external address visible at S1 is 192.0.2.201 and client B’s is 203.0.113.100. At S2, however, B’s external address is 10.0.1.1.

basic configuration of NATs, so we discuss them later in the context of NAT setup and configuration.

An application attempting to fix its address without help from the NAT performs the address fixing in a so-called unilateral fashion. Applications that do so are said to perform *UNilateral Self-Address Fixing* (UNSAF) [RFC3424]. As the name suggests, such methods are considered to be undesirable in the long run but a necessary evil for the time being. UNSAF involves a set of heuristics and is not guaranteed to work in all cases, especially because NAT behaviors vary significantly based on vendor and particular circumstance. The BEHAVE documents mentioned earlier are aimed at specifying more consistent NAT behavior. If widely adopted, UNSAF methods will work more reliably.

In most cases of interest, UNSAF methods operate in a client/server fashion similar to hole punching, but with added generality. Figure 7-7 illustrates some of the hazards that can arise in this situation. One issue is the lack of a single “outside” address realm for every NAT. In this example, there are two levels of NAT between client B and server S1. This situation can cause complications. For example, if an application on B wishes to obtain its “outside” address by using UNSAF with a server, it receives different answers depending on whether it contacts server S1 or S2. Finally, because UNSAF uses servers that are distinct from

the NATs, there is always the possibility that the NAT behavior reported will change over time or become inconsistent with what the UNSAF approach reports.

Given the various problems with NATs and UNSAF, the IAB, an elected group of architectural advisers within the IETF, has indicated that UNSAF protocol proposals must include responses to the concerns in their specifications:

1. Define a limited-scope problem that the “short-term” UNSAF proposal addresses.
2. Define an exit strategy/transition plan.
3. Discuss what design decisions make the approach “brittle.”
4. Identify requirements for longer-term, sound technical solutions.
5. Discuss any noted practical issues or experiences known.

This is an unusual list of requirements imposed on a protocol specification, but it results from long-standing interoperability problems between different NATs and NAT traversal techniques. Despite all the aforementioned problems, UNSAF methods are commonly used, partly because a wide range of NATs are found in operation today with little consistent behavior. We now look at how these methods are used as building blocks to form robust, general-purpose NAT traversal techniques to maximize the chances that communication among systems behind NATs, even between systems across multiple NATs such as the one illustrated in Figure 7-7, will be possible.

7.4.3 Session Traversal Utilities for NAT (STUN)

One of the primary workhorses for UNSAF and NAT traversal is called *Session Traversal Utilities for NAT* (STUN) [RFC5389]. STUN has evolved from a previous version called *Simple Tunneling of UDP through NATs*, now known as “classic STUN.” Classic STUN has been used with VoIP/SIP applications for some time but has been revised to be a tool that can be used by other protocols for performing NAT traversal. Applications requiring a complete solution for NAT traversal are recommended to begin with other mechanisms we discuss in Section 7.4.5 (e.g., ICE and SIP-Outbound). These frameworks may make use of STUN in one or more particular ways called STUN *usages*. Usages may extend the set of basic STUN operations, message types, or error codes defined in [RFC5389].

STUN is a relatively simple client/server protocol that is able to ascertain the external IP address and port numbers being used on a NAT in most circumstances. It can also keep NAT bindings current by using keepalive messages. It requires a cooperating server on the “other” side of a NAT to be effective, and several public STUN servers are configured with globally reachable IP addresses and are available for use on the Internet. The main job of a STUN server is to echo back STUN requests sent to it in a way that allows the client addressing information to

be fixed. As with UNSAF methods in general, the approach is not foolproof. However, the attraction of STUN is that it does not require modification of network routers, application protocols, or servers. It requires only that clients implement the STUN request protocol, and that at least one STUN server be available in an appropriate location. STUN was envisioned as a “temporary” measure (as were many standard protocols now in widespread use a decade or more after their creation) until a more sophisticated direct protocol was developed and implemented, or NATs became obsolete because of the adoption of IPv6.

STUN operates using UDP, TCP, or TCP with *Transport Layer Security* (TLS; see Chapter 18). STUN usage specifications define which transport protocols are supported for the particular usage. It uses port 3478 for UDP and TCP, and 3479 for TCP/TLS. The STUN base protocol has two types of transactions: *request/response transactions* and *indication transactions*. Indications do not require a response and can be generated by either the client or the server. All messages include a type, length, magic cookie with value 0x2112A442, and a random 96-bit transaction ID used for matching requests with responses or for debugging. Each message begins with two 0 bits and may contain zero or more *attributes*. STUN message types are defined in the context of *methods* that support a particular STUN usage. The various STUN parameters, including method and attribute numbers, are maintained by the IANA [ISP]. Attributes have their own types and can vary in length. The basic STUN header, most often located immediately following a UDP transport header in an IP packet, is shown in Figure 7-8.

The basic STUN header is 20 bytes in length (see Figure 7-8), and the *Message Length* field provides for an entire STUN message length of $2^{16} - 1$ bytes (the 20-byte header length is not included in the *Message Length* field), although messages are always padded to a multiple of 4 bytes so this field always has its 2 low-order bits set to 0. STUN messages sent over UDP/IP are supposed to form IP datagrams less than the path MTU, if known, to avoid fragmentation (see Chapter 10). If not known, the entire datagram length (including IP and UDP headers and any options) should be less than 576 bytes (IPv4) or 1280 bytes (IPv6). STUN has no provision for cases where a response might exceed the path MTU in the reverse direction, so servers should arrange to use messages of appropriate size.

STUN messages carried over UDP/IP are not reliable, so STUN applications are required to implement their own reliability. This is accomplished by resending messages thought to be lost. The retransmission interval is based on the estimated time to send and receive a message from the peer called the *round-trip time* (RTT). RTT computation and setting retransmission timers will be a major consideration when we discuss TCP (see Chapter 14). STUN uses a similar approach, but with minor modifications to the standard TCP values. See [RFC5389] for more details. Reliability issues for STUN over TCP/IP or TCP-with-TLS/IP are handled by TCP. Multiple pending STUN transactions can be supported over TCP-based connections.

STUN attributes are encoded in a TLV arrangement, a technique used by several other Internet protocols. The type and length portions of a TLV are each 16

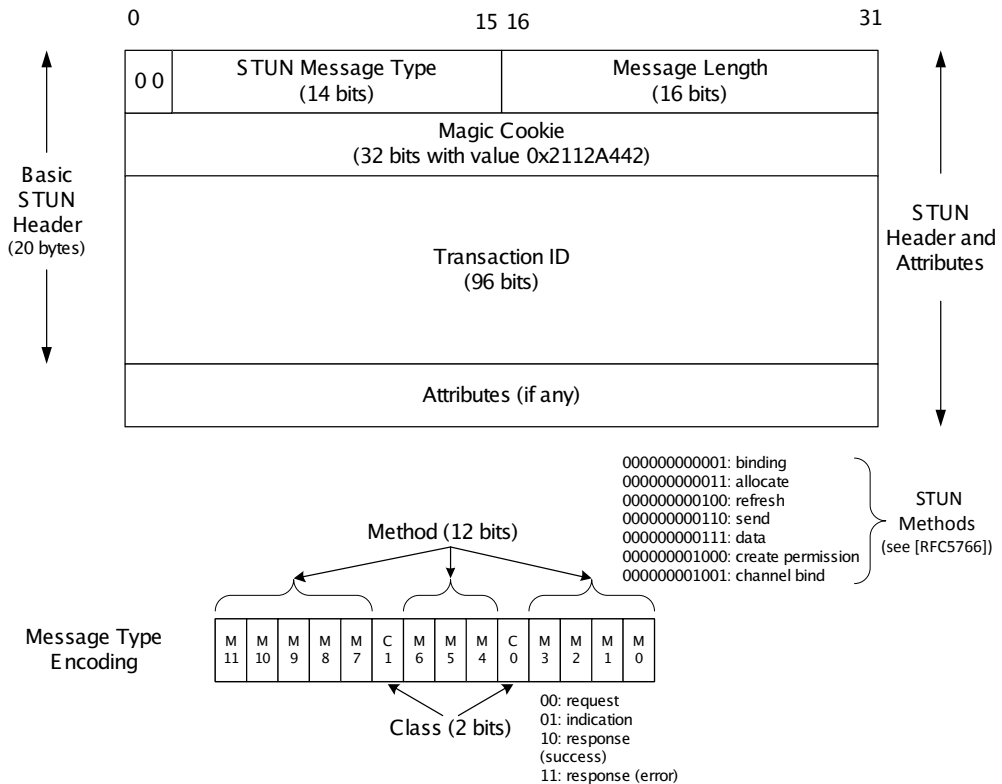


Figure 7-8 STUN messages always begin with two 0 bits and are usually encapsulated in UDP, although TCP is also allowed. The *Message Type* field gives both the method (e.g., binding) as well as class (request, response, error, or success). The *Transaction ID* is a random 96-bit number used to match requests with responses, or for debugging in the case of indications. Each STUN message can hold zero or more attributes, depending on the particular usage of STUN.

bits, and the value portion is variable-length (up to 64KB, if supported), but padded to the next multiple of 4 bytes (padding bits may be any value). The same attribute type may appear more than once in the same STUN message, although only the first is necessarily processed by a receiver. Attributes with type numbers below 0x8000 are called *comprehension-required* attributes, and the others are called *comprehension-optional* attributes. If a STUN agent receives a message containing comprehension-required attributes it does not know how to process, it generates an error. Most of the attributes defined to date are comprehension-required [ISP].

[RFC5389] defines a single STUN method called *binding*, which can be used in either request/response or indication transactions for address fixing and keeping NAT bindings current. It also defines 11 attributes, given in Table 7-2.

Table 7-2 STUN, defined in [RFC5389] and sometimes called STUN2, replaces classic STUN. These 11 attributes may be used by a STUN2-compliant client or server.

Name	Value	Purpose/Use
MAPPED-ADDRESS	0x0001	Contains an address family indicator and the reflexive transport address (IPv4 or IPv6)
USERNAME	0x0006	User name and password; used for message integrity checks (up to 513 bytes)
MESSAGE-INTEGRITY	0x0008	Message authentication code value on the STUN message (see Chapter 18 and [RFC5389])
ERROR-CODE	0x0009	Contains 3-bit error class, 8-bit error code value, and variable-length textual description of error
UNKNOWN-ATTRIBUTES	0x000A	Used with error messages to indicate the unknown attributes (one 16-bit value per attribute)
REALM	0x0014	Indicates the authentication “realm” name for long-term credentials
NONCE	0x0015	Nonrepeated value optionally carried in requests and responses (see Chapter 18) to prevent replay attacks
XOR-MAPPED-ADDRESS	0x0020	XORed version of MAPPED-ADDRESS
SOFTWARE	0x8022	Textual description of the software that sent the message (e.g., manufacturer and version number)
ALTERNATE-SERVER	0x8023	Provides an alternate IP address for a client to use; encoded as with MAPPED-ADDRESS
FINGERPRINT	0x8028	CRC-32 of message XORed with 0x5354554E; must be last attribute if used (optional)

Referring to Figure 7-5, a STUN client with addressing information $X:x$ is often interested in determining $X1':x1'$, called the *reflexive transport address* or *mapped address*. A STUN server at $Y1:y1$ includes the reflexive transport address in a MAPPED-ADDRESS attribute in a STUN message returned to the client. The MAPPED-ADDRESS attribute holds an 8-bit *Address Family* field, a 16-bit *Port Number* field, and either a 32-bit or 128-bit *Address* field, depending on whether IPv4 or IPv6 is indicated by the *Address Family* field (0x01 for IPv4; 0x02 for IPv6). This attribute is included to remain backward-compatible with classic STUN. The more important attribute is the XOR-MAPPED-ADDRESS attribute, which holds exactly the same value as the MAPPED-ADDRESS attribute, but XORed with the magic cookie value (for IPv4) or a concatenation with the magic cookie and transaction ID values (for IPv6). The reason for using XORed values in this way is to detect and bypass generic ALGs that look through packets and rewrite whatever IP addresses they find. Such ALGs are very brittle because they may rewrite information that protocols such as STUN require. Experience has shown that XORing IP addresses in the packet payload is usually sufficient to bypass such ALGs.

A STUN client, including most VoIP devices and “softphone” applications such as pjsua [PJSUA], is initially configured with the IP address(es) or names of one or more STUN servers. It is desirable to use STUN servers that are likely to “see” the same IP addresses as the peer to which the application ultimately wishes to talk, although that may be difficult to determine. Using STUN servers located on the public Internet (e.g., stun.ekiga.net, stun.xten.com, numb.viagenie.ca) is usually adequate. Some servers may be discovered using DNS Service (SRV) records (see Chapter 11). An example STUN binding request is given in Figure 7-9.

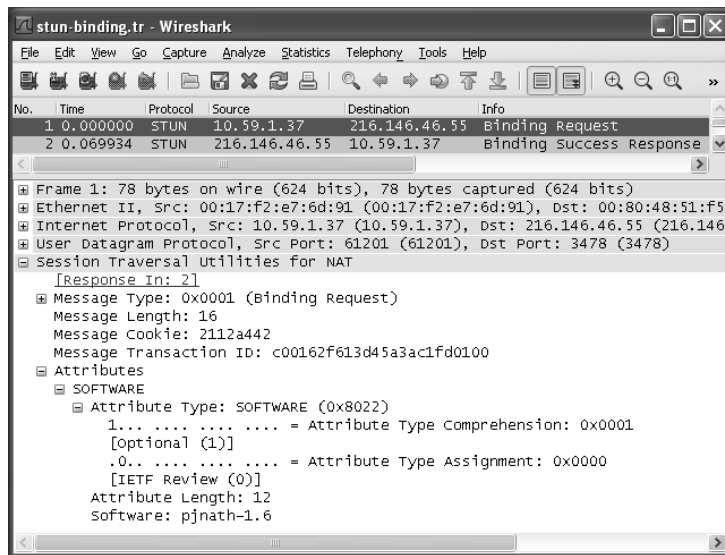


Figure 7-9 A STUN binding request. The request contains a 96-bit transaction ID and the SOFTWARE attribute that identifies the client making the request. The attribute contains 10 characters, but this value is rounded up to the next multiple of 4, giving an attribute value of 12. The message length of 16 also includes the 4 bytes used to include the attribute’s type and length (the STUN header is not included).

The sample STUN binding request in Figure 7-9 is initiated from a client. The transaction ID has been selected randomly, and the request is sent to numb.viagenie.ca (with IPv4 addresses 216.146.46.55 and 216.146.46.59), which is both a STUN and a TURN server (see Section 7.4.4). The request contains the SOFTWARE attribute that identifies the client application. In this case, the request was initiated by `pjnath-1.6`. This is the “PJSIP NAT helper” application included with `pjsua`. The message length includes 4 bytes for the attribute type and length, plus 12 bytes used to hold the attribute. The length of `pjnath-1.6` is only 10 bytes, but attribute lengths are always rounded up to the nearest 4-byte multiple. After passing through a NAT, the response is given as shown in Figure 7-10.

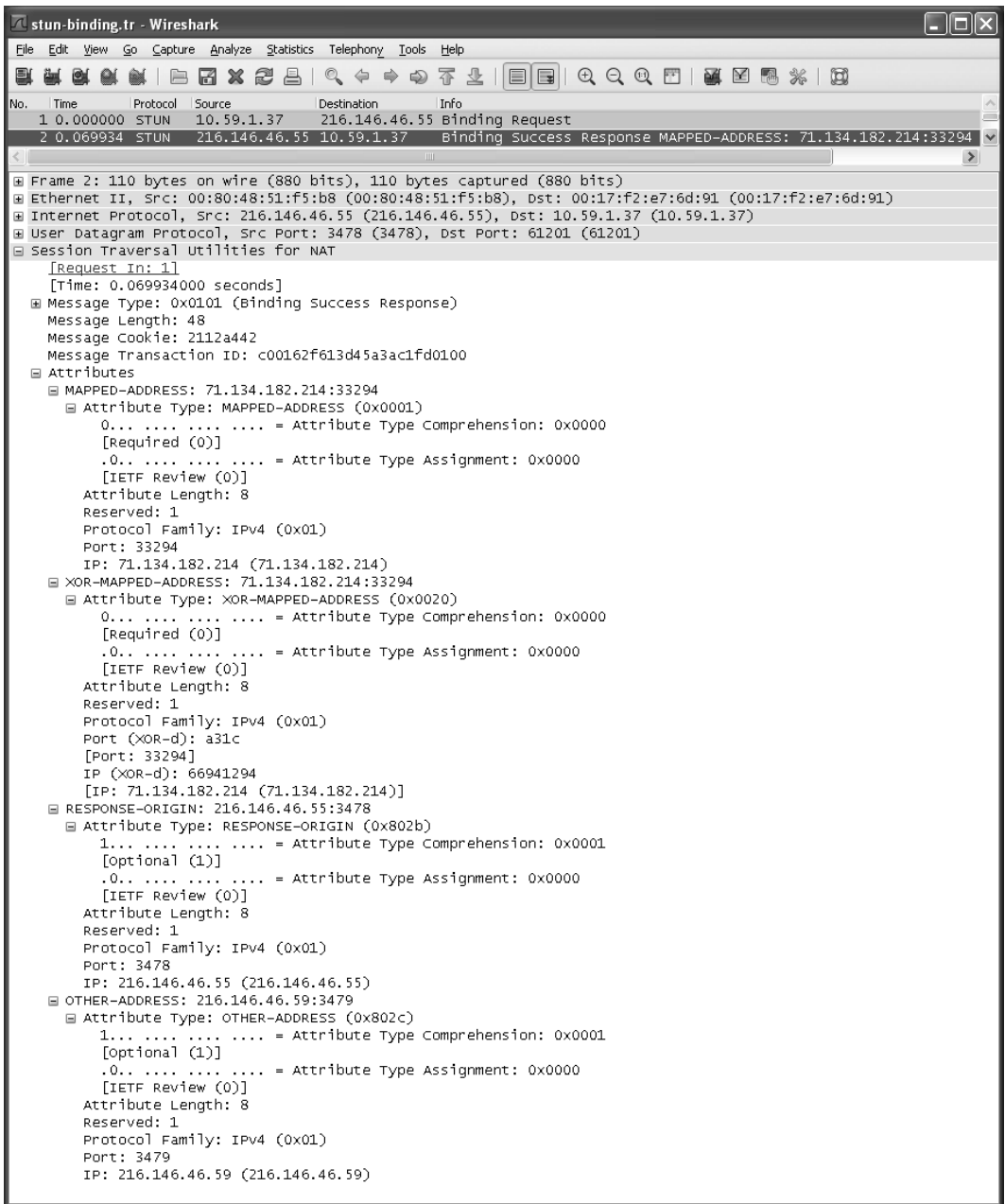


Figure 7-10 A STUN binding response containing four attributes. The MAPPED-ADDRESS and XOR-MAPPED-ADDRESS attributes contain the server-reflexive addressing information. The other attributes are used with an experimental NAT behavior discovery mechanism [RFC5780].

The binding response shown in Figure 7-10 gives useful information to the client, encoded as a collection of attributes. The MAPPED-ADDRESS and XOR-MAPPED address attributes indicate that the STUN server determined the server-reflexive address of 71.134.182.214:33294. The RESPONSE-ORIGIN and OTHER-ADDRESS attributes are used by an experimental facility for discovering NAT behavior [RFC5780]. The first gives the communication endpoint used to send the STUN message (216.146.46.55:3478, which matches the sending IPv4 address and UDP port number). The second attribute indicates which source IPv4 address and port number (216.146.45.59:3479) would have been used if the client requested “change address” or “change port” behavior. This latter attribute is equivalent to the now-deprecated CHANGED-ADDRESS attribute in classic STUN. If a change address or port is specified in a request, a cooperating STUN server attempts to use a different address when responding to the client, if possible.

STUN can be used to perform address fixing as well as a number of other functions called *mechanisms*, including DNS discovery, a method to redirect to an alternate server, and message integrity exchanges. Mechanisms are selected in the context of a particular STUN usage, so in general they are considered optional STUN features. One of the more important mechanisms provides authentication and message integrity. It has two forms: the *short-term credential mechanism* and the *long-term credential mechanism*.

Short-term credentials are intended to last for a single session; the particular duration is defined by the STUN usage. Long-term credentials last across sessions; they correspond to a login ID or account. Short-term credentials are often used in particular message exchanges, and long-term credentials are used when some particular resource is to be allocated (e.g., with TURN; see Section 7.4.4). Passwords are never sent in the clear where they could be intercepted.

The short-term credential mechanism uses the USERNAME and MESSAGE-INTEGRITY attributes. Both are required on any request. The USERNAME gives an indication of which credentials are required and allows the message sender to use the appropriate shared password in forming an integrity check on the message (a MAC computed on the message contents; see Chapter 18). When using short-term credentials, it is assumed that some form of credential information (e.g., user name and password) has been exchanged earlier. The credential is used for forming an integrity check on STUN messages that is encoded in the MESSAGE-INTEGRITY attribute. The ability to form a valid MESSAGE-INTEGRITY attribute value is an indication that the sender holds a current (“fresh”) copy of the appropriate credential.

The long-term credential mechanism ensures freshness in a different way, using a *digest challenge*. When using this mechanism, a client initially makes a request without any authentication material. The server rejects the request but provides a REALM attribute in response. This can be used by the client to determine which credential is needed to provide adequate authentication, as the client may have credentials for various services (e.g., multiple VoIP accounts). Along with the REALM, the server provides a never-reused NONCE value, which the client uses in

forming a subsequent request. This mechanism also uses a MESSAGE-INTEGRITY attribute, but its integrity function is computed by including the NONCE value. Thus, it is difficult for an eavesdropper that overheard a previous long-term credential exchange to simply replay a validated request (i.e., because the NONCE value is different). The use of NONCE values in authentication and related concerns are discussed in more detail in Chapter 18. The long-term credential mechanism cannot be used to protect STUN indications, as these transactions do not operate as request/response pairs.

7.4.4 Traversal Using Relays around NAT (TURN)

Traversal Using Relays around NAT (TURN) [RFC5766] provides a way for two or more systems to communicate even if they are located behind relatively uncooperative NATs. As a last-resort method to support communication in such circumstances, it involves a relay server that shuttles data between systems that could otherwise not communicate. Using extensions to STUN and some TURN-specific messages, it supports communication even when most other approaches have failed, provided a common server that is not behind a NAT can be reached by each client. If all NATs were compliant with the BEHAVE specifications, TURN would not be necessary. Direct communication methods (i.e., that do not use TURN) are almost always preferable to using TURN servers.

Referring to Figure 7-11, a TURN client behind a NAT contacts a TURN server, usually on the public Internet, and indicates the other systems (called *peers*) with which it wishes to communicate. Finding the server's address and the appropriate protocol to use for communication is accomplished using a special DNS NAPTR record (see Chapter 11 and [RFC5928]) or by manual configuration. The client obtains address and port information, called the *relayed transport address*, from the server, which are the address and port number used by the TURN server to communicate with the peers. The client also obtains its own server-reflexive transport address. Peers also have server-reflexive transport addresses that represent their external addresses. These addresses are needed by the client and server to perform the "plumbing" necessary to interconnect the client and its peers. The method used to exchange this addressing information is not defined within the scope of TURN. Instead, this information must be exchanged using some other mechanism (e.g., ICE; see Section 7.4.5) in order for TURN servers to be used effectively.

The client uses TURN commands to create and maintain *allocations* on the server. An allocation resembles a multiway NAT binding and includes the (unique) relayed transport address that each peer can use to reach the client. Server/peer data is sent using straightforward TURN messages traditionally carried in UDP/IPv4. Enhancements support TCP [RFC6062] and IPv6 (and also relaying between IPv4 and IPv6) [RFC6156]. Server/client data is encapsulated with an indication of corresponding peer(s) that sent or should receive the associated data. The client/server connection has been specified for UDP/IPv4, TCP/IPv4, and TCP/IPv6

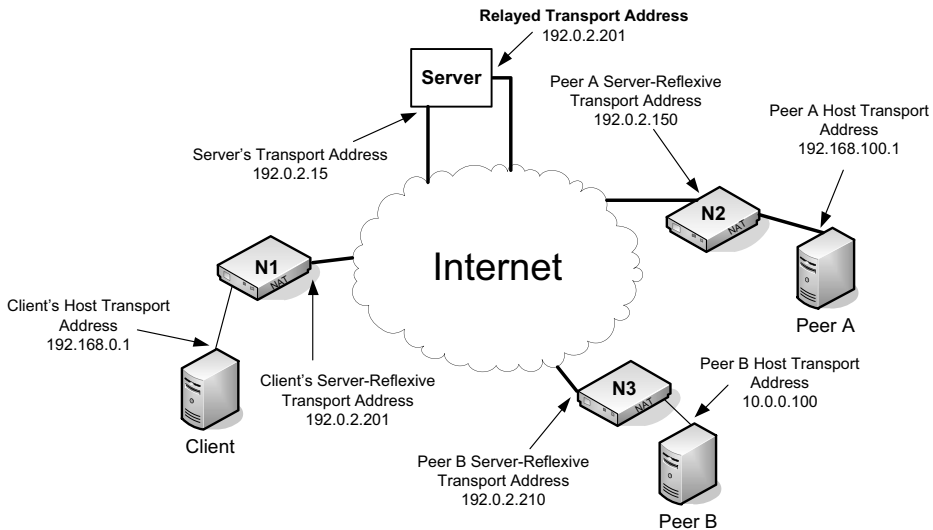


Figure 7-11 Based on [RFC5766], a TURN server helps clients behind “bad” NATs to communicate by relaying traffic. Traffic flowing between client and server may use TCP, UDP, or TCP with TLS. Traffic between the server and one or more peers uses UDP. Relaying is a last-resort measure for communication; direct methods are preferred if available.

with TLS. Establishing an allocation requires the client to be authenticated, usually using the STUN long-term credential mechanism.

TURN supports two methods for copying data between a client and its peers. The first encodes data using STUN methods called Send and Data, defined in [RFC5766], which are STUN indicators and therefore not authenticated. The other uses a TURN-specific concept called *channels*. Channels are communication paths between a client and a peer that have less overhead than the Send and Data methods. Messages carried over channels use a smaller, 4-byte header that is incompatible with the larger STUN-formatted messages ordinarily used by TURN. Up to 16K channels can be associated with an allocation. Channels were developed to help some applications such as VoIP that prefer to use relatively small packets to reduce latency and overhead.

In operation, the client makes a request to obtain an allocation using a TURN-defined STUN Allocate method. If successful, the server responds with a success indicator and the allocated relayed transport address. A request might be denied if the client fails to provide adequate authentication to the server. The client must now send refresh messages to keep the allocation alive. Allocations expire in 10 minutes if not refreshed, unless the client included an alternate lifetime value, encoded as a STUN LIFETIME attribute, in the allocation request. Allocations may be deleted by requesting an allocation with zero lifetime. When an allocation expires, so do all of its associated channels.

Allocations are represented using a “5-tuple.” At the client, the 5-tuple includes the client’s host transport address and port number, server transport address and port number, and the transport protocol used to communicate with the server. At the server, the same 5-tuple is used, except the client’s host transport address and port are replaced with its server-reflexive address and port. An allocation may have zero or more associated *permissions*, to limit the patterns of connectivity that are permitted through the TURN server. Each permission includes an IP address restriction such that only packets with the matching source address received at the TURN server have their data payloads forwarded to the corresponding client. Permissions are deleted if not refreshed within 5 minutes.

TURN enhances STUN with six methods, nine attributes, and six error response codes. These can be partitioned roughly into support for establishing and maintaining allocations, authentication, and manipulating channels. The six methods and their method numbers are as follows: Allocate (3), Refresh (4), Send (6), Data (7), CreatePermission (8), and ChannelBind (9). The first two establish and keep allocations alive. Send and Data use STUN messages to encapsulate data from client to server and vice versa, respectively. CreatePermission establishes or refreshes a permission, and ChannelBind associates a particular peer with a 16-bit channel number. The error messages indicate problems with TURN features such as authentication failure or running out of resources (e.g., channel numbers). The nine STUN attribute names, values, and purposes defined by TURN are given in Table 7-3.

Table 7-3 STUN attributes defined by TURN

Name	Value	Purpose/Use
CHANNEL-NUMBER	0x000C	Indicates what channel associated data belongs to
LIFETIME	0x000D	Requested allocation timeout (seconds)
XOR-PEER-ADDRESS	0x0012	A peer’s address and port, using XORed encoding
DATA	0x0013	Holds data for a Send or Data indication
XOR-RELAYED-ADDRESS	0x0016	Server’s address and port allocated for a client
EVEN-PORT	0x0018	Requests that the relayed transport addressing information use an even port; optionally requests allocation of the next port in sequence
REQUESTED-TRANSPORT	0x0019	Used in a client to request that a specific transport be used in forming the transport address; values are drawn from the IPv4 <i>Protocol</i> or IPv6 <i>Next Hop</i> header field values
DONT-FRAGMENT	0x001A	Requests that the server set the “don’t fragment” bit in the IPv4 header in packets sent to peers
RESERVATION-TOKEN	0x0022	Unique identifier for a relayed transport address held by the server; the value is provided to the client as a reference

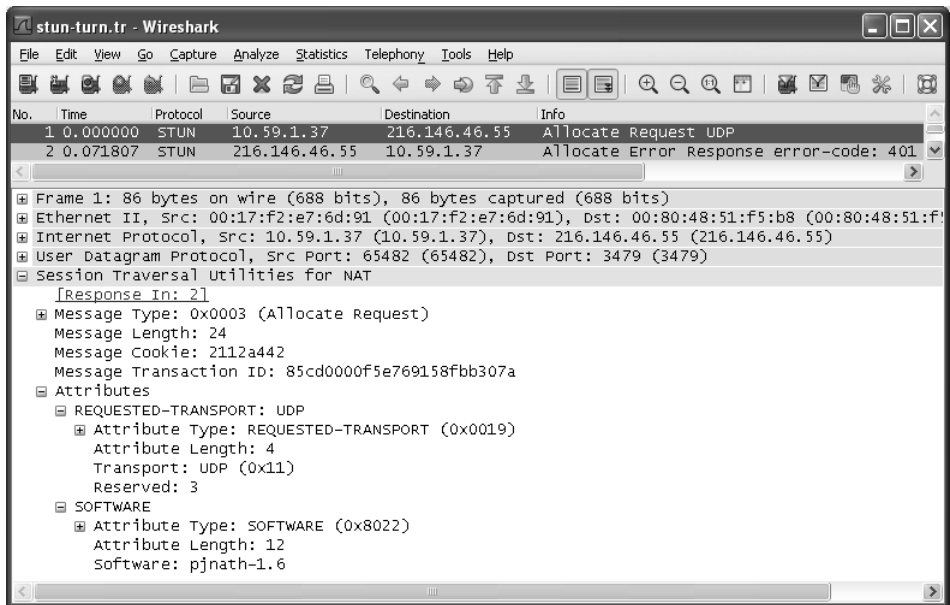


Figure 7-12 A TURN allocation request is a STUN message using message type 0x0003. This request also includes the REQUESTED-TRANSPORT and SOFTWARE attributes. It does not include authentication information. According to STUN long-term credentials, this request will fail.

A TURN request takes the form of a STUN message whose message type is an allocation request. Figure 7-12 shows an example. According to the STUN long-term credential mechanism, the initial allocation request shown in Figure 7-12 did not include authentication information, so it is rejected by the server. The rejection is indicated by an allocation error response, shown in Figure 7-13.

The error message in Figure 7-13 provides the REALM attribute (*viagenie.ca*) and the NONCE value the client requires to form its next request. The message also includes the MESSAGE-INTEGRITY attribute so the client can check that the message has not been modified and the requested REALM and NONCE are correct. A subsequent request includes the USERNAME, NONCE, and MESSAGE-INTEGRITY attributes. See Figure 7-14.

After receiving the request including long-term credentials, as shown in Figure 7-14, the server computes its own version of the message integrity value and compares the result against the MESSAGE-INTEGRITY attribute value. If they match, this is sufficient information for the TURN server to conclude that the client must hold the appropriate password. It then permits the allocation and indicates the result to the client (see Figure 7-15).

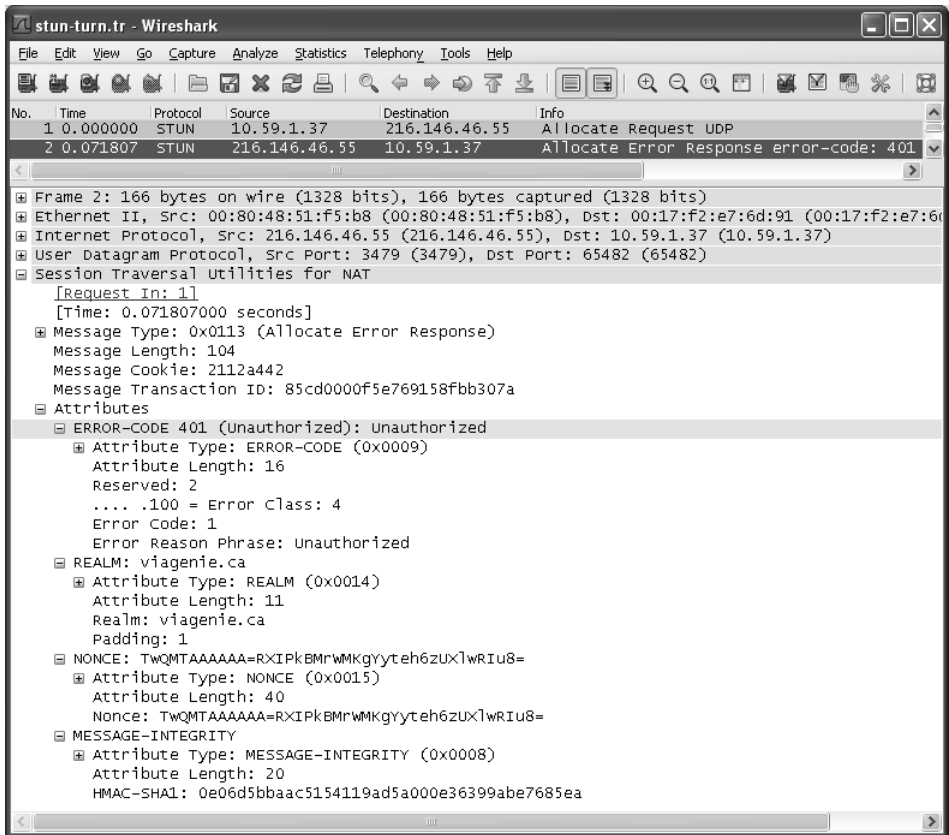


Figure 7-13 A TURN allocation error response includes the ERROR-CODE attribute with value 401 (Unauthorized). The message is integrity-protected and includes the REALM and NONCE attributes required by the client in forming another, authenticated allocation request.

The allocation request is successful, as shown in Figure 7-15, and the relayed transport address is 216.146.46.55:49261 (note that Wireshark performs the XOR operation to display the decoded address). At this point, the client can proceed to use the TURN server for relaying to peers. Once this is finished, the allocation can be removed. About 4s later, packets 5 and 6 in Figure 7-15 indicate the client's request to remove the allocation. The request is expressed as a refresh with lifetime set to 0. The server responds with a success indicator and removes the allocation. Note that the BANDWIDTH attribute has been included in the allocation and refresh success indicators. This attribute, defined by a draft version of [RFC5766] but ultimately deprecated, was intended to hold the peak bandwidth, in kilobytes per second, permitted on the allocation. This attribute may be redefined in the future.

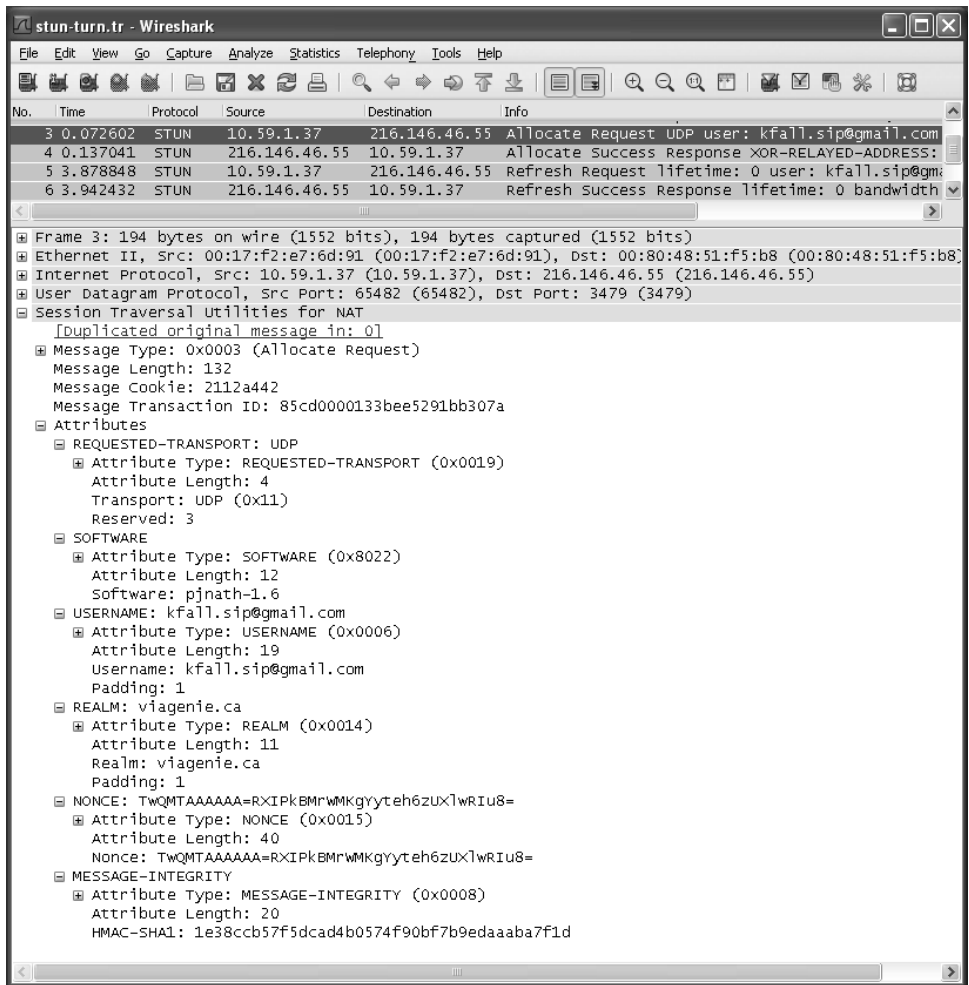


Figure 7-14 A second TURN allocation request includes the USERNAME, REALM, NONCE, and MESSAGE-INTEGRITY attributes. These are used by the server to verify integrity of the message and the identity of the client. If successful, the server authenticates the request and performs the allocation.

As suggested previously, TURN has the disadvantage that traffic must be relayed through the TURN server, and this can lead to inefficient routing (i.e., the TURN server may be far away from a client and peer that are proximal). In addition, certain other traffic contents are not passed through from peer to client using TURN. This includes ICMP values (see Chapter 8), *TTL (Hop Limit)* field values, and IP *DS Field* values. Also, a requesting TURN client must implement the STUN long-term credential mechanism and have some form of login credential or account assigned by the TURN server operator. This helps to avoid uncontrolled use of open TURN servers but creates somewhat greater configuration complexity.

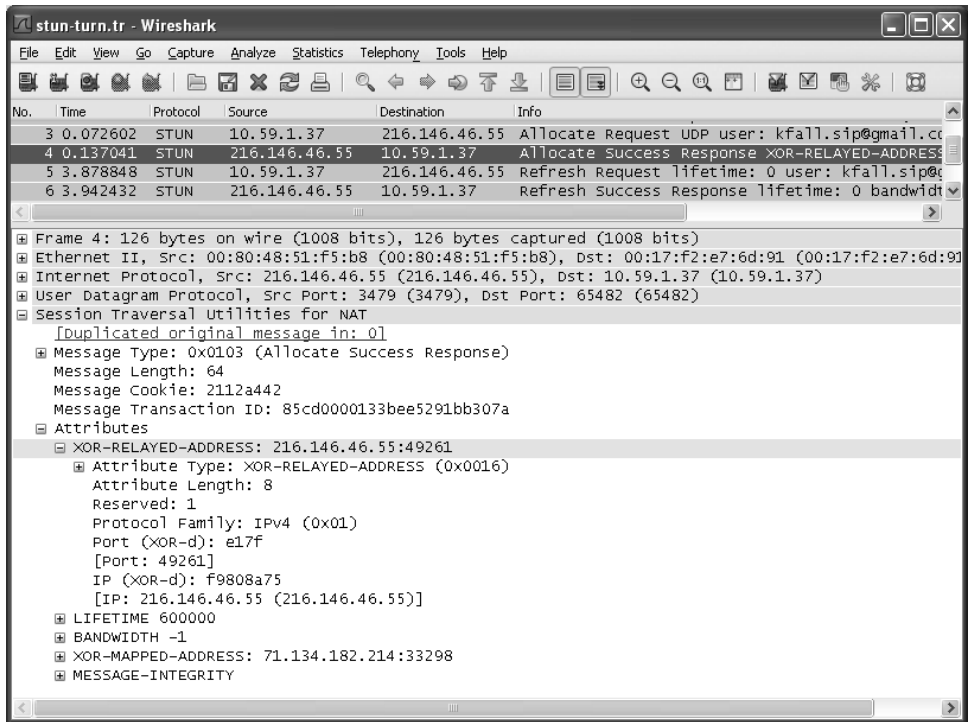


Figure 7-15 A TURN allocation success response. The message is integrity-protected and includes the XOR-RELAYED-ADDRESS attribute, identifying the port and address allocated by the TURN server. The allocation is deleted if not refreshed.

7.4.5 Interactive Connectivity Establishment (ICE)

Given the large variety of NATs deployed and the various mechanisms that may be necessary to traverse them, a generic facility called *Interactive Connectivity Establishment* (ICE) [RFC5245] has been developed to help UDP-based applications hosted behind a NAT establish connectivity. ICE is a set of heuristics by which an application can perform UNSAF in a relatively predictable fashion. In its operation, ICE makes use of other protocols such as TURN and STUN. A proposal extends the use of ICE to TCP-based applications [IDTI].

ICE works with and extends “offer/answer” protocols, such as the Session Description Protocol (SDP) used with unicast SIP connection establishment [RFC3264]. These protocols involve an offer of service with an accompanying set of service parameters followed by an answer that also includes a set of selected options. It is increasingly common to find ICE clients incorporated into VoIP applications that use SDP/SIP for establishing communications. However, in such circumstances, ICE is used for establishing NAT traversal for media streams (such as the audio or video portion of a call carried using RTP [RFC3550] or SRTP

[RFC3711]), while another mechanism, called *SIP Outbound* [RFC5626], handles the SIP signaling information such as who is being called. Although in practice ICE has been used primarily with SIP/SDP-based applications, it can also be used as a generic NAT traversal mechanism for other applications. One such example is the use of ICE (over UDP) with *Jingle* [XEP-0176], defined as an extension to the core *Extensible Messaging and Presence Protocol* (XMPP) [RFC6120].

Ordinarily, ICE works to establish communication between two SDP entities (called *agents*) by first determining a set of *candidate transport addresses* that each agent might use for communicating with the other. Referring to Figure 7-11, these addresses could be host transport, server-reflexive, or relayed addresses. ICE may make use of both STUN and TURN to determine the candidate transport addresses. ICE then orders these addresses according to a priority assignment algorithm. The algorithm arranges for addresses that provide direct connectivity to receive greater priority than those that require data relaying. ICE then provides the set of prioritized addresses to its peer agent, which engages in a similar behavior. Ultimately, two agents agree on the best set of usable address pairs and indicate the selected results to the other peer. Determination of which candidate transport addresses are available is accomplished using a sequence of *checks* encoded as STUN messages. ICE has several optimizations to decrease the latency of agreeing on the selected candidate, which are beyond the scope of this discussion.

ICE begins by attempting to discover all available candidate addresses. Addresses may be locally assigned transport addresses (multiple if the agent is multihomed), server-reflexive addresses, or relayed addresses determined by TURN. After assigning each address a priority, an agent sends the prioritized list to its peer using SDP. The peer performs the same operation, resulting in each agent having two prioritized lists. Each agent then forms an identical set of prioritized *candidate pairs* by pairing up the two lists. A set of checks are performed on the candidate pairs in a particular order to determine which addresses will ultimately be selected. Generally, the priority ordering prefers candidate pairs with fewer NATs or relays. The candidate pair ultimately selected is determined by a *controlling agent* assigned by ICE. The controlling agent *nominates* which valid candidate pairs are to be used, according to its order of preference. The controlling agent may try all pairs and subsequently make its choice (called *regular nomination*) or may use the first viable pair (called *aggressive nomination*). A nomination is expressed as a flag in a STUN message referring to a particular pair; aggressive nomination is performed by setting the nominate flag in every request.

Checks are sent as STUN binding request messages exchanged between the two agents using the addressing information being checked. Checks are initiated by timer, or scheduled as a result of an incoming check from a peer (called a *triggered check*). Responses arrive in the form of STUN binding responses that contain addressing information. In some circumstances this may reveal a new server-reflexive address to the agent (e.g., because a different NAT is used between agents from the one that was used when the candidate addresses were first determined using STUN or TURN servers). Should this happen, the agent gains a new address

called a *peer-reflexive candidate*, which ICE adds to the set of candidate addresses. ICE checks are integrity-checked using STUN's short-term credential mechanism and use the STUN FINGERPRINT attribute. When TURN is used, the ICE client uses TURN permissions to limit the TURN binding to the remote candidate address of interest.

ICE incorporates the concept of different implementations. *Lite* implementations are designed for deployment in systems that do not employ NAT. They do not ever act as a controlling agent unless interacting with another Lite implementation. They also do not perform the checks mentioned earlier as do *full* implementations. The type of an ICE implementation is indicated in the STUN messages it sends. All ICE implementations must comply with STUN [RFC5389], but Lite implementations will only ever act as STUN servers. ICE extends STUN with the attributes described in Table 7-4.

Table 7-4 STUN attributes defined by ICE

Name	Value	Purpose/Use
PRIORITY	0x0024	Computed priority of associated candidate address
USE-CANDIDATE	0x0025	Indicates selection of candidate by controlling agent
ICE-CONTROLLED	0x8029	Indicates sender of message is controlled agent
ICE-CONTROLLING	0x802A	Indicates sender of message is controlling agent

A check is a STUN binding request containing the PRIORITY attribute. The value is equal to the value assigned by the algorithm described in Section 4.1.2 of [RFC5245]. The ICE-CONTROLLING and ICE-CONTROLLED attributes are included in STUN requests when the sender is the controlling or controlled agent, respectively. A controlling agent may also include a USE-CANDIDATE attribute. If present, this attribute indicates which candidate the controlling agent wishes to select for subsequent use.

7.5 Configuring Packet-Filtering Firewalls and NATs

Although NATs frequently require little configuration (unless port forwarding is being used), firewalls usually do, and sometimes they require extensive configuration. In most home networks the same device is providing NAT, IP routing, and firewall capabilities and may require some configuration. Although the configuration is logically separate for each of these, they are sometimes merged, either in configuration files, command-line interfaces, Web page controls, or other network management tools.

9

Broadcasting and Local Multicasting (IGMP and MLD)

9.1 Introduction

We mentioned in Chapter 2 that there are four kinds of IP addresses: *unicast*, *anycast*, *multicast*, and *broadcast*. IPv4 may use all of them, and IPv6 uses any except the last form. In this chapter we discuss broadcasting and multicasting in more detail, including how link-layer addressing can be used to send multicast or broadcast traffic efficiently from one computer to several others. We also examine the *Internet Group Management Protocol* (IGMP) [RFC3376] and the IPv6 Multicast Listener Discovery (MLD) [RFC3810] protocols, which are used to inform IPv4 and IPv6 multicast routers which multicast addresses are in use on a subnetwork. One topic we do not cover in this chapter (or this book) is how multicast routing is implemented in wide area networks such as the global Internet. At the present time, multicast is used more in enterprise and local networks than in the wide area. While the protocols we discuss in this chapter are prerequisites for a complete understanding of wide area multicasting, the wide area routing protocols are comparatively complex and would unnecessarily complicate the explanation of the important local area case. The reader interested in exploring these issues is referred to [EGW02].

Broadcasting and multicasting provide two services for an application: delivery of packets to multiple destinations, and solicitation/discovery of servers by clients.

- **Delivery to multiple destinations**

There are many applications that deliver information to multiple recipients: interactive conferencing and dissemination of mail or news to multiple recipients, for example. Without broadcasting or multicasting, these types of services tend to use TCP today (delivering a separate copy to each destination, which can be very inefficient).

- **Solicitation of servers by clients**

Using broadcasting or multicasting, an application can send a request for a server without knowing any particular server's IP address. This capability is very useful during configuration when little is known about the local networking environment. A laptop, for example, might need to get its initial IP address and find its nearest router using DHCP (see Chapter 6).

Although both broadcasting and multicasting can provide these important capabilities, multicasting is generally preferable to broadcasting because multicasting involves only those systems that support or use a particular service or protocol, and broadcasting does not. Thus, a broadcast request affects *all* hosts that are reachable within the scope of the broadcast, whereas multicast affects only those hosts that are likely to be interested in the request. These concepts will become clearer as we explore the details of broadcasting and multicasting. For now, keep in mind that there is a trade-off between the higher overhead and simplicity of broadcast and the improved efficiency but greater complexity associated with multicast.

Broadcasting has been supported by the IPv4 protocol since its inception, and multicast was added with the publication of [RFC1112]. IPv6 supports multicasting but does *not* support broadcasting. Generally, only user applications that use the UDP transport protocol (Chapter 10) take advantage of broadcasting and multicasting, where it makes sense for an application to send a single message to multiple recipients. TCP is a connection-oriented protocol that implies a connection between two hosts (specified by IP addresses) and one process on each host (specified by port numbers). TCP can use unicast and anycast addresses (recall that anycast addresses behave like unicast addresses), but not broadcast or multicast addresses.

Note

Broadcasting and multicasting are also used by important system processes such as routing protocols, ARP, ND in IPv6, and others. Although IP multicasting support was once an "add-on," requiring users to patch their systems to make use of it, modern operating systems include the capability by default. Multicasting is an important but arguably optional feature in IPv4, but it is mandatory in IPv6 because of its use in ND (see Chapter 8), a service critical even to unicast communication.

9.2 Broadcasting

Broadcasting refers to sending a message to all possible receivers in a network. In principle, this is simple: a router simply forwards a copy of any message it receives out of every interface other than the one on which the message arrived. Things are slightly more complicated when multiple hosts are attached to the same local area

network. In this case, features of the link layer may be used to make broadcasting somewhat more efficient.

Consider a set of hosts on a network such as an Ethernet that supports broadcasting at the link layer. Each Ethernet frame contains the source and destination MAC addresses (48-bit values). Normally, each IP packet is destined for a single host, so unicast addressing is used and the destination's unique MAC address is determined using ARP or IPv6 ND. When a frame is sent to a unicast destination in this way, communication between any two hosts does not bother any of the remaining hosts on the network. For switched Ethernet networks, these are the types of addresses found in the station caches in switches and bridges (see Chapter 3). There are times, however, when a host wants to send a frame to every other host on the network (or VLAN)—this is called a *broadcast*. We saw this with ARP in Chapter 4.

9.2.1 Using Broadcast Addresses

On an Ethernet or similar network, a multicast MAC address has the low-order bit of the high-order byte turned on. In hexadecimal this looks like 01:00:00:00:00:00. We may consider the Ethernet broadcast address ff:ff:ff:ff:ff:ff as a special case of the Ethernet multicast address. From Chapter 2 recall that in IPv4, each subnet has a local subnet-directed broadcast address formed by placing all 1 bits in the host portion of the address, and the special address 255.255.255.255 corresponds to a local network (also called “limited”) broadcast.

9.2.1.1 Example

In Linux, the IPv4 subnet-directed broadcast address associated with each interface can be found or set with the `ifconfig` command. We can see it displayed as follows:

```
Linux% ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:08:74:93:C8:3C
          inet addr:10.0.0.13  Bcast:10.0.0.127  Mask:255.255.255.128
          inet6 addr: 2001:5c0:9ae2:0:208:74ff:fe93:c83c/64
                  Scope:Global
          inet6 addr: fe80::208:74ff:fe93:c83c/64
                  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:426469 errors:0 dropped:0 overruns:1 frame:0
          TX packets:779338 errors:0 dropped:0 overruns:0 carrier:0
          collisions:298048 txqueuelen:1000
          RX bytes:44414543 (42.3 MiB)  TX bytes:1094425223 (1.0 GiB)
          Interrupt:19 Base address:0xec00
```

Here, the address 10.0.0.127 is the (subnet-directed) broadcast address used on the network to which device `eth0` is attached. This address is formed by taking the network prefix (10.0.0.0/25) and combining it with $32 - 25 = 7$ bits of 1s in

the host portion of the address: 10.0.0.0 OR 0.0.0.127 = 10.0.0.127. A simple utility called `ipcalc` is available on some systems to perform this calculation.

To see how simple broadcasting works, we can send an ICMPv4 Echo Request message using the `ping` program to the broadcast address of 10.0.0.127 indicated by the output of the `ifconfig` command:

```
Linux# ping -b 10.0.0.127
WARNING: pinging broadcast address
PING 10.0.0.127 (10.0.0.127) 56(84) bytes of data.
64 bytes from 10.0.0.6: icmp_seq=1 ttl=64 time=1.05 ms
64 bytes from 10.0.0.113: icmp_seq=1 ttl=64 time=1.55 ms (DUP!)
64 bytes from 10.0.0.120: icmp_seq=1 ttl=64 time=3.09 ms (DUP!)

--- 10.0.0.127 ping statistics ---
1 packets transmitted, 1 received, +2 duplicates,
0% packet loss, time 0ms
```

We mentioned in Chapter 8 that in this type of broadcast, all the hosts on the local LAN (or VLAN) are affected. Here we receive replies from three other hosts on the network, and the `ping` program notes that more responses were received than the number of requests sent (the `DUP!` indication). To see the addresses being used, we can investigate the action using Wireshark (see Figure 9-1).

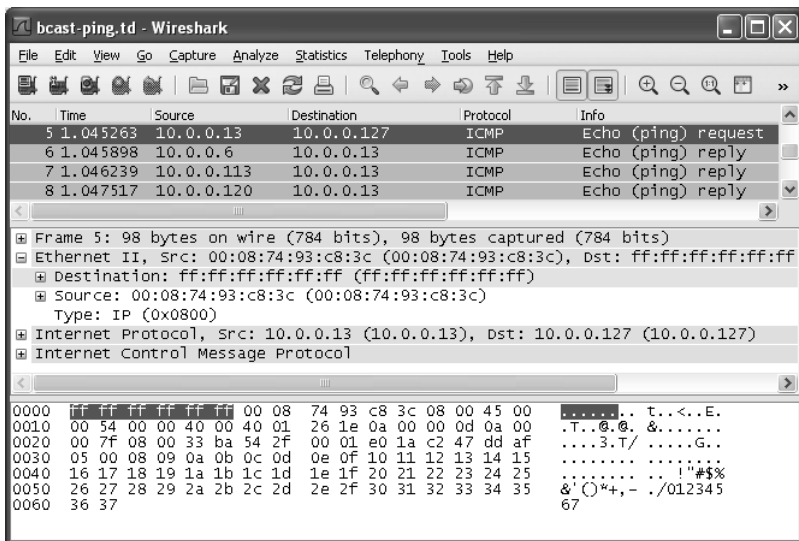


Figure 9-1 An ICMPv4 Echo Request message sent to the directed broadcast address on the local subnetwork is encapsulated in a link-layer broadcast frame with a destination address of all 1s.

The Echo Request message is sent to the address 10.0.0.127. The IPv4 implementation determines this to be the subnet-directed broadcast address by consulting information in the local routing table and interface configuration information, and it sends the datagram using the link-layer broadcast address ff:ff:ff:ff:ff:ff, so no ARP request is needed to determine the MAC addresses for each destination. In fact, the sender is unaware of what hosts will respond until they do. It knows only that 10.0.0.127 is a broadcast address and that it should therefore use a broadcast link-layer destination address when sending. The source addresses at both the IP and link layers are entirely conventional unicast; multicast addresses are used only as destination addresses.

In this particular example, notice that each of the responses generated is directed at 10.0.0.13, the unicast address of the original sender, and that each response includes the IPv4 address of the responder: 10.0.0.6, 10.0.0.113, and 10.0.0.120. This is a simple example of a more general principle: broadcast addressing (and multicast addressing, as we shall see shortly) can be used to discover systems or services that are otherwise unknown. In this example, the outgoing broadcast ping request discovered three hosts that are willing to respond to broadcast Echo Request messages.

9.2.2 Sending Broadcast Datagrams

Generally speaking, applications using broadcast use the UDP protocol (or ICMPv4 protocol) and invoke an ordinary set of API calls to send traffic. The only exception is that when invoking the API calls, a special flag (SO_BROADCAST) is used in some operating systems to indicate that the application really does intend to send broadcast datagrams. For example, in Linux, failing to use the `-b` flag when attempting to do a broadcast ping causes the following output:

```
Linux% ping 10.0.0.127
Do you want to ping broadcast? Then -b
```

This error is caused because the SO_BROADCAST flag is provided through the API only when the `-b` option is provided in the command line. This helps to avoid accidentally generating broadcast traffic that could temporarily congest a network.

To determine which interfaces are used for broadcasting, the IPv4 forwarding table (called “routing table” here) is consulted. The following is an example of a Windows Vista routing table (later versions of Windows use an identical format) showing the interface list and broadcast-related routing information (other information has been removed for clarity):

```
C:\> netstat -rn
=====
Interface List
 10 ...02 00 4c 4f 4f 50 ..... Microsoft Loopback Adapter
   9 ...00 13 02 20 b9 18 ..... Intel(R) PRO/Wireless 3945ABG Network
                               Connection
```

```

 8 ...00 14 22 f4 19 5f . .... Broadcom 440x 10/100 Integrated
                               Controller
 1 . ..... Software Loopback Interface 1
12 ...00 00 00 00 00 00 00 e0 Microsoft ISATAP Adapter
13 ...00 00 00 00 00 00 00 e0 Microsoft ISATAP Adapter #2
11 ...00 00 00 00 00 00 00 e0 isatap.
                               {2523E0D6-A8E2-42F1-8188-6AA108FEA1EA}
=====
```

IPv4 Route Table

=====

Active Routes:

Network	Destination	Netmask	Gateway	Interface	Metric
0.0.0.0		0.0.0.0	10.0.0.1	10.0.0.57	25
10.0.0.127		255.255.255.255	On-link	10.0.0.57	281
127.255.255.255		255.255.255.255	On-link	127.0.0.1	306
169.254.255.255		255.255.255.255	On-link	169.254.57.240	286
255.255.255.255		255.255.255.255	On-link	127.0.0.1	306
255.255.255.255		255.255.255.255	On-link	169.254.57.240	286
255.255.255.255		255.255.255.255	On-link	10.0.0.57	281

The first portion of this output shows seven different network interfaces that may be used for carrying network traffic. The first is the virtual loopback interface, the next is a Wi-Fi wireless interface, the third is a wired Ethernet interface (that is disconnected), the fourth is another loopback interface, and the next three are used as part of the nonstandard *Intra-Site Automatic Tunnel Addressing Protocol* (ISATAP) [RFC5214][RFC5579]. ISATAP is used in supporting IPv6 hosts separated by an IPv4 network.

Moving on to the routing table, we see that there are seven entries that could be used to determine where broadcast traffic should be sent. The first entry is the default route (mask 0.0.0.0), so it matches any destination. This could be used by broadcasts directed beyond the local network, if such a facility were enabled. This type of directed broadcast, which travels beyond the local network, is usually disabled by routers to avoid a number of security problems, as suggested by [RFC2644].

The next three entries are the directed subnet broadcast addresses associated with the three interfaces having IPv4 addresses 10.0.0.57, 127.0.0.1, and 169.254.57.240, respectively. The last two are software loopback interfaces. These entries show how Windows expresses a directed subnet broadcast route as the network prefix combined with all 1s bits in the host part as the destination, and a /32 or 255.255.255.255 subnet mask. The **Gateway** column indicates **On-link**, so traffic is delivered using direct delivery (see Chapter 5) on the interface identified in the **Interface** column. In these cases, there is not more than one match for each subnet-directed broadcast address, so the **Metric** column is not consulted.

The last three entries are routing entries for the limited broadcast address, 255.255.255.255. In some ways, this address acts like a multicast address because

network. Thus, it is not immediately obvious which interface(s) should be used for sending traffic destined for the limited broadcast address. Unfortunately, Section 3.3.6 of the Host Requirements RFC [RFC1122] provides little guidance:

There has been discussion on whether a datagram addressed to the Limited Broadcast address ought to be sent from all the interfaces of a multihomed host. This specification takes no stand on the issue.

As a consequence, the way outgoing traffic to the limited broadcast address is handled is operating-system-specific. Most systems pick a single broadcast-capable interface to use for sending such traffic. Linux and FreeBSD behave this way. FreeBSD actually converts the limited broadcast address into a subnet-directed broadcast address of the “primary” (first configured) interface, although an application can disable this behavior using the `IP_ONESBCAST` API option. Windows, for example, has behaved differently in different versions. Up to Windows 2000, limited broadcasts were forwarded over multiple interfaces. With Windows XP and later, the default behavior is to send over a single interface. In this example, there are multiple possible matching routes for such traffic, so the entry with the lowest metric (interface `10.0.0.57`) is used.

9.3 Multicasting

To reduce the amount of overhead involved in broadcasting, it is possible to send traffic only to those receivers that are interested in it. This is called *multicasting*. Fundamentally, this is accomplished by either having the sender indicate the receivers, or instead having the receivers independently indicate their interest. The network then becomes responsible for sending traffic only to intended/interested recipients. Implementing multicast is considerably more challenging than broadcast because *multicast state* (information) must be maintained by hosts and routers as to what traffic is of interest to what receivers. In the TCP/IP model of multicasting, receivers indicate their interest in what traffic they wish to receive by specifying a multicast address and optional list of sources. This information is maintained as *soft state* (see Chapter 4) within hosts and routers, meaning that it must be updated regularly or it will time out and be deleted. When this happens, delivery of multicast traffic either ceases or reverts to broadcast.

The inefficiencies of broadcast apply not only to wide area networks, where they can be extremely severe, but also to local area and enterprise networks. Every host that can be reached on the same LAN or VLAN must process broadcast packets. IP multicasting provides a more efficient way to carry out the same types of tasks. If IP multicasting is used properly, only those hosts involved or interested in the communication need to process the associated packets, traffic is carried only on those links where it will be used, and only one copy of any multicast datagram is carried on any such link. To make multicasting work, applications that wish

to be involved in a communication require a mechanism to notify their protocol implementations of their desires. The host software can then arrange to receive packets matching the applications' criteria.

IP multicasting originated using a design based on the way group addressing works in link-layer networks such as Ethernet. In this approach, each station selects the group address for which it is willing to accept traffic, irrespective of the sender. This approach is also sometimes called any-source multicast (ASM) because of the insensitivity to the identity of the sender. As IP multicasting has evolved, an alternative form that is sensitive to the identity of the sender called source-specific multicast (SSM) [RFC4607] has been developed that allows end stations to explicitly include or exclude traffic sent to a multicast group from a particular set of senders. The SSM service model is easier to implement than ASM, primarily because in wide area multicasting it is easier to determine the location of a single source than the locations of many sources. In the local area, however, much of the machinery involved in supporting either ASM or SSM is identical, so we treat them together and explain the few differences when they are important. We begin by investigating how IP multicast traffic makes use of MAC-layer multicast addresses on multicast-capable IEEE LAN technology.

9.3.1 Converting IP Multicast Addresses to 802 MAC/Ethernet Addresses

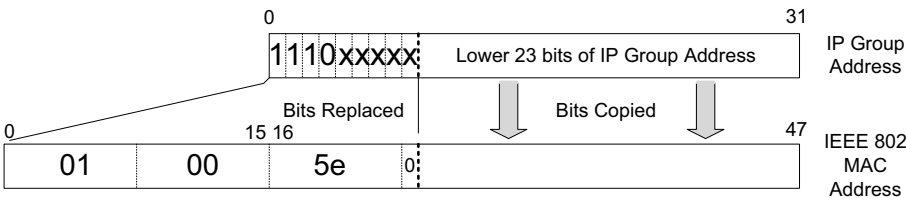
When using unicast addresses on Ethernet-like networks, ARP (see Chapter 4) is usually used to determine a local destination's MAC address given its IPv4 address. In IPv6, ND serves a similar role (see Chapter 8). When we looked at broadcasting earlier, we noticed that there is a single well-known broadcast MAC address that can always be used to reach all stations on a LAN or VLAN. What destination MAC address should be placed in a link-layer frame when we wish to send multicast traffic? Ideally, we would not have to use a protocol message to determine the appropriate MAC address but could instead simply map an IP multicast address directly to some corresponding MAC address. To see how this is done, we shall focus on IEEE 802 networks, especially Ethernet and Wi-Fi. These networks represent the most common types of networks where IP multicasting is used. We will first discuss how the mapping works with IPv4, and then move on to the slightly different method used with IPv6.

To carry IP multicast efficiently on a link-layer network, there should be a one-to-one mapping between packets and addresses at the IP layer and frames at the link layer. The IANA owns the IEEE Organizationally Unique Identifier (abbreviated OUI, or more informally Ethernet address prefix) 00:00:5e. With it, IANA is given the right to use group (multicast) MAC addresses starting with 01:00:5e as well as unicast addresses starting with 00:00:5e. This prefix is used as the high-order 24 bits of the Ethernet address, meaning that this block includes unicast addresses in the range 00:00:5e:00:00:00 through 00:00:5e:ff:ff:ff and group addresses in the range 01:00:5e:00:00:00 through 01:00:5e:ff:ff:ff. Other organizations besides IANA own address blocks as well, but only IANA devotes some of its space to support of IP multicasting.

The IANA allocates half of its group block to identifying IPv4 multicast traffic on IEEE 802 LANs. This means that the Ethernet addresses corresponding to IPv4 multicasting are in the range 01:00:5e:00:00:00 through 01:00:5e:7f:ff:ff.

Note
Our notation here uses the Internet standard bit order as the bits appear in memory. This is what most programmers and system administrators deal with. The IEEE documentation uses the transmission order of the bits.

The mapping of IPv4 addresses to their corresponding IEEE 802-style link-layer addresses can be seen in Figure 9-2.



Example: 224.0.1.17 → 01:00:5e:01:11

Figure 9-2 The IPv4-to-IEEE-802 MAC multicast address mapping uses the lower-order 23 bits of the IPv4 group address as the suffix of a MAC address starting with 01:00:5e. Because only 23 of the 28 group address bits are used, 32 groups are mapped to the same MAC-layer address.

Recall from Chapter 2 that all IPv4 multicast addresses are contained within the address space from 224.0.0.0 to 239.255.255.255 (formerly known as class D address space). All such addresses share a common 4-bit sequence of 1110 in the high-order bits. Thus, there are $32 - 4 = 28$ bits available to encode the entire space of $2^{28} = 268,435,456$ multicast IPv4 addresses (also called group IDs). For IPv4, the IANA policy of allocating half of its group addresses for use in supporting IPv4 multicast means that all 268,435,456 IPv4 multicast group IDs need to be mapped into a link-layer address space containing only $2^{23} = 8,388,608$ unique entries. The mapping therefore is *nonunique*. That is, more than one IPv4 group ID is mapped to the same MAC-layer group address. Specifically, $2^{28} / 2^{23} = 2^5 = 32$ distinct IPv4 multicast group IDs are mapped to each group address. For example, both the multicast addresses 224.128.64.32 (hexadecimal e0.80.40.20) and 224.0.64.32 (hexadecimal e0.00.40.20) are mapped into the Ethernet address 01:00:5e:00:40:20.

For IPv6, the 16-bit OUI hexadecimal prefix is 33:33. This means that the last 32 bits of the IPv6 address can be used to form the link-layer address. Thus, any address ending with the same 32 bits maps to the same MAC address (see Figure 9-3). Given

that all IPv6 multicast addresses begin with `ff`, and the subsequent 8 bits are used for flags and scope information, this leaves $128 - 16 = 112$ bits for representing 2^{112} groups. Thus, with the 32 bits of MAC-layer address available to encode these groups, there can be as many as $2^{112}/2^{32} = 2^{80}$ groups that map to the same MAC address!

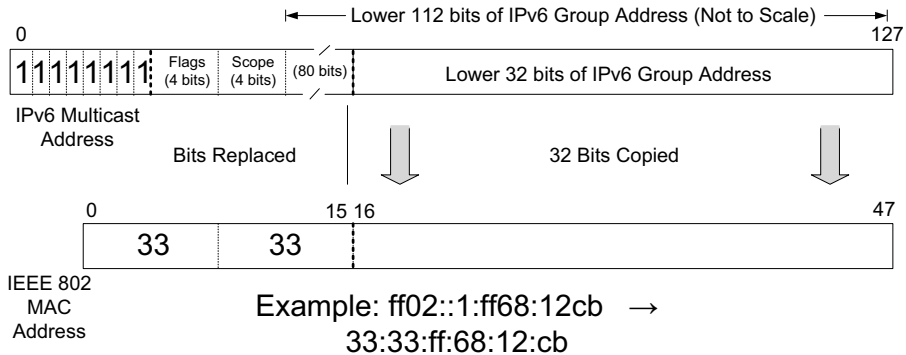


Figure 9-3 The IPv6-to-IEEE-802 MAC multicast address mapping uses the low-order 32 bits of the IPv6 multicast address as the suffix of a MAC address starting with `33:33`. Because only 32 of the 112 multicast address bits are used, 2^{80} groups are mapped to the same MAC-layer address.

9.3.2 Examples

In a previous example, we used a subnet broadcast address to determine all the hosts on the local subnet that would respond to a broadcast ICMPv4 Echo Request message. Here, because we can use multicast addressing to determine hosts that offer a particular service, we can send an ICMPv4 echo request to those hosts that respond to the *Multicast DNS* (mDNS [CK11]) address `224.0.0.251`:

```
Linux% ping 224.0.0.251
PING 224.0.0.251 (224.0.0.251) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=60 time=1.10 ms
64 bytes from 10.0.0.11: icmp_seq=1 ttl=60 time=1.60 ms (DUP!)
64 bytes from 10.0.0.120: icmp_seq=1 ttl=64 time=2.59 ms (DUP!)
--- 224.0.0.251 ping statistics ---
1 packets transmitted, 1 received, +2 duplicates,
0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.109/1.767/2.590/0.615 ms
```

Here, hosts `10.0.0.2`, `10.0.0.11`, and `10.0.0.120` all respond, indicating that they are subscribed to the mDNS group. Notice that these hosts are not the same ones that responded when we used the broadcast address of `10.0.0.127`. This is not so surprising, as not all hosts support the mDNS protocol.

Note

Multicast DNS (mDNS) is a service designed to support zero configuration (effortless system and device configuration). mDNS has been supported on Apple systems where it is part of Bonjour. Microsoft has promoted an alternative protocol that includes similar features known as Link Local Multicast Name Resolution (LLMNR) [RFC4795]. Neither protocol is currently an Internet standard within the IETF, but at present mDNS enjoys a longer history than LLMNR. See Chapter 11 for more details.

For IPv6, we can perform a similar operation using an ICMPv6 Echo Request message:

```
Linux% ping6 -I eth0 ff02::fb
PING ff02::fb(ff02::fb) from fe80::208:74ff:fe93:c83c eth0:
 56 data bytes
64 bytes from fe80::217:f2ff:fee7:6d91: icmp_seq=1 ttl=64 time=2.76 ms

--- ff02::fb ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 2.768/2.768/2.768/0.000 ms
```

Note that in this case, we provide the outgoing interface as input to the ping6 program. This allows the program to select the appropriate outgoing IPv6 address in Windows XP. As we can see in Figure 9-4, the address selected is a link-local address associated with the eth0 device.

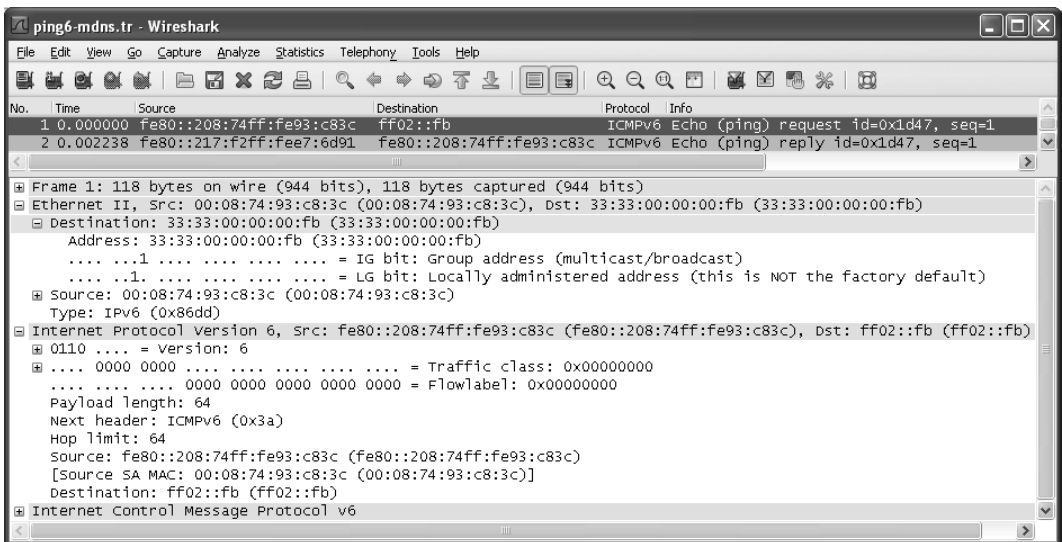


Figure 9-4 An ICMPv6 Echo Request message is sent from a link-local unicast address associated with the eth0 network interface to the multicast address ff02::fb. The reply includes the sender's IPv6 link-local IPv6 address.

The packets are identified as ICMPv6 Echo Request/Reply messages with the *Identifier* field set to 0x1d47 and *Sequence Number* field set to 1. The source IPv6 addresses are link-local in all cases. The destination address of the request is the multicast address ff02::fb, which is mapped to the MAC address 33:33:00:00:00:fb. The Echo Reply message is sent directly to the link-local IPv6 unicast address of the sender, fe80::208:74ff:fe93:c83c, from the responder's link-local unicast address, fe80::217:f2ff:fee7:6d91. Note that the sender of the Echo Reply message arranges to use a source IPv6 address of the same scope (see the discussion on source address selection in Chapter 5, and compare Figure 9-4 with Figure 5-16).

9.3.3 Sending Multicast Datagrams

When sending any IP packet, a decision must be made as to which source address and interface to use. This is especially true for IPv6, where having multiple addresses per interface is considered normal. To help determine this, we can look at the forwarding table present in the host. In either Windows or Linux, the `netstat` command can be used. Here are the IPv4 and IPv6 routing tables as output on Windows Vista (later versions use an identical format):

```
C:\> netstat -rn
... interface list ...
```

IPv4 Route Table

```
=====
Active Routes:
Network Destination  Netmask          Gateway         Interface       Metric
0.0.0.0             0.0.0.0          10.0.0.1        10.0.0.57       25
224.0.0.0           240.0.0.0        On-link         127.0.0.1       306
224.0.0.0           240.0.0.0        On-link         169.254.57.240  286
224.0.0.0           240.0.0.0        On-link         10.0.0.57       281
255.255.255.255     255.255.255.255 On-link         127.0.0.1       306
255.255.255.255     255.255.255.255 On-link         169.254.57.240  286
255.255.255.255     255.255.255.255 On-link         10.0.0.57       281
=====
```

```
Persistent Routes:
None
```

IPv6 Route Table

```
=====
Active Routes:
If Metric Network Destination      Gateway
9    281  ::/0          fe80::204:5aff:fe9f:9e80
1    306  ff00::/8      On-link
10   286  ff00::/8      On-link
9    281  ff00::/8      On-link
=====
```

```
Persistent Routes:
None
```

From this table we can see that a default route for IPv4 traffic goes to 10.0.0.1 using interface 10.0.0.57. Although this does match multicast traffic, there are other entries that are more specific. The entries listed as 224.0.0.0/4 (subnet mask 240.0.0.0) indicate that three different interfaces can carry outgoing multicast traffic. The interface with the lowest metric (10.0.0.57, with metric 281) is the most preferred, so it is used unless an application specifies otherwise. For IPv6, all multicast addresses begin with ff, and there are no broadcast addresses, so interfaces 1, 9, and 10 can all be used. Interface 9 (which happens to be the same interface used for IPv4 and the default for IPv6 unicast traffic) has the lowest metric. Additional information indicating which interfaces have which IP addresses can be determined using the Windows command `ipconfig /all`.

The output on Linux is separate for different protocol families (such as IPv4 and IPv6). It is generated by different arguments to the `netstat` command, to indicate which version of IP (or other) protocol is of interest. For IPv4, there is nothing to show, as there is no special entry for multicast; a conventional default route handles the multicast traffic. For IPv6, however, we can see the following:

```
Linux% netstat -rn -A inet6
Kernel IPv6 routing table
Destination Next Hop          Flags Metric Ref    Use  Iface
ff00::/8      ::                          U          256  0      0  eth0
```

In this case, there is no direct “next hop,” so the unspecified address (::) is listed in the table, but we can see that the outgoing interface is `eth0`. The **Flags** column contains only **U**, indicating that the route is usable, but the lack of a **G** flag indicates that it is an on-link route, not requiring forwarding to a router.

9.3.4 Receiving Multicast Datagrams

Fundamental to multicasting is the concept of a process *joining* or *leaving* one or more multicast groups on a given interface on a host. (We use the term *process* to mean a program being executed by the operating system, often on behalf of a user.) Membership in a multicast group on a given interface is dynamic—it changes over time as processes join and leave groups. In addition to joining or leaving groups, additional methods are needed if a process wishes to specify sources it cares to hear from or exclude. These are required parts of any API on a host that supports multicasting. We use the qualifier “interface” because membership in a group is associated with an interface. A process can join the same group on multiple interfaces, multiple groups on the same interface, or any combination thereof.

9.3.4.1 Example

It is possible to determine what multicast groups are in use on each interface using an operating-system-specific command. In Windows, the commands are part of the `net sh` package. For IPv6, this works as follows (for IPv4, replace `ipv6` with `ip`):

```
C:\> netsh interface ipv6 show joins
Interface 1: Loopback Pseudo-Interface 1
Scope      References  Last  Address
-----
0           1      Yes  ff02::c

Interface 8: Local Area Connection
Scope      References  Las  Address
-----
0           0      Yes  ff01::1
0           0      Yes  ff02::1
0           1      Yes  ff02::c
0           1      Yes  ff02::1:3
0           1      Yes  ff02::1:ffdc:fc85
```

Here we can see how IPv6 uses several multicast addresses per interface. The first interface is a loopback, local interface. The only multicast group used on it is the link-local scoped Simple Service Discovery Protocol (SSDP) multicast address, which we saw in Chapter 7.

Note

SSDP is described in an (expired) Internet draft [GCLG99] authored by Microsoft and Hewlett-Packard. SSDP also operates on IPv4, using address 239.255.255.250 and UDP port 1900.

On the other network interface, the addresses ff01::1 (node-local All Nodes address) and ff02::1 (link-local All Nodes address) show joins for all nodes, and ff02::c shows the use of SSDP. The next address, ff02::1:3, is for support of LLMNR, a local multicast name resolution system mentioned previously and discussed in more detail in Chapter 11. Finally, the address ff02::1:ffdc:fc85 is the Solicited-Node multicast address for this node, used by IPv6 ND. Recall that in IPv6, determining a neighbor's MAC address is accomplished using multicast ICMPv6 ND messages, as opposed to the ARP mechanism used in IPv4.

On Linux, the `netstat` command displays the IP group memberships:

```
Linux% netstat -gn
IPv6/IPv4 Group Memberships
Interface      RefCnt Group
-----
lo              1      224.0.0.1
eth1            1      224.0.0.1
lo              1      ff02::1
eth1            1      ff02::1:ff2a:1988
eth1            1      ff02::1
```

The output from this command includes the join information for multiple interfaces and for both IPv4 and IPv6. In this case, we see 224.0.0.1 (All Hosts) on both the Ethernet interface (eth1) as well as the local loopback interface (lo).

We can also see the link-local scope All Nodes bindings for each interface. Finally, the Solicited-Node address is `ff02::1:ff2a:1988`.

Note

With IP multicasting, a process may send to a multicast group without joining it. More commonly, processes do join the multicast groups with which they are interacting, and on one or more specific interfaces. There is a special option in the socket API (`IP_MULTICAST_LOOP`) to alter the way multicast traffic is handled among processes on the same host that are members of the same group on the same interface. In UNIX, this option applies to the send path, meaning that if the option is enabled, other processes on the same host receive the multicast datagrams, even if they have the option disabled. Conversely, on Windows, the option applies on the receive path, meaning that any processes enabling the option receive multicast traffic from other applications on the same host even if they have the option disabled.

9.3.5 Host Address Filtering

To understand how the operating system processes received multicast datagrams for multicast groups that programs have joined, recall from Chapter 3 that *filtering* takes place on each host's network interface card (NIC), each time a frame is presented to it (e.g., by a bridge or switch) for possible reception. Figure 9-5 indicates how this occurs.

In a typical switched Ethernet environment, broadcast and multicast frames are replicated on all segments within a VLAN, along a spanning tree formed among the switches. Such frames are delivered to the NIC on each host which checks the correctness of the frame (using the CRC) and makes a decision about whether to receive the frame and deliver it to the device driver and network stack. Normally the NIC receives only those frames whose destination address is either the hardware address of the interface or the broadcast address. However, when multicast frames are involved, the situation is somewhat more complicated.

NICs tend to come in two varieties. One type performs filtering based on the hash values of the multicast hardware addresses in which the host software has expressed interest, which means that some unwanted frames can always get through because of hash collisions. The other type listens for a finite table of multicast addresses, meaning that if the host needs to receive frames destined for more multicast addresses than can fit in the table, the NIC is put into a "multicast-promiscuous" mode, in which case all multicast traffic is given to the host software. Hence, both types of interfaces require that the device driver or higher-layer software perform checking that the received frame is really wanted. Even if the interface performs perfect multicast filtering (based on the 48-bit hardware address), because the mapping from a multicast IPv4 or IPv6 address to a 48-bit hardware address is not unique, filtering is still required. Despite this imperfect address mapping and hardware filtering, multicasting is still more efficient than broadcasting.

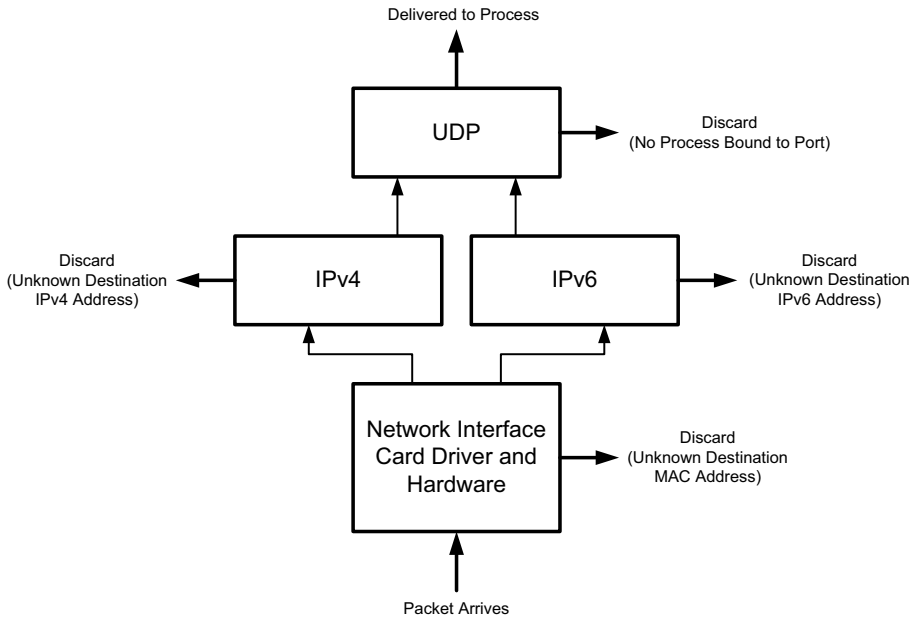


Figure 9-5 Each layer implements filtering on some portion of the received message. MAC address filtering can take place in either software or hardware. Cheaper NICs tend to impose a larger processing burden on software because they perform fewer functions in hardware.

For NICs that support a multi-entry address table, the destination address on each received frame is compared against this table, and if the address is found in the table, the frame is received and processed by the device driver. The entries of this table are managed by the device driver software in combination with other layers of the protocol stack (such as the IPv4 and IPv6 implementations). Another method of implementing this type of filtering is to apply a hash function to the destination address, forming an index into a (smaller) binary vector. When the indexed entry in the vector contains a 1 bit, the corresponding address is deemed to be acceptable and the frame is processed further. This approach can save memory on the NIC, but because of collisions in the hash function, some frames may be considered admissible when they should not be. This is not a fatal problem, however, because higher layers of the stack also perform filtering, and no frames are ever discarded when they should not have been (i.e., there are no false negatives, but there may be false positives).

Note

The specific capabilities of an NIC vary based on manufacturer. As an example, the Intel 82583V Ethernet controller includes a 16-entry exact match table (unicast or multicast), a 4096-bit hash filter for multicast destinations, and support for

both promiscuous reception and promiscuous multicast reception in addition to filtering based on up to 4096 VLAN tags.

Once the NIC hardware has verified a frame as acceptable (i.e., the CRC is correct, any VLAN tags match, and the destination MAC address matches an address entry in one or more of the NIC's tables), the frame is passed to the device driver, where additional filtering is performed. First, the frame type must specify a protocol that is supported (e.g., IPv4, IPv6, ARP, etc.). Second, additional multicast filtering may be performed to check whether the host belongs to the addressed multicast group (indicated by the destination IP address). This is necessary for NICs that may generate false positives.

The device driver then passes the frame to the next layer, such as IP, if the frame type specifies an IP datagram. IP performs more filtering, based on the source and destination IP addresses, and passes the datagram up to the next layer (such as TCP or UDP) if all is well. Each time UDP receives a datagram from IP, it performs filtering based on the destination port number, and sometimes the source port number, too. If no process is currently using the destination port number, the datagram is discarded and an ICMPv4 or ICMPv6 Port Unreachable message is normally generated. (TCP performs similar filtering based on its port numbers.) If the UDP datagram has a checksum error, UDP silently discards it.

One of the primary motivations behind the development of the multicast addressing features was to avoid the overhead of broadcasting. Consider an application that is designed to use UDP broadcasts. If there are 50 hosts on the network (or VLAN), but only 20 are participating in the application, every time one of the 20 sends a UDP broadcast, the other 30 nonparticipating hosts have to process the broadcast, all the way up through the UDP layer, before the UDP datagram is discarded. The UDP datagram is discarded by these 30 hosts because the destination port number is not in use. The intent of multicasting is to reduce this load on hosts with no interest in the application. With multicasting, a host specifically joins one or more multicast groups. If possible, the NIC is told which multicast groups the host belongs to, and only those multicast frames associated with the IP-layer multicast groups are allowed through the filter in the NIC. All of this machinery offers less overhead imposed on the host, in exchange for additional complexity in managing multicast addresses and group memberships.

9.4 The Internet Group Management Protocol (IGMP) and Multicast Listener Discovery Protocol (MLD)

So far we have discussed how multicast datagrams are transmitted, filtered, and received from a host's perspective. When multicast datagrams are to be forwarded over a wide area network or within an enterprise across multiple subnets, we require that *multicast routing* be enabled by one or more multicast routers. This complicates the situation considerably, because multicast routers require

10

User Datagram Protocol (UDP) and IP Fragmentation

10.1 Introduction

UDP is a simple, datagram-oriented, transport-layer protocol that preserves message boundaries. It does not provide error correction, sequencing, duplicate elimination, flow control, or congestion control. It can provide error detection, and it includes the first true *end-to-end* checksum at the transport layer that we have encountered. This protocol provides minimal functionality itself, so applications using it have a great deal of control over how packets are sent and processed. Applications wishing to ensure that their data is reliably delivered or sequenced must implement these protections themselves. Generally, each UDP output operation requested by an application produces exactly one UDP datagram, which causes one IP datagram to be sent. This is in contrast to a stream-oriented protocol such as TCP (see Chapter 15), where the amount of data written by an application may have little relationship to what actually gets sent in a single IP datagram or what is consumed at the receiver.

[RFC0768] is the official specification of UDP, and it has remained as a standard without significant revisions for more than 30 years. As mentioned, UDP provides no error correction: it sends the datagrams that the application writes to the IP layer, but there is no guarantee that they ever reach their destination. In addition, there is no protocol mechanism to prevent high-rate UDP traffic from negatively impacting other network users. Given this lack of reliability and protection, we might be tempted to conclude that there are no benefits to using UDP at all. This is not true, however. Because of its connectionless character, it has less overhead than other transport protocols. In addition, broadcast and multicast operations (see Chapter 9) are much more straightforward using a connectionless transport such as UDP. Finally, the ability of an application to choose its own unit of retransmission can be an important consideration (see [CT90], for example).

Figure 10-1 shows the encapsulation of a UDP datagram as a single IPv4 datagram. The IPv6 encapsulation is similar, but other details differ slightly and we discuss them in Section 10.5. The IPv4 *Protocol* field has the value 17 to indicate UDP. IPv6 uses the same value in the *Next Header* field. Later in this chapter we will examine what happens when the size of the UDP datagram exceeds the MTU size and the datagram must be fragmented into more than one IP-layer packet.

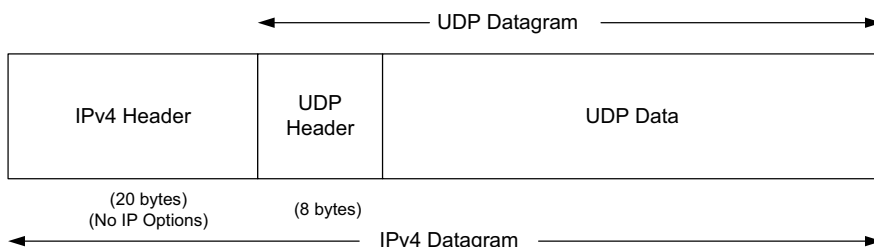


Figure 10-1 Encapsulation of a UDP datagram in a single IPv4 datagram (the typical case with no IPv4 options). The IPv6 encapsulation is similar; the UDP header follows the header chain.

10.2 UDP Header

Figure 10-2 shows a UDP datagram, including the payload and UDP header (which is always 8 bytes in size).

Port numbers act as *mailboxes* and help a protocol implementation identify the sending and receiving processes (see Chapter 1). They are purely *abstract*—they do not correspond to any physical entity on a host. In UDP, port numbers are positive 16-bit numbers, and the source port number is optional; it may be set to 0 if the sender of the datagram never requires a reply. Transport protocols such as TCP, UDP, and SCTP [RFC4960] use the destination port number to help demultiplex incoming data from IP. Because IP demultiplexes the incoming IP datagram to a particular transport protocol based on the value of the *Protocol* field in the IPv4 header or *Next Header* field in the IPv6 header, this means that the port numbers can be made independent among the transport protocols. That is, TCP port numbers are used only by TCP, and the UDP port numbers only by UDP, and so on. A straightforward consequence of this separation is that two completely distinct servers can use the same port number and IP address, as long as they use different transport protocols.

Note

Despite this independence, if a well-known service is provided (or can conceivably be provided) by both TCP and UDP, the port number is normally allocated to be the same for both transport protocols. This is purely for convenience and is not required by the protocols. See [IPORT] for details on how port numbers are formally assigned.

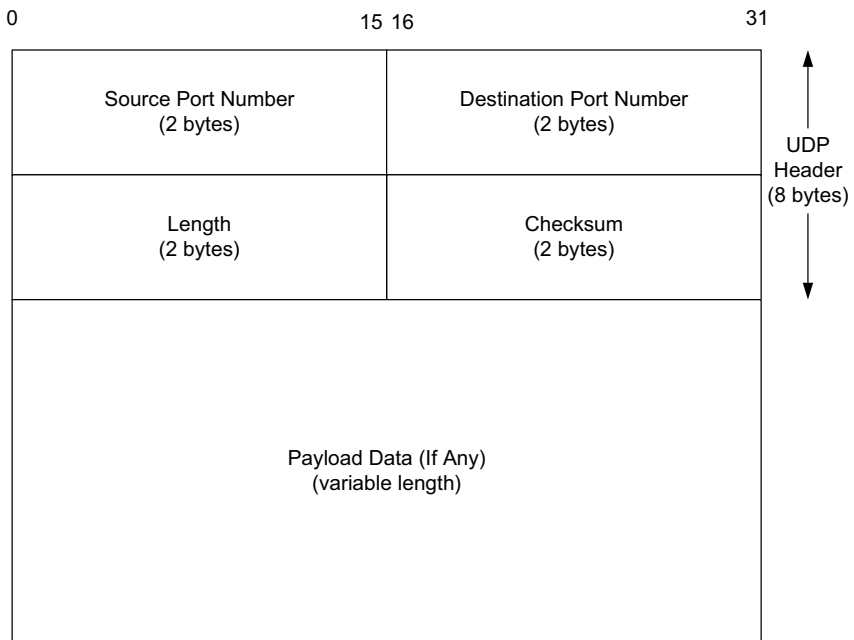


Figure 10-2 The UDP header and payload (data) area. The *Checksum* field is end-to-end and is computed over the UDP pseudo-header, which includes the *Source* and *Destination IP Address* fields from the IP header. Thus, any modification made to those fields (e.g., by NAT) requires a modification to the UDP checksum.

Referring to Figure 10-2, the UDP *Length* field is the length of the UDP header and the UDP data in bytes. The minimum value for this field is 8 except when UDP is used with IPv6 jumbograms (see Section 10.5). Sending a UDP datagram with 0 bytes of data is acceptable, although rare. Note that the UDP *Length* field is redundant; the IPv4 header contains the datagram's total length (see Chapter 5), and the IPv6 header contains the payload length. The length of a UDP/IPv4 datagram is then the total length of the IPv4 datagram minus the length of the IPv4 header. A UDP/IPv6 datagram's length is the value of the *Payload Length* field contained in the IPv6 header minus the lengths of any extension headers (unless jumbograms are being used). In either case, the UDP *Length* field should match the length computed from the IP-layer information.

10.3 UDP Checksum

The UDP checksum is the first end-to-end transport-layer checksum we have encountered (ICMP has an end-to-end checksum but is not a true transport protocol). It covers the UDP header, the UDP data, and a pseudo-header (defined later in this section). It is computed at the initial sender and checked at the final destination.

It is not modified in transit (except when it passes through a NAT, as described in Chapter 7). Recall that the checksum in the IPv4 header covers only the header (i.e., it does not cover any data in the IP packet) and is recomputed at each IP hop (required because the IPv4 *TTL* field is decremented by routers when the datagram is forwarded). Transport protocols (e.g., TCP, UDP) use checksums to cover their headers *and* data. With UDP, the checksum is optional (although strongly suggested), while with the others it is mandatory. When UDP is used with IPv6, computation and use of the checksum are mandatory because there is no header checksum at the IP layer. To provide error-free data to applications, a transport-layer protocol such as UDP must always compute a checksum or use some other error detection mechanism before delivering the data to a receiving application.

Although the basics for calculating the UDP checksum are similar to what we described in Chapter 5 for the general Internet checksum (the one's complement of the one's complement sum of 16-bit words), there are two small special details. First, the length of the UDP datagram can be an odd number of bytes, whereas the checksum algorithm adds 16-bit words (always an even number of bytes). The procedure for UDP is to append a (virtual) pad byte of 0 to the end of odd-length datagrams, just for the checksum computation and verification. This pad byte is not actually transmitted and is thus called "virtual" here.

The second detail is that UDP (as well as TCP) computes its checksum over a 12-byte *pseudo-header* derived (solely) from fields in the IPv4 header or a 40-byte pseudo-header derived from fields in the IPv6 header. This pseudo-header is also virtual and is used only for purposes of the checksum computation (at both the sender and the receiver). It is never actually transmitted. This pseudo-header includes the source and destination addresses and *Protocol* or *Next Header* field (which should contain the value 17) from the IP header. Its purpose is to let the UDP layer verify that the data has arrived at the correct destination (i.e., that IP has not accepted a misaddressed datagram, and that IP has not given UDP a datagram that is for another transport protocol). Figure 10-3 shows what is covered when computing the UDP checksum, including the pseudo-header along with the UDP header and payload.

Note

The careful reader will note that this causes a so-called *layering violation*. That is, the UDP protocol (transport layer) is directly processing bits "owned" by IP (network layer). While true, it is of only minor consequence to protocol implementations, which in general have IP-layer information readily available when data is passed to (or from) UDP. It is of far greater concern for NATs (see Chapter 7), especially if UDP datagrams are fragmented.

Figure 10-3 shows a datagram with an odd data length, requiring a pad byte for the checksum computation. Note that the length of the UDP datagram appears twice in the checksum computation. If the value of the calculated checksum

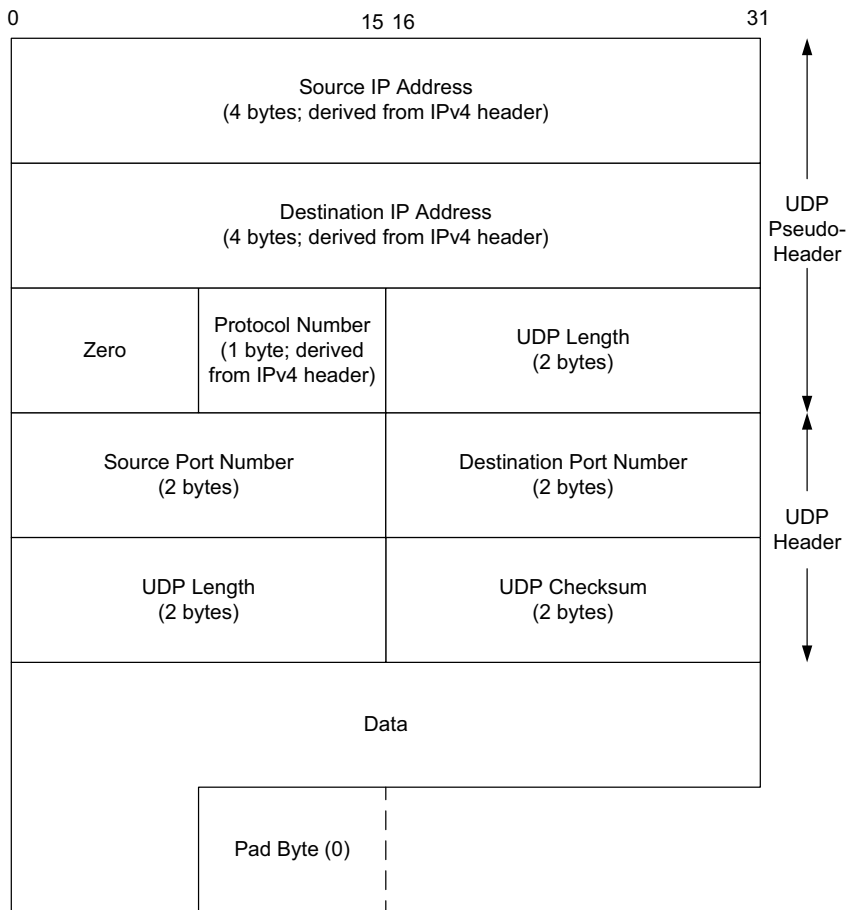


Figure 10-3 Fields used in computing the checksum for UDP/IPv4 datagrams, including the pseudo-header, the UDP header, and data. If the data is not an even number of bytes, it is padded with one 0 byte for purposes of computing the checksum. The pseudo-header and any pad bytes are not transmitted with the datagram.

happens to be 0x0000, it is stored in the header as all 1 bits (0xFFFF), which is equivalent in one's complement arithmetic (see Chapter 5). Upon receipt, a *Checksum* field value of 0x0000 indicates that the sender did not compute a checksum. If the sender *did* compute a checksum and the receiver detects a checksum error, the UDP datagram is silently discarded. No error message is generated, although some statistical counts may be updated. (This is what happens if an IPv4 header checksum error is detected.)

Despite UDP checksums being optional in the original UDP specification, they are currently required to be enabled on hosts by default [RFC1122]. During the 1980s some computer vendors turned off UDP checksums by default to speed

up their implementation of Sun's Network File System (NFS), which uses UDP. While this might not cause problems in many cases because of the presence of layer 2 CRC protection (which is stronger than the Internet checksum; see Chapter 3), it is considered bad form (and a violation of the RFCs) to disable checksums by default. Early experience in the Internet revealed that when datagrams pass through routers, all bets are off with respect to their correctness. Believe it or not, there have been routers with software and hardware bugs that have modified bits in the datagrams being forwarded. These errors are undetectable in a UDP datagram if the end-to-end UDP checksum is disabled. Also realize that some older data-link protocols (e.g., serial line IP, or SLIP) do not have any form of data-link checksum, thereby leaving open the possibility that IP packets could be undetectably modified unless another checksum is employed.

Note

[RFC1122] requires that UDP checksums be enabled by default. It also states that an implementation must verify a received checksum if the sender calculated one (i.e., if the received checksum is not 0).

Given the structure of the pseudo-header, it is clear that when a UDP/IPv4 datagram passes through a NAT, not only is the IP-layer header checksum modified, but the UDP pseudo-header checksum must be appropriately modified because the IP-layer addressing and/or UDP-layer port numbers may have changed. NATs therefore routinely perform "layering violations" by modifying multiple layers of protocol within packets at the same time. Of course, given that the pseudo-header is itself a layering violation, a NAT has little choice. The particular rules that apply when UDP traffic is processed by a NAT are given in [RFC4787]. We also discussed them briefly in Chapter 7.

Recently there has been interest in relaxation of the UDP checksum for applications that are partially insensitive to errors (multimedia applications being the typical case). The discussion relates to whether having a *partial checksum* is a valuable concept. A partial checksum covers only a portion of the payload specified by the application. We discuss this in Section 10.6 in the context of UDP-Lite.

10.4 Examples

We will use the `sock` program [SOCK] to generate some UDP datagrams that we can watch with `tcpdump`. In the first example, we are running a server on the discard port (9) on the destination machine. In the second example, we have disabled the server, and the client is informed of this fact as illustrated here. Very few UDP-based services are made available in typical machine configurations because of security concerns, so the second part of the example is not unusual.

```
Linux% sock -v -u -i 10.0.0.3 discard
connected on 10.0.0.5.46274 to 10.0.0.3
wrote 1024 bytes
...
(1023 more times)

Linux% sock -v -u -i 10.0.0.3 discard
connected on 10.0.0.5.46294 to 10.0.0.3
wrote 1 bytes
write returned -1, expected 1024: Connection refused
```

When we execute the `sock` program, we specify the verbose mode, `-v`, to see the ephemeral port numbers, specify UDP `-u` instead of the default TCP, and use the `-i` option to send data instead of trying to read and write standard input and output. The default number of datagrams (1024) is sent to the destination host with IP address 10.0.0.3. In this case we have arranged a server to process incoming datagrams to the discard port. To capture the traffic sent, we use the following command on a host with access to the traffic stream:

```
Linux# tcpdump -n -p -s 1500 -vvv host 10.0.0.3 and \( udp or icmp \)
```

This command captures any UDP or ICMP traffic between the two machines (and possibly additional traffic not illustrated). The `-s 1500` option directs `tcpdump` to collect packets up to 1500 bytes in length (longer than the 1024 bytes we are sending, in this case), and the `-vvv` option indicates verbose printing. The `-n` option tells `tcpdump` to not convert IP addresses to machine names, and the `-p` option avoids placing the default network interface into promiscuous mode. The resulting `tcpdump` output is illustrated in Listing 10-1 (some lines have been wrapped for clarity).

Listing 10-1 `tcpdump` output showing packets from the first `sock` command (server running)

```
1 22:52:53.102838 10.0.0.5.46274 > 10.0.0.3.9:
    [udp sum ok] udp 1024 (DF) (ttl 64, id 24462, len 1052)
2 22:52:53.102964 10.0.0.5.46274 > 10.0.0.3.9:
    [udp sum ok] udp 1024 (DF) (ttl 64, id 24463, len 1052)
3 22:52:53.103091 10.0.0.5.46274 > 10.0.0.3.9:
    [udp sum ok] udp 1024 (DF) (ttl 64, id 24464, len 1052)
4 22:52:53.103215 10.0.0.5.46274 > 10.0.0.3.9:
    [udp sum ok] udp 1024 (DF) (ttl 64, id 24465, len 1052)
... repeated 1020 times ...
```

This output shows four 1052-byte UDP/IPv4 datagrams (1024 bytes of UDP payload plus 8 bytes of UDP header and the 20-byte IPv4 header) sent from IPv4 address 10.0.0.5 and port 46274 to port 9 (the discard port), with an inter-packet time of about 100µs. In addition, we may observe that UDP checksums are enabled and are valid (checked by `tcpdump`), that the *Don't Fragment* (DF) bit field is turned on, the IPv4 *TTL* field is 64, and the IPv4 *Identification* field is different (and increasing by 1) for each datagram. No ICMP traffic is generated,

and it would appear that all data was successfully delivered to the destination machine; although because there are no acknowledgments, we do not know with certainty. We shall see in Chapter 13 that the other major transport protocol, TCP, normally uses a handshake with the other end before the first byte of data can be sent and uses subsequent acknowledgments to know what data has been successfully transferred to the receiver.

The second time we run the `sock` program with the same arguments, but this time we send our datagrams to the discard service after the server has been disabled. Listing 10-2 shows the trace for this example (some lines have been wrapped for clarity).

Listing 10-2 `tcpdump` output showing ICMP Destination Unreachable (Port Unreachable) message from host (server disabled)

```
1 22:55:07.223094 10.0.0.5.46294 > 10.0.0.3.9:
    [udp sum ok] udp 1024 (DF) (ttl 64, id 37874, len 1052)

2 22:55:07.223134 10.0.0.3 > 10.0.0.5: icmp:
    10.0.0.3 udp port 9 unreachable for
    10.0.0.5.46294 > 10.0.0.3.9:
        udp 1024 (DF) (ttl 64, id 37874, len 1052)
    [tos 0xc0] (ttl 255, id 63302, len 576)
```

In this example we see somewhat different behavior. Here, only a single UDP datagram is sent, and an ICMP message is returned in response. Although all other parameters are the same, no server is running to receive the incoming datagrams. In this case, the underlying UDP implementation causes an ICMPv4 Destination Unreachable (Port Unreachable) message (see Chapter 8) to be generated and returned to the sender. This message includes a copy of the first 556 bytes of the original (“offending”) datagram. If the ICMP message is not discarded by the intervening network (accidentally or on purpose by firewalls), the sending application (if it is still running when the ICMP message arrives) can learn of the absence of the receiver and print an error, as indicated in the listing at the beginning of this section (i.e., the `write returned -1` message). Note that the returning ICMP error message contains enough information for the sending host to ascertain which port was not reachable. Finally, note that the source UDP port number changes each time the program is run. First it was 46274 and then it was 46294. We mentioned in Chapter 1 that the ephemeral port numbers used by clients are suggested to be in the range 49152 through 65535, so here we observe noncompliant behavior.

Note

For Linux, the local port parameter range can be easily modified by changing the contents of the file `/proc/sys/net/ipv4/ip_local_port_range`. In Windows Vista and later, the `netsh` command can be used to set the dynamic port range [KB929851]. See [IPORT] for current port numbers.

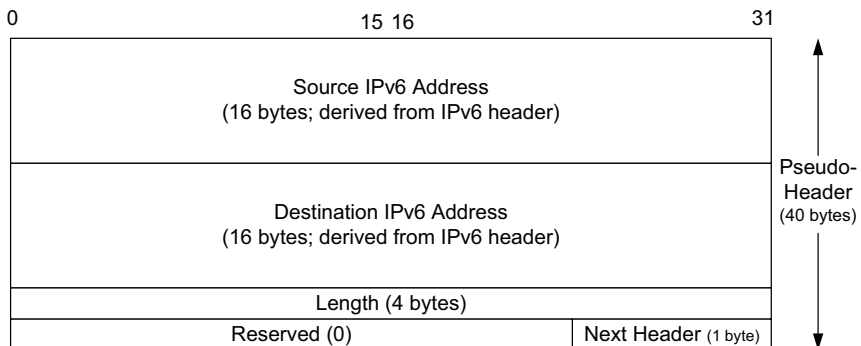


Figure 10-4 The UDP (and TCP) pseudo-header used with IPv6 ([RFC2460]). The pseudo-header includes the source and destination IPv6 addresses and a larger 32-bit *Length* field value. The pseudo-header checksum is required when UDP is used with IPv6 because the IPv6 header lacks a checksum. The *Next Header* field is copied from the last IPv6 header of the chain.

10.5 UDP and IPv6

Given its simplicity, UDP requires only small changes when operating over IPv6 instead of IPv4. The most obvious differences are the 128-bit addresses used by IPv6 and the corresponding effect on the construction of the pseudo-header. A related but more subtle distinction is that in IPv6, no IP-layer header checksum is present. Thus, if UDP were to operate with checksums disabled, there would be *no end-to-end check whatsoever* on the correctness of the IP-layer addressing information. For this reason, when UDP is used with IPv6, a pseudo-header checksum, common to both UDP and TCP, is required (by Section 8 of [RFC2460]). The construction of the pseudo-header is given in Figure 10-4. Note that the *Length* field has expanded from its IPv4 counterpart to 32 bits. Recall from earlier that this field is redundant for UDP, but we shall see in Chapter 13 that it is not redundant when used with TCP (either TCP/IPv4 or TCP/IPv6) and has thus been retained for use with both UDP/IPv6 and TCP/IPv6.

Expanding the discussion regarding the IPv6 packet length, two aspects of IPv6's packet size can affect UDP. First, in IPv6, the minimum MTU size is 1280 bytes (as opposed to the 576 bytes required by IPv4 as the minimum size required to be supported by all hosts). Second, IPv6 supports *jumbograms* (packets larger than 65,535 bytes). If we inspect the IPv6 header and option set (see Chapter 5), we can observe that with jumbograms, 32 bits are available to hold the payload length. This implies that a single UDP/IPv6 datagram could be very large indeed. As described in [RFC2675], this poses a problem for the UDP *Length* field in the UDP header, which is only 16 bits long. As such, when encapsulated in IPv6, a UDP/IPv6 datagram exceeding 65,535 bytes has its UDP *Length* field value set to 0. Note that the size of the *Length* field in the pseudo-header is still large enough

(32 bits). Computing the value of this field for IPv6 jumbograms involves taking the total length of the UDP header plus data. Checking this field when receiving a packet involves computing the size of the UDP datagram (header plus data) by subtracting the size of all IPv6 extension headers from the value found in the Jumbo Payload option, which gives the length of the IPv6 payload (i.e., the total datagram length minus the 40-byte IPv6 header). In the “unexpected” case where the *Length* field in the UDP header is 0 but no Jumbo Payload option is present, the UDP length can be inferred based on the nonzero IPv6 *Payload Length* field (see Section 4 of [RFC2675]).

10.5.1 Teredo: Tunneling IPv6 through IPv4 Networks

Although it was once thought that a worldwide transition to IPv6 might happen quickly, this has not materialized exactly as forecast. Consequently, a number of (theoretically temporary) *transition mechanisms* [RFC4213][RFC5969] have been proposed to ease the transition burden. One such mechanism is called *6to4* [RFC3056], whereby IPv6 packets used by hosts are encapsulated in IPv4 packets that may be delivered over an IPv4-only infrastructure. One problem with 6to4 is that it suffers from the same types of NAT traversal problems as other applications on the Internet. It is also known to have scaling problems that make its continued use unattractive [RFC6343]. Although methods we have seen such as ICE (see Chapter 7) could conceivably be used for handling this issue, a special protocol called *Teredo* (originally called “shipworm” but renamed based on the Latin name for a common genus of shipworm to avoid confusion with computer worms) has been devised especially to address this problem [RFC4380][RFC5991][RFC6081]. It is popular because of its widespread availability in modern versions of Microsoft Windows.

Teredo (also called *Teredo tunneling*) transports IPv6 datagrams in the payload area of UDP/IPv4 datagrams for systems that have no other IPv6 connectivity options. An example scenario is given in Figure 10-5. Teredo *clients* are IPv4/IPv6 hosts that implement a Teredo tunneling interface. Such interfaces are assigned special Teredo addresses using the 2001::/32 IPv6 prefix after having successfully engaged in a “qualification” procedure, described in the next paragraph. Teredo *servers*, which serve a general purpose similar to STUN servers (Chapter 7), are used to help establish direct tunnels of Teredo-encapsulated IPv6 packets through NATs. Teredo *relays* serve a purpose similar to TURN servers and consequently may take significant processing resources if used by many clients. Note that servers must include all of the capabilities of relays, but not vice versa. Using Teredo relays is a “last-resort” option for IPv6 connectivity. Nodes cease to perform Teredo tunneling if they discover that they have any other IPv6 connectivity option (e.g., direct or using 6to4).

Referring to Figure 10-5, a Teredo client is initially configured with the name or IPv4 address and UDP port number (usually 3544) of a Teredo server. Teredo was initially developed by Microsoft, and a Teredo server is available using the

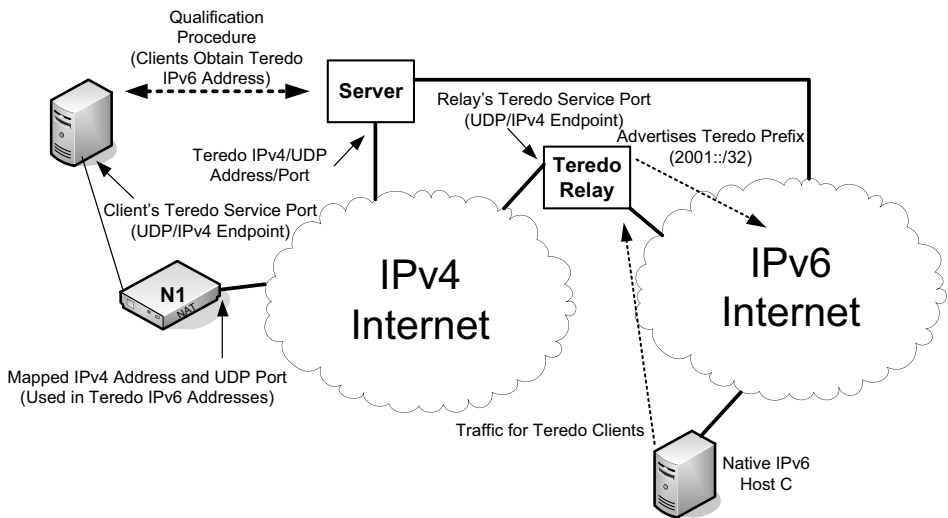


Figure 10-5 Teredo, an IPv6 transition mechanism, encapsulates IPv6 datagrams and optional trailers within the payload area of UDP/IPV4 datagrams to carry IPv6 traffic across IPv4-only infrastructures. The server helps clients obtain an IPv6 address and determine their mapped addresses and port numbers. Relays, if required, can forward traffic between Teredo, 6to4, and native IPv6 clients.

name `teredo.ipv6.microsoft.com`. When ready to obtain an address, it starts the *qualification procedure*. The client begins by sending an ICMPv6 RS packet (see Chapter 8) from one of its link-local IPv6 addresses using its Teredo service port, the agent responsible for encapsulating and decapsulating IPv6 traffic within UDP/IPV4. The encapsulation format is the Origin Indication format, one of two shown in Figure 10-6.

Successful responses are ICMPv6 RA messages that use the Origin Indication Encapsulation format from Figure 10-6. The RA contains a Prefix Information option with a valid Teredo prefix (see Chapter 2). The Origin Indication provides the client with knowledge of its own mapped address and port information. The source address of the RA is a valid link-local IPv6 address of the server. The destination is the client's link-local IPv6 address used as the source of the RS message. Assuming that all goes well, the client is now "qualified" and can build its Teredo IPv6 address based on the prefix and origin information provided by the server. The Teredo address is an IPv6 address constructed from various parameters using the format of Figure 10-7.

A Teredo address (see Figure 10-7) contains the Teredo prefix (2001::/32), the IPv4 address of the Teredo server, a 16-bit *Flags* field detailed in the next paragraph, followed by the mapped port number and mapped IPv4 address. The last two values are the addressing information of the client as seen from the Teredo server and are usually determined by the client's outermost NAT. The actual

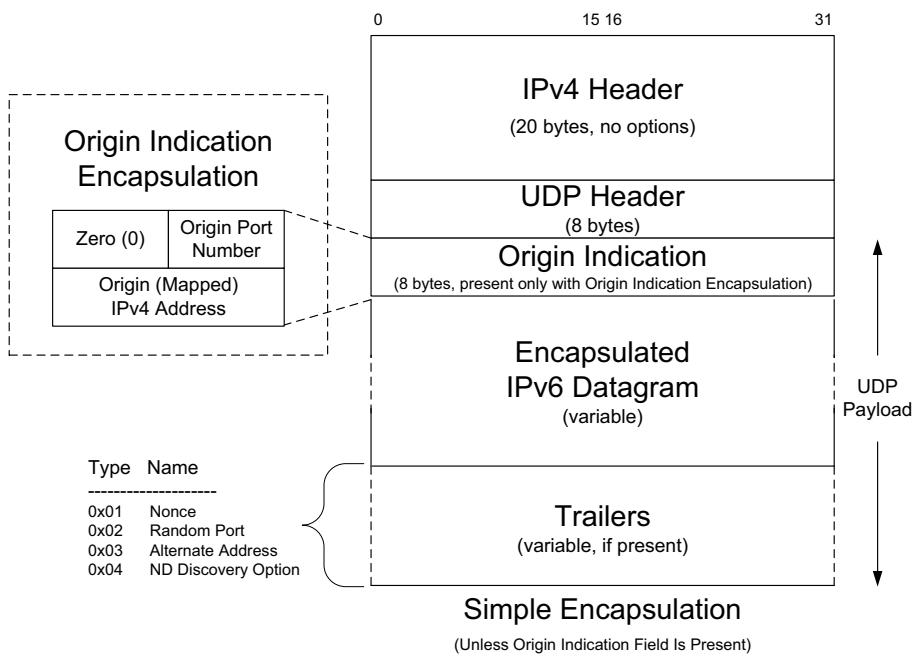


Figure 10-6 The Simple Encapsulation and Origin Indication Encapsulation formats used by Teredo. The Origin Indication Encapsulation carries UDP address and port number information between the UDP header and encapsulated IPv6 datagram. This information is used to inform Teredo clients about their mapped addresses and port numbers when creating a Teredo address. Addresses and port numbers are “obfuscated” by inverting each bit present to fend off NATs that attempt to rewrite this information. Zero or more trailers may be present, encoded as TLV triples. They are used to implement a number of Teredo extensions (e.g., support for symmetric NATs).

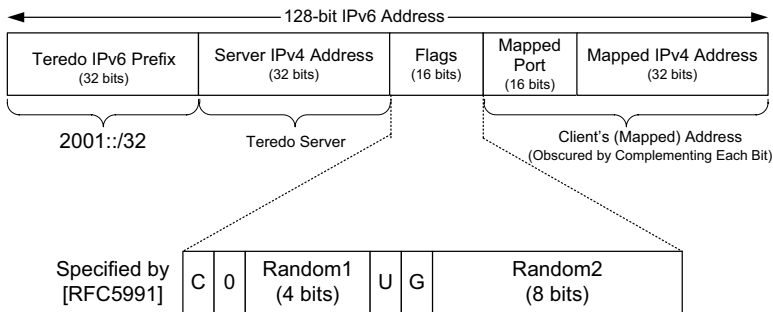


Figure 10-7 Teredo clients use IPv6 addresses from the 2001::/32 Teredo prefix. The subsequent bits contain the Teredo server’s IPv4 address, 16 flag bits that identify the type of NAT encountered and random bits to help thwart address-guessing attacks, and 16 bits containing the client’s mapped port number and the client’s mapped 32-bit IPv4 address. The last two values are “obfuscated.”

address and port number information is bitwise-inverted to cause indiscriminate NATs to not rewrite them.

The 16-bit *Flags* field has been used to indicate the type of NAT discovered during the qualification process. Some NATs (formerly called symmetric NATs—the types of NATs that have either address-dependent mapping or address- and port-dependent mapping along with either address-dependent or address- and port-dependent filtering behavior) work with Teredo only when extensions are supported (see later in this section), but the most common types for household networks (including “cone NATs”—NATs with endpoint-independent mapping and endpoint-independent filtering behavior) work without such extensions. Originally, the C (cone NAT) bit field was used to indicate if a cone NAT was encountered and to arrange appropriate support, but this usage is now deprecated and the field should be set to 0 (clients ignore the field; servers inspect it to look for legacy clients). The next bit field is set to 0. The *U* (*Universal*) and *G* (*Group*) bit fields are available for future use but are also currently set to 0. The *Random1* and *Random2* field values are chosen as random numbers according to [RFC5991] to make Teredo addresses harder to guess (a security measure intended to reduce random probes by potential attackers).

Once a qualified client builds its Teredo address, it can send IPv6 traffic. For details on what happens when qualification fails or a secure qualification is to be used, see [RFC4380]. In general, a Teredo client may wish to communicate with another client on the same link, another client within the IPv4 Internet, or with a host on the IPv6 Internet. In each case, Teredo provides some UDP/IPv4-based alternative to IPv6 ND. For clients on the same link, Teredo uses an IPv4 multicast discovery protocol that operates using the multicast address 224.0.0.253. Special Teredo “bubble” packets (those with no data payload) are used to determine if a destination is on the same link. Such bubbles appear as minimum-size Teredo packets using the Simple Encapsulation format of Figure 10-6. They contain an IPv6 header with the *Destination IP Address* field set to the target of the communication. The IPv6 packet contains an IPv6 header with no payload or additional extensions (the *Next Header* field is set to 0x3b, indicating none). For clients within the IPv4 Internet, recall that the Teredo IPv6 address contains the IPv4-mapped address and port number. Thus, it is straightforward for one client to send a Teredo-encapsulated packet to another’s NAT. For NATs that are restrictive, Teredo uses bubble packets to perform hole punching and establish UDP NAT mappings (see Chapter 7 and [RFC6081]).

When a qualified client has a packet to send to an IPv6 host (i.e., one that does not use a Teredo address), it first determines whether it already knows a Teredo relay for the destination. If so, the packet is sent using Simple Encapsulation. If not, the client formats an ICMPv6 Echo Request containing a large (e.g., 64-bit) random number and sends it to the IPv6 destination by way of the Teredo server. The server forwards this packet to the destination IPv6 host. The receiving host sees an incoming IPv6 datagram with the source address equal to the Teredo address of the client. It forms an Echo Reply, which is routed to the nearest Teredo relay. The

relay then forwards the reply back to the client. The receiving client observes the IPv4 address of the relay and updates a cache to indicate that subsequent packets destined for the IPv6 host should use the relay address it just determined.

As of [RFC6081], Teredo can support a number of optional extensions, several of which help to support Teredo operation with symmetric NATs. The extensions are protocol behavior modifications and include the following: Symmetric NAT Support (SNS), UPnP-Enabled Symmetric NAT (UP), Port-Preserving Symmetric NAT (PP), Sequential Port-Symmetric NAT (SP), Hairpinning (HP), and Server Load Reduction (SLR). The extensions can be used independently, except that both the UP and PP extensions depend on the SNS extension. The various NAT types that can be supported with various extension combinations are given in a table (see Section 3 of [RFC6081]).

To implement the extensions, one or more *trailers* may be present in a Teredo message. Trailers are encoded as an ordered list of TLV combinations, using the same basic format as for ICMPv6 ND options (Figure 8-41), which contain an 8-bit *Type* field and an 8-bit *Length* field. The two highest-order bits of the *Type* field encode what processing should be performed if the host does not recognize the trailer type. The bit pattern 01 indicates that the host should discard the packet; all others indicate that the unknown trailer should be skipped and others should be processed in order. The official list of trailer type values is maintained by the IANA [TTYPES]. The trailers currently defined are listed here in Table 10-1.

Table 10-1 Teredo trailers are carried after the IPv6 payload encapsulated in a UDP/IPv4 datagram. Each trailer has a type value, name, and associated explanation. In some cases, the length value is a constant.

Type	Length	Name	Use	Notes
0x00	Reserved	(Unassigned)	(Unassigned)	(Unassigned)
0x01	0x04	Nonce	SNS, UP, PP, SP, HP	32-bit nonce for protection against replays (see Chapter 18)
0x02	Reserved	(Unassigned)	(Unassigned)	(Unassigned)
0x03	[8, 26]	Alternate Address	HP	Additional addresses/ports usable by Teredo clients behind the same NAT
0x04	0x04	ND Option	SLR	Allows NAT refresh using direct bubbles (that carry NS messages)
0x05	0x02	Random Port	PP	Sender's predicted mapped port

The Nonce trailer contains a 32-bit random value that is unique for each message. It is a security measure to guard against replay attacks (see Chapter 18) and is used with either HP or SNS (IPv4 address, port) pairs. Each pair is 6 bytes long, and the trailer can hold from one to four such pairs. These pairs identify

The *Checksum Coverage* field in Figure 10-8 is the number of bytes (starting from the first byte of the UDP-Lite header) covered by the checksum. Except for the special value 0, the minimum value is 8, because the UDP-Lite header itself is always required to be covered by the checksum. The value 0 indicates that the entire payload is covered by the checksum, as with conventional UDP. There is a slight issue with IPv6 jumbograms because of the limited space used to hold the *Checksum Coverage* field. For such datagrams, the number of bytes covered can be at most 64KB or the entire datagram (i.e., when the *Checksum Coverage* field has value 0). Special socket API options are used for applications to specify the use of UDP-Lite (IPPROTO_UDPLITE) and the amount of checksum coverage requested (using the SOL_UDPLITE, UDPLITE_SEND_CSCOV, and UDPLITE_RECV_CSCOV options to `setsockopt`).

10.7 IP Fragmentation

As we described in Chapter 3, link-layer framing normally imposes an upper limit on the maximum size of a frame that can be transmitted. To keep the IP datagram abstraction consistent and isolated from link-layer details, IP employs *fragmentation* and *reassembly*. Whenever the IP layer receives an IP datagram to send, it determines which local interface the datagram is to be sent over next (via a forwarding table lookup; see Chapter 5) and what MTU is required. IP compares the outgoing interface's MTU with the datagram size and performs fragmentation if the datagram is too large. Fragmentation in IPv4 can take place at the original sending host and at any intermediate routers along the end-to-end path. Note that datagram fragments can themselves be fragmented. Fragmentation in IPv6 is somewhat different because only the source is permitted to perform fragmentation. We saw an example of IPv6 fragmentation in Chapter 5.

When an IP datagram is fragmented, it is not reassembled until it reaches its final destination. Two reasons have been given for this, the second more compelling than the first. First, not performing reassembly within the network alleviates the forwarding software (or hardware) in routers from implementing this feature. Second, it is possible for different fragments of the same datagram to follow different paths to their common destination. If this happens, no single router along the path would in general be capable of reassembling the original datagram because it would see only a subset of the fragments. The first argument is not terribly convincing at face value given the current performance levels of routers, but it is even less convincing when one considers that most routers must ultimately be capable of functioning as end hosts anyhow (e.g., when being managed or configured). The second argument remains compelling.

10.7.1 Example: UDP/IPv4 Fragmentation

An application using UDP may need to worry about the size of the resulting IP datagram it creates if it wishes to avoid IP-layer fragmentation. In particular, if

the size of the resulting datagram exceeds the link's MTU, the IP datagram is split across multiple IP packets, which can lead to performance issues because *if any fragment is lost, the entire datagram is lost*. Figure 10-9 illustrates the situation when a 3020-byte UDP/IPv4 datagram is split into multiple IPv4 packets.

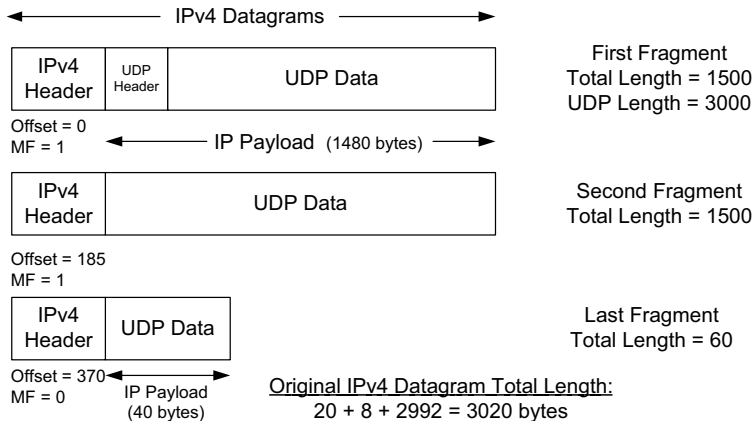


Figure 10-9 A single UDP datagram with 2992 UDP payload bytes is fragmented into three UDP/IPv4 packets (no options). The UDP header that contains the source and destination port numbers appears only in the first fragment (a complicating factor for firewalls and NATs). Fragmentation is controlled by the *Identification*, *Fragment Offset*, and *More Fragments (MF)* fields in the IPv4 header.

In Figure 10-9, we conclude that the original UDP datagram included 2992 bytes of application (UDP payload) data and 8 bytes of UDP header, resulting in an IPv4 *Total Length* field value of 3020 bytes (recall that this size includes a 20-byte IPv4 header as well). When this datagram was fragmented into three packets, 40 extra bytes were created (20 bytes for each of the newly created IPv4 fragment headers). Thus, the total number of bytes sent is 3060, an increase in IP-layer overhead of about 1.3%. The *Identification* field value (set by the original sender) is copied to each fragment and is used to group them together when they arrive. The *Fragment Offset* field gives the offset of the first byte of the fragment payload byte in the original IPv4 datagram (in 8-byte units). Clearly, the first fragment always has offset 0. Here, we observe the second fragment with offset 185 ($185 * 8 = 1480$). The size of 1480 is the size of the first fragment less the size of the IPv4 header. A similar analysis applies to the third fragment. Finally, the *MF* bit field indicates whether more fragments in the datagram should be expected and is 0 only in the final fragment. When the fragment with *MF* = 0 is received, the reassembly process can ascertain the length of the original datagram, as a sum of the *Fragment Offset* field value (times 8) and the IPv4 *Total Length* field value (minus the IPv4 header length). Because each *Offset* field is relative to the original datagram, the

reassembly process can handle fragments that arrive out of order. When a datagram is fragmented, the *Total Length* field in the IPv4 header of each fragment is changed to be the total size of that fragment.

Although IP fragmentation looks transparent, there is one feature mentioned earlier that makes it less than desirable: if one fragment is lost, the entire datagram is lost. To understand why this happens, realize that IP itself has no error correction mechanism of its own. Mechanisms such as timeout and retransmission are left as the responsibility of the higher layers. (TCP performs timeout and retransmission; UDP does not. Some UDP-based *applications* perform timeout and retransmission themselves, but this happens at a layer above UDP.) When a fragment of a TCP segment is lost, TCP retransmits the entire TCP segment, which corresponds to an entire IP datagram. There is no way to resend only one fragment of a datagram. Indeed, if the fragmentation was done by an intermediate router, and not the originating system, there is no way for the originating system to know how the datagram was fragmented. For this reason, fragmentation is often avoided. [KM87] provides arguments for avoiding fragmentation.

Using UDP, it is easy to generate IP fragmentation. (We shall see later that TCP tries to avoid fragmentation and that it is nearly impossible for an application to force TCP to send segments large enough to require fragmentation.) We can use our `sock` program and increase the size of the datagram until fragmentation occurs. On an Ethernet, the maximum amount of data in a frame is ordinarily 1500 bytes (see Chapter 3), which leaves at most 1472 bytes for application data to avoid fragmentation, assuming 20 bytes for the IPv4 header and 8 bytes for the UDP header.¹ We will run our `sock` program with data sizes of 1471, 1472, 1473, and 1474 bytes. We expect the last two to cause fragmentation:

```
Linux% sock -u -i -n1 -w1471 10.0.0.3 discard
Linux% sock -u -i -n1 -w1472 10.0.0.3 discard
Linux% sock -u -i -n1 -w1473 10.0.0.3 discard
Linux% sock -u -i -n1 -w1474 10.0.0.3 discard
```

Listing 10-3 illustrates the `tcpdump` output (some lines are wrapped for clarity).

Listing 10-3 UDP fragmentation on a 1500-byte MTU Ethernet link

```
1 23:42:43.562452 10.0.0.5.46530 > 10.0.0.3.9:
    udp 1471 (DF) (ttl 64, id 61350, len 1499)
2 23:42:50.267424 10.0.0.5.46531 > 10.0.0.3.9:
    udp 1472 (DF) (ttl 64, id 62020, len 1500)
3 23:42:57.814555 10.0.0.5 > 10.0.0.3:
    udp (frag 37671:1@1480) (ttl 64, len 21)
4 23:42:57.814715 10.0.0.5.46532 > 10.0.0.3.9:
    udp 1473 (frag 37671:1480@0+) (ttl 64, len 1500)
```

1. Recall the assumption that no options are used. For IPv4 datagrams with options, the header exceeds 20 bytes, up to a maximum of 60 bytes.

```
5 23:43:04.368677 10.0.0.5 > 10.0.0.3:
    udp (frag 37672:2@1480) (ttl 64, len 22)
6 23:43:04.368838 10.0.0.5.46535 > 10.0.0.3.9:
    udp 1474 (frag 37672:1480@0+) (ttl 64, len 1500)
```

The first two UDP datagrams (packets 1 and 2) fit into 1500-byte Ethernet frames (using the typical “DIX” or “Ethernet” encapsulation) and are not fragmented. In the third case, the length of the IPv4 datagram corresponding to the application write of 1473 bytes is 1501, which must be fragmented (packets 3 and 4). Similarly, the datagram generated by the write of 1474 bytes is 1502 bytes long and is also fragmented (packets 5 and 6).

When it captures a fragmented datagram, `tcpdump` prints additional information. First, the outputs `frag 37671` (packets 3 and 4) and `frag 37672` (packets 5 and 6) specify the value of the *Identification* field in the IPv4 header. The next number in the fragmentation information (between the colon and the @ sign in packets 4 and 6) is the IPv4 packet size, excluding the IPv4 header. The first fragment of both datagrams contains 1480 bytes of data: 8 bytes for the UDP header and 1472 bytes of user data. (The 20-byte option-free IPv4 header makes the packet exactly 1500 bytes.) The second fragment of the first fragmented datagram (packet 3) contains 1 byte of data (the remaining byte of user data). The second fragment of the second fragmented datagram (packet 5) contains the remaining 2 bytes of user data. Fragmentation requires that the data portion of the generated fragments (that is, everything excluding the IPv4 header) be a multiple of 8 bytes for all fragments other than the final one. In this example, 1480 is a multiple of 8. (Constrast this case with the IPv6 fragmentation example in Chapter 5, where the 1500-byte Ethernet MTU was not able to be fully utilized.)

The number following the @ is the offset of the data in the fragment from the start of the datagram. The first fragment of each new fragmented datagram starts with offset 0 (packets 4 and 6), and the second fragment of both datagrams starts at byte offset 1480 (packets 3 and 5). The + sign following an offset value means that there are more fragments composing this datagram, corresponding to the *MF* bit field being set to 1 in the 3-bit *Flags* field in the IPv4 header.

One observation that may be surprising is that the fragments with larger offsets are delivered *prior* to the first fragments. In effect, the sender has intentionally reordered the fragments. Upon reflection, we realize that this behavior can be beneficial. If the last fragment is delivered first, the receiving host is able to ascertain the maximum amount of buffer space it will require in order to reassemble the entire datagram. Given that the reassembly process is robust to reordering anyhow, this presents no major problem. On the other hand, there are techniques that would like to take advantage of higher-layer information available in the first fragment (including UDP port numbers) that is not present in the later fragments [KEWG96].

Finally, note that packets 3 and 5 (fragments other than the first) omit the source and destination UDP port numbers. In order for `tcpdump` to print the port

numbers associated with fragments other than the first, it would have to reassemble fragmented datagrams to recover the port numbers that appear only in the UDP header located in the first fragments (which it does not do).

10.7.2 Reassembly Timeout

The IP layer must start a timer when any fragment of a datagram first arrives. If this were not done, fragments that never arrive (as we see in Listing 10-4) could eventually cause the receiver to run out of buffers and can constitute a form of attack opportunity. The example in the listing was created with a special program that constructs and sends only the first two fragments of an ICMPv4 Echo Request message separated by a delay but then never sends any additional fragments. Listing 10-4 illustrates the response (some lines have been wrapped for clarity).

Listing 10-4 Timeout during IPv4 fragment reassembly

```
1 17:35:59.609387 10.0.0.5 > 10.0.0.3:
    icmp: echo request (frag 28519:380@0+) (ttl 255, len 400)
2 17:36:19.617272 10.0.0.5 > 10.0.0.3:
    icmp (frag 28519:380@376+) (ttl 255, len 400)
3 17:36:29.602373 10.0.0.3 > 10.0.0.5:
    icmp: ip reassembly time exceeded for 10.0.0.5 > 10.0.0.3:
    icmp: echo request (frag 28519:380@0+) (ttl 255, len 400)
    [tos 0xc0](ttl 64, id 38816, len 424)
```

Here we see that the first fragment (in both time and sequence space) is sent, with total length 400. A second fragment is sent 20s later, but no final fragment is ever sent. Thirty seconds after receiving the first fragment, the target machine responds with an ICMPv4 Time Exceeded (code 1) message, telling the sender that the datagram has been discarded by including a copy of the first fragment. A normal timeout value is 30 or 60s. As we can see, the timer starts when any of the fragments is received and is not reset when new fragments arrive. Thus, the timer places a sort of bound on the maximum span of time by which fragments of the same datagram can be separated.

Note

Historically, most Berkeley-UNIX-derived IP implementations simply never generated this error. While these implementations *did* set a timer, and *did* discard all fragments when the timer expired, the ICMP error was never generated. Another detail one sometimes encounters is that an implementation is not required to generate the ICMP error unless the *first* fragment has been received (i.e., the one with the 0 *Fragment Offset* field). The reason is that the receiver of the ICMP error cannot tell which user process sent the datagram that was discarded, because the transport-layer header is not available. It is assumed that higher-layer protocols will eventually time out and retransmit if necessary.

10.8 Path MTU Discovery with UDP

Let us examine the interaction between an application using UDP and the path MTU discovery mechanism (PMTUD) [RFC1191]. For a protocol such as UDP, in which the calling application is generally in control of the outgoing datagram size, it is useful if there is some way to determine an appropriate datagram size if fragmentation is to be avoided. Conventional PMTUD uses ICMP PTB messages (see Chapter 8) in determining the largest packet size along a routing path that can be used without inducing fragmentation. These messages are typically processed below the UDP layer and are not directly visible to an application, so either an API call is used for the application to learn the best current estimate of the path MTU size for each destination with which it has communicated, or the IP layer can perform PMTUD independently without the application knowing. The IP layer often caches PMTUD information on a per-destination basis and times it out if it is not refreshed.

10.8.1 Example

In the following example, we use the `sock` program to create a UDP datagram that produces a 1501-byte IPv4 datagram. Both our host system and the attached LAN support an MTU larger than 1500 bytes, but the outgoing link to the Internet at the router does not. The command attempts to send three UDP messages to the `echo` service (UDP port 7) in quick succession.

```
Linux% sock -u -i -n 3 -w1473 www.cs.berkeley.edu echo
```

Listing 10-6 illustrates the corresponding packet trace we can see using `tcpdump` at the sender (some lines are wrapped for clarity).

Listing 10-6 `tcpdump` output illustrating ICMP PTB message. The suggested MTU is included.

```
1 14:42:18.359366 IP (tos 0x0, ttl 64, id 18331, offset 0, flags [DF],
   proto UDP (17), length 1501)
   12.46.129.28.33954 > 128.32.244.172.7: UDP, length 1473

2 14:42:18.359384 IP (tos 0x0, ttl 64, id 18332, offset 0, flags [DF],
   proto UDP (17), length 1501)
   12.46.129.28.33954 > 128.32.244.172.7: UDP, length 1473

3 14:42:18.359402 IP (tos 0x0, ttl 64, id 18333, offset 0, flags [DF],
   proto UDP (17), length 1501)
   12.46.129.28.33954 > 128.32.244.172.7: UDP, length 1473

4 14:42:18.360156 IP (tos 0x0, ttl 255, id 23457, offset 0,
   flags [none], proto ICMP (1), length 56)
   12.46.129.1 > 12.46.129.28: ICMP
   128.32.244.172 unreachable - need to frag (mtu 1500), length 36
```

```
IP (tos 0x0, ttl 63, id 18331, offset 0, flags [DF],
proto UDP (17), length 1501)
12.46.129.28.33954 > 128.32.244.172.7: UDP, length 1473
```

In Listing 10-6 we see three UDP datagrams of 1473 UDP (application) payload bytes each. Each produces a 1501-byte (unfragmented) IPv4 datagram. Each of these datagrams has the IPv4 *DF* bit field turned on (the default on this system), so when one of them reaches a router (IPv4 address 12.46.129.1), an ICMPv4 PTB message is produced, which includes the suggested next-hop MTU of 1500 bytes. We may also observe that the ICMPv4 messages produced contain the UDP/IPv4 headers (and first 8 data bytes) from our discarded (“offending”) datagrams. In this example, our `sock` program sent its datagrams so quickly (in under a millisecond) that it completed its execution before any of the ICMP messages were returned and processed.

Note

The 1500-byte MTU is now a common minimum MTU among ISPs. Some ISPs that incorporate PPPoE for address assignment and management use smaller, 1492-byte MTUs. The PPPoE header (see Chapter 3) comprises 6 bytes, and the following PPP header is 2, leaving $1500 - 6 - 2 = 1492$ bytes for the encapsulated datagram.

If we use another destination host (one about which we have no path MTU history), and we add additional delay between writes, we can observe different behavior. Using the `sock` command with the `-p 2` option, which adds 2s of delay between each send, we use the following two (identical) commands:

```
Linux% sock -u -i -n 3 -w1473 -p 2 www.wisc.edu echo
write returned -1, expected 1473: Message too long
Linux% sock -u -i -n 3 -w1473 -p 2 www.wisc.edu echo
```

The `tcpdump` output, using an alternative version of `tcpdump`, for these commands is given in Listing 10-7 (some lines are wrapped for clarity).

Listing 10-7 Illustration of successful Path MTU discovery on 3000-byte MTU link adapting to 1500-byte path MTU

```
1 17:22:16.331023 IP (tos 0x0, ttl 64, id 58648, offset 0, flags [DF],
  proto: UDP (17), length: 1501)
  12.46.129.28.33955 > 144.92.9.185.7: UDP, length 1473

2 17:22:16.331581 IP (tos 0x0, ttl 255, id 38518, offset 0,
  flags [none], proto: ICMP (1), length: 56)
  12.46.129.1 > 12.46.129.28: ICMP
  144.92.9.185 unreachable - need to frag (mtu 1500), length 36
```

```
IP (tos 0x0, ttl 63, id 58648, offset 0, flags [DF],
proto: UDP (17), length: 1501)
12.46.129.28.33955 > 144.92.9.185.7: UDP, length 1473

3 17:22:24.284866 IP (tos 0x0, ttl 64, id 53776, offset 0, flags [+],
proto: UDP (17), length: 1500)
12.46.129.28.33955 > 144.92.9.185.7: UDP, length 1473

4 17:22:24.284873 IP (tos 0x0, ttl 64, id 53776, offset 1480,
flags [none], proto: UDP (17), length: 21)
12.46.129.28 > 144.92.9.185: udp

5 17:22:26.293554 IP (tos 0x0, ttl 64, id 53777, offset 0, flags [+],
proto: UDP (17), length: 1500)
12.46.129.28.33955 > 144.92.9.185.7: UDP, length 1473

6 17:22:26.293559 IP (tos 0x0, ttl 64, id 53777, offset 1480,
flags [none], proto: UDP (17), length: 21)
12.46.129.28 > 144.92.9.185: udp

7 17:22:28.301469 IP (tos 0x0, ttl 64, id 53778, offset 0, flags [+],
proto: UDP (17), length: 1500)
12.46.129.28.33955 > 144.92.9.185.7: UDP, length 1473

8 17:22:28.301474 IP (tos 0x0, ttl 64, id 53778, offset 1480,
flags [none], proto: UDP (17), length: 21)
12.46.129.28 > 144.92.9.185: udp
```

In Listing 10-7 we can see that the first time we ran our program it resulted in an error due to the ICMPv4 PTB message. The extra time provided within and between runs provides an opportunity for the PTB message to reach the sending host and for the error condition to be delivered back to the sender for processing. Interestingly, when we run the program a second time, the path MTU has been discovered to be 1500 bytes and the system is able to send the program's three datagrams using fragmentation (packets 3, 5, and 7 indicate the first fragments of the three datagrams). After 15 minutes (not illustrated), the path MTU information is considered stale, the datagram is sent unfragmented, another ICMPv4 PTB message is returned, and the process repeats.

Note

[RFC1191] recommends a PMTU value determined using PMTUD to be considered stale after 10 minutes. Path MTU discovery can sometimes cause problems because firewalls and filtering gateways may drop ICMP traffic indiscriminately, which can harm the PMTU discovery algorithm. Because of this, it is possible to disable PMTU discovery on a system-wide or finer-granularity basis. On Linux, the file `/proc/sys/net/ipv4/ip_no_pmtu_disc` can have a 1 written to it to disable the feature. On Windows, it involves editing the registry entry `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\Tcpip\Parameters\`

EnablePMTUDiscovery to include the value 0. An alternative to conventional PMTUD that does not use ICMP has also been developed [RFC4821]; we will discuss it in the context of TCP in Chapter 15.

10.9 Interaction between IP Fragmentation and ARP/ND

Using UDP, we can see the relationship between induced IP fragmentation and typical implementations of ARP. Recall that ARP is used to map IP-layer addresses to corresponding MAC-layer addresses on the same IPv4 subnet (see Chapter 4). The questions with which we are concerned include, When multiple fragments are to be sent, how many ARP messages should be generated, and how many of the fragments are held until a pending ARP request/response is completed? (Similar questions apply with IPv6 ND.) Returning to our host and LAN using a 1500-byte MTU, we use the following two commands to see the answer:

```
Linux% sock -u -i -n1 -w8192 10.0.0.20 echo
Linux% sock -u -i -n1 -w8192 10.0.0.3 echo
```

These arguments cause our sock program to generate a single UDP datagram with 8192 bytes of user data. We expect this to generate six fragments on an Ethernet using a 1500-byte MTU size. We also make sure that the ARP cache is empty before running the program, so that an ARP request and reply must be exchanged before any fragments are sent (see Listing 10-8; some lines are wrapped for clarity).

Listing 10-8 ARP and fragmentation on Ethernet with 1500-byte MTU

```
1 15:45:49.063561 arp who-has 10.0.0.20 tell 10.0.0.5
2 15:45:50.059523 arp who-has 10.0.0.20 tell 10.0.0.5
3 15:45:51.059505 arp who-has 10.0.0.20 tell 10.0.0.5
---
4 15:46:08.555725 arp who-has 10.0.0.3 tell 10.0.0.5
5 15:46:08.555973 arp reply 10.0.0.3 is-at 0:0:c0:c2:9b:26
6 15:46:08.555992 10.0.0.5 > 10.0.0.3:
    udp (frag 27358:1480@2960+) (ttl 64, len 1500)
7 15:46:08.555998 10.0.0.5 > 10.0.0.3:
    udp (frag 27358:1480@1480+) (ttl 64, len 1500)
8 15:46:08.556004 10.0.0.5.32808 > 10.0.0.3.7:
    udp 8192 (frag 27358:1480@0+) (ttl 64, len 1500)
```

For this experiment, we happen to know that there is no running host assigned address 10.0.0.20, so we should expect no reply. In the first part of Listing 10-8 (packets 1–3), we observe three ARP requests spaced approximately 1s apart. No host responds after three requests are sent, so the ARP requestor gives up. In the next case, an ARP response is received in about 250µs, and a fragment is sent about 20µs thereafter. After this, the remaining fragments are sent very closely

together, within about 6 μ s of each other. Recall that in this system (Linux), the last fragment is sent first.

Note

Historically, the interaction between fragmentation and ARP has been problematic. For example, in some cases an ARP request was sent for each fragment, and in many cases only one of the fragments was queued pending the ARP response (thus losing the datagram, as all but one of its fragments were discarded). The first problem was addressed in [RFC1122], which requires an implementation to prevent this type of ARP flooding. The recommended maximum rate is one per second. The second problem is also discussed in [RFC1122], but this states only that the link layer “SHOULD save (rather than discard) at least one (the latest) packet of each set of packets destined to the same unresolved IP address, and transmit the saved packet when the address has been resolved.” This approach can lead to unnecessary packet loss and has been addressed in individual implementations by providing a larger queue for packets while their ARP requests are pending.

10.10 Maximum UDP Datagram Size

Theoretically, the maximum size of an IPv4 datagram is 65,535 bytes, imposed by the 16-bit *Total Length* field in the IPv4 header (see Chapter 5). With an optionless IPv4 header of 20 bytes and a UDP header of 8 bytes, this leaves a maximum of 65,507 bytes of user data in a UDP datagram. For IPv6, the 16-bit *Payload Length* field permits an effective UDP payload of 65,527 bytes (8 of the 65,535 IPv6 payload bytes are used for the UDP header), assuming jumbograms are not being used. There are two main reasons why a full-size datagram of these sizes may not be delivered end-to-end, however. First, the system’s local protocol implementation may have some limitation. Second, the receiving application may not be prepared to handle such large datagrams.

10.10.1 Implementation Limitations

Protocol implementations provide an API to applications that pick some default buffer size for sending and receiving. Some implementations provide defaults that are less than the maximum IP datagram size, and some actually do not support sending datagrams larger than a few tens of kilobytes (although this problem is not common).

The sockets API [UNP3] provides a set of functions that an application can call to set or query the size of the receive and send buffers. For a UDP socket, this size is directly related to the maximum size of UDP datagram the application can read or write. Typical default values are 8192 bytes or 65,535 bytes, but these can generally be made larger by invoking the `setsockopt()` API call.

We mentioned in Chapter 5 that a host is required to provide enough buffering to receive at least a 576-byte IPv4 datagram on reassembly. Many UDP applications are designed to restrict their application data size to 512 bytes or less (resulting in IPv4 datagrams under 576 bytes), to stay below this limit. Examples employing such limitations to their UDP datagram size include the DNS (see Chapter 11) and DHCP (see Chapter 6).

10.10.2 Datagram Truncation

Just because UDP/IP is capable of sending and receiving a datagram of a given (large) size does not mean the receiving application is prepared to read that size. UDP programming interfaces allow the application to specify the maximum number of bytes to return each time a network read operation completes. What happens if the received datagram exceeds the size specified?

In most cases, the answer to this question is that the API *truncates* the datagram, discarding any excess data in the datagram beyond the number of bytes specified by the receiving application. However, the exact behavior varies from implementation to implementation. Some systems provide the unconsumed portion of the datagram in subsequent read operations, and others inform the caller of how much data was truncated (or, in yet other cases, that *some* data was truncated, but not exactly how much).

Note

In Linux, the MSG_TRUNC option may be given to the sockets API to discover how much data was truncated. On HP-UX, MSG_TRUNC is instead a flag set when a read call returns that some data was truncated. The sockets API under SVR4 (including Solaris 2.x) does *not* truncate the datagram. Any excess data is returned in subsequent reads. The application is not notified that multiple reads are being fulfilled from a single UDP datagram.

When we discuss TCP we shall see that it provides a continuous stream of bytes to the application, without any message boundaries. Thus, an application consumes however much data it requests, provided sufficient data is available (if not, it usually waits).

10.11 UDP Server Design

There are some characteristics of UDP that affect the design and implementation of networking application software wishing to use it [RFC5405]. Servers typically interact with the operating system, and most need a way to handle multiple clients at the same time. Client design and implementation are usually simpler, and therefore we will not discuss them here.

In the typical client/server scenario, a client starts, immediately communicates with a single server, and is done. Servers, on the other hand, start and then go to sleep, waiting for a client's request to arrive. They awaken when a client's datagram arrives, which usually requires the server to evaluate the request and possibly perform further processing. Our interest here is not in the programming aspects of clients and servers ([UNP3] covers all those details) but in the protocol features of UDP that affect the design and implementation of a server using UDP. (We examine the details of TCP server design in Chapter 13.) Although some of the features we describe depend slightly on the implementation of UDP being used, the features are common to most implementations.

10.11.1 IP Addresses and UDP Port Numbers

What arrives at a UDP server from a client is a UDP datagram. The IP header contains the source and destination IP addresses, and the UDP header contains the source and destination UDP port numbers. When an application receives a UDP message, the IP and UDP headers have been stripped off; the application must be told by the operating system in some other way who sent the message (the source IP address and port number), if it intends to furnish a reply. This feature allows a UDP server to handle multiple clients.

Some servers need to know *to whom* the datagram was sent, that is, the destination IP address. While it may seem obvious that such information would immediately be known by a server without looking into the received datagram, this is not always the case. For example, because of multihoming, IP address aliasing, and ordinary IPv6 usage with multiple scopes, a host may have multiple IP addresses, and a single server may receive incoming datagrams using any of them (this is in fact the common case). Any server wishing to perform its tasks differently depending on the destination IP address selected by the client would require access to the destination IP address information. In addition, some services may respond differently if the destination address is broadcast or multicast (e.g., the Host Requirements RFC [RFC1122] states that a TFTP server should ignore received datagrams that are sent to a broadcast address).

Note

A DNS server is one type of server that is sensitive to the destination IP address. It can use this information to arrange a particular sorting order on the address mappings it returns. This behavior of DNS is described in more detail in Chapter 11.

The lesson here is that even though an API may deliver all the data contained in a transport-layer datagram, additional information from the various layers (typically addressing information) may be required for a server to operate most effectively. This issue is not unique to UDP, of course, but because it is the first transport-layer protocol we study, it is worthwhile to point out now.

UDP servers designed for use with both IPv4 and IPv6 must consider the fact that these two types of addresses have significantly different lengths and require different data structures. In addition, the interoperability mechanism of encoding IPv4 addresses in IPv6 addresses may allow the use of IPv6 sockets to handle both IPv4 and IPv6 addressing. See [UNP3] for more details.

10.11.2 Restricting Local IP Addresses

Most UDP servers *wildcard* their local IP address when they bind a UDP endpoint. This means that an incoming UDP datagram destined for the server's port is accepted on any local IP address (any IP address in use on the local machine, including the local loopback address). For example, we can start an IPv4 UDP server on port 7777:

```
Linux% sock -u -s 7777
```

We can then use the `netstat` command to see the state of the endpoint (see Listing 10-9).

Listing 10-9 `netstat` listing of IPv4 UDP servers using wildcarded address bindings

```
Linux% netstat -l --udp -n
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address
udp        0      0 *:7777                  0.0.0.0:*
```

We have deleted several lines of output other than the one in which we are interested. The `-l` flag reports on all listening sockets (servers). The `--udp` flag provides data relating only to the UDP protocol. The `-n` flag prints IP addresses rather than fully expanded host names.

Note

While not all systems provide exactly these (Linux) flags for `netstat`, most provide the `netstat` command with some combination of flags to obtain similar results. On BSD, the `-l` and `-p udp` flags are supported. On Windows, the `-n`, `-a`, and `-p udp` flags can be used.

The local address is printed as `*:7777`, where the asterisk means that the local IP address has been wildcarded. When the server creates its endpoint, it can specify one of the host's local IP addresses, including a broadcast address, as the local IP address for the endpoint. In such cases, incoming UDP datagrams are then passed to this endpoint only if the destination IP address matches the specified local address. With our `sock` program, if we specify an IP address before the port number, that IP address becomes the local IP address for the endpoint. For example, the command

```
Linux% sock -u -s 127.0.0.1 7777
```

restricts the server to accepting only datagrams arriving on the local loopback interface (127.0.0.1), which can be generated only on the same host. The `netstat` output in Listing 10-10 shows this case.

Listing 10-10 `netstat` listing of UDP IPv4 server bound to only the local loopback interface

Active Internet connections (only servers)				
Proto	Recv-Q	Send-Q	Local Address	Foreign Address
udp	0	0	127.0.0.1:7777	0.0.0.0:*

If we try to send this server a datagram from a host on the same Ethernet, an ICMPv4 Port Unreachable message is returned, and the sending application receives an error. The server never sees the datagram.

```
Linux% sock -u -v 10.0.0.3 6666
connected on 10.0.0.5.50997 to 10.0.0.3.6666
123
error: Connection refused
```

10.11.3 Using Multiple Addresses

It is possible to start different servers on the same port number, each with a different local IP address. Normally, however, the system must be told by the application that it is OK to reuse the same port number in this way.

Note

With the sockets API, the `SO_REUSEADDR` socket option must be specified. This is done in our `sock` program by specifying the `-A` option.

Even if we have only one true network interface, we can establish additional IP addresses for it to use. Here, our host has a native IPv4 address of 10.0.0.30, but we will give it two additional addresses:

```
Linux# ip addr add 10.0.2.13 scope host dev eth0
Linux# ip addr add 10.0.2.14 scope host dev eth0
```

Now our host has four unicast IPv4 addresses: its native address, the two we have just added, plus its local loopback address. We can start three different instances of the UDP on the same port using our `sock` program on the same UDP port (8888):

```
Linux% sock -u -s -A 10.0.2.13 8888
Linux% sock -u -s -A 10.0.2.14 8888
Linux% sock -u -s -A 8888
```

The servers must be started with the `-A` flag, telling the system that it is OK to reuse the same addressing information. The `netstat` output in Listing 10-11 shows the addresses and port numbers on which the servers are listening.

Listing 10-11 Restricted and wildcarded UDP servers on the same UDP port

Active Internet connections (only servers)					
Proto	Recv-Q	Send-Q	Local Address	Foreign Address	
udp	0	0	10.0.2.13:8888	0.0.0.0:*	
udp	0	0	0.0.0.0:8888	0.0.0.0:*	
udp	0	0	10.0.2.14:8888	0.0.0.0:*	

In this scenario, the only IPv4 datagrams that will go to the server with the wildcarded local address are those destined for 10.0.0.30, the directed broadcast address (e.g., 10.255.255.255), the limited broadcast address (255.255.255.255), or the local loopback address (127.0.0.1), because the restricted servers cover all other possibilities.

There is a priority implied when an endpoint with a wildcard address exists. An endpoint with a specific IP address that matches the destination IP address is always chosen over a wildcard. The wildcard endpoint is used only when a specific match is not found.

10.11.4 Restricting Foreign IP Address

In all the `netstat` output that we showed earlier, the foreign IP address (i.e., the one not local to the host where the server is running) and foreign port number are shown as `0.0.0.0:*`, meaning that the endpoint will accept an incoming UDP datagram from any IPv4 address and any port number. However, there is an option to restrict the foreign address. This means that the endpoint receives UDP datagrams only from that specific IPv4 address and port number. Note that this restriction can be added once a server has heard from a client, in order to filter out additional traffic from other clients. Our `sock` program uses the `-f` option to specify the foreign IPv4 address and port number:

```
Linux% sock -u -s -f 10.0.0.14.4444 5555
```

This sets the foreign IPv4 address to `10.0.0.14` and the foreign port number to `4444`. The server's port is `5555`. If we run `netstat`, we see that the local address has also been set, even though we did not specify it explicitly (see Listing 10-12).

Listing 10-12 Restricting the foreign address causes assignment of a local address.

Linux% netstat --udp -n					
Active Internet connections (w/o servers)					
Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
udp	0	0	10.0.0.30:5555	10.0.0.14:4444	ESTABLISHED

This is a typical side effect of specifying the foreign IP address and foreign port: if the local address has not been chosen when the foreign address is specified, the local address is chosen automatically. Its value becomes the IP address of the interface chosen by IP routing to reach the specified foreign IP address. Indeed, in this example the primary IPv4 address for the Ethernet that is connected to the foreign address is 10.0.0.30. Note that as a consequence of the endpoints being determined and the foreign address restricted, the `State` column now indicates that the association is `ESTABLISHED`.

Table 10-2 summarizes the three types of address bindings that a UDP server can establish.

Table 10-2 Types of address bindings for a UDP server

Local Address	Foreign Address	Description
<i>local_IP.local_port</i>	<i>foreign_IP.foreign_port</i>	Restricted to one client
<i>local_IP.local_port</i>	<i>*.* (wildcard)</i>	Restricted to one local IP address and port (but for any client)
<i>*.local_port</i>	<i>*.* (wildcard)</i>	Restricted to local port only

In all cases, *local_port* is the server's port and *local_IP* must be one of the locally assigned IP addresses. The ordering of the three rows in the table is the order that the UDP module applies when trying to determine which local endpoint receives an incoming datagram. The most specific binding (the first row) is tried first, and the least specific (the last row with both IP addresses wildcarded) is tried last.

10.11.5 Using Multiple Servers per Port

Although it is not specified in the RFCs, by default most implementations allow only one application endpoint at a time to be associated with any one (local IP address, UDP port number) pair for a given *address family* (i.e., IPv4 or IPv6). When a UDP datagram arrives at a host destined for its IP address and an active port number, one copy is delivered to that single endpoint (e.g., a listening application). The IP address of the endpoint can be the wildcard, as shown earlier, but only a single application can receive datagrams for the address(es) specified. If we then try to start another server with the same wildcarded local address and the same port using the same address family, it does not work:

```
Linux% sock -u -s 12.46.129.3 8888 &
Linux% sock -u -s 12.46.129.3 8888
can't bind local address: Address already in use
```

In support of multicasting (see Chapter 9), multiple endpoints can be allowed to use the same (local IP address, UDP port number) pair, although the application normally must tell the API that this is OK (i.e., our `-A` flag to specify the `SO_REUSEADDR` socket option illustrated previously).

Note

4.4BSD requires the application to set a different socket option (`SO_REUSEPORT`) to allow multiple endpoints to share the same port. Furthermore, each endpoint must specify this option, including the first one to use the port.

When a UDP datagram arrives whose destination IP address is a broadcast or multicast address, and there are multiple endpoints at the destination IP address and port number, one copy of the incoming datagram is passed to each endpoint. (The endpoint's local IP address can be the wildcard, which matches any destination IP address.) But if a UDP datagram arrives whose destination IP address is a *unicast* address (i.e., an ordinary address), only a single copy of the datagram is delivered to *one* of the endpoints. Which endpoint gets the unicast datagram is implementation-dependent, but this policy helps to allow multithreaded and multiprocess servers to operate without being invoked multiple times on the same incoming request.

10.11.6 Spanning Address Families: IPv4 and IPv6

It is possible to write servers that span not only protocols (such as servers that respond to both TCP and UDP) but also across address families. That is, we may write a UDP server that responds to incoming requests for IPv4 as well as for IPv6. While this may seem entirely straightforward (IPv6 addresses are just additional IP addresses on the same host that happen to be 128 bits long), there is a subtlety related to the sharing of the port space. On some systems, the port space between IPv6 and IPv4 for UDP (and TCP) is *shared*. This means that if a service binds to a UDP port using IPv4, it is also allocated the same port in the IPv6 port space (and vice versa), preventing other services from using it (unless the `SO_REUSEADDR` socket option is used, as mentioned before). Furthermore, because IPv6 addresses can encode IPv4 addresses in an interoperable way (see Chapter 2), wildcard bindings in IPv6 may receive incoming IPv4 traffic.

Note

The situation is implementation-specific. In Linux, all port space is shared, and any wildcard IPv6 binding implies a corresponding IPv4 binding. In FreeBSD, the `IPV6_V6ONLY` socket option may be used to ensure that bindings are present only in the IPv6 space. Programmers should consult the socket interface for IPv6 for whichever operating environment they are supporting. C language bindings are described in [RFC3493].

10.11.7 Lack of Flow and Congestion Control

Most UDP servers are *iterative* servers. This means that a single server thread (or process) handles all the client requests on a single UDP port (e.g., the server's well-known port). Normally there is a limited-size input queue associated with each UDP port that an application is using. This means that requests arriving at about the same time from different clients are automatically queued by UDP. The received UDP datagrams are passed to the application (when it asks for the next one) in the order in which they were received (i.e., FCFS—first come, first served).

It is possible, however, for this queue to overflow, causing the UDP implementation to discard incoming datagrams. This can happen even if only one client is being served because UDP provides no *flow control* (that is, no way for the server to tell the client to slow down). Because UDP is a connectionless protocol with no reliability mechanism of its own, applications are not told when the UDP input queue overflows. The excess datagrams are just discarded by UDP.

Another concern arises from the fact that queues are also present in the IP routers between the sender and the receiver—in the middle of the network. When these queues become full, traffic may be discarded in a fashion similar to that of the UDP input queue. When this happens, the network is said to be *congested*. Congestion is undesirable because it affects all network users with traffic that traverses the point where congestion is occurring, as opposed to the UDP input case mentioned previously, where only a single application server was affected. UDP poses a special concern for congestion because it has no way of being informed that it should slow down its sending rate if the network is being driven into congestion. (It also has no mechanism for slowing down, even if it were told to do so.) Thus, it is said to lack *congestion control*. Congestion control is a complex subject and still an active area of research. We will return to considerations of congestion control when we discuss TCP (see Chapter 16).

10.12 Translating UDP/IPv4 and UDP/IPv6 Datagrams

In Chapter 7 we discussed a framework for translating IP datagrams from IPv4 to IPv6 and vice versa. Chapter 8 described how this framework applies to ICMP. When UDP passes through a translator, the translation takes place as described in Chapter 7, except there are issues specific to the UDP checksum. For UDP/IPv4 datagrams, the UDP header's *Checksum* field is allowed to be 0 (uncomputed), whereas in UDP/IPv6 this is not allowed. Consequently, complete datagrams arriving with a zero checksum being translated from IPv4 to IPv6 result in either a UDP/IPv6 datagram with a fully computed pseudo-header checksum being generated, or with the arriving packet being dropped. The translator is supposed to provide a configuration option to select which is desired, as the overhead of generating such checksums may be objectionable. Packets containing a nonzero checksum being translated in either direction require the checksum to be updated if a non-checksum-neutral address mapping is used (see Chapter 7).

Fragmented datagrams present another challenge. For stateless translators, a fragmented UDP/IPv4 datagram with a zero checksum cannot be translated, as the appropriate UDP/IPv6 checksum cannot be computed. Such datagrams are dropped. Stateful translators (i.e., NAT64) can reassemble a number of fragments and compute the required checksum. Fragmented UDP/IP datagrams with computed checksums are handled as ordinary fragments in either direction, as specified in Chapter 7. Large UDP/IPv4 datagrams that require fragmentation to fit within the IPv6 minimum MTU after translation are also handled as conventional IPv4 datagrams (i.e., they are fragmented as needed).

10.13 UDP in the Internet

If we attempt to characterize the amount of UDP traffic in the Internet, we find that useful, publicly available data is somewhat hard to come by, and that the breakdown of traffic load by protocol varies from site to site. That said, studies such as [FKMC03] find that UDP accounts for between 10% and 40% of Internet traffic observed, and that as peer-to-peer applications gain in popularity, the use of UDP is also on the rise [Z09], although TCP traffic still dominates in terms of packets and bytes.

In [SMC02], fragmentation of Internet traffic is found to be most common with UDP (68.3% of the fragmented traffic is UDP), although very little traffic overall is fragmented (about 0.3% of packets, 0.8% of bytes). The authors report that the most common type of traffic that is fragmented is UDP-based multimedia traffic (53%; Microsoft's Media Player is responsible for about half of this) and encapsulated/tunneled traffic such as that present in VPN tunnels (about 22%). Furthermore, about 10% of the fragmentation is *reverse-order* (we said this earlier in the examples where the last IP fragment was sent prior to the first), and the most commonly seen fragment size is 1500 bytes (79%), followed by 1484 bytes (18%) and 1492 bytes (1%).

Note

The 1500-byte MTU is related to the native usable payload size for Ethernet. The 1484 size was produced by Digital Equipment Corporation's GigaSwitch (now defunct), which represented significant portions of the topology measured at the time.

The causes of fragmentation appear to derive from two factors: careless encapsulation and lack of path MTU discovery and adaptation for applications that like to use large messages. The former case relates to multiple levels of encapsulation across many protocol layers that add additional headers, forcing IP packets that initially fit into 1500-byte MTUs (the most common size) to no longer fit (e.g., application traffic carried over VPN tunnels). The second factor arises for applications

that use larger packets (e.g., video applications) that end up being fragmented. A curious (and unfortunate) finding in the [SMC02] study is that numerous UDP packets with the IPv4 *DF* bit field turned on (presumably trying to perform PMTU discovery) are encapsulated in UDP packets that do not (thereby defeating the attempt and leaving the responsible application ignorant of the fact).

10.14 Attacks Involving UDP and IP Fragmentation

Most attacks involving UDP relate to exhaustion of some shared resource (buffers, link capacity, etc.) or exploitation of bugs in protocol implementations causing system crashes or other undesired behavior. Both fall into the broad category of DoS attacks: the successful attacker is able to cause services to be made unavailable to legitimate users. The most straightforward DoS attack with UDP is simply generating massive amounts of traffic as fast as possible. Because UDP does not regulate its sending traffic rate, this can negatively impact the performance of other applications sharing the same network path. This can happen even without malicious intent.

A more sophisticated form of DoS attack frequently associated with UDP is a *magnification* attack. This type of attack generally involves an attacker sending a small amount of traffic that induces other systems to generate much more. In the so-called *fraggle* attack, a malicious UDP sender forges the IP source address to be that of a victim and sets the destination address to a form of broadcast (e.g., the directed broadcast address). UDP packets are sent to a service that generates traffic in response to an incoming datagram. When the servers implementing these services respond, they direct their messages to the IP address contained in the *Source IP Address* field of the arriving UDP packet. In this case, the source address is that of the victim, and so the victim host is subject to being overloaded by the multiple UDP traffic responders. Variants of this magnification attack are numerous, including inducing a character-generating service to be coupled to the echo service, thereby causing traffic to be “ping-ponged” forever. This attack is closely related to the ICMP smurf attack (see Chapter 8).

Several attacks involving IP fragmentation have appeared. IP fragmentation processing is somewhat more complex than UDP processing, so it is not so surprising that bugs in its implementation have been found and exploited. One form of attack involves sending fragments that contain no data whatsoever. This attack exploited a bug in IPv4 reassembly code and caused some systems to crash. Another attack on the IPv4 reassembly layer is the *teardrop* attack, which involves carefully constructing a series of fragments with overlapping *Fragment Offset* fields that crash or otherwise badly affect some systems. A variant of this involves overlapping fragment offsets that overwrite the UDP header from an earlier fragment. Overlapping fragments are now prohibited with IPv6 [RFC5722]. Finally, the also-related *ping of death* attack (typically constructed with ICMPv4 Echo Request but also applicable to UDP) operates by creating an IPv4 datagram that on

11

Name Resolution and the Domain Name System (DNS)

11.1 Introduction

The protocols we have studied so far operate using IP addresses to identify the hosts that participate in a distributed application. These addresses (especially IPv6 addresses) are cumbersome for humans to use and remember, so the Internet supports the use of *host names* to identify hosts, both clients and servers. In order to be used by protocols such as TCP and IP, host names are converted into IP addresses using a process known as *name resolution*. There are different forms of name resolution in the Internet, but the most prevalent and important one uses a distributed database system known as the *Domain Name System* (DNS) [MD88]. DNS runs as an application on the Internet, using IPv4 or IPv6 (or both). For scalability, DNS names are hierarchical, as are the servers that support name resolution.

DNS is a distributed client/server networked database that is used by TCP/IP applications to map between host names and IP addresses (and vice versa), to provide electronic mail routing information, service naming, and other capabilities. We use the term distributed because no single site on the Internet knows all of the information. Each site (university department, campus, company, or department within a company, for example) maintains its own database of information and runs a server program that other systems across the Internet (clients) can query. The DNS provides the protocol that allows clients and servers to communicate with each other and also a protocol for allowing servers to exchange information.

From an application's point of view, access to the DNS is through an application library called a *resolver*. In general, an application must convert a host name to an IPv4 and/or IPv6 address before it can ask TCP to open a connection or send a unicast datagram using UDP. The TCP and IP protocol implementations know nothing about the DNS; they operate only with the addresses.

In this chapter we will take a look at how the names in DNS are set up, how resolvers and servers communicate using the Internet protocols (mainly UDP), and some of the other resolution mechanisms that are used in Internet environments. We do not cover all of the administrative details of running a name server or all of the options available with resolvers and servers. Such information is available from various other sources, including Albitz and Liu's *DNS and BIND* text [AL06] and in [RFC6168]. We discuss the details of DNS security (DNSSEC) in Chapter 18.

11.2 The DNS Name Space

The set of all names used with DNS constitutes the DNS *name space*. This space is partitioned hierarchically and is case insensitive, similar to computer file system folders (directories) and files. The current DNS name space is a tree of domains with an unnamed root at the top. The top echelons of the tree are the so-called top-level domains (TLDs), which include *generic TLDs* (gTLDs), *country-code TLDs* (ccTLDs), and *internationalized country-code TLDs* (IDN ccTLDs), plus a special *infrastructure TLD* called, for historical reasons, *ARPA* [RFC3172]. These form the top levels of a naming tree with the form shown in Figure 11-1.

There are five commonly used groups of TLDs, and one group of specialized domains being used for *internationalized domain names* (IDNs).¹ The history of IDNs, one piece of the “internationalization” or “i18n” of the Internet, is long and somewhat complicated. Across the world, there are multiple languages, and each uses one or more written scripts. While the Unicode standard [U11] aims to capture the entire set of characters, many characters look the same but have different Unicode values. Furthermore, characters written as text may flow from right to left, left to right, or (when combining certain texts with others) in both directions. Couple these (and other) somewhat technical concerns with concerns regarding equity and international law and politics, and a considerable hurdle results. The interested reader may wish to consult the IAB's review of IDNs [RFC4690], published in 2006, for more information. Current information is available from [IIDN].

The gTLDs are grouped into categories: *generic*, *generic-restricted*, and *sponsored*. The generic gTLDs (*generic* appears twice) are open for unrestricted use. The others (*generic-restricted* and *sponsored*) are limited to various sorts of uses or are constrained as to what entity may assign names from the domain. For example, *EDU* is used for educational institutions, *MIL* and *GOV* are used for military and government institutions of the United States, and *INT* is used for international organizations (such as *NATO*). Table 11-1 provides a summary of the 22 gTLDs from [GTLD] as of mid-2011. There is a “new gTLD” program in the works that may significantly expand the current set, possibly to several hundred or even thousand. This program and policies relating to TLD management in general are maintained by the Internet Corporation for Assigned Names and Numbers (ICANN) [ICANN].

1. Figure 11-1 also shows 11 test IDN domains, which are still available.

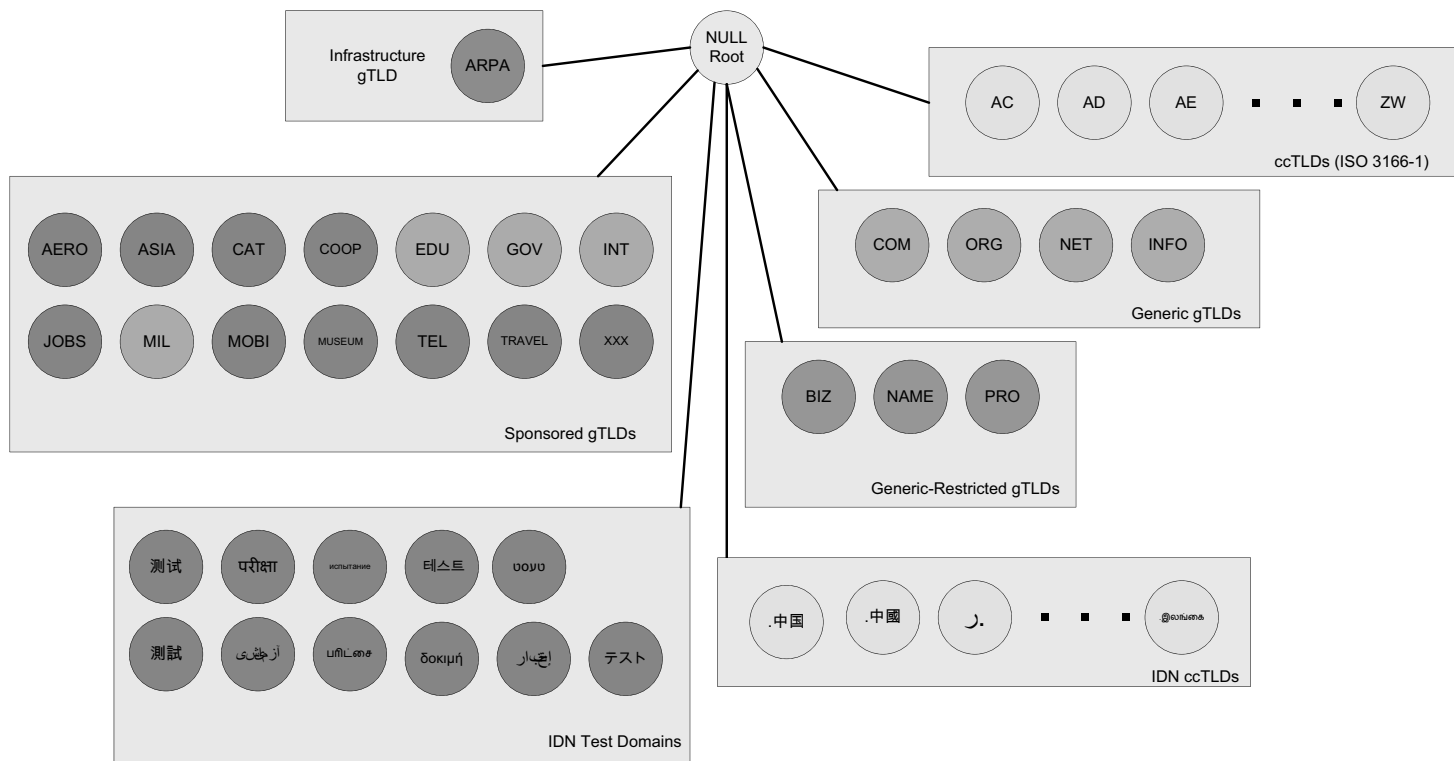


Figure 11-1 The DNS name space forms a hierarchy with an unnamed root at the top. The top-level domains (TLDs) include generic TLDs (gTLDs), country-code TLDs (ccTLDs), internationalized TLDs (IDN ccTLDs), and a special infrastructure TLD called *ARPA*.

Table 11-1 The generic top-level domains (gTLDs), circa 2011

TLD	First Use (est.)	Use	Example
AERO	December 21, 2001	Air-transport industry	www.sita.aero
ARPA	January 1, 1985	Infrastructure	18.in-addr.arpa
ASIA	May 2, 2007	Pan-Asia and Asia Pacific	www.seo.asia
BIZ	June 26, 2001	Business uses	neustar.biz
CAT	December 19, 2005	Catalan linguistic/cultural community	www.domini.cat
COM	January 1, 1985	Generic	icanhascheezburger.com
COOP	December 15, 2001	Cooperative associations	www.ems.coop
EDU	January 1, 1985	Post-secondary educational institutions recognized by U.S.A.	hpu.edu
GOV	January 1, 1985	U.S. government	whitehouse.gov
INFO	June 25, 2001	Generic	germany.info
INT	November 3, 1988	International treaty organizations	nato.int
JOBS	September 8, 2005	Human resource managers	intel.jobs
MIL	January 1, 1985	U.S. military	dtic.mil
MOBI	October 30, 2005	Customers/providers of mobile products/services	flowers.mobi
MUSEUM	October 30, 2001	Museums	icom.museum
NAME	August 16, 2001	Individuals	www.name
NET	January 1, 1985	Generic	ja.net
ORG	December 9, 2002	Generic	slashdot.org
PRO	May 6, 2002	Credentialed professionals/entities	nic.pro
TEL	March 1, 2007	Contact data for businesses/individuals	telnic.tel
TRAVEL	July 27, 2005	Travel industry	cancun.travel
XXX	April 15, 2011	Adult entertainment industry	whois.nic.xxx

The ccTLDs include the two-letter country codes specified by the ISO 3166 standard [ISO3166], plus five that are not: *uk*, *su*, *ac*, *eu*, and *tp* (the last one is being phased out). Because some of these two-letter codes are suggestive of other uses and meanings, various countries have been able to find commercial wind-falls from selling names within their ccTLDs. For example, the domain name *cnn.tv* is really a registration in the Pacific island of Tuvalu, which has been selling domain names associated with the television entertainment industry. Creating a name in such an unconventional way is sometimes called a *domain hack*.

11.2.1 DNS Naming Syntax

The names below a TLD in the DNS name tree are further partitioned into groups known as *subdomains*. This is very common practice, especially for the ccTLDs. For

example, most educational sites in England use the suffix `.ac.uk`, whereas names for most for-profit companies there end in the suffix `.co.uk`. In the United States, city government Web sites tend to use the subdomain `ci.city.state.us` where *state* is the two-letter abbreviation for the name of the state and *city* is the name of the city. For example, the site `www.ci.manhattan-beach.ca.us` is the site of Manhattan Beach, California's, city government in the United States.

The example names we have seen so far are known as fully qualified domain names (FQDNs). They are sometimes written more formally with a trailing period (e.g., `mit.edu.`). This trailing period indicates that the name is complete; no additional information should be added to the name when performing a name resolution. In contrast to the FQDN, an *unqualified domain name*, which is used in combination with a default domain or domain search list set during system configuration, has one or more strings appended to the end. When a system is configured (see Chapter 6), it is typically assigned a default domain extension and search list using DHCP (or, less commonly, the RDNSS and DNSSL RA options). For example, the default domain `cs.berkeley.edu` might be configured in systems at the computer science department at UC Berkeley. If a user on one of these machines types in the name `vangogh`, the local resolver software converts this name to the FQDN `vangogh.cs.berkeley.edu.` before invoking a resolver to determine `vangogh`'s IP address.

A domain name consists of a sequence of *labels* separated by periods. The name represents a location in the name hierarchy, where the period is the hierarchy delimiter and descending down the tree takes place from right to left in the name. For example, the FQDN

```
www.net.in.tum.de.
```

contains a host name label (`www`) in a four-level-deep domain (`net.in.tum.de`). Starting from the root, and working from right to left in the name, the TLD is `de` (the ccTLD for Germany), `tum` is shorthand for Technische Universität München, `in` is shorthand for *informatik* (German for “computer science”), and finally `net` is shorthand for the networks group within the computer science department. Labels are case-insensitive for matching purposes, so the name `ACME.COM` is equivalent to `acme.com` or `AcMe.cOm` [RFC4343]. Each label can be up to 63 characters long, and an entire FQDN is limited to at most 255 (1-byte) characters. For example, this domain name:

```
thelongestdomainnameintheworldandthensomeandthensomemoreandmore.com
```

was allegedly submitted as a potential world record for the longest name, with a label of length 63, but was judged to have been of insufficient merit to justify a place in the Guinness World Records.

The hierarchical structure of the DNS name space allows different administrative authorities to manage different parts of the name space. For example, creating

a new DNS name of the form `elevator.cs.berkeley.edu` would likely require dealing with the owner of the `cs.berkeley.edu` subdomain only. The `berkeley.edu` and `edu` portions of the name space would not require alteration, so the owners of those would not need to be bothered. This feature of DNS is one key aspect of its *scalability*. That is, no single entity is required to administer all the changes for the entire DNS name space. Indeed, creating a hierarchical structure for names was one of the first responses in the Internet community to the pressures of scaling and a major motivator for the structure used today. The original Internet naming scheme was flat (i.e., no hierarchy), and a single entity was responsible for assigning, maintaining, and distributing the list of nonconflicting names. Over time, as more names were required and more changes were being made, this approach became unworkable [MD88].

11.3 Name Servers and Zones

Management responsibility for portions of the DNS name space is assigned to individuals or organizations. A person given responsibility for managing part of the active DNS name space (one or more domains) is supposed to arrange for at least two *name servers* or *DNS servers* to hold information about the name space so that users of the Internet can perform queries on the names. The collection of servers forms the DNS (service) itself, a distributed system whose primary job is to provide name-to-address mappings. However, it can also provide a wide array of additional information.

The unit of administrative delegation, in the language of DNS servers, is called a *zone*. A zone is a subtree of the DNS name space that can be administered separately from other zones. Every domain name exists within some zone, even the TLDs that exist in the *root zone*. Whenever a new record is added to a zone, the DNS administrator for the zone allocates a name and additional information (usually an IP address) for the new entry and enters these into the name server's database. At a small campus, for example, one person could do this each time a new server is added to the network, but in a large enterprise the responsibility would have to be delegated (probably by departments or other organizational units), as one person likely could not keep up with the work.

A DNS server can contain information for more than one zone. At any hierarchical change point in a domain name (i.e., wherever a period appears), a different zone and containing server may be accessed to provide information for the name. This is called a *delegation*. A common delegation approach uses a zone for implementing a second-level domain name, such as `berkeley.edu`. In this domain, there may be individual hosts (e.g., `www.berkeley.edu`) or other domains (e.g., `cs.berkeley.edu`). Each zone has a designated owner or responsible party who is given authority to manage the names, addresses, and subordinate zones within the zone. Often this person manages not only the contents of the zone but also the name servers that contain the zone's database(s).

Zone information is supposed to exist in at least two places, implying that there should be at least two servers containing information for each zone. This is for redundancy; if one server is not functioning properly, at least one other server is available. All of these servers contain identical information about a zone. Typically, among the servers, a *primary* server contains the zone database in a disk file, and one or more *secondary* servers obtain copies of the database in its entirety from the primary using a process called a *zone transfer*. DNS has a special protocol for performing zone transfers, but copies of a zone's contents can also be obtained using other means (e.g., the `rsync` utility [RSYNC]).

11.4 Caching

Name servers contain information such as name-to-IP-address mappings that may be obtained from three sources. The name server obtains the information directly from the zone database, as the result of a zone transfer (e.g., for a slave server), or from another server in the course of processing a resolution. In the first case, the server is said to contain *authoritative* information about the zone and may be called an *authoritative server* for the zone. Such servers are identified by name within the zone information.

Most name servers (except some of the root and TLD servers) also *cache* zone information they learn, up to a time limit called the *time to live* (TTL). They use this cached information to answer queries. Doing so can greatly decrease the amount of DNS message traffic that would otherwise be carried on the Internet [J02]. When answering a query, a server indicates whether the information it is returning has been derived from its cache or from its authoritative copy of the zone. When cached information is returned, it is common for a server to also include the domain names of the name servers that can be contacted to retrieve authoritative information about the corresponding zone.

As we shall see, each DNS record (e.g., name-to-IP-address mapping) has its own TTL that controls how long it can be cached. These values are set and altered by the zone administrator when necessary. The TTL dictates how long a mapping can be cached anywhere within DNS, so if a zone changes, there still may exist cached data within the network, potentially leading to incorrect DNS resolution behavior until expiry of the TTL. For this reason, some zone administrators, anticipating a change to the zone contents, first reduce the TTL before implementing the change. Doing so reduces the window for incorrect cached data to be present in the network.

It is worth mentioning that caching is applied both for successful resolutions and for unsuccessful resolutions (called *negative caching*). If a request for a particular domain name fails to return a record, this fact is also cached. Doing so can help to reduce Internet traffic when errant applications repeatedly make requests for names that do not exist. Negative caching was changed from optional to mandatory by [RFC2308].

In some network configurations (e.g., those using older UNIX-compatible systems), the cache is maintained in a nearby name server, not in the resolvers resident in the clients. Placing the cache in the server allows any hosts on the LAN that use the nearby server to benefit from the server's cache but implies a small delay in accessing the cache over the local network. In Windows and more recent systems (e.g., Linux), the client can maintain a cache, and it is made available to all applications running on the same system. In Windows, this happens by default, and in Linux, it is a service that can be enabled or disabled.

On Windows, the local system's cache parameters may be modified by editing the following registry entry:

```
HKLM\SYSTEM\CurrentControlSet\Services\DNSSCache\Parameters
```

The DWORD value `MaxNegativeCacheTtl` gives the maximum number of seconds that a negative DNS result remains in the resolver cache. The DWORD value `MaxCacheTtl` gives the maximum number of seconds that a DNS record may remain in the resolver cache. If this value is less than the TTL of a received DNS record, the lesser value controls how long the record remains in cache. These two registry keys do not exist by default, so they must be created in order to be used.

In Linux and other systems that support it, the *Name Service Caching Daemon* (NSCD) provides a client-side caching capability. It is controlled by the `/etc/nscd.conf` file that can indicate which types of resolutions (for DNS and some other services) are cached, along with some cache parameters such as TTL settings. In addition, the file `/etc/nsswitch.conf` controls how name resolution for applications takes place. Among other things, it can control whether local files, the DNS protocol (see Section 11.5), and/or NSCD is employed for mappings.

11.5 The DNS Protocol

The DNS protocol consists of two main parts: a query/response protocol used for performing queries against the DNS for particular names, and another protocol for name servers to exchange database records (zone transfers). It also has a way to notify secondary servers that the zone database has evolved and a zone transfer is necessary (DNS Notify), and a way to dynamically update the zone (dynamic updates). By far, the most typical usage is a simple query/response to look up the IPv4 address that corresponds to a domain name.

Most often, DNS name resolution is the process of mapping a domain name to an IPv4 address, although IPv6 addresses mappings work in essentially the same way. DNS query/response operations are supported over the distributed DNS infrastructure consisting of servers deployed locally at each site or ISP, and a special set of *root servers*. There is also a special set of *generic top-level domain servers*

used for scaling some of the larger gTLDs, including COM and NET. As of mid-2011, there are 13 root servers named by the letters A through M (see [ROOTS] for more information about them); 9 of them have IPv6 addresses. There are also 13 gTLD servers, also labeled A through M; 2 of them have IPv6 addresses. By contacting a root server and possibly a gTLD server, the name server for any TLD in the Internet can be discovered. These servers are mutually coordinated to provide the same information. Some of them are not a single physical server but instead a group of servers (over 50 for the J root server) that use the same IP address (i.e., using IP anycast addressing; see Chapter 2).

A full resolution that is unable to benefit from preexisting cached entries takes place among several entities, as shown in Figure 11-2.

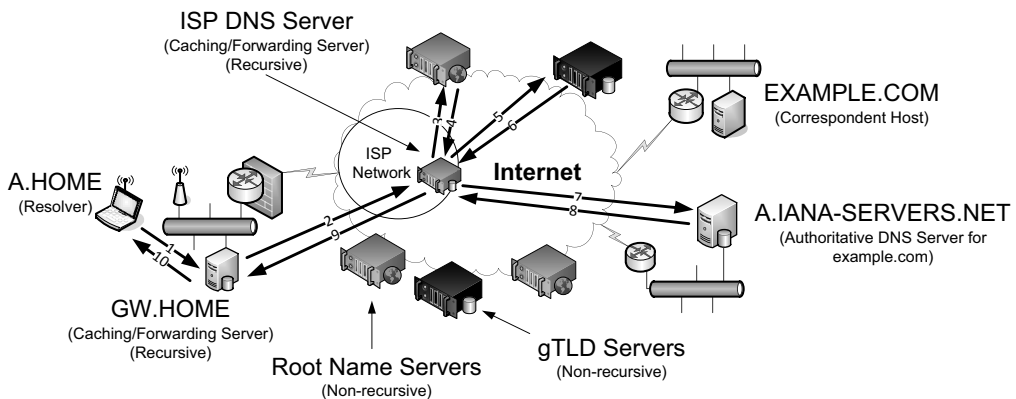


Figure 11-2 A typical recursive DNS query for `EXAMPLE.COM` from `A.HOME` involves up to ten messages. The local recursive server (`GW.HOME` here) uses a DNS server provided by its ISP. That server, in turn, uses an Internet root name server and a gTLD server (for COM and NET TLDs) to find the name server for the `EXAMPLE.COM` domain. That name server (`A.IANA-SERVERS.NET` here) provides the required IP address for the host `EXAMPLE.COM`. All of the recursive servers cache any information learned for later use.

Here, we have a laptop called `A.HOME` residing nearby the DNS server `GW.HOME`. The domain `HOME` is private, so it is not known to the Internet—only locally at the user's residence. When a user on `A.HOME` wishes to connect to the host `EXAMPLE.COM` (e.g., because a Web browser has been instructed to access the page `http://EXAMPLE.COM`), `A.HOME` must determine the IP address for the server `EXAMPLE.COM`. Assuming it does not know this address already (it might if it has accessed the host recently), the resolver software on `A.HOME` first makes a request to its local name server, `GW.HOME`. This is a request to convert the name `EXAMPLE.COM` into an address and constitutes message 1 (labeled on an arrow in Figure 11-2).

Note

If the A.HOME system is configured with a default domain search list, there may be additional queries. For example, if .HOME is a default search domain used by A.HOME, the first DNS query may be for the name EXAMPLE.COM.HOME, which will fail at the GW.HOME name server, which is authoritative for .HOME. A subsequent query will typically remove the default extension, resulting in a query for EXAMPLE.COM.

If GW.HOME does not already know the IP address for EXAMPLE.COM or the name servers for either the EXAMPLE.COM domain or the COM TLD, it forwards the request to another DNS server (called *recursion*). In this case, a request (message 2) goes to an ISP-provided DNS server. Assuming that this server also does not know the required address or other information, it contacts one of the root name servers (message 3). The root servers are not recursive, so they do not process the request further but instead return the information required to contact a name server for the COM TLD. For example, it might return the name A.GTLD-SERVERS.NET and one or more of its IP addresses (message 4). With this information, the ISP-provided server contacts the gTLD server (message 5) and discovers the name and IP addresses of the name servers for the domain EXAMPLE.COM (message 6). In this case, one of the servers is A.IANA-SERVERS.NET.

Given the correct server for the domain, the ISP-provided server contacts the appropriate server (message 7), which responds with the requested IP address (message 8). At this point, the ISP-provided server can respond to GW.HOME with the required information (message 9). GW.HOME is now able to complete the initial query and responds to the client with the desired IPv4 and/or IPv6 address(es) (message 10).

From the perspective of A.HOME, the local name server was able to perform the request. However, what really happened is a *recursive query*, where the GW.HOME and ISP-provided servers in turn made additional DNS requests to satisfy A.HOME's query. In general, most name servers perform recursive queries such as this. The notable exceptions are the root servers and other TLD servers that do not perform recursive queries. These servers are a relatively precious resource, so encumbering them with recursive queries for every machine that performs a DNS query would lead to poor global Internet performance.

11.5.1 DNS Message Format

There is one basic DNS message format [RFC6195]. It is used for all DNS operations (queries, responses, zone transfers, notifications, and dynamic updates), as illustrated in Figure 11-3.

The basic DNS message begins with a fixed 12-byte header followed by four variable-length *sections*: questions (or queries), answers, authority records, and additional records. All but the first section contain one or more *resource records*

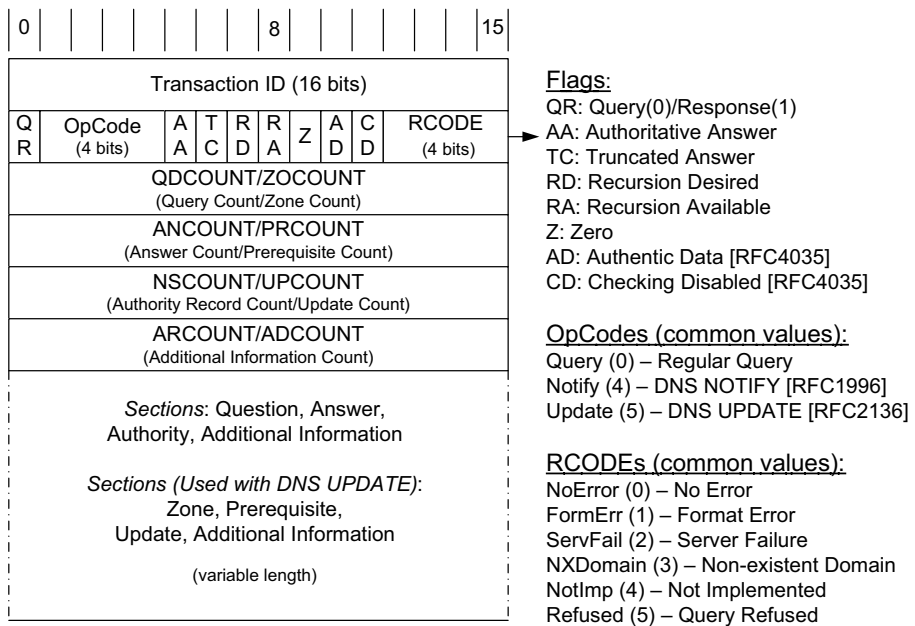


Figure 11-3 The DNS message format has a fixed 12-byte header. The entire message is usually carried in a UDP/IPv4 datagram and limited to 512 bytes. DNS UPDATE (DNS with dynamic updates) uses the field names *ZOCOUNT*, *PRCOUNT*, *UPCOUNT*, and *ADCOUNT*. A special extension format (called EDNS0) allows messages to be larger than 512 bytes, which is required for DNSSEC (see Chapter 18).

(RRs), which we discuss in detail in Section 11.5.6. (The question section contains a data item that is very close in structure to an RR.) RRs can be cached; questions are not.

In the fixed-length header, the *Transaction ID* field is set by the client and returned by the server. It lets the client match responses to requests. The second 16-bit word includes a number of flags and other subfields. Beginning from the left-most bit, *QR* is a 1-bit field: 0 means the message is a query; 1 means it is a response. The next is the *OpCode*, a 4-bit field. The normal value is 0 (a standard query) for requests and responses. Other values are: 4 (notify), and 5 (update). Other values (1–3) are deprecated or never seen in operational use. Next is the *AA* bit field that indicates an “authoritative answer” (as opposed to a cached answer). *TC* is a 1-bit field that means “truncated.” With UDP, this flag being set means that the total size of the reply exceeded 512 bytes, and only the first 512 bytes of the reply were returned.

RD is a bit field that means “recursion desired.” It can be set in a query and is then returned in the response. It tells the server to perform a recursive query. If the bit is not set, and the requested name server does not have an authoritative answer, the requested name server returns a list of other name servers to contact

for the answer. At this point, the overall query may be continued by contacting the list of other name servers. This is called an *iterative query*. *RA* is a bit field that means “recursion available.” This bit is set in the response if the server supports recursion. Root servers generally do *not* support recursion, thereby forcing clients to perform iterative queries to complete name resolution. The *Z* bit field must be 0 for now but is reserved for future use.

The *AD* bit field is set to true if the contained information is *authenticated*, and the *CD* bit is set to true if security checking is disabled (see Chapter 18). The *Response Code* (or *RCODE*) field is a 4-bit field with the return code whose possible values are given in [DNSPARAM]. The common values include 0 (no error) and 3 (name error or “nonexistent domain,” written as NXDOMAIN). A list of the first 11 error codes is given in Table 11-2 (values 11 through 15 are unassigned). Additional types are defined using a special extension (see Section 11.5.2). A name error is returned only from an authoritative name server and means that the domain name specified in the query does not exist.

Table 11-2 The first ten error types used with the *RCODE* field

Value	Name	Reference	Description and Purpose
0	NoError	[RFC1035]	No error
1	FormErr	[RFC1035]	Format error; query cannot be interpreted
2	ServFail	[RFC1035]	Server failure; error in processing at server
3	NXDomain	[RFC1035]	Nonexistent domain; unknown domain referenced
4	NotImp	[RFC1035]	Not implemented; request not supported in server
5	Refused	[RFC1035]	Refused; server unwilling to provide answer
6	YXDomain	[RFC2136]	Name exists but should not (used with updates)
7	YXRRSet	[RFC2136]	RRSet exists but should not (used with updates)
8	NXRRSet	[RFC2136]	RRSet does not exist but should (used with updates)
9	NotAuth	[RFC2136]	Server not authorized for zone (used with updates)
10	NotZone	[RFC2136]	Name not contained in zone (used with updates)

The next four fields are 16 bits in size and specify the number of entries in the question, answer, authority, and additional information sections that complete the DNS message. For a query, the number of questions is normally 1 and the other three counts are 0. For a reply, the number of answers is at least 1. Questions have a name, type, and class. (Class supports non-Internet records, but we ignore this for our purposes. The type identifies the type of object being looked up.) All of the other sections contain zero or more RRs. RRs contain a name, type, and class information, but also the TTL value that controls how long the data can be cached. We shall discuss the most important RR types in detail once we have a look at how DNS encodes names and selects which transport protocol to use when carrying DNS messages.

11.5.1.1 Names and Labels

The variable-length sections at the end of a DNS message contain a collection of questions, answers, authority information (names of name servers that contain authoritative information for certain data), and additional information that may be useful to reduce the number of necessary queries. Each question and each RR begins with a *name* (called the domain name or owning name) to which it refers. Each name consists of a sequence of *labels*. There are two categories of label types: *data labels* and *compression labels*. Data labels contain characters that constitute a label; compression labels act as pointers to other labels. Compression labels help to save space in a DNS message when multiple copies of the same string of characters are present across multiple labels.

11.5.1.2 Data Labels

Each data label begins with a 1-byte count that specifies the number of bytes that immediately follow. The name is terminated with a byte containing the value 0, which is a label with a length of 0 (the label of the root). For example, the encoding of the name `www.pearson.com` would be as shown in Figure 11-4.

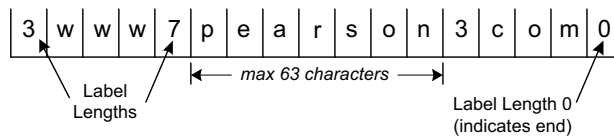


Figure 11-4 DNS names are encoded as a sequence of labels. This example encodes the name `www.pearson.com`, which (technically) has four labels. The end of the name is identified by a 0-length label of the nameless root.

For data labels, each label *Length* byte must be in the range of 0 to 63, as labels are limited to 63 bytes. No padding is used for labels, so the total name length could be odd. Although these labels are sometimes called “text” labels, they are capable of containing non-ASCII values. This use, however, is uncommon and not recommended. Indeed, even the internationalized domain names, which can encode Unicode characters [RFC5890][RFC5891], use a curious encoding syntax called “punycode” [RFC3492] that expresses Unicode characters using the ASCII character set. To be completely safe, it is recommended to follow the requirements in [RFC1035], which suggest that labels “start with a letter, end with a letter or digit, and have as interior characters only letters, digits and hyphen.”

11.5.1.3 Compression Labels

In many cases, a DNS response carries information in the answer, authority, and additional information sections relating to the same domain name. If data labels were used, the same characters would be repeated in the DNS message when referring to the same name. To avoid this redundancy and save space, a compression

scheme is used. Anywhere the label portion of a domain name can occur, the single preceding count byte (which is normally between 0 and 63) instead has its 2 high-order bits turned on, and the remaining bits are combined with the bits in the subsequent byte to form a 14-bit pointer (offset) in the DNS message. The offset gives the number of bytes from the beginning of the DNS message where a data label (called the *compression target*) is to be found that should be substituted for the compression label. Compression labels are thus able to point to a location up to 16,383 bytes from the beginning. Figure 11-5 illustrates how we might encode the domain names `usc.edu` and `uc la.edu` using compression labels.

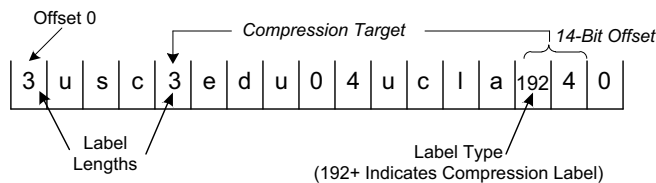


Figure 11-5 A compression label can reference other labels to save space. This is accomplished by setting the 2 high-order bits of the byte preceding the label contents. This signals that the following 14 bits are used in providing an offset for the replacement label. In this example, `usc.edu` and `uc la.edu` share the `edu` label.

In Figure 11-5 we see how the common label `edu` can be shared by the two domain names. Assuming the names start at offset 0, data labels are used to encode `usc.edu` as described previously. The next name is `uc la.edu`, and the label `uc la` is encoded using a data label. However, the label `edu` may be reused from the encoding of `usc.edu`. This is accomplished by setting the 2 high-order bits of the label *Type* byte to 1 and encoding the offset of `edu` in the remaining 14 bits. Because the first occurrence of `edu` is at offset 4, we only need to set the first byte to 192 (6 bits of 0) and the next byte to 4. The example in Figure 11-5 shows a savings of only 4 bytes, but it is clear how compression of larger common labels can result in more substantial savings.

11.5.2 The DNS Extension Format (EDNS0)

The basic DNS message format described so far can be restrictive in a number of ways. It has fixed-length fields, a total length limitation of 512 bytes when used with UDP (not including UDP or IP headers), and limited space (the 4-bit *RCODE* field) for indicating error types. An extension mechanism called *EDNS0* (because there could be future extensions beyond the index 0) is specified in [RFC2671]. While its use is not ubiquitous at present, it is necessary for supporting DNS security (DNSSEC; see Chapter 18), so it is likely to receive more widespread deployment over time.

EDNS0 specifies a particular type of RR (called an *OPT pseudo-RR* or *meta-RR*) that is added to the additional data section of a request or response to indicate the use of EDNS0. At most one such record may be present in any DNS message. We will discuss the particular format of an OPT pseudo-RR when we discuss the other RR types in Section 11.5.6. For now, the important thing to note is that if a UDP DNS message includes an OPT RR, it is permitted to exceed the 512-byte length limitation and may contain an expanded set of error codes.

EDNS0 also defines an extended label type (extending beyond the data labels and compression labels mentioned earlier). Extended labels have their first 2 bits in the label *Type/Length* byte set to 01, corresponding to values between 64 and 127 (inclusive). An experimental binary labeling scheme (type 65) was used at one time but is now not recommended. The value 127 is reserved for future use, and values above 127 are unallocated.

11.5.3 UDP or TCP

The well-known port number for DNS is 53, for both UDP and TCP. The most common format uses the UDP/IPv4 datagram structure shown in Figure 11-6.

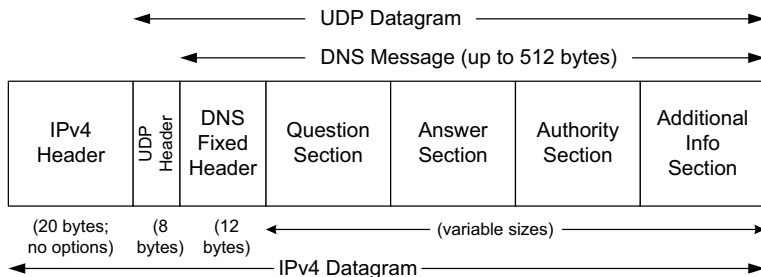


Figure 11-6 DNS messages are typically encapsulated in a UDP/IPv4 datagram and are limited to 512 bytes in size unless TCP and/or EDNS0 is used. Each section (except the question section) contains a set of resource records.

When a resolver issues a query and the response comes back with the *TC* bit field set (“truncated”), the size of the true response exceeded 512 bytes, so only the first 512 bytes are returned by the server. The resolver may issue the request again, using TCP, which now must be a supported configuration [RFC5966]. This allows more than 512 bytes to be returned because TCP breaks up large messages into multiple segments.

When a secondary name server for a zone starts up, it normally performs a zone transfer from the primary name server for the zone. Zone transfers can also be initiated by a timer or as a result of a DNS NOTIFY message (see Section 11.5.8.3). Full zone transfers use TCP as they can be large. Incremental zone transfers, where only the updated entries are transferred, may use UDP at first but switch to TCP if the response is too large, just like a conventional query.

When UDP is used, both the resolver and the server application software must perform their own timeout and retransmission. A recommendation for how to do this is given in [RFC1536]. It suggests starting with a timeout of at least 4s, and that subsequent timeouts result in an exponential increase of the timeout (a bit like TCP’s algorithms; see Chapter 14). Linux and UNIX-like systems allow a change to be made to the retransmission timeout parameters by altering the contents of the `/etc/resolv.conf` file (by setting the `timeout` and `attempts` options).

11.5.4 Question (Query) and Zone Section Format

The question or query section of a DNS message lists the question(s) being referenced. The format of each question in the question section is shown in Figure 11-7. There is normally just one, although the protocol can support more. The same structure is also used for the zone section in dynamic updates (see Section 11.5.7), but with different names.

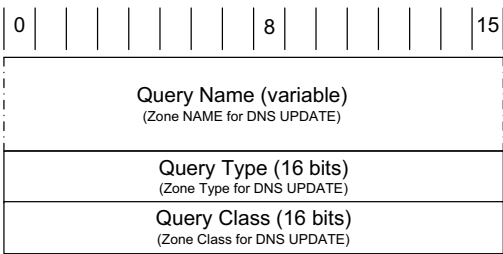


Figure 11-7 The query (or question) section of a DNS message does not contain a TTL because it is not cached.

The *Query Name* is the domain name being looked up, using the encoding for labels we described before. Each question has a *Query Type* and *Query Class*. The class value is 1, 254, or 255, indicating the Internet class, no class, or all classes, respectively, for all cases in which we are interested (other values are not typically used for TCP/IP networks). The *Query Type* field holds a value indicating the type of query being performed using the values from Table 11-2. The most common query type is A (or AAAA if IPv6 DNS resolution is enabled), which means that an IP address is desired for the query name. It is also possible to create a query of type ANY, which returns all RRs of any type in the same class that match the query name.

11.5.5 Answer, Authority, and Additional Information Section Formats

The final three sections in the DNS message, the answer, authority, and additional information sections, contain sets of RRs. RRs in these sections can, for the most part, have *wildcard* domain names as owning names. These are domain names in which the asterisk label—a data label containing only the asterisk character

[RFC4592]—appears first (i.e., leftmost). Each resource record has the form shown in Figure 11-8.

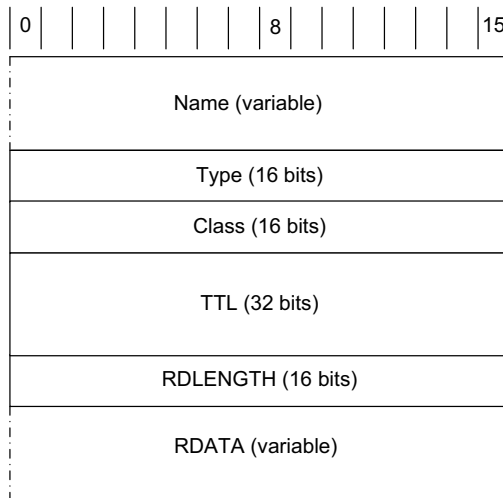


Figure 11-8 The format of a DNS resource record. For DNS in the Internet, the *Class* field always contains the value 1. The *TTL* field gives the maximum amount of time the RR can be cached (in seconds).

The *Name* field (sometimes called the “owning name,” “owner,” or “record owner’s name”) is the domain name to which the following resource data corresponds. It is in the same format we described earlier for names and labels. The *Type* field specifies one of the RR type codes (see Section 11.5.6). These are the same as the query type values we described earlier. The *Class* field is 1 for Internet data. The *TTL* field is the number of seconds for which the RR can be cached. The *Resource Data Length* (*RDLENGTH*) field specifies the number of bytes contained in the *Resource Data* (*RDATA*) field. The format of this data depends on the type. For example, A records (type 1) have a 32-bit IPv4 address in the *RDATA* area. We discuss other RR types later.

[RFC2181] defines the term *Resource Record Set* (RRSet) to be a set of resource records that share the same name, class, and type but not the same data. This occurs, for example, when a host has more than one address record for its name (e.g., because it has more than one IP address). TTLs for RRs in the same RRSet must be equal.

11.5.6 Resource Record Types

Although DNS is most commonly used to determine the IP address(es) that correspond to a particular name, it can also be used for the opposite purpose and for a number of other things. It can be used with both IPv4 and IPv6 and can even provide a distributed database function for other than Internet data (other classes,

in DNS terminology [RFC6195]). The wide range of capabilities provided by DNS is largely attributable to its ability to have different types of resource records.

There are many types of resource records (see [DNSPARAMS] for the complete list), and a single name may have multiple matching RRs. Table 11-3 provides a listing of the most common RR types used with conventional DNS (i.e., DNS without the DNSSEC security extensions).

Table 11-3 The popular resource record and query types used in DNS protocol messages. Additional records (not shown) are used when DNS security (DNSSEC) is employed.

Value	RR Type	Reference	Description and Purpose
1	A	[RFC1035]	Address record for IPv4 (32-bit IPv4 address)
2	NS	[RFC1035]	Name server; provides name of authoritative name server for zone
5	CNAME	[RFC1035]	Canonical name; maps one name to another (to provide a form of name aliasing)
6	SOA	[RFC1035]	Start of authority; provides authoritative information for the zone (name servers, e-mail address of contact, serial number, zone transfer timers)
12	PTR	[RFC1035]	Pointer; provides address to (canonical) name mapping; used with <code>in-addr.arpa</code> and <code>ip6.arpa</code> domains for IPv4 and IPv6 reverse queries
15	MX	[RFC1035]	Mail exchanger; provides name of e-mail handling host for a domain
16	TXT	[RFC1035] [RFC1464]	Text; provides a variety of information (e.g., used with SPF anti-spam scheme to identify authorized e-mail servers)
28	AAAA	[RFC3596]	Address record for IPv6 (128-bit IPv6 address)
33	SRV	[RFC2782]	Server selection; transport endpoints of a generic service
35	NAPTR	[RFC3403]	Name authority pointer; supports alternative name spaces
41	OPT	[RFC2671]	Pseudo-RR; supports larger datagrams, labels, return codes in EDNS0
251	IXFR	[RFC1995]	Incremental zone transfer
252	AXFR	[RFC1035] [RFC5936]	Full zone transfer; carried over TCP
255	(ANY)	[RFC1035]	Request for all (any) records

Resource records are used for many purposes but can be divided into three broad categories: data types, query types, and meta types. Data types are used to convey information stored in the DNS such as IP addresses and the names of authoritative name servers. Query types use the same values as data types, with a few additional values (e.g., AXFR, IXFR, and *). They can be used in the question section we described previously. Meta types designate transient data associated with a particular single DNS message. The OPT RR is the only meta type we

discuss in this chapter (all others are covered in Chapter 18). The most common data-type RRs include A, NS, SOA, MX, CNAME, PTR, TXT, AAAA, SRV, and NAPTR. The NS records are used to relate the DNS name space to the servers that perform resolution, and they contain the names of authoritative name servers for a zone. The A and AAAA records are used to provide an IPv4 or IPv6 address, respectively, given a particular name. The CNAME record provides a way to have an alias for another domain name. SRV and NAPTR records help applications to discover the location of servers supporting particular services, and to use alternative naming schemes (beyond DNS) to access such services. We shall explore each of these record types in the following sections.

11.5.6.1 Address (A, AAAA) and Name Server (NS) Records

Arguably, the most important records within DNS are the address (A, AAAA) and name server (NS) records. The A records contain 32-bit IPv4 addresses, and AAAA (called “quad-A”) records contain IPv6 addresses. An NS record contains the name of an authoritative DNS server that contains information for a particular zone. Because the name of a DNS server alone is not sufficient to perform a query, the IP address(es) of these servers is also typically provided as a so-called glue record in the additional information section of DNS responses. Indeed, such glue records are required to avoid loops whenever the names of the authoritative name servers use the same domain name for which they are authoritative. (Consider how `ns1.example.com` would be resolved if the name server for `example.com` was `ns1.example.com`.) We can see the structure of A, AAAA, and NS records using the `dig` tool provided on most Linux/UNIX-like systems. Here, we make a request for records of any type associated with the domain name `rfc-editor.org`:

```
Linux% dig +nostats -t ANY rfc-editor.org
```

```
; <<>> DiG 9.6.0-P1 <<>> +nostats -t ANY rfc-editor.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 53052
;; flags: qr rd ra; QUERY: 1, ANSWER: 12, AUTHORITY: 0, ADDITIONAL: 2

;; QUESTION SECTION:
;rfc-editor.org.      IN ANY

;; ANSWER SECTION:
...
rfc-editor.org.  1654 IN AAAA 2001:1890:1112:1::2f
rfc-editor.org.  1654 IN A   64.170.98.47
rfc-editor.org.  1654 IN NS  ns0.ietf.org.
rfc-editor.org.  1654 IN NS  ns1.hkg1.afiliast.net.
...
;; ADDITIONAL SECTION:
ns0.ietf.org.    756 IN   A       64.170.98.2
ns0.ietf.org.    756 IN   AAAA    2001:1890:1112:1::14
```

In the command's output, the first two lines indicate the version of the `dig` program being used and the options provided to it, plus implied options (`+cmd` means that this information itself should be printed). The next portion indicates data in the DNS reply message: the `QUERY` opcode, `NOERROR` status indicating no errors were encountered, and a transaction ID of 53052. In the *OpCode* field, `QUERY` is used for both queries and responses. Next, the `flags` line indicates that the message is a query response (`qr` flag) and not a query and that recursion was desired in the original query (`rd` flag) and is provided by the responding server (`ra` flag). The message contains a section with one query, and 12 resource records in the answer section (only 4 are shown). There are no RRs in the authority section, meaning that this response is likely from a caching server (the RRs are not authoritative). Different results might be obtained by interacting with different servers. The additional information section contains IPv4 and IPv6 addresses for one of the authoritative servers, should we wish to contact it. The question section contains a copy of our original query: type `ANY` for domain name `rfc-editor.org`.

Among the four RRs in the answer section shown, we find one `A` type, one `AAAA` type, and two `NS` types. From this information we can see that the domain name `rfc-editor.org` is a host with IPv4 address `64.170.98.47` and IPv6 address `2001:1890:1112:1::2f`. It is also a subdomain, as indicated by the presence of the `NS` records. We can quickly guess and verify that there is at least one host in this subdomain using the following command:

```
Linux% host ftp.rfc-editor.org
ftp.rfc-editor.org has address 64.170.98.47
```

This example indicates a few interesting aspects of `A`, `AAAA`, and `NS` records. First, it is possible for a single domain name to have records of each of these types (and more). This is fairly common for IPv6-capable servers that are the “well-known” servers for a particular organization. We can also see that each record has a TTL value, and they differ considerably, except for those in the same `RRSet`. The TTL for the records in the answer section is 1654s (about half an hour), and the TTL for records in the additional information section is 756s (about 12 minutes). Note that the TTL value of a cached record is never more than the TTL of the same record retrieved from the authoritative source. TTLs for cached records “decay” until the record is retrieved again from an authoritative server. As a result, retrieving a cached record multiple times from the same server usually shows a decreasing TTL value.

11.5.6.2 Example

Now that we have seen the DNS message format, transport protocol options, and RR types for basic queries and responses, let us see an example. We start with a simple case to see the communication between a resolver on a client, a local name server, and a remote name server managed by an ISP. This scenario demonstrates the importance of caching in DNS. The topology is shown in Figure 11-9.

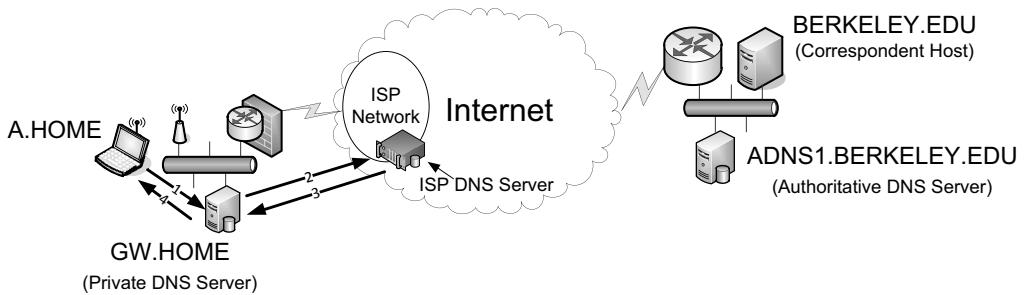


Figure 11-9 A simple DNS query/response example. The local DNS server (GW.HOME) provides recursion to the client (A.HOME), and uses the DNS server provided at the ISP when requested data is not present in the cache.

On our Windows client (A.HOME) we begin with a command that removes any DNS data cached by the resolver libraries. We then perform a query for the address (A record type) of the domain name `berkeley.edu`:

```
C:\> ipconfig /flushdns
Windows IP Configuration
```

Successfully flushed the DNS Resolver Cache.

```
C:\> nslookup
Default Server: gw
Address: 10.0.0.1
```

```
> set type=a
> berkeley.edu.
Server: gw
Address: 10.0.0.1
```

```
Non-authoritative answer:
Name: berkeley.edu
Address: 169.229.131.81
```

The first command is specific to Windows and removes data cached by the client's resolver software. The `nslookup` program, available on both Windows and Linux/UNIX-based systems, provides a basic way to query the DNS for specific data. Upon execution, it indicates which name server it is using for resolution (here the server is `gw` at the address `10.0.0.1`). Using the `set` command, we arrange to query for A records, and then query for the name `berkeley.edu`. Once again, `nslookup` indicates which server it uses for the resolution. It then also gives us an indication that the answer is nonauthoritative (i.e., it is being provided by a caching server) and the requested address is `169.229.131.81`.

To see what happens with the DNS protocol at the packet level, we use Wireshark and have a look at the first packet in detail, as shown in Figure 11-10.

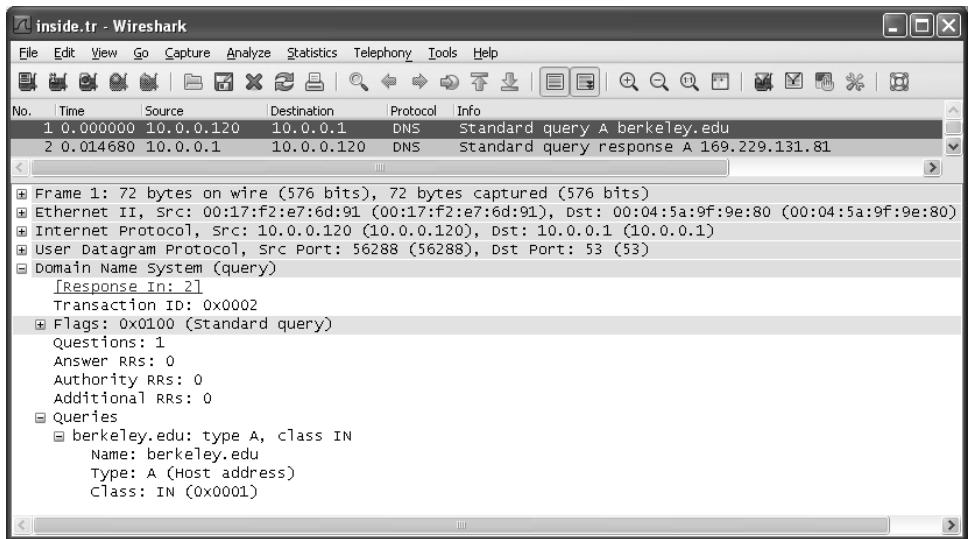


Figure 11-10 A UDP/IPv4 datagram containing a DNS standard query for the IPv4 address associated with `berkeley.edu`..

There are two messages in the trace: a standard query and a standard query response. In the first message (the query), the source IPv4 address is 10.0.0.120 (a DHCP-assigned address at the client; see Chapter 6), and the destination is 10.0.0.1 (the DNS server). The query is a UDP/IPv4 datagram with source port 56288 (an ephemeral port) and destination port 53 (the well-known DNS port). In terms of its full encapsulation, the request is an Ethernet frame containing 72 bytes. This size can be derived by summing the following parts: Ethernet header (14 bytes), IPv4 header (20 bytes), UDP header (8 bytes), DNS fixed header (12 bytes), query type (2 bytes), query class (2 bytes), plus the data labels for `berkeley` (9 bytes) and `edu` (4 bytes), plus the trailing 0 byte.

Turning to the details of the DNS header, the transaction ID is 0x0002 and forms the first 2 bytes of the DNS header, located at the start of the UDP payload. Only a single flag (recursion requested, the default) is set, so this message is a query. The message contains a standard query with one question. The other sections are empty. The question itself is for the name `berkeley.edu` and is seeking information of type A (address records) in the IN (Internet) class. After receiving this message, the name server process running on 10.0.0.1, unable to directly respond because it does not know the address, forwards the query to the next (upstream) name server it is configured to use. In this particular case, that name server is at the address 206.13.28.12 (see Figure 11-11).

In Figure 11-11 we see a query similar to the one sent by the client, but in this case the source IPv4 address is 70.231.136.162 (the ISP-side IPv4 address of `GW.HOME`). The destination address is 206.13.28.12, the IPv4 address of the ISP-provided DNS server, and the source port is an ephemeral port on the local DNS server (60961).

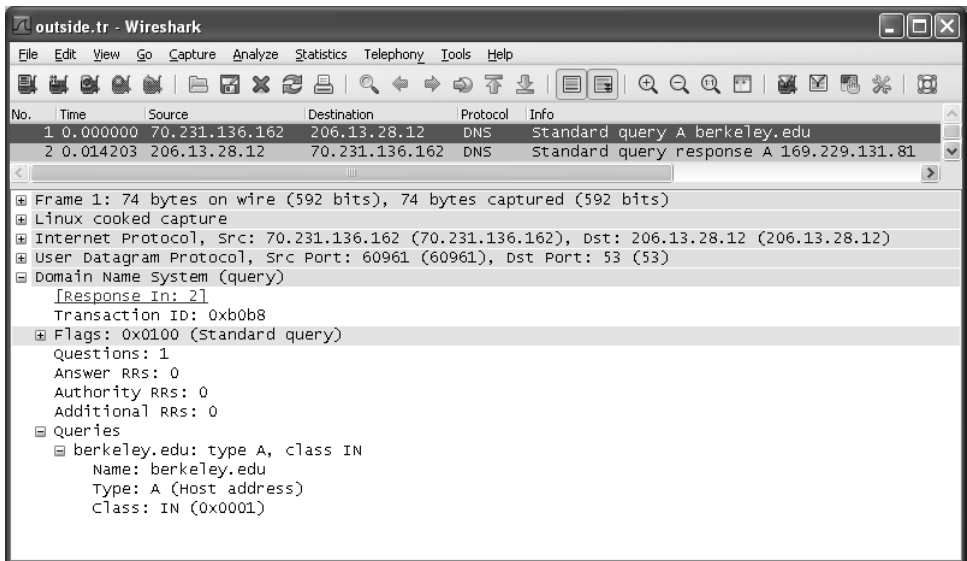


Figure 11-11 A DNS request generated at GW.HOME being sent to the ISP name server as a consequence of recursion.

The transaction ID is generated anew and set to 0xb0b8. Note that Wireshark indicates that the response to the query is contained in packet number 2.

Packet 2 in Figure 11-12 is the first DNS response we have seen. First, we note that the UDP source port number is 53, but the destination port is the ephemeral port number 60961. The transaction ID matches the query (0xb0b8), but the *Flags* field now contains the value 0x8180 (response, recursion requested, and recursion available are all set). The question section contains a copy of the question for which answers are being provided and typically matches the original query sent by the client exactly (e.g., case is preserved). There is one RR in the answer section. It is of type A (address), has a TTL of 10 minutes and a data length of 4 bytes (the size of an IPv4 address), and the value is 169.229.131.81, the IPv4 address we requested for `berkeley.edu`. Note that the authority flag is *not* set, and the authority section of the reply is empty. This response is based upon cached data; it is not authoritative for the domain. At this point, the local name server also caches the value (but only for up to 10 minutes as specified by the TTL in the RR it received) and responds to the requesting client (see Figure 11-13).

The response in Figure 11-13, packet 2, is much like the one from 206.13.28.12, except it is now sent from 10.0.0.1 to our original client at 10.0.0.120, and the transaction ID matches the one in the original DNS request. Note also that from the client's point of view the entire round-trip time of the transaction was about 14.7ms, but we know that most of that time (14.2ms) was taken up in the transaction between the local name server (GW.HOME) and the ISP's name server (206.13.28.12).

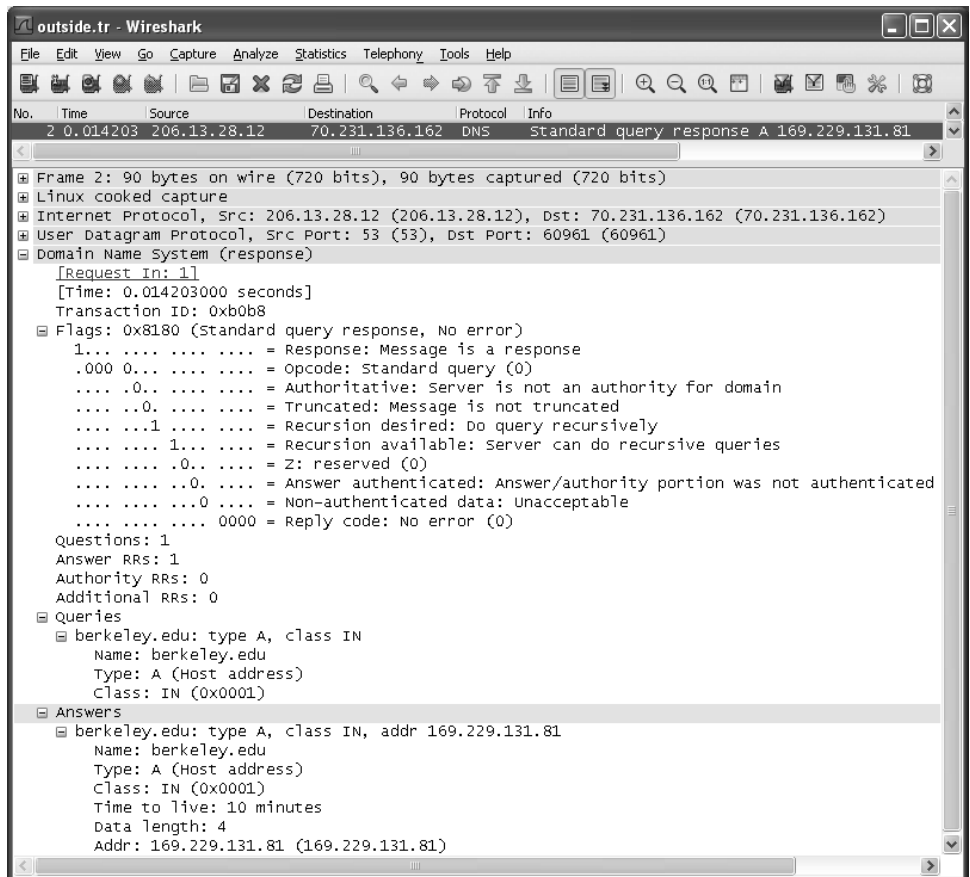


Figure 11-12 A standard DNS response sent from the ISP's DNS server back to GW.HOME.

11.5.6.3 Canonical Name (CNAME) Records

The CNAME record stands for *canonical name* record and is used to introduce an alias for a single domain name into the DNS naming system. For example, the name `www.berkeley.edu` may have a CNAME record that maps to some other machine (e.g., `www.w3.berkeley.edu`), so that if the Web server is located at a different computer, a relatively simple change to the DNS database may be all that is required for the rest of the world to find the new system. It is now common practice to use CNAME records to establish aliases for common services. As a result, names such as `www.berkeley.edu`, `ftp.sun.com`, `mail.berkeley.edu`, and `www.ucsd.edu` are all CNAME entries in the DNS that refer to other RRs.

Within a CNAME RR, the RDATA section contains the "canonical name" associated with the domain name (alias). Such names use the same type of encoding as other names (e.g., data labels and compression labels). When a CNAME RR is present for a particular name, no other data is permitted [RFC1912] (unless DNSSEC

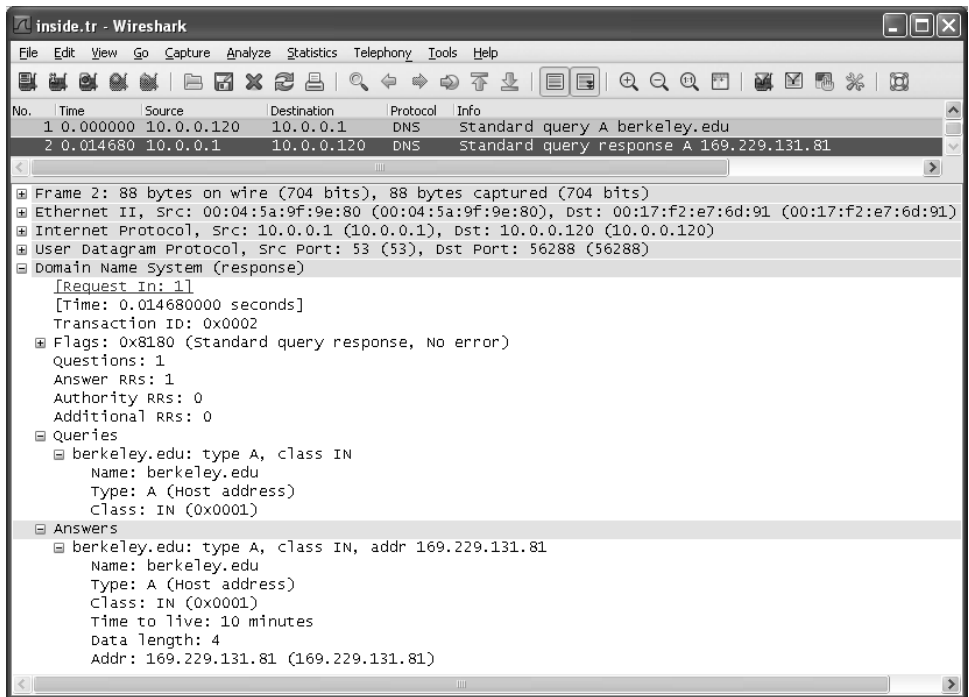


Figure 11-13 A response generated by GW.HOME and destined for the client. This message completes the recursive DNS transaction.

is in use; see Chapter 18). Domain names of CNAME RRs may not be used in all places that regular domain names can (e.g., as the target of an NS RR). Also, the canonical name may itself be a CNAME (called *CNAME chaining*), but this is usually discouraged, as it can cause DNS resolvers to make more queries than would otherwise be necessary. Nonetheless, there are certain services that make use of this feature. For example, the high-volume site www.whitehouse.gov (at the time of writing) uses a *content delivery network* (CDN)² provided by the Akamai Corporation. When we look up this domain name, we find the following:

```
Linux% host -t any www.whitehouse.gov
www.whitehouse.gov is an alias for www.whitehouse.gov.edgesuite.net.
Linux% host -t any www.whitehouse.gov.edgesuite.net
www.whitehouse.gov.edgesuite.net is an alias for a1128.h.akamai.net.
Linux% host -t any a1128.h.akamai.net
a1128.h.akamai.net has address 92.123.65.42
a1128.h.akamai.net has address 92.123.65.51
```

2. A content delivery network typically includes a number of synchronized content caches located in particular topological locations in the network. CDNs attempt to minimize latency for consumers accessing content in exchange for payment from content providers.

Thus, CNAME chains can be used with DNS. However, because of their potential performance impact, such chains are often limited by resolvers to a few “links” (such as five). Long chains are likely the result of an error in execution or a misunderstanding, as it is hard to imagine why they should be necessary under normal circumstances.

Note

There is a standard resource record called DNAME (type 39) [RFC2672][IDDN]. DNAME records act like CNAME records but for an entire zone. For example, all names of the form NAME.example.com could be mapped to NAME.newexample.com using a single DNAME resource record. However, DNAME records do not apply to the top-level record itself (example.com here).

11.5.6.4 Reverse DNS Queries: PTR (Pointer) Records

Although the most critical function of DNS is to provide mappings from names to IP addresses, there are many circumstances where the reverse mapping is required. For example, a server receiving an incoming TCP/IP connection request is able to ascertain the source IP address of the connection from the incoming IP datagram, but the name(s) corresponding to the address are not carried in the connection itself; such name(s) must be looked up in some other way. Fortunately, a clever use of the DNS can provide this capability.

The PTR RR type is used in response to reverse DNS queries, which are typically necessary when converting an IP address to a name. This uses the special `in-addr.arpa` (`ip6.arpa` for IPv6) domain, in a special way. Consider an IPv4 address such as 128.32.112.208. In the classful address structure (see Chapter 2), this address is taken from the 128.32 class B address space. To determine the name corresponding to this address, the address is first reversed, and then the special domain is added. In this example, a query for a PTR record using the name

`208.112.32.128.in-addr.arpa.`

would be used. In effect, this is a query for the “host” 208 in the “domain” `112.32.128.in-addr.arpa.`. We shall see more examples of reverse DNS queries later in this section.

Note

The regular DNS name space, which usually uses NS, A, and AAAA records, is not automatically linked with the “reverse” name space supported by PTR records. Thus it is possible (and even relatively common) to have an existing forward resolution that does not have a corresponding reverse mapping set up (or has a different one). Some services check to see that both directions are set up with equivalent mappings and may deny service under such circumstances.

Recall that IPv4 addresses are typically written in the “dotted-decimal” format and IPv6 addresses are written in the hex format (e.g., 169.229.131.81 and 2001:503:a83e::2:30, respectively). These addresses can be thought of as names existing in a left-to-right hierarchy. For example, the address 169.229.131.81 has the top-down hierarchy (reading left to right) 169, 229, 131, 81. By reversing the dotted-decimal IPv4 address and treating it as a DNS name, we can employ DNS to perform the mapping from IP address to name(s). So, the name 81.131.229.169 would effectively be the reversal of the IPv4 address 169.229.131.81. For IPv6, the scheme is similar, but any suppressed zeros are expanded, and each hexadecimal digit becomes a character. For example, the reversal of 2001:503:a83e::2:30 would be 0.3.0.0.2.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.e.3.8.a.3.0.5.0.1.0.0.2. Fortunately, users rarely have to type in these names directly.

As mentioned previously, the special domains `.in-addr.arpa` (for IPv4) and `.ip6.arpa` (for IPv6) are used in conjunction with the PTR (“pointer”) RR type in support of these types of names and reverse DNS lookups. For example, consider the following commands:

```
C:\> nslookup
Default Server: gw
Address: 10.0.0.1
> server c.in-addr-servers.arpa
Default Server: c.in-addr-servers.arpa
Address: 196.216.169.10
> set type=ptr
> 81.131.229.169.in-addr.arpa.
Server: c.in-addr-servers.arpa
Address: 196.216.169.10

169.in-addr.arpa nameserver = w.arin.net
169.in-addr.arpa nameserver = t.arin.net
169.in-addr.arpa nameserver = dill.arin.net
169.in-addr.arpa nameserver = x.arin.net
169.in-addr.arpa nameserver = z.arin.net
169.in-addr.arpa nameserver = y.arin.net
169.in-addr.arpa nameserver = u.arin.net
169.in-addr.arpa nameserver = v.arin.net
```

This example shows how the `.in-addr.arpa` domain is set up. According to [RFC5855], the `in-addr-servers.arpa` and `ip6-servers.arpa` domains are used in forming the domain names associated with the servers that provide reverse DNS mappings for IPv4 and IPv6, respectively. As of 2011, there are five such servers for each version of IP: `X.in-addr-servers.arpa` and `X.ip6-servers.arpa`, where `X` is any letter a through f (inclusive).

Although the ten servers we have mentioned contain authoritative data for reverse mappings, they do not contain the information we are looking for. In our example, the first server contacted instead told us to contact one of the eight name servers maintained by ARIN, the American Registry for Internet Numbers, which is authoritative for IPv4 addresses that start with 169. If we in turn contact one

of these servers, we find that a PTR query for `81.131.229.169.in-addr.arpa` gives the following response:

```
> server w.arin.net
Default Server: w.arin.net
Address: 72.52.71. 2
Default Server: w.arin.net
Address: 2001:470:1a::2
> 81.131.229.169.in-addr.arpa.
Server: w.arin.net
Address: 72.52.71.2

229.169.in-addr.arpa nameserver = adns1.berkeley.edu.
229.169.in-addr.arpa nameserver = phloem.uoregon.edu.
229.169.in-addr.arpa nameserver = aodns1.berkeley.edu.
229.169.in-addr.arpa nameserver = adns2.berkeley.edu.
```

Here we can surmise that the network prefix `169.229/16` is owned by an educational institution called Berkeley, that the campus maintains three name servers covering its `in-addr.arpa` space, and that the University of Oregon also provides a copy. Continuing by contacting one of these servers, we find our answer (this time using the Linux version of `nslookup` with slightly different output):

```
Linux% nslookup
> set type=ptr
> server adns1.berkeley.edu
Default Server: adns1.berkeley.edu
Address: 128.32.136.3#53
Default Server: adns1.berkeley.edu
Address: 2607:f140:ffff:fffe::3#53
> 81.131.229.169.in-addr.arpa.
Server: adns1.berkeley.edu
Address: 128.32.136.3#53

81.131.229.169.in-addr.arpa      name = webfarm.Berkeley.EDU
```

Here we obtain the result we were looking for, that the IPv4 address `169.229.131.81` has the name `webfarm.Berkeley.EDU`. The DNS server uses port 53, as indicated by the `#53` following the IP addresses. This output makes it obvious that accessing the DNS with UDP/IPv4 (as opposed to UDP/IPv6) can still provide mappings for IPv6 addresses using “quad-A” (AAAA) DNS records because we can see that the IPv6 address of the server is `2607:f140:ffff:fffe::3`.

If there were not a separate branch of the DNS tree for handling the address-to-name translation, there would be essentially no way to do the reverse translation other than starting at the root of the tree and trying every top-level domain. This is clearly an unreasonable option, given the current size of the Internet. The `in-addr.arpa` solution is effective and fairly efficient, although the reversed bytes of the IPv4/IPv6 address and the special domains can be confusing.

Fortunately, as mentioned before, users can typically avoid having to type or refer to them. Even application writers do not typically have to manipulate addresses to perform reverse queries, as library functions (such as the C library function `getnameinfo()`) perform this task.

It is worth mentioning here that PTR queries have become a significant concern for the global DNS servers. Consider a home network using one of the private address prefixes such as 10.0.0.0/8 (IPv4) or fc00::/7 (IPv6). When a system receives an incoming connection request from another system on the same privately addressed subnet, it may wish to resolve the source address to a name and does so by performing a PTR query. If the query is not answered by the local DNS server, it will likely propagate to the global Internet. For this reason (and a few others), [RFC6303] specifies that local name servers—especially those operating in networks using private IP addressing that are attached to the Internet—provide PTR mappings for the private address space defined in [RFC1918] for IPv4 and [RFC4193] for IPv6 (i.e., in IN-ADDR.ARPA and D.F.IP6.ARPA, respectively).

11.5.6.5 Classless in-addr.arpa Delegation

When organizations join the Internet and obtain authority to fill in a portion of the DNS name space, they often also obtain authority for a portion of the `in-addr.arpa` name space corresponding to their IPv4 addresses on the Internet. In the case of UC Berkeley, authority includes the network prefix 169.229/16, which, using older terminology, is “class B” network number 169.229. Thus, UC Berkeley would be expected to populate a portion of the DNS tree with PTR records using names ending in `229.169.in-addr.arpa`. This works fine for cases where the address prefix assigned to the organization is one of the older class A, B, or C styles where the number of bits is an integral multiple of 8. However, many organizations today are given prefix lengths of greater than 24 bits or greater than 16 bits (but less than 24). In these cases, the address range is not easily written as a simple reversal of the IP address. Instead, some method of conveying the network prefix length must be included as well.

The standard method for implementing this, given by [RFC2317], is to append the length of the prefix to the reversed octets and use it as the first label in the domain name. For example, assume that a site is assigned the prefix 12.17.136.128/25, a prefix that includes 128 addresses. According to [RFC2317], two types of records should be provided. First, for each name of the form `X.136.17.12.in-addr.arpa` (where `X` is at least 128 and not more than 255), a CNAME RR is created, likely maintained by a site's ISP, according to the following pattern:

[illegible]

Here we can see how the network prefix is encoded, with the / notation associated with the second label in the domain name (for this example). These entries are typically placed by an ISP and allow for delegations on non-byte-aligned address ranges. In this example, the customer is now able to provide mappings for the zone `128.128/25.136.17.12.in-addr.arpa`. We can trace the delegation as follows:

```
C:\> nslookup
Default Server:  gw
Address:  10.0.0.1
> server f.in-addr-servers.arpa
Default Server:  f.in-addr-servers.arpa
Addresses:  193.0.9.1
> set type=ptr
> 129.128/25.136.17.12.in-addr.arpa.
Server:  f.in-addr-servers.arpa
Address:  193.0.9.1
12.in-addr.arpa nameserver = dbru.br.ns.els-gms.att.net
12.in-addr.arpa nameserver = cbru.br.ns.els-gms.att.net
12.in-addr.arpa nameserver = cmtu.mt.ns.els-gms.att.net
12.in-addr.arpa nameserver = dmtu.mt.ns.els-gms.att.net
> server dbru.br.ns.els-gms.att.net.
Default Server:  dbru.br.ns.els-gms.att.net
Address:  199.191.128.106

> 129.128/25.136.17.12.in-addr.arpa.
128/25.136.17.12.in-addr.arpa  nameserver = ns2.intel-research.net
128/25.136.17.12.in-addr.arpa  nameserver= ns1.intel-research.net

> server ns1.intel-research.net.
Server:  ns1.intel-research.net
Address:  12.155.161.131
> 129.128/25.136.17.12.in-addr.arpa.

129.128/25.136.17.12.in-addr.arpa
                        name = dmz.slouter.seattle.intel-research.net
128/25.136.17.12.in-addr.arpa
                        nameserver = bldmzsvr.berkeley.intel-research.net
128/25.136.17.12.in-addr.arpa
                        nameserver = sldmzsvr.intel-research.net
bldmzsvr.berkeley.intel-research.net internet address = 12.155.161.131
sldmzsvr.intel-research.net internet address = 12.17.136.131
```

In this example, we wish to find out the name for the host associated with IPv4 address `12.17.136.129`. We have already seen that it has a CNAME RR pointing to the canonical name `129.128/25.136.17.12.in-addr.arpa.` We instruct our resolver to use one of the root servers (F) and arrange for the query type to be for a PTR RR. At this point we request a resolution for `129.128/25.136.17.12.in-addr.arpa.` The root name server does not have this information, and it does not perform recursion, so it returns the name of the authoritative servers for the domain

12.in-addr.arpa.. Picking one of them (DBRU), we again try to resolve our question. This time we find two name servers (ns1 and ns2). Picking one of these, we are able to resolve the PTR request. It resolves to the name `dmz.slouter.seattle.intel-research.net`.

11.5.6.6 Authority (SOA) Records

In DNS, each zone has an authority record, using an RR type called *start of authority* (SOA). These records provide authoritative links between portions of the DNS name space and the servers that provide the zone information allowing various queries to be performed for addresses and other information. The SOA RR is used to identify the name of the host providing the official permanent database, the responsible party's e-mail address (where "." is used instead of @), zone update parameters, and the default TTL. The default TTL is applied to RRs in the zone that are not otherwise assigned an explicit per-RR TTL.

The zone update parameters include a serial number, refresh time, retry time, and expire time. The serial number is increased (by at least 1), usually by the network administrator, anytime there is a change to the zone contents. It is used by secondary servers to determine if they should initiate a zone transfer (when they do not have a copy of the zone contents with largest serial number). The refresh time tells secondary servers how long to wait before checking the SOA record from the primary and its version number to determine if a zone transfer is required. The retry and expire times are used in the case of zone transfer failure. The retry value gives the time (in seconds) a secondary will wait before retrying. The expire time is an upper bound (in seconds) that a secondary server will keep retrying zone transfers before giving up. If it gives up, such a server ceases to respond to queries for the zone. In general, a zone can contain a mix of IPv4 and IPv6 data and can be accessed using either version of IP. In this example, we use IPv6 (using `nslookup` on an IPv6-only Windows host):

```
C:\> nslookup
Default Server: gw
Address: fe80::204:5aff:fe9f:9e80

> set type=soa
> berkeley.edu.
Server: gw
Address: fe80::204:5aff:fe9f:9e80

Non-authoritative answer:
berkeley.edu
    primary name server = ns-master1.berkeley.edu
    responsible mail addr = hostmaster.berkeley.edu
    serial = 2009050116
    refresh = 10800 (3 hours)
    retry = 1800 (30 mins)
    expire = 3600000 (41 days 16 hours)
    default TTL = 300 (5 mins)
```

```

> server adns1.berkeley.edu.
Default Server: adns1.berkeley.edu
Addresses: 2607:f140:ffff:fffe::3
           128.32.136.3

> berkeley.edu.
Server: adns1.berkeley.edu
Addresses: 2607:f140:ffff:fffe::3
           128.32.136.3

berkeley.edu
primary name server = ns-master1.berkeley.edu
responsible mail addr = hostmaster.berkeley.edu
serial = 2009050116
refresh = 10800 (3 hours)
retry = 1800 (30 mins)
expire = 3600000 (41 days 16 hours)
default TTL = 300 (5 mins)
berkeley.edu nameserver = ns.v6.berkeley.edu
berkeley.edu nameserver = aodns1.berkeley.edu
berkeley.edu nameserver = adns2.berkeley.edu
berkeley.edu nameserver = phloem.uoregon.edu
berkeley.edu nameserver = adns1.berkeley.edu
berkeley.edu nameserver = ucb-ns.NYU.edu
ns.v6.berkeley.edu internet address = 128.32.136.6
ns.v6.berkeley.edu AAAA IPv6 address = 2607:f140:ffff:fffe::6
adns1.berkeley.edu internet address = 128.32.136.3
adns1.berkeley.edu AAAA IPv6 address = 2607:f140:ffff:fffe::3
adns2.berkeley.edu internet address = 128.32.136.14
adns2.berkeley.edu AAAA IPv6 address = 2607:f140:ffff:fffe::e
aodns1.berkeley.edu internet address = 192.35.225.133
aodns1.berkeley.edu AAAA IPv6 address =
                        2607:f010:3f8:8000:214:4fff:fe45:e6a2
phloem.uoregon.edu internet address = 128.223.32.35
phloem.uoregon.edu AAAA IPv6 address = 2001:468:d01:20::80df:2023

```

Here we can see that not only did we receive the SOA record, but we also received a list of six authoritative name servers, and the IPv4/IPv6 addresses (glue records) for five of them (the address for the NYU server is not given, as glue records for NYU.edu would be in a different zone supported by a different server). As this is one of the more interesting responses we have seen, let us look at the packet contents corresponding to the request sent to the authoritative name server, `adns1.berkeley.edu` (see Figure 11-14).

This trace contains two packets, and we have chosen to display the reply, which is the more interesting of the two. A query for an SOA RR was sent to the host `2607:f140:ffff:fffe::3` (`adns1.Berkeley.EDU`) from the local system's globally scoped IPv6 datagram address `2001:5c0:1101:ed00:5571:5f81:e0a6:4978`. The response is carried in an IPv6 datagram with 491 bytes total length (the *Payload Length* field is 451). This particular packet contains the IPv6 header (40 bytes), UDP header (8 bytes), plus the DNS message (443 bytes). The DNS message includes one question, one answer, six authority RRs, and ten additional RRs.

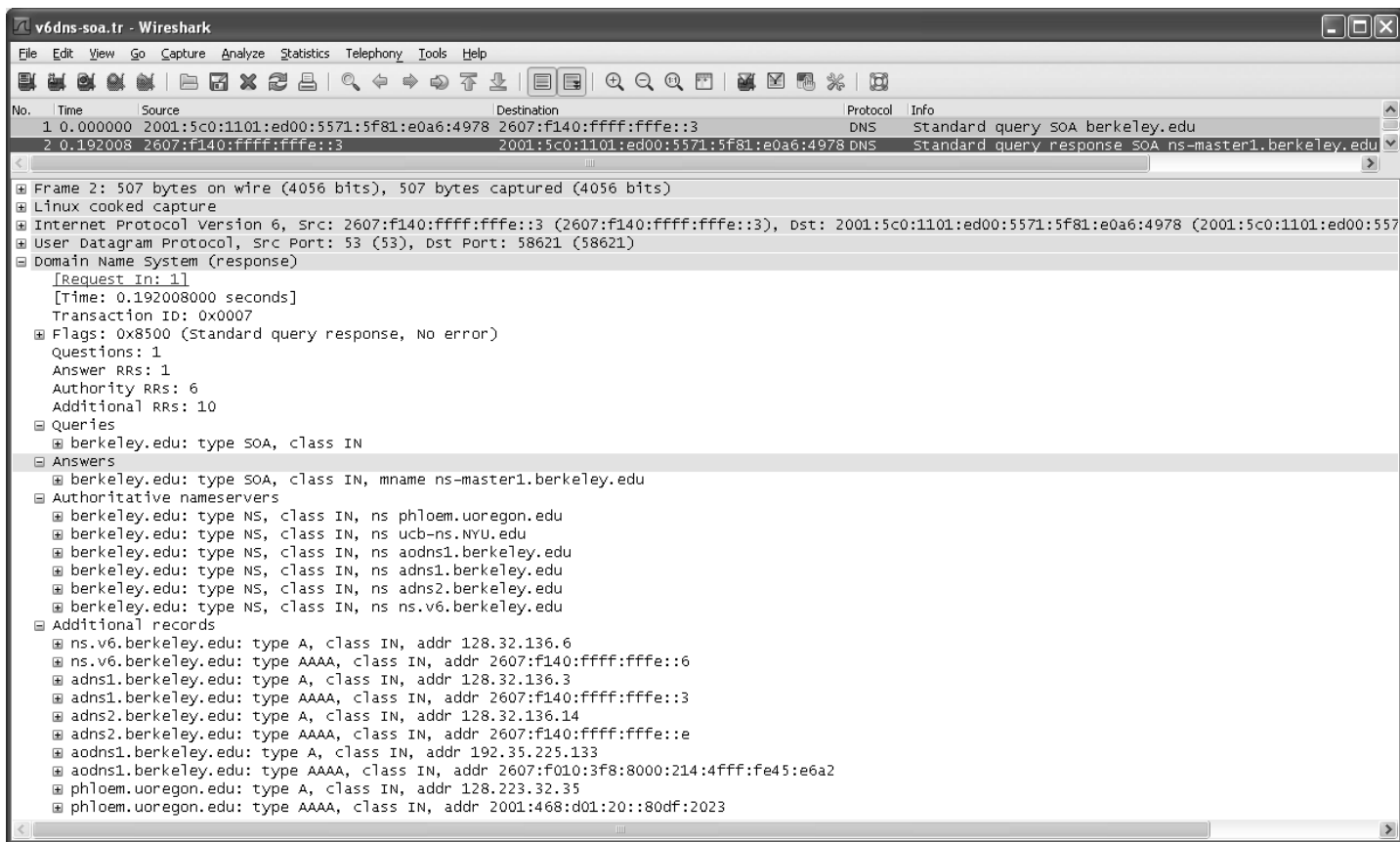


Figure 11-14 Response to a DNS query for an SOA record using IPv6. The response includes IPv4 and IPv6 addresses for the zone.

The question section contains the labels `berkeley` and `edu` and is 18 bytes long. The answer section contains the relevant information for the `berkeley.edu` domain described earlier and is able to take advantage of compression labels thanks to the contents of the question section. The total length for this section is 58 bytes. The authority section contains six NS records identifying name servers. This information takes another 135 bytes. The additional information section includes five A records and five AAAA records for a total of 220 bytes. The size of the *RDATA* field for each AAAA record is 16 bytes, so although the IPv6 address can be written in textual form with the `::` convention to save space, it is not encoded this way in the packet. Instead, the full 128 bits of the address are used.

11.5.6.7 Mail Exchanger (MX) Records

An MX record provides the name of a *mail exchanger*—a host willing to engage in the *Simple Mail Transfer Protocol* (SMTP) [RFC5321] to receive incoming e-mail on behalf of users associated with a domain name. When the Internet was still developing, some sites did not have permanent connections but instead would dial up and connect to hosts that did have permanent Internet connections. In such scenarios, the e-mail destination might be disconnected from the network when e-mail was in transit, so another host would hold on to the mail until the destination was attached. This was one motivation for the inclusion of MX records in the DNS—to allow sending hosts to deliver e-mail to an intermediary (“relay server”) even if the true destination was not available. Today, MX records are still used, and mail agents prefer to deliver e-mail to the host(s) listed in an MX record associated with a particular domain name.

MX records include a *preference* value, so that more than one MX record may be present for a particular domain name. The preference value allows a sending agent to sort the hosts in preference order (smaller is more preferable) in deciding which host to use as an e-mail destination. For example, we can use the `host` command again to query the DNS for MX records associated with the domain name `cs.ucla.edu`:

```
Linux% host -t MX cs.ucla.edu ns3.dns.ucla.edu
Using domain server:
Name: ns3.dns.ucla.edu
Address: 2607:f600:8001:1::ff:fe01:35#53
Aliases:

cs.ucla.edu mail is handled by 13 Pelican.cs.ucla.edu.
cs.ucla.edu mail is handled by 3 Moa.cs.ucla.edu.
cs.ucla.edu mail is handled by 13 Mailman.cs.ucla.edu.
```

Here we can see that an e-mail addressed to `person@cs.ucla.edu` is handled by one of three mail servers configured in the DNS. All of these mail servers are part of the `cs.ucla.edu` domain, but in general mail servers do not have to be named with the same domain as the e-mail they are handling. These three servers can be grouped into two parts: one with preference 3 and one set with preference

13. The server with the smaller preference number is preferred, so the sender first tries `MoA.cs.ucla.edu`. If that fails, it tries either `Pelican` or `Mailman`, selected at random.

It is possible that none of the MX record target hosts is reachable. This is an error condition. It is also possible that there are no MX records present, but there are CNAME, A, or AAAA records for the domain name. If there is a CNAME record, the target of the CNAME is used in place of the original domain name. If there are A or AAAA records, the mail agent may connect to these addresses. Each is considered to have a preference of zero (called *implicit MX*). MX record targets must be domain names that resolve to A or AAAA records; they cannot point to CNAMEs [RFC5321].

11.5.6.8 *Fighting Spam: The Sender Policy Framework (SPF) and Text (TXT) Records*

For outgoing e-mail, MX records allow the DNS to help determine the names of mail relays and servers for a domain. More recently, the DNS has been leveraged by receiving mail agents to determine which relaying or sending mail servers are authorized to send mail from a particular domain name. This is used to help combat spam (unwanted e-mail) that is sent by a rogue mail agent pretending to be an authorized mail sender.

E-mail received by a mail server is rejected, stored, or forwarded to another mail server. Rejection can happen for a number of reasons, such as a protocol error or lack of available storage space at the receiver. It can also be rejected because the sending mail client does not appear to be the proper one for sending e-mail. This capability is supported by the *Sender Policy Framework* (SPF) and documented in [RFC4408], an experimental RFC. There is another framework known as *Sender ID* [RFC4406] that incorporates SPF's functions. It is also experimental but less widely deployed.

Version 1 of SPF uses DNS TXT or SPF (type 99) resource records. Records are set up and published in the DNS by a domain's owner to indicate which servers are authorized to send mail originating from the domain. Although the SPF record type is a more "proper" place to carry SPF-related information in some sense, some DNS client implementations do not process SPF records properly, so to avoid this complication TXT records are used. TXT records hold simple strings associated with a domain name. Historically they have held strings useful for human consumption, to aid in debugging or identifying the owner or location of a domain. Today, they are usually processed by programs such as the SPF application.

SPF supports a rich syntax to express criteria used to match against details about an incoming mail message and the connection in which it is carried. For example, UC Berkeley uses the following SPF entry (some lines have been wrapped for clarity):

```
Linux% host -t txt berkeley.edu
berkeley.edu descriptive text
      "v=spf1 ip4:169.229.218.128/25 ip6:2607:F140:0:1000::/64
      include:outboundmail.convio.net ~all"
```

In this example, the information being provided is for SPF version 1 (indicated by the `v=spf1` string in the version section) and uses a TXT RR. When a receiving mail agent receives e-mail purportedly coming from the domain `berkeley.edu`, it performs a DNS query for records of type TXT against the `berkeley.edu` domain. The value of the text record contains the matching criteria (called *mechanisms*) and other information (called *modifiers*). Preceding each mechanism is a *qualifier* that determines the consequence of a matching mechanism. Processing of SPF records takes place using a function called `check_host()`. The function evaluates various mechanisms and completes when the first matching mechanism is encountered. Ultimately, `check_host()` provides a return value that is one of the following: None, Neutral, Pass, Fail, SoftFail, TempError, PermError. The None and Neutral return values indicate that no information was available or that information was available but that no result is asserted. These are handled identically. Pass indicates a match, as described in the next paragraph. Fail indicates that the sending host is not authorized to send mail from the domain. SoftFail is somewhat ambiguous but is to be treated “somewhere between a ‘Fail’ and a ‘Neutral,’” according to [RFC4408]. The TempError return indicates some transient failure (e.g., communication failure) that is likely to abate. The PermError return indicates that there was a problem in the SPF configuration, usually due to a malformed TXT or SPF record for the domain.

Reading from left to right in the example, the string `v=spf1` is a modifier indicating that the SPF version is 1. The `ip4` mechanism specifies that the SMTP sender has an IPv4 address from the prefix `169.229.218.128/25`. The `ip6` mechanism specifies any sending host with IPv6 address prefix `2607:F140:0:1000::/64`. Finally, the `include` mechanism incorporates, by reference, the TXT records with `outboundmail.convio.net`:

```
Linux% host -t txt outboundmail.convio.net
outboundmail.convio.net descriptive text
      "v=spf1 +ip4:66.45.103.0/25 +ip4:69.48.252.128/25
      +ip4:209.163.168.192/26 ~all"
outboundmail.convio.net descriptive text
      "spf2.0/pr
      +ip4:66.45.103.0/25 +ip4:69.48.252.128/25
      +ip4:209.163.168.192/26 ~all"
```

Note that these TXT records are used for both SPF and for Sender ID (which uses the value of `spf2.0/pr` in the version section). The first record is used by SPF. The `+` qualifier indicates that a match results in a Pass indication. Any mechanism missing a qualifier is assumed to have the `+` qualifier. Other possible qualifiers include `-` (Fail), `~` (Soft Fail), and `?` (Neutral). If none of the matching mechanisms produces a Pass result, the final mechanism (`all`) matches any condition. The tilde character (`~`) before the `all` criterion indicates that a SoftFail return should be generated if `all` is the only matching mechanism. The exact way a soft failure is handled is dependent on the receiving e-mail software. Note that

even with SPF support, validation is provided only on the sending domain and system, and not on the sending user. In Chapter 18 we will look at DKIM, which provides SPF-like capabilities but uses cryptography for authentication.

11.5.6.9 Option (OPT) Pseudo-Records

In conjunction with EDNS0, described previously, a special *OPT pseudo-RR* has been defined [RFC2671]. It is “pseudo” in the sense that it pertains only to the contents of a single DNS message and is not conventional DNS RR data. Consequently, OPT RRs are not cached, forwarded, or persistently stored, and they may appear only once (or not at all) in a DNS message. If one is present in a DNS message, it is found in the additional information section.

An OPT RR contains a 10-byte fixed portion followed by a variable portion. The fixed portion includes 16 bits indicating the RR type (41), 16 bits indicating the UDP payload size, 32 bits constituting an extended *RCODE* field and flags area, and 16 bits giving the size of the variable portion in bytes. These fields are located in the same relative positions as the *Name*, *Type*, *Class*, *TTL*, and *RDLLEN* fields, respectively, in a conventional RR (see Figure 11-8). OPT RRs use a null domain name in the *Name* field (0 bytes). The extended *RCODE* and *Flags* area (32 bits, corresponding to the *TTL* field in Figure 11-8) is subdivided into an 8-bit area to hold an extra 8 high-order bits augmenting the *RCODE* field in Figure 11-3, and an 8-bit *Version* field (currently set to 0 to indicate EDNS0). The remaining 16 bits are not yet defined and must be 0. The additional 8 bits provide an extended set of possible DNS error types, and these values are given in Table 11-4. (Note that value 16 is defined by two distinct RFCs.)

Table 11-4 Extended *RCODE* values. Most are used to support security extensions.

Value	Name	Reference	Description and Purpose
16	BADVERS	[RFC2671]	Bad EDNS version
16	BADSIG	[RFC2845]	Bad TSIG signature (see Chapter 18)
17	BADKEY	[RFC2845]	Bad TSIG key (see Chapter 18)
18	BADTIME	[RFC2845]	Bad TSIG signature (time problem; see Chapter 18)
19	BADMODE	[RFC2930]	Bad TKEY mode (see Chapter 18)
20	BADNAME	[RFC2930]	Duplicate key name (see Chapter 18)
21	BADALG	[RFC2930]	Algorithm not supported (see Chapter 18)

As we have mentioned, OPT RRs contain a variable-length *RDATA* field. This field is used to hold an extensible list of attribute-value pairs. The current set of attributes, meanings, and defining RFCs is maintained by the IANA [DNSPAR-AMS]. One such option, called NSID (EDNS option code 3) [RFC5001], indicates a special identifying value for a responding DNS server. The format of this value is not defined by standard but is instead configurable by the system administrator

of the DNS server. This capability may be useful in circumstances where an anycast address is used to identify a group of servers. The NSID is able to identify a specific responding server using a value other than the sending IP address. We shall see more examples of OPT RRs and EDNS0 usage when we look at DNSSEC in Chapter 18.

11.5.6.10 Service (SRV) Records

[RFC2782] defines the *service* (SRV) resource record. SRV RRs generalize the MX record format to describe the host, protocols, and port numbers used to contact a particular service. An SRV RR is ordinarily structured as follows:

```
_Service._Proto.Name TTL IN SRV Prio Weight Port Target
```

The *Service* identifier is the official name of a service. The *Proto* identifier is the transport protocol used to access the service, usually TCP or UDP. The *TTL* value is a conventional RR *TTL*, and IN and SRV indicate the Internet class and SRV RR type, respectively. The *Prio* value is a 16-bit unsigned value and works like the priority value in MX records (lower numbers represent higher priorities). The *Weight* value is used to choose an RR among several whose priority values are equal. The idea is that the weight is to be used as a weighted probability to select the particular entry for load balancing, so larger weights indicate a greater probability of selection. The *Port* is the TCP or UDP (or other transport protocol's) port number. The *Target* is the domain name of the target host where the service is being provided. The *Name* identifier is the containing domain in which a particular service is to be found. One of the purposes of SRV records is to identify when multiple individual servers in a domain support the same service.

For example, if a client would like to determine the host and port where the `ldap` service is available using the TCP protocol in the domain `example.com`, it would perform a query for SRV records using the domain name `_ldap._tcp.example.com`. Here is a real-world example:

```
Linux% host -t srv _ldap._tcp.openldap.org
_ldap._tcp.openldap.org has SRV record 0 0 389 www.openldap.org.
```

In this example, we are looking for a server providing the *Lightweight Directory Access Protocol* (LDAP) [RFC4510] service over TCP within the domain `openldap.org`. We find that it can be accessed at the server `www.openldap.org` using TCP port 389 (the default LDAP port). The *Priority* and *Weight* values are 0, as there are no alternative servers.

[RFC2782] did not specify a new IANA registry for SRV *Service* and *Proto* values. So, by default, the names correspond to the names maintained in IANA's "Service Name and Transport Protocol Port Number" registry [ISPR], and the *Proto* values are either `_tcp` or `_udp`. There are a few exceptions, however. [RFC5509] establishes conventions for SIP-based presence and instant messaging using the

following SRV *Service* and *Proto* names: `_im._sip` and `_pres._sip`. [RFC6186] defines the following SRV *Service* names for e-mail user agents to easily discover the contact information for IMAPS, SMTP, IMAP, and POP3 servers (the first two are ordinarily preferred when setting up an e-mail client): `_submission`, `_imap`, `_imaps`, `_pop3`, `_pop3s`. Although [RFC6186] doesn't require these names to use TCP as the corresponding *Proto* value, this is currently the only real option. For example, a user configuring a new *mail user agent* (MUA, essentially an e-mail program) might specify only the domain `example.com`. The MUA implementation would then likely perform DNS queries for at least `_submission._tcp.example.com` and `_imaps._tcp.example.com`.

11.5.6.11 Name Authority Pointer (NAPTR) Records

The *Name Authority Pointer* (NAPTR) RR type is used when DNS supports a *Dynamic Delegation Discovery System* (DDDS) [RFC3401]. A DDS is a general, abstract algorithm for applying dynamically retrieved string transformation rules to strings provided by applications and using the results, most often, for locating resources. Each DDS application customizes the operation of the general DDS rules for its particular use case. A DDS includes a rules database and a set of algorithms for forming strings that are used with the database to produce output strings. DNS is one such database [RFC3403], and with it the NAPTR resource record type is used to hold the transformation rules. One such DDS application has been defined for use with DNS to handle multinational telephone numbers and convert them to a standard *Uniform Resource Identifier* (URI) format [RFC3986] using ENUM (see Section 11.5.6.12).

In a DDS, an *algorithm* [RFC3402] directs how an *application-unique string* (AUS) is processed by rules contained in a database. The result can be either a *terminal string* (complete output) or another (nonterminal) string used to retrieve another rule that is applied to the AUS. In all, the collection forms a powerful string rewriting system that can be used to encode nearly anything that has a sufficiently regular syntax. The essence of this algorithm is captured in Figure 11-15.

The process illustrated in Figure 11-15 starts by applying the first Well-Known Rule to the AUS, which is uniquely identified for each application. The result forms a key used to retrieve another rule from a database. Rules are string-rewriting patterns and flags that are applied to the AUS, but never to the result of a rewritten string. The particular way this works is dependent on the application, but usually the rules are regular expression substitutions, similar to those used with the UNIX `sed` program [DR97]. When using the DNS as a database for supporting a DDS [RFC3403], the case in which we are interested, the keys are domain names and the rules are stored in NAPTR resource records. Each NAPTR RR contains the following fields: *Order*, *Preference*, *Flags*, *Services*, *Regular Expression* (sometimes abbreviated *Regexp*), and *Replacement*.

The *Order* field is a 16-bit unsigned integer specifying which NAPTR record to use before others (lower numbers are preferred to higher ones), as the DNS architecture does not guarantee the ordering of any particular set of resource records.

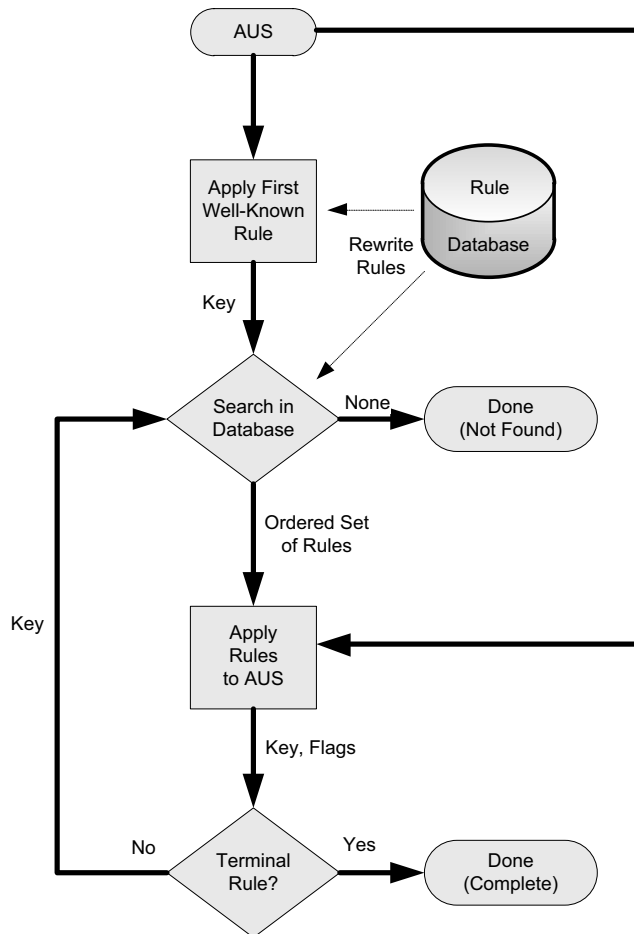


Figure 11-15 Abstract operation of the DDDS algorithm. Non-terminal records are permitted to form loops. Each iteration involves a string rewrite operation on the application's unique string.

The *Preference* field is used to influence the order of records containing the same order number. The *Order* field is supposed to place a mandatory ordering on RRs, whereas the preference number is advisory. The *Flags* field contains an unordered list of single characters from the set A–Z and 0–9 (case-insensitive). The particular application using NAPTR records (e.g., ENUM, described in the next section) defines the interpretation of the *Flags* field. The *Services* field is defined by the application to indicate which type of service is being described. The *Regular Expression* field contains a substitution expression that is applied to the AUS to form the identity of another server to use for another NAPTR lookup (non-terminal case) or the output string (terminal case). The *Replacement* field (which exists only when the *Regular Expression* does not) indicates the next server to query

for NAPTR records. It is encoded as a separate FQDN (no name compression is used within the DNS message). The uses for these two final (mutually exclusive) fields are very similar for historical reasons in the development of the NAPTR RR.

To get a better sense of how NAPTR processing works with applications, we will have a brief look at the ENUM and SIP DDDS applications, the URI/URN DDDS applications, and alternatives for regular NAPTR records called S-NAPTR and U-NAPTR. Specifying a DDDS entails specifying the application's AUS, first Well-Known Rule, expected output, valid databases, flags, and service parameters.

11.5.6.12 ENUM and SIP

In the ENUM DDDS [R06][RFC6116][RFC6117][RFC5483], which is used to map phone numbers to URI information, the AUS is an E.164-format telephone number (up to 15 digits starting with the + character). The initial + character differentiates E.164 numbers acceptable for use with the ENUM DDS from numbers in other name spaces. The first Well-Known Rule starts by removing any dashes or other non-digit characters in the AUS. The DDDS database is the DNS, where keys are domain names created from the AUS (which now consists only of digits) as follows: dot (.) characters are inserted between each digit and the result is reversed. Then, the suffix `.e164.arpa` is added. For example, the E.164 number `+1-415-555-1212` would be transformed to the key `2.1.2.1.5.5.5.5.1.4.1.e164.arpa`. The resulting domain name is used to query for NAPTR records.

The final output, possibly after multiple loops of the DDDS algorithm shown in Figure 11-15, is an absolute (not relative) URI. The only flag defined is the *U* flag, indicating a terminal rule that produces a URI. The lack of any flag indicates a non-terminal rule, sometimes called a *non-terminal NAPTR* (NTN). The service parameters, encoded in the *Service* field of the NAPTR record, are of the form `E2U+Service`, which derives from the string `E2U` (an indicator for E.164 to URI) plus a *Service* name subfield providing information about particular services associated with the number. Together, they form an *enumservice* identifier, and such services are registered with the IANA [ENUM][RFC6117]. Many have been created, including *enumservices* for fax, instant messaging, and presence indicators.

To see how this all works, we can construct a query for the number `+420738511111` at the University of Ostrava in the Czech Republic (lines are wrapped for clarity):

```
Linux% host -t naptr 1.1.1.1.5.8.3.7.0.2.4.e164.arpa
1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa has NAPTR record
    50 50 "u" "E2U+sip" "!^\\+(.*)$!sip:\\1@osuc.cz!" .
1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa has NAPTR record
    100 50 "u" "E2U+sip" "!^\\+(.*)$!sip:\\1@cesnet.cz!" .
1.1.1.1.1.5.8.3.7.0.2.4.e164.arpa has NAPTR record
    200 50 "u" "E2U+h323" "!^\\+(.*)$!h323:\\1@gk1ext.cesnet.cz!" .
```

Here we see the contents of three NAPTR records in the ENUM DDDS application, two for the SIP service and one for the H.323 service, used for Internet telephony. The order numbers are 50 and 100 for the SIP entries and 200 for the H.323 entry,

showing how it is possible using ENUM and NAPTR records to have multiple services associated with a single telephone number, and how the provider of the NAPTR records can indicate a preferred ordering of more than one gateway providing the same service.

Note

SIP is an IETF-specified protocol used for signaling and is especially popular for facilitating the connection of multimedia clients and servers. H.323 is an ITU-specified protocol for multimedia conferencing and communication, including a signaling sub-protocol. It is widely implemented in teleconferencing equipment. In this example and those that follow, the host program produces output that can be used as input to a zone file for a DNS server such as BIND. As a consequence, the output shows extra escape “\” characters (which appear as “\\”) that are not present in the actual DNS responses provided by the server.

To better understand how a NAPTR record’s rules are applied to the AUS, we will look at the second SIP record from the preceding example. After the DNS query is performed and the NAPTR RR is received, the string appearing between the first and second ! characters is used as a regular expression match and replacement. Thus, the string +420738511111 is matched against the regular expression `^\+(.*)$`. According to the matching rules for regular expressions, the match is successful, so the string rewrite rule becomes `sip:\1@cesnet.cz`. The special variable `\1` is replaced with the substring matching the first regular expression contained in parenthesis characters, `()`, which in this case is everything in the AUS except for the initial + character. In summary, the AUS +420738511111 is transformed into the URI `sip:420738511111@cesnet.cz`.

Once this URI is formed, the natural next step is for the driving application to contact a SIP server. However, SIP can itself be carried over different transport protocols, so the next step uses another DDDS that is tailored for SIP [RFC3263]. In this application, NAPTR records contain targets that identify the domain that should be used to perform SRV record queries. Continuing with the preceding example:

```
Linux% host -t naptr cesnet.cz
cesnet.cz has NAPTR record 200 50 "s" "SIP+D2T" "" _sip._tcp.cesnet.cz.
cesnet.cz has NAPTR record 100 50 "s" "SIP+D2U" "" _sip._udp.cesnet.cz.
```

Here we see the use of the `s` flag in the NAPTR, indicating that an SRV record is the result. The *Regexp* field is not used, so the result is a simple domain name substitution, given by the string in the *Replacement* field. The *Service* field is of the form `SIP+D2x` or `SIPS+D2x` where `SIP` and `SIPS` indicate the use of the SIP protocol and SIP protocol with security (TLS; see Chapter 18), respectively, and `x` is the single-letter identifier of the transport protocol: `U` for UDP, `T` for TCP, and `S` for SCTP [RFC4960]. In this example, the application would first attempt to look up and use the SRV record corresponding to SIP/UDP and would resort to SIP/TCP if that fails because the UDP entry has a lower preference value.

11.5.6.13 URI/URN Resolution

Although ENUM may be the most mature use of NAPTR records in the DNS, there are also DDDS applications defined for resolving URIs [RFC3404] and for persistent, location-independent URIs called *Uniform Resource Names* (URNs) [RFC2141]. All URIs (including URNs) consist of a scheme name followed by a substring compliant with semantics that are specific to the scheme. The current list of official schemes is maintained by the IANA [URI]. The URI and URN applications are so similar that it is worth considering them together. For the URI/URN DDDS application, then, the AUS is the URI or URN for which an authoritative “resolution” server is being located. The first Well-Known Rule for the URI application is simply the scheme name. For URNs, it is the name space identifier (the substring that appears after the `urn:` scheme identifier and before the next colon character). For example, `http://www.pearson.com` is a URI using the scheme (key) `http`, and the URN `urn:foo:foospace` would use `foo` as the first key. Four possible flags are currently defined: *S*, *A*, *U*, and *P*. The first three are terminal and indicate that the result is the domain name to use for fetching an SRV record, an IP address, or a URI, respectively. The *P* flag indicates that processing of the DDDS algorithm is to be discontinued and some application-specific processing (defined elsewhere) begins. All such flags are mutually exclusive. As with ENUM, the lack of any flag indicates an NTN.

Support for the URI/URN DDDS is still evolving. If we take a current (2011) look into the DNS, we can see how some of the schemes have been populated into the `uri.arpa` TLD:

```
Linux% host -t naptr http.uri.arpa
http.uri.arpa has NAPTR record 0 0 "" "" "!^http://([^/?#]*).*!\1!i" .
Linux% host -t naptr ftp.uri.arpa
ftp.uri.arpa has NAPTR record 0 0 "" "" "!^ftp://([^/?#]*).*!\1!i" .

Linux% host -t naptr mailto.uri.arpa
mailto.uri.arpa has NAPTR record 0 0 "" "" "!^mailto:(.*)@(.*)$\1\2!i" .
Linux% host -t naptr urn.uri.arpa
urn.uri.arpa has NAPTR record 0 0 "" "" "/urn:([^:]+)\1/i" .
```

The first three of these NAPTR records contain rewrite rules and no flags. Thus, they essentially indicate that the application should extract the domain name from the corresponding URI and continue the DDDS algorithm. The trailing `i` flag after the last `!` character indicates that case checking is to be performed in an insensitive way. For example, `maIlto:person@example.com` is rewritten to be just `example.com`. The fourth record is used to extract the URN name space ID and continue processing. For URNs, there are a small number (two at present) of NAPTR records in the DNS set up in `urn.arpa`:

```
Linux% host -t naptr pin.urn.arpa
pin.urn.arpa has NAPTR record 100 100 "" "" "" pin.verisignlabs.com.
Linux% host -t naptr uci.urn.arpa
uci.urn.arpa has NAPTR record 100 100 "" "" "" uci.or.kr.
```

These URN name spaces appear to be receiving little attention at present, and it is still unclear to what extent URNs will be widely used, as there are now competing methods for expressing and locating objects using persistent identifiers (e.g., see [P10]). Nevertheless, more than 40 URN name spaces have been defined [URN], so there continues to be community interest in establishing name spaces, even though few have corresponding global, active NAPTR records.

11.5.6.14 S-NAPTR and U-NAPTR

A common issue arises when an application wishes to determine the particular host, protocol, and port number to use for reaching a service within a domain. For example, a mail-reading application running on a user's computer in the `example.com` domain may need to find a server offering the IMAP service. A convention has arisen to simply prepend the service name to the domain (e.g., `imap.example.com`). Using CNAME, A, or AAAA records is somewhat inflexible, because these record types do not convey any indication of which transport protocol or port number to use. SRV records go further by providing another layer of indirection, but their targets may contain only domain names for which an A or AAAA record is subsequently retrieved. Using NAPTR records instead provides more flexibility through an additional layer of indirection and allows for other target record types (such as SRV records) to be used.

The NAPTR structure and rewrite capabilities have caused concern for some implementers and operators given the complexity of the regular expressions. In an effort to simplify the situation yet still provide a method beyond basic SRV records for locating services, *straightforward* NAPTR (S-NAPTR) [RFC3958] specifies a DDS application for mapping domain “labels” that contain a service name using certain simplifying restrictions on the contents of the NAPTR records.

For S-NAPTR, the AUS is a domain label for which an authoritative server for a particular service is sought. The first Well-Known Rule is the identity function. The expected output is the information necessary to contact a particular application service within a domain (e.g., protocol, host, port). Only *S* and *A* terminal flags are permitted, which indicate an SRV RR or a domain name (which is to be used to form a subsequent request for an A or AAAA RR), respectively. The service parameters are taken from a set maintained in an IANA registry [SNP], and the *Regexp* field is not used. Only the *Replacement* field is active. S-NAPTR is used in conjunction with the *Internet Registry Information Service* (IRIS) [RFC3981], an XML-based text application protocol for exchanging information pertaining to domain name and other registration information whose database is contained within the `iris.arpa` portion of the DNS name space; for example:

```
Linux% host -t naptr areg.iris.arpa
reg.iris.arpa NAPTR
      100 10 "" "AREG1:iris.xpc:iris.lwz" "" areg.nro.net.
```

This example uses S-NAPTR (no regular expression) to indicate that in order to perform an ISIS query for AREG1-type data (see [RFC4698]), a subsequent NAPTR query should be initiated to `areg.nro.net`.

Experience and further consideration of S-NAPTR led to the development of *URI-enabled NAPTR* (U-NAPTR) [RFC4848], which relaxes some of the restrictions of S-NAPTR but maintains all of its other features and registries. Most important, an additional *U* flag is permitted, which enables the NAPTR record target to be a URI and thus allows the use of regular expressions. This is similar to the fully generic version of NAPTR, except U-NAPTR regular expressions are restricted to the following form: `!.*!<URI>!`. That is, the entire AUS is replaced with a URI. U-NAPTR is being used in conjunction with the *Location-to-Service Translation protocol* (LoST) [RFC5222], which can be used to determine the correct service given a point of network attachment and geographical location. Such information is useful in public safety applications where geography dictates the particular jurisdiction and responsible parties that should provide emergency services.

11.5.7 Dynamic Updates (DNS UPDATE)

It is possible to dynamically update a zone, called *DNS UPDATE*, using a protocol defined in [RFC2136]. It supports the ability to specify *prerequisites* in conjunction with an update request. Prerequisites are evaluated at the server; if they are not true, the update is not performed and an error message is returned.

DNS UPDATE is accomplished by sending *dynamic update* DNS messages to an authoritative DNS server for a zone. The structure of such messages is the same as for a conventional DNS message, except the header fields and sections have different names (see Figure 11-3). The sections indicate the zone being updated, prerequisites that require various RRs to be present (or not) for the update to take effect, and the *update information*. In an update, the header mirrors the format for a query, but the *Opcode* field is set to Update (5). The header fields *ZOCOUNT*, *PRCOUNT*, *UPCOUNT*, and *ADCOUNT* contain counts of the following: zones to be updated (this will have the value 1), prerequisites to consider, updates to be made, and additional information records, respectively. [RFC2136] also defines a collection of RCODE values carried in DNS response messages capable of indicating conditions relating to problems with the prerequisites or server (values 6–10 in Table 11-2).

The zone section of an update message (see Figure 11-7) indicates the zone's name, a type, and a class. The type value will be 6 to indicate the presence of an SOA record, which identifies the zone. The class value will be 1 (Internet) for any update message with which we are concerned. All records being updated must be in the same zone.

The prerequisite section of an update message contains one or more prerequisites, expressed using the format for RRs we discussed previously in Section 11.5.5. There are five types of prerequisites: *RRSet exists* (value-dependent and value-independent varieties), *RRSet does not exist*, *name is in use*, and *name is not in use*. Recall that an RRSet is a group of RRs from the same zone sharing a common name, class, and type. To express the semantics of a prerequisite, a combination of an RR's class, type, and RDATA values are set according to Table 11-5.

The *RRSet exists* type means that at least one RRSet exists in the zone specified in the zone section that matches the name and type of the corresponding RR in

Table 11-5 *RR Class and Type fields used in prerequisite section to indicate prerequisite type*

Prerequisite Type (Semantics)	Class Setting	Type Setting	RDATA Setting
RRSet exists (value-independent)	ANY	Same as zone's type	Empty
RRSet exists (value-dependent)	Same as zone's class	Type being checked	RRSet being checked
RRSet does not exist	NONE	Type being checked	Empty
Name is in use	ANY	ANY	Empty
Name is not in use	NONE	ANY	Empty

the prerequisite section. In the value-dependent case, the prerequisite is true only if the matching RRs also contain matching RDATA values. The *RRSet does not exist* type means that no RRSet in the zone specified in the zone section matches the name and type of the RR in the Prerequisites section. The last two cases (*Name is in use* and *Name is not in use*) refer only to the domain name; the type value is not used. The values for NONE and ANY as DNS classes are 254 and 255, respectively.

Following the Prerequisite section, the Update section contains RRs to be added or deleted from the zone specified in the zone section. There are four types of updates, encoded as an RR with various combinations of values in the *Class*, *Type*, and *RDATA* fields, as indicated in Table 11-6.

Table 11-6 *RR Class and Type fields used in Update section to indicate update type*

Use	Class Setting	Type Setting	RDATA
Add RR to RRSet	Same as zone's class	Type of RR being added	RDATA of RR being added
Delete RRSet	ANY	Type of RRSet to delete	Empty (TTL and RDLENGTH also zero)
Delete all RRSets from a name	ANY	ANY	Empty (TTL and RDLENGTH also zero)
Delete RR from RRSet	NONE	Type of RR being deleted	Matching RDATA to delete

The update section contains a collection of RRs that are processed provided no errors have occurred due to prerequisites or server problems. Each RR encodes an addition or deletion operation. Modifications can be performed as a deletion followed by an addition. To see an example of DNS UPDATE, we can induce a Windows machine to perform a dynamic DNS update using the following command:

```
C:\> ipconfig /registerdns
```

Windows clients issue updates for their computer name and domain name by default, but this behavior can also be enabled for IPv4 on a per-DNS-suffix basis by checking the box labeled "Use this connection's DNS suffix in DNS registration"

under the DNS section of the Advanced TCP/IP Settings, found on the General tab of the Internet Protocol (TCP/IP) Properties menu associated with each interface enabled for TCP/IP. For IPv6, the same procedure is used, but on the IPv6 Properties menu. In the example shown in Figure 11-16, we can see how the machine named *vista* updates the local zone *dyn.home* as it issues the DNS update message shown.

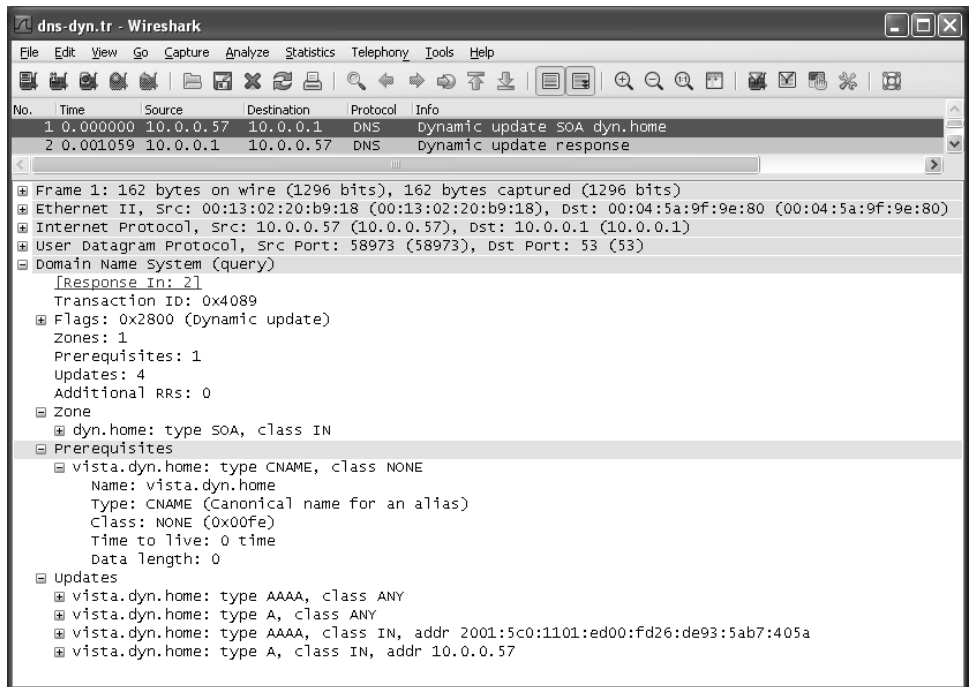


Figure 11-16 A DNS dynamic update contains an SOA record in the zone section and RRs in the update section. This case includes new IPv4 and IPv6 addresses for the host *vista.dyn.home*.

Figure 11-16 shows how a dynamic update is encoded. The DNS server at 10.0.0.1 (running BIND9 [AL06] in this example) is configured to allow dynamic updates. The zone section contains an SOA record identifying the zone to be updated (*vista.dyn.home*). The prerequisite section contains an RR with a zero-length RDATA section and 0 TTL value. The RR corresponds to the type of prerequisite in the third row of Table 11-5 (*RRset does not exist*) because its type is not ANY (it is CNAME) and its class is set to NONE (254).

In this particular case, the addresses 10.0.0.57 and 2001:5c0:1101:ed00:fd26:de93:5ab7:405a are to be associated with the name *vista.dyn.home*. This is accomplished by first deleting the AAAA and A RRsets (corresponding to row 2 in Table 11-6), and then adding the AAAA and A RRsets (corresponding to row 1 in Table 11-6) for the desired addresses.

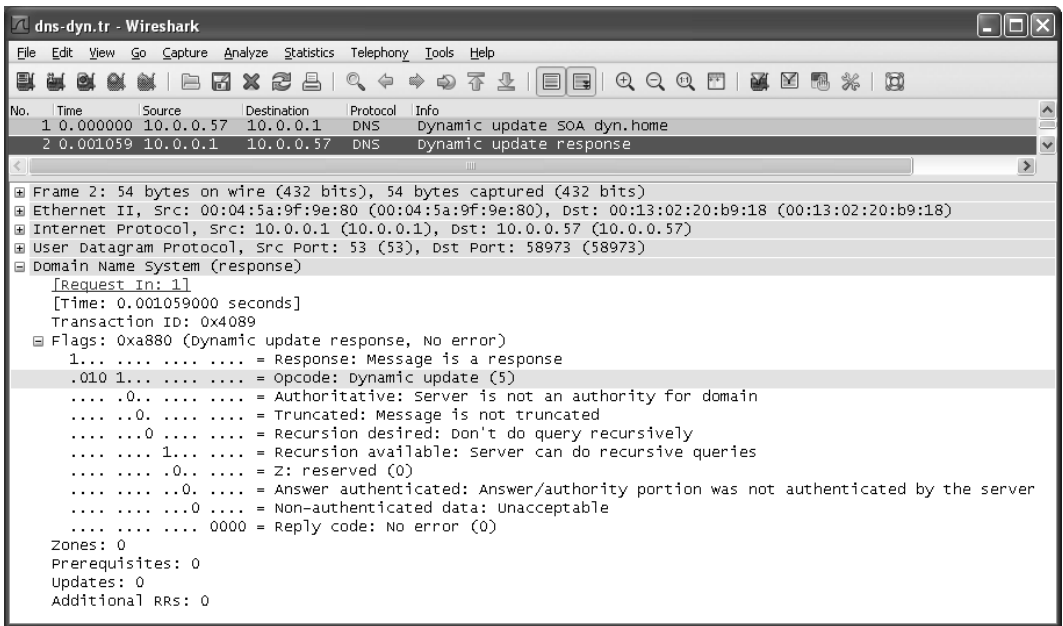


Figure 11-17 The response to a dynamic update request includes a transaction ID and status flag set.

Responses to DNS updates are straightforward and compact. The response for the update shown in Figure 11-16 is illustrated in Figure 11-17.

The *Flags* field indicates a successful update (no error). The transaction ID (0x4089) is used to ensure that the update response matches a corresponding request. Note that on Linux, the `nsupdate` program can be used to update a cooperative DNS server. DNS servers cooperate with a requested update only if an authentication and access control procedure indicates that the request is acceptable. This can be as simple as nothing or listing the IP addresses of clients at the server, neither of which is very secure, or using somewhat more complex and secure methods that provide *transaction authentication* (see TSIG and SIG(0) in Chapter 18).

11.5.8 Zone Transfers and DNS NOTIFY

A zone transfer is used to copy a set of RRs for a zone from one server to another (generally from the master server to slave servers). The purpose of doing so is to keep multiple servers in sync with respect to a zone's contents. Multiple servers provide resiliency to failure, in case a server should go down. Performance can also be improved as multiple servers can be used to share the processing load for incoming queries. Finally, the latency of a DNS query/response can potentially be reduced if servers are placed in locations close to clients (i.e., where the network latency between resolver and server is small).

As originally specified, zone transfers are initiated after *polling*, where slaves periodically contact masters to see if a zone transfer is necessary by comparing the zones' version numbers. A later method says if a zone transfer needs to be initiated using an asynchronous update mechanism when the zone contents change. This is called *DNS NOTIFY*. Once a zone transfer is initiated, either the entire zone is transferred (using DNS AXFR messages) [RFC5936], or an *incremental zone transfer* option may be used (using DNS IXFR messages) [RFC1995]. The general scheme operates according to the illustration in Figure 11-18.

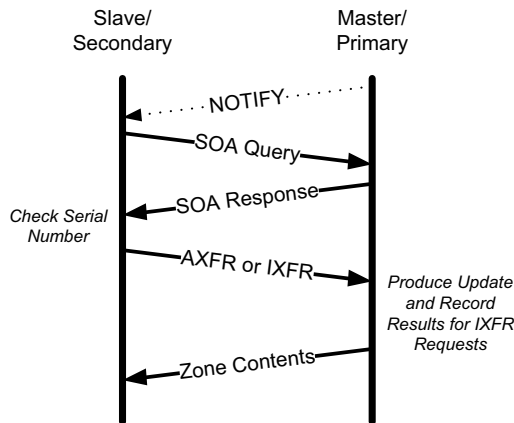


Figure 11-18 A DNS zone transfer copies the contents of zones between servers. An optional notification can cause a slave to request a full or incremental zone transfer.

We will now have a closer look at each of the options, including full and incremental zone transfers, plus DNS Notify.

11.5.8.1 Full Zone Transfers (AXFR Messages)

Full zone transfers are controlled by the zone transfer parameters carried in a zone's SOA record: primary name server, serial number, and the refresh, retry, and expire intervals. When configured, a slave server attempts to contact the primary server to see if a zone transfer is necessary. Contacts are attempted periodically, according to the refresh interval. They are also attempted when a server first starts. If a contact is not successful (no response from the server), retries are attempted periodically according to the retry interval (generally shorter than the refresh interval). The entire zone contents are flushed if not refreshed within the expire interval, effectively incapacitating the server for the zone.

An All Zone Transfer (AXFR) DNS message (a standard query containing type AXFR in the Question section) is used to request a complete zone transfer using TCP. To see such a message, we may arrange for a request to be initiated using the `host` program in our local network:

```
Linux% host -l home.
Using domain server:
Name: 10.0.0.1
Address: 10.0.0.1#53
Aliases:

home name server gw.home.
ap.home has address 10.0.0.6
gw.home has address 10.0.0.1
...
```

The `-l` flag asks the `host` program to perform a full zone transfer from a local DNS server. The program initiates a TCP-based query/response dialogue, illustrated in Figure 11-19.

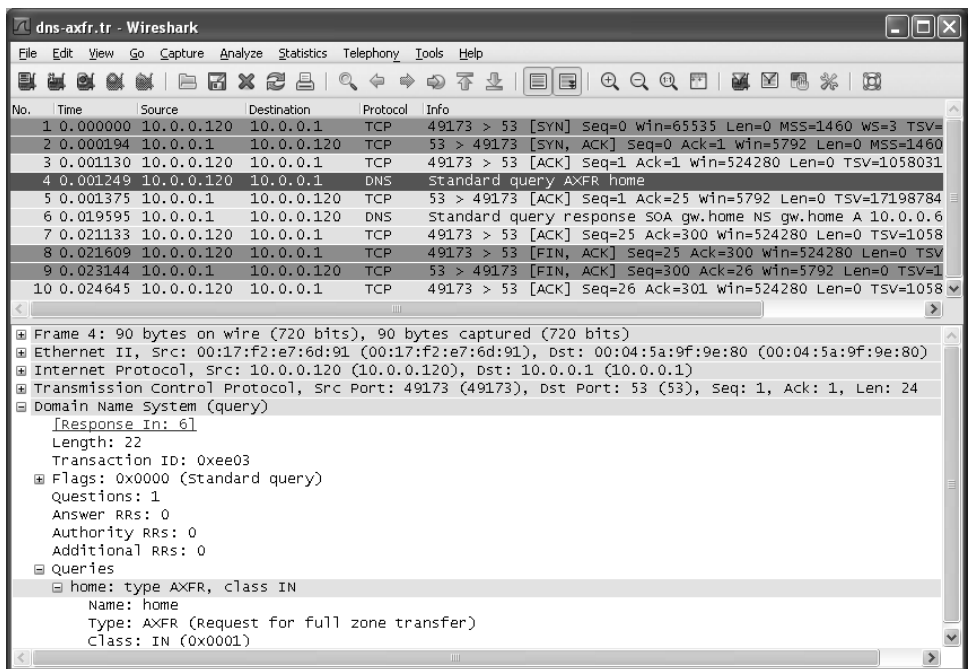


Figure 11-19 A DNS request for a full zone transfer uses the AXFR record type and TCP as a transport protocol.

In Figure 11-19 we can see how the zone transfer takes place using TCP. The first three TCP segments are part of the standard TCP connection establishment process (see Chapter 13). The fourth (decoded) packet is the request. It is a normal DNS standard, with type AXFR and class IN (Internet). The query is for the domain name `home`. The response to this query is contained in message 6, following the TCP ACK (see Figure 11-20).

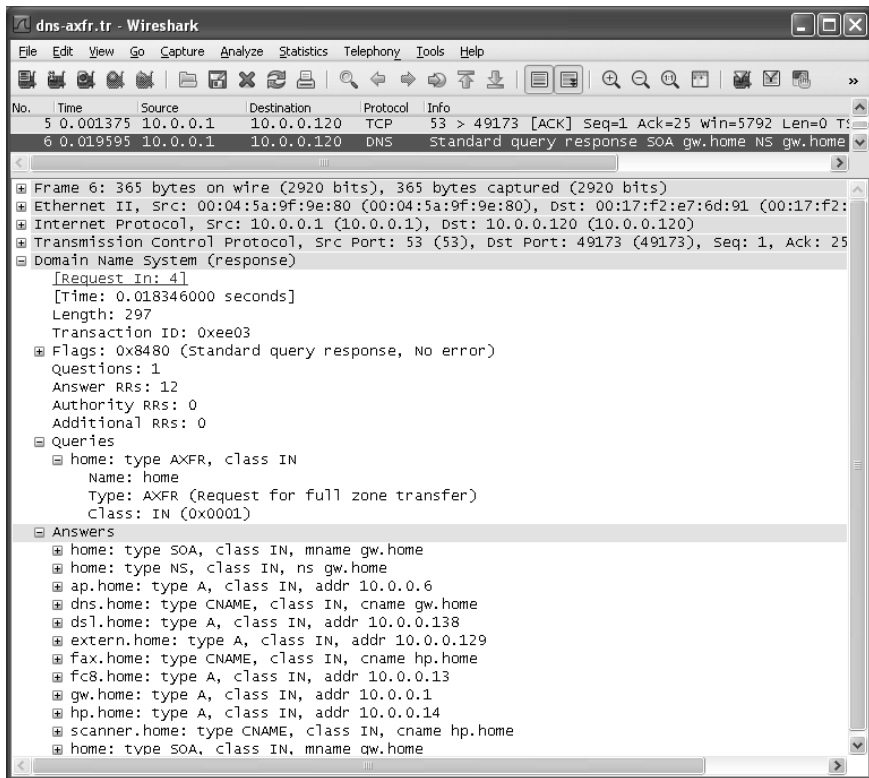


Figure 11-20 The successful response for a full zone transfer request includes all of the records for the zone. The transaction takes place using TCP, as the zone contents may be large and a reliable copy is required.

In Figure 11-20 we can see how the entire zone is carried in the response. After receiving the response, the client's TCP ACKs the data and initiates a TCP connection close. The connection is closed gracefully using the FIN-ACK handshake (packets 8–10). See Chapter 13 for more details on the standard TCP connection establishment and clearing.

Although it used to be possible to perform such zone transfers with virtually any DNS server, they are now typically restricted to the authoritative servers in a zone (e.g., those listed in NS records for the zone). The reason for this restriction is privacy and security—knowledge of the hosts within the zone might help an attacker target particular services or hosts.

11.5.8.2 Incremental Zone Transfers (IXFR Messages)

To improve the efficiency of zone transfers, [RFC1995] defines the use of *incremental* zone transfers. Using incremental zone transfers and the IXFR message type, only the changes in a zone are provided. To execute an incremental zone transfer, the client (e.g., slave server) must provide its current serial number for the zone. In

the following example, we can emulate a requesting server by providing the serial number and using the `dig` program:

```
Linux% dig +short @10.0.0.1 -t ixfr=1997022700 home.
gw.home. hostmaster.gw.home. 1997022700 10800 15 604800 10800
```

The command line indicates that output from the command should be short, 10.0.0.1 is the address of the DNS server to use, and an incremental zone transfer starting with serial number 1997022700 should be performed. This example creates an exchange similar to the one illustrated in Figures 11-19 and 11-20 for AXFR, except in this case the serial number of the request matches the current serial number (see Figure 11-21).

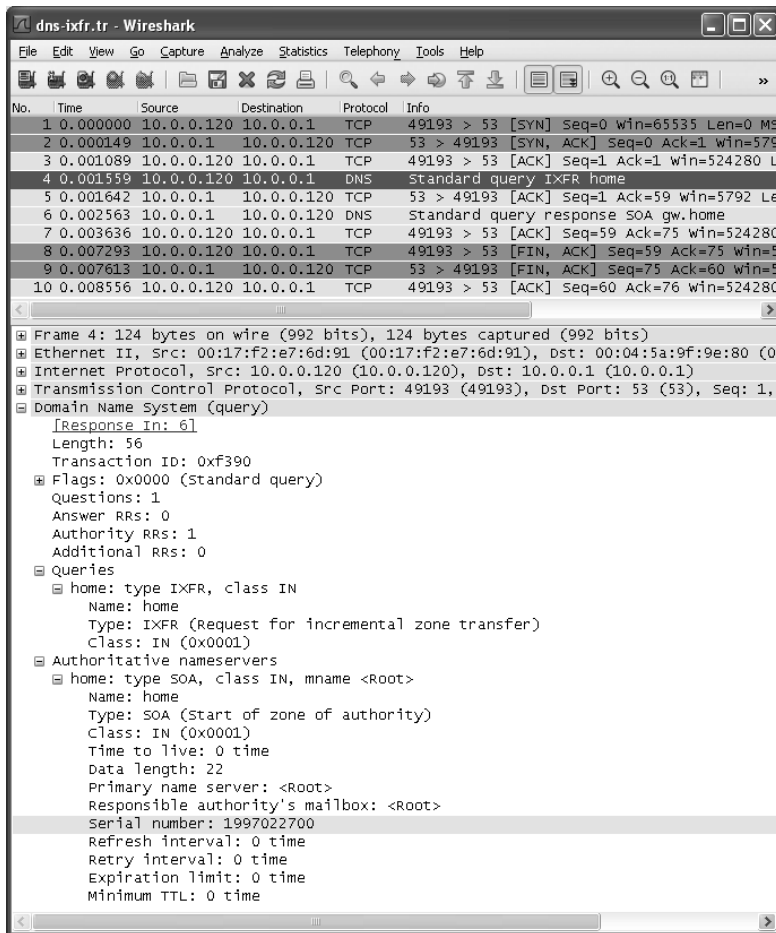


Figure 11-21 An incremental zone transfer request (IXFR record type) carried on TCP. The serial number is used to determine which records, if any, have changed since an earlier zone transfer took place.

Figure 11-22 shows how the IXFR request includes a mostly empty SOA RR in the authority section. The SOA record includes the serial number specified (1997022700). The response (packet 6) contains no real information because this serial number matches the current one at the server.

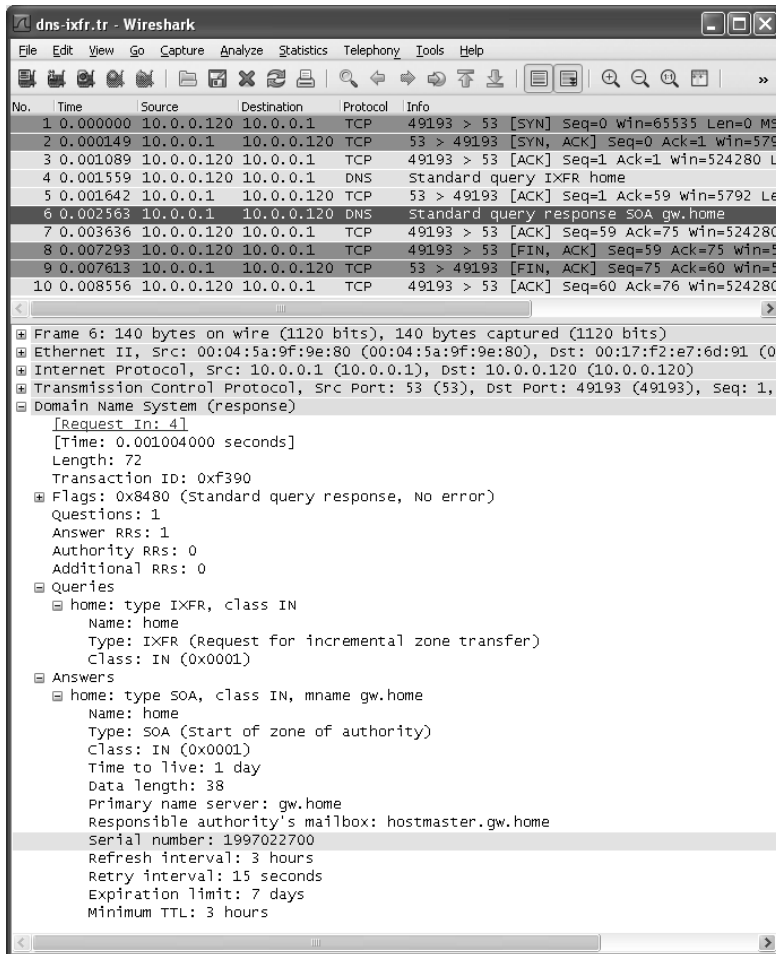


Figure 11-22 The response to an IXFR request when the serial number is current contains only an SOA record and no additional information.

The response in Figure 11-22 contains only the SOA RR in the answer section. Unlike the one contained in the query, this one is filled in with the complete SOA fields (e.g., mailbox, zone transfer parameters). However, there are no additional answers because the current serial number for the zone matches that of the request. Thus, the requesting client is assumed to be up-to-date and not in need of any additional information or a zone transfer.

11.5.8.3 DNS NOTIFY

As mentioned previously, polling has traditionally been used to determine the need for zone transfers, meaning that the slave servers would check with a master periodically (the “refresh” interval) to see if the zone had been updated (indicated by a different serial number), in which case a zone transfer would be initiated. This is a somewhat wasteful process because many useless polls may occur before the zone is updated. To improve the situation, [RFC1996] developed the *DNS NOTIFY* mechanism. DNS NOTIFY allows a server with modified zone contents to notify slave servers that an update has been made and a zone transfer should be initiated. More specifically, if enabled, a notification message is sent to a set of interested servers if the SOA RR for a zone changes (e.g., if the serial number increases). This allows zone transfers to be initiated easily when required. Using a local (home) name server, we can see how this works (see Figure 11-23).

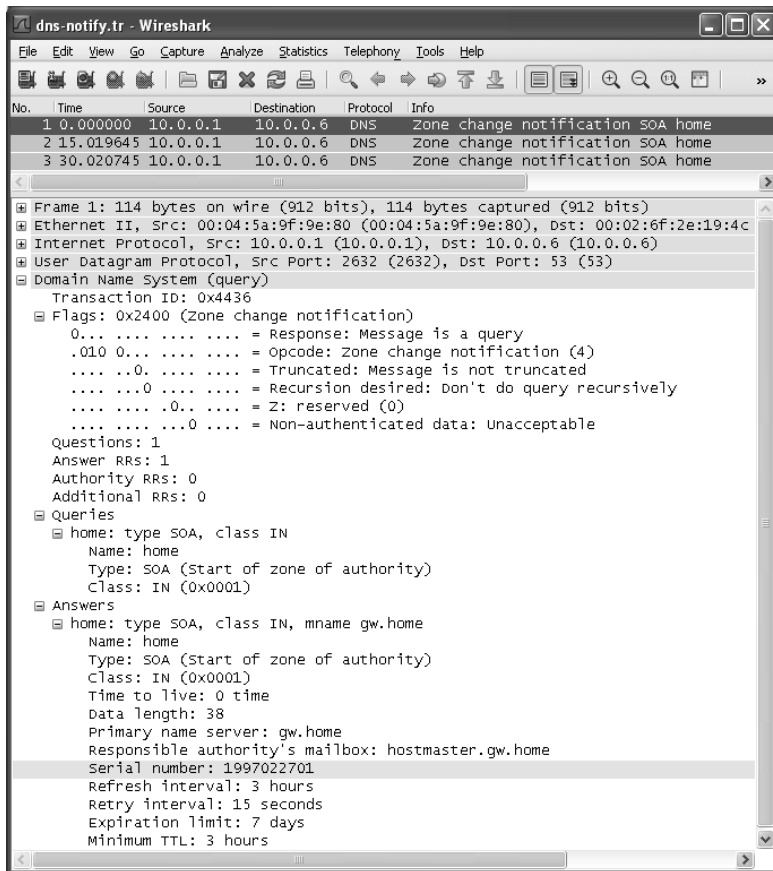


Figure 11-23 A DNS NOTIFY indicating an update to the zone file. There are two retransmissions spaced 15s apart (contrary to the method suggested in the standard).

This example illustrates the simple DNS NOTIFY message sent to a host in the server's *notify set* of servers that should be informed of a zone change. The message is a UDP/IPv4 DNS query message with the *Flags* field indicating a zone change notification. The query section contains the type and class for an SOA record, and the answer section contains the current SOA RR for the zone (with TTL 0), including the serial number. This provides sufficient information for a notified server to determine that a zone transfer may be necessary. Note that a single server may receive notifications from multiple other servers as they update their zone information. This does not present a problem for the protocol's operation.

The DNS NOTIFY mechanism defaults to using UDP, an unreliable protocol. In this particular example, the notify set contains only the address 10.0.0.11, which does not run a DNS server. Consequently, the message is resent every 15s hoping for a response that never arrives.

Note

The time between retransmissions and the total number of retransmissions to attempt are suggested by [RFC1996] to be 60s and five retransmissions, respectively. It also suggests that a timer backoff method (additive or exponential) be used. Here we can see that the BIND9 implementation fails to respect these suggestions, as the two retransmissions are 15s apart.

Responses are simply DNS response messages with no useful information except the transaction ID; they are used only to complete the protocol and cancel retransmissions at the sending server.

11.6 Sort Lists, Round-Robin, and Split DNS

So far we have discussed how domain names are set up, the types of resource records DNS supports, and the DNS protocol used to fetch and update a zone. One subtle point to consider is what data is returned and in what order in response to a DNS query. A DNS server could return all matching data to any client in whatever order the server finds most convenient. However, special configuration options and behaviors are available in most DNS server software to achieve certain operational, privacy, or performance goals. Consider the topology shown in Figure 11-24.

The type of topology shown in Figure 11-24 is typical of a small enterprise. There is a private network and a public network including a DNS server. In addition, there is a pair of hosts on the DMZ (A and B), one on the internal network (C) and one on the Internet (R). A multihomed host (M) spans the DMZ and internal networks. M therefore has two IP addresses drawn from two different network prefixes.

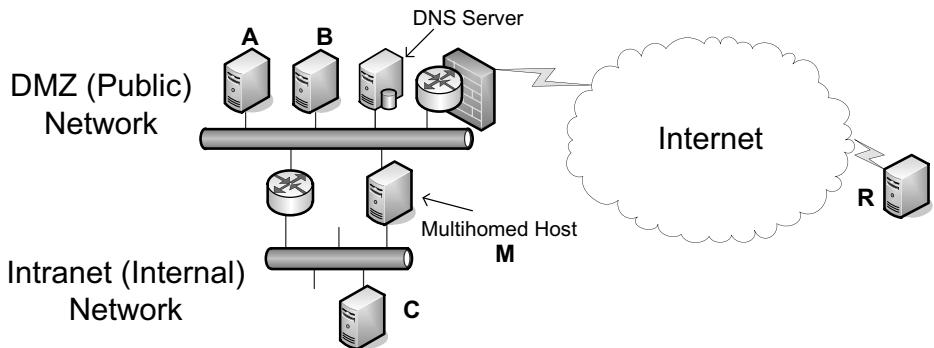


Figure 11-24 In a small enterprise topology, DNS may be configured to return different addresses depending on the requesting IP address.

A host wishing to contact M performs a DNS lookup that returns two addresses—one associated with the internal network and one with the DMZ. Naturally, it would be more efficient if A, B, and R reached M via the DMZ and C reached M via the internal network. This generally happens if the DNS server orders its returned address records based on the source IP address of the request. (It could also use the destination IP address, especially if M uses multiple IP addresses from different subnets on the same network interface.) If the requesting system uses a source IP address with the same network prefix as the source of a returning address record, the DNS server places the set of such matching records early in the returned message. This behavior encourages the client to find the “closest” IP address for a particular server it is attempting to contact, because most simple applications attempt to contact the first address found among the returned address records. The precise behavior can usually be controlled using a so-called `sortlist` or `rrset-order` directive (options used in configuration files for resolvers and servers). Such sorting behavior may also happen automatically if performed by the DNS server software by default.

A somewhat related situation arises when one service is offered using more than one server such that the incoming connections are load-balanced (i.e., divided among the servers). In the preceding example, imagine that a service is offered on both A and B. Such a service may be identified by the URL `http://www.example.com`. Requesting clients (like R) perform a DNS query on the domain name `www.example.com`, and the DNS server eventually returns a set of address records. To achieve load balancing, the DNS server may be configured to use *DNS round-robin*, which means that the server permutes the order of the returned address records. Doing so encourages each new client to access the service on a different server from the previous client. While this helps to balance load, it is far from perfect. When records are cached, the desired effect may not occur because of reuse of existing cached address records. In addition, this scheme may balance the number of connections well across servers, but not the load. Different

12

TCP: The Transmission Control Protocol (Preliminaries)

12.1 Introduction

So far we have been discussing protocols that do not include their own mechanisms for delivering data reliably. They may *detect* that erroneous data has been received, using a mathematical function such as a checksum or CRC, but they do not try very hard to repair errors. With IP and UDP, no error repair is done at all. With Ethernet and other protocols based on it, the protocol provides some number of retries and then gives up if it cannot succeed.

The problem of communicating in environments where the communication medium may lose or alter the messages being delivered has been studied for years. Some of the most important theoretical work on the topic was developed by Claude Shannon in 1948 [S48]. This work, which popularized the term bit and became the foundation of the field of *information theory*, helps us understand the fundamental limits on the amount of information that can be moved across an information channel that is *lossy* (that may delete or alter bits). Information theory is closely related to the field of *coding theory*, which provides ways of encoding information so that it is as resilient as possible to errors in the communications channel. Using *error-correcting codes* (basically, adding redundant bits so that the real information can be retrieved even if some bits are damaged) to correct communications problems is one very important method for handling errors. Another is to simply “try sending again” until the information is finally received. This approach, called *Automatic Repeat Request* (ARQ), forms the basis for many communications protocols, including TCP.

12.1.1 ARQ and Retransmission

If we consider not only a single communication channel but the multihop cascade of several, we realize that not only may we have the types of errors mentioned so far (packet bit errors), but there may be others. These problems might arise at an intermediate router and are the types of problems we brought up when discussing IP: packet reordering, packet duplication, and packet erasures (drops). An error-correcting protocol designed for use over a multihop communications channel (such as IP) must cope with all of these problems. Let us now explore the protocol mechanisms that can be brought to bear on them. After we discuss these in the abstract, we shall explore how they are used by TCP in the Internet.

A straightforward method of dealing with packet drops (and bit errors) is to resend the packet until it is received properly. This requires a way to determine (1) whether the receiver has received the packet and (2) whether the packet it received was the same one the sender sent. The method for a receiver to signal to a sender that it has received a packet is called an *acknowledgment*, or ACK. In its most basic form, the sender sends a packet and awaits an ACK. When the receiver receives the packet, it sends the ACK. When the sender receives the ACK, it sends another packet, and the process continues. Interesting questions to ask here are (1) How long should the sender wait for an ACK? (2) What if the ACK is lost? (3) What if the packet was received but had errors in it?

As we shall see, the first question turns out to be deep. Deciding how long to wait relates to how long the sender should *expect* to wait for an ACK. Determining this may be difficult; we postpone the discussion of techniques for it until we discuss TCP in detail later (see Chapter 14). The answer to question 2 is easier: if an ACK is dropped, the sender cannot readily distinguish this case from the case in which the original packet is dropped, so it simply sends the packet again. Of course, the receiver may receive two or more copies in that case, so it must be prepared to handle that situation (see the next paragraph). As for the third question, we can appeal to the codes mentioned in Section 12.1. It is generally much easier to use codes to *detect* errors in a large packet (with high probability) using only a few bits than it is to correct them. Simpler codes are typically not capable of correcting errors but are capable of detecting them. That is why checksums and CRCs are so popular. In order to detect errors in a packet, then, we use a form of checksum. When a receiver receives a packet containing an error, it refrains from sending an ACK. Eventually, the sender resends the packet, which ideally arrives undamaged.

Even with the simple scenario presented so far, there is the possibility that the receiver might receive *duplicate* copies of the packet being transferred. This problem is addressed using a *sequence number*. Basically, every unique packet gets a new sequence number when it is sent at the source, and this sequence number is carried along in the packet itself. The receiver can use this number to determine whether it has already seen the packet and if so, discard it.

The protocol described so far is reliable but not very efficient. Consider what happens when the time to deliver even a small packet from sender to receiver (the

delay or latency) is large (e.g., a second or two, which is not unusual for satellite links) and there are several packets to send. The sender is able to inject a single packet into the communications path but then must stop until it hears the ACK. This protocol is therefore called “stop and wait.” Its throughput performance (data sent on the network per unit time) is proportional to M/R where M is the packet size and R is the round-trip time (RTT), assuming no packets are lost or irreparably damaged in transit. For a fixed-size packet, as R goes up, the throughput goes down. If packets are lost or damaged, the situation is even worse: the “goodput” (useful amount of data transferred per unit time) can be considerably less than the throughput.

For a network that doesn’t damage or drop many packets, the cause for low throughput is usually that the network is not being kept busy. The situation is similar to using an assembly line where new work cannot enter the line until a complete product emerges. Most of the line goes idle. If we take this comparison one step further, it seems obvious that we would do better if we could have more than one work unit in the line at a time. It is the same for network communication—if we could have more than one packet in the network, we would keep it “more busy,” leading to higher throughput.

Allowing more than one packet to be in the network at a time complicates matters considerably. Now the sender must decide not only when to inject a packet into the network, but also how many. It also must figure out how to keep the timers when waiting for ACKs, and it must keep a copy of each packet not yet acknowledged in case retransmissions are necessary. The receiver needs to have a more sophisticated ACK mechanism: one that can distinguish which packets have been received and which have not. The receiver may need a more sophisticated buffering (packet storage) mechanism—one that allows it to hold “out-of-sequence” packets (those packets that have arrived earlier than those expected because of loss or reordering), unless it simply wants to throw away such packets, which is very inefficient. There are other issues that may not be so obvious. What if the receiver is slower than the sender? If the sender simply injects many packets at a very high rate, the receiver might just drop them because of processing or memory limitations. The same question can be asked about the routers in the middle. What if the network infrastructure cannot handle the rate of data the sender and receiver wish to use?

12.1.2 Windows of Packets and Sliding Windows

To handle all of these problems, we begin with the assumption that each unique packet has a sequence number, as described earlier. We define a *window* of packets as the collection of packets (or their sequence numbers) that have been injected by the sender but not yet completely acknowledged (i.e., the sender has not received an ACK for them). We refer to the *window size* as the number of packets in the window. The term *window* comes from the idea that if you lined up all the packets sent during a communication session in a long row but had only a small aperture

through which to view them, you would see only a subset of them—like peering through a window. The sender's window (and the line of other packets) can be graphically depicted as shown in Figure 12-1.

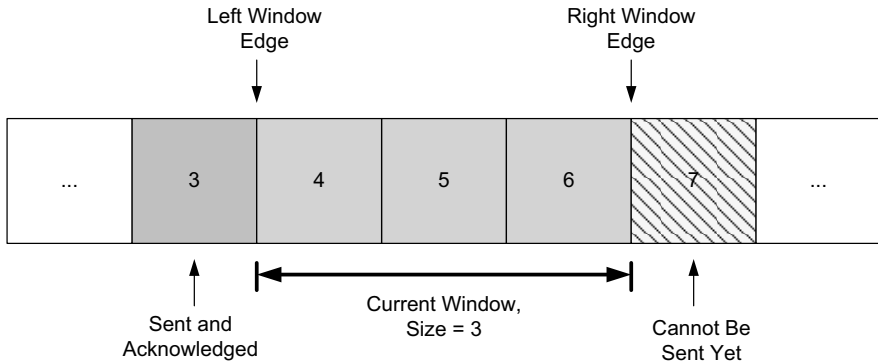


Figure 12-1 The sender's window, showing which packets are eligible to be sent (or have already been sent), which are not yet eligible, and which have already been sent and acknowledged. In this example, the window size is fixed at three packets.

This figure shows the current window of three packets, for a total window size of 3. Packet number 3 has already been sent and acknowledged, so the copy of it that the sender was keeping can now be released. Packet 7 is ready at the sender but not yet able to be sent because it is not yet “in” the window. If we now imagine that data starts to flow from the sender to the receiver and ACKs start to flow in the reverse direction, the sender might next receive an ACK for packet 4. When this happens, the window “slides” to the right by one packet, meaning that the copy of packet 4 can be released and packet 7 can be sent. This movement of the window gives rise to another name for this type of protocol, a *sliding window* protocol.

The sliding window approach can be used to combat many of the problems described so far. Typically, this window structure is kept at both the sender and the receiver. At the sender, it keeps track of what packets can be released, what packets are awaiting ACKs, and what packets cannot yet be sent. At the receiver, it keeps track of what packets have already been received and acknowledged, what packets are expected (and how much memory has been allocated to hold them), and which packets, even if received, will not be kept because of limited memory. Although the window structure is convenient for keeping track of data as it flows between sender and receiver, it does not provide guidance as to how large the window should be, or what happens if the receiver or network cannot handle the sender's data rate. We shall now see how these are related.

12.1.3 Variable Windows: Flow Control and Congestion Control

To handle the problem that arises when a receiver is too slow relative to a sender, we introduce a way to force the sender to slow down when the receiver cannot keep up. This is called *flow control* and is usually handled in one of two ways. One way, called *rate-based* flow control, gives the sender a certain data rate allocation and ensures that data is never allowed to be sent at a rate that exceeds the allocation. This type of flow control is most appropriate for streaming applications and can be used with broadcast and multicast delivery (see Chapter 9).

The other predominant form of flow control is called *window-based* flow control and is the most popular approach when sliding windows are being used. In this approach, the window size is not fixed but is instead allowed to vary over time. To achieve flow control using this technique, there must be a method for the receiver to signal the sender how large a window to use. This is typically called a *window advertisement*, or simply a *window update*. This value is used by the sender (i.e., the receiver of the window advertisement) to adjust its window size. Logically, a window update is separate from the ACKs we discussed previously, but in practice the window update and ACK are carried in a single packet, meaning that the sender tends to adjust the size of its window at the same time it slides it to the right.

If we consider the effect of changing the window size at the sender, it becomes clear how this achieves flow control. The sender is allowed to inject W packets into the network before it hears an ACK for any of them. If the sender and receiver are sufficiently fast, and the network loses no packets and has an infinite capacity, this means that the transfer rate is proportional to (SW/R) bits/s, where W is the window size, S is the packet size in bits, and R is the RTT. When the window advertisement from the receiver clamps the value of W at the sender, the sender's overall rate can be limited so as to not overwhelm the receiver. This approach works fine for protecting the receiver, but what about the network in between? We may have routers with limited memory between the sender and the receiver that have to contend with slow network links. When this happens, it is possible for the sender's rate to exceed a router's ability to keep up, leading to packet loss. This is addressed with a special form of flow control called *congestion control*.

Congestion control involves the sender slowing down so as to not overwhelm the network between itself and the receiver. Recall that in our discussion of flow control, we used a window advertisement to signal the sender to slow down for the receiver. This is called *explicit* signaling, because there is a protocol field specifically used to inform the sender about what is happening. Another option might be for the sender to *guess* that it needs to slow down. Such an approach would involve *implicit* signaling—that is, it would involve deciding to slow down based on some other evidence.

The problem of congestion control in datagram-style networks, and more generally *queuing theory* to which it is closely related, has remained a major research topic for years, and it is unlikely to ever be solved completely for all circumstances. It is also not practical to discuss all the options and methods of performing flow

control here. The interested reader is referred to [J90], [K97], and [K75]. In Chapter 16 we will explore the particular congestion control technique used with TCP in more detail, along with a number of variants that have arisen over the years.

12.1.4 Setting the Retransmission Timeout

One of the most important performance issues the designer of a retransmission-based reliable protocol faces is how long to wait before concluding that a packet has been lost and should be resent. Stated another way, What should the retransmission timeout be? Intuitively, the amount of time the sender should wait before resending a packet is about the sum of the following times: the time to send the packet, the time for the receiver to process it and send an ACK, the time for the ACK to travel back to the sender, and the time for the sender to process the ACK. Unfortunately, in practice, none of these times are known with certainty. To make matters worse, any or all of them vary over time as additional load is added to or removed from the end hosts or routers.

Because it is not practical for the user to tell the protocol implementation what the values of all the times are (or to keep them up-to-date) for all circumstances, a better strategy is to have the protocol implementation try to estimate them. This is called *round-trip-time estimation* and is a statistical process. Basically, the true RTT is likely to be close to the sample mean of a collection of samples of RTTs. Note that this average naturally changes over time (it is not stationary), as the paths taken through the network may change.

Once some estimate of the RTT is made, the question of setting the actual timeout value, used to trigger retransmissions, remains. If we recall the definition of a mean, it can never be the extreme value of a set of samples (unless they are all the same). So, it would not be sensible to set the retransmission timer to be exactly equal to the mean estimator, as it is likely that many actual RTTs will be larger, thereby inducing unwanted retransmissions. Clearly, the timeout should be set to something larger than the mean, but exactly what this relationship is (or even if the mean should be directly used) is not yet clear. Setting the timeout too large is also undesirable, as this leads back to letting the network go idle, reducing throughput. We shall defer further exploration of this topic to Chapter 14, where we explore how TCP, in particular, approaches this problem.

12.2 Introduction to TCP

Given the background we now have regarding the issues affecting reliable delivery in general, let us see how they play out in TCP and what type of service it provides to Internet applications. We also look at the fields in the TCP header, noticing how many of the concepts we have seen so far (e.g., ACKs, window advertisements) are captured in the header description. In the chapters that follow, we examine all of these header fields in more detail.

Our description of TCP starts in this chapter and continues in the next five chapters. Chapter 13 describes how a TCP connection is established and terminated. Chapter 14 details how TCP estimates the per-connection RTT and how the retransmission timeout is set based on this estimate. Chapter 15 looks at the normal transfer of data, starting with “interactive” applications (such as chat). It then covers window management and flow control, which apply to both interactive and “bulk” data flow applications (such as file transfer), along with TCP’s *urgent mechanism*, which allows a sender to mark certain data in the data stream as special. Chapter 16 takes a look at congestion control algorithms in TCP that help to reduce packet loss when the network is very busy. It also discusses some modifications that have been proposed to increase throughput on fast networks or improve resiliency on lossy (e.g., wireless) networks. Finally, Chapter 17 shows how TCP keeps connections active even when no data is flowing.

The original specification for TCP is [RFC0793], although some errors in that RFC are corrected in the Host Requirements RFC, [RFC1122]. Since then, specifications for TCP have been revised and extended to include clarified and improved congestion control behavior [RFC5681][RFC3782][RFC3517][RFC3390][RFC3168], retransmission timeouts [RFC6298][RFC5682][RFC4015], operation with NATs [RFC5382], acknowledgment behavior [RFC2883], security [RFC6056][RFC5927][RFC5926], connection management [RFC5482], and urgent mechanism implementation guidelines [RFC6093]. There have also been a rich variety of experimental modifications covering retransmission behaviors [RFC5827][RFC3708], congestion detection and control [RFC5690][RFC5562][RFC4782][RFC3649][RFC2861], and other features. Finally, there is an effort to explore how TCP might take advantage of multiple simultaneous network-layer paths [RFC6182].

12.2.1 The TCP Service Model

Even though TCP and UDP use the same network layer (IPv4 or IPv6), TCP provides a totally different service to the application layer from what UDP does. TCP provides a *connection-oriented*, reliable, byte stream service. The term *connection-oriented* means that the two applications using TCP must establish a TCP connection by contacting each other before they can exchange data. The typical analogy is dialing a telephone number, waiting for the other party to answer the phone and saying “Hello,” and then saying “Who’s calling?” There are exactly two endpoints communicating with each other on a TCP connection; concepts such as broadcasting and multicasting (see Chapter 9) are not applicable to TCP.

TCP provides a byte stream abstraction to applications that use it. The consequence of this design decision is that no record markers or message boundaries are automatically inserted by TCP (see Chapter 1). A record marker corresponds to an indication of an application’s write extent. If the application on one end writes 10 bytes, followed by a write of 20 bytes, followed by a write of 50 bytes, the application at the other end of the connection cannot tell what size the individual writes were. For example, the other end may read the 80 bytes in four reads of 20 bytes at a time or in some other way. One end puts a stream of bytes into TCP, and

the identical stream of bytes appears at the other end. Each endpoint individually chooses its read and write sizes.

TCP does not interpret the contents of the bytes in the byte stream at all. It has no idea if the data bytes being exchanged are binary data, ASCII characters, EBCDIC characters, or something else. The interpretation of this byte stream is up to the applications on each end of the connection. TCP does, however, support the urgent mechanism mentioned before, although it is no longer recommended for use.

12.2.2 Reliability in TCP

TCP provides reliability using specific variations on the techniques just described. Because it provides a byte stream interface, TCP must convert a sending application's stream of bytes into a set of packets that IP can carry. This is called *packetization*. These packets contain sequence numbers, which in TCP actually represent the byte offsets of the first byte in each packet in the overall data stream rather than packet numbers. This allows packets to be of variable size during a transfer and may also allow them to be combined, called *repacketization*. The application data is broken into what TCP considers the best-size chunks to send, typically fitting each segment into a single IP-layer datagram that will not be fragmented. This is different from UDP, where each write by the application usually generates a UDP datagram of that size (plus headers). The chunk passed by TCP to IP is called a *segment* (see Figure 12-2). In Chapter 15 we shall see how TCP decides what size a segment should be.

TCP maintains a mandatory checksum on its header, any associated application data, and fields from the IP header. This is an end-to-end pseudo-header checksum whose purpose is to detect any bit errors introduced in transit. If a segment arrives with an invalid checksum, TCP discards it without sending any acknowledgment for the discarded packet. The receiving TCP might acknowledge a *previous* (already acknowledged) segment, however, to help the sender with its congestion control computations (see Chapter 16). The TCP checksum uses the same mathematical function as is used by other Internet protocols (UDP, ICMP, etc.). For large data transfers, there is some concern that this checksum is not really strong enough [SP00], so careful applications should apply their own error protection methods (e.g., stronger checksums or CRCs) or use a middleware layer to achieve the same result (e.g., see [RFC5044]).

When TCP sends a group of segments, it normally sets a single retransmission timer, waiting for the other end to acknowledge reception. TCP does not set a different retransmission timer for every segment. Rather, it sets a timer when it sends a window of data and updates the timeout as ACKs arrive. If an acknowledgment is not received in time, a segment is retransmitted. In Chapter 14 we will look at TCP's adaptive timeout and retransmission strategy in more detail.

When TCP receives data from the other end of the connection, it sends an acknowledgment. This acknowledgment may not be sent immediately but is normally delayed a fraction of a second. The ACKs used by TCP are *cumulative* in the sense that an ACK indicating byte number *N* implies that all bytes up to number *N*

(but not including it) have already been received successfully. This provides some robustness against ACK loss—if an ACK is lost, it is very likely that a subsequent ACK is sufficient to ACK the previous segments.

TCP provides a *full-duplex* service to the application layer. This means that data can be flowing in each direction, independent of the other direction. Therefore, each end of a connection must maintain a sequence number of the data flowing in each direction. Once a connection is established, every TCP segment that contains data flowing in one direction of the connection also includes an ACK for segments flowing in the opposite direction. Each segment also contains a window advertisement for implementing flow control in the opposite direction. Thus, when a TCP segment arrives on a connection, the window may slide forward, the window size may change, and new data may have arrived. As we shall see in Chapter 13, a fully active TCP connection is bidirectional and symmetric; data can flow equally well in either direction.

Using sequence numbers, a receiving TCP discards duplicate segments and reorders segments that arrive out of order. Recall that any of these anomalies can happen because TCP uses IP to deliver its segments, and IP does not provide duplicate elimination or guarantee correct ordering. Because it is a byte stream protocol, however, TCP never delivers data to the receiving *application* out of order. Thus, the receiving TCP may be forced to hold on to data with larger sequence numbers before giving it to an application until a missing lower-sequence-numbered segment (a “hole”) is filled in.

We will now begin to look at some of the details of TCP. In this chapter we will only introduce the encapsulation and header structure for TCP. Other details appear in the next five chapters. TCP can be used with IPv4 or IPv6, and the pseudo-header checksum it uses (similar to UDP’s) is mandatory for use with either IPv4 or IPv6.

12.3 TCP Header and Encapsulation

TCP is encapsulated in IP datagrams as shown in Figure 12-2.

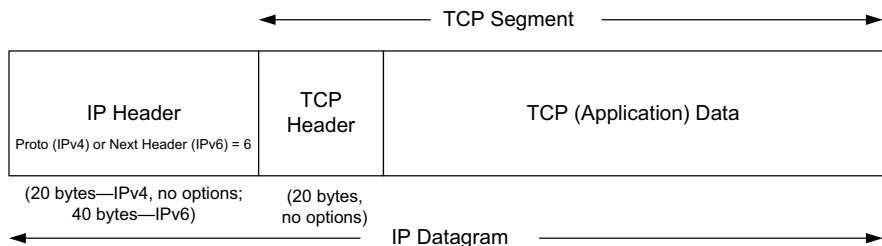


Figure 12-2 The TCP header appears immediately following the IP header or last IPv6 extension header and is often 20 bytes long (with no TCP options). With options, the TCP header can be as large as 60 bytes. Common options include Maximum Segment Size, Time-stamps, Window Scaling, and Selective ACKs.

The header itself is considerably more complicated than the header we saw for UDP in Chapter 10. This is not very surprising, as TCP is a significantly more complicated protocol that must keep each end of the connection informed (synchronized) about the current state. It is shown in Figure 12-3.

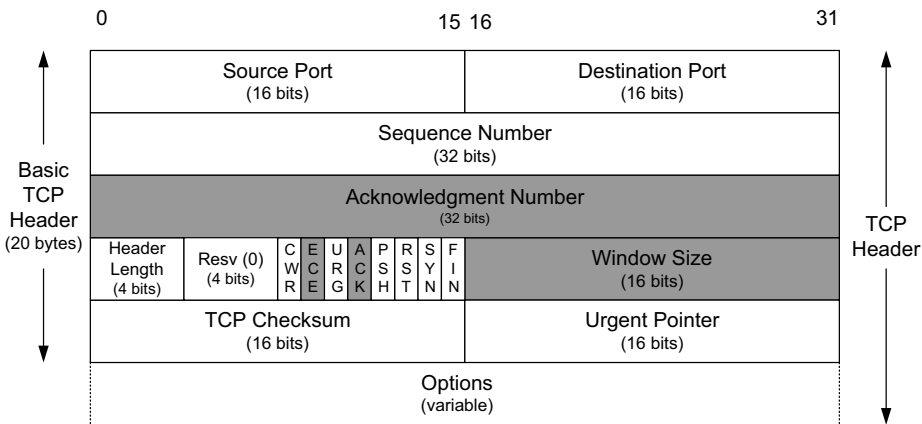


Figure 12-3 The TCP header. Its normal size is 20 bytes, unless options are present. The *Header Length* field gives the size of the header in 32-bit words (minimum value is 5). The shaded fields (*Acknowledgment Number*, *Window Size*, plus *ECE* and *ACK* bits) refer to the data flowing in the opposite direction relative to the sender of this segment.

Each TCP header contains the source and destination port number. These two values, along with the source and destination IP addresses in the IP header, uniquely identify each connection. The combination of an IP address and a port number is sometimes called an *endpoint* or *socket* in the TCP literature. The latter term appeared in [RFC0793] and was ultimately adopted as the name of the Berkeley-derived programming interface for network communications (now frequently called “Berkeley sockets”). It is a *pair* of sockets or endpoints (the 4-tuple consisting of the client IP address, client port number, server IP address, and server port number) that uniquely identifies each TCP connection. This fact will become important when we look at how a TCP server can communicate with multiple clients (see Chapter 13).

The *Sequence Number* field identifies the byte in the stream of data from the sending TCP to the receiving TCP that the first byte of data in the containing segment represents. If we consider the stream of bytes flowing in one direction between two applications, TCP numbers each *byte* with a sequence number. This sequence number is a 32-bit unsigned number that wraps back around to 0 after reaching $(2^{32}) - 1$. Because every byte exchanged is numbered, the *Acknowledgment Number* field (also called the *ACK Number* or *ACK field* for short) contains the next sequence number that the sender of the acknowledgment expects to receive. This is therefore the sequence number of the last successfully received byte of data plus 1. This field is valid only if the *ACK* bit field (described later in this section) is on,

which it usually is for all but initial and closing segments. Sending an ACK costs nothing more than sending any other TCP segment because the 32-bit *ACK Number* field is *always* part of the header, as is the ACK bit field.

When a new connection is being established, the SYN bit field is turned on in the first segment sent from client to server. Such segments are called *SYN segments*, or simply *SYNs*. The *Sequence Number* field then contains the first sequence number to be used on that direction of the connection for subsequent sequence numbers and in returning ACK numbers (recall that connections are all bidirectional). Note that this number is *not* 0 or 1 but instead is another number, often randomly chosen, called the *initial sequence number* (ISN). The reason for the ISN not being 0 or 1 is a security measure and will be discussed in Chapter 13. The sequence number of the first byte of data sent on this direction of the connection is the ISN plus 1 because the SYN bit field *consumes* one sequence number. As we shall see later, consuming a sequence number also implies reliable delivery using retransmission. Thus, SYNs and application bytes (and FINs, which we will see later) are reliably delivered. ACKs, which do not consume sequence numbers, are not.

TCP can be described as “a sliding window protocol with cumulative positive acknowledgments.” The *ACK Number* field is constructed to indicate the largest byte received in order at the receiver (plus 1). For example, if bytes 1–1024 are received OK, and the next segment contains bytes 2049–3072, the receiver cannot use the regular *ACK Number* field to signal the sender that it received this new segment. Modern TCPs, however, have a *selective acknowledgment* (SACK) option that allows the receiver to indicate to the sender out-of-order data it has received correctly. When paired with a TCP sender capable of *selective repeat*, a significant performance benefit may be realized [FF96]. In Chapter 14 we will see how TCP uses *duplicate acknowledgments* (multiple segments with the same ACK field) to help with its congestion control and error control procedures.

The *Header Length* field gives the length of the header in 32-bit words. This is required because the length of the *Options* field is variable. With a 4-bit field, TCP is limited to a 60-byte header. Without options, however, the size is 20 bytes.

Currently eight bit fields are defined for the TCP header, although some older implementations understand only the last six of them.¹ One or more of them can be turned on at the same time. We briefly mention their use here and discuss each of them in more detail in later chapters.

1. **CWR**—Congestion Window Reduced (the sender reduced its sending rate); see Chapter 16.
2. **ECE**—ECN Echo (the sender received an earlier congestion notification); see Chapter 16.
3. **URG**—Urgent (the *Urgent Pointer* field is valid—rarely used); see Chapter 15.

1. Note that [RFC3540], an experimental RFC, also defines the least significant of the Resv bits as a nonce sum (NS). See Section 16.12.

4. **ACK**—Acknowledgment (the *Acknowledgment Number* field is valid—always on after a connection is established); see Chapters 13 and 15.
5. **PSH**—Push (the receiver should pass this data to the application as soon as possible—not reliably implemented or used); see Chapter 15.
6. **RST**—Reset the connection (connection abort, usually because of an error); see Chapter 13.
7. **SYN**—Synchronize sequence numbers to initiate a connection; see Chapter 13.
8. **FIN**—The sender of the segment is finished sending data to its peer; see Chapter 13.

TCP's flow control is provided by each end advertising a window size using the *Window Size* field. This is the number of bytes, starting with the one specified by the ACK number, that the receiver is willing to accept. This is a 16-bit field, limiting the window to 65,535 bytes, and thereby limiting TCP's throughput performance. In Chapter 15 we will look at the Window Scale option that allows this value to be scaled, providing much larger windows and improved performance for high-speed and long-delay networks.

The *TCP Checksum* field covers the TCP header and data and some fields in the IP header, using a pseudo-header computation similar to the one used with ICMPv6 and UDP that we discussed in Chapters 8 and 10. It is mandatory for this field to be calculated and stored by the sender, and then verified by the receiver. The TCP checksum is calculated with the same algorithm as the IP, ICMP, and UDP ("Internet") checksums.

The *Urgent Pointer* field is valid only if the *URG* bit field is set. This "pointer" is a positive offset that must be added to the *Sequence Number* field of the segment to yield the sequence number of the *last* byte of urgent data. TCP's urgent mechanism is a way for the sender to provide specially marked data to the other end.

The most common *Option* field is the Maximum Segment Size option, called the MSS. Each end of a connection normally specifies this option on the first segment it sends (the ones with the *SYN* bit field set to establish the connection). The MSS option specifies the maximum-size segment that the sender of the option is willing to receive in the reverse direction. We describe the MSS option in more detail in Chapter 13 and some of the other TCP options in Chapters 14 and 15. Other common options we investigate include SACK, Timestamp, and Window Scale.

In Figure 12-2 we note that the data portion of the TCP segment is optional. We will see in Chapter 13 that when a connection is established, and when a connection is terminated, segments are exchanged that contain only the TCP header (with or without options) but no data. A header without any data is also used to acknowledge received data, if there is no data to be transmitted in that direction (called a *pure ACK*), and to notify the communication peer of a change in the window size (called a *window update*). There are also some cases resulting from timeouts when a segment can be sent without any data.

13

TCP Connection Management

13.1 Introduction

TCP is a unicast *connection-oriented* protocol. Before either end can send data to the other, a connection must be established between them. In this chapter, we take a detailed look at what a TCP connection is, how it is established, and how it is terminated. Recall that TCP's service model is a byte stream. TCP detects and repairs essentially all the data transfer problems that may be introduced by packet loss, duplication, or errors at the IP layer (or below).

Because of its management of *connection state* (information about the connection kept by both endpoints), TCP is a considerably more complicated protocol than UDP (see Chapter 10). UDP is a *connectionless* protocol that involves no connection establishment or termination. One of the major differences we shall see between the two is the amount of detail required to handle the various TCP states properly: when connections are created, terminated normally, and reset without warning. In other chapters we will look at what happens once the connection is established and data is transferred.

During connection establishment, several *options* can be exchanged between the two endpoints regarding the parameters of the connection. Some options are allowed to be sent only when the connection is established, and others can be sent later. Recall from Chapter 12 that the TCP header has a limited space for holding options (40 bytes).

13.2 TCP Connection Establishment and Termination

A TCP *connection* is defined to be a 4-tuple consisting of two IP addresses and two port numbers. More precisely, it is a pair of *endpoints* or *sockets* where each endpoint is identified by an (IP address, port number) pair.

A connection typically goes through three phases: setup, data transfer (called *established*), and teardown (closing). As we will see, some of the difficulty in creating a robust TCP implementation is handling all of the transitions between and among these phases correctly. A typical TCP connection establishment and close (without any data transfer) is shown in Figure 13-1.

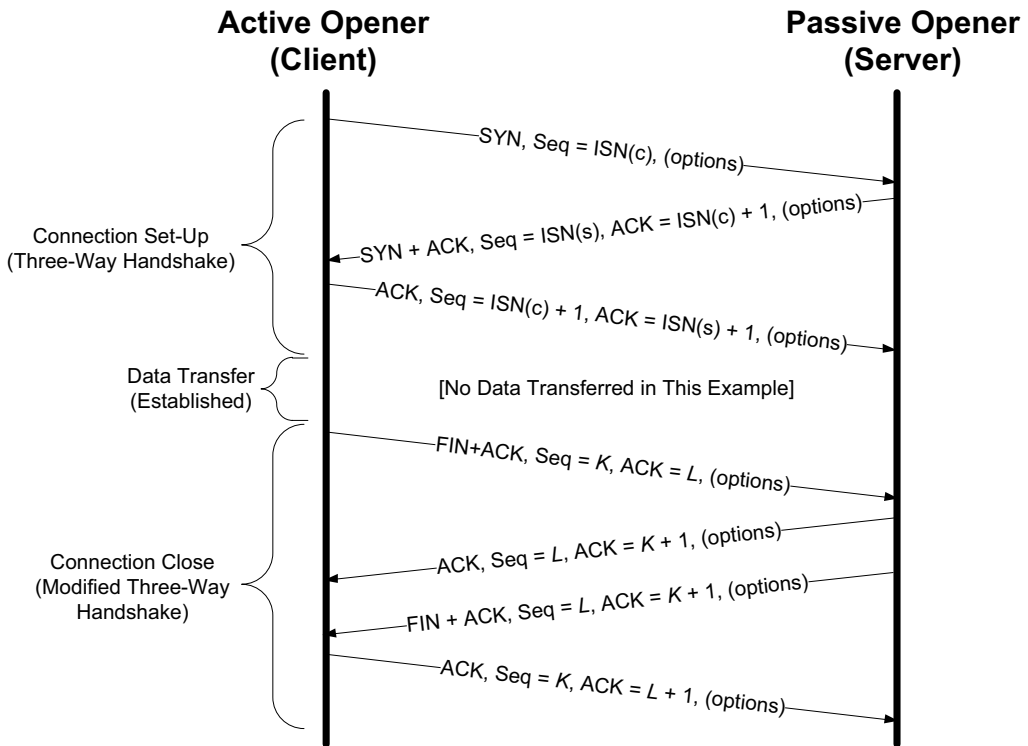


Figure 13-1 A normal TCP connection establishment and termination. Usually, the client initiates a three-way handshake to exchange initial sequence numbers carried on SYN segments for the client and server (ISN(c) and ISN(s), respectively). The connection terminates after each side has sent a FIN and received an acknowledgment for it.

The figure shows a timeline of what happens during connection establishment. To establish a TCP connection, the following events usually take place:

1. The *active opener* (normally called the client) sends a SYN segment (i.e., a TCP/IP packet with the SYN bit field turned on in the TCP header) specifying the port number of the peer to which it wants to connect and the client's

initial sequence number or ISN(c) (see Section 13.2.3). It typically sends one or more options at this point (see Section 13.3). This is segment 1.

2. The server responds with its own SYN segment containing its initial sequence number (ISN(s)). This is segment 2. The server also acknowledges the client's SYN by ACKing ISN(c) plus 1. A SYN consumes one sequence number and is retransmitted if lost.
3. The client must acknowledge this SYN from the server by ACKing ISN(s) plus 1. This is segment 3.

These three segments complete the connection establishment. This is often called the *three-way handshake*. Its main purposes are to let each end of the connection know that a connection is starting and the special details that are carried as options, and to exchange the ISNs.

The side that sends the first SYN is said to perform an *active open*. As mentioned, this is typically a client. The other side, which receives this SYN and sends the next SYN, performs a *passive open*. It is most commonly called the server. (In Section 13.2.2 we describe a supported but unusual *simultaneous open* when both sides can do an active open at the same time and become both clients and servers.)

Note

TCP supports the capability of carrying application data on SYN segments. This is rarely used, however, because the Berkeley sockets API does not support it.

Figure 13-1 also shows how a TCP connection is closed (also called cleared or terminated). Either end can initiate a close operation, and simultaneous closes are also supported but are rare. Traditionally, it was most common for the client to initiate a close (as shown in Figure 13-1). However, other servers (e.g., Web servers) initiate a close after they have completed a request. Usually a close operation starts with an application indicating its desire to terminate its connection (e.g., using the `close()` system call). The closing TCP initiates the close operation by sending a FIN segment (i.e., a TCP segment with the *FIN* bit field set). The complete close operation occurs after both sides have completed the close:

1. The *active closer* sends a FIN segment specifying the current sequence number the receiver expects to see (K in Figure 13-1). The FIN also includes an ACK for the last data sent in the other direction (labeled L in Figure 13-1).
2. The *passive closer* responds by ACKing value $K + 1$ to indicate its successful receipt of the active closer's FIN. At this point, the application is notified that the other end of its connection has performed a close. Typically this results in the application initiating its own close operation. The passive closer then effectively becomes another active closer and sends its own FIN. The sequence number is equal to L .

3. To complete the close, the final segment contains an ACK for the last FIN. Note that if a FIN is lost, it is retransmitted until an ACK for it is received.

While it takes three segments to establish a connection, it takes four to terminate one. It is also possible for the connection to be in a half-open state (see Section 13.6.3), although this is not common. This reason is that TCP's data communications model is bidirectional, meaning it is possible to have only one of the two directions operating. The *half-close* operation in TCP closes only a single direction of the data flow. Two half-close operations together close the entire connection. The rule is that either end can send a FIN when it is done sending data. When a TCP receives a FIN, it must notify the application that the other end has terminated that direction of data flow. The sending of a FIN is normally the result of the application issuing a close operation, which typically causes both directions to close.

The seven segments we have seen are baseline overheads for any TCP connection that is established and cleared "gracefully." (There are more abrupt ways to tear down a TCP connection using special reset segments, which we cover later.) When a small amount of data needs to be exchanged, it is now apparent why some applications prefer to use UDP because of its ability to send and receive data without establishing connections. However, such applications are then faced with handling their own error repair features, congestion management, and flow control.

13.2.1 TCP Half-Close

As we have mentioned, TCP supports a half-close operation. Few applications require this capability, so it is not common. To use this feature, the API must provide a way for the application to say, essentially, "I am done sending data, so send a FIN to the other end, but I still want to receive data from the other end, until it sends me a FIN." The Berkeley sockets API supports half-close, if the application calls the `shutdown()` function instead of calling the more typical `close()` function. Most applications, however, terminate both directions of the connection by calling `close`. Figure 13-2 shows an example of a half-close being used. We show the client on the left side initiating the half-close, but either end can do this.

The first two segments are the same as for a regular close: a FIN by the initiator, followed by an ACK of the FIN by the recipient. The operation then differs from Figure 13-1, because the side that receives the half-close can still send data. We show only one data segment, followed by an ACK, but any number of data segments can be sent. (We talk more about the exchange of data segments and acknowledgments in Chapter 15.) When the end that received the half-close is done sending data, it closes its end of the connection, causing a FIN to be sent, and this delivers an end-of-file indication to the application that initiated the half-close. When this second FIN is acknowledged, the connection is completely closed.

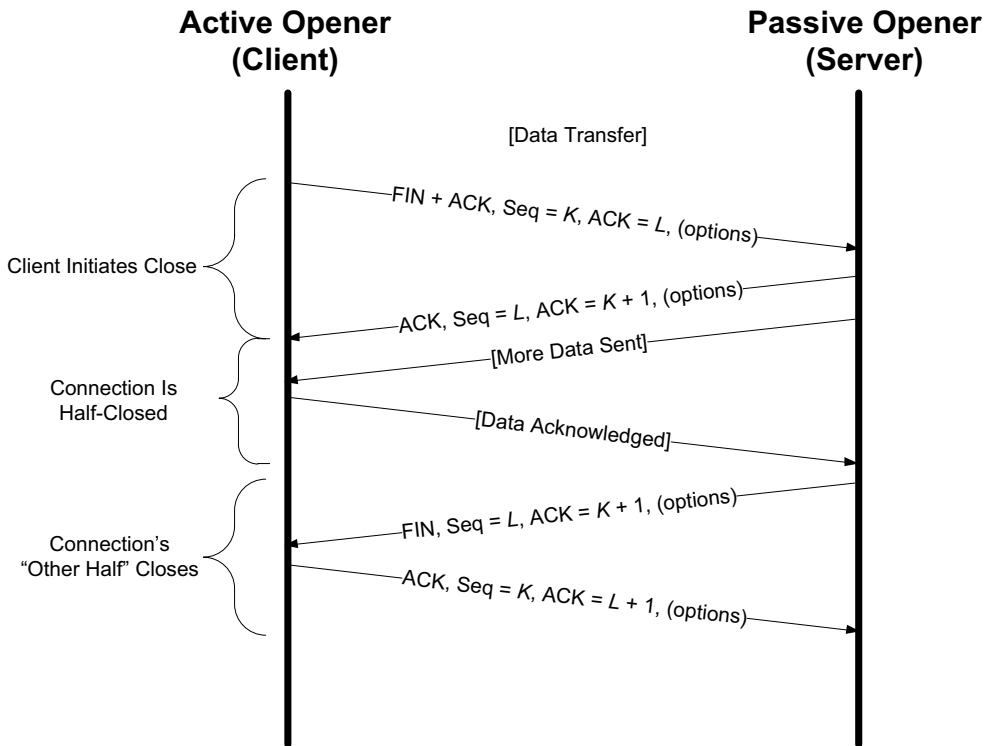


Figure 13-2 With the TCP half-close operation, one direction of the connection can terminate while the other continues until it is closed. Few applications use this feature.

13.2.2 Simultaneous Open and Close

It is possible, although highly improbable unless specifically arranged, for two applications to perform an active open to each other at the same time. Each end must have transmitted a SYN before receiving a SYN from the other side; the SYNs must pass each other on the network. This scenario also requires each end to have an IP address and port number that are known to the other end, which is rare (except for the firewall “hole-punching” techniques we saw in Chapter 7). If this happens, it is called a *simultaneous open*.

For example, a simultaneous open occurs when an application on host A using local port 7777 performs an active open to port 8888 on host B, while at the same time an application on host B using local port 8888 performs an active open to port 7777 on host A. This is *not* the same as connecting a client on host A to a server on host B, while at the same time having a client on host B connect to a conventional server on host A. In that case, both servers perform passive opens, not active opens, and the clients assign themselves different ephemeral port numbers. This results in two distinct TCP connections. Figure 13-3 shows the segments exchanged during a simultaneous open.

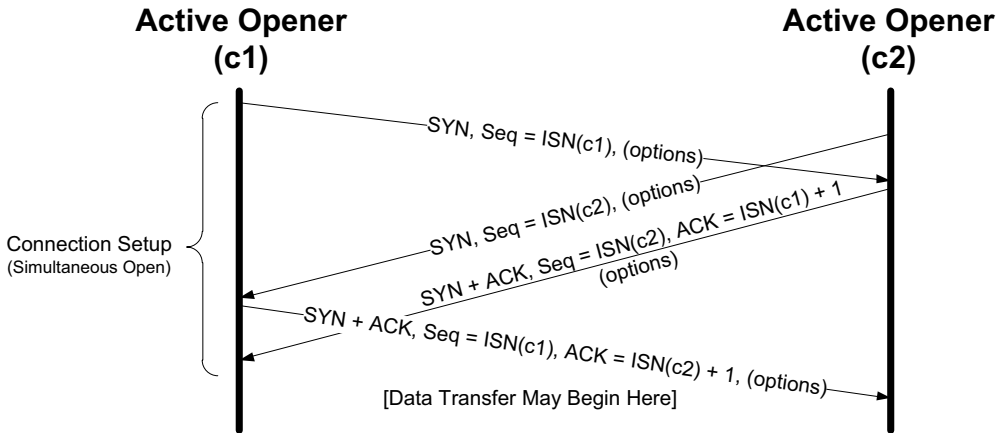


Figure 13-3 Segments exchanged during simultaneous open. One additional segment is required compared to the ordinary connection establishment procedure. The *SYN* bit field is on in each segment until an ACK for it is received.

A simultaneous open requires the exchange of four segments, one more than the normal three-way handshake. Also note that we do not call either end a client or a server, because both ends act as client and server. A *simultaneous close* is not very different. We said earlier that one side (often, but not always, the client) performs the active close, causing the first FIN to be sent. In a simultaneous close, both do. Figure 13-4 shows the segments exchanged during a simultaneous close.

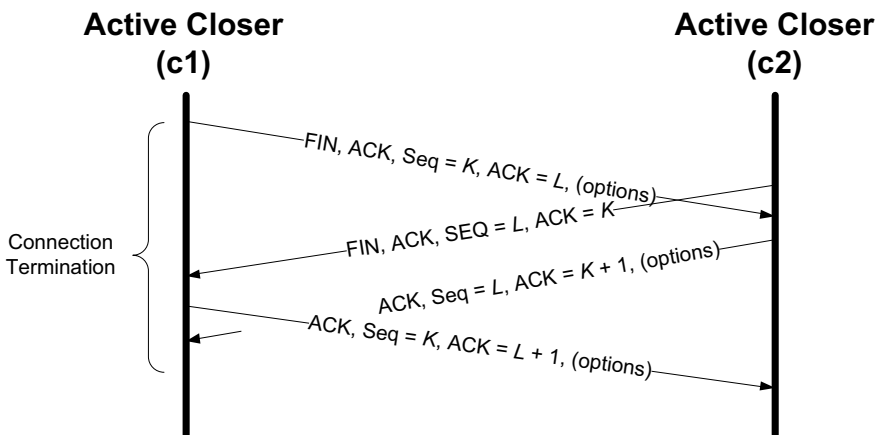


Figure 13-4 Segments exchanged during simultaneous close work like a conventional close, but the segment ordering is interleaved.

With a simultaneous close the same number of segments are exchanged as in the normal close. The only real difference is that the segment sequence is interleaved instead of sequential. Later we will see that simultaneous open and close operations use particular states in the TCP implementation that are not commonly exercised.

13.2.3 Initial Sequence Number (ISN)

When a connection is open, any segment with the appropriate two IP addresses and port numbers is accepted as valid provided the sequence number is valid (i.e., within the window) and the checksum is OK. This brings up the question of whether it might be possible to have TCP segments being routed through the network that could show up later and disrupt a connection. This concern is addressed by careful selection of the ISN, which we now investigate.

Before each end sends its SYN to establish the connection, it chooses an ISN for that connection. The ISN should change over time, so that each connection has a different one. [RFC0793] specifies that the ISN should be viewed as a 32-bit counter that increments by 1 every 4 μ s. The purpose of doing this is to arrange for the sequence numbers for segments on one connection to not overlap with sequence numbers on another (new) identical connection. In particular, new sequence numbers must not be allowed to overlap between different *instantiations* (or *incarnations*) of the *same* connection.

The idea of different instantiations of the same connection becomes clear when we recall that a TCP connection is identified by a pair of endpoints, creating a 4-tuple of two address/port pairs. If a connection had one of its segments delayed for a long period of time and closed, but then opened again with the same 4-tuple, it is conceivable that the delayed segment could reenter the new connection's data stream as valid data. This would be most troublesome. By taking steps to avoid overlap in sequence numbers between connection instantiations, we can try to minimize this risk. It does suggest, however, that an application with a very great need for data integrity should employ its own CRCs or checksums at the application layer to ensure that its own data has been transferred without error. This is generally good practice in any case, and it is commonly done for large files.

As we shall see, knowing the connection 4-tuple as well as the currently active window of sequence numbers is all that is required to form a TCP segment that is considered valid to a communicating TCP endpoint. This represents a form of vulnerability for TCP: anyone can forge a TCP segment and, if the sequence numbers, IP addresses, and port numbers are chosen appropriately, can interrupt a TCP connection [RFC5961]. One way of repelling this is to make the initial sequence number (or ephemeral port number [RFC6056]) relatively hard to guess. Another is encryption (see Chapter 18).

In modern systems, the ISN is typically selected in a semirandom way. An interesting discussion of the subtleties of doing this properly is contained in CERT Advisory CA-2001-09 [CERTISN]. Linux goes through a fairly elaborate process to

select its ISNs. It uses a clock-based scheme but starts the clock at a random offset for each connection. The random offset is chosen as a cryptographically hashed function on the connection identifier (4-tuple). A secret input to the hash function changes every 5 minutes. Of the 32 bits in the ISN, the top-most 8 bits are a sequence number of the secret, and the remaining bits are generated by the hash. This produces an ISN that is difficult to guess, but also one that increases over time. Windows reportedly uses a similar scheme based on RC4 [S96].

13.2.4 Example

Now that we have a basic idea of how a TCP connection is established and cleared, let us look at the packet-level details. To do so we make a TCP connection to a nearby Web server running on the machine with IPv4 address 10.0.0.2. The client is the Telnet application on Windows:

```
C:\> telnet 10.0.0.2 80
Welcome to Microsoft Telnet Client
Escape Character is 'CTRL+]'
... wait about 4.4 seconds ...
Microsoft Telnet> quit
```

The `telnet` command establishes a TCP connection with the host having IPv4 address 10.0.0.2 on the port corresponding to the `http` or Web service (port 80). When the Telnet program connects to a port other than 23 (the well-known port for the Telnet protocol [RFC0854]), it does not engage in the application protocol. Instead, it merely copies bytes from its input to its TCP connection and vice versa. When a Web server receives the incoming connection request, the first thing it does is await a request for a Web page. In this case, we do not provide one, so the server does not produce any data. This is ideal for us, because for now we are interested only in the connection establishment and termination packet exchange. Figure 13-5 shows the Wireshark output for the segments generated by this command.

In the figure, we can see that the client begins with a SYN segment containing an ISN of 685506836 and window advertisement of 65535. This segment also contains several options we discuss in Section 13.3. The second segment is both a SYN from the server and an ACK for the client. The sequence number (server's ISN) is 1479690171 and the ACK number is 685506837, 1 more than the client's ISN. This indicates successful receipt of the client's ISN. This segment also includes a window advertisement indicating that the server is willing to accept up to 64,240 bytes. Completion of the three-way handshake takes place with segment 3, which contains ACK number 1479690172. Remember that ACK numbers are cumulative and always indicate the sequence number the sender of the ACK expects to see *next* (not the one that it last received).

After a pause of about 4.4s, the Telnet application is instructed to close the connection. This results in the client's TCP sending the FIN in segment 4. The sequence number of the FIN is 685506837, which is ACKed in segment 5 (with

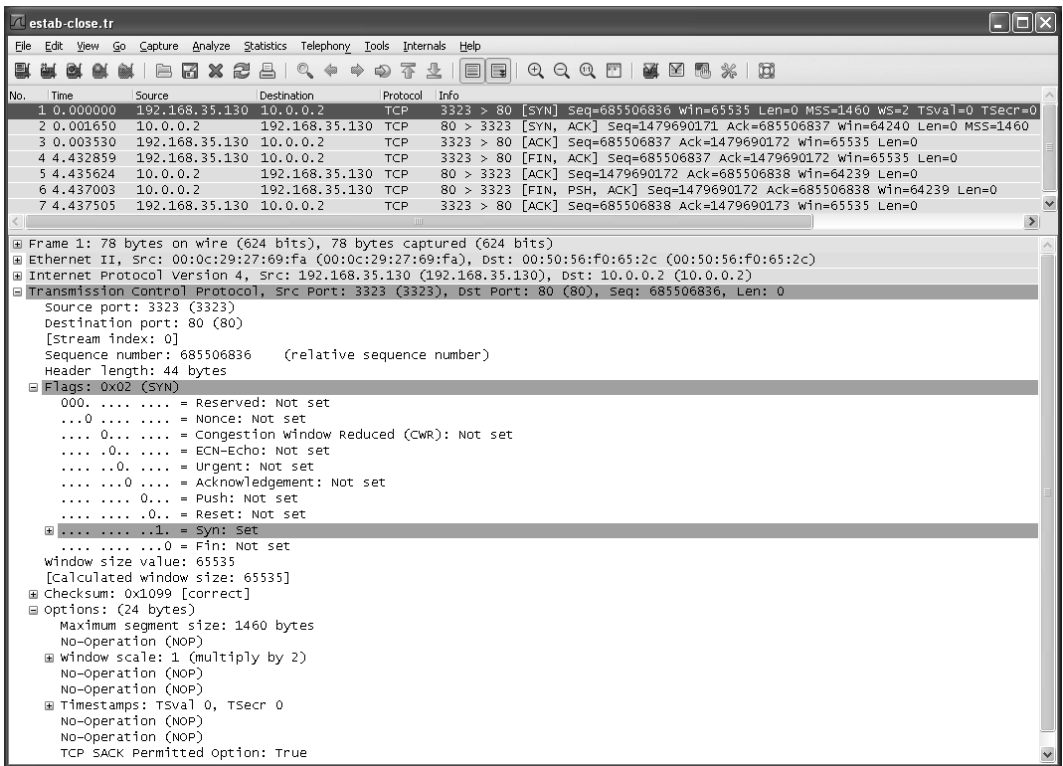


Figure 13-5 A TCP connection between 192.168.35.130 and 10.0.0.2 is established and cleared without sending any data. The *PSH* (*Push*) bit indicates that segment 6 is sending all data from its buffer (which is none).

ACK number 685506838). Shortly thereafter the server sends its own FIN with sequence number 1479690172. This segment also (redundantly) ACKs the client's FIN once again. Note that the *PSH* bit field is on. This has no real effect on the closing of the connection but usually indicates that the server has no additional data to send. The final segment ACKs the server's FIN by including ACK number 1479690173.

Note

[RFC1025] calls a segment with the maximum number of features enabled (e.g., flags and options) a “Kamikaze” packet. Other colorful terms include “nastygram,” “Christmas tree packet,” and “lamp test segment.”

One thing we can see in Figure 13-5 is that the SYN segments contain one or more options. These take up additional space in the TCP header. For example, the length of the first TCP header is 44 bytes, 24 bytes greater than the minimum size. TCP

has several supported options, which we detail after we see what happens when a connection cannot be established.

13.2.5 Timeout of Connection Establishment

There are several circumstances in which a connection cannot be established. One obvious case is when the server host is down. To simulate this scenario, we issue our `telnet` command to a nonexistent host in the same subnet. If we do this without modifying the ARP table, the client exits with a “No route to host” error message, generated because no ARP reply is ever returned for the ARP request (see Chapter 4). If, however, we place an ARP entry for a nonexistent host in the ARP table first, the ARP request is not sent, and the system immediately attempts to contact the nonexistent host with TCP/IP. First, the commands:

```
Linux# arp -s 192.168.10.180 00:00:1a:1b:1c:1d
Linux% date; telnet 192.168.10.180 80; date
Tue Jun  7 21:16:34 PDT 2009
Trying 192.168.10.180...
telnet: connect to address 192.168.10.180: Connection timed out
Tue Jun  7 21:19:43 PDT 2009
Linux%
```

Here the MAC address `00:00:1a:1b:1c:1d` was chosen simply as a MAC address not being used on the LAN; it is of no special consequence. The timeout occurs about 3.2 minutes after the initial command. Because there is no host to respond, all of the segments generated are from the client. Listing 13-1 shows the output using Wireshark in packet summary (text) mode.

Listing 13-1 Wireshark output for connection establishment that times out

No.	Time	Source	Destination	Protocol	Info
1	0.000000	192.168.10.144	192.168.10.180	TCP	32787 > http
2	2.997928	192.168.10.144	192.168.10.180	TCP	32787 > http
3	8.997962	192.168.10.144	192.168.10.180	TCP	32787 > http
4	20.997942	192.168.10.144	192.168.10.180	TCP	32787 > http
5	44.997936	192.168.10.144	192.168.10.180	TCP	32787 > http
6	92.997937	192.168.10.144	192.168.10.180	TCP	32787 > http

The interesting point in this output is how frequently the client’s TCP sends a SYN to try to establish the connection. The second segment is sent 3s after the first, the third is sent 6s after the second, the fourth is sent 12s after the third, and so on. This behavior is called *exponential backoff*, and we saw something like it before when we discussed the behavior of Ethernet’s CSMA/CD media access control protocol (see Chapter 3). In that case, it was a little different, however, because here each backoff is deterministically (i.e., always) twice the previous backoff, whereas in Ethernet, the *maximum* backoff is doubled and the actual backoff is chosen randomly.

The number of times to retry an initial SYN can be configured on some systems and usually has a fairly small value such as 5. In Linux, the system configuration variable `net.ipv4.tcp_syn_retries` gives the maximum number of times to attempt to resend a SYN segment during an active open. A corresponding value called `net.ipv4.tcp_synack_retries` gives the maximum number of times to attempt to resend a SYN + ACK segment when responding to a peer's active open request. It can also be used on an individual connection basis by setting the Linux-specific `TCP_SYNCNT` socket option. Its default value is five retries, as we see here. The exponential backoff timing between these retransmissions is part of TCP's congestion management response. We shall examine it in detail when we discuss Karn's algorithm (see Chapter 16).

13.2.6 Connections and Translators

In Chapter 7 we discussed how conventional NAT translates the addresses and port numbers used by protocols such as TCP and UDP. We also examined how IP packets can be translated between IPv6 and IPv4. When NAT is used with TCP, the pseudo-header checksum usually requires adjustment (except in cases where a checksum-neutral address modifier is used). This is also true for other protocols that use pseudo-header checksums, because the computation involves information at the transport layer as well as the network layer.

When a TCP connection is first established, a NAT (or NAT64) can ascertain this fact because of the presence of the *SYN* bit field in a segment. It can also determine when a connection has become fully established by looking for subsequent SYN + ACK and ACK segments containing the appropriate sequence numbers. The same applies for the termination of a connection. By implementing a portion of the TCP state machine in a NAT (see, for example, Sections 3.5.2.1 and 3.5.2.2 of [RFC6146]), the connection can be tracked, including the current states, sequence numbers in each direction, and corresponding ACK numbers. Such state tracking is typical for NAT implementations.

Further complications arise when a NAT acts as an editor and rewrites contents in the transport protocol's data payload. For TCP, this may involve removing or adding bytes to the data stream, and consequently affecting the sequence numbers (and segment) lengths. Doing so also necessarily affects the checksum, but it also affects the data sequence. If data is inserted or removed from the data stream by the NAT, these values can be adjusted appropriately. Doing so is somewhat fragile because if the NAT state becomes desynchronized with the state in the end hosts, the connection will not operate properly.

13.3 TCP Options

The TCP header can contain options (see Figure 12-3). The only options defined in the original TCP specification are the *End of Option List* (EOL), the *No Operation* (NOP), and the *Maximum Segment Size* (MSS) options. Since then, several options have been

Table 13-1 The TCP option values. Up to 40 bytes are available to hold options.

Kind	Length	Name	Reference	Description and Purpose
0	1	EOL	[RFC0793]	End of Option List
1	1	NOP	[RFC0793]	No Operation (used for padding)
2	4	MSS	[RFC0793]	Maximum Segment Size
3	3	WSOPT	[RFC1323]	Window Scaling Factor (left-shift amount on window)
4	2	SACK-Permitted	[RFC2018]	Sender supports SACK options
5	Var.	SACK	[RFC2018]	SACK block (out-of-order data received)
8	10	TSOPT	[RFC1323]	Timestamps option
28	4	UTO	[RFC5482]	User Timeout (abort after idle time)
29	Var.	TCP-AO	[RFC5925]	Authentication option (using various algorithms)
253	Var.	Experimental	[RFC4727]	Reserved for experimental use
254	Var.	Experimental	[RFC4727]	Reserved for experimental use

defined. The entire list is maintained by the IANA [TPARAMS]; Table 13-1 gives the current options of interest (i.e., those with standards-track RFC descriptions).

Every option begins with a 1-byte *kind* that specifies the type of option. Options that are not understood are simply ignored, according to [RFC1122]. The options with a *kind* value of 0 and 1 occupy a single byte. The other options have a *len* byte that follows the *kind* byte. The length is the total length, including the *kind* and *len* bytes. The reason for the NOP option is to allow the sender to pad fields to a multiple of 4 bytes, if it needs to. Remember that the TCP header's length is always required to be a multiple of 32 bits because the TCP *Header Length* field uses that unit. The EOL option indicates the end of the list and that no further processing of the options list is to be performed. Now we will have a look at the other options.

13.3.1 Maximum Segment Size (MSS) Option

The maximum segment size (MSS) is the largest segment that a TCP is willing to receive from its peer and, consequently, the largest size its peer should ever use when sending. The MSS value counts only TCP data bytes and does not include the sizes of any associated TCP or IP header [RFC0879]. When a connection is established, each end usually announces its MSS in an MSS option carried with its SYN segment. The option allows for 16 bits to be used to specify the MSS value. If no MSS option is provided, a default value of 536 bytes is used. Recall the rule that requires any host to be capable of processing IPv4 datagrams at least as large as 576. With minimum-size IPv4 and TCP headers, a TCP using a sending MSS size of 536 bytes produces an IPv4 datagram of size $20 + 20 + 536 = 576$ bytes.

The MSS values in Figure 13-5 are all 1460, which is typical for IPv4. The resulting IPv4 datagram is normally 40 bytes larger (1500 bytes total, the typical

MTU size for Ethernet and path MTU for the Internet): 20 bytes for the TCP header and 20 bytes for the IPv4 header. When IPv6 is used, the MSS is usually 1440, 20 bytes less because of the larger IPv6 header. The special MSS value of 65535 can be used with IPv6 jumbograms to indicate an effective MSS of infinity [RFC2675]. In this case the SMSS will be determined as the PMTU minus 60 bytes (40 bytes for the IPv6 header and 20 bytes for the TCP header). Note that the MSS option is not a negotiation between one TCP and its peer; it is a limit. When one TCP gives its MSS option to the other, it is indicating its unwillingness to accept any segments larger than that size for the duration of the connection.

13.3.2 Selective Acknowledgment (SACK) Options

In Chapter 12 we introduced the concept of a sliding window, and we described how TCP handles its sequence numbers and acknowledgments. Because it uses cumulative ACKs, TCP is never able to acknowledge data it has received correctly but that is not contiguous, in terms of sequence numbers, with data it has received previously. In such cases, the TCP receiver is said to have *holes* in its received data queue. A receiving TCP prevents applications from consuming data beyond a hole because of the byte stream abstraction it provides.

If a TCP sender were able to learn of the existence of holes (and out-of-sequence data blocks beyond holes in the sequence space) at the receiver, it could better select which particular TCP segments to retransmit when segments are lost or otherwise missing at the receiver. The TCP selective acknowledgment (SACK) options [RFC2018][RFC2883] provide this capability. The scheme works effectively, however, only if the TCP sender logic is able to make effective use of the SACK information it receives from a SACK-capable receiver.

A TCP learns that its peer is capable of advertising SACK information by receiving the SACK-Permitted option in a SYN (or SYN + ACK) segment. Once this has taken place, the TCP receiving out-of-sequence data may provide a SACK option that describes the out-of-sequence data to help its peer perform retransmissions more efficiently. SACK information contained in a SACK option consists of a range of sequence numbers representing data blocks the receiver has successfully received. Each range is called a *SACK block* and is represented by a pair of 32-bit sequence numbers. Thus, a SACK option containing n SACK blocks is $(8n + 2)$ bytes long. Two bytes are used to hold the kind and length of the SACK option.

Because of the limited amount of space available in the option space of a TCP header, the maximum number of SACK blocks available to be sent in a single segment is three (assuming the Timestamps option is also used, described in Section 13.3.4, which is typical for modern TCP implementations). Although the SACK-Permitted option is only ever sent in a SYN segment, the SACK blocks themselves may be sent in any segment once the sender has sent the SACK-Permitted option. Because the operation of SACK is most easily (and importantly) related to the error and congestion control operations of TCP, we discuss it in further detail when we cover these topics in Chapters 14 and 16.

13.3.3 Window Scale (WSALE or WSOPT) Option

The *Window Scale* option (denoted WSALE or WSOPT) [RFC1323] effectively increases the capacity of the TCP *Window Advertisement* field from 16 to about 30 bits. Instead of changing the field size, however, the header still holds a 16-bit value, and an option is defined that applies a *scaling factor* to the 16-bit value. This factor effectively left-shifts the window field value by the scale factor. This, in effect, multiplies the window value by the value 2^s , where s is the scale factor. The 1-byte shift count is between 0 and 14 (inclusive). A shift count of 0 indicates no scaling. The maximum scale value of 14 provides for a maximum window of 1,073,725,440 bytes ($65,535 \times 2^{14}$), close to 1,073,741,823 ($2^{30} - 1$), effectively 1GB. TCP then maintains the “real” window size internally as a 32-bit value.

This option can appear only in a SYN segment, so the scale factor is fixed in each direction when the connection is established. To enable window scaling, both ends must send the option in their SYN segments. The end doing the active open sends the option in its SYN, but the end doing the passive open can send the option only if the received SYN specifies the option. The scale factor can be different in each direction. If the end doing the active open sends a nonzero scale factor but does not receive a Window Scale option from the other end, it sets its send and receive scale values to 0. This lets systems that do not understand the option interoperate with systems that do.

Assume we are using the Window Scale option, with a shift count of S for sending and a shift count of R for receiving. Then every 16-bit advertised window that we receive from the other end is left-shifted by R bits to obtain the real advertised window size. Every time we send a window advertisement to the other end, we take our real 32-bit window size and right-shift it S bits, placing the resulting 16-bit value in the TCP header.

The shift count is automatically chosen by TCP, based on the size of the receive buffer. The size of this buffer is set by the system, but the capability is normally provided for the application to change it. The Window Scale option is most relevant when TCP is used to provide bulk data transfer over networks with large-bandwidth-delay products (i.e., those with a product of round-trip time and bandwidth being relatively large). Thus, we shall discuss the importance and use of this option more in Chapter 16.

13.3.4 Timestamps Option and Protection against Wrapped Sequence Numbers (PAWS)

The *Timestamps* option (sometimes called the *Timestamp* option and written as TSOPT or TSOpt) lets the sender place two 4-byte timestamp values in every segment. The receiver reflects these values in the acknowledgment, allowing the sender to calculate an estimate of the connection's RTT for each ACK received. (We must say “each ACK received” and not “each segment” because TCP often acknowledges multiple segments per ACK; we will see this in Chapter 15.) When using the Timestamps option, the sender places a 32-bit value in the Timestamp

Value field (called TSV or TSval) in the first part of the TSOPT, and the receiver echoes this back unchanged in the second *Timestamp Echo Retry* field (called TSER or TSecr). TCP headers containing this option increase by 10 bytes (8 bytes for the two timestamp values and 2 to indicate the option value and length).

The timestamp is a monotonically increasing value. Because the receiver simply echoes what it receives, it does not care what the timestamp units or values actually are. This option does not require any form of clock synchronization between the two hosts. [RFC1323] recommends that the sender increment the timestamp value by at least 1 every second. Figure 13-6 shows the Timestamps option, as displayed by Wireshark.

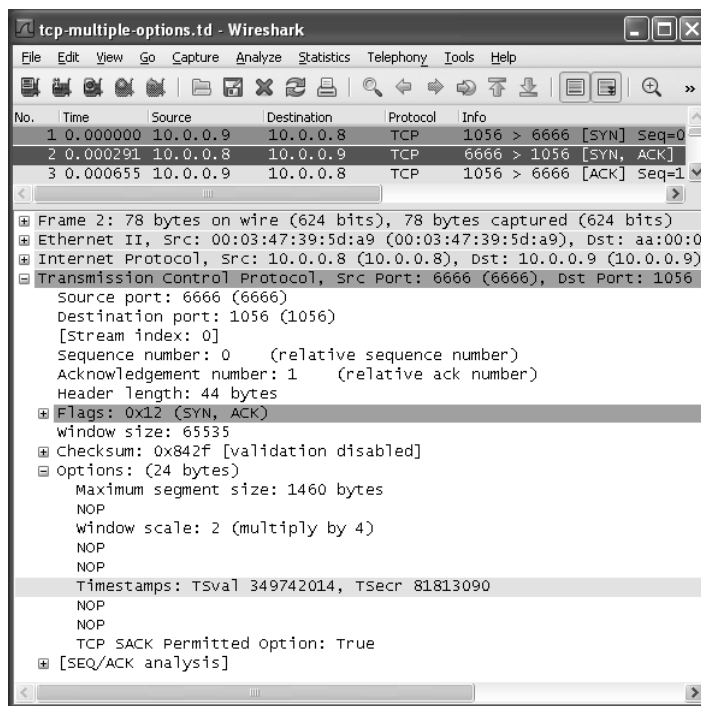


Figure 13-6 A TCP connection with the Timestamps, Window Scaling, and MSS options being used. The TCP header is 44 bytes long. The initial SYN (packet 1) starts with the TSV set to 81813090. The second packet, highlighted, echoes this value back to the active opener and includes its own value of 349742014.

Here, both ends participate by generating and echoing back the other's timestamps. The first segment (client's SYN) uses an initial timestamp value of 81813090. This value is placed in the TSV. The second portion, TSER, has a value of 0 on the first segment because the client does not know the server's timestamp value yet.

The main reason for wishing to calculate a good estimate of the connection's RTT is to set the retransmission timeout, which tells TCP when it should try resending a segment that is likely lost. In Chapter 12 we discussed the need to set this timeout based on some function of the RTT. With the Timestamps option, we can get relatively fine-grain measurements of the RTT. Prior to the creation of the Timestamps option, most TCPs would perform just one RTT sample per window of data. With the Timestamps option, more samples can be taken, leading to the potential of a better RTT estimate (see [RFC1323] and [RFC6298]).

Because the Timestamps option is most relevant to the setting of the retransmission timer, we discuss its use for that purpose in more detail when we discuss retransmission in Chapter 14. We say “for that purpose” because although the Timestamps option allows for more frequent RTT samples, it also provides a way for the receiver to avoid receiving old segments and considering them as valid. This is called *Protection Against Wrapped Sequence Numbers* (PAWS), and it is described in [RFC1323] along with the Timestamps option. We'll now take a look at how it works.

Consider a TCP connection using the Window Scale option with the largest possible window, about 1GB (2^{30}). Also assume that the Timestamps option is being used and that the timestamp value assigned by the sender increments by 1 for each window that is sent. (This is conservative. Normally the timestamp increments faster than this.) Table 13-2 shows the possible data flow between the two hosts when transferring 6GB. To avoid lots of ten-digit numbers, we use the notation G to mean a multiple of 1,073,741,824. We also use the notation from `tcpdump` that *J:K* means byte *J* through and including byte *K* - 1.

Table 13-2 The TCP Timestamps option can disambiguate segments with the same sequence numbers by providing an extra 32 bits of effective sequence number space.

Time	Bytes Sent	Send Seq. No.	Send Timestamp	Receive
A	0G:1G	0G:1G	1	OK
B	1G:2G	1G:2G	2	OK, but one segment lost and retransmitted
C	2G:3G	2G:3G	3	OK
D	3G:4G	3G:4G	4	OK
E	4G:5G	0G:1G	5	OK
F	5G:6G	1G:2G	6	OK, but retransmitted segment reappears

The 32-bit *Sequence Number* field wraps between times D and E. We assume that one segment gets lost at time B and is retransmitted. We also assume that this lost segment reappears at time F. This assumes that the time difference between the segment getting lost and reappearing is less than the maximum time a segment can live in the network (called the MSL; see Section 13.5.2); otherwise the

segment would have been discarded by some router when its TTL expired. As we mentioned earlier, it is only with relatively high-speed connections that this problem appears, where old segments can reappear and contain sequence numbers currently being transmitted.

We can also see from Table 13-2 that using the Timestamps option prevents this problem. The receiver considers the timestamp as a 32-bit extension of the sequence number. Because the lost segment that reappears at time F has a timestamp of 2, which is less than the most recent valid timestamp (5 or 6), it is discarded by the PAWS algorithm. The PAWS algorithm does not require any form of time synchronization between the sender and the receiver. All the receiver needs is for the timestamp values to be monotonically increasing, and to increase by at least 1 per window of data.

13.3.5 User Timeout (UTO) Option

The *User Timeout* (UTO) option is a relatively new TCP capability described in [RFC5482]. The UTO value (also called `USER_TIMEOUT`) specifies the amount of time a TCP sender is willing to wait for an ACK of outstanding data before concluding that the remote end has failed. `USER_TIMEOUT` has traditionally been a local configuration parameter for TCP [RFC0793]. The UTO option allows one TCP to signal its `USER_TIMEOUT` value to its connection peer. This allows the receiving TCP to adjust its behavior (e.g., to tolerate a longer period of disrupted connectivity prior to aborting a connection). NAT devices could also interpret such information to help set their connection activity timers.

UTO option values are advisory; just because one end of a connection might wish to use a large or small UTO value does not mean that the other end needs to comply. [RFC1122] refines the definition of `USER_TIMEOUT` and suggests that a TCP reaching a threshold of three (R1) retransmissions should notify the requesting application, and that after 100s (R2) the connection should be closed. Some implementations have an API function to change R1 and R2. Because long UTOS might lead to resource exhaustion concerns and short UTOS might result in some connections being torn down early (a type of DoS attack), upper and lower limits are placed on the possible UTO values. The way to set `USER_TIMEOUT`, then, is as follows:

```
USER_TIMEOUT = min(U_LIMIT, max(ADV_UTO, REMOTE_UTO, L_LIMIT))
```

where `ADV_UTO` is the UTO option advertised to the remote TCP, `REMOTE_UTO` is the peer's advertised UTO option value, `U_LIMIT` is the local system's upper UTO limit, and `L_LIMIT` is the local system's UTO lower limit. Note that this formula does not guarantee that each end of the same connection will arrive at the same `USER_TIMEOUT` value. In all cases the `L_LIMIT` value must be greater than the associated connection's retransmission timeout (RTO) value (see Chapter 14), and it is recommended to be set to 100s to retain compatibility with [RFC1122].

UTO options are included on SYN segments when a connection is established, on the first non-SYN segments, and whenever the `USER_TIMEOUT` value is changed. The option value is expressed as a 15-bit value in units of seconds or minutes following a bit field (“granularity”) that indicates that the value is in minutes (1) or seconds (0). As a relatively new option, it is not yet widely deployed.

13.3.6 Authentication Option (TCP-AO)

There is an option used to enhance the security of TCP connections. It is designed to enhance and replace an earlier mechanism called TCP-MD5 [RFC2385]. Called the *TCP Authentication Option* (TCP-AO) [RFC5925], it uses a cryptographic hash algorithm (see Chapter 18), in combination with a secret value known to each end of a TCP connection, to authenticate each segment. TCP-AO improves upon TCP-MD5 by supporting a variety of cryptographic algorithms and identifying changing of keys using in-band signaling. It does not provide a comprehensive key management solution, however. That is, each end still has to have a way to establish a shared set of keys prior to operation.

When sending, the TCP derives a traffic key from the shared secret key and computes the hash value according to a particular cryptographic algorithm [RFC5926]. A receiver, equipped with the same secret key, is likewise able to derive the traffic key and use it to ensure that an arriving segment has not been modified in transit (with high probability). This option is intended as a strong countermeasure to a variety of TCP spoofing attacks (see Section 13.8). However, because it requires creation and distribution of a shared key (and is a relatively new option), it is not yet widely deployed.

13.4 Path MTU Discovery with TCP

In Chapter 3, we described the concept of the path MTU. It is the minimum MTU on any network segment that is currently in the path between two hosts. Knowing the path MTU can help protocols such as TCP avoid fragmentation. In Chapter 10, we looked at how discovery of the path MTU (PMTUD) is accomplished based on ICMP messages, but in that case UDP is not usually able to adapt its datagram size because the application specifies the size (i.e., not the transport protocol). TCP, in providing the byte stream abstraction it implements, determines what segment size to use and as a result has a much greater degree of control over the size of IP datagrams that are ultimately generated.

In this section we will examine how PMTUD is used by TCP. Our discussion will apply to both TCP/IPv4 and TCP/IPv6. More details are provided by [RFC1191] and [RFC1981], respectively. A method that avoids the use of ICMP, called *Packetization Layer Path MTU Discovery* (PLPMTUD), can also be used by TCP [RFC4821] or by other transport protocols. We shall use the ICMPv6 Packet Too Big (PTB) terminology to refer to either ICMPv4 Destination Unreachable (Fragmentation Required) or ICMPv6 Packet Too Big messages.

TCP's regular PMTUD process operates as follows: When a connection is established, TCP uses the minimum of the MTU of the outgoing interface, or the MSS announced by the other end, as the basis for selecting its send maximum segment size (SMSS). PMTUD does not allow TCP to exceed the MSS announced by the other end. If the other end does not specify an MSS, the sender assumes a default of 536 bytes, but this situation is now rare. It is also possible for an implementation to save path MTU information on a per-destination basis to help in selecting its segment size. Note that the path MTU in each direction of a connection could be different.

Once the initial SMSS is chosen, all IPv4 datagrams sent by TCP on that connection have the IPv4 *DF* bit field set. For TCP/IPv6, this is not necessary because there is no *DF* bit field; all datagrams are assumed to have it set implicitly. If a PTB is received, TCP decreases the segment size and retransmits using a different segment size. If the PTB contains the suggested next-hop MTU, the segment size can be set to the next-hop MTU minus the sizes of the IPv4 (or IPv6) and TCP headers. If the next-hop MTU value is not present (e.g., an older ICMP error was returned that lacks this information), the sender may try a variety of values (e.g., binary search for a usable value). This also affects TCP's congestion control management (see Chapter 16). For PLPMTUD the situation is similar, except PTB messages are not used. Instead, the protocol performing PMTUD must be able to detect message discards quickly and perform its own datagram size adjustments.

Because routes can change dynamically, when some time has passed since the last decrease of the segment size, a larger value (up to the initial SMSS) can be tried. Guidance in [RFC1191] and [RFC1981] recommends that this time interval be about 10 minutes.

There are a number of problems with PMTUD when it operates in an Internet environment with firewalls that block PTB messages [RFC2923]. Of the various operational problems with PMTUD, *black holes* have been the most problematic, although the situation is improving (in [LS10], 80% of systems studied were able to properly process PTB messages). PMTUD black holes arise when a TCP implementation that depends on the delivery of ICMP messages to adjust its segment size never receives them. This could be for several reasons, including a firewall or NAT configuration that prohibits such ICMP messages from being forwarded. The consequence is a TCP connection that cannot proceed once it starts to use larger packets. It can be difficult to diagnose because only large packets cannot be forwarded. The smaller ones (such as SYN and SYN + ACK packets used to establish the connection) generally succeed. Some TCP implementations have "black hole detection," which amounts to trying a smaller segment size when a segment is retransmitted several times.

13.4.1 Example

We can see the correct behavior of PMTUD when an intermediate router has an MTU less than either of the endpoints' MSS. To create this situation, we begin with a router (a Linux host with local address 10.0.0.1) that has a PPPoE interface to a

DSL service provider. The PPPoE link uses an MTU of 1492 (1500 bytes for Ethernet, minus 6 bytes of PPPoE overhead, minus another 2 bytes of PPP overhead; see Chapter 3). Figure 13-7 is an illustration of the topology.

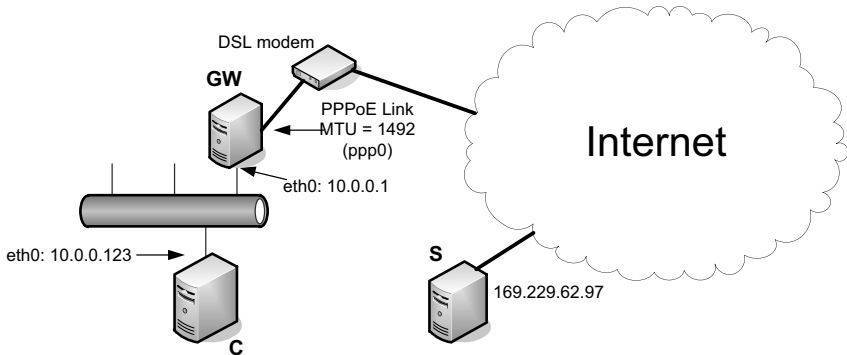


Figure 13-7 The PPPoE encapsulation drops the path MTU of most TCP connections to 1492 bytes from what might otherwise have been 1500 bytes (the typical MTU for Ethernet). To demonstrate TCP's use of PMTUD, we set the MTU even smaller (288 bytes).

In order to induce this behavior specifically, we can reduce the MTU size on the PPPoE link from 1492 to, say, 288 bytes. On the GW machine, the following command accomplishes this task:

```
Linux(GW)# ifconfig ppp0 mtu 288
```

In addition, we need to tell the client system (C) that small segments are allowed:

```
Linux(C)# sysctl -w net.ipv4.route.min_pmtu=68
```

If we did not perform this second operation, Linux would clamp its minimum path MTU at the default value of 552 bytes, which helps avoid certain small MTU attacks (see Section 13.8). The consequence of doing so in our example here is that any packets larger than 288 bytes would be fragmented. To avoid this, and to demonstrate PMTUD more effectively, we remove this minimum. We then start a file transfer from machine C (address 10.0.0.123) to the server S on the Internet (address 169.229.62.97). Listing 13-2 shows a `tcpdump` packet trace from this exchange. Several lines have been wrapped and extraneous fields have been removed for clarity.

Listing 13-2 The path MTU discovery mechanism finds an appropriate segment size to use when transiting the network where the middle link has a smaller MTU than the endpoints.

```
1 20:20:21.992721 IP (tos 0x0, ttl 45, id 43565, offset 0, flags [DF],
    proto 6, length: 588)
    169.229.62.97.22 > 10.0.0.123.1027: P [tcp sum ok]
    41:577(536) ack 23
```

```

2 20:20:21.993727 IP (tos 0x0, ttl 64, id 57659, offset 0, flags [DF],
    proto 6, length: 588)
    10.0.0.123.1027 > 169.229.62.97.22: P [tcp sum ok]
    23:559(536) ack 577

3 20:20:21.994093 IP (tos 0xc0, ttl 64, id 57547, offset 0, flags
    [none], proto 1, length: 576)
    10.0.0.1 > 10.0.0.123: icmp 556:
    169.229.62.97 unreachable - need to frag (mtu 288) for
    IP (tos 0x0, ttl 63, id 57659, offset 0, flags [DF],
    proto 6, length: 588)
    10.0.0.123.1027 > 169.229.62.97.22:
    P 23:559(536) ack 577

4 20:20:21.994884 IP (tos 0x0, ttl 64, id 57660, offset 0, flags [DF],
    proto 6, length: 288)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    23:259(236) ack 577

...

5 20:20:22.488856 IP (tos 0x0, ttl 45, id 6712, offset 0, flags [DF],
    proto 6, length: 836)
    169.229.62.97.22 > 10.0.0.123.1027: P [tcp sum ok]
    857:1641(784)ack 855

...

6 20:20:29.672947 IP (tos 0x8, ttl 64, id 57679, offset 0, flags [DF],
    proto 6, length: 1452)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    1431:2831(1400) ack 2105

7 20:20:29.674123 IP (tos 0xc8, ttl 64, id 57548, offset 0, flags
    [none], proto 1, length: 576)
    10.0.0.1 > 10.0.0.123: icmp 556:
    169.229.62.97 unreachable - need to frag (mtu 288) for
    IP (tos 0x8, ttl 63, id 57679, offset 0, flags [DF],
    proto 6, length: 1452)
    10.0.0.123.1027 > 169.229.62.97.22: .
    1431:2831(1400) ack 2105

8 20:20:29.673751 IP (tos 0x8, ttl 64, id 57680, offset 0, flags [DF],
    proto 6, length: 1452)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    2831:4231(1400) ack 2105

9 20:20:29.675180 IP (tos 0xc8, ttl 64, id 57549, offset 0, flags
    [none], proto 1, length: 576)
    10.0.0.1 > 10.0.0.123: icmp 556:
    169.229.62.97 unreachable - need to frag (mtu 288) for
    IP (tos 0x8, ttl 63, id 57680, offset 0, flags [DF],
    proto 6, length: 1452)
    10.0.0.123.1027 > 169.229.62.97.22: .
    2831:4231(1400) ack 2105

```

```
10 20:20:29.674932 IP (tos 0x8, ttl 64, id 57681, offset 0, flags
    [DF], proto 6, length: 288)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    1431:1667(236) ack 2105

11 20:20:29.675143 IP (tos 0x8, ttl 64, id 57682, offset 0, flags
    [DF], proto 6, length: 288)
    10.0.0.123.1027 > 169.229.62.97.22: . [tcp sum ok]
    1667:1903(236) ack 2105
```

In the `tcpdump` output, the connection has already been set up and MSS options have been exchanged. All packets on the connection have the *DF* bit field set, so both ends are performing PMTUD. The remote side's first packet is 588 bytes long, which transitions the router successfully in one piece, despite our configuration of the MTU on the PPPoE links being 288 bytes. The reason for this is asymmetry in the MTU configuration. Although the local end of the PPPoE link is using a maximum *transmission* unit of 288 bytes, the other end is using a larger size SMSS, presumably 1492 bytes. This leaves us in the situation where our outgoing packets need to be small (288 bytes or less), and packets traveling in the reverse direction can be larger.

When the local end attempts to send a larger packet of size 588 bytes with the *DF* bit field turned on, a PTB message is generated by the router (10.0.0.1), indicating that the appropriate MTU for the next-hop link is 288 bytes. The TCP responds by sending its next packet with size 288 bytes, as instructed. To then send the rest of the sequence numbers it attempted to send in its 588-byte packet, it sends two additional packets, of sizes 288 and 116. We see a similar pattern of sizes repeats during the course of the file transfer.

The PMTU discovery process is one of the only ways TCP explicitly attempts to adapt its segment size after a connection has started, at least when large amounts of data are transferred. The size of a segment can affect the overall throughput performance, as can the window size. We discuss how these affect overall performance in Chapter 15.

13.5 TCP State Transitions

We have described numerous rules regarding the initiation and termination of a TCP connection, and we have seen which types of segments are sent during different phases of a connection. The rules that determine what TCP does are determined by what state TCP is in. The current state is changed based on various stimuli, such as segments that are transmitted or received, timers that expire, application reads or writes, or information from other layers. These rules can be summarized in TCP's state transition diagram.

13.5.1 TCP State Transition Diagram

TCP's state transition diagram is shown in Figure 13-8. States are indicated by ovals and transitions between states by arrows. Each endpoint of a connection transitions through the states. Some transitions are triggered by the receipt of a segment with certain control bit fields set (e.g., *SYN*, *ACK*, *FIN*). Some transitions

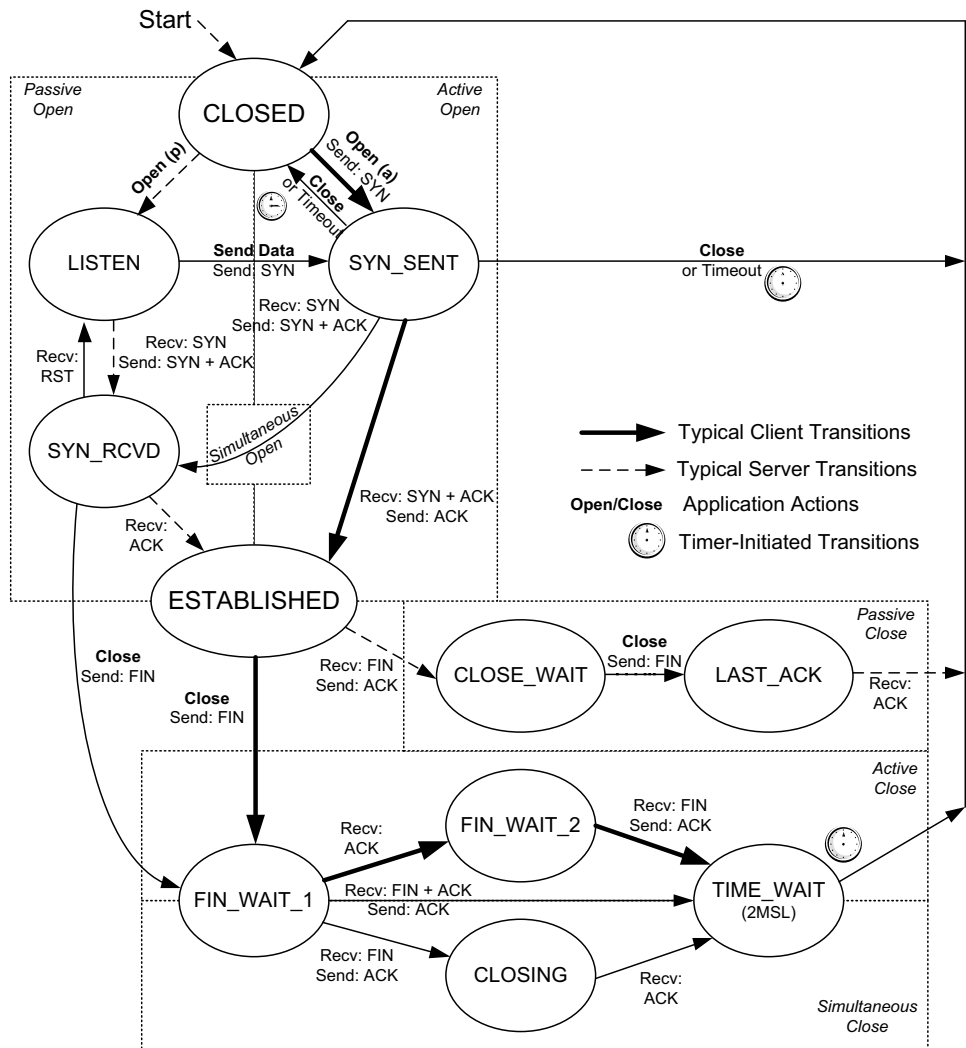


Figure 13-8 The TCP state transition diagram (also called finite state machine). Arrows represent transitions between states due to segment transmission, segment reception, or timers expiring. The bold arrows indicate typical client behavior, and the dashed arrows indicate typical server behavior. The boldface directives (e.g., open, close) are actions performed by applications.

also cause a segment with particular control bit fields set to be sent. Other transitions may be triggered by application actions or by timers expiring. Each of these cases is indicated in the diagram as a textual annotation near the associated transition arrow. When initialized, TCP starts in the CLOSED state. Usually an immediate transition takes it to either the SYN_SENT or LISTEN state, depending on whether the TCP is asked to perform an active or passive open, respectively.

Note in this diagram that only a subset of the state transitions is “typical.” We have marked the normal client transitions with a darker solid arrow, and the normal server transitions with a dashed arrow. The two transitions leading to the ESTABLISHED state correspond to opening a connection, and the two transitions leading from the ESTABLISHED state are for the termination of a connection. The ESTABLISHED state is where data transfer can occur between the two ends in both directions. Chapters 14–17 describe what happens in this state.

We have labeled the FIN_WAIT_1, FIN_WAIT_2, and TIME_WAIT states as being (at least partially) in a box called “Active Close.” These are the set of states entered when the local application initiates a close request. Two other states (CLOSE_WAIT and LAST_ACK) are collected in a dashed box with the label “Passive Close.” These states correspond to waiting for a peer to acknowledge a FIN segment and perform its close. Simultaneous close, which is a form of double active close, uses the CLOSING state.

The names of the 11 states (CLOSED, LISTEN, SYN_SENT, etc.) in this figure are based on the names output by the `netstat` command in UNIX, Linux, and Windows, which are themselves based on the names originally used in [RFC0793]. The state CLOSED is not really an “official” state but has been added as a useful starting point and ending point for the diagram.

The state transition from LISTEN to SYN_SENT is legal in the TCP protocol but is not supported by Berkeley sockets and is rarely seen. The transition from SYN_RCVD back to LISTEN is valid only if the SYN_RCVD state was entered from the LISTEN state (the normal scenario), not from the SYN_SENT state (a simultaneous open). This means that if we perform a passive open (enter LISTEN), receive a SYN, send a SYN with an ACK (enter SYN_RCVD), and then receive a reset instead of an ACK, the endpoint returns to the LISTEN state and waits for another connection request to arrive.

Figure 13-9 shows the normal TCP connection establishment and termination, detailing the different states through which the client and server pass. It is a simpler version of Figure 13-1 showing the relevant states but not the options or ISN details. We assume in Figure 13-9 that the client on the left side does an active open and the server on the right side does a passive open. Although we show the client doing the active close, as we mentioned earlier, either side can do the active close.

13.5.2 TIME_WAIT (2MSL Wait) State

The TIME_WAIT state is also called the 2MSL wait state. It is a state in which TCP waits for a time equal to twice the *Maximum Segment Lifetime* (MSL), sometimes called *timed wait*. Every implementation must choose a value for the MSL. It is

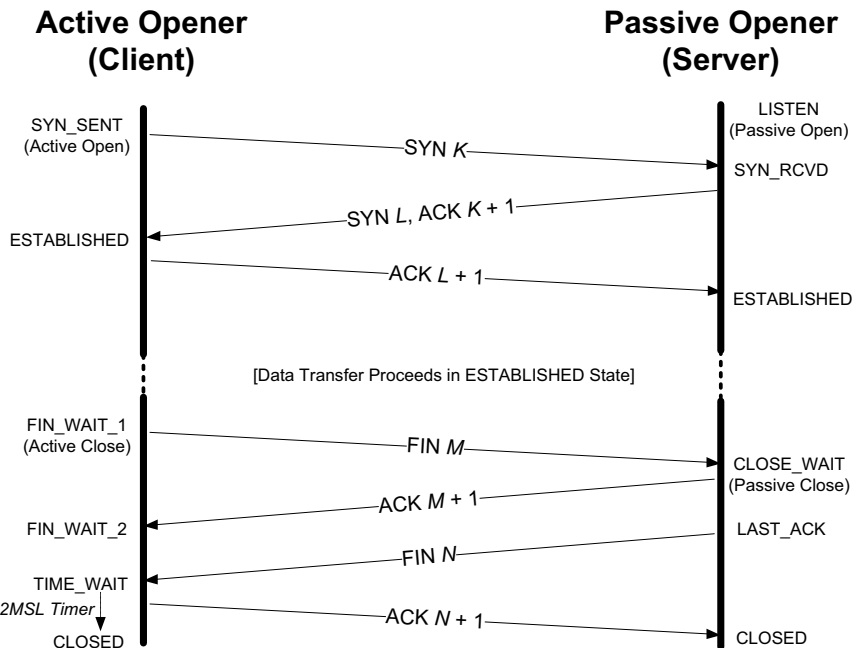


Figure 13-9 TCP states corresponding to normal connection establishment and termination

the maximum amount of time any segment can exist in the network before being discarded. We know that this time limit is bounded, because TCP segments are transmitted as IP datagrams, and the IP datagram has the *TTL* field or *Hop Limit* field that limits its effective lifetime (see Chapter 5). [RFC0793] specifies the MSL as 2 minutes. Common implementation values, however, are 30s, 1 minute, or 2 minutes. In most cases, the value can be modified. On Linux, the value `net.ipv4.tcp_fin_timeout` holds the 2MSL wait timeout value (in seconds). On Windows, the following registry key:

```
HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\TcpTimedWaitDelay
```

holds the timeout. It is permitted to be in the range of 30 to 300s. For IPv6, replace the term `Tcpip` with `Tcpip6`.

Given the MSL value for an implementation, the rule is: When TCP performs an active close and sends the final ACK, that connection must stay in the `TIME_WAIT` state for twice the MSL. This lets TCP resend the final ACK in case it is lost. The final ACK is resent not because the TCP retransmits ACKs (they do not consume sequence numbers and are not retransmitted by TCP), but because the other side will retransmit its `FIN` (which does consume a sequence number). Indeed, TCP will always retransmit `FIN`s until it receives a final ACK.

Another effect of this 2MSL wait state is that while the TCP implementation waits, the endpoints defining that connection (client IP address, client port number, server IP address, and server port number) cannot be reused. That connection can be reused only when the 2MSL wait is over, or when a new connection uses an ISN that exceeds the highest sequence number used on the previous instantiation of the connection [RFC1122], or if the use of the Timestamps option allows the disambiguation of segments from a previous connection instantiation to not otherwise be confused [RFC6191]. Unfortunately, some implementations impose a more stringent constraint. In these systems, a local port number cannot be reused while that port number is the local port number of *any* endpoint that is in the 2MSL wait state on the system. We will see examples of this constraint in Listings 13-3 and 13-4.

Most implementations and APIs provide a way to bypass this restriction. With the Berkeley sockets API, the `SO_REUSEADDR` socket option enables the bypass operation. It lets the caller assign itself a local port number even if that port number is part of some connection in the 2MSL wait state. We will see, however, that even with this bypass mechanism for one socket (address, port number pair), the rules of TCP still (should) prevent this port number from being reused by another instantiation of the same connection that is in the 2MSL wait state. Any delayed segments that arrive for a connection while it is in the 2MSL wait state are discarded. Because the connection defined by the address/port 4-tuple in the 2MSL wait state cannot be reused during this time period, when a valid connection is finally established, we know that delayed segments from an earlier instantiation of this connection cannot be misinterpreted as being part of the new connection.

For interactive applications, it is normally the client that does the active close and enters the `TIME_WAIT` state. The server usually does the passive close and does not go through the `TIME_WAIT` state. The implication is that if we terminate a client, and restart the same client immediately, that new client cannot reuse the same local port number. This is not ordinarily a problem, because clients normally use ephemeral ports assigned by the operating system and do not care what the assigned port number is. (Recall, it is actually a recommended practice for them to be randomized for security reasons [RFC6056].) This is important to know because a client that makes a large number of connections quickly (especially to the same server) could conceivably have to delay while other connections terminate if ephemeral ports are in short supply.

With servers, however, the situation is different. They almost always use well-known ports. If we terminate a server process that has a connection established and immediately try to restart it, the server cannot assign its assigned port number to its endpoint (it gets an “Address already in use” binding error), because that port number is part of a connection that is in a 2MSL wait state. It may take from 1 to 4 minutes for the server to be able to restart, depending on the local system’s value for the MSL. We can see this scenario using our `sock` program. In Listing 13-3 we start the server, connect to it from a client, and then terminate the server.

Listing 13-3 A TCP connection must complete a 2MSL delay in the TIME_WAIT state before a port number can be reused by another process.

```
Linux% sock -v -s 6666
```

(now a client on another computer connects to this server)

```
connection on 192.168.10.144.6666 from 192.168.10.140.2623
```

(server stopped by typing interrupt character)

(now server is restarted)

```
Linux% sock -v -s 6666
```

```
can't bind local address: Address already in use
```

```
Linux% netstat -n -t
```

Active Internet connections (w/o servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	192.168.10.144:6666	192.168.10.140:2623	TIME_WAIT

(wait one minute and restart server again)

```
Linux% sock -v -s 6666
```

When we try to restart the server, the program outputs an error message indicating that it cannot bind its port number because the address is already in use. This really means that the address and port number combination is already in use; it is in a 2MSL wait state because of the previous connection. This is the more stringent restriction on port number reuse mentioned before. The output from the `netstat` command shows that the connection is in the TIME_WAIT state. Although clients do not typically experience as many issues with 2MSL wait states as servers do, we can demonstrate the same issue by having the client specify its own port number, as shown in Listing 13-4.

Listing 13-4 A client cannot reuse a port number while it is still being used by another connection in the 2MSL wait state.

(start server in one window)

```
Linux% sock -s -v 6666
```

(connect to it from another window)

```
Linux% sock -v 127.0.0.1 6666
```

(server identifies incoming connection)

```
connection on 127.0.0.1.6666 from 127.0.0.1.2091
```

(client identifies connection establishment, and is interrupted)

```
connected on 127.0.0.1.2091 to 127.0.0.1.6666
```

```
^C
```

(server identifies connection has terminated and exits)

```
connection closed by peer
```

```
Linux%
```

(client is restarted, specifying same port number as before)

```
Linux% sock -b 2091 -v 127.0.0.1 6666  
bind() error: Address already in use
```

(wait 30 seconds and try again)

```
Linux% sock -b 2091 -v 192.168.10.144 6666  
connect() error: Connection refused
```

The first time we execute the client we specify the `-v` option to see what the local (ephemeral) port number assigned to the client is (2091). The second time we execute the client we specify the `-b` option, telling the client to assign itself 2091 as its local port number instead of being given another ephemeral port number by the operating system. As we expect, the client cannot do this, because port 2091 is part of a connection that is in a 2MSL wait state. Once the wait is over (1 minute on this Linux machine), the client attempts to connect again, but the server exited when the connection was interrupted the first time, so it is refused. We shall see how TCP reset segments are used to signal this connection refused condition in Section 13.6.

We mentioned earlier that most systems provide a way of overriding the default behavior, which allows processes to bind to ports even if those ports are part of connections in the 2MSL wait state. Now we try the same scenario as before, but using the `-A` option to `sock`, which enables the bypass mechanism:

```
Linux% sock -A -v -s 6666  
Linux% sock -A -v -s 6666
```

In this example, we start the server with the `-A` option, which enables the `SO_REUSEADDR` socket option that we mentioned. By doing this, we allow the server to bind to its port even though it is part of a connection in the 2MSL wait state. If we try to use the client right away with the same port, however, the following happens:

```
Linux% sock -b 32840 -v 127.0.0.1 6666  
bind() error: Address already in use
```

Once again, the endpoint 127.0.0.1.32840 is in use, so the client fails. If, however, we also use the `-A` option for the client, we can force the connection to work:

```
Linux% sock -A -b 32840 -v 127.0.0.1 6666  
Connected on 127.0.0.1.32840 to 127.0.0.1.6666  
TCP_MAXSEG = 16383
```

Here we see that even though the same connection (4-tuple) is being used again before the 2MSL wait state expires, the use of the `-A` option has forced the connection to be allowed. Of course, this is all taking place on the same computer, so the operating system is able to ascertain what processes represent what ends

of the connections in the 2MSL wait state and (potentially, at least) keep them separate. What if we try the same thing again but establish the connection from another host? Here we test this idea:

(start server on first machine)

```
Linux% sock -v -s 6666
```

(connect to it from second - Windows - machine)

```
C:\> sock -A -v 10.0.0.1 6666
```

(server identifies incoming connection)

```
connection on 10.0.0.1.6666 from 10.0.0.3.2172
```

(client identifies connection establishment, and is interrupted)

```
connected on 10.0.0.3.2172 to 10.0.0.1.6666
```

```
^C
```

```
C:\>
```

(server identifies connection has terminated and exits)

```
connection closed by peer
```

```
Linux%
```

(client is restarted, specifying same port number as before)

```
C:\> sock -A -b 2091 -v 10.0.0.1 6666
```

```
connect() error: Address already in use
```

```
C:\> sock -A -b 2091 -v 10.0.0.1 6666
```

```
connect() error: Address already in use
```

(wait 30 seconds and try again)

```
C:\> sock -A -b 2091 -v 10.0.0.1 6666
```

```
connect() error: Connection refused
```

This example is similar to the previous one, except the client and server are on different machines. We observe that irrespective of the `-A` flag on the client, the 2MSL wait time is induced. Here the 2MSL wait lasts for 30s. After that, the client attempts to contact the server, which has already exited.

One interesting thing happens if we switch the client and server machines. We will now use Windows as the server and Linux as the client and repeat the experiment:

(start server on Windows machine)

```
C:\> sock -v -s 6666
```

(connect to it from second - Linux - machine)

```
Linux% sock -A -v 192.168.10.145 6666
```

(server identifies incoming connection)

```
connection on 192.168.10.145.6666 from 192.168.10.145.32843
```

(client identifies connection establishment, and is interrupted)

```
connected on 192.168.10.144.32843 to 192.168.10.145.6666
```

```
^C
```

```
Linux%
```

(server identifies connection has terminated and exits)

```
connection closed by peer
```

```
C:\>
```

(client is restarted, specifying same port number as before)

```
Linux% sock -A -b 32843 -v 192.168.10.144 6666
```

```
bind() error: Connection refused
```

At this point we would expect local port 32843 to be unavailable, but because of the way `-A` works on Linux, we are allowed to make use of it. This is a violation of the original TCP specification, but it is allowed by [RFC1122] and [RFC6191], as mentioned before. These specifications allow a new connection request to arrive and be accepted for a connection that is in the `TIME_WAIT` state, if there is a strong reason to believe that segments on the new connection will not be confused with segments on the previous instantiation of the connection based on a combination of the sequence numbers and timestamps. [RFC1337] and the appendix of [RFC1323] show some of the pitfalls related to this rule.

13.5.3 Quiet Time Concept

The 2MSL wait provides protection against delayed segments from an earlier instantiation of a connection being interpreted as part of a new connection that uses the same local and foreign IP addresses and port numbers. But this works only if a host with connections in the 2MSL wait does not crash.

What if a host with connections in the `TIME_WAIT` state crashes, reboots within the MSL, and immediately establishes new connections using the same local and foreign IP addresses and port numbers corresponding to the local connections that were in the `TIME_WAIT` state before the crash? In this scenario, delayed segments from the connections that existed before the crash can be misinterpreted as belonging to the new connections created after the reboot. This can happen regardless of how the initial sequence number is chosen after the reboot.

To protect against this scenario, [RFC0793] states that TCP should wait an amount of time equal to the MSL before creating any new connections after a reboot or crash. This is called the *quiet time*. Few implementations abide by this because most hosts take longer than the MSL to reboot after a crash. Also, if applications use their own checksums or encryption, errors such as these are easily detected.

13.5.4 FIN_WAIT_2 State

In the `FIN_WAIT_2` state, TCP has sent a FIN and the other end has acknowledged it. Unless a half-close is being performed, the TCP must wait for the application on the other end to recognize that it has received an end-of-file notification and close its end of the connection, which causes a FIN to be sent. Only when the application performs this close (and its FIN is received) does the active closing TCP move from the `FIN_WAIT_2` to the `TIME_WAIT` state. This means that one end of the connection can remain in this state forever. The other end is still in the `CLOSE_WAIT` state and can remain there forever, until the application decides to issue its close.

Many implementations prevent this infinite wait in the `FIN_WAIT_2` state as follows: If the application that does the active close does a complete close, not a half-close indicating that it expects to receive data, a timer is set. If the connection is idle when the timer expires, TCP moves the connection into the `CLOSED` state. In Linux, the variable `net.ipv4.tcp_fin_timeout` can be adjusted to control the number of seconds to which the timer is set. Its default value is 60s.

13.5.5 Simultaneous Open and Close Transitions

We have seen the normal uses for the `SYN_SENT` and `SYN_RCVD` states that correspond to sending and receiving SYN segments, respectively. As illustrated in Figure 13-3, TCP was purposely designed to handle simultaneous opens that result in a single connection. When a simultaneous open occurs, the state transitions differ from those shown in Figure 13-9. Both ends send a SYN at about the same time, entering the `SYN_SENT` state. When each end receives its peer's SYN segments, the state changes to `SYN_RCVD`, and each end resends a SYN and acknowledges the received SYN. When each end receives the SYN plus the ACK, the state changes to `ESTABLISHED`.

For a simultaneous close, in terms of Figure 13-6, both ends go from `ESTABLISHED` to `FIN_WAIT_1` when the application issues the close. This causes both FINs to be sent, and they probably pass each other somewhere in the network. When its peer's FIN arrives, each end transitions from `FIN_WAIT_1` to the `CLOSING` state, and each endpoint sends its final ACK. Upon receiving a final ACK, each endpoint's state changes to `TIME_WAIT`, and the 2MSL wait is initiated.

13.6 Reset Segments

We mentioned the *RST* bit field in the TCP header in Chapter 12. A segment having this bit set to “on” is called a “reset segment” or simply a “reset.” In general, a reset is sent by TCP whenever a segment arrives that does not appear to be correct for the referenced connection. (We use the term *referenced connection* to mean the

connection specified by the 4-tuple in the TCP and IP headers of the reset.) Resets ordinarily result in a fast teardown of a TCP connection. We can construct scenarios to demonstrate the use of reset segments.

13.6.1 Connection Request to Nonexistent Port

A common case for generating a reset segment is when a connection request arrives and no process is listening on the destination port. We saw this previously when we encountered the “connection refused” error messages. These are common with TCP. In the case of UDP, we saw in Chapter 10 that an ICMP Destination Unreachable (Port Unreachable) message is generated when a datagram arrives for a destination port that is not in use. TCP uses a reset segment instead.

An example of this is trivial to generate—we use the Telnet client and specify a port number that is not in use on the destination. This destination can just as well be the local computer:

```
Linux% telnet localhost 9999
Trying 127.0.0.1...
telnet: connect to address 127.0.0.1: Connection refused
```

This error message is output by the Telnet client immediately. Listing 13-5 shows the packet exchange corresponding to this command.

Listing 13-5 Reset generated by attempt to open connection to nonexistent port

```
1 22:15:16.348064 127.0.0.1.32803 > 127.0.0.1.9999:
    S [tcp sum ok] 3357881819:3357881819(0) win 32767
    <mss 16396,sackOK,timestamp 16945235 0,nop,wscale 0>
    (DF) [tos 0x10] (ttl 64, id 42376, len 60)
2 22:15:16.348105 127.0.0.1.9999 > 127.0.0.1.32803:
    R [tcp sum ok] 0:0(0) ack 3357881820 win 0
    (DF) [tos 0x10] (ttl 64, id 0, len 40)
```

The values we need to examine in Listing 13-5 are the *Sequence Number* field and *ACK Number* field in the reset (second) segment. Because the *ACK* bit field was not on in the arriving SYN segment, the sequence number of the reset is set to 0 and the ACK number is set to the incoming ISN plus the number of data bytes in the segment. Although there is no data in the arriving segment, the SYN bit logically occupies 1 byte of sequence number space; therefore, in this example the ACK number in the reset segment is set to the ISN, plus the data length (0), plus 1 for the SYN bit.

For a reset segment to be accepted by a TCP, the *ACK* bit field must be set and the *ACK Number* field must be within the valid window (see Chapter 12). This helps to prevent a simple attack in which anyone able to generate a reset matching the appropriate connection (4-tuple) could disrupt a connection [RFC5961].

13.6.2 Aborting a Connection

We saw in Figure 13-1 that the normal way to terminate a connection is for one side to send a FIN. This is sometimes called an *orderly release* because the FIN is sent after all previously queued data has been sent, and there is normally no loss of data. But it is also possible to abort a connection by sending a reset instead of a FIN at any time. This is sometimes called an *abortive release*.

Aborting a connection provides two features to the application: (1) any queued data is thrown away and a reset segment is sent immediately, and (2) the receiver of the reset can tell that the other end did an abort instead of a normal close. The API being used by the application must provide a way to generate the abort instead of a normal close.

The sockets API provides this capability by using the “linger on close” socket option (SO_LINGER) with a 0 linger value. Essentially this means “Linger for no time in making sure data gets to the other side, then abort.” In the following example, we show what happens when a remote command that generates a large amount of output is canceled by the user:

```
Linux% ssh linux cat /usr/share/dict/words
Aarhus
Aaron
Ababa
aback
abaft
abandon
abandoned
abandoning
abandonment
abandons
... continues ...
^C
Killed by signal 2.
```

Here the user has decided to abort the output of this command. The `words` file has 45,427 words in it, so this command was probably some sort of mistake. When the user types the interrupt character, the system indicates that the process (here, the SSH program) has been killed by signal number 2. This signal is called SIGINT and usually terminates a program when it is delivered. Listing 13-6 shows the `tcpdump` output for this example. (We have deleted many of the intermediate packets, because they add nothing to the discussion.)

Listing 13-6 Aborting a connection with a reset (RST) instead of a FIN

```
Linux# tcpdump -vvv -s 1500 tcp

1 22:33:06.386747 192.168.10.140.2788 > 192.168.10.144.ssh:
  S [tcp sum ok] 1520364313:1520364313(0) win 65535
  <mss 1460,nop,nop,sackOK>
  (DF) (ttl 128, id 43922, len 48)
```



```

2 22:33:06.386855 192.168.10.144.ssh > 192.168.10.140.2788:
    S [tcp sum ok] 181637276:181637276(0) ack 1520364314
    win 5840
    <mss 1460,nop,nop,sackOK>
    (DF) (ttl 64, id 0, len 48)

```

```

3 22:33:06.387676 192.168.10.140.2788 > 192.168.10.144.ssh:
    . [tcp sum ok] 1:1(0) ack 1 win 65535
    (DF) (ttl 128, id 43923, len 40)

```

(... ssh encrypted authentication exchange and bulk data transfer ...)

```

4 22:33:13.648247 192.168.10.140.2788 > 192.168.10.144.ssh:
    R [tcp sum ok] 1343:1343(0) ack 132929 win 0
    (DF) (ttl 128, id 44004, len 40)

```

Segments 1–3 show the normal connection establishment. When the interrupt character is hit, the connection is aborted. The reset segment contains a sequence number and acknowledgment number. Also notice that the reset segment elicits no response from the other end—it is not acknowledged at all. The receiver of the reset aborts the connection and advises the application that the connection was reset. This often results in the error indication “Connection reset by peer” or a similar message.

13.6.3 Half-Open Connections

A TCP connection is said to be *half-open* if one end has closed or aborted the connection without the knowledge of the other end. This can happen anytime one of the peers crashes. As long as there is no attempt to transfer data across a half-open connection, the end that is still up does not detect that the other end has crashed.

Another common cause of a half-open connection is when one host is powered off instead of shut down properly. This happens, for example, when PCs are being used to run remote login clients and are switched off at the end of the day. If there was no data transfer going on when the power was cut, the server will never know that the client disappeared (it would still think the connection is in the ESTABLISHED state). When the user comes in the next morning, powers on the PC, and starts a new session, a new occurrence of the server is started on the server host. This can lead to many half-open TCP connections on the server host. (In Chapter 17 we will see a way for one end of a TCP connection to discover that the other end has disappeared using TCP’s keepalive option.)

We can easily create a half-open connection. In this case, we do so on the client rather than the server. We will execute the Telnet client on 10.0.0.1, connecting to the Sun RPC Service (sunrpc, port 111) server at 10.0.0.7 (see Listing 13-7). We type one line of input and watch it go across with `tcpdump`, and then we disconnect the Ethernet cable on the server’s host and reboot the server host. This simulates the server host crashing. (We disconnect the Ethernet cable before rebooting the server to prevent it from sending a FIN out of the open connections,

which some TCPs do when they are shut down.) After the server has rebooted, we reconnect the cable and try to send another line from the client to the server. After rebooting, the server's TCP has lost all memory of the connections that existed before, so it knows nothing about the connection that the data segment references. The rule of TCP is that the receiver responds with a reset.

Listing 13-7 The server host is disconnected and rebooted, leaving a half-open connection at the client. When it receives additional data on the connection it now knows nothing about, the server responds with a reset segment, closing the connection at both ends.

```
Linux% telnet 10.0.0.7 sunrpc
Trying 10.0.0.7...
Connected to 10.0.0.7.
Escape character is '^'.
foo
(Ethernet cable disconnected and server rebooted)
bar
Connection closed by remote host
```

Listing 13-8 shows the `tcpdump` output for this example.

Listing 13-8 Reset in response to data segment on a half-open connection

```
1 23:15:48.804142 IP (tos 0x10, ttl 64, id 20095, offset 0,
   flags [DF], proto 6, length: 60)
   10.0.0.1.1310 > 10.0.0.7.sunrpc:
   S [tcp sum ok] 2365970104:2365970104(0) win 5840
   <mss 1460,sackOK,timestamp 3849492679 0,nop,wscale 2>

2 23:15:48.804742 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF],
   proto 6, length: 60)
   10.0.0.7.sunrpc > 10.0.0.1.1310:
   S [tcp sum ok] 2093796387:2093796387(0) ack 2365970105 win 5792
   <mss 1460,sackOK,timestamp 654784 3849492679,nop,wscale 0>

3 23:15:48.805028 IP (tos 0x10, ttl 64, id 20097, offset 0,
   flags [DF], proto 6, length: 52)
   10.0.0.1.1310 > 10.0.0.7.sunrpc:
   . [tcp sum ok] 1:1(0) ack 1 win 1460
   <nop,nop,timestamp 3849492680 654784>

4 23:15:51.999394 IP (tos 0x10, ttl 64, id 20099, offset 0,
   flags [DF], proto 6, length: 57)
   10.0.0.1.1310 > 10.0.0.7.sunrpc:
   P [tcp sum ok] 1:6(5) ack 1 win 1460
   <nop,nop,timestamp 3849495875 654784>

5 23:15:51.999874 IP (tos 0x0, ttl 64, id 12773, offset 0,
   flags [DF], proto 6, length: 52)
   10.0.0.7.sunrpc > 10.0.0.1.1310:
   . [tcp sum ok] 1:1(0) ack 6 win 5792
   <nop,nop,timestamp 656421 3849495875>
```

```
6 23:17:19.419611 arp who-has 10.0.0.7 (Broadcast) tell 0.0.0.0
7 23:17:20.419142 arp who-has 10.0.0.7 (Broadcast) tell 0.0.0.0
8 23:17:21.427458 arp reply 10.0.0.7 is-at 00:e0:00:88:ad:d6

9 23:17:21.921745 arp who-has 10.0.0.1 tell 10.0.0.7
10 23:17:21.921892 arp reply 10.0.0.1 is-at 00:04:5a:9f:9e:80

11 23:17:23.437114 arp who-has 10.0.0.7 (Broadcast) tell 10.0.0.7

12 23:17:34.804196 arp who-has 10.0.0.7 tell 10.0.0.1
13 23:17:34.804650 arp reply 10.0.0.7 is-at 00:e0:00:88:ad:d6

14 23:17:43.684786 IP (tos 0x10, ttl 64, id 20101, offset 0,
    flags [DF], proto 6, length: 57)
    10.0.0.1.1310 > 10.0.0.7.sunrpc:
    P [tcp sum ok] 6:11(5) ack 1 win 1460
    <nop,nop,timestamp 3849607577 656421>

15 23:17:43.685277 IP (tos 0x10, ttl 64, id 0, offset 0,
    flags [DF], proto 6, length: 40)
    10.0.0.7.sunrpc > 10.0.0.1.1310:
    R [tcp sum ok] 2093796388:2093796388(0) win 0
```

Segments 1–3 are the normal connection establishment. Segment 4 sends the line “foo” to the `sunrpc` server (the 5 bytes required include a carriage return and newline character), and segment 5 is the acknowledgment.

At this point we disconnect the Ethernet cable from the server (address 10.0.0.7), reboot, and reconnect the cable. This takes about 90s. We then type the next line of input to the client (“bar”), and when we type the return key the line is sent to the server (the first TCP segment after the ARP traffic in Listing 13-9). This elicits a reset response from the server, which no longer has any knowledge of the existence of the connection.

Note that when the host reboots, it uses gratuitous ARP (see Chapter 4) in order to determine if its IPv4 address is already in use on the segment, and to supply it to others. It also requests the MAC address for IPv4 address 10.0.0.1 because that is its default router to the Internet.

13.6.4 TIME-WAIT Assassination (TWA)

As mentioned previously, the `TIME_WAIT` state is intended to allow any datagrams lingering from a closed connection to be discarded. During this period, the waiting TCP usually has little to do; it merely holds the state until the 2MSL timer expires. If, however, it receives certain segments from the connection during this period, or more specifically an RST segment, it can become desynchronized. This is called *TIME-WAIT Assassination* (TWA) [RFC1337]. Consider the exchange of packets shown in Figure 13-10.

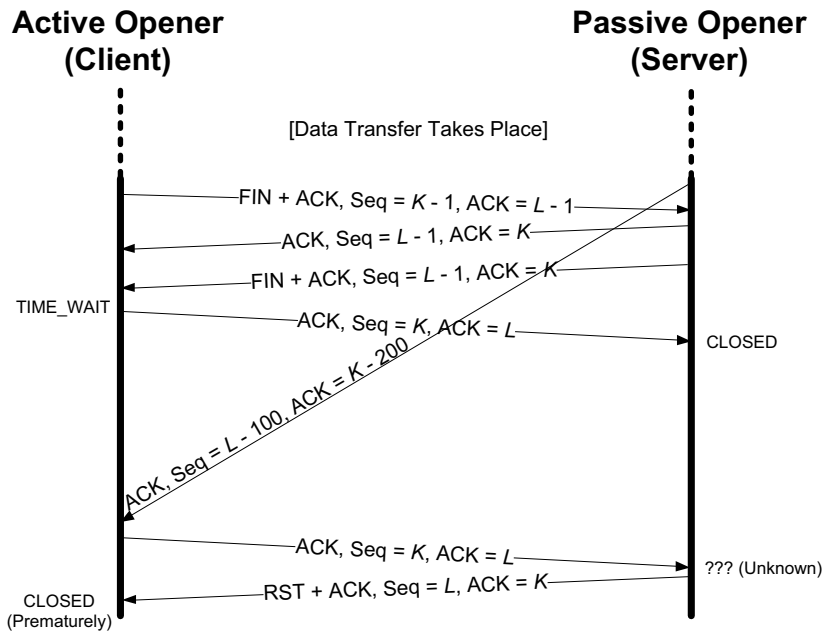


Figure 13-10 An RST segment can “assassinate” the **TIME_WAIT** state and force the connection to close prematurely. Various methods exist to resist this problem, including ignoring RST segments when in the **TIME_WAIT** state.

In the example shown in Figure 13-10, the server has completed its role in the connection and cleared any state. The client remains in the **TIME_WAIT** state. When the FIN exchange completes, the client’s next sequence number is K and the server’s is L . The late-arriving segment is sent from the server to the client using sequence number $L - 100$ and containing ACK number $K - 200$. When the client receives this segment, it determines that both the sequence number and ACK values are “old.” When receiving such old segments, TCP responds by sending an ACK with the most current sequence number and ACK values (K and L , respectively). However, when the server receives this segment, it has no information whatsoever about the connection and therefore replies with an RST segment. This is no problem for the server, but it causes the client to prematurely transition from **TIME_WAIT** to **CLOSED**. Most systems avoid this problem by simply not reacting to reset segments while in the **TIME_WAIT** state.

13.7 TCP Server Operation

We said in Chapter 1 that most TCP servers are concurrent. When a new connection request arrives at a server, the server accepts the connection and invokes

14

TCP Timeout and Retransmission

14.1 Introduction

Efficiency and performance are issues that we have not discussed much so far. We have primarily been concerned with correctness of operation. In this chapter and the next two, we will be focusing not only on the basic tasks TCP performs, but also on how well it performs them. The TCP protocol provides a reliable data delivery service between two applications using an underlying network layer (IP) that may lose, duplicate, or reorder packets. In order to provide an error-free exchange of data, TCP resends data it believes has been lost. To decide what data it needs to resend, TCP depends on a continuous flow of acknowledgments from receiver to sender. When data segments or acknowledgments are lost, TCP initiates a *retransmission* of the data that has not been acknowledged. TCP has two separate mechanisms for accomplishing retransmission, one based on time and one based on the structure of the acknowledgments. The second approach is usually much more efficient than the first.

TCP sets a timer when it sends data, and if the data is not acknowledged when the timer expires, a *timeout* or *timer-based retransmission* of data occurs. The timeout occurs after an interval called the *retransmission timeout* (RTO). It has another way of initiating a retransmission called *fast retransmission* or *fast retransmit*, which usually happens without any delay. Fast retransmit is based on inferring losses by noticing when TCP's cumulative acknowledgment fails to advance in the ACKs received over time, or when ACKs carrying selective acknowledgment information (SACKs) indicate that out-of-order segments are present at the receiver. Generally speaking, when the sender believes that the receiver might be missing some data, a choice needs to be made between sending new (unsent) data and retransmitting. In this chapter we look closely at how TCP determines that a segment

is lost and what to send in response. The issue of how *much* to send is deferred until Chapter 16, where we discuss TCP's congestion control procedures that are commonly invoked when packet loss is suspected. Here, we investigate how the RTO is set based on measurements of a connection's round-trip time (RTT), the mechanics of a timer-based retransmission, and how TCP's fast retransmission mechanism works. We also look at how SACKs are used to help a TCP sender determine what data is missing at the receiver, the effect of reordering and duplication of IP packets on TCP's behavior, and the way TCP can change its packet size when retransmitting. We also look briefly at some attacks that can be mounted to fool TCP into behaving more aggressively or more passively.

14.2 Simple Timeout and Retransmission Example

We have already seen some examples of timeout and retransmission. (1) In the ICMP Destination Unreachable (Port Unreachable) example in Chapter 8 we saw the TFTP client using UDP employing a simple (and poor) timeout and retransmission strategy: it assumed 5s was an adequate timeout period and retransmitted every 5s. (2) In the attempted connection to a nonexistent host in Chapter 13, we saw that when TCP tried to establish the connection it retransmitted its SYN segment using a longer and longer delay between each successive retransmission. (3) In Chapter 3, we saw what happens when Ethernet encounters a collision. All of these mechanisms are initiated by the expiration of a timer.

We shall first look at the timer-based retransmission strategy used by TCP. We will establish a connection, send some data to verify that everything is OK, isolate one end of the connection, send some more data, and watch what TCP does. In this case, we will use Wireshark to see how the connection progresses (see Figure 14-1).

Segments 1, 2, and 3 correspond to the normal TCP connection establishment handshake. When the Web server completes the connection establishment, it remains silent, awaiting a Web request. Before we provide the request, we isolate (disconnect) the server host. The input at the client side is as follows:

```
Linux% telnet 10.0.0.10 80
Trying 10.0.0.10...
Connected to 10.0.0.10.
Escape character is '^J'.
GET / HTTP/1.0
Connection closed by foreign host.
```

This request cannot be delivered to the server, so it remains in TCP's queue at the client for quite some time. During this period, the `netstat` command on the client indicates that the queue is not empty:

```
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 18 10.0.0.9:1043 10.0.0.10:www ESTABLISHED
```

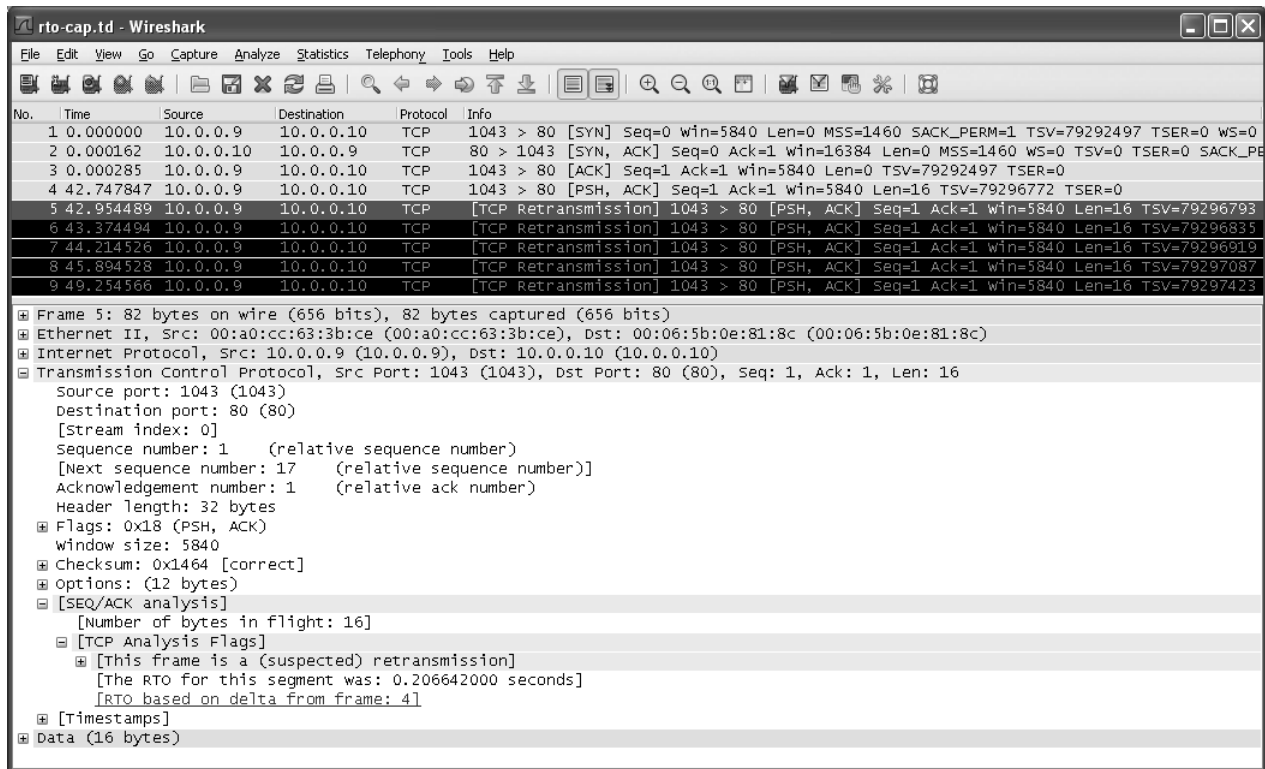


Figure 14-1 A simple example of TCP's timeout and retransmission mechanism. The first retransmit occurs at time 42.954, followed by other retransmissions at times 43.374, 44.215, 45.895, and 49.255. The intervals between successive retransmissions are 206ms, 420ms, 841ms, 1.68s, and 3.36s, respectively. These times represent a doubling of the timeout between successive retransmissions of the same segment.

Here we see that 18 bytes are in the send queue, waiting to be delivered to the Web server. The 18 bytes consist of the characters displayed in the preceding request, plus two sets of carriage-return and newline characters. Details of the rest of the output, including addresses and state information, are described in the following paragraphs.

Segment 4 is the client's first attempt to send the Web request, at 42.748s. The next try is at 42.954, 0.206s later. Then it launches another try at 43.374, which is 0.420s later. Additional retries (retransmissions) occur at 44.215, 45.895, and 49.255s. These represent time differences of 0.841, 1.680, and 3.360s, respectively.

This doubling of time between successive retransmissions is called a *binary exponential backoff*, and we saw it in Chapter 13 during a failed TCP connection establishment attempt. We shall explore it in more detail later. If we measure the elapsed time between the initial request and the time at which the connection is finally aborted, the total time is about 15.5 minutes. After that, the following error message is displayed at the client:

```
Connection closed by foreign host.
```

Logically, TCP has two thresholds to determine how persistently it will attempt to resend the same segment. These thresholds are described in the Host Requirements RFC [RFC1122], and we mentioned them briefly in Chapter 13. Threshold R1 indicates the number of tries TCP will make (or the amount of time it will wait) to resend a segment before passing “negative advice” to the IP layer (e.g., causing it to reevaluate the IP route it is using). Threshold R2 (larger than R1) dictates the point at which TCP should abandon the connection. These thresholds are suggested to be at least three retransmissions and 100s, respectively. For connection establishment (sending SYN segments), these values may be different from those for data segments, and the R2 value for SYN segments is required to be at least 3 minutes.

In Linux, the R1 and R2 values for regular data segments are available to be changed by applications or can be changed using the system-wide configuration variables `net.ipv4.tcp_retries1` and `net.ipv4.tcp_retries2`, respectively. These are measured in the number of retransmissions, and not in units of time. The default value for `tcp_retries2` is 15, which corresponds roughly to 13–30 minutes, depending on the connection's RTO. The default value for `net.ipv4.tcp_retries1` is 3. For SYN segments, `net.ipv4.tcp_syn_retries` and `net.ipv4.tcp_synack_retries` bounds the number of retransmissions of SYN segments; their default value is 5 (roughly 180s). Windows also has a number of variables that affect the overall behavior of TCP, including values for R1 and R2. These are all available by modifying values under the following registry keys [WINREG]:

```
HKLM\System\CurrentControlSet\Services\Tcpip\Parameters  
HKLM\System\CurrentControlSet\Services\Tcpip6\Parameters
```


Of immediate interest is the value called `TcpMaxDataRetransmissions`. This corresponds to the value of `tcp_retries2` in Linux. It has a default value of 5. Even in the simple retransmission example we have seen so far, TCP is required to assign some timeout value to its retransmission timer to dictate how long it should await an ACK for data it sends. If TCP were only ever used in one static environment, it would be possible to determine one particular correct value for the timeout value. Because TCP needs to operate in a large variety of environments, which themselves may change over time, TCP needs to determine this timeout value based on the current situation. For example, if a network link failed and traffic were rerouted, the RTT would change (possibly in a major way). In other words, TCP needs to *dynamically* determine its RTO. We consider this problem next.

14.3 Setting the Retransmission Timeout (RTO)

Fundamental to TCP's timeout and retransmission procedures is how to set the RTO based upon measurement of the RTT experienced on a given connection. If TCP retransmits a segment earlier than the RTT, it may be injecting duplicate traffic into the network unnecessarily. Conversely, if it delays sending until much longer than one RTT, the overall network utilization (and single-connection throughput) drops when traffic is lost. Knowing the RTT is made more complicated because it can change over time, as routes and network usage vary. TCP must track these changes and modify its timeout accordingly in order to maintain good performance.

Because TCP sends acknowledgments when it receives data, it is possible to send a byte with a particular sequence number and measure the time required to receive an acknowledgment that covers that sequence number. Each such measurement is called an RTT *sample*. The challenge for TCP is to establish a good estimate for the range of RTT values given a set of samples that vary over time. The second step is how to set the RTO based on these values. Getting this "right" is very important for TCP's performance.

The RTT is estimated for each TCP connection separately, and one retransmission timer is pending whenever any data is in flight that consumes a sequence number (including SYN and FIN segments). The proper way to set this timer has been a subject of research for years, and improvements are made on an occasional basis. In this section, we will explore some of the more important milestones in the evolution of the method used to compute the RTO. We begin with the first ("classic") method, as detailed in [RFC0793].

14.3.1 The Classic Method

The original TCP specification [RFC0793] had TCP update a *smoothed RTT estimator* (called *SRTT*) using the following formula:

$$SRTT \leftarrow \alpha(SRTT) + (1 - \alpha) RTT_s$$

Here, $SRTT$ is updated based on both its existing value and a new sample, RTT_s . The constant α is a smoothing or scale factor with a recommended value between 0.8 and 0.9. $SRTT$ is updated every time a new measurement is made. With the original recommended value for α , it is clear that 80% to 90% of each new estimate is from the previous estimate and 10% to 20% is from the new measurement. This type of average is also known as an *exponentially weighted moving average* (EWMA) or *low-pass filter*. It is convenient for implementation reasons because it requires only one previous value of $SRTT$ to be stored in order to keep the running estimate.

Given the estimator $SRTT$, which changes as the RTT changes, [RFC0793] recommended that the RTO be set to the following:

$$RTO = \min(ubound, \max(lbound, (SRTT)\beta))$$

where β is a delay variance factor with a recommended value of 1.3 to 2.0, $ubound$ is an upper bound (suggested to be, e.g., 1 minute), and $lbound$ is a lower bound (suggested to be, e.g., 1s) on the RTO. We shall call this assignment procedure the *classic method*. It generally results in the RTO being set either to 1s, or to about twice $SRTT$. For relatively stable distributions of the RTT, this was adequate. However, when TCP was run over networks with highly variable RTTs (e.g., early packet radio networks in this case), it did not perform so well.

14.3.2 The Standard Method

In [J88], Jacobson detailed problems with the classic method further—basically, that the timer specified by [RFC0793] cannot keep up with wide fluctuations in the RTT (and in particular, it causes unnecessary retransmissions when the real RTT is much larger than expected). Unnecessary retransmissions add to the network load, when the network is already loaded, as indicated by the increasing sample RTT.

To address this problem, the method used to assign the RTO was enhanced to accommodate a larger variability in the RTT. This is accomplished by keeping track of an estimate of the variability in the RTT measurements in addition to the estimate of its average. Setting the RTO based on both a mean and a variability estimator provides a better timeout response to wide fluctuations in the round-trip times than just calculating the RTO as a constant multiple of the mean.

Figures 5 and 6 in [J88] show a comparison of the [RFC0793] RTO values for some actual round-trip times, versus the RTO calculations we show next, which take into account the variability of the round-trip times. If we think of the RTT measurements made by TCP as samples of a statistical process, estimating both the mean and variance (or standard deviation) helps to make better predictions about the possible future values the process may take on. A good prediction for the range of possible values for the RTT helps TCP determine an RTO that is neither too large nor too small in most cases.

As described by Jacobson, the *mean deviation* is a good approximation to the standard deviation, but it is easier and faster to compute. Calculating the standard deviation requires executing a square root mathematical operation on the variance, which was considered to be too expensive for a fast TCP implementation. (This is not the whole story, really. See the fascinating history of “the debate” in [G04].) We therefore need running estimates of both the average as well as the mean deviation. This leads to the following equations that are applied to each RTT measurement M (called RTT_s earlier):

$$\begin{aligned} srtt &\leftarrow (1 - g)(srtt) + (g)M \\ rttvar &\leftarrow (1 - h)(rttvar) + (h)(|M - srtt|) \\ RTO &= srtt + 4(rttvar) \end{aligned}$$

Here, the value $srtt$ effectively replaces the earlier value of $SRTT$, and the value $rttvar$, which becomes an EWMA of the mean deviation, is used instead of β to help determine the RTO. This set of equations can also be written in a form that requires a smaller number of operations when implemented on a conventional computer:

$$\begin{aligned} Err &= M - srtt \\ srtt &\leftarrow srtt + g(Err) \\ rttvar &\leftarrow rttvar + h(|Err| - rttvar) \\ RTO &= srtt + 4(rttvar) \end{aligned}$$

As suggested, $srtt$ is the EWMA for the mean and $rttvar$ is the EWMA for the absolute error, $|Err|$. Err is the difference between the measured value M and the current RTT estimator $srtt$. Both $srtt$ and $rttvar$ are used to calculate the RTO, which varies over time. The gain g is the weight given to a new RTT sample M in the average $srtt$ and is set to $1/8$. The gain h is the weight given to a new mean deviation sample (absolute difference of the new sample M from the running average $srtt$) for the deviation estimate $rttvar$ and is set to $1/4$. The larger gain for the deviation makes the RTO go up faster when the RTT changes. The values for g and h are chosen as (negative) powers of 2, allowing the overall set of computations to be implemented in a computer using fixed-point integer arithmetic with shift and add operations instead of multiplies and divides.

Note

[J88] specified $2 * rttvar$ in the calculation of RTO, but after further research, [J90] changed the value to $4 * rttvar$, which is what appeared in the BSD Net/1 implementation and ultimately in the standard [RFC6298].

Comparing the classic method with Jacobson's, we see that the calculations of the RTT average are similar (α is 1 minus the gain g) but a different gain is used. Also, Jacobson's calculation of the RTO depends on both the smoothed RTT and the smoothed deviation, whereas the classic method used a simple multiple of the smoothed RTT. This is the basis for the way many TCP implementations compute their RTOs to this day, and because of its adoption as the basis for [RFC6298] we shall call it the *standard method*, although there are slight refinements in [RFC6298], which we shall now discuss.

14.3.2.1 Clock Granularity and RTO Bounds

TCP has a continuously running "clock" that is used when taking RTT measurements. As with initial sequence numbers, real TCP connections do not start their clocks at zero and the clock does not have infinite precision. Rather, the TCP clock is usually the value of a variable that is updated as the system clock advances, not necessarily one-for-one. The length of the TCP's clock "tick" is called its *granularity*. Traditionally, this value was relatively large (about 500ms), but more recent implementations use finer-granularity clocks (e.g., 1ms for Linux).

The granularity can affect the details of making RTT measurements and also how the RTO is set. In [RFC6298], the granularity is used to refine how updates to the RTO are made. In addition, a lower bound is placed on the RTO. The equation used is as follows:

$$\text{RTO} = \max(\text{srtt} + \max(G, 4(\text{rttvar})), 1000)$$

where G is the timer granularity and 1000ms represents a lower bound on the total RTO (recommended by rule (2.4) of [RFC6298]). Consequently, the RTO is always at least 1s. An optional upper bound is also allowed, provided it has a value of at least 60s.

14.3.2.2 Initial Values

We have seen how the estimators are updated as time progresses, but we also need to know how to set their initial values. Before the first SYN exchange, TCP has no good idea what value to use for setting the initial RTO. It also does not know what to use as the initial values for its estimators, unless the system has provided hints at this information (some systems cache this information in the forwarding table; see Section 14.9). According to [RFC6298], the initial setting for the RTO should be 1s, although 3s is used in the event of a timeout on the initial SYN segment. When the first RTT measurement M is received, the estimators are initialized as follows:

$$\text{srtt} \leftarrow M$$

$$\text{rttvar} \leftarrow M/2$$

We now have enough detail to see how the estimators are initialized and maintained. The procedures depend on obtaining RTT samples, which would appear to be straightforward. We now look at why this might not always be the case.

14.3.2.3 Retransmission Ambiguity and Karn's Algorithm

A problem measuring an RTT sample can occur when a packet is retransmitted. Say a packet is transmitted, a timeout occurs, the packet is retransmitted, and an acknowledgment is received for it. Is the ACK for the first transmission or the second? This is an example of the *retransmission ambiguity problem*. It happens because unless the Timestamps option is being used, an ACK provides only the ACK number with no indication of which copy (e.g., first or second) of a sequence number is being ACKed.

The paper [KP87] specifies that when a timeout and retransmission occur, we cannot update the RTT estimators when the acknowledgment for the retransmitted data finally arrives. This is the “first part” of Karn's algorithm. It eliminates the acknowledgment ambiguity problem by removing the ambiguity for purposes of computing the RTT estimate. It is a requirement in [RFC6298].

If we were to simply ignore retransmitted segments entirely when setting the RTO, however, we would be failing to take into account some useful information being provided by the network (i.e., that it is probably experiencing some form of inability to deliver packets quickly). In such cases, it would be beneficial to reduce the load on the network by decreasing the retransmission rate, at least until packets are no longer being lost. This reasoning is the basis for the exponential backoff behavior we saw in Figure 14-1.

TCP applies a *backoff factor* to the RTO, which doubles each time a subsequent retransmission timer expires. Doubling continues until an acknowledgment is received for a segment that was not retransmitted. At that time, the backoff factor is set back to 1 (i.e., the binary exponential backoff is canceled), and the retransmission timer returns to its normal value. Doubling the backoff factor on subsequent retransmissions is the “second part” of Karn's algorithm. Note that when TCP times out, it also invokes congestion control procedures that alter its sending rate. (Congestion control is discussed in detail in Chapter 16.) Karn's algorithm, then, really consists of two parts. As quoted directly from the 1987 paper [KP87]:

When an acknowledgement arrives for a packet that has been sent more than once (i.e., is retransmitted at least once), ignore any round-trip measurement based on this packet, thus avoiding the retransmission ambiguity problem. In addition, the backed-off RTO for this packet is kept for the next packet. Only when it (or a succeeding packet) is acknowledged without an intervening retransmission will the RTO be recalculated from SRTT.

This algorithm has been a required procedure in a TCP implementation for some time (since [RFC1122]). There is an exception, however, when the TCP Timestamps option is being used (see Chapter 13). In that case, the acknowledgment ambiguity problem can be avoided and the first part of Karn's algorithm does not apply.

14.3.2.4 RTT Measurement (RTTM) with the Timestamps Option

The TCP Timestamps option (TSOPT), in addition to providing a basis for the PAWS algorithm we saw in Chapter 13, can be used for *round-trip time measurement* (RTTM) [RFC1323]. The basic format of the TSOPT was described in Chapter 13. It allows the sender to include a 32-bit number in a TCP segment that is returned in a corresponding acknowledgment.

The timestamp value (TSV) is carried in the TSOPT of the initial SYN and returned in the TSER part of the TSOPT in the SYN + ACK, which is how the initial values for *srtt*, *rttvar*, and RTO are determined. Because the initial SYN “counts” as data (i.e., it is retransmitted if lost and consumes a sequence number), its RTT is measured. TSOPTs are also carried in other segments, so the connection’s RTT can be estimated on an ongoing basis. This seems straightforward enough but is made more complex because TCP does not always provide an ACK for each segment it receives. For example, TCP often provides one ACK for every other segment (see Chapter 15) when large volumes of data are transferred. In addition, when data is lost, reordered, or successfully retransmitted, the cumulative ACK mechanism of TCP means that there is not necessarily any fixed correspondence between a segment and its ACK. To handle these challenges, TCPs that use this option (most of them today—Linux and Windows included), employ the following algorithm for taking RTT samples:

1. The sending TCP includes a 32-bit timestamp value in the TSV portion of the TSOPT in each TCP segment it sends. This field contains the value of the sender’s TCP “clock” when the segment is transmitted.
2. A receiving TCP keeps track of the received TSV value to send in the next ACK it generates (in a variable typically named *TsRecent*) and the ACK number in the last ACK that it sent (in a variable named *LastACK*). Recall that ACK numbers represent the next in-order sequence number the receiver (i.e., sender of the ACK) expects to see.
3. When a new segment arrives, if it contains the sequence number matching the value in *LastACK* (i.e., it is the next expected segment), the segment’s TSV is saved in *TsRecent*.
4. Whenever the receiver sends an ACK, a TSOPT is included such that the timestamp value contained in *TsRecent* is placed in the TSER part of the TSOPT in the ACK.
5. A sender receiving an ACK that advances its window subtracts the TSER from its current TCP clock and uses the difference as a sample value to update its RTT estimators.

Timestamps are enabled by default in FreeBSD, Linux, and in response to systems that use them for later versions of Windows. In Linux, the system configuration variable `net.ipv4.tcp_timestamps` dictates whether or not they are used

(value 0 for not used, value 1 for used). In Windows, their use is controlled by the `Tcp1323Opts` value in the registry area mentioned earlier. If it has the value 0, timestamps are disabled. If its value is 2, timestamps are enabled. This key has no default value (it is not in the registry by default). The default behavior is to use timestamps if a peer uses them when initiating a connection.

14.3.3 The Linux Method

The Linux RTT estimation procedure works somewhat differently from the standard method. It uses a clock granularity of 1ms, which is finer than that of many other implementations, along with the `TSOPT`. The combination of frequent measurements of the RTT and the fine-grain clock contributes to a more accurate estimate of the RTT but also tends to minimize the value of *rttvar* over time [LS00]. This happens because when a large enough number of mean deviation samples are accumulated, they tend to cancel each other out. This is one consideration for setting the RTO that differs somewhat from the standard method. Another relates to the way the standard method increases *rttvar* when an RTT sample is significantly *below* the existing RTT estimate *srtt*.

To understand the second issue better, recall that the RTO is usually set to the value $srtt + 4(rttvar)$. Consequently, any large change in *rttvar* causes the RTO to increase, whether the latest RTT sample is greater or less than *srtt*. This is counter-intuitive—if the actual RTT has dropped significantly, it is not desirable to have the RTO increase as a consequence. Linux deals with this issue by limiting the impact of significant downward drops in RTT sample values on the value of *rttvar*. We will now look at the details for the procedure Linux uses to set its RTO; the procedure addresses both of the issues just discussed.

Linux keeps the variables *srtt* and *rttvar*, as with the standard method, but also two new ones called *mdev* and *mdev_max*. The value *mdev* keeps the running estimate of the mean deviation using the standard algorithm for *rttvar* described before. The value *mdev_max* holds the maximum value of *mdev* seen over the last measured RTT and is never allowed to be less than 50ms. In addition, *rttvar* is regularly updated to ensure that it is at least as large as *mdev_max*. Consequently, the RTO never dips below 200ms.

Note

The minimum RTO can be changed. `TCP_RTO_MIN`, which is a kernel configuration constant, can be changed prior to recompiling and installing the kernel. Some Linux versions also allow it to be changed using the `ip route` command. When TCP is used in data-center networks where RTTs may be a few microseconds, 200ms minimum RTO can lead to severe performance degradations due to slow TCP recovery after packet loss in local switches. This is the so-called TCP “incast” problem. Various solutions exist to this problem, including modification of the TCP timer granularity and minimum RTO to be on the order of microseconds [V09]. Such small minimum RTO values are not recommended for use on the global Internet.

Linux updates *rttvar* to the value of *mdev_max* whenever the maximum increases. It always sets the RTO to be the sum of *srtt* and $4(rttvar)$ and ensures that the RTO never exceeds TCP_RTO_MAX, which defaults to 120s. See [SK02] for more details. We can see how the details of all of this work in Figure 14-2. This figure also shows how the Timestamps option operates.

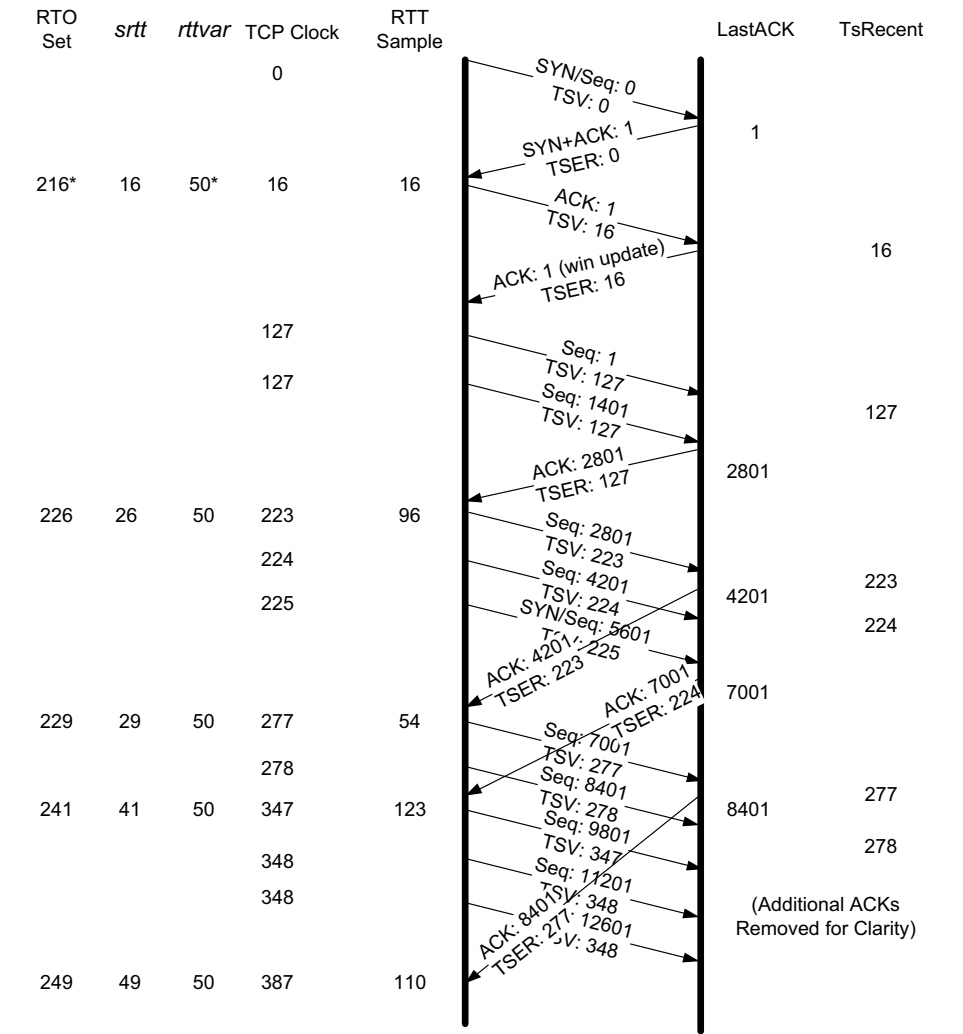


Figure 14-2 The TCP Timestamps option carries a copy of the TCP clock at the sender. ACKs return this value to the sender, which uses the difference (current clock - returned timestamp) to update its *srtt* and *rttvar* estimates. For clarity, only one set of timestamps is depicted. In this Linux system, the *rttvar* value is constrained to be at least 50 (millisecond) units, and the RTO has a lower bound of 200ms.

In Figure 14-2 we see a TCP connection using the Timestamps option as it starts up. The sender is a Linux 2.6 system and the receiver is a FreeBSD 5.4 system. Sequence numbers and timestamp values are depicted as relative values for clarity. In addition, only the sender's timestamps are shown. The figure is not drawn exactly to temporal scale, in order to make the numerical values easier to read. Based on the initial RTT measurement in this example, Linux makes the following updates:

- $srtt = 16\text{ms}$
- $mdev = (16/2)\text{ms} = 8\text{ms}$
- $rttvar = mdev_max = \max(mdev, \text{TCP_RTO_MIN}) = \max(8, 50) = 50\text{ms}$
- $\text{RTO} = srtt + 4(rttvar) = 16 + 4(50) = 216\text{ms}$

After the initial SYN exchange, the sender supplies an ACK for the receiver's SYN and the receiver responds with a window update. As neither of these packets contains data (or SYN or FIN bit fields, which are counted as data), they are not timed, and no RTT estimator update is performed when the window update arrives back at the sender. Segments that do not contain data are not reliably delivered by TCP, meaning they are not retransmitted if lost. These types of segments do not require a retransmission timer to be set, because they are never retransmitted.

Note

It is worth mentioning that TCP options, by themselves, are also not retransmitted or reliably delivered. Only when options are specifically arranged to be present in data segments (including SYN and FIN segments) will options be retransmitted if lost, and then only as a side effect.

When the application performs its first write, the sending TCP emits two segments, each with a TSV value equal to 127. The values are identical in these two segments because the TCP clock has advanced less than 1ms (the sending TCP's clock granularity) between the first and second transmission. It is not unusual to see the clock fail to advance, or advance by small amounts, when the sender is sending multiple segments "back-to-back" in this fashion.

The *LastACK* variable at the receiver holds the ACK number last sent by the receiver. In this example, *LastACK* starts with the value 1 because the last ACK sent was the SYN + ACK packet sent during connection establishment. When the first full-size segment arrives, its sequence number matches the *LastACK* value, so the *TsRecent* variable is updated to contain the value 127 from the arriving segment's TSV. The arrival of the second segment does not update the *TsRecent* variable because its *Sequence Number* field does not match the value in *LastACK*. The ACK sent in response to the arriving packets includes the value of *TsRecent* in its TSER, and its transmission also causes the receiver to update the *LastACK* variable to the ACK number, 2801.

When this ACK arrives, TCP is able to make its second RTT measurement. It takes the current TCP clock and subtracts the TSER value from the arriving packet, forming the measurement m : $m = 223 - 127 = 96$. With this measurement, the Linux TCP updates the connection variables as follows:

- $mdev = mdev (3/4) + |m - srtt|(1/4) = 8(3/4) + |80|(1/4) = 26\text{ms}$
- $mdev_max = \max(mdev_max, mdev) = \max(50, 26) = 50\text{ms}$
- $srtt = srtt (7/8) + m(1/8) = 16(7/8) + 96(1/8) = 14 + 12 = 26\text{ms}$
- $rttvar = mdev_max = 50\text{ms}$
- $RTO = srtt + 4(rttvar) = 26 + 4(50) = 226\text{ms}$

As mentioned previously, Linux TCP has several special modifications to the classic RTT estimation algorithm that merit discussion. At the time the classic algorithms were developed, the typical granularity of the TCP clock was 500ms and the Timestamps option was not in widespread use. It was typical to take only one RTT sample per window and update the estimators accordingly. This is still used if timestamps are not available or not enabled.

If only one RTT sample is taken per window, the $rttvar$ term changes relatively slowly. With timestamps and per-packet timestamp measurements, many more measurements can take place. Because it is common for the RTT to vary little from one packet to the next in the same window of data, taking so many measurements in a small period of time (e.g., when the window is large) can lead to the mean deviation estimate being small (near zero, thanks to the law of large numbers [F68]). To address this issue, Linux maintains the $mdev$ variable as the running mean deviation estimate but sets the RTO based on the $rttvar$, which is increased to the maximum value of $mdev$ during one window of data and also clamped to be at least 50ms. $Rttvar$ is allowed to decrease only one time, from one window to the next.

The standard approach uses a heavy weight (factor of 4) given to the $rttvar$ term, and consequently the RTO tends to increase, even when the RTT is decreasing. With a coarse-granularity clock (e.g., 500ms) this may have relatively little effect because there are so few values the RTO can take on. However, with a finer-granularity clock, such as the 1ms used by Linux, this can be of concern. To address this issue, Linux handles the case where the RTT is decreasing by giving less weight to the new sample if it is below the “lower end” of the estimated RTT range ($srtt - mdev$). The complete relationship is as follows:

$$\begin{aligned}
 &\text{if } (m < (srtt - mdev)) \\
 &\quad mdev = (31/32) * mdev + (1/32) * |srtt - m| \\
 &\text{else} \\
 &\quad mdev = (3/4) * mdev + (1/4) * |srtt - m|
 \end{aligned}$$

The conditional determines if the new RTT sample is below the bottom of the range of what an RTT measurement is expected to be. If so, the new sample indicates that the connection may be experiencing a significantly reducing RTT. To avoid increasing *mdev* (and consequently *rttvar* and RTO) in such cases, the new mean deviation sample, $|srtt - ml|$, is given an 8x reduced weight versus its normal weighting. Overall, this results in avoiding the problem of increasing the RTO in cases where the RTT is decreasing. For an in-depth discussion of these issues, please see [LS00] and [SK02]. In [RKS07], the authors evaluated the TCP RTT estimation algorithms with various operating systems on 2.8 million TCP flows. They conclude that the Linux estimator is the most effective among those studied, largely because of its relatively quick convergence, but that it can also be tuned most effectively by reducing the influence of RTT variance on setting the RTO.

Returning now to Figure 14-2, when ACK 7001 is generated at the receiver, we see that its TSER contains a copy of a TSV value, not from the most recently arriving segment, but instead from the oldest segment that has not been ACKed. When returned to the sender, this ACK causes the RTT sample to be measured from the first of the two segments, rather than from the last one sent. This is how the timestamp algorithm works with delayed or otherwise erratic ACKs. When the RTT sample from the oldest packet is measured, the RTT sample is taken to be the time the sender should wait to expect an ACK, rather than the actual network RTT. This is important because the sender needs to base its RTO on the rate at which it can expect ACKs from the receiver, which may be less than the packet sending rate.

14.3.4 RTT Estimator Behaviors

As we have seen, substantial innovation and engineering have been invested in how to set TCP's RTO and how to estimate the RTT. Figure 14-3 shows how the more popular estimators work, based on applying the standard and Linux algorithms to a synthetic data set. The 1s RTO minimum recommended by [RFC6298] has been removed for the standard method for illustration. Most real-world TCP implementations today violate this directive anyhow [RKS07].

The graph shows a time-series plot of 200 synthetic values drawn from two Gaussian probability distributions, $N(200, 50)$ and $N(50, 50)$. The first distribution is used for the first 100 points, and the second is used for the second 100 points. Any negative samples were made positive by sign inversion (applicable only to the second distribution). Each plus (+) indicates a specific sample value. The significant drop in sample values after sample 100 is apparent, and it is easy to see how the Linux approach drops the RTO almost immediately after sample 100, while the standard approach requires another 20 samples.

If we focus now on the Linux *rttvar* line, we can see that it remains relatively constant. This is because of the 50ms minimum on the *mdev_max* value (and consequently the *rttvar* value). This has the effect of making the Linux RTO value always at least 200ms, and all unnecessary retransmissions are avoided (although the timer may not fire as quickly, leading to reduced performance when packets

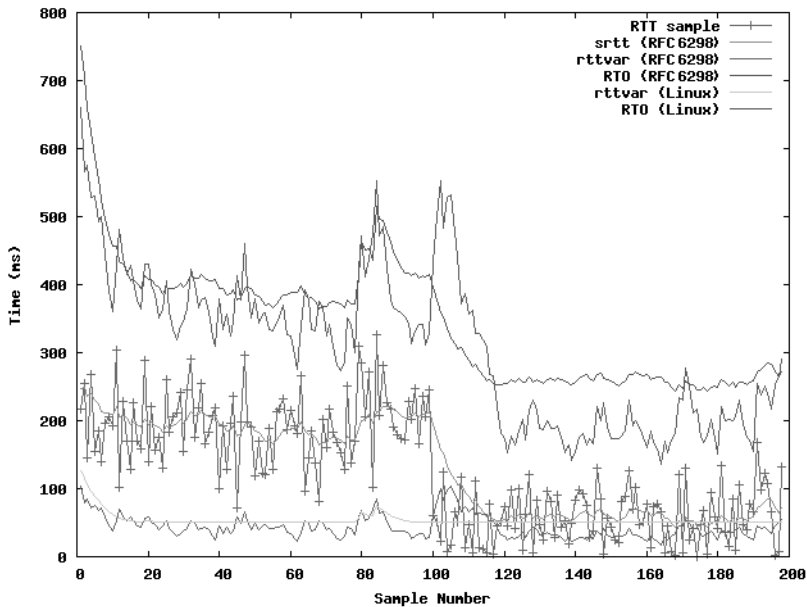


Figure 14-3 The Linux and standard RTO assignment and RTT estimation algorithms applied to synthetic (pseudorandom) sample points. The first 100 points are drawn from an $N(200, 50)$ distribution, and the second 100 are drawn from an $N(50, 50)$ distribution with negative values turned positive. Linux avoids the increase in RTO when the mean drops after sample 100. With Linux, the minimum RTO is effectively set to 200ms, so after sample 120, the standard method is tighter. Linux avoids setting the RTO too low in all cases for this example. The standard approach runs into potential problems at samples 78 and 191.

are lost). The standard approach runs into potential problems at samples 78 and 191, where a *spurious retransmission* could take place. We shall discuss this problem later.

14.3.5 RTTM Robustness to Loss and Reordering

The TSOPT has been shown to work properly when packets are not lost, whether or not the receiver delays some ACKs. The algorithm also operates correctly in the following cases:

- **Out-of-order segments:** When a receiver receives an out-of-order segment, typically because of the loss of a previous segment, an ACK is supposed to be generated immediately to help the fast retransmit algorithm (see Section 14.5) operate. This ACK includes as its TSER value the TSV value from the most recent in-order segment that arrived at the receiver (i.e., the most recent one to advance the window, which is generally *not* the arriving

out-of-order segment). This tends to cause the sender's RTT sample values to increase, leading to a corresponding increase in the sender's RTO. When packets are being reordered, this is beneficial because it tends to allow the sender a bit more time to realize that packets are reordered rather than lost before initiating a retransmission.

- **Successful retransmissions:** When a receiver receives a segment that fills a hole in its receive buffer (e.g., because of the successful arrival of a retransmission), the window generally jumps forward. In this case, the value carried in the TSER of the corresponding ACK is from the most recently arriving segment. This is useful because if an older segment's TSV were used, it might be more than one RTO's worth of time old, leading to a large unwanted bias in the sender's RTT estimate.

The example in Figure 14-4 illustrates these points. Assume that three segments, each containing 1024 bytes, are received in the following order: segment 1 with bytes 1–1024, segment 3 with bytes 2049–3072, and then segment 2 with bytes 1025–2048.

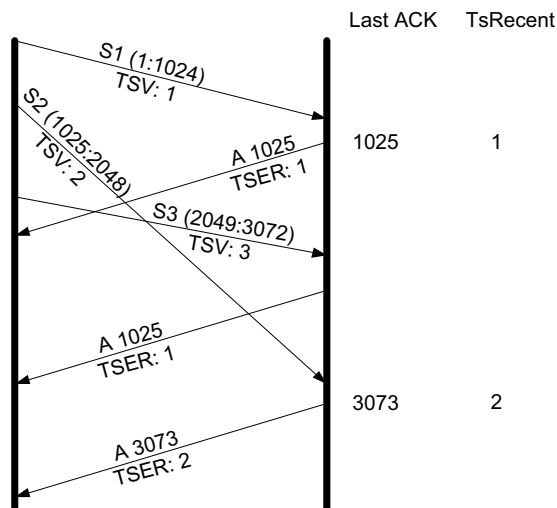


Figure 14-4 When segments are reordered, the returned timestamp is that of the last segment to advance the receiver's window (not the largest timestamp to arrive at the receiver). This biases the sender's RTO toward overestimating the RTT during periods of packet reordering and reduces its aggressiveness.

The ACKs sent back in Figure 14-4 are ACK 1025 with the timestamp from segment 1 (a normal ACK for data that was expected), ACK 1025 with the timestamp from segment 1 (a duplicate ACK in response to the in-window but out-of-sequence segment), then ACK 3073 with the timestamp from segment 2 (not the

timestamp from segment 3). This has the effect of overestimating the RTT when segments are reordered (or lost). A larger RTT estimate leads to a larger RTO, making the sender less aggressive to retransmit. This is especially desirable in cases where packet reordering occurs, because aggressive retransmissions are likely to be spurious.

So, we have seen that the Timestamps option allows the sender to make estimates of the RTT even when there are packet delays, losses, and reorderings. The sender can measure the RTT using whatever values it wishes to in the option, but these units must at least be proportional to real time and of a reasonable granularity to be compatible with TCP sequence numbers and plausible link rates (see [RFC1323] for more details on this). In particular, to be useful to the sender, the TCP clock must “tick” at least once for any plausible RTT. On the other hand, it should not change faster than once every 59ns. If it did, the 32-bit TSV value holding the TCP clock value could wrap around within the maximum time permitted by the IP layer for a single packet to exist (255s) [ID1323b]. Assuming all this to be correct, the RTO value can now be used to trigger retransmissions.

14.4 Timer-Based Retransmission

Once a sending TCP has established its RTO based upon measurements of the time-varying values of effective RTT, whenever it sends a segment it ensures that a retransmission timer is set appropriately. When setting a retransmission timer, the sequence number of the so-called timed segment is recorded, and if an ACK is received in time, the retransmission timer is canceled. The next time the sender emits a packet with data in it, a new retransmission timer is set, the old one is canceled, and the new sequence number is recorded. The sending TCP therefore continuously sets and cancels one retransmission timer per connection; if no data is ever lost, no retransmission timer ever expires.

Note

This observation proved somewhat of a surprise to the designers of the host operating systems. In a typical operating system, timers are used to signal a wide variety of events, and the implementation of the timer facility is tuned to efficiently set up and expire timers (which invoke system functions). For TCP, however, the requirement is for efficient setting and resetting or canceling of timers; if TCP is working well, timers never expire.

When TCP fails to receive an ACK for a segment it has timed on a connection within the RTO, it performs a timer-based retransmission. We have seen this already in Figure 14-1. TCP considers a timer-based retransmission as a fairly major event; it reacts very cautiously when it happens by quickly reducing the rate at which it sends data into the network. It does this in two ways. The first way is

to reduce its sending window size based on congestion control procedures (see Chapter 16). The other way is to keep increasing a multiplicative backoff factor applied to the RTO each time a retransmitted segment is again retransmitted. This is implemented in the “second part” of Karn’s algorithm mentioned previously. In particular, the RTO value is (temporarily) multiplied by the value γ to form the backed-off timeout when multiple retransmissions of the same segment occur:

$$RTO = \gamma RTO$$

In ordinary circumstances, γ has the value 1. On subsequent retransmissions, γ is doubled: 2, 4, 8, and so forth. There is typically a maximum backoff factor that γ is not allowed to exceed (Linux ensures that the used RTO never exceeds the value `TCP_RTO_MAX`, which defaults to 120s). Once an acceptable ACK is received, γ is reset to 1.

14.4.1 Example

We can see the action of the retransmission timer by creating a connection similar to the one we looked at in Figures 14-1 and 14-2, but where we purposely drop the segment with sequence number 1401 twice (see Figure 14-5).

For this example, we send the TCP segments through a special function that is able to drop them a certain number of times based on their TCP sequence numbers. This adds a bit of extra delay to the RTT as compared with Figure 14-2. The connection starts out as before, except when the pair of segments with sequence numbers 1 and 1401 is sent, the second packet is dropped. Presumably the first of these segments reaches the receiver, but the receiver is delaying ACKs and does not respond immediately. Lacking a response in 219ms, the sender’s retransmission timer expires, causing the packet with sequence number 1 to be resent (this time with TSV value 577). Its arrival elicits an ACK from the receiver, which returns to the sender. Because this ACK acknowledges data and moves the sender’s window forward, its TSER value is used to update the *srtt* and RTO values to 34 and 234, respectively.

The next three ACKs are generated in response to packets that arrive at the receiver. The ACKs with the asterisks (*) are all duplicate ACKs and contain SACK information. We will discuss the effect of duplicate ACKs and SACKs in Sections 14.5 and 14.6. For now, because these ACKs do not move the sender’s window forward, their TSER values are not used.

With the eventual retransmission and arrival of segment 1401 (at TCP clock time 911) at the receiver, the repair period is complete, and the receiver responds with ACK number 7001, indicating that all data has been received.

The retransmission timer provides a form of “last-resort restart” for a TCP connection that has ceased to move data through the network regularly. In most cases it is unnecessary (and undesirable) to have retransmission timers trigger retransmissions because the RTO is generally established to be larger than the

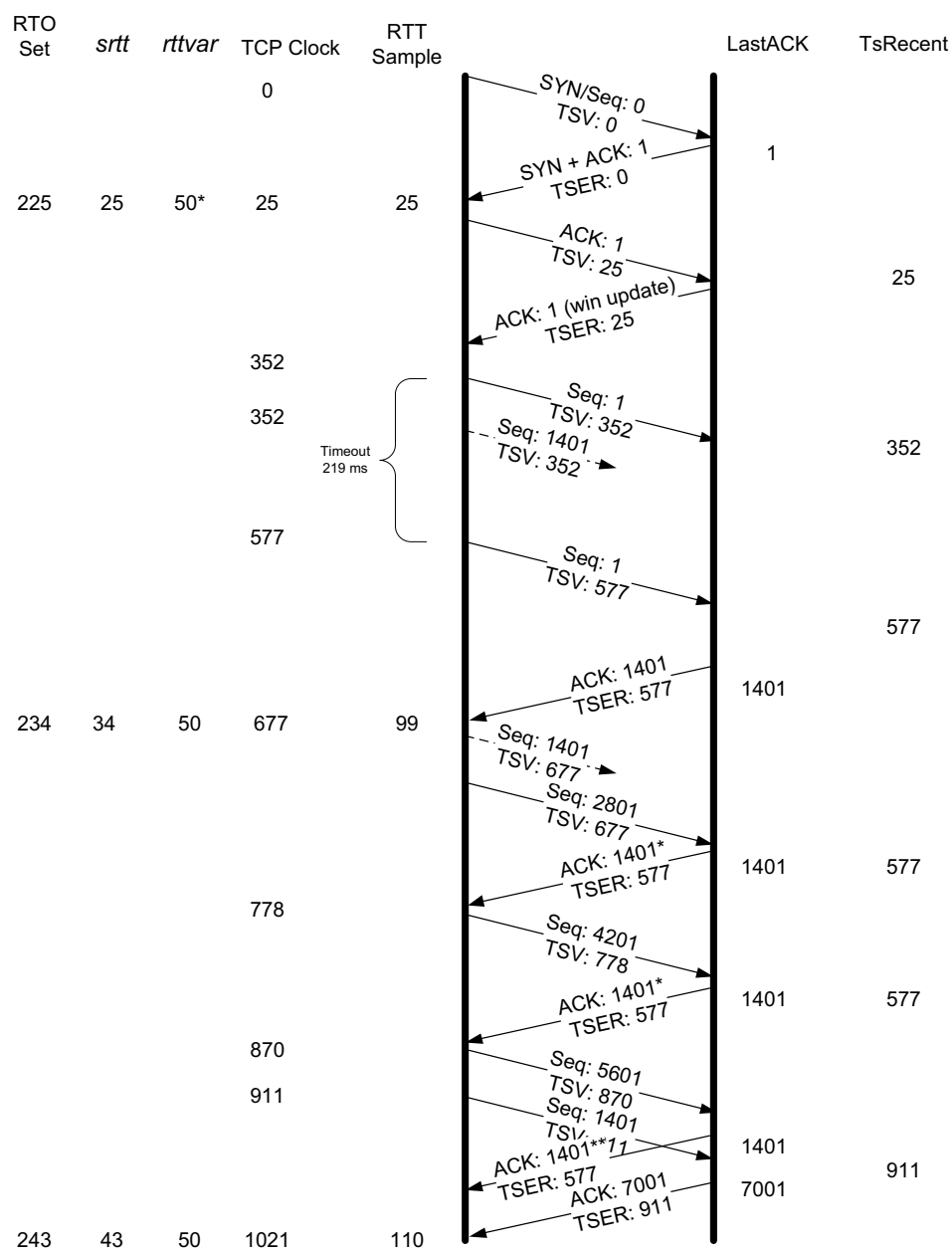


Figure 14-5 Segment 1401 is forcibly dropped twice. This results in a timer-based retransmission at the sender. The *srtt*, *rttvar*, and RTO values are updated only by a returning ACK that advances the sender's window. ACKs with asterisks (*) include SACK information.

typical RTT (by about a factor of 2 or more), so a timer-based retransmission often leads to underutilization of the network capacity. Fortunately, TCP has another method for detecting and repairing lost packets, which is almost always more efficient than timer-based retransmissions. It is called fast retransmit because it does not require the expiration of a retransmission timer to be invoked.

14.5 Fast Retransmit

Fast retransmit [RFC5681] is a TCP procedure that can induce a packet retransmission based on feedback from the receiver instead of requiring a retransmission timer to expire. As a result, packet loss can often be more quickly and efficiently repaired using fast retransmit than with timer-based retransmission. A typical TCP implements both fast retransmit and timer-based retransmission. Before we describe fast retransmit in more detail, it is important to realize that TCP is required to generate an immediate acknowledgment (a “duplicate ACK”) when an out-of-order segment is received, and that the loss of a segment implies out-of-order arrivals at the receiver when subsequent data arrives. When this happens, a hole is created at the receiver. The sender’s job then becomes filling the receiver’s holes as quickly and efficiently as possible.

The duplicate ACKs sent immediately when out-of-order data arrives are not delayed. The reason is to let the sender know that a segment was received out of order, and to indicate what sequence number is expected (i.e., where the hole is). When SACK is used, these duplicate ACKs typically contain SACK blocks as well, which can provide information about more than one hole.

A duplicate ACK (with or without SACK blocks) arriving at a sender is a potential indicator that a packet sent earlier has been lost. As we discuss in Section 14.8 in more detail, duplicate ACKs can also appear when there is packet reordering in the network—if a receiver receives a packet for a sequence number beyond the one it is expecting next, the expected packet could be either missing or merely delayed. Because we generally do not know which one, TCP waits for a small number of duplicate ACKs (called the *duplicate ACK threshold* or *dupthresh*) to be received before concluding that a packet has been lost and initiating a fast retransmit. Traditionally, *dupthresh* has been a constant (with value 3), but some nonstandard implementations (including Linux) alter this value based on the current measured level of reordering (see Section 14.8).

A TCP sender observing at least *dupthresh* duplicate ACKs retransmits one or more packets that appear to be missing without waiting for a retransmission timer to expire. It may also send additional data that has not yet been sent. This is the essence of the fast retransmit algorithm. Packet loss inferred by the presence of duplicate ACKs is assumed to be related to network congestion, and congestion control procedures (discussed in Chapter 16) are invoked along with fast retransmit. Without SACK, no more than one segment is typically retransmitted until an acceptable ACK is received. With SACK, ACKs contain additional information

allowing the sender to fill more than one hole in the receiver per RTT. We explore the use of SACK with fast retransmit after illustrating an example of the basic fast retransmit algorithm.

14.5.1 Example

In the following example, we create a TCP connection similar to the one from Figure 14-4, except this time we drop segments 23801 and 26601 and SACK is disabled. We will see how TCP uses the basic fast retransmit algorithm to repair these holes. The sender is a Linux 2.6 system and the receiver is a FreeBSD 5.4 system. The plot in Figure 14-6 from Wireshark's Statistics | TCP Stream Graph | Time-Sequence Graph (tcptrace) screen shows fast retransmit in action.

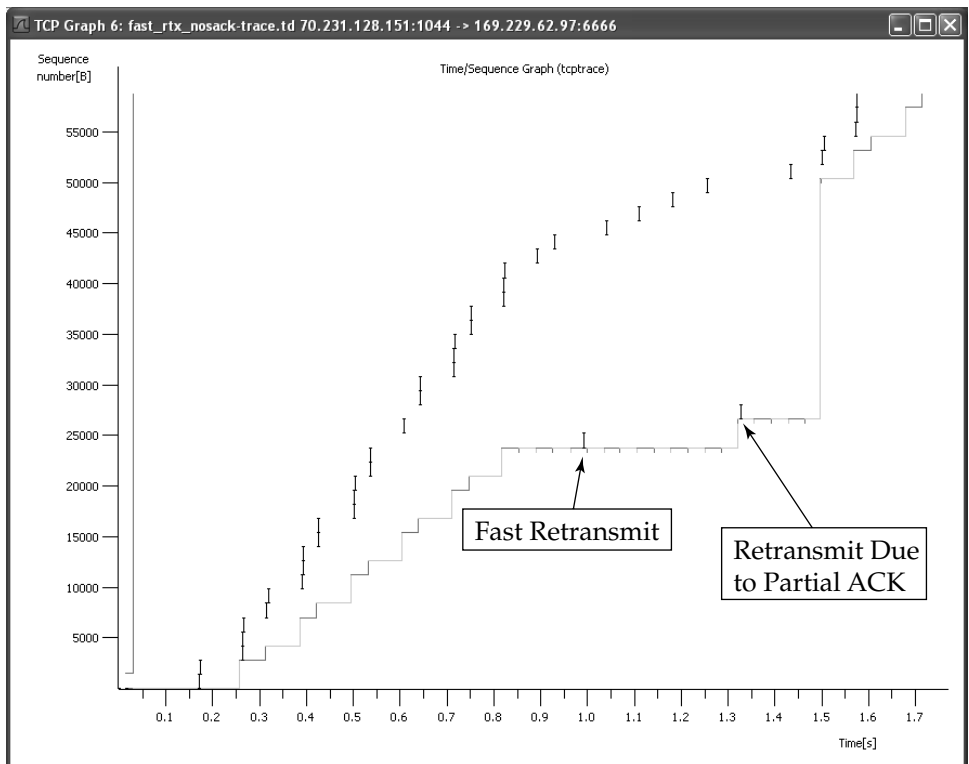


Figure 14-6 In this plot, TCP sequence numbers are on the *y*-axis and time is on the *x*-axis. Outgoing segments are displayed as darker line segments, and the incoming ACK numbers appear as lighter gray segments. Fast retransmit is triggered by the arrival of the third duplicate ACK at time 0.993s. This connection does not use SACK, so it is able to repair at most only one hole per RTT. Additional duplicate ACKs arriving after the third cause the sender to send new segments (not retransmissions). A “partial ACK” arriving at time 1.32 causes the next retransmission.

This plot indicates the relative sending sequence number on the y -axis and the elapsed time on the x -axis. The black vertical I-shaped extents indicate the span of sequence numbers present in the transmitted segment. The blue lines in Wireshark (lower light gray line in Figure 14-6) indicate ACK numbers in returning packets. At approximately time 1.0, sequence number 23801 is retransmitted because of the fast retransmit algorithm (the initial transmission is not visible because it was dropped by the process at the sender below the TCP protocol layer). The retransmission is triggered by the arrival of the third duplicate ACK, as illustrated by the repeated lower line segments. The retransmit can also be seen using the basic analysis screen of Wireshark (see Figure 14-7).

No.	Time	Source	Destination	Protocol	Info
40	0.815379	169.229.62.97	70.231.128.1	TCP	6666 > 1044 [ACK] Seq=1 Ack=23801 win=231616 Len=0 TSV=488;
41	0.820951	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=37801 Ack=1 win=5808 Len=1400 TSV=29'
42	0.821692	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=39201 Ack=1 win=5808 Len=1400 TSV=29'
43	0.822282	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=40601 Ack=1 win=5808 Len=1400 TSV=29'
44	0.853283	169.229.62.97	70.231.128.1	TCP	[TCP Window Update] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
45	0.890255	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#1] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
46	0.893441	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=42001 Ack=1 win=5808 Len=1400 TSV=29'
47	0.925942	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#2] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
48	0.929728	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=43401 Ack=1 win=5808 Len=1400 TSV=29'
49	0.963646	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#3] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
50	0.992938	70.231.128.151	169.229.62.9	TCP	[TCP Retransmission] 1044 > 6666 [ACK] Seq=23801 Ack=1 win=2
51	0.998380	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#4] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
52	1.036083	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#5] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
53	1.040409	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=44801 Ack=1 win=5808 Len=1400 TSV=29'
54	1.069333	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#6] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
55	1.106530	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#7] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
56	1.110553	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=46201 Ack=1 win=5808 Len=1400 TSV=29'
57	1.142507	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#8] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
58	1.177764	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#9] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
59	1.182439	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=47601 Ack=1 win=5808 Len=1400 TSV=29'
60	1.212966	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#10] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
61	1.250471	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#11] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
62	1.254697	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=49001 Ack=1 win=5808 Len=1400 TSV=29'
63	1.286622	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 44#12] 6666 > 1044 [ACK] Seq=1 Ack=23801 win=2
64	1.321104	169.229.62.97	70.231.128.1	TCP	6666 > 1044 [ACK] Seq=1 Ack=26601 win=230216 Len=0 TSV=488;
65	1.321602	169.229.62.97	70.231.128.1	TCP	[TCP window update] 6666 > 1044 [ACK] Seq=1 Ack=26601 win=2
66	1.326378	70.231.128.151	169.229.62.9	TCP	[TCP Retransmission] 1044 > 6666 [ACK] Seq=26601 Ack=1 win=2
67	1.356099	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 65#1] 6666 > 1044 [ACK] Seq=1 Ack=26601 win=2
68	1.392063	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 65#2] 6666 > 1044 [ACK] Seq=1 Ack=26601 win=2
69	1.430013	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 65#3] 6666 > 1044 [ACK] Seq=1 Ack=26601 win=2
70	1.434094	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=50401 Ack=1 win=5808 Len=1400 TSV=29'
71	1.463026	169.229.62.97	70.231.128.1	TCP	[TCP Dup ACK 65#4] 6666 > 1044 [ACK] Seq=1 Ack=26601 win=2
72	1.497273	169.229.62.97	70.231.128.1	TCP	6666 > 1044 [ACK] Seq=1 Ack=50401 win=209216 Len=0 TSV=488;
73	1.497989	169.229.62.97	70.231.128.1	TCP	[TCP Window Update] 6666 > 1044 [ACK] Seq=1 Ack=50401 win=2
74	1.501522	70.231.128.151	169.229.62.9	TCP	1044 > 6666 [ACK] Seq=51801 Ack=1 win=5808 Len=1400 TSV=29'

Figure 14-7 The TCP exchange showing relative sequence numbers. Packets 50 and 66 are retransmissions. Packet 50 is retransmitted because of the fast retransmit algorithm, which triggers as a result of three duplicate ACKs. No retransmission timer is required, so recovery is relatively quick.

The first line of Figure 14-7 (number 40) indicates the first time ACK 23801 is received. Wireshark highlights (in red, appearing as black in Figure 14-7) other “interesting” TCP packets. Such packets differ from what would be expected for

a TCP transfer with no losses or other anomalies. We see window updates, duplicate ACKs, and retransmissions. The window update at time 0.853 is an ACK with a duplicate sequence number (because no data is being carried) but contains a change to the TCP flow control window. The window changes from 231,616 bytes to 233,016 bytes. Thus, it is not counted toward the three-duplicate-ACK threshold required to initiate a fast retransmit. Window updates merely provide a copy of the window advertisement. We will look at these in more detail in Chapter 15.

The packets arriving at times 0.890, 0.926, and 0.964 are all duplicate ACKs for sequence number 23801. The arrival of the third of these duplicate ACKs triggers the fast retransmit of segment 23801 at time 0.993. This can also be seen using Wireshark's Statistics | Flow Graph feature (see Figure 14-8).

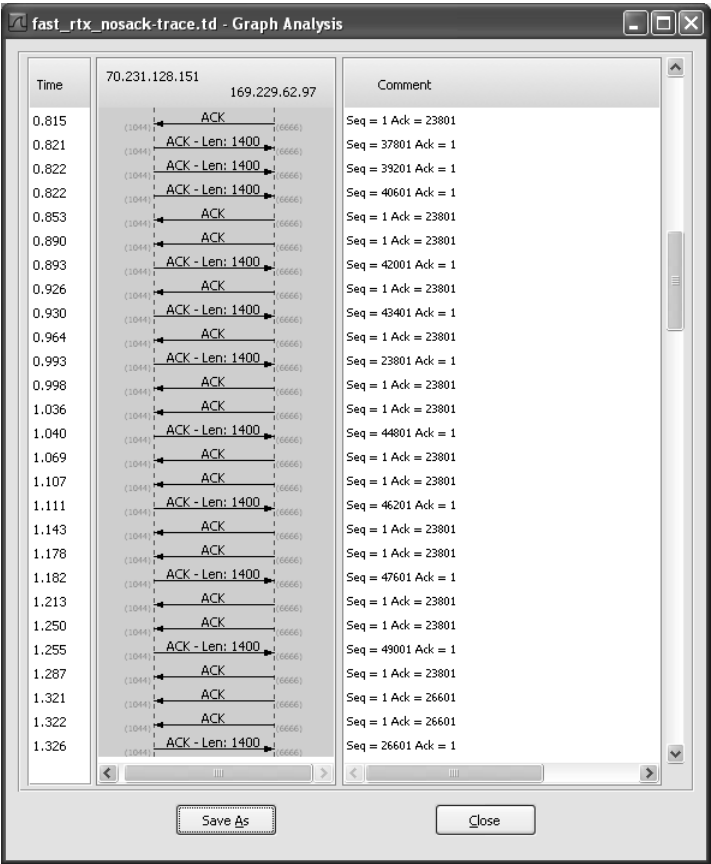


Figure 14-8 The retransmission at time 0.993 is triggered by the fast retransmit algorithm after receiving duplicate ACKs at times 0.890, 0.926, and 0.964. The ACK at time 0.853 is not considered a duplicate ACK because it contains a window update.

Here we see, in a slightly different way, the same fast retransmit at time 0.993. We can also see the second retransmission that takes place at time 1.326. This second retransmission takes place because of the arrival of the ACK at time 1.322.

The second retransmission is somewhat different from the first. When the first retransmission takes place, the sending TCP notes the highest sequence number it had sent just before it performed the retransmission ($43401 + 1400 = 44801$). This is called the *recovery point*. TCP is considered to be recovering from loss after a retransmission until it receives an ACK that matches or exceeds the sequence number of the recovery point. In this example, the ACKs at times 1.322 and 1.321 are not for 44801, but instead for 26601. This number is larger than the previous highest ACK value seen (23801), but not enough to meet or exceed the recovery point (44801). This type of ACK is called a *partial ACK* for this reason. When partial ACKs arrive, the sending TCP immediately sends the segment that appears to be missing (26601 in this case) and continues this way until the recovery point is matched or exceeded by an arriving ACK. If permitted by congestion control procedures (see Chapter 16), it may also send new data it has not yet sent.

This example illustrates the behavior of a TCP not using SACKs, when using fast retransmit, and when performing additional retransmits during recovery based on the “NewReno” sending algorithm [RFC3782]. Because no SACKs are being used, the sender can learn of at most one receiver hole per round-trip time, indicated by the increase in the ACK number of returning packets, which can only occur once a retransmission filling the receiver’s lowest-numbered hole has been received and ACKed.

The precise behavior during recovery varies, depending on the type and configuration of the TCP sender and receiver. This example illustrates a non-SACK sender using the NewReno algorithm, a fairly common arrangement. With NewReno, partial ACKs keep the sender in recovery as described. With older TCP variants (plain Reno), there is no such concept, and any acceptable ACK brings the TCP out of recovery. Doing so can present some performance problems for TCP, and these are discussed in detail in Chapter 16. NewReno and SACK, which we discuss next, are sometimes called “advanced loss recovery” techniques to distinguish them from the older approaches.

14.6 Retransmission with Selective Acknowledgments

With the standardization of the Selective Acknowledgment options in [RFC2018], a SACK-capable TCP receiver is able to describe data it has received with sequence numbers beyond the cumulative *ACK Number* field it sends in the primary portion of the TCP header. As we mentioned before, gaps between the ACK number and other in-window data cached at the receiver are called holes. Data with sequence numbers beyond the holes are called out-of-sequence data because that data is not contiguous, in terms of its sequence numbers, with the other data the receiver has already received.

The job of a sending TCP is to fill the holes in the receiver by retransmitting any data the receiver is missing, yet to be as efficient as possible by not resending data the receiver already has. In many circumstances, the properly operating SACK sender is able to fill these holes more quickly and with fewer unnecessary retransmissions than a comparable non-SACK sender because it does not have to wait an entire RTT to learn about additional holes. When the SACK option is being used, an ACK can be augmented with up to three or four SACK blocks that contain information about out-of-sequence data at the receiver. Each SACK block contains two 32-bit sequence numbers representing the first and last sequence numbers (plus 1) of a continuous block of out-of-sequence data being held at the receiver.

A SACK option that specifies n blocks has a length of $8n + 2$ bytes, so the 40 bytes available to hold TCP options can specify a maximum of four blocks. It is expected that SACK will often be used in conjunction with the TSOPT, which takes an additional 10 bytes (plus 2 bytes of padding), meaning that SACK is typically able to include only three blocks per ACK.

With three distinct blocks, up to three holes can be reported to the sender. If not limited by congestion control (see Chapter 16), all three could be filled within one round-trip time using a SACK-capable sender. An ACK packet containing one or more SACK blocks is sometimes called simply a “SACK.”

14.6.1 SACK Receiver Behavior

A SACK-capable receiver is allowed to generate SACKs if it has received the SACK-Permitted option during the TCP connection establishment (see Chapter 13). Generally speaking, a receiver generates SACKs whenever there is any out-of-order data in its buffer. This can happen either because data was lost in transit, or because it has been reordered and newer data has arrived at the receiver before older data. We consider the first case here and discuss the second one later.

The receiver places in the first SACK block the sequence number range contained in the segment it has *most recently received*. Because the space in a SACK option is limited, it is best to ensure that the most recent information is always provided to the sending TCP, if possible. Other SACK blocks are listed in the order in which they appeared as first blocks in previous SACK options. That is, they are filled in by repeating the most recently sent SACK blocks (in other segments) that are not subsets of another block about to be placed in the option being constructed.

The purpose of including more than one SACK block in a SACK option and repeating these blocks across multiple SACKs is to provide some redundancy in the case where SACKs are lost. If SACKs were never lost, [RFC2018] points out that only one SACK block would be required per SACK for full SACK functionality. Unfortunately, SACKs and regular ACKs are sometimes lost and are not retransmitted by TCP unless they contain data (or the SYN or FIN control bit fields are turned on).

14.6.2 SACK Sender Behavior

Although it is necessary for a SACK-capable receiver to generate proper SACK information to make full use of SACK, it is not sufficient for a TCP connection to benefit from SACKs. A SACK-capable sender must be used that treats the SACK blocks appropriately and performs *selective retransmission* by sending only those segments missing at the receiver, a process also called *selective repeat*. The SACK sender keeps track of any cumulative ACK information it receives (like any TCP sender), plus any SACK information it receives. It uses the SACK information it receives in ACKs generated at the receiver to avoid retransmitting data the receiver reports that it already has. One way it can do this is to keep a “SACKed” indication for each segment in its retransmission buffer that is set whenever a corresponding range of sequence numbers arrives in a SACK.

When a SACK-capable sender has the opportunity to perform a retransmission, usually because it has received a SACK or seen multiple duplicate ACKs, it has the choice of whether it sends new data or retransmits old data. The SACK information provides the sequence number ranges present at the receiver, so the sender can infer what segments likely need to be retransmitted to fill the receiver’s holes. The simplest approach is to have the sender first fill the holes at the receiver and then move on to send more new data [RFC3517] if the congestion control procedures allow. This is the most common approach.

There is one exception to this behavior. In [RFC2018], the current specification for SACK options, SACK blocks are considered *advisory*. This means that a receiver could provide a SACK to the sender indicating that some sequence numbers have been received successfully and then change its mind later (“renege”). Because of this, the SACK sender is not able to free its retransmission buffer of data it has received only a SACK for; it is permitted to free a block of data only once the regular TCP ACK number of the receiver has passed by the highest sequence number of this data. The rule also affects what TCP is supposed to do when a retransmission timer expires. When a sending TCP initiates a timer-based retransmission, any information regarding out-of-sequence data at the receiver derived from SACKs is supposed to be forgotten. If out-of-sequence data remains at the receiver, the ACK for the retransmitted segment contains additional SACK blocks the sender can then use. Fortunately, renegeing is rare and discouraged.

14.6.3 Example

To understand how the use of SACK alters the sender and receiver behaviors, we repeat the preceding fast retransmit experiment with the same setup (dropping sequence numbers 23601 and 28801), but this time the sender and receiver are using SACK. To get an immediate idea of what happens, we again use Wireshark’s TCP sequence number (`tcpt race`) plot function (see Figure 14-9).

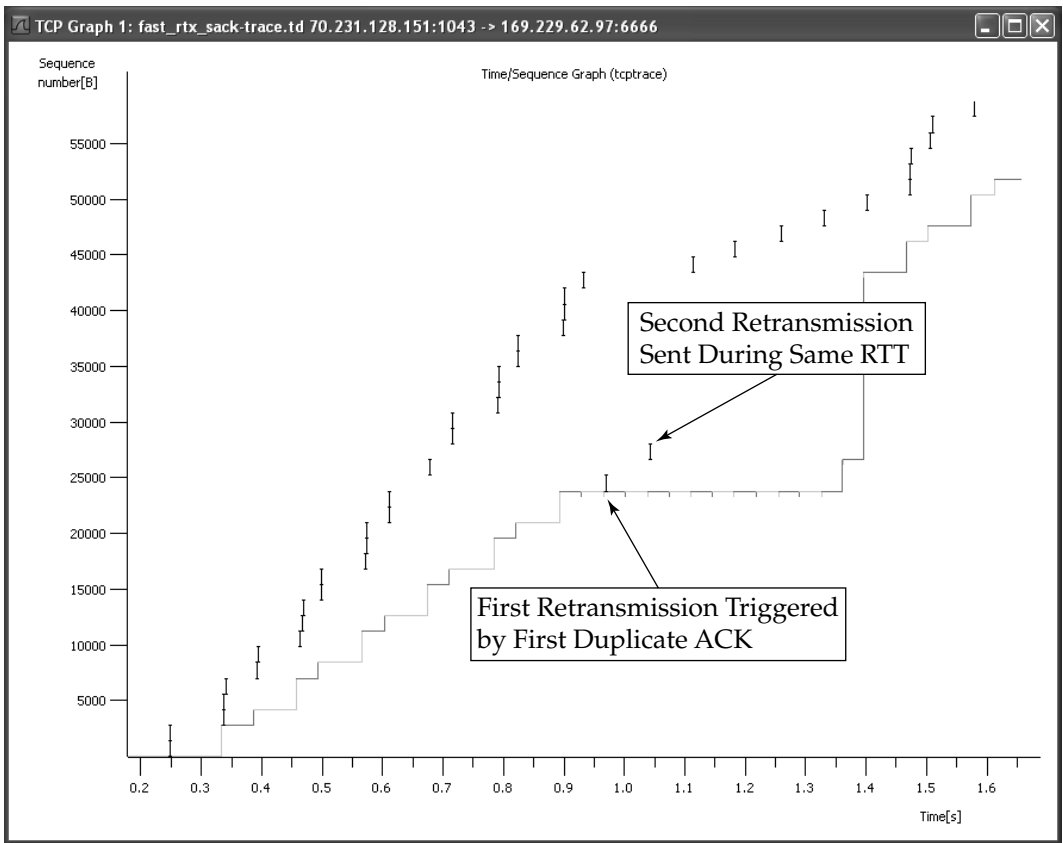


Figure 14-9 Fast retransmit is triggered by the arrival of the first duplicate ACK containing SACK information. The arrival of the next ACK allows the sender to learn of the second missing segment and retransmit it within the same RTT.

Figure 14-9 is similar to Figure 14-6, but the SACK sender has not had to wait an RTT to retransmit lost segment 28801 after retransmitting segment 23601. This is a result of the SACK information contained in the arriving ACKs. We will look at those in detail later, but first we verify the negotiation of the SACK-Permitted option during connection setup. This can be seen in Figure 14-10.

As expected, the receiver indicates its ability to use SACKs with the SACK-Permitted option. The SYN packet from the sender, the first packet of the trace, also contains an identical option. These options are present only at connection setup, and thus they only ever appear in segments with the SYN bit field set.

Once the connection is permitted to use SACKs, packet loss generally causes the receiver to start producing SACKs. For example, Wireshark shows the contents of the SACK options when the first SACK is selected (see Figure 14-11).

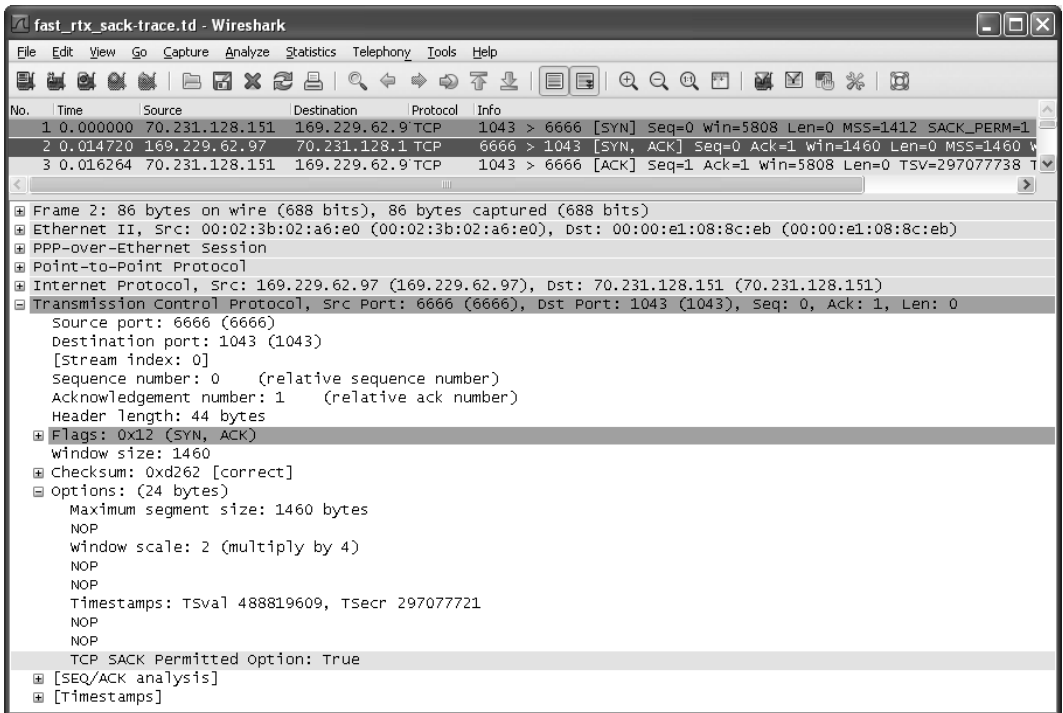


Figure 14-10 The SACK-Permitted option is exchanged in SYN segments to indicate the capability to generate and process SACK information. Most modern TCPs support the MSS, Timestamps, Window Scale, and SACK-Permitted options during connection establishment.

Figure 14-11 shows the series of events after the first SACK is received. Wireshark indicates SACK information by indicating the left edge and right edge of the SACK range. Here we see that the ACK for 23801 contains a SACK block of [25201,26601], indicating a hole at the receiver. The receiver is missing the sequence number range [23801,25200], which corresponds to the single 1400-byte packet starting with sequence number 23801. Note that this SACK is a window update and is not counted as a duplicate ACK for the reasons discussed earlier. It does not trigger fast retransmit.

The SACK arriving at time 0.967 contains two SACK blocks: [28001,29401] and [25201,26601]. Recall that the first SACK blocks from previous SACKs are repeated in later positions in subsequent SACKs for robustness against ACK loss. This SACK is a duplicate ACK for sequence number 23801 and suggests that the receiver now requires two full-size segments starting with sequence numbers 23801 and 26601. The sender reacts immediately by initiating fast retransmit, but because of congestion control procedures (see Chapter 16), the sender sends only one retransmission, for segment 23801. With the arrival of two additional ACKs, the sender is permitted to send its second retransmission, for segment 26601.

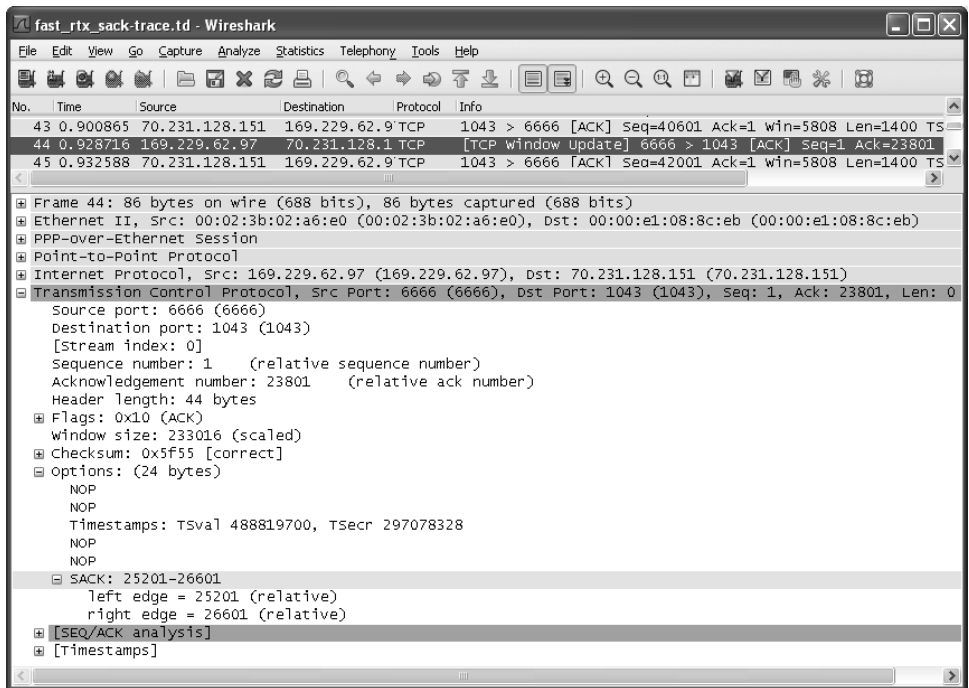


Figure 14-11 The first ACK containing SACK information indicates an out-of-order block with sequence number range 25201 to 26601.

A TCP SACK sender uses the recovery point idea introduced with NewReno. In this example, the highest sequence number sent prior to the retransmission is 43400, which is lower than in the NewReno example from Figure 14-5. For this implementation of SACK fast retransmit, three duplicate ACKs are not required; the TCP initiates its retransmission earlier. The recovery exit is essentially the same, though. Once the ACK for sequence number 43401 is received at time 1.3958, recovery is complete.

It is interesting to note that the potential for better control of the sender using SACKs does not always lead to increased overall throughput performance. This fact is suggested by looking at the two examples we have seen. The NewReno (non-SACK) sender completes the data transfer of 131,074 bytes in 3.592s. The SACK sender completes it in 3.674s. These two measurements are not directly comparable, however, because they did not face precisely the same network conditions (this was not a simulation but rather a live test), although the conditions were largely similar. The benefits of SACKs are more pronounced when the RTT is large and packet loss is severe. Under such circumstances, the benefits of being able to fill more than one hole per RTT are likely to be more significant.

14.7 Spurious Timeouts and Retransmissions

Under a number of circumstances, TCP may initiate a retransmission even when no data has been lost. Such undesirable retransmissions are called *spurious retransmissions* and are caused by *spurious timeouts* (timeouts firing too early) and other reasons such as packet reordering, packet duplication, or lost ACKs. Spurious timeouts can occur when the real RTT has recently increased significantly, beyond the RTO. This happens more frequently in environments where lower-layer protocols have widely varying performance (e.g., wireless) and was a concern mentioned in [KP87]. Here we focus primarily on spurious retransmissions caused by spurious timeouts. The effects of reordering and duplication on TCP are deferred until the following section.

A number of approaches have been suggested to deal with spurious timeouts. They generally involve a *detection* algorithm and a *response* algorithm. The detection algorithm attempts to determine whether a timeout or timer-based retransmission was spurious. The response algorithm is invoked once a timeout or retransmission is deemed spurious. Its purpose is to undo or mitigate some action that is otherwise normally performed by TCP when a retransmission timer expires. In this chapter we discuss only the segment retransmission behavior. The response algorithms typically involve congestion control changes as well, and those aspects are discussed in Chapter 16.

Figure 14-12 illustrates a highly simplified exchange that shows what happens to a basic TCP when a spurious retransmission occurs because of a delay spike in the ACK path after segment 8 is sent. After the retransmission of segment 5 occurs because of a timeout, there are still ACKs in flight from the original transmissions of segments 5 through 8. In this illustration, sequence and ACK numbers are based on packets instead of bytes, with ACKs indicating what has already arrived instead of what is expected next, for simplicity. When they arrive, TCP begins to retransmit additional segments that have already been received, starting with the segment following the ACKed segment. This causes TCP to behave in an undesirable “go-back-N” behavior pattern and in turn causes a collection of duplicate ACKs to be generated and returned to the sender, possibly triggering fast retransmit as well. Several techniques have been developed to mitigate these problems. We now have a look at some of the more popular ones.

14.7.1 Duplicate SACK (DSACK) Extension

With a non-SACK TCP, an ACK can indicate only the highest in-sequence segment back to the sender. With SACK, it can signal other (out-of-order) segments as well. The basic SACK mechanism we discussed previously does not say what happens when a receiver receives duplicate data segments. Such segments can be the result of spurious retransmissions, duplication within the network, or other reasons.

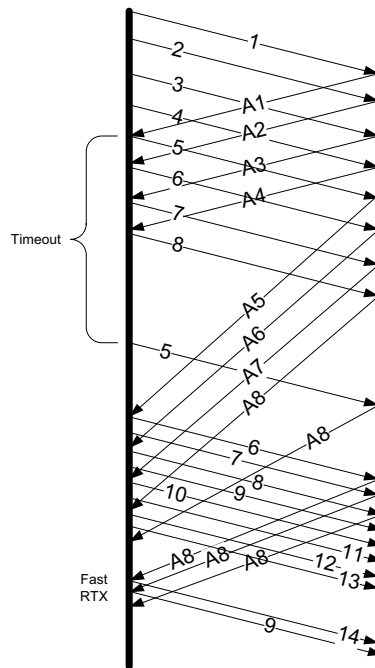


Figure 14-12 A delay spike occurs after the transmission of packet 8, causing a spurious retransmission timeout and retransmission of packet 5. After retransmission, an ACK for the first copy of 5 arrives. The retransmission for 5 creates a duplicate packet at the receiver, followed by an undesirable “go-back-N” behavior whereby packets 6, 7, and 8 are retransmitted even though they are already present at the receiver.

DSACK or D-SACK, which stands for *duplicate SACK* [RFC2883], is a rule, applied at the SACK receiver and interoperable with conventional SACK senders, that causes the first SACK block to indicate the sequence numbers of a duplicate segment that has arrived at the receiver. The main purpose of DSACK is to determine when a retransmission was not necessary and to learn additional facts about the network. With it, a sender has at least the possibility of inferring whether packet reordering, loss of ACKs, packet replication, and/or spurious retransmissions are taking place.

The implementation of DSACK is compatible with conventional SACK in the sense that no separate negotiation is required to make use of it. For it to work properly, a change is made to the content of SACKs sent from the receiver and a corresponding change to the logic at the sender. If a non-DSACK TCP shares a connection with a DSACK TCP, they will interoperate, but without any of the benefits of DSACK.

The change to the SACK receiver is to allow a SACK block to be included even if it covers sequence numbers *below* (or equal to) the cumulative *ACK Number* field.

This was not the original intent of SACK, but its capability is well matched to this purpose. (It applies equally well in cases where the DSACK information is above the cumulative *ACK Number* field; this happens for duplicated out-of-order segments.) DSACK information is included in only a single ACK, and such an ACK is called a DSACK. DSACK information is not repeated across multiple SACKs as conventional SACK information is. As a consequence, DSACKs are less robust to ACK loss than regular SACKs.

Exactly what a sender given DSACK information is supposed to do with it is not specified by [RFC2883]. An experimental algorithm is given in [RFC3708] for detecting spurious retransmissions using DSACK but does not provide any response algorithm. One option it mentions is to use the Eifel Response Algorithm, which we investigate in Section 14.7.4 after introducing a few other detection algorithms.

14.7.2 The Eifel Detection Algorithm

At the beginning of this chapter, we discussed the retransmission ambiguity problem. The experimental *Eifel Detection Algorithm* [RFC3522] deals with this problem using the TCP TSOPT to detect spurious retransmissions. After a retransmission timeout occurs, Eifel awaits the next acceptable ACK. If the next acceptable ACK indicates that the first copy of a retransmitted packet (called the *original transmit*) was the cause for the ACK, the retransmission is considered to be spurious.

The Eifel Detection Algorithm is able to detect spurious behavior earlier than the approach using only DSACK because it relies on ACKs generated as a result of packets arriving *before* loss recovery is initiated. DSACKs, conversely, are able to be sent only after a duplicate segment has arrived at the receiver and able to be acted upon only after the DSACK is returned to the sender. Detecting spurious retransmissions early can offer advantages, because it allows the sender to avoid most of the go-back-N behavior mentioned earlier.

The mechanics of the Eifel Detection Algorithm are simple. It requires the use of the TCP TSOPT. When a retransmission is sent (either a timer-based retransmission or a fast retransmit), the TSV value is stored. When the first acceptable ACK covering its sequence number is received, the incoming ACK's TSER is examined. If it is smaller than the stored value, the ACK corresponds to the original transmission of the packet and not the retransmission, implying that the retransmission must have been spurious. This approach is fairly robust to ACK loss as well. If an ACK is lost, any subsequent ACKs still have TSER values less than the stored TSV of the retransmitted segment. Thus, a retransmission can be deemed spurious as a result of any of the window's worth of ACKs arriving, so a loss of any single ACK is not likely to cause a problem.

The Eifel Detection Algorithm can be combined with DSACKs. This can be beneficial in the situation where an entire window's worth of ACKs are lost but both the original transmit and retransmission have arrived at the receiver. In this particular case, the arriving retransmit causes a DSACK to be generated. The Eifel

Detection Algorithm would by default conclude that the retransmission is spurious. It is thought, however, that if so many ACKs are being lost, allowing TCP to believe the retransmission was *not* spurious is useful (e.g., to induce it to start sending more slowly—a consequence of the congestion control procedures we discuss in Chapter 16). Thus, arriving DSACKs cause the Eifel Detection Algorithm to conclude that the corresponding retransmission is *not* spurious.

14.7.3 Forward-RTO Recovery (F-RTO)

Forward-RTO Recovery (F-RTO) [RFC5682] is a standard algorithm for detecting spurious retransmissions. It does not require any TCP options, so when it is implemented in a sender, it can be used effectively even with an older receiver that does not support the TCP TSOPT. It attempts to detect only spurious retransmissions caused by expiration of the retransmission timer; it does not deal with the other causes for spurious retransmissions or duplications mentioned before.

F-RTO makes a modification to the action TCP ordinarily takes after a timer-based retransmission. These retransmissions are for the smallest sequence number for which no ACK has yet been received. Ordinarily, TCP continues sending additional adjacent packets in order as additional ACKs arrive. This is the go-back-N behavior described previously.

F-RTO modifies the ordinary behavior of TCP by having TCP send new (so far unsent) data after the timeout-based retransmission when the first ACK arrives. It then inspects the second arriving ACK. If either of the first two ACKs arriving after the retransmission was sent are duplicate ACKs, the retransmission is deemed OK. If they are both acceptable ACKs that advance the sender's window, the retransmission is deemed to have been spurious. This approach is fairly intuitive. If the transmission of new data results in the arrival of acceptable ACKs, the arrival of the new data is moving the receiver's window forward. If such data is only causing duplicate ACKs, there must be one or more holes at the receiver. In either case, the reception of new data at the receiver does not harm the overall data transfer performance (provided there are sufficient buffers at the receiver).

14.7.4 The Eifel Response Algorithm

The *Eifel Response Algorithm* [RFC4015] is a standard set of operations to be executed by a TCP once a retransmission has been deemed spurious. Because the response algorithm is logically decoupled from the Eifel Detection Algorithm, it can be used with any of the detection algorithms we just discussed. The Eifel Response Algorithm was originally intended to operate for both timer-based and fast retransmit spurious retransmissions but is currently specified only for timer-based retransmissions.

Although the Eifel Response Algorithm can be used with any of the detection algorithms, it behaves somewhat differently based on whether a spurious timeout was detected early (e.g., by the Eifel or F-RTO detection algorithms) or

later (e.g., by DSACKs). The former cases are called spurious timeouts and operate by inspecting ACKs for original transmissions. The latter are called *late* spurious timeouts and are based on ACKs for retransmissions invoked as a result of (spurious) timeouts.

The response algorithm operates on the first retransmission timer event only. It is not executed if a subsequent timeout occurs before recovery is complete. After the retransmission timer expires, it takes a snapshot of the values in *srtt* and *rttvar* and records them in new variables *srtt_prev* and *rttvar_prev* as follows:

$$srtt_prev = srtt + 2(G)$$

$$rttvar_prev = rttvar$$

These variables are assigned on any timer expiration but are used only when the timeout is determined to be spurious. If so, they help form the basis for setting the new RTO. In the formula, the value *G* represents the TCP clock granularity. *srtt_prev* is set to *srtt* plus twice the timer granularity based on the following chain of reasoning: The spurious timeout may have been invoked because the value of *srtt* is just a tad too small. If it were just a bit larger, no timeout would have happened. Adding the term $2(G)$ to *srtt* deals with this situation by storing a slightly increased value into *srtt_prev*, which is used later for setting the RTO.

After the *srtt_prev* and *rttvar_prev* values are stored, one of the detection algorithms is invoked. The result of running the algorithm produces a value assigned to a special variable called *SpuriousRecovery*. If the algorithm detects a spurious timeout, *SpuriousRecovery* is set to *SPUR_T0*. If it detects a late spurious timeout, it sets *SpuriousRecovery* to *LATE_SPUR_T0*. Otherwise, the timeout is not spurious, and ordinary TCP timeout processing continues.

If *SpuriousRecovery* is *SPUR_T0*, TCP can take action before recovery is complete. It does this by adjusting the sequence number of the next segment it is about to send (called *SND.NXT*) to the first new, unsent segment (called *SND.MAX*). This avoids the undesirable go-back-*N* behavior after the initial retransmission discussed previously. If the detection algorithm detects a late spurious timeout, an ACK for the initial retransmission has already taken place, so *SND.NXT* is not changed. In either case, however, the congestion control state is reset (see Chapter 16). In addition, once an acceptable ACK is received for a segment transmitted after the retransmission timer expires, the values of *srtt*, *rttvar*, and RTO can be updated as follows:

$$srtt \leftarrow \max(srtt_prev, m)$$

$$rttvar \leftarrow \max(rttvar_prev, m/2)$$

$$RTO = srtt + \max(G, 4(rttvar))$$

Here, *m* is a sample of the RTT of the connection based on the arrival of the first acceptable ACK for data sent after the timeout. The motivation for these

modifications is that the real RTT may have changed so significantly that the RTT history in the current estimators is no longer a valid basis for setting the RTO. If the real path RTT has increased abruptly (e.g., because of wireless handoff to a new base station), the current *srtt* and *rttvar* values are likely to be too small and should be reinitialized. On the other hand, an increase in path RTT could be only temporary, implying that reinitializing *srtt* and *rttvar* might not be such a good idea because they are likely to be approximately correct.

These equations try to balance between the two situations by reassigning the moving averages *srtt* and *rttvar* only if the new RTT samples are larger. Doing so effectively throws out the previous history of the RTT (and RTT variance). The values of *srtt* and *rttvar* can only increase as a result of the response algorithm. If the RTT does not appear to be increasing, the running estimators remain unchanged, essentially ignoring the fact that a timeout has occurred. The RTO is reassigned in the conventional way in any case, and a new retransmission timer is set for this timeout value.

14.8 Packet Reordering and Duplication

Most of the issues discussed so far relate to how TCP handles packet loss. This is a relatively common issue, and a great deal of work has gone into making TCP robust to packet drops. As we began to see in the last section, other packet delivery anomalies such as duplication and reordering can also affect TCP's operation. In both of these cases, we wish TCP to be able to distinguish between packets that are reordered or duplicated and those that are lost. As we shall now see, this is sometimes not so simple.

14.8.1 Reordering

Packet reordering can occur in an IP network because IP provides no guarantee that relative ordering between packets is maintained during delivery. This can be beneficial (to IP at least), because IP can choose another path for traffic (e.g., that is faster) without having to worry about the consequences that doing so may cause traffic freshly injected into the network to pass ahead of older traffic, resulting in the order of packet arrivals at the receiver not matching the order of transmission at the sender. There are other reasons packet reordering may occur. For example, some high-performance routers employ multiple parallel data paths within the hardware [BPS99], and different processing delays among packets can lead to a departure order that does not match the arrival order.

Reordering may take place in the forward path or the reverse path of a TCP connection (or in some cases both). The reordering of data segments has a somewhat different effect on TCP as does reordering of ACK packets. Recall that because of asymmetric routing, it is frequently the case that ACKs travel along

15

TCP Data Flow and Window Management

15.1 Introduction

Chapter 13 dealt with the establishment and termination of TCP connections, and Chapter 14 examined how TCP ensures reliable delivery using retransmissions of data that has been lost. We now examine the dynamics of TCP data transfers, focusing initially on interactive connections and then introducing flow control and associated window management procedures that are used in conjunction with congestion control (see Chapter 16) for bulk data transfers.

An “interactive” TCP connection is one in which user input such as keystrokes, short messages, or joystick/mouse movements need to be delivered between a client and a server. If small segments are used to carry such user input, the protocol imposes more overhead because there are fewer useful payload bytes per packet exchanged. On the other hand, filling packets with more data usually requires them to be delayed, which can have a negative impact on delay-sensitive applications such as online games and collaboration tools. We shall investigate techniques with which the application can trade off between these two issues.

After discussing interactive communications, we discuss the methods used by TCP for achieving flow control by dynamically adapting the window size to ensure that a sender does not overrun a receiver. This issue primarily impacts bulk data transfer (i.e., noninteractive communications) but can also affect interactive applications. In Chapter 16 we will explore how the concept of flow control can be extended to protect not only the receiver, but also the network between the sender and the receiver.

15.2 Interactive Communication

The amount of network traffic carried in a particular portion of the Internet over a certain amount of time is usually measured in terms of bytes or packets. There is considerable variation in these numbers. For example, local area traffic differs from wide area traffic, and traffic between different sites tends to vary. Studies of TCP traffic [P05][F03] usually find that 90% or more of all TCP segments contain *bulk data* (e.g., Web, file sharing, electronic mail, backups) and the remaining portion contains *interactive data* (e.g., remote login, network games). Bulk data segments tend to be relatively large (1500 bytes or larger), while interactive data segments tend to be much smaller (tens of bytes of user data).

TCP handles both types of data using the same protocol and packet format, but different algorithms come into play for each. In this section, we will look at how interactive data is transferred by TCP, using the `ssh` (secure shell) application as one example. Secure shell [RFC4251] is a remote login protocol that provides strong security (privacy and authentication based on cryptography). It has mostly replaced the earlier UNIX `rlogin` and Telnet programs that provide remote login service but without strong security.

As we investigate `ssh`, we will see how delayed acknowledgments work and how the *Nagle algorithm* reduces the number of small packets across wide area networks. The same algorithms apply to other applications supporting remote login capability such as Telnet, `rlogin`, and Windows Terminal Services.

Let us look at the flow of data when we type an interactive command on an `ssh` connection. The client captures what the user types and ships it over to the server to be interpreted, and the server ships any responses back to the client. The client encrypts the data it sends, meaning that the characters typed by the user are encoded before being transferred over the connection (see Chapter 18). The encoding makes determining the typed keys difficult for an eavesdropper. The client supports several encryption algorithms and different authentication methods. It also supports several other advanced features such as tunneling other protocols (see Chapter 3 and [RFC4254]).

Many newcomers to TCP/IP are surprised to find that each interactive keystroke normally generates a separate data packet. That is, the keystrokes are sent from the client to the server individually (one character at a time rather than one line at a time). Furthermore, `ssh` invokes a shell (command interpreter) on the remote system (the server), which echoes the characters that are typed at the client. A single typed character could thus generate four TCP segments: the interactive keystroke from the client, an acknowledgment of the keystroke from the server, the echo of the keystroke from the server, and an acknowledgment of the echo from the client back to the server (see Figure 15-1(a)).

Normally, however, segments 2 and 3 are combined—in Figure 15-1(b), the acknowledgment of the keystroke is sent along with the echo of the characters typed. We describe the technique that combines these (called *delayed acknowledgments* with *piggybacking*) in the next section.

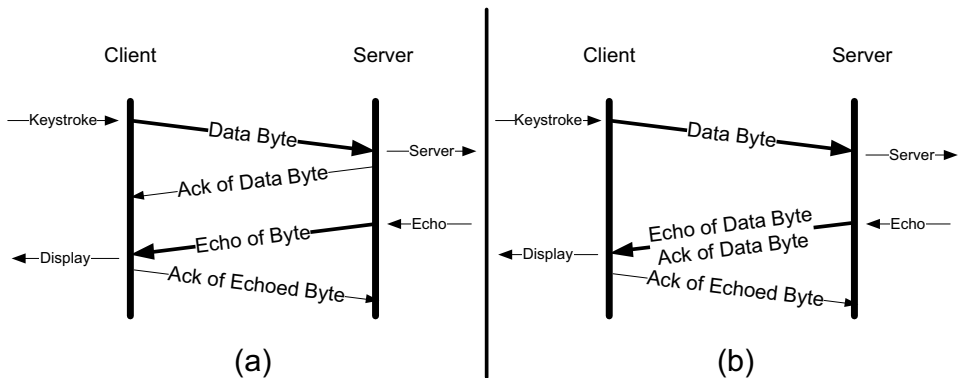


Figure 15-1 One possible way to remotely echo an interactive keystroke is a separate ACK and echo packet (a). A typical TCP coalesces the ACK for the data byte and the echo of the byte into a single packet (b).

We purposely use SSH for this example because it generates a packet for each character typed from the client to the server. If the user types especially fast, however, more than one character might be carried in a single packet. Figure 15-2 shows the flow of data using Wireshark when we type the `date` command across an active SSH connection to a Linux server.

No.	Time	Source	Destination	Protocol	Info
1	0.000000	70.231.141.59	169.229.62.97	SSH	Encrypted request packet len=48
2	0.014508	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=48
3	0.014769	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=49 Ack=49 win=4220 Len=0 TSV=913185368 TSER=114503261
4	1.736761	70.231.141.59	169.229.62.97	SSH	Encrypted request packet len=48
5	1.751620	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=48
6	1.751840	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=97 Ack=97 win=4220 Len=0 TSV=913187106 TSER=114503435
7	3.284481	70.231.141.59	169.229.62.97	SSH	Encrypted request packet len=48
8	3.299718	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=48
9	3.299937	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=145 Ack=145 win=4220 Len=0 TSV=913188654 TSER=114503590
10	4.982810	70.231.141.59	169.229.62.97	SSH	Encrypted request packet len=48
11	4.997635	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=48
12	4.997858	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=193 Ack=193 win=4220 Len=0 TSV=913190352 TSER=114503759
13	6.626947	70.231.141.59	169.229.62.97	SSH	Encrypted request packet len=48
14	6.642338	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=48
15	6.642557	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=241 Ack=241 win=4220 Len=0 TSV=913191997 TSER=114503924
16	6.644846	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=64
17	6.645054	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=241 Ack=305 win=4220 Len=0 TSV=913192000 TSER=114503924
18	6.646053	169.229.62.97	70.231.141.59	SSH	Encrypted response packet len=64
19	6.646251	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=241 Ack=369 win=4220 Len=0 TSV=913192001 TSER=114503924

Figure 15-2 TCP segments sent when the `date` command is typed on an already-established SSH connection.

In Figure 15-2, packet 1 carries the character `d` from the client to the server. Packet 2 is the acknowledgment of this character and its echo (combining the middle two segments as in Figure 15-1). Packet 3 is the acknowledgment of the echoed character. Packets 4–6 correspond to the character `a`, packets 7–9 to the character `t`, and packets 10–12 to the character `e`. Packets 13–15 correspond to the Enter (carriage

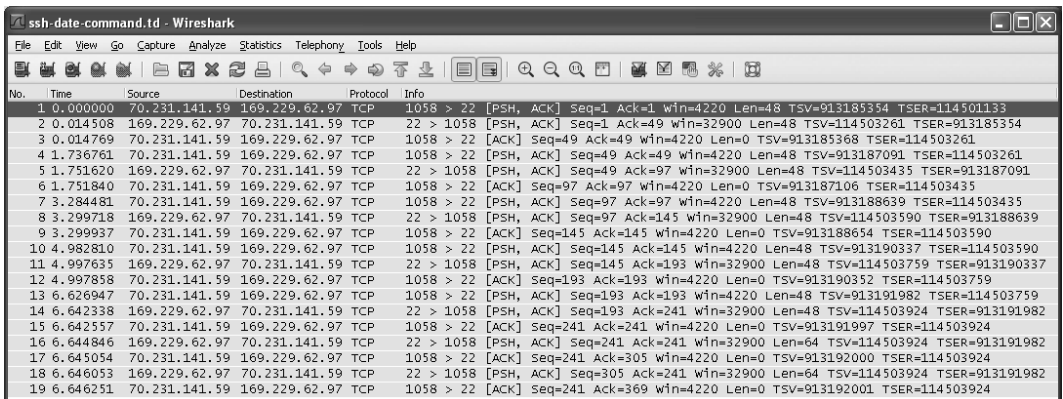
return) key. The delays between packets 3–4, 6–7, 9–10, and 12–13 are the human delays between typing each character, which were intentionally made unusually long (about 1.5s) in this case for illustration.

Notice that packets 16–19 are slightly different because they have grown in size from 48 bytes to 64 bytes. Packet 16 contains the output of the `date` command from the server. The 64 bytes are the encrypted version of the following 28 clear-text (not-yet-encrypted) characters:

```
Wed Dec 28 22:47:16 PST 2005
```

plus the carriage-return and line-feed characters at the end. The next packet sent from the server to the client (packet 18) contains the client's prompt on the server host: `Linux%`. Packet 19 acknowledges this data.

Figure 15-3 is the same trace as in Figure 15-2, except now more of the TCP-layer information is shown, indicating how TCP acknowledgments operate and the packet sizes used by ssh. Packet 1 (containing the `d` character) starts with the relative sequence number 0. Packet 2 ACKs the packet from line 1 by setting the ACK number to 48, the sequence number of the last successfully received byte plus 1. Packet 2 also sends the data byte with a sequence number of 0 from the server to the client, containing the echo of the `d` character. The echoed `d` is ACKed by the client in packet 3 by setting the ACK number to 48. We see that the connection has two streams of sequence numbers in use—one from the client to the server, and one in the reverse direction. We shall explore this in more detail when we discuss window advertisements.



No.	Time	Source	Destination	Protocol	Info
1	0.000000	70.231.141.59	169.229.62.97	TCP	1058 > 22 [PSH, ACK] Seq=1 Ack=1 win=4220 Len=48 TSV=913185354 TSER=114501133
2	0.014508	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=1 Ack=49 win=32900 Len=48 TSV=114503261 TSER=913185354
3	0.014769	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=49 Ack=49 win=4220 Len=0 TSV=913185368 TSER=114503261
4	1.736761	70.231.141.59	169.229.62.97	TCP	1058 > 22 [PSH, ACK] Seq=49 Ack=49 win=4220 Len=48 TSV=913187091 TSER=114503261
5	1.751620	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=49 Ack=97 win=32900 Len=48 TSV=114503435 TSER=913187091
6	1.751840	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=97 Ack=97 win=4220 Len=0 TSV=913187106 TSER=114503435
7	3.284481	70.231.141.59	169.229.62.97	TCP	1058 > 22 [PSH, ACK] Seq=97 Ack=97 win=4220 Len=48 TSV=913188639 TSER=114503435
8	3.299718	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=97 Ack=145 win=32900 Len=48 TSV=114503590 TSER=913188639
9	3.299937	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=145 Ack=145 win=4220 Len=0 TSV=913188654 TSER=114503590
10	4.982810	70.231.141.59	169.229.62.97	TCP	1058 > 22 [PSH, ACK] Seq=145 Ack=145 win=4220 Len=48 TSV=913190337 TSER=114503590
11	4.997635	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=145 Ack=193 win=32900 Len=48 TSV=114503759 TSER=913190337
12	4.997858	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=193 Ack=193 win=4220 Len=0 TSV=913190352 TSER=114503759
13	6.626947	70.231.141.59	169.229.62.97	TCP	1058 > 22 [PSH, ACK] Seq=193 Ack=193 win=4220 Len=48 TSV=913191982 TSER=114503759
14	6.642338	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=193 Ack=241 win=32900 Len=48 TSV=114503924 TSER=913191982
15	6.642557	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=241 Ack=241 win=4220 Len=0 TSV=913191997 TSER=114503924
16	6.644846	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=241 Ack=241 win=32900 Len=64 TSV=114503924 TSER=913191982
17	6.645054	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=241 Ack=305 win=4220 Len=0 TSV=913192000 TSER=114503924
18	6.646053	169.229.62.97	70.231.141.59	TCP	22 > 1058 [PSH, ACK] Seq=305 Ack=241 win=32900 Len=64 TSV=114503924 TSER=913191982
19	6.646251	70.231.141.59	169.229.62.97	TCP	1058 > 22 [ACK] Seq=241 Ack=369 win=4220 Len=0 TSV=913192001 TSER=114503924

Figure 15-3 The same trace as in Figure 15-2, except the protocol decode for ssh has been disabled, revealing the TCP sequence number information. Note that all data packets are 48 bytes in size except the last two. The size of 48 bytes relates to the cryptography used in ssh (see Chapter 18).

One other observation we can make about this trace is that each packet with data in it (not zero length) also has the *PSH* bit field set. As mentioned earlier, this flag is conventionally used to indicate that the buffer at the side sending the

packet has been emptied in conjunction with sending the packet. In other words, when the packet with the *PSH* bit field set left the sender, the sender had no more data to send.

15.3 Delayed Acknowledgments

In many cases, TCP does not provide an ACK for every incoming packet. This is possible because of TCP's cumulative ACK field (see Chapter 12). Using a cumulative ACK allows TCP to intentionally *delay* sending an ACK for some amount of time, in the hope that it can combine the ACK it needs to send with some data the local application wishes to send in the other direction. This is a form of *piggybacking* that is used most often in conjunction with bulk data transfers. Obviously a TCP cannot delay ACKs indefinitely; otherwise its peer could conclude that data has been lost and initiate an unnecessary retransmission.

Note

The Host Requirements RFC [RFC1122] states that TCP should implement a delayed ACK but the delay must be less than 500ms. Many implementations use a maximum of 200ms.

Delaying ACKs causes less traffic to be carried over the network than when ACKs are not delayed because fewer ACKs are used. A ratio of 2 to 1 is fairly common for bulk transfers. The use of delayed ACKs and the maximum amount of time TCP is allowed to wait before sending an ACK can be configured, depending on the host operating system. Linux uses a dynamic adjustment algorithm whereby it can change between ACKing every segment (called “quickack” mode) and conventional delayed ACK mode. On Mac OS X, the system variable `net.inet.tcp.delayed_ack` determines how delayed ACKs are to be used. The values work as follows: disable delay (0), always delay (1), ACK every other packet (2), and autodetect when to respond (3). The default is 3. On recent versions of Windows, the registry entries under

```
HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\Interfaces\IG
```

(where *IG* refers to the GUID of the particular network interface being referenced) for each interface GUID work a bit differently. The value for `TcpAckFrequency` (which needs to be added) can range from 0 to 255 and defaults to 2. It determines the number of ACKs outstanding before the delayed ACK timer is ignored. Setting the value to 1 effectively causes ACKs to be generated for every segment received. The ACK timer, when used, can be controlled with the `TcpDeLackTicks` registry entry. This value can be set in the range from 2 to 6 and defaults to 2. It is the number of hundreds of milliseconds to wait before sending a delayed ACK.

For the reasons mentioned earlier, TCP is generally set up to delay ACKs under certain circumstances, but not to delay them too long. We will see extensive use of delayed ACKs in Chapter 16, when we look at how TCP's congestion control behaves during bulk transfers with large packets. When smaller packets are used, such as for interactive applications, another algorithm comes into play. The combination of this algorithm with delayed ACKs can lead to poor performance if not handled carefully, so we will now look at it in more detail.

15.4 Nagle Algorithm

We saw in the previous section that as little as one keystroke at a time often flows from the client to the server across an ssh connection. When using IPv4, sending one single key press generates TCP/IPv4 packets of about 88 bytes in size (using the encryption and authentication from the example): 20 bytes for the IP header, 20 bytes for the TCP header (assuming no options), and 48 bytes of data. These small packets (called *tinygrams*) have a relatively high overhead for the network. That is, they contain relatively little useful application data compared to the rest of the packet contents. Such high-overhead packets are normally not a problem on LANs, because most LANs are not congested and such packets would not need to be carried very far. However, these tinygrams can add to congestion and lead to inefficient use of capacity on wide area networks. A simple and elegant solution was proposed by John Nagle in [RFC0896], now called the *Nagle algorithm*. First we will describe how it operates, and then we will discuss some pitfalls and problems that can occur as a result of using it with delayed ACKs.

The Nagle algorithm says that when a TCP connection has outstanding data that has not yet been acknowledged, small segments (those smaller than the SMSS) cannot be sent until all outstanding data is acknowledged. Instead, small amounts of data are collected by TCP and sent in a single segment when an acknowledgment arrives. This procedure effectively forces TCP into *stop-and-wait* behavior—it stops sending until an ACK is received for any outstanding data. The beauty of this algorithm is that it is *self-clocking*: the faster the ACKs come back, the faster the data is sent. On a comparatively high-delay WAN, where reducing the number of tinygrams is desirable, fewer segments are sent per unit time. Said another way, the RTT controls the packet sending rate.

We saw in Figure 15-3 that the RTT for a single byte to be sent, acknowledged, and echoed can be small (under 15ms). To generate data faster than this we would have to type more than 60 characters per second. This means that we rarely encounter any observable effects of this algorithm when sending data between two hosts with a small RTT, such as when they are on the same LAN.

To illustrate the effect of the Nagle algorithm, we can compare the behaviors of an application using TCP with the Nagle algorithm enabled and disabled. We modify a version of the ssh client for this purpose. Using a connection with a relatively large RTT of about 190ms, we can see the differences. First, we examine the case when Nagle is disabled (the default for ssh), as shown in Figure 15-4.

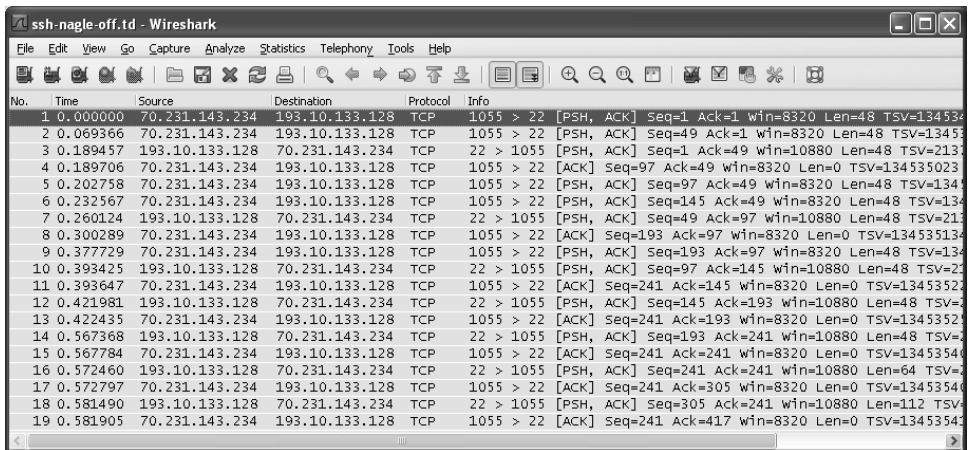


Figure 15-4 An ssh trace showing a TCP connection with approximately a 190ms RTT. The Nagle algorithm is disabled. Transmissions and ACKs are intermingled, and the exchange takes 0.58s using 19 packets. Many packets are relatively small (48 bytes of user data). Pure ACKs (segments with no data) indicate that command output at the server has been processed by the client.

The trace in Figure 15-4 begins after the initial authentication protocol has completed and the login session has begun. The `date` command is then typed. We see that 19 packets are captured, and the entire exchange lasts 0.58s. There are five ssh request packets, seven ssh response packets, and seven TCP-level pure ACKs (no data). If we repeat this measurement soon after (i.e., in similar network conditions), but instead leave the Nagle algorithm enabled, we see the behavior shown in Figure 15-5.

We can see immediately that the number of packets in Figure 15-5 is smaller than in Figure 15-4 (by eight). The other striking difference is the regularity of how the requests and responses are ordered and separated by time. Recall that the Nagle algorithm forces TCP to operate in a stop-and-wait fashion, so that the TCP sender cannot proceed until ACKs are received. If we look at the times for each request/response pair—0.0, 0.19, 0.38, and 0.57—we see that they follow a pattern; each is separated by almost exactly 190ms, which is very close to the RTT of the connection. The consequence of having to wait one RTT for each request/response adds to the overall time to complete the exchange (0.80s instead of the 0.58s when Nagle was disabled). This is the trade-off the Nagle algorithm makes: fewer and larger packets are used, but the required delay is higher. The different behaviors can be seen even more clearly in Figure 15-6.

The effect of the Nagle algorithm's stop-and-wait behavior can be seen clearly in Figure 15-6. The exchange on the left side keeps both directions of the connection busy, while with the Nagle algorithm enabled only one direction of the connection is busy at any given time.

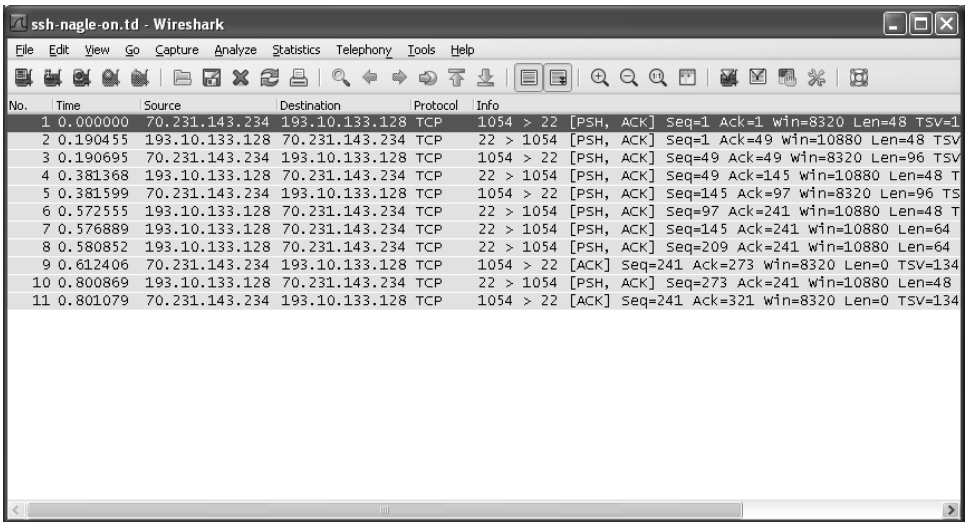


Figure 15-5 An ssh trace showing a TCP connection with a 190ms RTT and the Nagle algorithm in operation. Requests are followed in lockstep with responses, and the exchange takes 0.80s using 11 packets.

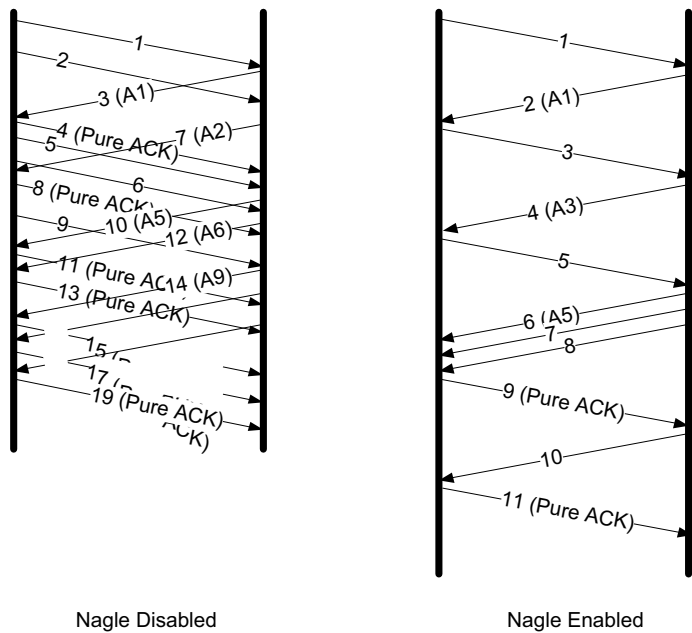


Figure 15-6 Comparing the use of the Nagle algorithm for TCP connections with a similar operating environment. With Nagle enabled, at most one packet is allowed to be outstanding at any given time. This reduces the number of small packets but increases delay.

15.4.1 Delayed ACK and Nagle Algorithm Interaction

If we consider what happens when the delayed ACK and Nagle algorithms are used together, we can construct an undesirable scenario. Consider a client using delayed ACKs that sends a request to a server, and the server responds with an amount of data that does not quite fit inside a single packet (see Figure 15-7).

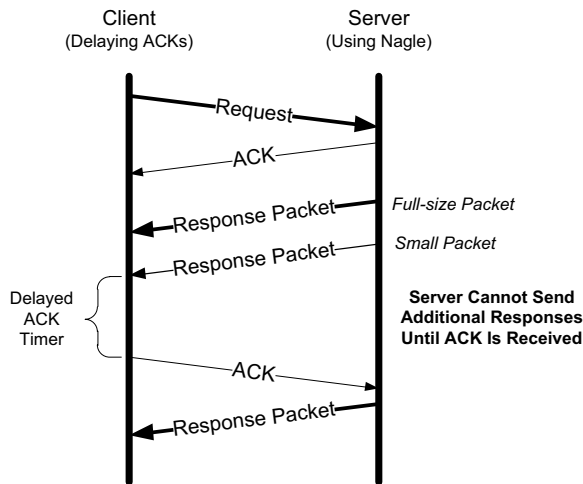


Figure 15-7 The interaction between the Nagle algorithm and delayed ACKs. A temporary form of deadlock can occur until the delayed ACK timer fires.

Here we see that the client, after receiving two packets from the server, withholds an ACK, hoping that additional data headed toward the server can be piggybacked. Generally, TCP is required to provide an ACK for two received packets only if they are full-size, and they are not here. At the server side, because the Nagle algorithm is operating, no additional packets are permitted to be sent to the client until an ACK is returned because at most one “small” packet is allowed to be outstanding. The combination of delayed ACKs and the Nagle algorithm leads to a form of *deadlock* (each side waiting for the other) [MMSV99][MM01]. Fortunately, this deadlock is not permanent and is broken when the delayed ACK timer fires, which forces the client to provide an ACK even if the client has no additional data to send. However, the entire data transfer becomes idle during this deadlock period, which is usually not desirable. The Nagle algorithm can be disabled in such circumstances, as we saw with SSH.

15.4.2 Disabling the Nagle Algorithm

As we might conclude from the previous example, there are times when the Nagle algorithm needs to be turned off. Typical examples include cases where as little delay as possible is required, for example, when a mouse movement or keystroke

must be delivered without delay to provide real-time feedback for a user whose display is handled remotely. Another example is in multiplayer online games, where character movements must be delivered as quickly as possible so as to not interfere with proper causality in the game (and to not delay it too much for other players).

The Nagle algorithm can be disabled in a number of ways. The ability to disable it is required by the Host Requirements RFC [RFC1122]. An application can specify the `TCP_NODELAY` option when using the Berkeley sockets API. In addition, it is possible to disable the Nagle algorithm on a system-wide basis. In Windows, this can be accomplished using the following registry key:

```
HKLM\SOFTWARE\Microsoft\MSMQ\Parameters\TCPNoDelay
```

This DWORD value, which must be added by the user, should be set to the value 1 in order to disable the Nagle algorithm. Message Queuing may have to be installed for this change to be effective [MMQ].

15.5 Flow Control and Window Management

Recall from Chapter 12 that a variable sliding window can be used to implement flow control. In Figure 15-8, a TCP client and server are interacting, providing each other with information about the data flow, including segment sequence numbers, ACK numbers, and window sizes (i.e., available space at the receiver).

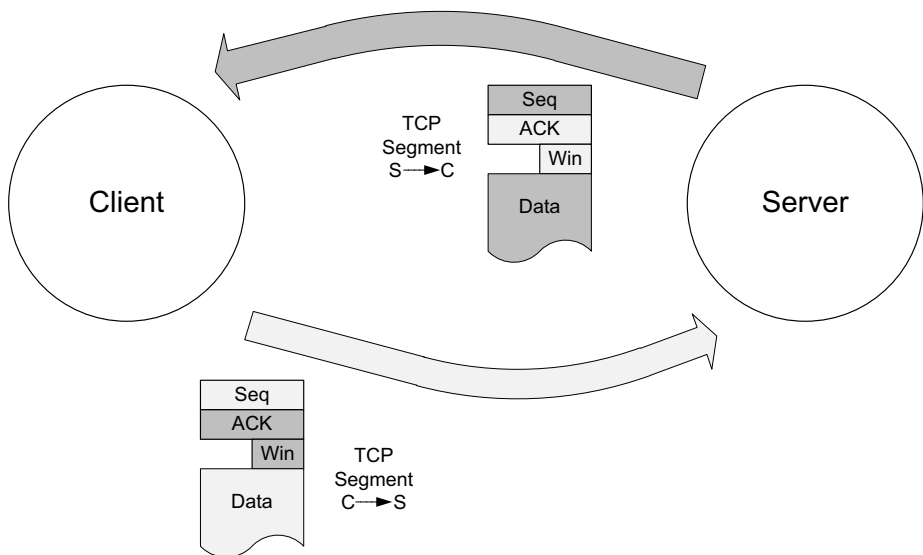


Figure 15-8 Each TCP connection is bidirectional. Data going in one direction causes the peer to respond with ACKs and window advertisements. The same is true for the reverse direction.

The two large arrows in Figure 15-8 indicate the direction of data flow (the direction in which TCP segments are sent). Recalling that every TCP connection has data flowing in both directions, we have two arrows, one in the client-to-server direction (C→S) and another in the server-to-client direction (S→C). Every segment contains ACK and window information and may also contain some user data. The fields used in the TCP header are shaded based on the direction of data flow they describe. For example, data flowing in the C→S direction is included in segments flowing along the bottom arrow, but the ACK number and window advertisement for this data are returned in segments following the top arrow. Every TCP segment (except those exchanged during connection establishment) includes a valid *Sequence Number* field, an *ACK Number* or *Acknowledgment* field, and a *Window Size* field (containing the window advertisement).

In each of the SSH examples in this chapter so far, we have seen an unchanging window advertisement conveyed from one TCP peer to the other. Examples include 8320 bytes, 4220 bytes, and 32,900 bytes. These sizes represent the amount of space the sender of the segment has reserved for storing incoming data the peer sends. When TCP-based applications are not busy doing other things, they are typically able to consume any and all data TCP has received and queued for them, leading to no change of the *Window Size* field as the connection progresses. On slow systems, or when the application has other things to accomplish, data may have arrived for the application, been acknowledged by TCP, and be sitting in a queue waiting for the application to read or “consume” it. When TCP starts to queue data in this way, the amount of space available to hold new incoming data decreases, and this change is reflected by a decreasing value of the *Window Size* field. Eventually, if the application does not read or otherwise consume the data at all, TCP must take some action to cause the sender to cease sending new data entirely, because there would be no place to put it on arrival. This is accomplished by sending a window advertisement of zero (no space).

The *Window Size* field in each TCP header indicates the amount of empty space, in bytes, remaining in the receive buffer. The field is 16 bits in TCP, but with the Window Scale option, values larger than 65,535 can be used (see Chapter 13). The largest sequence number the sender of a segment is willing to accept in the reverse direction is equal to the sum of the *Acknowledgment Number* and *Window Size* fields in the TCP header (scaled appropriately).

15.5.1 Sliding Windows

Each endpoint of a TCP connection is capable of sending and receiving data. The amount of data sent or received on a connection is maintained by a set of *window structures*. For each active connection, each TCP endpoint maintains a *send window structure* and a *receive window structure*. These structures are similar to the conceptual window structures described in Chapter 12, but here we describe them in more detail. Figure 15-9 shows a hypothetical TCP send window structure.

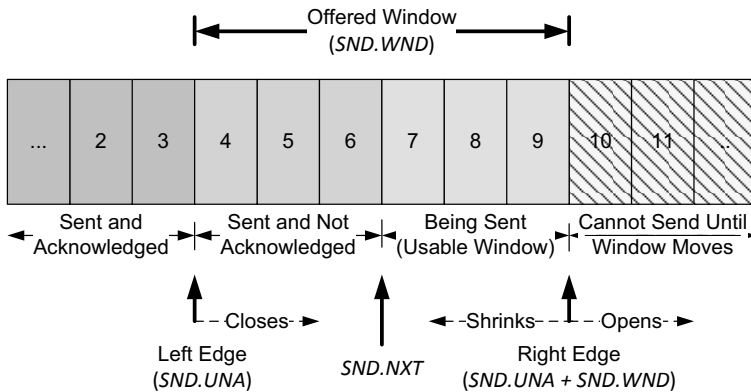


Figure 15-9 The TCP sender-side sliding window structure keeps track of which sequence numbers have already been acknowledged, which are in flight, and which are yet to be sent. The size of the offered window is controlled by the *Window Size* field sent by the receiver in each ACK.

TCP maintains its window structures in terms of bytes (not packets). In Figure 15-9 we have numbered the bytes 2 through 11. The window advertised by the receiver is called the *offered window* and covers bytes 4 through 9, meaning that the receiver has acknowledged all bytes up through and including number 3 and has advertised a window size of 6. Recall from Chapter 12 that the *Window Size* field contains a byte offset relative to the ACK number. The sender computes its *usable window*, which is how much data it can send immediately. The usable window is the offered window minus the amount of data already sent but not yet acknowledged. The variables `SND.UNA` and `SND.WND` are used to hold the values of the left window edge and offered window. The variable `SND.NXT` holds the next sequence number to be sent, so the usable window is equal to $(\text{SND.UNA} + \text{SND.WND} - \text{SND.NXT})$.

Over time this sliding window moves to the right, as the receiver acknowledges data. The relative motion of the two ends of the window increases or decreases the size of the window. Three terms are used to describe the movement of the right and left edges of the window:

1. The window *closes* as the left edge advances to the right. This happens when data that has been sent is acknowledged and the window size gets smaller.
2. The window *opens* when the right edge moves to the right, allowing more data to be sent. This happens when the receiving process on the other end reads acknowledged data, freeing up space in its TCP receive buffer.
3. The window *shrinks* when the right edge moves to the left. The Host Requirements RFC [RFC1122] strongly discourages this, but TCP must be able to cope with it. Section 15.5.3 on silly window syndrome shows an example where one side would like to shrink the window by moving the right edge to the left but cannot.

Because every TCP segment contains both an ACK number and a window advertisement, a TCP sender adjusts the window structure based on both values whenever an incoming segment arrives. The left edge of the window cannot move to the left, because this edge is controlled by the ACK number received from the other end that is cumulative and never goes backward. When the ACK number advances the window but the window size does not change (a common case), the window is said to advance or “slide” forward. If the ACK number advances but the window advertisement grows smaller with other arriving ACKs, the left edge of the window moves closer to the right edge. If the left edge reaches the right edge, it is called a zero window. This stops the sender from transmitting any data. If this happens, the sending TCP begins to *probe* the peer’s window (see Section 15.5.2) to look for an increase in the offered window.

The receiver also keeps a window structure, which is somewhat simpler than the sender’s. The receiver window structure keeps track of what data has already been received and ACKed, as well as the maximum sequence number it is willing to receive. The TCP receiver depends on this structure to ensure the correctness of the data it receives. In particular, it wishes to avoid storing duplicate bytes it has already received and ACKed, and it also wishes to avoid storing bytes that it should not have received (any bytes beyond the sender’s right window edge). The receiver’s window structure is illustrated in Figure 15-10.

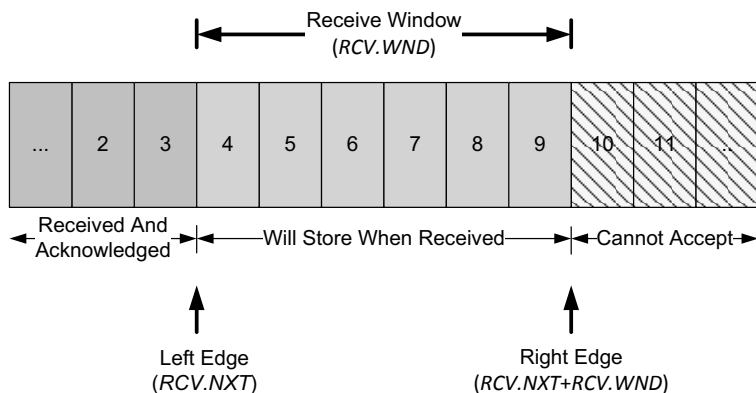


Figure 15-10 The TCP receiver-side sliding window structure helps the receiver know which sequence numbers to expect next. Sequence numbers in the receive window are stored when received. Those outside the window are discarded.

This structure also contains a left and right window edge like the sender’s window, but the in-window bytes (4–9 in this picture) need not be differentiated as they are in the sender’s window structure. For the receiver, any bytes received with sequence numbers less than the left window edge (called RCV.NXT) are discarded as duplicates, and any bytes received with sequence numbers beyond the

right window edge (RCV.WND bytes beyond RCV.NXT) are discarded as out of scope. Bytes arriving with any sequence number in the receive window range are accepted. Note that the ACK number generated at the receiver may be advanced only when segments fill in directly at the left window edge because of TCP's cumulative ACK structure. With selective ACKs, other in-window segments can be acknowledged using the TCP SACK option, but ultimately the ACK number itself is advanced only when data contiguous to the left window edge is received (see Chapter 14 for more details on SACK).

15.5.2 Zero Windows and the TCP Persist Timer

We have seen that TCP implements flow control by having the receiver specify the amount of data it is willing to accept from the sender: the receiver's advertised window. When the receiver's advertised window goes to zero, the sender is effectively stopped from transmitting data until the window becomes nonzero. When the receiver once again has space available, it provides a *window update* to the sender to indicate that data is permitted to flow once again. Because such updates do not generally contain data (they are a form of "pure ACK"), they are not reliably delivered by TCP. TCP must therefore handle the case where such window updates that would open the window are lost.

If an acknowledgment (containing a window update) is lost, we could end up with both sides waiting for the other: the receiver waiting to receive data (because it provided the sender with a nonzero window and expects to see incoming data) and the sender waiting to receive the window update allowing it to send. To prevent this form of deadlock from occurring, the sender uses a *persist timer* to query the receiver periodically, to find out if the window size has increased. The persist timer triggers the transmission of *window probes*. Window probes are segments that force the receiver to provide an ACK, which also necessarily contains a *Window Size* field. The Host Requirements RFC [RFC1122] suggests that the first probe should happen after one RTO and subsequent problems should occur at exponentially spaced intervals (i.e., similar to the "second part" of Karn's algorithm, which we discussed in Chapter 14).

Window probes contain a single byte of data and are therefore reliably delivered (retransmitted) by TCP if lost, thereby eliminating the potential deadlock condition caused by lost window updates. The probes are sent whenever the TCP persist timer expires, and the byte included may or may not be accepted by the receiver, depending on how much buffer space it has available. As with the TCP retransmission timer (see Chapter 14), the normal exponential backoff can be used when calculating the timeout for the persist timer. An important difference, however, is that a normal TCP never gives up sending window probes, whereas it may eventually give up trying to perform retransmissions. This can lead to a certain resource exhaustion vulnerability that we discuss in Section 15.7.

15.5.2.1 Example

To illustrate the use of the dynamic window size adjustment and flow control in TCP, we create a TCP connection and cause the receiving process to pause before consuming data from the network. For this experiment, we use a Mac OS X 10.6 sender and a Windows 7 receiver. The receiver runs our `sock` program with the `-P` flag as follows:

```
C:\> sock -i -s -P 20 6666
```

This arranges for the receiver to pause 20s prior to consuming data from the network. The result is that eventually the receiver's advertised window begins to close, as shown with packet 125 in Figure 15-11.

No.	Time	Source	Destination	Protocol	Info
2	0.003022	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [SYN, ACK] Seq=0 Ack=1 win=65535
15	0.010130	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=2473 win=65535 L
19	0.010421	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=4521 win=65535 L
23	0.010640	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=6569 win=65535 L
27	0.011881	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=8617 win=65535 L
31	0.012137	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=10665 win=65535
34	0.012929	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=12713 win=65535
38	0.015507	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=15609 win=65535
43	0.017863	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=18505 win=65535
48	0.022081	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=21401 win=65535
53	0.025970	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=24297 win=65535
58	0.026315	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=27193 win=65535
63	0.034158	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=30089 win=65535
68	0.049115	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=32985 win=65535
73	0.056894	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=35881 win=65535
78	0.058797	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=38777 win=65535
83	0.066892	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=41673 win=65535
88	0.069709	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=44569 win=65535
93	0.074032	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=47465 win=65535
98	0.075499	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=50361 win=65535
103	0.080786	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=53257 win=65535
108	0.088841	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=56153 win=65535
113	0.091330	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=59049 win=65535
117	0.094739	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=61945 win=65535
121	0.097035	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=64841 win=65535
125	0.098536	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=67737 win=64087
127	0.100571	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=70633 win=61191
128	0.102534	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=73529 win=58295
129	0.107267	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=76425 win=55399
130	0.107578	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=79321 win=52503
131	0.107778	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=82217 win=49607
132	0.108664	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=85113 win=46711
133	0.109498	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=88009 win=43815
134	0.114609	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=90905 win=40919
135	0.115865	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=93801 win=38023
136	0.118856	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=96697 win=35127

Figure 15-11 After a period when the advertised window does not change, acknowledgments continue but the window size grows smaller as the receiver's buffer fills up. If the receiving application fails to consume any data and the sender continues, the window eventually reaches zero.

In this trace we see that for more than 100 packets the receiver's window remains pegged at 64KB. This is because of an automatic window adjustment algorithm (see Section 15.5.4) that allocates memory to the receiving TCP even if not requested by the application. However, this eventually runs short, so we see the window begin to reduce starting with packet 125. A large number of ACKs follow, each reducing the window further while increasing the ACK number by 2896 bytes per ACK. This indicates that the receiving TCP is storing the data, but the application is not consuming it. If we look further into the trace, we see that eventually the receiver has no more space to hold the incoming data (see Figure 15-12).

No.	Time	Source	Destination	Protocol	Info
139	0.125602	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=105385 win=26439 Len=0 TSV=343880 TSER=949804632
140	0.126039	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=108281 win=23543 Len=0 TSV=343881 TSER=949804632
141	0.126403	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=111177 win=20647 Len=0 TSV=343881 TSER=949804632
142	0.126692	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=114073 win=17751 Len=0 TSV=343881 TSER=949804632
143	0.127250	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=116737 win=15087 Len=0 TSV=343882 TSER=949804632
144	0.127691	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=119633 win=12191 Len=0 TSV=343882 TSER=949804632
145	0.127963	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=121257 win=10567 Len=0 TSV=343882 TSER=949804632
146	0.128522	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=123305 win=8519 Len=0 TSV=343882 TSER=949804632
147	0.128897	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=125353 win=6471 Len=0 TSV=343882 TSER=949804632
148	0.129153	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=127401 win=4423 Len=0 TSV=343882 TSER=949804632
149	0.129531	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=129449 win=2375 Len=0 TSV=343883 TSER=949804632
150	0.131384	10.0.1.37	10.0.1.33	TCP	6666 > 53005 [ACK] Seq=1 Ack=131497 win=327 Len=0 TSV=343883 TSER=949804632
151	4.769161	10.0.1.33	10.0.1.37	TCP	[TCP window Full] 53005 > 6666 [ACK] Seq=131497 Ack=1 win=131070 Len=327 TSV=949804
152	4.979187	10.0.1.37	10.0.1.33	TCP	[TCP zeroWindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0 Len=0 TSV=344368 TSER=949804
153	9.770829	10.0.1.33	10.0.1.37	TCP	[TCP zeroWindowProbe] 53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1 TSV=949804
154	9.771573	10.0.1.37	10.0.1.33	TCP	[TCP zeroWindowProbeAck] [TCP zeroWindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0
155	14.772023	10.0.1.37	10.0.1.33	TCP	[TCP zeroWindowProbe] 53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1 TSV=949804
156	14.773429	10.0.1.37	10.0.1.33	TCP	[TCP zeroWindowProbeAck] [TCP zeroWindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0
157	19.773114	10.0.1.37	10.0.1.33	TCP	[TCP zeroWindowProbe] 53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1 TSV=949804
158	19.773914	10.0.1.37	10.0.1.33	TCP	[TCP zeroWindowProbeAck] [TCP zeroWindow] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=0
159	20.142523	10.0.1.37	10.0.1.33	TCP	[TCP window update] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=3732 Len=0 TSV=345884 T
160	20.142729	10.0.1.37	10.0.1.33	TCP	[TCP window update] 6666 > 53005 [ACK] Seq=1 Ack=131824 win=65535 Len=0 TSV=345884
161	20.142807	10.0.1.33	10.0.1.37	TCP	53005 > 6666 [ACK] Seq=131824 Ack=1 win=131070 Len=1448 TSV=949804833 TSER=345884
162	20.142833	10.0.1.33	10.0.1.37	TCP	53005 > 6666 [ACK] Seq=133272 Ack=1 win=131070 Len=1448 TSV=949804833 TSER=345884
163	20.142867	10.0.1.33	10.0.1.37	TCP	53005 > 6666 [ACK] Seq=134720 Ack=1 win=131070 Len=1448 TSV=949804833 TSER=345884

Figure 15-12 The receiver's buffer has filled up. When the receiving application starts reading again, a window update tells the sender that there is now an opportunity to transfer more data.

Here we can see that packet 151 fills the small 327-byte window, as indicated by the TCP Window Full comment provided by Wireshark. After about 200ms, at time 4.979, a zero window advertisement is produced, indicating that no more data can be received. This is no surprise, given that the sender has filled the last known available window and the receiving application will not consume any data until time 20.143.

After receiving the zero window advertisement, the sending TCP tries to probe the receiver three times at 5s intervals to see if the window has opened. At time 20, as instructed, the receiver begins to consume the data present in TCP's queue. This causes two window updates to be sent to the sender, indicating that further data transmission (up to 64KB) is now possible. Such segments are called window updates because they do not acknowledge any new data—they just advance the right edge of the window. At this point, the sender is able to resume normal data transmission and complete the transfer.

There are numerous points that we can summarize using Figures 15-11 and 15-12:

1. The sender does not have to transmit a full window's worth of data.
2. A single segment from the receiver acknowledges data and slides the window to the right at the same time. This is because the window advertisement is relative to the ACK number in the same segment.
3. The size of the window can decrease, as shown by the series of ACKs in Figure 15-11, but the right edge of the window does not move left, so as to avoid window shrinkage.
4. The receiver does not have to wait for the window to fill before sending an ACK.

In addition to these points, it is instructive to look at the throughput this connection achieves as a function of time. Using Wireshark's Statistics | TCP Stream Graph | Throughput Graph function, we observe the time series as shown in Figure 15-13.

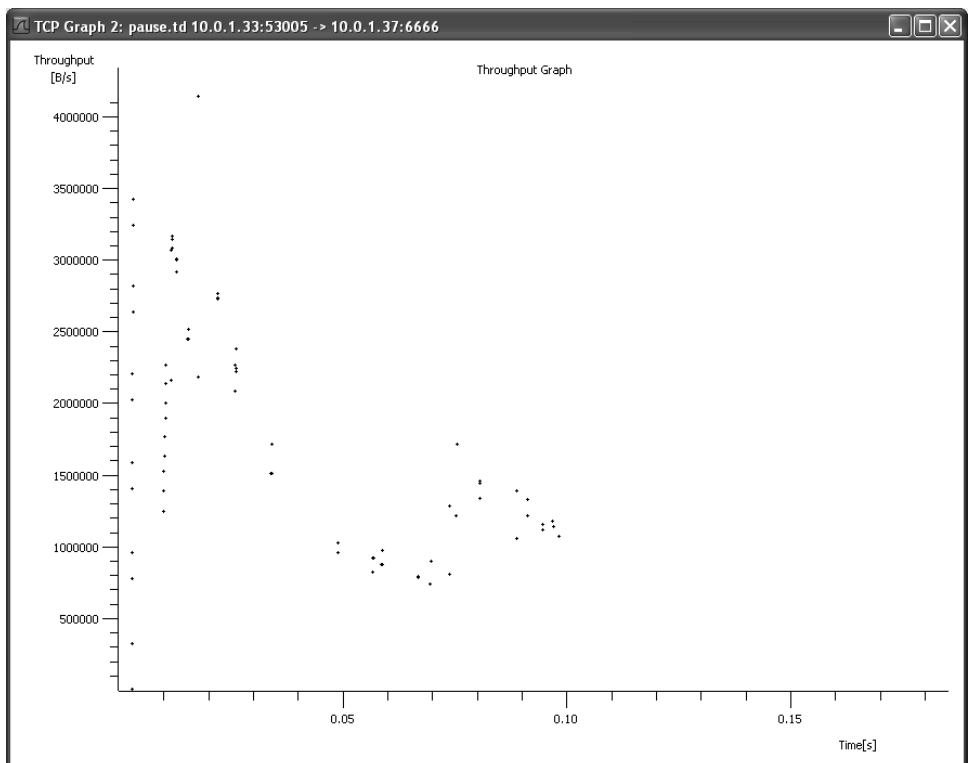


Figure 15-13 With a relatively large receive buffer, a significant amount of data can be transferred even before the receiving application reads any data from the network.

Here we see an interesting behavior. Even before the receiving application has consumed *any* data, the connection still achieves a throughput of approximately 1.3MB/s. This continues until approximately time 0.10. After that, the throughput is essentially zero until the receiver begins consuming data much later (after time 20).

15.5.3 Silly Window Syndrome (SWS)

Window-based flow control schemes, especially those that do not use fixed-size segments (such as TCP), can fall victim to a condition known as the *silly window syndrome* (SWS). When it occurs, small data segments are exchanged across the connection instead of full-size segments [RFC0813]. This leads to undesirable inefficiency because each segment has relatively high overhead—a small number of bytes relative to the number of bytes in the headers.

SWS can be caused by either end of a TCP connection: the receiver can advertise small windows (instead of waiting until a larger window can be advertised), and the sender can transmit small data segments (instead of waiting for additional data to send a larger segment). Correct avoidance of silly window syndrome requires a TCP to implement rules specifically for this purpose, whether operating as a sender or a receiver. TCP never knows ahead of time how a peer TCP will behave. The following rules are applied:

1. When operating as a receiver, small windows are not advertised. The receive algorithm specified by [RFC1122] is to not send a segment advertising a larger window than is currently being advertised (which can be 0) until the window can be increased by either one full-size segment (i.e., the receive MSS) or by one-half of the receiver's buffer space, whichever is smaller. Note that there are two cases where this rule can come into play: when buffer space has become available because of an application consuming data from the network, and when TCP must respond to a window probe.
2. When sending, small segments are not sent and the Nagle algorithm governs when to send. Senders avoid SWS by not transmitting a segment unless at least one of the following conditions is true:
 - a. A full-size (send MSS bytes) segment can be sent.
 - b. TCP can send at least one-half of the maximum-size window that the other end has ever advertised on this connection.
 - c. TCP can send everything it has to send and either (i) an ACK is not currently expected (i.e., we have no outstanding unacknowledged data) or (ii) the Nagle algorithm is disabled for this connection.

Condition (a) is the most straightforward and directly avoids the high-overhead segment problem. Condition (b) deals with hosts that always advertise tiny windows, perhaps smaller than the segment size. Condition (c) prevents TCP from

sending small segments when there is unacknowledged data waiting to be ACKed and the Nagle algorithm is enabled. If the sending application is doing small writes (e.g., smaller than the segment size), condition (c) avoids silly window syndrome.

These three conditions also let us answer the following question: If the Nagle algorithm prevents us from sending small segments while there is outstanding unacknowledged data, how small is small? From condition (a) we see that “small” means that the number of bytes is less than the SMSS (i.e., the largest packet size that does not exceed the PMTU or the receiver’s MSS). Condition (b) comes into play only with older, primitive hosts or when a small advertised window is used because of a limited receive buffer size.

Condition (b) of step 2 requires that the sender keep track of the maximum window size advertised by the other end. This is an attempt by the sender to guess the size of the other end’s receive buffer. Although the size of the receive buffer could decrease while the connection is established, in practice this is rare. Furthermore, recall that TCP avoids window shrinkage.

15.5.3.1 Example

We will now present a detailed example to see silly window syndrome avoidance in action; this example also involves the persist timer. We will use our `sock` program with a Windows XP sending host and a FreeBSD receiver, doing three 2048-byte writes to the network. The command at the sender is as follows:

```
C:\> sock -i -n 3 -w 2048 10.0.0.8 6666
```

The corresponding command at the receiver is

```
FreeBSD% sock -i -s -P 15 -p 2 -r 256 -R 3000 6666
```

This fixes the receive buffer at 3000 bytes, causes an initial delay of 15s before reading from the network, injects 2s of delay between each read, and sets each read amount to be 256 bytes. The reason for the initial pause is to let the receiver’s buffer fill, ultimately forcing the transmitter to stop. By having the receiver then perform small reads from the network, we expect to see it perform silly window syndrome avoidance. Figure 15-14 is the trace as displayed by Wireshark.

The contents of the entire connection are displayed in the figure. Packet lengths are described in terms of how many TCP payload bytes are included in each segment. During connection establishment, the receiver advertises a window of 3000 bytes with an MSS of 1460 bytes. The sender sends a 1460-byte packet (packet 4) at time 0.052 and 588 bytes (packet 5) at time 0.053. The sum of these sizes equals the 2048-byte write size used by the application. Packet 6 acknowledges both data packets from the sender and provides a window advertisement of 952 bytes ($3000 - 1460 - 588 = 952$).

The 952-byte window (packet 6) is not as large as a full MSS, so the Nagle algorithm at the sender prevents filling it immediately. Instead, we see a delay

No.	Time	Source	Destination	Protocol	Info
1	0.000000	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [SYN] Seq=0 win=65535 Len=0 MSS=1460 SACK_PERM=1
2	0.000332	10.0.0.8	10.0.0.100	TCP	6666 > 3699 [SYN, ACK] Seq=0 Ack=1 win=3000 Len=0 MSS=1460 SACK_PERM=1
3	0.001106	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=1 Ack=1 win=65535 Len=0
4	0.052667	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=1 Ack=1 win=65535 Len=1460
5	0.053057	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [PSH, ACK] Seq=1461 Ack=1 win=65535 Len=588
6	0.053145	10.0.0.8	10.0.0.100	TCP	6666 > 3699 [ACK] Seq=1 Ack=2049 win=952 Len=0
7	5.066108	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=2049 Ack=1 win=65535 Len=952
8	5.160715	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindow] 6666 > 3699 [ACK] Seq=1 Ack=3001 win=0 Len=0
9	6.970589	10.0.0.100	10.0.0.8	TCP	[TCP ZeroWindowProbe] 3699 > 6666 [ACK] Seq=3001 Ack=1 win=65535 Len=1
10	6.970754	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 3699 [ACK] Seq=1 Ack=3001 win=0
11	10.782520	10.0.0.100	10.0.0.8	TCP	[TCP ZeroWindowProbe] 3699 > 6666 [ACK] Seq=3001 Ack=1 win=65535 Len=1
12	10.782692	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 3699 [ACK] Seq=1 Ack=3001 win=0
13	18.408217	10.0.0.100	10.0.0.8	TCP	[TCP ZeroWindowProbe] 3699 > 6666 [ACK] Seq=3001 Ack=1 win=65535 Len=1
14	18.408384	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindow] [TCP ACKed lost segment] 6666 > 3699 [ACK] Seq=1 Ack=3002 win=0
15	25.061290	10.0.0.8	10.0.0.100	TCP	[TCP Window Update] 6666 > 3699 [ACK] Seq=1 Ack=3002 win=1535 Len=0
16	25.064378	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [PSH, ACK] Seq=3002 Ack=1 win=65535 Len=1460
17	25.161232	10.0.0.8	10.0.0.100	TCP	6666 > 3699 [ACK] Seq=1 Ack=4462 win=75 Len=0
18	30.043950	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=4462 Ack=1 win=65535 Len=75
19	30.141368	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindow] 6666 > 3699 [ACK] Seq=1 Ack=4537 win=0 Len=0
20	31.548523	10.0.0.100	10.0.0.8	TCP	[TCP ZeroWindowProbe] 3699 > 6666 [ACK] Seq=4537 Ack=1 win=65535 Len=1
21	31.548698	10.0.0.8	10.0.0.100	TCP	[TCP ACKed lost segment] 6666 > 3699 [ACK] Seq=1 Ack=4538 win=767 Len=0
22	36.574538	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=4538 Ack=1 win=65535 Len=767
23	36.671539	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindow] 6666 > 3699 [ACK] Seq=1 Ack=5305 win=0 Len=0
24	37.667677	10.0.0.100	10.0.0.8	TCP	[TCP ZeroWindowProbe] 3699 > 6666 [ACK] Seq=5305 Ack=1 win=65535 Len=1
25	37.667834	10.0.0.8	10.0.0.100	TCP	[TCP ACKed lost segment] 6666 > 3699 [ACK] Seq=1 Ack=5306 win=767 Len=0
26	42.784930	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=5306 Ack=1 win=65535 Len=767
27	42.881712	10.0.0.8	10.0.0.100	TCP	[TCP ZeroWindow] 6666 > 3699 [ACK] Seq=1 Ack=6073 win=0 Len=0
28	43.485876	10.0.0.100	10.0.0.8	TCP	[TCP ZeroWindowProbe] 3699 > 6666 [ACK] Seq=6073 Ack=1 win=65535 Len=1
29	43.486016	10.0.0.8	10.0.0.100	TCP	[TCP ACKed lost segment] 6666 > 3699 [ACK] Seq=1 Ack=6074 win=767 Len=0
30	43.486822	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [PSH, ACK] Seq=6074 Ack=1 win=65535 Len=71
31	43.581711	10.0.0.8	10.0.0.100	TCP	6666 > 3699 [ACK] Seq=1 Ack=6145 win=696 Len=0
32	43.711676	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [FIN, ACK] Seq=6145 Ack=1 win=65535 Len=0
33	43.711806	10.0.0.8	10.0.0.100	TCP	6666 > 3699 [ACK] Seq=1 Ack=6146 win=695 Len=0
34	55.212068	10.0.0.8	10.0.0.100	TCP	[TCP Window Update] 6666 > 3699 [ACK] Seq=1 Ack=6146 win=2232 Len=0
35	63.252390	10.0.0.8	10.0.0.100	TCP	6666 > 3699 [FIN, ACK] Seq=1 Ack=6146 win=3000 Len=0
36	63.253356	10.0.0.100	10.0.0.8	TCP	3699 > 6666 [ACK] Seq=6146 Ack=2 win=65535 Len=0

Figure 15-14 Trace of a TCP transfer illustrating silly window syndrome avoidance. The sender avoids filling the offered window at time 0.053 because of sender-side SWS avoidance. Instead, it waits until time 5.066, also acting effectively as a window probe. Receiver-side SWS avoidance can be seen by looking at packet 14, which advertises a zero window even though the receiver has consumed some data.

of 5s before any further action is taken. The sender waits for 5s, until the persist timer expires, before sending a window probe. Given that the sender is sending a packet anyhow, the sending TCP adds the permitted 952 bytes to fill the available window. This fills the window, as confirmed by the zero window advertisement contained in packet 8.

The next event in the trace is when TCP sends a window probe at time 6.970, about 2s after receiving the first zero window advertisement. The probe itself contains a single data byte and is labeled “TCP ZeroWindowProbe” by Wireshark, but the ACK for this does not move the ACK number forward (Wireshark labels this a “TCP ZeroWindowProbeAck”), so the byte has not been kept at the receiver. Another 1-byte probe is produced at time 10.782 (about 4s later), and another at time 18.408 (about 8s later), showing the characteristic exponential timeout back-off. Note that for this latter window probe, the single byte is acknowledged by the receiver.

At time 25.061, after the application has had a chance to perform six 256-byte reads (spaced 2s apart), a window update indicates that 1535 bytes (plus 1 for the ACK number) are now free in the receiver's buffer. This is "large enough" according to receiver-side SWS avoidance. The sender begins to fill the window, starting with a 1460-byte packet at time 25.064, resulting in an ACK at time 25.161 for byte 4462 with a window advertisement of only 75 bytes (packet 17). This advertisement appears to violate our rule that the amount advertised should be at least an MSS or (in the case of FreeBSD) one-quarter of the total buffer. The reason is to avoid window shrinkage. With the last window update (packet 15), the receiver advertises a right window edge of byte $(3002 + 1535) = 4537$. If the present ACK (packet 17) were to advertise less than 75 bytes, as would be required by receiver-side SWS avoidance, the right window edge would move left, a condition TCP is not supposed to allow. Consequently the 75-byte advertisement represents a form of override: avoiding window shrinkage is preferred to avoiding SWS.

We see the effect of sender-side SWS avoidance once again with the 5s delay between packets 17 and 18. The sender is forced to send the 75-byte packet and the receiver responds with another zero window advertisement. Packet 20, which appears a second later, is another window probe, which results in a window of 767 bytes. Another round of sender-side SWS avoidance results in a 5s delay; the sender fills the window, again resulting in a zero window; and the pattern repeats. The pattern is eventually broken because the sender has no more data to send. Packet 30 represents the last data sent, and the connection is eventually closed some 20s later (because of the 2s delays between each read at the receiving application).

To understand the relationships among the application behavior, the advertised window, and SWS avoidance, we can capture the connection's dynamics in tabular form. Table 15-1 gives the action at the sender and the receiver, as well as an estimated time when the receiving application performs its reads.

Table 15-1 Dynamics of the window advertisement and application to avoid silly window syndrome

Time	Packet Number	Action			Receive Buffer	
		TCP Sender	TCP Receiver	Application	Data	Available
0.000	1	SYN			0	3000
0.000	2		SYN + ACK 1 win 3000		0	3000
0.001	3	ACK			0	3000
0.052	4	1:1460(1460)			1460	1539
0.053	5	1461:2049(588)			2048	952
0.053	6		ACK 2049 win 952		2048	952
5.066	7	2049:3000(952)			3000	0
5.160	8		ACK 3001 win 0		3000	0

(continues)

Table 15-1 Dynamics of the window advertisement and application to avoid silly window syndrome (*continued*)

Time	Packet Number	Action			Receive Buffer	
		TCP Sender	TCP Receiver	Application	Data	Available
6.970	9	3001:3001(1)			3000	0
6.970	10		ACK 3001 win 0		3000	0
10.782	11	3001:3001(1)			3000	0
10.782	12		ACK 3001 win 0		3000	0
15				256 byte read	2744	256
17				256 byte read	2488	512
18.408	13	3001:3001(1)			2489	511
18.408	14		ACK 3002 win 0		2489	511
19				256 byte read	2233	767
21				256 byte read	1977	1023
23				256 byte read	1721	1279
25				256 byte read	1465	1535
25.061	15		ACK 3002 win 1535		1465	1535
25.064	16	3002:4461(1460)			2925	75
25.161	17		ACK 4462 win 75		2925	75
27				256 byte read	2669	331
29				256 byte read	2413	587
30.043	18	4462:4536(75)			2488	512
30.141	19		ACK 4537 win 0		2488	512
31				256 byte read	2232	768
31.548	20	4537:4537(1)			2233	767
31.548	21		ACK 4538 win 767		2233	767
33				256 byte read	1977	1023
35				256 byte read	1721	1279
36.574	22	4538:5304(767)			2488	512
36.671	23		ACK 5305 win 0		2488	512
37				256 byte read	2232	768
37.667	24	5305:5305(1)			2233	767
37.667	25		ACK 5306 win 767		2233	767
39				256 byte read	1977	1023

Table 15-1 Dynamics of the window advertisement and application to avoid silly window syndrome (*continued*)

Time	Packet Number	Action			Receive Buffer	
		TCP Sender	TCP Receiver	Application	Data	Available
41				256 byte read	1721	1279
42.784	26	5306:6073(767)			2488	512
42.881	27		ACK 6074 win 0		2488	512
43				256 byte read	2232	768
43.485	28	6073:6073(1)			2233	767
43.485	29		ACK 6074 win 767		2233	767
43.486	30	6074:6144(71)			2304	696
43.581	31		ACK 6145 win 696		2304	696
43.711	32	6145 (FIN)				
43.711	33		ACK 6146 win 695		2305	695
45,47,49,51 53,55				6x256 byte read	769	2231
55.212	34		ACK 6146 win 2232		768	2232
57,59,61				3x256 byte read	0	3000
63				0 byte read	0	3000
63.252	35		FIN		0	3000

In Table 15-1, the first column is the relative point in time for each action if it appears in the trace. Those times with three digits to the right of the decimal point are taken from the Wireshark output (refer to Figure 15-16). Those times with no digits to the right of the decimal point are the inferred times of the action on the receiving host, which are not represented in the trace.

The amount of data in the receiver's buffer (labeled "Data" in the table) increases when data arrives from the sender and decreases as the application reads (consumes) data from the buffer. What we want to follow are the window advertisements sent by the receiver to the sender, and what those window advertisements contain. This lets us see how the receiver avoids SWS.

As discussed previously, the first evidence of SWS avoidance is the 5s delay between segments 6 and 7, where the sender avoids trying to send with a 952-byte window until it is forced to. When this happens, the receiver fills up, causing a series of zero window advertisements and window probe exchanges. We can see the exponential backoff on the persist timer in action: probes are sent at times 6.970, 10.782, and 18.408. These are approximately 2, 4, and 8s from when the sender first received the zero window advertisement at time 5.160.

Although the application reads data at times 15 and 17, it has read only 512 bytes by time 18.408. The receiver-side SWS avoidance rules dictate that no window update should be provided to the sender because the available 512 bytes of buffer are neither half the size of the total buffer (3000 bytes) nor at least one MSS (1460 bytes). Lacking a window update, the sender sends a window probe at time 18.408 (segment 13). This probe is received and the byte is kept by the receiver, because some buffer space is available, as verified by the increasing ACK number between segments 12 and 14.

Although 511 bytes are available in the receiver's buffer, receiver-side SWS avoidance kicks in once again. The FreeBSD implementation of receiver SWS avoidance differentiates between when to send a window update and how to respond to a window probe. Although it follows the rules in [RFC1122] and sends a window update only when at least half of the total receive buffer (or an MSS) can be advertised, when responding to a window probe it advertises a larger window when the window is either at least an MSS size or when at least *one-fourth* of the total receive buffer size can be advertised. In either case, the 511 bytes are less than a full MSS and also less than $3000/4 = 750$ bytes, so this form of receiver-side SWS avoidance dictates that the window advertisement included in the ACK for segment 13 must contain the value 0.

By the time the application completes its sixth read at time 25, the receive buffer has 1535 bytes free (more than half of the total 3000-byte size), so a window update is sent (segment 15). The sender continues with a full-size segment (segment 16), for which it receives an ACK but a window advertisement of only 75 bytes. In the next 5s, both sender- and receiver-side SWS avoidance takes place. The sender waits for a larger window advertisement, and the application performs reads at times 27 and 29, but the 587 bytes of free receive buffer space are not enough to allow a window update to be sent. The sender therefore has to wait the entire 5s and eventually sends its 75 bytes, forcing the receiver again into SWS avoidance.

With the receiver not providing a window update, the sender's persist timer causes a window probe to be sent at time 31.548. In this case, the FreeBSD receiver responds with a nonzero window, of size 767 bytes (larger than one-fourth of the total receive buffer). This window is not large enough for the sender's SWS avoidance procedure, however, so the sender waits another 5s and the process repeats. Finally, at time 43.486, the last 71 bytes are sent and acknowledged. The acknowledgment contains a window advertisement of 696 bytes. Although it is less than one-quarter of the receiver's total buffer size, the advertisement is not made zero by receiver-side SWS avoidance in order to avoid window shrinkage.

The connection termination begins with segment 32, which contains no data. It is acknowledged immediately with a window advertisement of 695 bytes (the FIN consumed a sequence number at the receiver). After the application completes another six reads, the receiver provides a window update, but the sender is done sending and remains silent. The application performs another four reads, three of which return 256 bytes and the final one of which returns nothing, indicating the

end of arriving data. At this point, the receiver closes the connection, causing the FIN to be sent to the sender. The sender responds with the final ACK, completing the bidirectional closing of the connection.

Because the sending application issues a close operation after performing its three 2048-byte writes, the sender's end of the connection goes from the ESTABLISHED state to the FIN_WAIT_1 state after sending segment 32 (see Chapter 13). It then goes to the FIN_WAIT_2 state after receiving segment 33. Although it receives a window update while in this state, no action is taken, because it has already sent a FIN that has been acknowledged (there is no timer in this state). Instead, it merely sits in this state until receiving a FIN from the other end. This is why we see no further transmissions by the sender until it receives the FIN (segment 35).

15.5.4 Large Buffers and Auto-Tuning

In this chapter, we have seen that an application using a small receive buffer size may be doomed to significant throughput degradation compared to other applications using TCP in similar conditions. Even if the receiver specifies a large enough buffer, the sender might specify too small a buffer, ultimately leading to bad performance. This problem became so important that many TCP stacks now decouple the allocation of the receive buffer from the size specified by the application. In most cases, the size specified by the application is effectively ignored, and the operating system instead uses either a large fixed value or a dynamically calculated value.

In newer versions of Windows (Vista/7) and Linux, receive window *auto-tuning* [S98] is supported. With auto-tuning, the amount of data that can be outstanding in the connection (its bandwidth-delay product, an important concept we discuss in Chapter 16) is continuously estimated, and the advertised window is arranged to always be at least this large (provided enough buffer space remains to do so). This has the advantage of allowing TCP to achieve its maximum available throughput rate (subject to the available network capacity) without having to allocate excessively large buffers at the sender or receiver ahead of time. In Windows, the receiver's buffer size is auto-sized by the operating system by default. However, the behavior can be modified using the netsh command:

```
C:\> netsh interface tcp set heuristics disabled
```

```
C:\> netsh interface tcp set global autotuninglevel=X
```

where *X* is one of the following: *disabled*, *highlyrestricted*, *restricted*, *normal*, or *experimental*. The setting affects the automatic selection of the receiver's advertised window. In the disabled state, auto-tuning is not used, and the window size uses a default value. The restricted modes slow the window growth, and the normal setting allows it to grow relatively quickly. The experimental mode allows the window to grow very aggressively but is not recommended for

normal use because many Internet sites and some firewalls interfere with or fail to implement the TCP Window Scale option properly.

With Linux 2.4 and later, sender-side auto-tuning is supported. With version 2.6.7 and later, both receiver- and sender-side auto-tuning is supported. However, auto-tuning is subject to limits placed on the buffer sizes. The following Linux `sysctl` variables control the sender and receiver maximum buffer sizes. The values after the equal sign are the default values (which may vary depending on the particular Linux distribution), which should be increased if the system is to be used in high bandwidth-delay-product environments:

```
net.core.rmem_max = 131071
net.core.wmem_max = 131071
net.core.rmem_default = 110592
net.core.wmem_default = 110592
```

In addition, the auto-tuning parameters are given by the following variables:

```
net.ipv4.tcp_rmem = 4096 87380 174760
net.ipv4.tcp_wmem = 4096 16384 131072
```

Each of these variables contains three values: the minimum, default, and maximum buffer size used by auto-tuning.

15.5.4.1 Example

To demonstrate the behavior of receiver auto-tuning, we use a Windows XP sender (set to use large windows and window scaling) and a Linux 2.6.11 receiver that includes auto-tuning. At the sender, we issue the following command:

```
C:\> sock -n 512 -i 10.0.0.1 6666
```

At the receiver, we do not specify any setting for the receive buffer, but we do arrange for an initial delay of 20s before the application performs any reads:

```
Linux% sock -i -s -v -P 20 6666
```

To illustrate the growth of the receiver's advertised window, we can use Wireshark to sort the displayed packets based on the receiver's address (see Figure 15-15). During connection establishment, the receiver begins with an initial window size of 1460 bytes and an initial MSS of 1412 bytes. It is using window scaling, with a shift amount of 2 (not shown), leading to a maximum usable window of 256KB. We can see that after the initial packets, the window increases, which corresponds to the sender's increase in the data sending rate. We explore the sender's data rate control when we investigate TCP congestion control in Chapter 16. For now, we need only know that when the sender starts up, it typically starts by sending one packet and then increases the amount of outstanding data by one MSS packet for

No.	Time	Source	Destination	Protocol	Info
2	0.014114	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [SYN, ACK] Seq=0 Ack=1 win=1460 Len=0 MSS=1412 SACK_PERM
5	0.078100	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=1025 win=7888 Len=0
8	0.136991	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=2437 win=10712 Len=0
11	0.172750	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=3849 win=13536 Len=0
14	0.214482	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=5261 win=16360 Len=0
17	0.249142	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=6673 win=19184 Len=0
20	0.284069	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=8085 win=22008 Len=0
22	0.325937	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=9497 win=24832 Len=0
26	0.356503	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=10909 win=27656 Len=0
27	0.366867	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=11265 win=27656 Len=0
29	0.399342	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=12677 win=30480 Len=0
31	0.439019	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=14337 win=33304 Len=0
35	0.498645	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=16773 win=33304 Len=0
38	0.543485	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=18433 win=33304 Len=0
41	0.599412	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=20481 win=33304 Len=0
44	0.678770	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=23941 win=29844 Len=0
47	0.732456	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=25989 win=27796 Len=0
51	0.784443	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=28037 win=25748 Len=0
54	0.842815	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=30085 win=23700 Len=0
57	0.900957	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=31745 win=22040 Len=0
60	0.982327	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=35841 win=17944 Len=0
65	1.070205	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=39301 win=14484 Len=0
68	1.124690	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=41349 win=12436 Len=0
72	1.140705	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=41985 win=11800 Len=0
73	1.179343	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=43397 win=10388 Len=0
75	1.198797	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=44033 win=11392 Len=0
77	1.239476	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=5445 win=14216 Len=0
79	1.253527	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=46081 win=17040 Len=0
81	1.275912	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=47105 win=19864 Len=0
83	1.359920	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=50177 win=16792 Len=0
87	1.444195	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=53249 win=13720 Len=0
91	1.521311	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=57345 win=9624 Len=0
97	1.597698	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=60417 win=6552 Len=0
99	1.622798	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=61441 win=5744 Len=0
100	1.659010	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=62853 win=8568 Len=0
103	1.674557	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=63489 win=11340 Len=0
105	1.762732	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=66949 win=7880 Len=0
109	1.860567	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=70657 win=4172 Len=0
111	1.916495	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=73093 win=5740 Len=0
114	1.989704	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=74753 win=4080 Len=0
115	2.085549	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=77577 win=1256 Len=0
117	6.982406	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=0 Len=0
119	7.657160	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] 6666 > 1198 [ACK] Seq=1 A
121	9.053792	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 A
123	11.971445	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 A
125	17.568786	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 A
126	20.043057	128.32.37.244	10.0.0.100	TCP	[TCP Window Update] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=2920 Len=0

Figure 15-15 The Linux receiver performs receiver-side auto-tuning by increasing the window as more data is received. Because the application does not read for 20s, the window eventually closes.

each ACK it receives that indicates progress. Thus, it typically sends two MSS-size segments for each ACK it receives.

Looking at the pattern of the window advertisements—10712, 13536, 16360, 19184, . . .—we can see that the advertised window is increased by twice the MSS on each ACK, which mimics the way the sender’s congestion control scheme operates, as we shall see in Chapter 16. Provided enough memory is available at the receiver, the advertised window is always larger than what the sender is permitted to send according to its congestion control limitations. This is the best case—the minimal amount of buffer space is being used and advertised by the receiver that keeps the sender sending as fast as possible.

If the receiver exhausts its buffers, auto-tuning is compromised. In this example, by time 0.678 the pattern of window growth reverses, having achieved a maximum of 33,304 bytes. The window size is no longer increasing, but instead the buffer is filling up while the application pauses. When the application begins reading at time 20, the window size again increases and goes beyond the point where it was previously (see Figure 15-16).

The image shows a Wireshark packet capture titled 'win-scaledwin-linux-autotune.td - Wireshark'. The packet list pane displays the following data:

No.	Time	Source	Destination	Protocol	Info
117	6.982406	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=0 Len=0
119	7.657160	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=0 Len=0
121	9.053792	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=0 Len=0
123	11.971445	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=0 Len=0
125	17.568786	128.32.37.244	10.0.0.100	TCP	[TCP ZeroWindowProbeAck] [TCP ZeroWindow] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=0 Len=0
126	20.043057	128.32.37.244	10.0.0.100	TCP	[TCP window update] 6666 > 1198 [ACK] Seq=1 Ack=78833 win=2920
129	20.106509	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=80245 win=5744 Len=0
133	20.148410	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=81657 win=8568 Len=0
136	20.186607	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=83069 win=11392 Len=0
140	20.223310	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=84481 win=14216 Len=0
141	20.255382	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=85893 win=17040 Len=0
142	20.291812	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=87305 win=19864 Len=0
143	20.311257	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=88065 win=22688 Len=0
144	20.335689	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=89089 win=25512 Len=0
150	20.375068	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=90501 win=28336 Len=0
152	20.389615	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=91137 win=31160 Len=0
154	20.422357	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=92161 win=33984 Len=0
156	20.453392	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=93573 win=36808 Len=0
158	20.468183	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=94209 win=36808 Len=0
160	20.501434	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=95621 win=39632 Len=0
162	20.515513	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=96257 win=39632 Len=0
164	20.541105	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=97281 win=42456 Len=0
166	20.570910	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=98305 win=44504 Len=0
168	20.594064	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=99329 win=46552 Len=0
170	20.618938	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=100353 win=48600 Len=0
172	20.648033	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=101377 win=50648 Len=0
173	20.677835	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=102401 win=52696 Len=0
174	20.703447	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=103425 win=54744 Len=0
175	20.727638	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=104449 win=56792 Len=0
176	20.750252	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=105473 win=58840 Len=0
182	20.782296	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=106497 win=60888 Len=0
184	20.810362	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=107521 win=62936 Len=0
186	20.835262	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=108545 win=64984 Len=0
188	20.872206	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=109577 win=67808 Len=0
190	20.885752	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=110593 win=67808 Len=0
192	20.935266	128.32.37.244	10.0.0.100	TCP	6666 > 1198 [ACK] Seq=1 Ack=112641 win=67808 Len=0

Figure 15-16 With the application pausing before reading, auto-tuning is compromised because the receive buffer becomes full. As the application begins reading, the advertised window increases, exceeding its previous value.

The zero window advertisement (packet 117) forces the sender to perform a series of window probes, resulting in a series of zero window advertisements. After the application begins reading at time 20.043, a window update is sent to the sender. The window begins to grow once again, twice the MSS in bytes for each ACK. As the sender continues to send additional data and the receiver consumes it, the receiver continues to increase the advertised window until the value

67808 is reached, which is the largest value the receiver ever advertises on this connection. This version of Linux also measures the time between adjacent application read completions and compares this value against the estimated connection round-trip time. If the RTT estimate increases, the buffer size is also increased (it is not decreased if the RTT becomes smaller). This helps auto-tuning keep the receiver's advertised window ahead of the sender's window even when the connection's bandwidth-delay product is increasing.

The problem of TCP applications using too-small buffers became a significant one as faster wide area Internet connections became available. In the United States, with cross-country round-trip times of approximately 100ms, using a 64KB window over a 1Gb/s network limits TCP throughput to about 640KB/s instead of the calculated maximum of about 130MB/s (a 99% waste of bandwidth). Practically speaking, it is not uncommon to see a factor of 100 increase in throughput performance when moving from a TCP with limited buffers to one with larger buffers on such networks. Significant credit should be given to the Web100 project [W100]. It created a set of tools and software patches in an effort to maximize the available throughput performance an application can obtain from various TCP implementations.

15.6 Urgent Mechanism

We saw in Chapter 12 that the TCP header has a special *URG* bit field to indicate “urgent data.” An application is able to mark data as urgent by specifying a special option to the Berkeley sockets API (`MSG_OOB`) when it performs a write operation, although the use of urgent data is no longer recommended [RFC6093]. When the sender's TCP receives such a write request, it enters a special state called *urgent mode*. Upon entering urgent mode, it records the last byte the application specified as urgent data. This is used to set the *Urgent Pointer* field in each subsequent TCP header the sender generates until the application ceases writing urgent data and all the sequence numbers up to the urgent pointer have been acknowledged by the receiver. According to [RFC6093], the urgent pointer points to the sequence number of the byte of data following the last byte of urgent data. This resolves a longstanding ambiguity in various RFCs that included contradictory statements about the semantics of the *Urgent Pointer* field. When an IPv6 jumbogram is used, the *Urgent Pointer* value of 65535 may be used to indicate the end of urgent data is to be found at the end of the TCP data area [RFC2675], beyond the 64K byte offset expressible using the conventional 16-bit *Urgent Pointer* field.

A receiving TCP enters urgent mode when it receives a segment with the *URG* bit field set. The receiving application can discover whether its TCP has entered urgent mode using a standard socket API call (`select()`). The operation of the urgent mechanism has been a source of confusion because the Berkeley sockets API and documentation use the term *out-of-band* (OOB) data, although in reality TCP does not implement any true OOB capability. Instead, virtually all TCP

TCP Congestion Control

16.1 Introduction

In this chapter we investigate how TCP approaches the issue of *congestion control*, which is most important in the context of bulk data transfers. Congestion control is a set of behaviors determined by algorithms that each TCP implements in an attempt to prevent the network from being overwhelmed by too large an aggregate offered traffic load. The basic approach is to have TCP slow down when it has reason to believe the network is about to be congested (or is already so congested that routers are discarding packets). The challenge is to determine exactly when and how TCP should slow down, and when it can speed up again.

TCP is a protocol designed to provide reliable delivery of data from one system to another. We have already seen in Chapter 15 how a sending TCP can be made to slow down if its peer (receiving) TCP cannot keep up. This is accomplished by TCP's procedures for *flow control* and is realized by a sender adapting its sending rate based on the advertised *Window Size* field provided by a receiver in its ACKs. This provides explicit information about the state of the receiver back to the sender and allows it to avoid overrunning the receiver's buffers.

Consider what happens when the network between a collection of senders and receivers is asked to carry more traffic than it can handle. Either the senders must slow down or the network must ultimately throw some data away (or some combination thereof). This fact arises from the most basic observation from queuing theory as applied at a router: even if the router can store some data, if the long-term data arrival rate exceeds the long-term departure rate, any amount of intermediate storage will grow without bound. Stated more simply, if a router receives more data per unit time than it can send out, it must store that data. If this situation persists, eventually the storage will run out and the router will be forced to drop some of the data.

This situation, when a router is forced to discard data because it cannot handle the arriving traffic rate, is called *congestion*. The router is said to be *congested* when it is in this state, and even a single connection can drive one or more routers into

congestion. Left unaddressed, congestion can cause the performance of a network to be reduced so badly that it becomes unusable. In the very worst cases, it is said to be in a state of *congestion collapse*. To either avoid or at least react effectively to mitigate this situation, each TCP implements *congestion control procedures*. Different versions or variants of TCP (and the operating systems that host the TCP/IP stack) have somewhat different procedures and behaviors. We will discuss most of the better-known ones in this chapter.

16.1.1 Detection of Congestion in TCP

As we have seen, the primary mechanism TCP has available to combat packet loss is retransmission, induced either by a retransmission timer expiring, or by the fast retransmit algorithm (see Chapter 14). Consider, for a moment, the consequence of many TCP connections that share an Internet path simply retransmitting more packets while the network is in a state of congestion collapse. As you can imagine, this only makes the situation worse. It has been called the analog of pouring gasoline on a fire and is something to be avoided.

In order to deal with congestion, we would like to have sending TCPs slow down when congestion is present (or about to be) and, if the congestion has subsided, detect and use an appropriate amount of new bandwidth when it becomes available. In the Internet, this can be quite challenging, as there has traditionally been no explicit way for a sending TCP to learn about the state of the intermediate routers. In other words, there is no *explicit signaling* about congestion. Instead, if a typical TCP is to react somehow to congestion, it must first conclude that congestion is occurring. This is usually accomplished by detecting that one or more packets have been lost. In TCP, an assumption is made that a lost packet is an indicator of congestion, and that some response (i.e., slowing down in some way) is required. We shall see that TCP has been this way since the late 1980s. Other methods for detecting congestion, including measuring delay and network-supported *Explicit Congestion Notification* (ECN), which we discuss in Section 16.11, allow TCP to learn about congestion even before it has become so bad as to cause dropped packets. We discuss these approaches after studying the “classic” algorithms.

Note

In today’s wired networks, packet loss is caused primarily by congestion in routers or switches. With wireless networks, transmission and reception errors become a significant cause of packet loss. Determining whether loss is due to congestion or transmission errors has been an active research topic since the mid-1990s when wireless networks started to attain widespread use.

In Chapter 14 we saw how TCP can use timers, acknowledgments, and selective acknowledgments to detect and recover from dropped packets. When packets are detected as lost, it is TCP’s responsibility to resend them. We are now concerned

with what else TCP does when it observes a lost packet. In particular, we are interested in how it interprets this as a signal that congestion has occurred, and that it should slow down. Just *how* it slows down and *when* (and how it speeds back up again) are the main subjects of the following sections. We begin with the classic algorithm used on a new connection to establish the base data rate and continue with another classic algorithm that is used by TCP during its steady-state operation when performing large data transfers. We will also incorporate the recommended variations on these algorithms into the discussion and discuss other modifications that have been made over the years. We will also examine an extended trace in detail. We conclude with a discussion of some of the security issues related to TCP congestion control and summarize the most important points. The area of congestion control has been a fertile area for networking researchers [RFC6077], and several new papers on this subject tend to appear each year.

16.1.2 Slowing Down a TCP Sender

One detail we need to address right away is just how to slow down a TCP sender. We saw in Chapter 15 that the *Window Size* field in the TCP header is used to signal a sender to adjust its window based on the availability of buffer space at the receiver. We can go a step further and arrange for the sender to slow down if either the receiver is too slow or the network is too slow. This is accomplished by introducing a window control variable at the sender that is based on an estimate of the network's capacity and ensuring that the sender's window size never exceeds the minimum of the two. In effect, a sending TCP then sends at a rate equal to what the receiver or the network can handle, whichever is less.

The new value used to hold the estimate of the network's available capacity is called the *congestion window*, written more compactly as simply *cwnd*. The sender's actual (usable) window *W* is then written as the minimum of the receiver's advertised window *awnd* and the congestion window:

$$W = \min(cwnd, awnd)$$

With this relationship, the TCP sender is not permitted to have more than *W* unacknowledged packets or bytes outstanding in the network. The total amount of data a sender has introduced into the network for which it has not yet received an acknowledgment is sometimes called the *flight size*, which is always less than or equal to *W*. In general, *W* can be maintained in either packet or byte units.

Note

When TCP does not make use of selective acknowledgment, the restriction on *W* means that the sender is not permitted to send a segment with a sequence number greater than the sum of the highest acknowledged sequence number and the value of *W*. A SACK TCP sender treats *W* somewhat differently, using it as an overall limit to the flight size.

This all seems logical but is far from the whole story. Because both the state of the network and the state of the receiver change with time, the values of both *awnd* and *cwnd* change over time. In addition, because of the lack of explicit signals (see the preceding section), the “correct” value of *cwnd* is generally not directly available to the sending TCP. Thus, all of the values W , *cwnd*, and *awnd* must be empirically determined and dynamically updated. In addition, as we said before, we do not want W to be too big or too small—we want it to be set to about the *bandwidth-delay product* (BDP) of the network path, also called the *optimal* window size. This is the amount of data that can be stored in the network in transit to the receiver. It is equal to the product of the RTT and the capacity of the lowest capacity (“bottleneck”) link on the path from sender to receiver. Generally, the sending strategy is to keep the network busy by arranging to have an amount of data at least as large as the BDP in the network. Using an outstanding limit that substantially exceeds the BDP, however, is usually undesirable as it can lead to unwanted delays (see Section 16.10). On the Internet, determining the BDP for a connection can be challenging, given that routes, delay, and the level of statistical multiplexing (i.e., sharing of capacity) change as a function of time.

Note

Although handling congestion at the TCP sender is our primary area of interest, work has been done on handling the cases where congestion occurs on the reverse path, because of ACKs. In [RFC5690] a method is introduced to inform a TCP receiver of the ACK ratio it should use (i.e., how many packets it should receive before sending an ACK).

16.2 The Classic Algorithms

When a new TCP connection first starts out, it usually has no idea what the initial value for *cwnd* should be, as it has no idea how much network capacity is available for it to send its data. (There are some exceptions, such as systems that cache performance values that were determined earlier. These were called destination metrics in Chapter 14.) TCP learns the value for *awnd* with one packet exchange to the receiver, but without any explicit signaling, the only obvious way it has to learn a good value for *cwnd* is to try sending data at faster and faster rates until it experiences a packet drop (or other congestion indicator). This could be accomplished by either sending immediately at the maximum rate it can (subject to the value of *awnd*), or it could start more slowly. Because of the detrimental effects on the performance of other TCP connections sharing the same network path that could be experienced when starting at full rate, a TCP generally uses one algorithm to avoid starting so fast when it starts up to get to steady state. It uses a different one once it is in steady state.

The operation of TCP congestion control at a sender is driven or “clocked” by the receipt of ACKs. If a TCP is operating at steady state (with an appropriate value

of *cwnd*), receipt of an ACK indicates that one or more packets have been removed from the network, and consequently that an opportunity to send more has arisen. Following this line of reasoning, the TCP congestion behavior in steady state attempts to achieve a *conservation of packets* in the network (see Figure 16-1). The term conservation here is used in the sense it is in physics—that some quantity (e.g., momentum, energy) going into a system does not simply disappear or appear but rather can be found as long as proper accounting is performed.

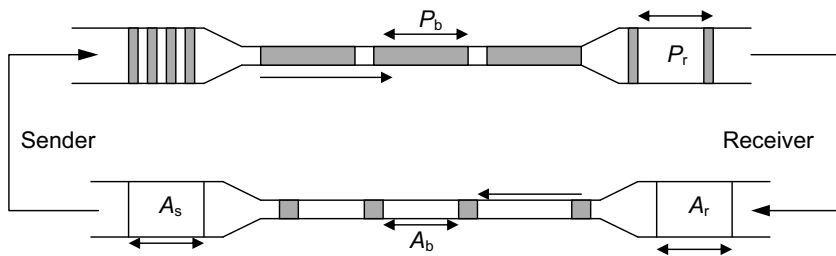


Figure 16-1 TCP congestion control operates on a principle of conservation of packets. Packets (P_b) are “stretched out” in time as they are sent from sender to receiver over links with constrained capacity. As they are received at the receiver spaced apart (P_r), ACKs are generated (A_r), which return to the sender. ACKs traveling from receiver to sender become spaced out (A_b) in relation to the inter-packet spacing of the packets. When ACKs reach the sender (A_s), their arrivals provide a signal or “ACK clock,” used to tell the sender it is time to send more. In steady state, the overall system is said to be “self-clocked.” The figure is adapted from [J88] and copied from S. Seshan’s CMU Lecture Notes dated March 22, 2005.

This idea is illustrated in Figure 16-1. We shall call the top and bottom objects “funnels.” The top funnel holds (larger) data packets traveling along the path from the sender to the receiver. The comparatively narrow width of the funnel depicts how packets are “stretched out” in time as they travel through a relatively slow link. The ends of the funnels (at sender and receiver) show the queues where packets are held before or after they travel along the path. The bottom funnel holds the ACKs sent by the receiver back to the sender that correspond to the data packets in the top funnel. When operating efficiently at steady state, there are no bunches of packets in the top or bottom funnels. In addition, there is no significant extra space between packets in the top funnel. Note that an arrival of an ACK at the sender “liberates” another data packet to be sent into the top funnel, and that this happens at just the right time (i.e., when the network is able to accept another packet). This relationship is sometimes called *self-clocking*, because the arrival of an ACK, called the *ACK clock*, triggers the system to take the action of sending another packet.

We now turn to the main two algorithms of TCP: slow start and congestion avoidance. These algorithms, based on the principles of packet conservation and ACK clocking, were first formally described in the classic paper by Jacobson [J88].

An update to the congestion avoidance algorithm was given by Jacobson a couple of years later [J90]. These algorithms do not operate at the same time—TCP executes only one at any given time, but it may switch back and forth between the two. We now explore these in more detail and examine what determines when each of them is used. We also look at how they have been modified and extended since they were initially implemented. Each TCP connection is able to individually execute these algorithms.

16.2.1 Slow Start

The slow start algorithm is executed when a new TCP connection is created or when a loss has been detected due to a retransmission timeout (RTO). It may also be invoked after a sending TCP has gone idle for some time. The purpose of slow start is to help TCP find a value for *cwnd* before probing for more available bandwidth using congestion avoidance and to establish the ACK clock. Typically, a TCP begins a new connection in slow start, eventually drops a packet, and then settles into steady-state operation using the congestion avoidance algorithm (Section 16.2.2). To quote from [RFC5681]:

Beginning transmission into a network with unknown conditions requires TCP to slowly probe the network to determine the available capacity, in order to avoid congesting the network with an inappropriately large burst of data. The slow start algorithm is used for this purpose at the beginning of a transfer, or after repairing loss detected by the retransmission timer.

A TCP begins in slow start by sending a certain number of segments (after the SYN exchange), called the *initial window* (IW). The value of IW was originally one SMSS, although with [RFC5681] it is allowed to be larger. The formula works as follows:

$IW = 2 * (SMSS)$ and not more than 2 segments (if $SMSS > 2190$ bytes)

$IW = 3 * (SMSS)$ and not more than 3 segments (if $2190 \geq SMSS > 1095$ bytes)

$IW = 4 * (SMSS)$ and not more than 4 segments (otherwise)

While this assignment for IW may allow several packets (e.g., three or four) in the initial window, we shall discuss the case where $IW = 1$ SMSS for simplicity. A TCP just starting out begins its connection, then, with $cwnd = 1$ SMSS, meaning the initial usable window W is also equal to SMSS. Note that in most cases SMSS is equal to the smaller of the receiver's MSS and the path MTU (less header sizes).

Assuming no packets are lost and each packet causes an ACK to be sent in response, an ACK is returned for the first segment, allowing the sending TCP to send another segment. However, slow start operates by incrementing *cwnd* by $\min(N, SMSS)$ for each good ACK received, where N is the number of previously

unacknowledged bytes ACKed by the received “good ACK.” A good ACK is one that returns a higher ACK number than has been seen so far.

Note

The number of bytes ACKed is used to support *Appropriate Byte Counting* (ABC) [RFC3465], an experimental specification recommended by [RFC5681]. It can be used to counter an “ACK division” attack, described in Section 16.12, where many small ACKs are used in an attempt to cause a TCP sender to send faster than normal. Linux uses the Boolean system configuration variable `net.ipv4.tcp_abc` to determine if ABC is enabled (default no). In recent versions of Windows, ABC defaults to on.

Thus, after one segment is ACKed, the *cwnd* value is ordinarily increased to 2, and two segments are sent. If each of those causes new good ACKs to be returned, 2 increases to 4, 4 to 8, and so on. In general, assuming no loss and an ACK for every packet, the value of *W* after *k* round-trip exchanges is $W = 2^k$. Rewriting, we can say that $k = \log_2 W$ RTTs are required to reach an operating window of *W*. This growth seems quite “fast” (increasing as an exponential function) but is still “slower” than what TCP would do if it were allowed to send immediately a window of packets equal in size to the receiver’s advertised window. (Recall that *W* is still never allowed to exceed *awnd*.)

If we imagine a TCP connection where the receiver’s advertised window is very large (say, infinitely large), *cwnd* is the primary governor of the sending rate (provided there is something for the sender to send). As we saw, this value grows exponentially fast in the RTT of the connection. So, eventually, *cwnd* (and thus *W*) could become so large that the corresponding window of packets sent overwhelms the network (recall that TCP’s throughput rate is proportional to W/RTT). When this happens, *cwnd* is reduced substantially (to half of its former value). In addition, this is the point at which TCP switches from operating in slow start to operating in congestion avoidance. The switch point is determined by the relationship between *cwnd* and a value called the *slow start threshold* (or *ssthresh*).

Figure 16-2 (left) illustrates the operation of slow start. The numbers are in units of the RTT of the connection. Assuming the connection starts out with one packet (top), one ACK is returned, allowing two packets to be sent during the second RTT. These packets cause two ACKs to be returned. The TCP sender increments *cwnd* by one segment for each ACK returned, so the process continues. The exponential growth of *cwnd* as a function of time is illustrated on the right. The second line shows how *cwnd* grows when every other packet is acknowledged, which is common when delayed ACKs are being used. In this case, the growth is still exponential but not as rapid. For this reason, some TCPs arrange to delay ACKs only after the connection has completed slow start. In Linux, this is called *quick acknowledgments* (“quickack mode”) and has been part of the basic TCP/IP stack since kernel version 2.4.4.

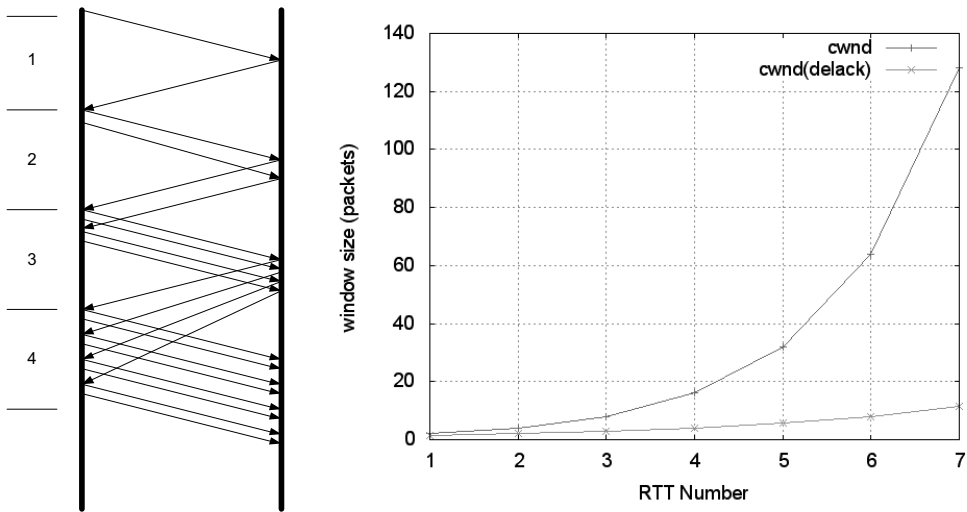


Figure 16-2 Operation of the classic slow start algorithm. In the simple case where ACKs are not delayed, every arriving good ACK allows the sender to inject two new packets (left). This leads to an exponential growth in the size of the sender's window as a function of time (right, upper line). When ACKs are delayed, such as when an ACK is produced for every other packet, the growth is still exponential but slower (right, lower line).

16.2.2 Congestion Avoidance

Slow start, just described, is used when initiating data flow across a connection or after a loss event invoked by a timeout. It increases *cwnd* fairly rapidly and helps to establish a value for *ssthresh*. Once this is achieved, there is always the possibility that more network capacity may become available for a connection. If such capacity were to be immediately used with large traffic bursts, other TCP connections with packets sharing the same queues in routers would likely experience significant packet drops, leading to overall instability in the network as many connections simultaneously experience packet drops and react with retransmissions.

To address the problem of trying to find additional capacity that may become available, but to not do so too aggressively, TCP implements the *congestion avoidance* algorithm. Once *ssthresh* is established and *cwnd* is at least at this level, a TCP runs the congestion avoidance algorithm, which seeks additional capacity by increasing *cwnd* by approximately one segment for each window's worth of data that is moved from sender to receiver successfully. This provides a much slower growth rate than slow start: approximately linear in terms of time, as opposed to slow start's exponential growth. More precisely, *cwnd* is usually updated as follows for each received nonduplicate ACK:

$$cwnd_{t+1} = cwnd_t + SMSS * SMSS / cwnd_t$$

Looking at this relationship briefly, assume $cwnd_0 = k \cdot SMSS$ bytes were sent into the network in k segments. After the first ACK arrives, $cwnd$ is updated to be larger by a factor of $(1/k)$:

$$cwnd_1 = cwnd_0 + SMSS \cdot SMSS / cwnd_0 = k \cdot SMSS + SMSS \cdot (SMSS / (k \cdot SMSS)) = k \cdot SMSS + (1/k) \cdot SMSS = (k + (1/k)) \cdot SMSS = cwnd_0 + (1/k) \cdot SMSS$$

Because the value of $cwnd$ grows slightly with each new ACK arrival, and this value is in the denominator of the expression in the first equation above, the overall growth rate of $cwnd$ is slightly sublinear. Nonetheless, we generally think of congestion avoidance growing the window linearly with respect to time (Figure 16-3), whereas slow start grows it exponentially with respect to time (Figure 16-2). This function is also called *additive increase* because a particular value (about one packet in this case) is added to $cwnd$ for each successfully received window's worth of data.

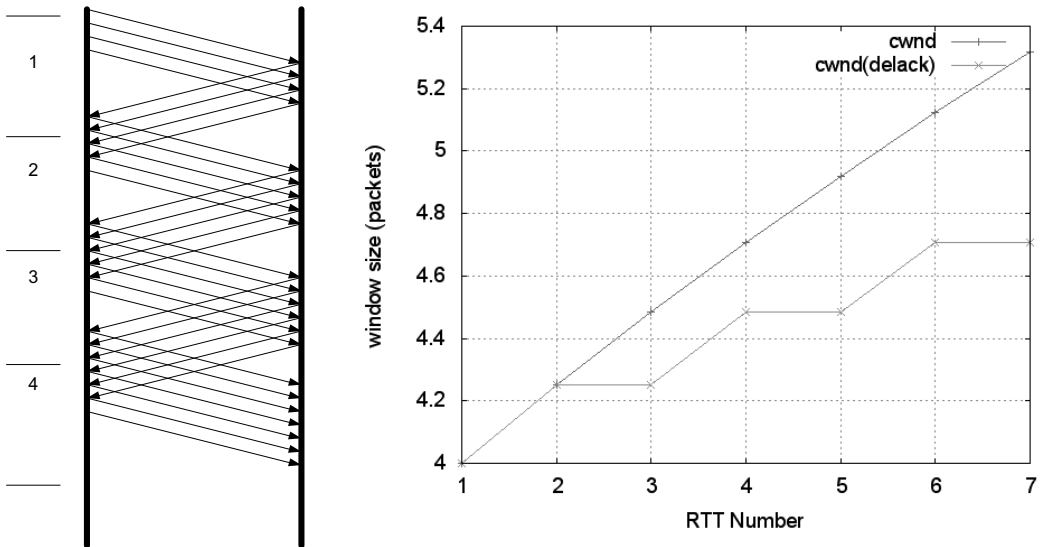


Figure 16-3 Operation of the congestion avoidance algorithm. In the simple case where ACKs are not delayed, every arriving good ACK allows the sender to inject approximately $1/W$ fraction of a new packet. This leads to approximately linear growth in the size of the sender's window as a function of time (right, upper line). When ACKs are delayed, such as when an ACK is produced for every other packet, the growth is still approximately linear but somewhat slower (right, lower line).

Figure 16-3 (left) illustrates the operation of congestion avoidance. Once again, the numbers are in units of the RTT of the connection. Assuming the connection sends four packets (top), four ACKs are returned, allowing $cwnd$ to grow slightly. By the second RTT period, the growth is enough to overcome the integer rounding and cause an increase of one $SMSS$ to $cwnd$, allowing one additional packet to be

sent. The growth of *cwnd* as a nearly linear function of time is illustrated on the right, on a linear-linear plot. The second line to the right shows how *cwnd* grows when every other packet is acknowledged, simulating the use of delayed ACKs. In this case, the growth is still about linear, but not as rapid.

The assumption of the algorithm is that packet loss caused by bit errors is very small (much less than 1%), and therefore the loss of a packet signals congestion somewhere in the network between the source and destination. If this assumption is false, which it sometimes is for wireless networks, TCP slows down even when no congestion is present. In addition, many RTTs may be required for the value of *cwnd* to grow large, which is required for efficient use of networks with high capacity. Fixing these issues with TCP has been a popular area for research, and we discuss some of the various approaches later.

16.2.3 Selecting between Slow Start and Congestion Avoidance

In normal operations, a TCP connection is always running either the slow start or the congestion avoidance procedure, but never the two simultaneously. We now turn to the question, What determines the algorithm TCP uses at any given time? We already know that slow start is used when a new connection is created or when a timeout-based retransmission occurs. We now turn to what controls the selection between slow start and congestion avoidance.

We mentioned *ssthresh* earlier. This threshold is a limit on the value of *cwnd* that determines which algorithm is in operation, slow start or congestion avoidance. When *cwnd* < *ssthresh*, slow start is used, and when *cwnd* > *ssthresh*, congestion avoidance is used. When they are equal, either can be used. The most important distinction between slow start and congestion avoidance, as we have seen, is how each modifies the value of *cwnd* when new ACKs arrive. What makes TCP somewhat tricky and interesting is that the value of *ssthresh* is not fixed but instead varies over time. Its main purpose is to remember the last “best” estimate of the operating window when no loss was present. Said another way, it holds the lower bound on TCP’s best estimate of the optimal window size.

The initial value of *ssthresh* may be set arbitrarily high (e.g., to *awnd* or higher), which causes TCP to always start with slow start. When a retransmission occurs, caused by either a retransmission timeout or the execution of fast retransmit, *ssthresh* is updated as follows:

$$ssthresh = \max(\text{flight size}/2, 2*SMSS) \quad [1]$$

Note

In Microsoft’s most recent (“Next Generation”) TCP/IP stack, this equation is reportedly changed to the somewhat more conservative relationship: *ssthresh* = $\max(\min(cwnd, awnd)/2, 2*SMSS)$ [NB08].

Here we see that if a retransmission is required, TCP assumes that the operating window must have been too large for the network to handle. Reducing the estimate of the optimal window size is accompanied by altering *ssthresh* to be about half of what the current window size is (but not ever below twice the SMSS). This usually results in lowering *ssthresh*, but it can also result in increasing *ssthresh*. If we examine the congestion avoidance procedure for TCP, we recall that if an entire window's worth of data is successfully exchanged, the value of *cwnd* is allowed to increase by approximately 1 SMSS. Thus, if *cwnd* has grown large over a considerable amount of time, setting *ssthresh* to half of the flight size could cause it to increase. This happens when TCP has discovered more usable bandwidth. The interplay between *ssthresh* and *cwnd*, in conjunction with the operation of slow start and congestion avoidance, gives TCP its characteristic behavior in the face of congestion. We now explore the complete, combined algorithms.

16.2.4 Tahoe, Reno, and Fast Recovery

The algorithms discussed so far, slow start and congestion avoidance, constitute the first congestion control algorithms applied to TCP. They were introduced in the late 1980s with the 4.2 release of UC Berkeley's version of UNIX, called the *Berkeley Software Distribution*, or BSD UNIX. Thus began the convention of naming various versions of TCP after U.S. cities, especially those where gambling is permitted.

The 4.2 release of BSD (called Tahoe) included a version of TCP that started connections in slow start, and if a packet was lost, detected by either a timeout or the fast retransmit procedure, the slow start algorithm was reinitiated. Tahoe was implemented by simply reducing *cwnd* to its starting value (1 SMSS at that time) upon any loss, forcing the connection to slow start until *cwnd* grew to the value *ssthresh*.

One problem with this approach is that for large BDP paths, this can cause the connection to significantly underutilize the available bandwidth while the sending TCP goes through slow start to get back to the point at which it was operating before the packet loss. To address this problem, the reinitiation of slow start on any packet loss was reconsidered. Ultimately, if packet loss is detected by duplicate ACKs (invoking fast retransmit), *cwnd* is instead reset to the last value of *ssthresh* instead of only 1 SMSS. (Slow start is still initiated on a timeout, which is generally the case for most TCP variants.) This approach allows the TCP to slow down to half of its previous rate without reverting to slow start.

In exploring the issue of large BDP paths further and thinking back to the conservation of packets principle mentioned before, it has been observed that any ACKs that are received, even while recovering after a loss, still represent opportunities to inject new packets into the network. This became the basis of the *fast recovery* procedure, which was released in conjunction with the popular 4.3 BSD Reno version of BSD UNIX. Fast recovery allows *cwnd* to (temporarily) grow by 1 SMSS for each ACK received while recovering. The congestion window is therefore

inflated for a period of time, allowing an additional new packet to be sent for each ACK received, until a good ACK is seen. Any nonduplicate (“good”) ACK causes TCP to exit recovery and reduces the congestion back to its pre-inflated value. TCP Reno became very popular and ultimately the basis for what might reasonably be called “standard TCP.”

16.2.5 Standard TCP

Although what constitutes “standard” TCP is subject to some debate, the algorithms we have discussed so far constitute the primary procedures identified with standard TCP operation. The slow start and congestion avoidance algorithms are usually implemented together, and the baseline overall behavior is given in [RFC5681]. This specification does not require the use of these exact algorithms, but a requirement is imposed that any TCP implementation not be more aggressive than these algorithms would allow.

To summarize the combined algorithm from [RFC5681], TCP begins a connection in slow start ($cwnd = IW$) with a large value of $ssthresh$, generally at least the value of $awnd$. Upon receiving a good ACK (one that acknowledges new data), TCP updates the value of $cwnd$ as follows:

$$\begin{array}{lll} cwnd += SMSS & (\text{if } cwnd < ssthresh) & \text{Slow start} \\ cwnd += SMSS * SMSS / cwnd & (\text{if } cwnd > ssthresh) & \text{Congestion avoidance} \end{array}$$

When fast retransmit is invoked because of receipt of a third duplicate ACK (or other signal, if conventional fast retransmit initiation is not used), the following actions are performed:

1. $ssthresh$ is updated to no more than the value given in equation [1].
2. The fast retransmit algorithm is performed, and $cwnd$ is set to $(ssthresh + 3 * SMSS)$.
3. $cwnd$ is temporarily increased by $SMSS$ for each duplicate ACK received.
4. When a good ACK is received, $cwnd$ is reset back to $ssthresh$.

The actions in steps 2 and 3 constitute *fast recovery*. Step 2 first adjusts $cwnd$, which usually causes it to be reduced to half of its former value, and then temporarily inflates it to take into account the fact that the receipt of each duplicate ACK indicates that some packet has left the network (and thus should permit another to be inserted). This step is also where *multiplicative decrease* occurs, as $cwnd$ is ordinarily multiplied by some value (0.5 here) to form its new value. Step 3 continues the inflation process, allowing the sender to send additional packets (assuming $awnd$ is not exceeded). In step 4, the TCP is assumed to have recovered, so the temporary inflation is removed (and so this step is sometimes called “deflation”).

Slow start is always used in two cases: when a new connection is started, and when a retransmission timeout occurs. It can also be invoked when a sender has been idle for a relatively long time or there is some other reason to suspect that *cwnd* may not accurately reflect the current network congestion state (see Section 16.3.5). In this case, the initial value of *cwnd* is set to the *restart window* (*RW*). In [RFC5681], the recommended value of $RW = \min(IW, cwnd)$. Other than this case, when slow start is invoked, *cwnd* is set to *IW*.

16.3 Evolution of the Standard Algorithms

The classic and standard TCP algorithms made a tremendous contribution to the operation of TCP, essentially addressing the major problem of Internet congestion collapse.

Note

The problem of Internet congestion collapse was a serious concern during the years 1986–1988. In October 1986 the NSFNET backbone, an important component of the early Internet, had been observed to operate with an effective capacity some 1000 times less than it should have (called the “NSFNET meltdown”). The primary reason for the problem was aggressive retransmissions during times of loss without any controls. This behavior drove the network into a persistently congested state where packet loss was massive (causing more retransmissions) and throughput was low. Adoption of the classic congestion control algorithms effectively eliminated this problem.

However, there remained several areas for improvement. Given TCP’s popularity, a growing amount of effort was put into ensuring that TCP could be made to work well under a wider range of conditions. We now mention several of these that are found in many TCP implementations today.

16.3.1 NewReno

One problem with fast recovery is that when multiple packets are dropped in a window of data, once one packet is recovered (i.e., successfully delivered and ACKed), a good ACK can be received at the sender that causes the temporary window inflation in fast recovery to be erased before all the packets that were lost have been retransmitted. ACKs that trigger this behavior are called *partial ACKs*. A Reno TCP reacting to a partial ACK by reducing its inflated congestion window can go idle until a retransmission timer fires. To understand why this happens, recall that (non-SACK) TCP depends on the signal of three (or *dupthresh*) duplicate ACKs to trigger its fast retransmit procedure. If there are not enough packets in the network, it is not possible to trigger this procedure on packet loss, ultimately

leading to the expiration of the retransmission timer and invocation of the slow start procedure, which drastically impacts TCP throughput performance.

To address this problem with Reno, a modification called *NewReno* [RFC3782] has been developed. This procedure modifies fast recovery by keeping track of the highest sequence number from the last transmitted window of data (the recovery point, which we first saw in Chapter 14). Only when an ACK with an ACK number at least as large as the recovery point is received is the inflation of fast recovery removed. This allows a TCP to continue sending one segment for each ACK it receives while recovering and reduces the occurrence of retransmission timeouts, especially when multiple packets are dropped in a single window of data. NewReno is a popular variant of modern TCPs—it does not suffer from the problems of the original fast recovery and is significantly less complicated to implement than SACKs. With SACKs, however, a TCP can perform better than NewReno when multiple packets are lost in a window of data, but doing this requires careful attention to the congestion control procedures, which we discuss next.

16.3.2 TCP Congestion Control with SACK

With the introduction of SACKs and selective repeat to TCP, a sender is able to make better decisions about what segments to send in order to fill holes at the receiver (see Chapter 14). In filling the receiver's holes, the sender generally sends each of the missing segments, in order, until all of the retransmissions for the lost segments have been received successfully. This procedure differs from the basic fast retransmit/recovery procedure mentioned previously in a somewhat subtle way.

In the case of fast retransmit/recovery, when a packet is lost, the sending TCP transmits only the segment it believes is lost and is able to send new data if the window W allows. Because the window is inflated for each arriving ACK during fast recovery, with larger windows TCP typically is able to send some additional data after performing its retransmission. With SACK TCP, the sender can be informed of multiple missing segments and would theoretically be able to send them all immediately because they would all be in the valid window. However, this might involve sending too much data into the network at once, thereby compromising the congestion control. The following issue arises with SACK TCP: using only *cwnd* as a bound on the sender's sliding window to indicate how many (and which) packets to send during recovery periods is not sufficient. Instead, the selection of *which* packets to send needs to be decoupled from the choice of *when* to send them. Said another way, SACK TCP underscores the need to separate the congestion management from the selection and mechanism of packet retransmission. Conventional (non-SACK) TCP mixes these together.

One way to implement this decoupling is to have a TCP keep track of how much data it has injected into the network separately from the maintenance of the window. In [RFC3517] this is called the *pipe* variable, an estimate of the flight size. Importantly, the pipe variable counts bytes (or packets, depending on the

implementation) of transmissions *and* retransmissions, provided they are not known to be lost. Assuming a large value of *awnd*, a SACK TCP is permitted to send a segment anytime the following relationship holds true: $cwnd - pipe \geq SMSS$. In other words, *cwnd* is still used to place a limit on the amount of data that can be outstanding in the network, but the amount of data estimated to be in the network is accounted for separately from the window itself. How SACK TCP using this approach to congestion control compares with conventional TCP was first explored in detail with a series of simulations in [FF96].

16.3.3 Forward Acknowledgment (FACK) and Rate Halving

For TCP variants based on Reno (including NewReno), the typical behavior is that when *cwnd* is reduced after a fast retransmit, ACKs for at least one-half of the current window's outstanding data must be received before the sending TCP is allowed to continue transmitting. This is an expected consequence of reducing the congestion window by half immediately when a loss is detected. It causes the sending TCP to wait for about half of an RTT and then send any new data during the second half of the same RTT, a more bursty behavior than is really required.

In an effort to avoid the initial pause after loss but not violate the convention of emerging from recovery with a congestion window set to half of its size on entry, *forward acknowledgment* (FACK) was described in [MM96]. It consists of two algorithms called "overdamping" and "rampdown." Since the initial proposal, the authors updated their approach to form a unified and improved algorithm they call *rate halving*, based on earlier work by Hoe [H96]. To ensure that it works as effectively as possible, they further govern its behavior by adding bounding parameters, resulting in the complete algorithm being called *Rate-Halving with Bounding Parameters* (RHBP) [PSCRH].

The basic operation of RHBP allows the TCP sender to send one packet for every two duplicate ACKs it receives during one RTT. This causes the recovering TCP to have sent the appropriate amount of data by the end of the recovery period, but it spaces or *paces* this data evenly, rather than bunching all the transmissions into the second half of the RTT period. Avoiding the bunching or burstiness is advantageous because bursts tend to persist across multiple RTTs, stressing router buffers more than required.

To keep an accurate estimate of the flight size, RHBP uses information from SACKs to determine the FACK: the highest sequence number known to have reached the receiver, plus 1. Taking the difference between the highest sequence number about to be sent by the sender (SND.NXT in Figure 15-9) and the FACK gives an estimate of the flight size, not including retransmissions.

With RHBP, a distinction is made between the *adjustment interval* (the period when *cwnd* is modified) and the *repair interval* (when some segments are retransmitted). The adjustment interval is entered immediately upon a loss or congestion indicator. The final value for *cwnd* when the interval completes is half of the correctly delivered portion of the window of data in the network at the time

of detection. The following expression allows the RHBP sender to transmit, if satisfied:

$$(SND.NXT - \text{fack} + \text{retran_data} + \text{len}) < \text{cwnd}$$

This expression captures the flight size, including retransmissions, and ensures that if injecting another packet of length *len*, *cwnd* will not be exceeded. Provided all the data prior to the FACK is indeed no longer in the network (i.e., is lost or stored at the receiver), this causes the SACK sender to be appropriately controlled by *cwnd*. However, it can be overly aggressive if packets have been reordered in the network because the holes indicated by SACK have not been lost.

In Linux, FACK and rate halving are implemented and enabled by default. FACK is activated only when SACK is enabled and the Boolean configuration variable `net.ipv4.tcp_fack` is set to 1. When reordering is detected in the network, the more aggressive behavior of FACK is disabled.

Rate halving is one of several ways of pacing TCP's sending procedure to avoid or limit burstiness. Although it offers a number of benefits, it also has a few problems. In [ASA00], the authors analyze TCP pacing in some detail using simulations, concluding that in many cases it offers inferior performance to TCP Reno. Furthermore, rate-halving TCP has been known to exhibit poor performance when the connection may become limited by the receiver's advertised window [MM05].

16.3.4 Limited Transmit

In [RFC3042], the authors propose *limited transmit*, a small modification to TCP designed to help it perform better when the usable window is small. Recall from the experience with Reno TCP that when operating with a small window, there may not be enough packets in the network to trigger the fast retransmit/recovery algorithms when loss occurs, as these algorithms typically require three duplicate ACKs to be observed prior to initiation.

With limited transmit, a TCP with unsent data is permitted to send a new packet for each pair of consecutive duplicate ACKs it receives. Doing this helps to keep at least a minimal number of packets in the network—enough so that fast retransmit can be triggered upon packet loss. This is advantageous to TCP because waiting for an RTO (which can be a relatively large amount of time—several hundred milliseconds) can degrade throughput performance considerably. As of [RFC5681], limited transmit is now a recommended TCP behavior. Note that rate halving is one form of limited transmit.

16.3.5 Congestion Window Validation (CWV)

One of the issues with congestion management in TCP arises when the TCP sender stops sending for a period of time, either because it has no more data to send, or because it has been prevented from sending when it wants to for some

other reason. If all goes well, a sender never pauses, and it continues sending data and receiving ACKs from its peer. This continuous feedback enables it to keep a reasonably current (within one RTT) estimate of what *cwnd* and *ssthresh* should be.

If the TCP sender has been sending for some time, its *cwnd* may have grown to a substantial size. If it then fails to send for some time but resumes later, the large *cwnd* may allow the sender to inject an undesirably large number of packets (i.e., a high-rate burst) into the network without delay. Furthermore, if the pause is sufficiently long, its last *cwnd* value may no longer be appropriate for the path and congestion state.

In [RFC2861], the authors propose an experimental *Congestion Window Validation* (CWV) mechanism. Essentially, the sender's current value of *cwnd* decays over a period of nonuse, and *ssthresh* maintains the "memory" of it prior to the initiation of the decay. To understand the scheme, a distinction is made between an *idle* sender and an *application-limited* sender. The idle sender has stopped producing data it wants to send into the network; ACKs for all the data it has sent so far have been received. Thus, the connection is truly quiescent—no data is flowing, so no ACKs are either, except for occasional window updates (see Chapter 15). The application-limited sender does have more data to send but has been unable to for some reason. This could be because the sending computer is busy doing other tasks, or because some mechanism or protocol layer below TCP is preventing data from being sent. This case results in underutilization of the allowed congestion window, but the connection is not completely quiescent. In particular, ACKs may still be returning for previously sent data.

The CWV algorithm work as follows: Whenever a new packet is to be sent, the time since the last send event is measured to determine if it exceeds one RTO. If so,

- *ssthresh* is modified but not reduced—it is set to $\max(ssthresh, (3/4)*cwnd)$.
- *cwnd* is reduced by half for each RTT of idle time but is always at least 1 SMSS.

For application-limited periods that are not idle, the following similar behavior is used:

- The amount of window actually used is stored in *W_{used}*.
- *ssthresh* is modified but not reduced—it is set to $\max(ssthresh, (3/4)*cwnd)$.
- *cwnd* is set to the average of *cwnd* and *W_{used}*.

Both of these changes decay the value of *cwnd* while "remembering" it in *ssthresh*. The first case can dramatically affect *cwnd* in one operation, if the application has been idle for a long time. Handling the congestion window in this way can lead to better performance under some circumstances. As the authors report, reducing the burst of packets that can arise after an idle period eases the pressure

on potentially limited buffer space in routers, ultimately leading to fewer dropped packets. Note that because *cwnd* is decayed and *ssthresh* is not, the typical consequence of applying this algorithm is to place the sender into slow start after a long enough pause. CWV is enabled by default in Linux TCP implementations.

16.4 Handling Spurious RTOs—the Eifel Response Algorithm

As we saw in Chapter 15, when TCP encounters a large delay spike, it can experience a retransmission timeout even if no packet has been lost. Such spurious retransmissions arise in a number of circumstances relating to changes in the underlying link layer (such as cellular handoff) or sudden onset of severe congestion contributing to a large increase in RTT. When this happens, the TCP adjusts *ssthresh* and enters slow start by setting *cwnd* to *IW*. If no packets have been lost, ACKs arriving after the RTO cause *cwnd* to grow relatively quickly, but TCP still sends unnecessary retransmissions and underutilizes the capacity until *cwnd* and *ssthresh* resettle.

To avoid the performance problems associated with spurious retransmissions, several methods have been proposed to detect them. We discussed some of them (e.g., DSACK, Eifel, F-RTO) in Chapter 14. Any one of these, or possibly others that may be developed, can be coupled with a *response algorithm* used to “undo” the changes TCP makes to its congestion control variables after a timeout. One popular (i.e., in the IETF standards track) response algorithm is the *Eifel Response Algorithm* [RFC4015].

Eifel comprises both a detection and a response algorithm, which are logically disjoint. Any TCP implementation using the Eifel Response Algorithm is compelled to use some detection algorithm specified in a standards-track or experimental RFC (i.e., one that is documented).

The Eifel Response Algorithm is aimed at handling the retransmission timer and congestion control state after a retransmission timer has expired. Here we discuss only the congestion-related portions of the response algorithm. It is initiated after the first timeout-based retransmission is sent. Its purpose is to undo a change to *ssthresh* when a retransmission is deemed to be spurious. In all cases, before *ssthresh* is modified as a result of the RTO, it is captured in a special variable as follows: $pipe_prev = \min(flight\ size, ssthresh)$. Once this has been accomplished, a detection algorithm, such as one of those mentioned previously, is invoked in order to determine if the RTO is spurious. If it is, the following steps are executed when an ACK arrives after the retransmission:

1. If a received good ACK includes an ECN-Echo flag, stop (see Section 16.11).
2. $cwnd = flight\ size + \min(bytes_acked, IW)$ (assuming *cwnd* is measured in bytes).
3. $ssthresh = pipe_prev$.

17

TCP Keepalive

17.1 Introduction

Many newcomers to TCP/IP are surprised to learn that no data whatsoever flows across an idle TCP connection. That is, if neither process at the ends of a TCP connection is sending data to the other, nothing is exchanged between the two TCP endpoints. There is no polling, for example, as you might find with other networking protocols. This means that we can start a client process that establishes a TCP connection with a server and walk away for hours, days, weeks, or months, and the connection should remain up. In theory, intermediate routers can crash and reboot, data lines may go down and back up, but as long as neither host at the ends of the connection reboots (or changes its IP address), the connection remains established. This is how TCP/IP was designed.

Note

The previous statement assumes that neither application—neither the client nor the server—has application-level timers to detect inactivity, causing either application to terminate. It also assumes that no intermediate router is keeping state about the connection (such as a NAT box) that is required for proper operation that it might delete because of inactivity or lose because of system failure. In today's Internet, these are big assumptions.

Under some circumstances, it is useful for a client or server to become aware of the termination or loss of connection with its peer. In other circumstances, it is desirable to keep a minimal amount of data flowing over a connection, even if the applications do not have any to exchange. TCP *keepalive* provides a capability useful for both cases. Keepalive is a method for TCP to probe its peer without affecting the content of the data stream. It is driven by a *keepalive timer*. When the timer fires, a *keepalive probe* (*keepalive* for short) is sent, and the peer receiving the probe responds with an ACK.

Note

Keepalives are not part of the TCP specification. The Host Requirements RFC [RFC1122] says that this is because they could (1) cause perfectly good connections to break during transient Internet failures, (2) consume unnecessary bandwidth, and (3) cost money for an Internet path that charges for packets. Nevertheless, most implementations provide the keepalive capability.

TCP keepalive is a controversial feature. Many feel that polling of the other end has no place in TCP and should be done by the application, if desired. On the other hand, if many applications require such functionality, it is convenient to place it in TCP so that its implementation can be shared. The keepalive is an optionally enabled feature that can cause an otherwise good connection between two processes to be terminated because of a temporary loss of connectivity in the network joining the two end systems. For example, if the keepalive probes are sent during the time that an intermediate router has crashed and is rebooting, TCP incorrectly thinks its peer host has crashed.

The keepalive feature was originally intended for server applications that might tie up resources on behalf of a client and want to know if the client host crashes or goes away. Using TCP keepalive to detect dead clients is most useful for servers that expect to have a relatively short-duration dialogue with a noninteractive client (e.g., Web servers, POP and IMAP e-mail servers). Servers implementing more interactive-style services that last for a long time (e.g., remote login such as ssh and Windows Remote Desktop) might wish to avoid using keepalives.

A common example showing the utility of the keepalive feature nowadays is when a user uses the ssh (secure shell) remote login program to log in to a remote host through a NAT router. If the user were to establish the connection, do some work, then just power off the computer at the end of the day, without logging off, a half-open connection would be left. In Chapter 13 we showed that sending data across a half-open connection causes a reset to be returned, but that was from the server end, where the client was sending the data. If the client disappears, leaving the half-open connection on the server's end, and the server is waiting for some data from the client, the server will wait forever. The keepalive feature is intended to detect these half-open connections from the server side.

Another reason for using keepalives is somewhat the reverse. If the user does not power off the computer but instead leaves a connection open all night (and wishes to continue using it the next day), the connection goes idle for many hours. In Chapter 7 we discussed how most NAT routers include a timeout mechanism that flushes the state of a connection after some period of inactivity. If the NAT timeout is less than the several hours before the user returns to use the login session, and the NAT is not smart enough to probe the end station to make sure it is still active, or the NAT crashes, the connection is terminated. To avoid this common problem, ssh can be configured to use TCP keepalives. ssh also has the ability to use *application-managed* keepalives, and the two behave differently, especially with respect to their security properties. (Please see Section 17.3 for more on this.)

17.2 Description

Either end of a TCP connection may request keepalives, which are turned off by default, for their respective direction of the connection. A keepalive can be set for one side, both sides, or neither side. There are several configurable parameters that control the operation of keepalives. If there is no activity on the connection for some period of time (called the *keepalive time*), the side(s) with keepalive enabled sends a keepalive probe to its peer(s). If no response is received, the probe is repeated periodically with a period set by the *keepalive interval* until a number of probes equal to the number *keepalive probes* is reached. If this happens, the peer's system is determined to be unreachable and the connection is terminated.

A keepalive probe is an empty (or 1-byte) segment with sequence number equal to one less than the largest ACK number seen from the peer so far. Because this sequence number has already been ACKed by the receiving TCP, the arriving segment does no harm, but it elicits an ACK that is used to determine whether the connection is still operating. Neither the probe nor its ACK contains any new data (it is "garbage" data), and neither is retransmitted by TCP if lost. [RFC1122] dictates that because of this fact, the lack of response for a single keepalive probe should not be considered sufficient evidence that the connection has stopped operating. This is the reason for the *keepalive probes* parameter setting mentioned previously. Note that some (mostly older) TCP implementations do not respond to keepalives lacking the "garbage" byte of data.

Anytime it is operating, a TCP using keepalives may find its peer in one of four states:

1. The peer host is still up and running and reachable. The peer's TCP responds normally and the requestor knows that the other end is still up. The requestor's TCP resets the keepalive timer for later (equal to the value of the *keepalive time*). If there is application traffic across the connection before the next timer expires, the timer is reset back to the value of *keepalive time*.
2. The peer's host has crashed and is either down or in the process of rebooting. In either case, its TCP is not responding. The requestor does not receive a response to its probe, and it times out after a time specified by the *keepalive interval*. The requestor sends a total of *keepalive probes* of these probes, *keepalive interval* time apart, and if it does not receive a response, the requestor considers the peer's host as down and terminates the connection.
3. The client's host has crashed and rebooted. In this case, the server receives a response to its keepalive probe, but the response is a reset segment, causing the requestor to terminate the connection.
4. The peer's host is up and running but is unreachable from the requestor for some reason (e.g., the network cannot deliver traffic and may or may not

inform the peers of this fact using ICMP). This is effectively the same as state 2, because TCP cannot distinguish between the two. All TCP can tell is that no replies are received to its probes.

The requestor does not have to worry about the peer's host being shut down gracefully and then rebooting (as opposed to crashing). When the system is shut down by an operator, all application processes are terminated (i.e., the peer's process), which causes the peer's TCP to send a FIN on the connection. Receiving the FIN would cause the requestor's TCP to report an end-of-file to the requestor's process, allowing the requestor to detect this scenario and exit.

In the first state the requestor's application has no idea that keepalive probes are taking place (except that it chose to enable keepalives in the first place). Everything is handled at the TCP layer. It is transparent to the application until one of states 2, 3, or 4 is determined. In these three cases, an error is returned to the requestor's application by its TCP. (Normally the requestor has issued a read from the network, waiting for data from the peer. If the keepalive feature returns an error, it is returned to the requestor as the return value from the read.) In scenario 2 the error is something like "Connection timed out," and in scenario 3 we expect "Connection reset by peer." The fourth scenario may look as if the connection timed out, or may cause another error to be returned, depending on whether an ICMP error related to the connection is received and how it is processed (see Chapter 8). We look at all four scenarios in the next section.

The values of the variables *keepalive time*, *keepalive interval*, and *keepalive probes* can usually be changed. Some systems allow these changes on a per-connection basis, while others allow them to be set only system-wide (or both in some cases). In Linux, these values are available as `sysctl` variables with the names `net.ipv4.tcp_keepalive_time`, `net.ipv4.tcp_keepalive_intvl`, and `net.ipv4.tcp_keepalive_probes`, respectively. The defaults are 7200 (seconds, or 2 hours), 75 (seconds), and 9 (probes).

In FreeBSD and Mac OS X, the first two values are also available as `sysctl` variables called `net.inet.tcp.keepidle` and `net.inet.tcp.keepintvl`, with default values 7,200,000 (milliseconds, or 2 hours) and 75,000 (milliseconds, or 75s), respectively. These systems also have a Boolean variable called `net.inet.tcp.always_keepalive`. If this value is enabled, all TCP connections have the keepalive function enabled, even if the application did not request it. In these systems, the number of probes is a fixed default value: 8 (FreeBSD) or 9 (Mac OS X).

In Windows, these values are available for modification via registry entries under the system key:

```
HKLM\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters
```

The value `KeepAliveTime` defaults to 7,200,000ms (2 hours); `KeepAliveInterval` defaults to 1000ms (1s). If there is no response to ten keepalive probes, Windows terminates the connection.

Note that [RFC1122] places certain restrictions on the use of keepalives. In particular, the *keepalive time* must be configurable and must not default to less than 2 hours. In addition, keepalives must not be enabled unless an application requests one (although this behavior is violated if the `net.inet.tcp.always_keeplive` variable is set). Linux does not provide a native facility for adding keepalives to applications that do not request it, but a special library can be *pre-loaded* (i.e., loaded prior to ordinary shared libraries) to get this effect [LKA].

17.2.1 Keepalive Examples

We shall now go through states 2, 3, and 4 from the previous section, to see the packets exchanged using the keepalive mechanism. The operation in state 1 will be illustrated in the course of looking at the others.

17.2.1.1 Other End Crashes

Let us see what happens when the server host crashes and does not reboot. To simulate this we will do the following steps:

1. Using the `regedit` program on a Windows client, modify the registry key, and set `KeepAliveTime` to 7000ms (7s). This may require the system to be rebooted to accept the new value.
2. Establish an ssh connection between the Windows client and a Linux server using an option that enables TCP keepalives.
3. Verify that data can go across the connection.
4. Watch the client's TCP send keepalive packets every 7s, and see them acknowledged by the server's TCP.
5. Disconnect the network cable from the server, and leave it disconnected until the example is complete. This makes the client think the server host has crashed.
6. We expect the client to send ten keepalive probes, 1s apart, before declaring the connection dead.

Here is the interactive output on the client:

```
C:\> ssh -o TCPKeepAlive=yes 10.0.1.1
(password prompt and login continues)
Write failed: Connection reset by peer (about 15 seconds after disconnect)
```

Figure 17-1 shows the results using Wireshark. In this example, the connection has already been established. The Wireshark output begins with a keepalive (packet 1) that is not identified as such. At this point, Wireshark has not processed enough packets to determine that the one sequence number in packet 1 is below

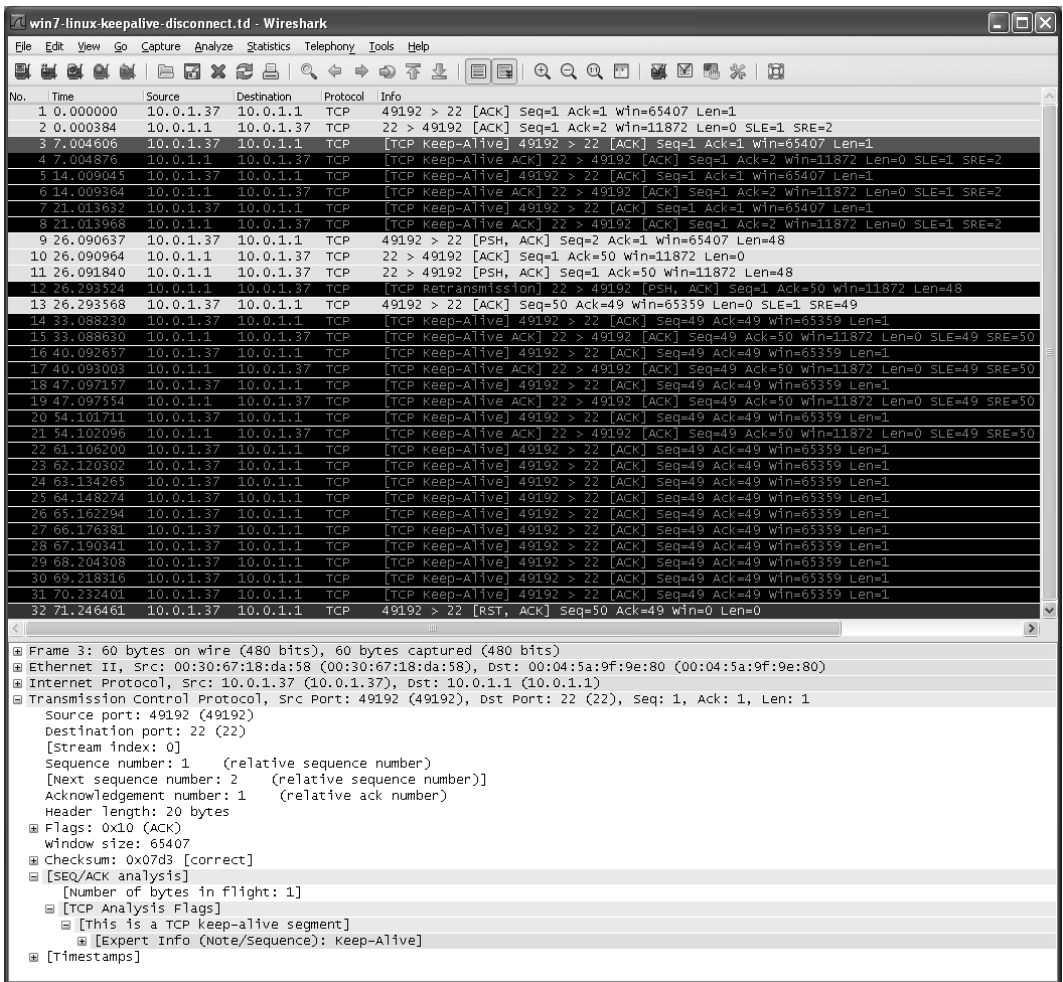


Figure 17-1 TCP keepalives are generated every 7s after the connection becomes idle. Each contains a below-window sequence number that is ACKed by the peer. A cable disconnection after 1 minute causes subsequent keepalives to not be ACKed. The client tries ten times before giving up and terminating the connection. The termination is signaled to the server by the final reset segment (which the server cannot hear). This example also illustrates the use of DSACKs at the server and a spurious retransmission caused by the client delaying ACKs.

the receiver's left window edge and is therefore a keepalive. Packet 2 contains an ACK number that allows Wireshark to process the sequence numbers in subsequent packets appropriately.

Most of this connection consists of keepalives and corresponding ACKs. Packets 1, 3, 5, 7, 14, 16, 18, 20, and 22–31 are all keepalives. Packets 2, 4, 6, 8, 15, 17, 19, and 21 are the corresponding ACKs. Keepalives are sent periodically every 7s

provided they are ACKed. When no ACK is returned for a keepalive, the sender switches to a 1s interval for sending keepalives, according to the default value of `KeepAliveInterval`. This starts with packet 23 at time 62.120. The sender produces ten unacknowledged keepalives in total (packets 22–31). After that, it terminates the connection, which results in the final reset segment (packet 32) that is never received by the disconnected receiver. The user receives the following output when the connection terminates:

```
Write failed: Connection reset by peer
```

This is a clear indication that the connection has terminated, but it is not entirely accurate. It was really the sender that terminated the connection, but it did so based on the lack of response from the receiver.

Apart from the use of keepalive segments, there are some other interesting features of this connection we will mention briefly. First, the server uses DSACKs (see Chapter 14). Each ACK contains the sequence number range of the previously received in-window segment. Next, a small bit of data is exchanged at time 26.09. The data represents a single key press. It is sent to the server, ACKed by the server, and echoed back. The data is encrypted, causing the packets containing data to be 48 bytes in user data size (see Chapter 18).

Interestingly, the echoed character is sent twice. We can see that packet 11, which contains the echoed character is not ACKed immediately. Recall from Chapter 14 that Linux uses an RTO of at least 200ms. Here we see that the Linux server retransmits the echoed character 200ms later, which produces an immediate response from the client. Because this test was performed on an uncongested LAN, it is highly unlikely that segment 11 was dropped. Instead, it appears that Linux produced a spurious retransmission due to the client delaying ACKs. This is a similar sort of hazard we saw when exploring the poor interaction between the Nagle algorithm and delayed ACKs we discussed in Chapter 15. Here, the dynamic results in an unnecessary delay of about 200ms.

17.2.1.2 Other End Crashes and Reboots

In this example we will see what happens when the peer crashes and reboots. The initial scenario is the same as the previous one, except this time we set `KeepAliveTime` to 120,000 (2 minutes). We establish a connection and then wait just over 2 minutes to allow a keepalive message to be sent and ACKed. Then we disconnect the server from the network, reboot it, and then reconnect it. We expect the next keepalive probe to generate a reset from the server, because the server now knows nothing about this connection. Figure 17-2 presents the trace as displayed by Wireshark.

In this example, the connection has been established and small amounts of data are exchanged starting at seconds 0.00 and 3.46. Then the connection goes idle. After 2 minutes have elapsed (the *keepalive time*), the client sends the first keepalive probe at time 123.47, containing the “garbage” byte below the receiver’s

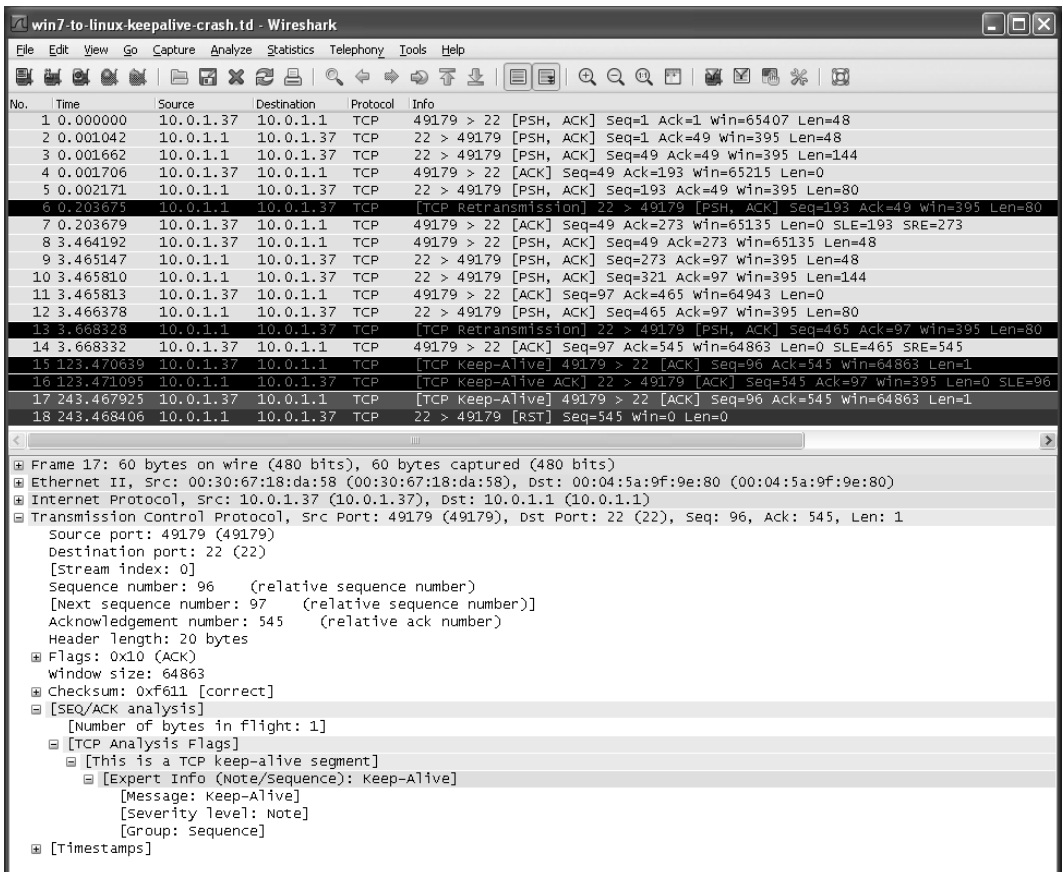


Figure 17-2 The server has rebooted between keepalives sent by the client. The last keepalive elicits a reset segment because the server no longer knows anything about the connection.

left window edge. It is acknowledged, and the server is disconnected, rebooted, and reconnected. At time 243.47, 120s later, the client sends its second keepalive probe. Although this reaches the server, the server no longer has any knowledge about the connection and responds with a reset segment (packet 18). This informs the client that the connection is no longer active, and the user is provided the same “Connection reset by peer” error message we saw before.

17.2.1.3 Other End Is Unreachable

In this case, the server has not crashed but becomes unreachable during the interval when the keepalive probes are sent. An intermediate router may have crashed, a phone line may be temporarily out of order, or something similar. To simulate this example we will use our sock program with the keepalive option set to

establish a connection to a Web server. We will use a Mac OS X client and an LDAP server (port 389) running on `ldap.mit.edu`. After shortening the client's *keepalive time* (for convenience) and opening the connection, we disconnect the network to see the effects. Here are the command lines and output at the client:

```
Mac# sysctl -w net.inet.tcp.keepidle=75000
Mac% sock -K ldap.mit.edu 389
recv error: Operation timed out          about 14 minutes later
```

The trace is displayed using Wireshark (see Figure 17-3).

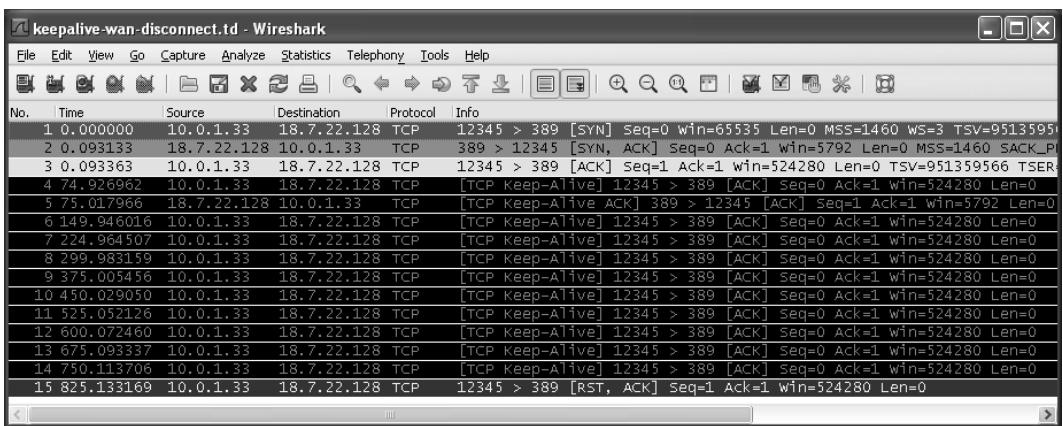


Figure 17-3 The WAN connection is taken down after the first keepalive probe is acknowledged. Another probe is sent every 75s. After nine keepalives are sent without a response, the connection is terminated and the client sends a reset to its peer. For the client, the situation is very similar to when the server crashes, as illustrated in Figure 17-1.

In this figure we can see the entire connection. After the initial three-way handshake, the connection remains idle and a keepalive is sent and acknowledged at about time 75 (packet 4). This first keepalive is triggered by the value of the `net.inet.tcp.keepidle` variable. Shortly thereafter, the network is severed. Neither end of the connection produces data, so the next event is another keepalive sent by the client at time 150 (75s later, the value of the `net.inet.tcp.keepintvl` variable). This pattern repeats with packets 7–14, with no ACKs present, even though the server is up and running. Finally, the client gives up 75s after its ninth unacknowledged keepalive probe. The connection termination is indicated to the server by a reset segment at the end (packet 15). Of course, the server is unable to receive this packet because the network is not operating.

When a client TCP using keepalives is unable to communicate across the network with its peer, as this example shows, it retries some number of times before giving up. This is essentially the same behavior we saw when the other

end crashed. In most cases, the sending TCP cannot tell the difference. There are some exceptions, such as when ICMP indicates that the destination has become unreachable or otherwise unavailable because of problems in the network, but these conditions are relatively rare because ICMP is often blocked. As a result, mechanisms such as TCP keepalive (or similar mechanisms implemented by applications) are used to detect disconnection periods.

17.3 Attacks Involving TCP Keepalives

As we mentioned before, SSH (version 2) has an application-level form of keep-alive called *server alive messages* and *client alive messages*. These are different from TCP keepalive messages because they are sent over an encrypted channel at the application layer and contain data. TCP keepalives contain no user-level data, so the use of encryption is limited at best. The consequence is that TCP keepalives may be spoofed. When TCP keepalives are spoofed, the victim can be coerced into keeping resources allocated for a period longer than intended.

Although it may be a relatively minor concern, TCP keepalives are driven off a timer based on the various configuration parameters discussed earlier, and not off the dynamically adjusted retransmission timer used to retransmit segments with data. A passive observer could notice the existence of keepalives and their inter-arrival times to conceivably learn information about the configuration parameters (possibly identifying the type of sending system, called *fingerprinting*) or about the network topology (i.e., whether downstream routers are forwarding traffic or not). These issues could be of concern in some environments.

17.4 Summary

As we said earlier, the keepalive feature has been somewhat controversial. Protocol experts continue to debate whether it belongs in the transport layer or should be handled entirely by the application. All popular TCP implementations now include the keepalive feature, which applications may optionally use to establish a “heartbeat” of traffic moving across a connection. Doing so can help a server by allowing it to detect nonresponsive clients and can help clients by keeping connections active (e.g., to keep NAT state active) even if no application-layer data is flowing.

Keepalives operate by sending a probe packet (usually containing a “garbage” byte, although zero-length probes are also possible) across a connection after the connection has been idle for some relatively long period of time, often 2 hours. Four different scenarios can occur: the other end is still there, the other end has crashed, the other end has crashed and rebooted, or the other end is currently unreachable. We saw each of these scenarios with an example.

In the first two keepalive examples that we examined, had keepalives not been used, and without any application-level timer or activity, TCP would never have known that the other end had crashed (or crashed and rebooted). In the final example, however, nothing was wrong with the other end; the connection was temporarily down. We must be aware of this limitation when using keepalives and consider whether or not such behavior is desired.

Attacks against the keepalive mechanism include causing a system to keep resources allocated longer than intended and possibly learning some otherwise hidden information about the end systems (although such information may be of limited use to an attacker). In addition, by default TCP does not use its own encryption, so keepalives and keepalive ACKs can be spoofed, whereas application-level keepalives that employ encryption (e.g., SSH) cannot.

17.5 References

[LKA] <http://libkeepalive.sourceforge.net>

[RFC1122] R. Braden, ed., "Requirements for Internet Hosts," Internet RFC 1122, Oct. 1989.